2019
MU

Strictly as per the New Revised Syllabus (Rev - 2016) of
Mumbai University w.e.f. academic year 2018-2019
(As per Choice Based Credit and Grading System)

Theory of Computer Science

Semester V - Computer Engineering

Same Subject, Same Author with New Publication

Dilip Kumar Sultania

With Solved Latest University Question Papers upto Dec. 2018.

TechKnowledge™ Publications

Scanned by CamScanner

Theory of Computer Science

(Code : CSC504)

Semester V - Computer Engineering
(Mumbai University)

Strictly as per the Choice Based Credit and Grading System (Revise 2016) of Mumbai University w.e.f. academic year 2018-2019

Dilip Kumar Sultania

B.Tech.(hons.) Computer Science and Engineering
I.I.T. , Kharagpur.

THE JAGDISH BOOK DEPOT
Near Punjab National Bank,
Shivaji Path, Thane (W).
Mob. No. - 9820841360/9324557619

TechKnowledge™ Publications

MO46A Price ₹ 455/-

(Book Code : MO46A)

Scanned by CamScanner

Theory of Computer Science

Dilip Kumar Sultania

(Semester V, Computer Engineering, MU)

Copyright © by Author. All rights reserved. No part of this publication may be reproduced, copied, or stored in a retrieval system, distributed or transmitted in any form or by any means, including photocopy, recording, or other electronic or mechanical methods, without the prior written permission of the publisher.

This book is sold subject to the condition that it shall not, by the way of trade or otherwise, be lent, resold, hired out, or otherwise circulated without the publisher's prior written consent in any form of binding or cover other than which it is published and without a similar condition including this condition being imposed on the subsequent purchaser and without limiting the rights under copyright reserved above.

First Edition : August 2018
Second Revised Edition : July 2019 **(TechKnowledge Publications)**

This edition is for sale in India, Bangladesh, Bhutan, Maldives, Nepal, Pakistan, Sri Lanka and designated countries in South-East Asia. Sale and purchase of this book outside of these countries is unauthorized by the publisher.

Printed at : 37/2, Ashtavinayak Industrial Estate, Near Pari Company, Narhe, Pune, Maharashtra State, India. Pune – 411041

ISBN 978-93-89299-95-3

Published by
TechKnowledge Publications

Head Office : B/5, First floor, Maniratna Complex, Taware Colony, Aranyeshwar Corner, Pune - 411 009. Maharashtra State, India
Ph : 91-20-24221234, 91-20-24225678.

[CSC504] (FID : MO46) (Book Code : MO46A)

Scanned by CamScanner

Preface

My Dear Students,

I am extremely happy to come out with this edition of **"Theory of Computer Science"** for you. The topics within the chapters have been arranged in a proper sequence to ensure smooth flow of the subject.

A large number of solved examples have also been included. Therefore, we are sure that this book will cater all your needs for this subject.

I present this book in the loving memory of **Late Shri. Pradeepji Lunawat**, our source of inspiration and a strong foundation of **"TechKnowledge Publications"**. He will always be remembered in our heart and motivate us to achieve our milestone.

I am thankful to the staff members of TechKnowledge Publications and others for their efforts to make this book as good as it is. I have jointly made every possible efforts to eliminate all the errors in this book. However if you find any, please let me know, because that will help me to improve further.

I am also thankful to my family members and friends for patience and encouragement.

- Dilip Kumar Sultania

□□□

(Book Code : MO46A)

Scanned by CamScanner

Syllabus

Theory of Computer Science

(Mumbai University)

Course Code	Course Name	Credits
CSC504	Theory of Computer Science	4

Course objectives :

1. Acquire conceptual understanding of fundamentals of grammars and languages.
2. Build concepts of theoretical design of deterministic and non-deterministic finite automata and push down automata.
3. Develop understanding of different types of Turing machines and applications.
4. Understand the concept of Undecidability.

Course outcomes : On successful completion of course learner will be able to :

1. Identify the central concepts in theory of computation and differentiate between deterministic and nondeterministic automata, also obtain equivalence of NFA and DFA.
2. Infer the equivalence of languages described by finite automata and regular expressions.
3. Devise regular, context free grammars while recognizing the strings and tokens.
4. Design pushdown automata to recognize the language.
5. Develop an understanding of computation through Turing Machine.
6. Acquire fundamental understanding of decidability and undecidability.

(Book Code : MO46A)

Scanned by CamScanner

Prerequisite : Discrete Mathematics

Module No.	Unit No.	Topics	Theory Hrs.	Tutorial Hrs.
1.0		**Basic Concepts and Finite Automata**	09	03
	1.1	• Alphabets, Strings, Languages, Closure properties. • Finite Automata (FA) and Finite State machine (FSM).		
	1.2	• Deterministic Finite Automata (DFA) and Nondeterministic Finite Automata (NFA) : Definitions, transition diagrams and Language recognizers • NFA to DFA Conversion • Equivalence between NFA with and without ε- transitions • Minimization of DFA • FSM with output: Moore and Mealy machines, Equivalence • Applications and limitations of FA **(Refer chapters 1 and 2)**		
2.0		**Regular Expressions and Languages**	06	02
	2.1	• Regular Expression (RE) • Equivalence of RE and FA, Arden's Theorem • RE Applications		
	2.2	• Regular Language (RL) • Closure properties of RLs • Decision properties of RLs • Pumping lemma for RLs **(Refer chapter 3)**		

Scanned by CamScanner

Module No.	Unit No.	Topics	Theory Hrs.	Tutorial Hrs.
3.0		**Grammars**	08	03
	3.1	• Grammars and Chomsky hierarchy.		
	3.2	• Regular Grammar (RG) • Equivalence of Left and Right linear grammar • Equivalence of RG and FA		
	3.3	**Context Free Grammars (CFG)** • Definition, Sentential forms, Leftmost and Rightmost derivations, Parse tree, Ambiguity. • Simplification and Applications. • Normal Forms: Chomsky Normal Forms (CNF) and • Greibach Normal Forms (GNF). • CFLs - Pumping lemma, Closure properties **(Refer chapters 4 and 5)**		
4.0		**Pushdown Automata(PDA)**	04	01
	4.1	• Definition, Transitions ,Language of PDA • Language acceptance by final state and empty stack • PDA as generator, decider and acceptor of CFG. • Deterministic PDA , Non-Deterministic PDA • Application of PDA. **(Refer chapter 6)**		

(Book Code : MO46A)

Scanned by CamScanner

Module No.	Unit No.	Topics	Theory Hrs.	Tutorial Hrs.
5.0		**Turing Machine (TM)**	09	03
	5.1	• Definition, Transitions • Design of TM as generator, decider and acceptor • Variants of TM : Multitrack, Multitape • Universal TM. • Equivalence of Single and Multi Tape TMs • Applications, Power and Limitations of TMs • Context Sensitivity and Linear Bound Automata. **(Refer chapter 7)**		
6.0		**Undecidability**	03	01
	6.1	• Decidability and Undecidability • Recursive and Recursively Enumerable Languages. • Halting Problem • Rice's Theorem • Post Correspondence Problem **(Refer chapter 8)**		
		Total	**39**	**13**

□□□

Scanned by CamScanner

Module 1

Chapter 1 : Basic Concepts and Finite Automata 1-1 to 1-21

1.1 Basics of Languages 1-1
1.1.1 Kleene Closure 1-3
1.1.2 Recursive Definition of a Language 1-4
1.1.3 Prefix and Suffix of a String 1-5
1.2 Grammar (Context Free) 1-5
1.2.1 Notations 1-6
1.2.2 The Language of a Grammar 1-7
1.2.2.1 Sentential Form 1-7
1.2.2.2 Parse Tree 1-9
1.3 Chomskey Hierarchy 1-13
1.3.1 Type 3 or Regular Grammar 1-13
1.3.2 Type 2 or Context Free Grammar 1-14
1.3.3 Type 1 or Context Sensitive Grammar 1-14
1.3.4 Type 0 or Unrestricted Grammar 1-15
1.4 Finite Automata (FA) 1-15
1.4.1 Working of a Finite Automata 1-15
1.4.2 Some Important Terms 1-17
1.4.2.1 Alphabet 1-17
1.4.2.2 Strings (words) 1-17
1.4.2.3 Languages 1-19
1.4.3 Application of Finite Automata 1-19
1.5 Finite State Machine (FSM) 1-20
1.6 University Questions and Answers 1-20
• Chapter Ends... 1-21

Scanned by CamScanner

Chapter 2 : Finite Automata

2-1 to 2-151

2.1 Introduction to Finite Automata 2-1

2.1.1 Working of a Finite Automata 2-2

2.1.2 Some Important Terms 2-3

2.1.2.1 Alphabet 2-3

2.1.2.2 Strings (words) 2-3

2.1.2.3 Languages 2-5

2.1.3 Application of Finite Automata 2-5

2.2 Deterministic Finite Automata (DFA) 2-6

2.2.1 Definition of a DFA 2-6

2.2.2 Representation of a DFA 2-6

2.2.3 Designing a DFA 2-8

2.2.4 Examples on DFA 2-12

2.2.4.1 Examples on Counting of Symbols 2-12

2.2.4.2 Examples on Substring 2-20

2.2.4.3 Examples of Divisibility 2-33

2.2.5 Language of DFA 2-38

2.2.6 Equivalence of DFAs 2-41

2.2.7 Closure Property of Language Accepted by a DFA 2-44

2.2.7.1 Union, Intersection, Difference 2-44

2.2.7.2 Complementation 2-48

2.2.8 Minimization of DFA 2-48

2.2.8.1 Algorithm for Minimization DFA's 2-49

2.3 Non-deterministic Finite Automata 2-56

2.3.1 Definition of NFA 2-56

2.3.2 Processing of a String by NFA 2-57

2.3.3 NFA to DFA Conversion 2-64

Scanned by CamScanner

2.3.4 NFA with ε-Transitions 2-88
2.3.4.1 Equivalence of ε-NFA and NFA 2-89
2.3.4.2 The Formal Notation for an ε-NFA 2-90
2.3.4.3 ε-Closures 2-91
2.3.4.4 ε-NFA to DFA 2-93
2.3.5 Difference between NFA and DFA 2-102
2.4 Finite Automata as Output Devices 2-102
2.4.1 A Sample Mealy Machine 2-103
2.4.2 Formal Definition of a Mealy Machine 2-104
2.4.3 A Sample Moore Machine 2-105
2.4.4 Formal Definition of a Moore Machine 2-106
2.4.5 Conversion of a Mealy Machine into a Moore Machine 2-117
2.4.6 Conversion of a Moore Machine into a Mealy Machine 2-126
2.5 Minimization of a Mealy Machine 2-132
2.5.1 Difference between Moore and Mealy Machines 2-135
2.6 University Questions and Answers 2-136
• Chapter Ends... 2-151

Module 2

Chapter 3 : Regular Expressions and Languages **3-1 to 3-84**

3.1 Introduction 3-1
3.2 Finite Automata Representing a Regular Expression 3-4
3.2.1 Composite Finite State Automata 3-4
3.3 Determination of Regular Expression 3-9
3.3.1 Language Generated by a Regular Expression 3-9
3.3.2 Basic Properties of Regular Expressions 3-9
3.4 DFA to Regular Expression 3-29
3.4.1 State/Loop Elimination Process 3-30
3.4.1.1 A Generic One State Machine 3-30
3.4.1.2 A Generic Two State Machine 3-30

Scanned by CamScanner

3.4.2 Arden's Theorem 3-47
3.4.2.1 Application of Arden's Theorem 3-48
3.5 FA Limitations 3-53
3.6 Pumping Lemma for Regular Language 3-53
3.6.1 Definition of Pumping Lemma 3-54
3.6.2 Interpretation of Pumping Lemma 3-54
3.6.3 Proof of Pumping Lemma 3-55
3.6.4 Application of Pumping Lemma 3-56
3.7 Closure Properties of Regular Language 3-64
3.7.1 Regular Language is Closed under Union 3-65
3.7.2 Regular Language is Closed under Concatenation 3-66
3.7.3 Regular Language is Closed under Kleene Star 3-67
3.7.4 Regular Language is Closed under Complementation 3-67
3.7.5 Regular Language is Closed under Intersection 3-67
3.7.6 Regular Languages are Closed under Difference 3-68
3.7.7 Regular Languages are Closed under Reversal 3-68
3.8 Decision Properties of Regular Languages 3-71
3.9 Myhill - Nerode Theorem 3-73
3.10 Application of RE 3-73
3.10.1 R.E. in Unix 3-73
3.10.2 Lexical Analysis 3-74
3.11 University Questions and Answers 3-74
• Chapter Ends... 3-84

Module 3

Chapter 4 : Regular Grammar (RG) **4-1 to 4-27**

4.1 Regular Grammar 4-1
4.2 DFA to Right-Linear Regular Grammar 4-2
4.2.1 Right Linear Grammar to DFA 4-4

Scanned by CamScanner

4.3 DFA to Left-Linear Grammar 4-8
4.3.1 Left Linear Grammar to DFA 4-10
4.4 Right Linear Grammar to Left Linear Grammar 4-12
4.5 Left Linear Grammar to Right Linear Grammar 4-15
4.6 University Questions and Answers 4-27
• Chapter Ends... 4-27

Chapter 5 : Context Free Grammars (CFG) 5-1 to 5-116

5.1 An Example to Explain Grammar 5-1
5.2 Context Free Grammar 5-5
5.2.1 Notations 5-5
5.2.2 The Language of a Grammar 5-6
5.2.2.1 Sentential Form 5-7
5.2.2.2 Parse Tree 5-8
5.2.3 Writing Grammar for a Language 5-14
5.2.3.1 Union Rule for Grammar 5-21
5.2.3.2 Concatenation Rule for Grammar 5-22
5.3 Ambiguous Grammar 5-39
5.4 Simplification of CFG 5-54
5.4.1 Elimination of Useless Symbols 5-54
5.4.1.1 Non-generating Symbols 5-54
5.4.1.2 Non-reachable Symbols 5-56
5.4.2 Elimination of ε-productions 5-58
5.4.3 Elimination of Unit Productions 5-61
5.5 Normal Forms for CFG 5-69
5.5.1 Chomsky Normal Form (CNF) 5-69
5.5.1.1 Algorithm for CFG to CNF Conversion 5-69
5.5.2 Greibach Normal Form (GNF) 5-79
5.5.2.1 Removing Left Recursion 5-80
5.5.2.2 Algorithm for Conversion from CFG to GNF 5-81

Scanned by CamScanner

5.6 Pumping Lemma for CFG 5-96
5.7 Properties of Context-free Languages 5-100
5.7.1 Closure Properties 5-100
5.7.1.1 CFL is Closed under Union 5-100
5.7.1.2 CFL is Closed under Concatenation 5-101
5.7.1.3 CFL is Closed under Kleene Star 5-101
5.7.1.4 CFL is not Closed under Intersection 5-102
5.7.1.5 CFL is not Closed under Complementation 5-102
5.7.1.6 Intersection of CFL and RL 5-103
5.7.1.7 CFL is Closed under Reversal 5-104
5.7.2 Algorithmic Properties (Decision Properties) 5-104
5.8 University Questions and Answers 5-104
• Chapter Ends... 5-116

Module 4

Chapter 6 : Pushdown Automata (PDA) **6-1 to 6-70**

6.1 Introduction to Pushdown Automata (PDA) 6-1
6.2 The Formal Definition of PDA 6-3
6.3 Instantaneous Description of a PDA 6-6
6.4 The Language of a PDA 6-9
6.4.1 Acceptance by Final State 6-9
6.4.2 Acceptance by Empty Stack 6-9
6.5 Non-deterministic PDA (NPDA) 6-25
6.6 Pushdown Automata and Context Free Language 6-32
6.6.1 Construction of PDA from CFG 6-32
6.6.2 Construction of CFG from PDA 6-38
6.7 Deterministic Pushdown Automata (DPDA) 6-55
6.7.1 Regular Language and DPDA 6-55

Scanned by CamScanner

6.8 Application of PDA 6-57
6.9 Parsing 6-57
6.9.1 Top-Down Parsing 6-57
6.9.2 Bottom-Up Parsing 6-61
6.10 Multistack DPDA 6-62
6.11 Power of Various Machines 6-63
6.12 University Questions and Answers 6-63
• Chapter Ends... 6-70

Module 5

Chapter 7 : Turing Machine (TM) **7-1 to 7-76**

7.1 Introduction to Turing Machine 7-1
7.2 The Formal Definition of Turing Machine 7-4
7.2.1 A String Accepted by TM 7-6
7.2.2 Instantaneous Descriptions for Turing Machines 7-8
7.3 Turing Machines as Computer of Functions 7-32
7.4 Extension of Turing Machine 7-59
7.4.1 Two-way Infinite Turing Machine 7-59
7.4.2 A Turing Machine with Multiple Heads 7-60
7.4.3 Multi-Tape Turing Machine 7-60
7.4.4 Non-deterministic Turing Machine 7-64
7.5 Universal Turing Machine 7-66
7.5.1 Church-Turing Hypothesis 7-67
7.5.2 Power of Various Machines 7-67
7.6 Linear Bounded Automata 7-68
7.7 University Questions and Answers 7-68
• Chapter Ends... 7-76

Scanned by CamScanner

Module 6

Chapter 8 : Undecidability 8-1 to 8-23

8.1 Recursive and Recursively Enumerable Language 8-1

8.1.1 Turing Acceptable Language 8-1

8.2 Enumerating a Language 8-6

8.2.1 Finite and Infinite Sets 8-6

8.3 Compare Type 0, Type 1, Type 2 and Type 3 Grammars 8-10

8.4 Un-decidability 8-11

8.4.1 Halting Problem of a Turing Machine 8-11

8.4.2 Un-decidability of Post Correspondence Problem 8-13

8.4.3 Rice's Theorem 8-18

8.4.4 Greibach Theorem 8-18

8.5 Partial Recursive Function 8-18

8.6 University Questions and Answers 8-20

- Chapter Ends... 8-23
- **Appendix A :** Solved University Question Paper of Dec. 2018 ..... A-1 to A-4

□□□

Scanned by CamScanner

CHAPTER 1

Module 1

Basic Concepts and Finite Automata

1.1 Basics of Languages

A subset of strings over an alphabet is a language.

An **alphabet** is a finite, non empty set of symbols. Conventionally, the symbol Σ is used for an alphabet.

Common alphabet include :

1. $\Sigma\{0,1\}$, the binary alphabet.
2. $\Sigma\{A,B,...Z\}$, the set of uppercase letters.
3. $\Sigma\{0,1,2,...9\}$, the decimal alphabet.

A **string** is a finite sequence of symbols from the alphabet. For example :

1. 10110 is a string from binary alphabet.
2. 5920 is a string from decimal alphabet.
3. 'STAMP' is a string from roman alphabet.

A string having no symbol is known as an empty sting. An empty string is denoted by ε.

- The **exponential** notation is used to express the set of all strings of a certain length.

  Σ^k = set of strings of length k, each of whose symbol is in Σ.

For example,

If $\Sigma = \{0, 1\}$

then $\Sigma^1 = \{0, 1\}$

$\Sigma^2 = \{00, 01, 10, 11\}$

and $\Sigma^0 = \{\varepsilon\}$

- The **reversal** of string ω is denoted by ω^R.

Scanned by CamScanner

For example,

If $\omega = xyz$

Then $\omega^R = zyx$.

- A string v is a **substring** of a string ω if and only if there are strings x and y such that,

  $\omega = x\ v\ y$, where both x and y could be null

  Every string is a substring of itself.

  For example,

  if ω = 'abcdef' is a string.

  then 'cde' is a substring of ω.

- For each string ω, the string ω^i is defined as,

  $\omega^0 = \varepsilon$, the empty string

  $\omega^{i+1} = \omega^i \cdot \omega$ for each $i \geq 0$ where, i is a natural number.

  For example,

  if $\omega = abb$

  then $\omega^0 = \varepsilon$

  $\omega^1 = abb$

  $\omega^2 = \omega \cdot \omega = abbabb$

  $\omega^3 = \omega^2 \cdot \omega = abbabbabb$

- The set of all strings over an alphabet Σ is denoted by Σ^*.

  $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \ldots\ldots\ldots\ldots$

  Σ^0: ε

  Σ^1: String of length 1

  Σ^2: String of length 2

  For example,

  if $\Sigma = \{0, 1\}$

  then $\Sigma^* = \{\in, 0, 1, 00, 01, 10, 11, 000, 001, \ldots.\}$

- A subset of strings over an alphabet Σ is a **language**. In other words, any subset of Σ^* is a language.

  If Σ is an alphabet, and $L \subseteq \Sigma^*$, then L is a language over Σ.

  For example,

  $L_1 = [\omega \in \{0, 1\}^* \mid \omega$ has an equal number of 0's and 1's]

  i.e., L_1 is a language over alphabets {0, 1}, having equal number of 0's and 1's.

  $\therefore\ L_1 = \{\varepsilon, 01, 10, 0011, 0101, 1100, 1010, \ldots\}$

Scanned by CamScanner

Thus, a language can be specified by :

$$L = \{\omega \in \Sigma^* \mid \omega \text{ has the given property}\}$$

Some examples of language are :

1. A language of all strings, where the string represents a binary number.
   {0, 1, 00, 01, 10, 11, ...}
2. The set of strings of 0's and 1's with an equal number of each.
   {ε, 01, 10, 0011, 0101, 1010, 1100, ...}
3. The set of strings of 0's and 1's, ending in 11.
   {11, 011, 111, 0011, 0111, 1011, ...}
4. Σ^* is a language for any alphabet Σ.
5. ϕ, the empty language, is a language over an alphabet.
6. {ε}, the language consisting of only the empty string, is also a language over any alphabet.

1.1.1 Kleene Closure

Given an alphabet Σ, the Kleene closure of Σ is a language given by

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 .....$$

Σ^0: ε
Σ^1: Strings of length 1
Σ^2: Strings of length 2

For example :

1. If $\Sigma = \{x\}$, then
   $\Sigma^* = \{\varepsilon, x, xx, ...\}$
2. If $\Sigma = \{0, 1\}$, then
   $\Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, ...\}$

If L is a language, then L^* is the set of all finite strings formed by concatenating words from L. Any word can be used any number of times including zero time. ε is a member of L^*.

Example : If L = {aa, b} then

$$L^* = \{\varepsilon, b, aa, bb, bbb, baa, aab, ...\}$$

It may be noted that a always appears in pair.

- If $\Sigma = \phi$ (an empty language)
  Then,
  $\Sigma^* = \{\varepsilon\}$

Scanned by CamScanner

- L^+ is known as the positive closure of L. L^+ is the set of all finite strings formed by concatenating words from L. Any word can be used one or more times.

Example : If L = {aa, b} then

$$L^+ = \{b, aa, bb, bbb, baa, aab, ...\}$$

Example 1.1.1 : Let L be a language. It is clear from the definition that $L^+ \subseteq L^*$. Under what circumstances are they equal ?

Solution :

L^* always contains an epsilon.

L^+ may not contain an epsilon.

Thus, the only difference between L^+ and L^* is that L^+ may or may not contain an **epsilon** but L^* will always contain an **epsilon**.

If L itself contains an **epsilon** then there will be no difference between L^+ and L^*.

1.1.2 Recursive Definition of a Language

A language over an alphabet Σ can be described recursively. A recursive definition has three steps :

1. Specify some basic objects in the set.
2. Specify the rules for constructing more objects from the objects already known.
3. Declaration that no objects except those constructed as given above are allowed in the set.

Example : Let us try to define a language of positive integers

$$L = \{1, 2, 3, ...\}$$

The object 2 can be derived from 1 by adding 1 to it.

The object 3 can be derived from object 2 by adding 1 to it.

Thus, if 1 is taken as the base object then we can derive any positive integer from it.

Rules are given below :

Rule 1 : 1 is a positive integer

Rule 2 : If x is a positive integer, then so is x + 1.

Scanned by CamScanner

Example 1.1.2 : Define a language of polynomials recursively. Give derivation for $5x^2 + 7x$.

Solution :

A polynomial is a finite sum of terms, where each term is of the form $c \cdot x^n$, c and n are integers.

We should be able to derive a term.

A polynomial can be derived by adding such terms.

Rule 1 : Any number is a term

Rule 2 : The variable x is a term

Rule 3 : If p and q are terms then pq is also a term

Rule 4 : A term is polynomial

Rule 5 : If the terms p and q are in polynomial, then they are p + q and p – q.

Deriving $5x^2 + 7x$

5 is a term	– (by Rule 1)
x is a term	– (by Rule 2)
5x is a term	– (by Rule 3)
$(5x) \cdot x$ is a term	– (by Rule 3)
$5x^2$ is a polynomial	– (by Rule 4)
7 is a term	– (by Rule 1)
7x is a term	– (by Rule 3)
$5x^2 + 7x$ is a polynomial	– (by Rule 5)

1.1.3 Prefix and Suffix of a String

A prefix of a string is a substring of leading symbols of the string.

Example : The prefixes of the string "abc" are {∈, a, ab, abc}

A suffix of a string is a substing of trailing symbols of the string.

Example : The suffixes of the string "abc" are {∈, c, bc, abc}

1.2 Grammar (Context Free)

A context free grammar G is a quadruple (V, T, P, S),

where,

V is a set of variables.

T is a set of terminals.

P is a set of productions.

S is a special variable called the start symbol $S \in V$.

A production is of the form

$V_i \rightarrow \alpha_i$ where $V_i \in V$ and α_i is a string of terminals and variables.

Scanned by CamScanner

1.2.1 Notations

1. Terminals are denoted by lower case letters a, b, c … or digits 0, 1, 2 …. etc.
2. Non-terminals (variables) are denoted by capital letters A, B, …, V, W, X …
3. A string of terminals or a word $w \in L$ is represented using u, v, w, x, y, z.
4. A sentential form is a string of terminals and variables and it is denoted by α, β, γ etc.

CFG explained through an example

Let us consider English sentences of the form :

1. Mohan eats. 2. Soham plays. 3. Ram reads.

The first word of in the above sentences is a noun and the second word is a verb. A sentence of the above form can be written as

<sentence> → <noun> <verb>

Here, noun can be replaced with Mohan, Soham or Ram and <verb> can be replaced with reads, plays or eats. We can write :

<noun> → Mohan
<noun> → Soham
<noun> → Ram
<verb> → eats
<verb> → plays
<verb> → reads

In the above example :

1. <sentence>, <noun> and <verb> are variables or non-terminals.
2. Mohan, Soham, Ram, eats, plays and reads are terminals.
3. The variable <sentence> is the start symbol as a sentence will be formed using the start symbol <sentence>.

Formally, the grammar can be written as :

G = (V, T, P, S), where
V = {<sentence>, <noun>, <verb>}
T = {Mohan, Soham, Ram, eats, plays, reads}
P = {<sentence> → <noun> <verb>,
<noun> → Mohan,
<noun> → Soham,
<noun> → Ram,
<verb> → eats,
<verb> → plays,
<verb> → reads
}

Scanned by CamScanner

S = <sentence>

Several productions of <noun> and <verb> can be merged together and the set of productions can be re-written as :

P = {<sentence> → <noun> <verb>,
<noun> → Mohan | Soham | Ram,
<verb> → eats | plays | reads
}

1.2.2 The Language of a Grammar

Every grammar generates a language. A word of a language is generated by applying productions a finite number of times. Derivation of a string should start from the start symbol and the final string should consist of terminals.

If G is a grammar with start symbol S and set of terminals T, then the language of G is the set

$$L(G) = \left\{ w \mid w \in T^* \text{ and } S \underset{G}{\overset{*}{\Rightarrow}} w \right\}$$

If the production rule is applied once, then we write $\alpha \underset{G}{\Rightarrow} \beta$. When it is applied a number of times then we write $\alpha \underset{G}{\overset{*}{\Rightarrow}} \beta$

Derivations are represented either in the

1. Sentential form OR 2. Parse tree form

1.2.2.1 Sentential Form

Let us consider a grammar given below :

S → A1B (Production 1-1)
A → 0A | ε (Production 1-2)
B → 0B | 1B | ε (Production 1-3)

where, G is given by (V, T, P, S)
with, V = {S, A, B}
T = {0, 1}
P = {Productions 1-1, 1-2 and 1-3}
S = Start symbol

Let us try to generate the string 00101 from the given grammar.

S → A1B [Starting production]
→ 0A1B [Using the production A → 0A]

Scanned by CamScanner

$\rightarrow$ 00A1B [Using the production $A \rightarrow 0A$]

$\rightarrow$ 001B [Using the production $A \rightarrow \varepsilon$]

$\rightarrow$ 0010B [Using the production $B \rightarrow 0B$]

$\rightarrow$ 00101B [Using the production $B \rightarrow 1B$]

$\rightarrow$ 00101 [Using the production $B \rightarrow \varepsilon$]

Thus the string $00101 \in L(G)$.

In sentential form, derivation starts from the start symbol through a finite application of productions.

A string α derived so far consists of terminals and non-terminals.

$$S \overset{*}{\underset{G}{\Rightarrow}} \alpha \mid \alpha \in (V \cup T)^*$$

- A final string consists of terminals.
- In left sentential form, leftmost symbol is picked up for expansion.
- In right sentential form, rightmost symbol is picked up for expansion.
- A string can be derived in many ways. But we restrict ourselves to :
  1. Leftmost derivation.
  2. Rightmost derivation.

In leftmost derivation the leftmost variable of α (sentential form) is picked for expansion.

In rightmost derivation the rightmost variable of α (sentential form) is picked for expansion.

Example 1.2.1 : For the grammar given below

$S \rightarrow A1B$

$A \rightarrow 0A \mid \varepsilon$

$B \rightarrow 0B \mid 1B \mid \varepsilon$

Give leftmost and rightmost derivation of the string 1001.

Solution :

(i) Leftmost derivation of 1001 (Leftmost variable is picked up for expansion.)

$S \rightarrow$ A1B [Derivation starts from the start symbol]

$\rightarrow$ 1B [Using the production $A \rightarrow \varepsilon$]

$\rightarrow$ 10B [Using the production $B \rightarrow 0B$]

$\rightarrow$ 100B [Using the production $B \rightarrow 0B$]

$\rightarrow$ 1001B [Using the production $B \rightarrow 1B$]

$\rightarrow$ 1001 [Using the production $B \rightarrow \varepsilon$]

Scanned by CamScanner

(ii) Rightmost derivation of 1001 (Rightmost variable is picked up for expansion)

$S \rightarrow A1B$ [Derivation starts from the start symbol]
$\rightarrow A10B$ [Using the production $B \rightarrow 0B$]
$\rightarrow A100B$ [Using the production $B \rightarrow 0B$]
$\rightarrow A1001B$ [Using the production $B \rightarrow 1B$]
$\rightarrow A1001$ [Using the production $B \rightarrow \varepsilon$]
$\rightarrow 1001$ [Using the production $A \rightarrow \varepsilon$]

1.2.2.2 Parse Tree

A set of derivations applied to generate a word can be represented using a tree. Such a tree is known as a parse tree. A parse tree representation gives us a better understanding of :

1. Recursion 2. Grouping of symbols

A parse tree is constructed with the following condition :

1. Root of the tree is represented by start symbol.
2. Each interior mode is represented by a variable belonging to V.
3. Each leaf mode is represented by a terminal or ε.

A string generated by a parse tree is seen from left to right.

Example 1.2.2 : For the grammar given below

$S \rightarrow A1B$

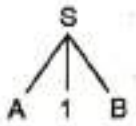
$A \rightarrow 0A \mid \varepsilon$

$B \rightarrow 0B \mid 1B \mid \varepsilon$

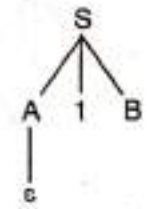
Give parse tree for leftmost and rightmost derivation of the string 1001.

Solution :

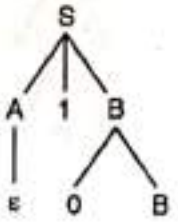
(i) Parse tree for leftmost derivation of 1001.



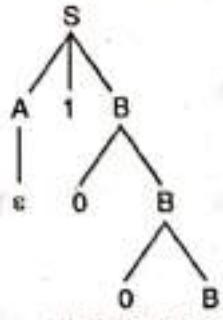
(a) $S \rightarrow A\ 1\ B$



(b) $A \rightarrow \varepsilon$



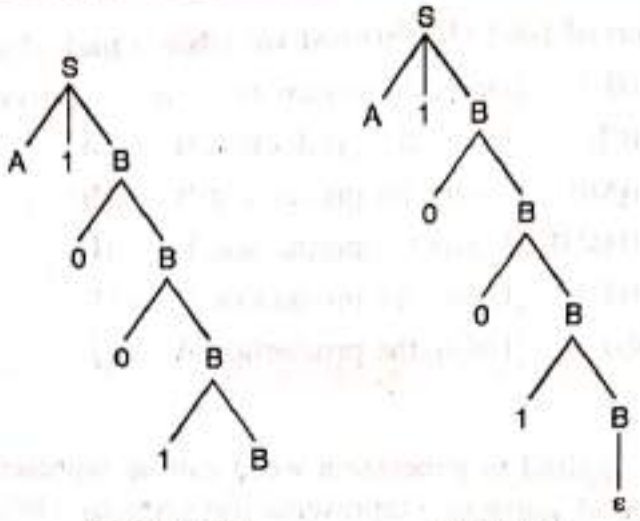
(c) $B \rightarrow 0B$



(d) $B \rightarrow 0B$

Fig. Ex. 1.2.2 Contd...

Scanned by CamScanner

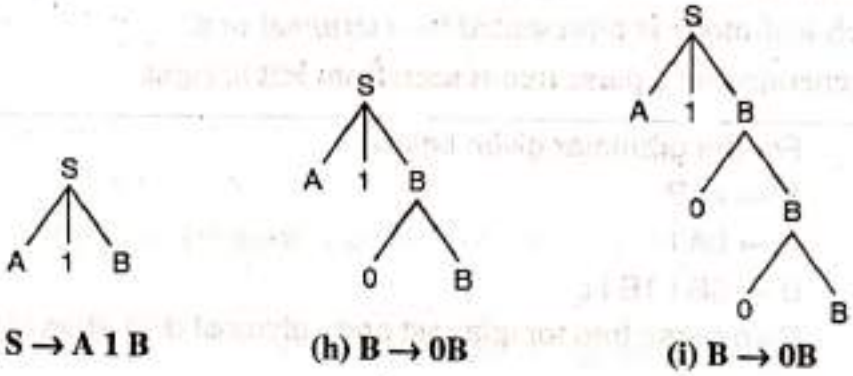


(e) $B \rightarrow 1B$ (f) $B \rightarrow \varepsilon$

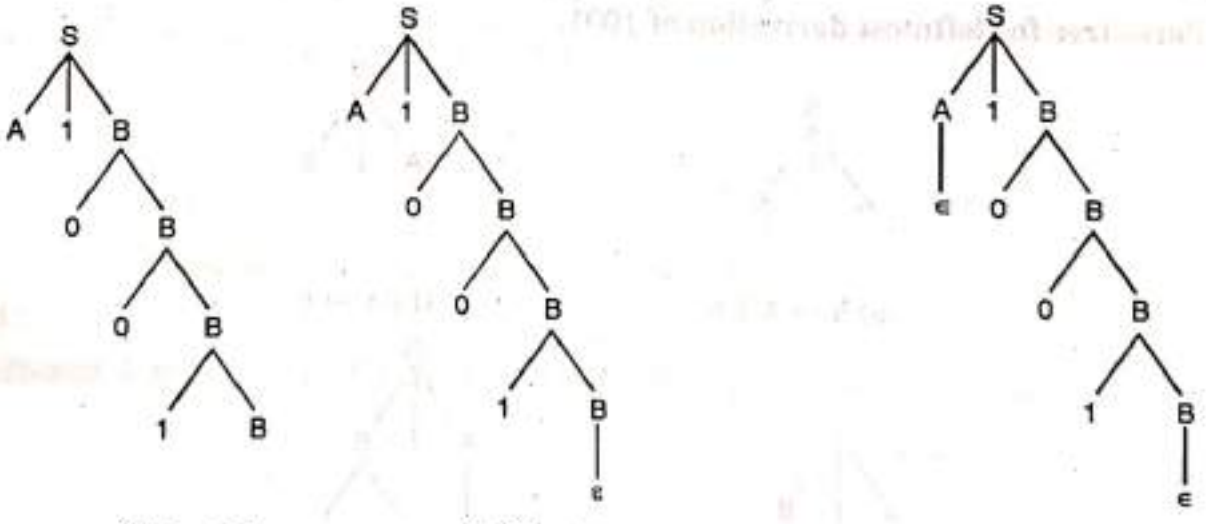
Fig. Ex. 1.2.2

Leftmost derivation of 1001 is shown by series of Figs. Ex. 1.2.2(a) to (f).

(ii) **Parse tree for rightmost derivation of 1001.**



(g) $S \rightarrow A\ 1\ B$ (h) $B \rightarrow 0B$ (i) $B \rightarrow 0B$



(j) $B \rightarrow 1B$ (k) $B \rightarrow \varepsilon$ (l) $A \rightarrow \varepsilon$

Fig. Ex. 1.2.2

Rightmost derivation of 1001 is shown by a series of Figs. Ex. 1.2.2(g) to (*l*).

It may be noted that the final parse trees in Fig. Ex. 1.2.2(f) and (*l*) are identical.

Scanned by CamScanner

Example 1.2.3 : For the grammar given below S → 0S1 | 01

Give derivation of 000111.

Solution :

1. Derivation in sentential form

S → 0S1 [Using production S → 0S1]

S → 00S11 [Using production S → 0S1]

S → 000111 [Using production S → 01]

2. Derivation using parse tree

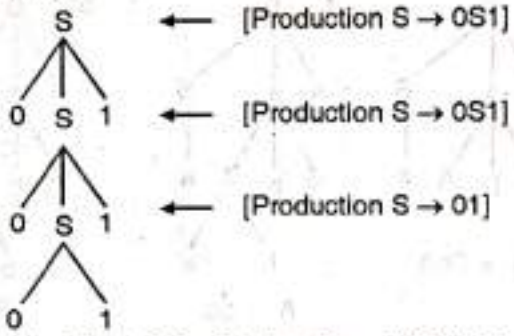


Fig. Ex. 1.2.3 : Derivation of 000111

Example 1.2.4 : Let G = (V, T, P, S) be the CFG having following set of productions. Derive the string "aabbaa" using leftmost derivation and rightmost derivation.

S → aAS | a, A → SbA | SS | ba

Solution :

(i) Leftmost derivation

Leftmost derivation of aabbaa is being shown with the help of the parse tree.

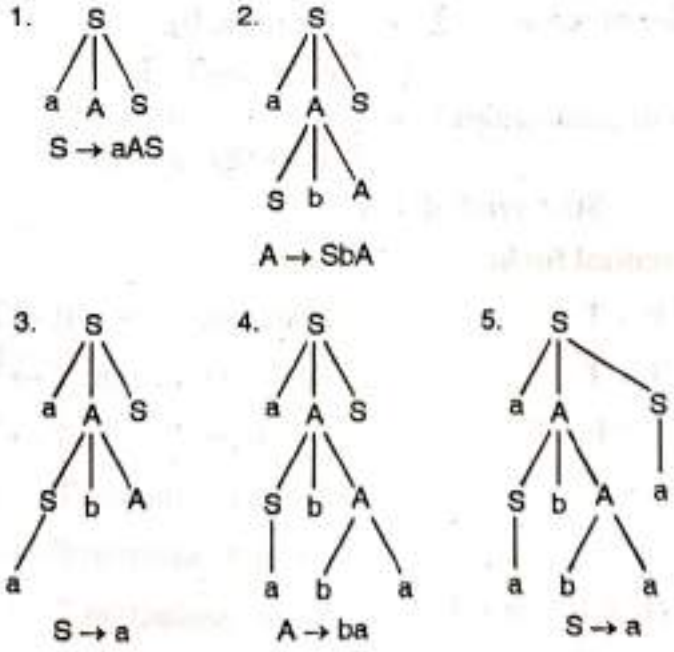


Fig. Ex. 1.2.4(a)

S → aAS → aSbAS → aabAS → aabbas → aabbaa

Scanned by CamScanner

(ii) Rightmost derivation

Rightmost derivation of aabbaa is being shown with the help of the parse tree.

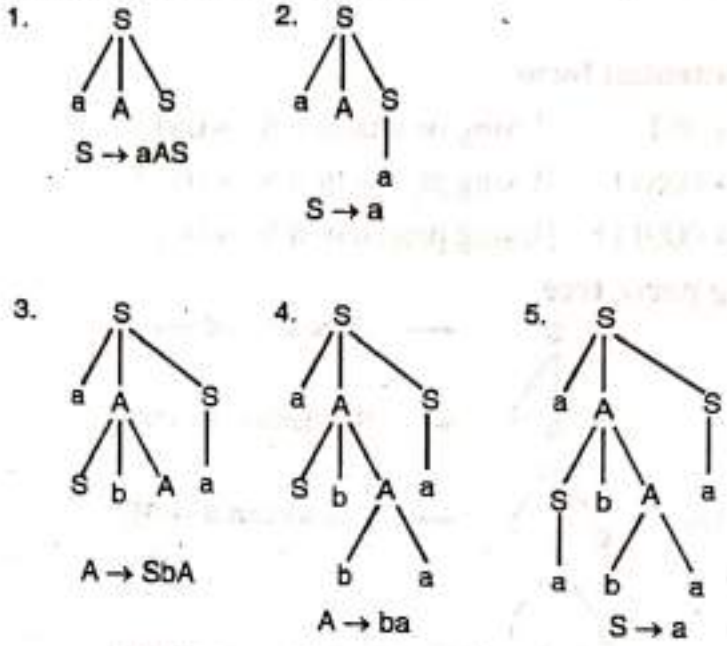


Fig. Ex. 1.2.4(b)

$S \rightarrow aAS \rightarrow aAa \rightarrow aSbAa \rightarrow aSbbaa \rightarrow aabbaa$

Example 1.2.5 : For the grammar given below

$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow (E) \mid a \mid b$

Give the derivation of $(a + b) * a + b$.

Solution : For the grammar given in Example 1.2.5,

Set of variables V = {E, T, F}

Set of terminals Σ = {+, *, (,), a, b}

$$\text{Set of productions } P = \left\{ \begin{array}{l} E \rightarrow E + T \mid T, \\ T \rightarrow T * F \mid F, \\ F \rightarrow (E) \mid a \mid b \end{array} \right\}$$

Start symbol = E

(i) Derivation in sentential form

$E \rightarrow E + T$ [Using production $E \rightarrow E + T$]

$\rightarrow T + T$ [Using production $E \rightarrow T$]

$\rightarrow T * F + T$ [Using production $T \rightarrow T * F$]

$\rightarrow F * F + T$ [Using production $T \rightarrow F$]

$\rightarrow (E) * F + T$ [Using production $F \rightarrow (E)$]

$\rightarrow (E + T) * F + T$ [Using production $E \rightarrow E + T$]

$\rightarrow (T + T) * F + T$ [Using production $E \rightarrow T$]

$\rightarrow (F + F) * F + F$ [Using production $T \rightarrow F$]

Scanned by CamScanner

$\rightarrow (a + b) * a + b$ [Using production $F \rightarrow a \mid b$]

(ii) Derivation using parse tree

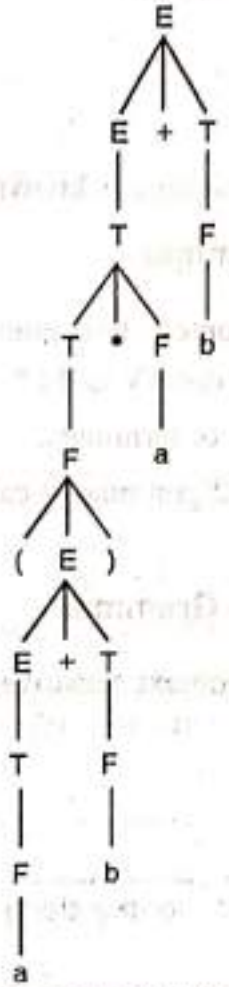


Fig. Ex. 1.2.5 : Parse tree for (a + b) * a + b

1.3 Chomskey Hierarchy

MU - Dec. 2009, Dec. 2012, May 2013, May 2014, Dec. 2014, May 2015, Dec. 2016, May 2017, Dec. 2017

A grammar can be classified on the basis of production rules. Chomsky classified grammars into the following types :

1. Type 3 : Regular grammar
2. Type 2 : Context free grammar
3. Type 1 : Context sensitive grammar
4. Type 0 : Unrestricted grammar.

1.3.1 Type 3 or Regular Grammar

A grammar is called Type 3 or regular grammar if all its productions are of the following forms :

$A \rightarrow \varepsilon$

Scanned by CamScanner

$A \rightarrow a$

$A \rightarrow aB$

$A \rightarrow Ba$

Where, $a \in \Sigma$ and $A, B \in V$.

A language generated by Type 3 grammar is known as regular language.

1.3.2 Type 2 or Context Free Grammar

A grammar is called Type 2 or context free grammar if all its productions are of the following form $A \rightarrow \alpha$ where $A \in V$ and $\alpha \in (V \cup T)^*$.

V is a set of variables and T is a set of terminals.

The language generated by a Type 2 grammar is called a context free language, a regular language but not the reverse.

1.3.3 Type 1 or Context Sensitive Grammar

A grammar is called a Type 1 or context sensitive grammar if all its productions are of the following form.

$\alpha \rightarrow \beta$,

Where, β is atleast as long as α.

Example 1.3.1 : Write a set of production for the strings of the form $a^n b^n c^n$.

Solution :

The set of productions is given by :

$$P = \left\{ \begin{array}{l} S \rightarrow aSBC \mid aBC \\ CB \rightarrow BC,\ aB \rightarrow ab,\ bB \rightarrow bb, \\ bC \rightarrow bc,\ cC \rightarrow cc \end{array} \right\}$$

A string of the form $a^n b^n c^n$ can be generated as given below.

$S \overset{*}{\Rightarrow} a^{n-1} S(BC)^{n-1}$ [by applying $S \rightarrow aSBC$, $(n-1)$ times]

$\Rightarrow a^{n-1} aBC(BC)^{n-1}$ [by applying $S \rightarrow aBC$]

$\Rightarrow a^{n-1} abC(BC)^{n-1}$ [by applying $aB \rightarrow ab$]

$\Rightarrow a^{n-1} abB^{n-1} C^n$ [by applying $CB \rightarrow BC$, $n-1$ times]

$\Rightarrow a^n b^{n-1} bC^n$ [by applying $bB \rightarrow bb$ several times]

$\Rightarrow a^n b^{n-1} bcC^{n-1}$ [by applying $bC \rightarrow bc$]

$\Rightarrow a^n b^n c^n$ [by applying $cC \rightarrow cc$ several times]

Scanned by CamScanner

1.3.4 Type 0 or Unrestricted Grammar

MU - May 2013

Productions can be written without any restriction in a unrestricted grammar. If there is production of the $\alpha \rightarrow \beta$, then length of α could be more than length of β.

- Every grammar also is a Type 0 grammar.
- A Type 2 grammar is also a Type 1 grammar
- A Type 3 grammar is also a Type 2 grammar.

1.4 Finite Automata (FA)

MU - Dec. 2012

Finite automaton or finite state machine can be thought of as a severely restricted model of a computer.

- Every computer has central processing unit; it executes a program stored in a memory. This program normally accepts some input and delivers processed result.
- The word Automata is for automation. A system where energy and information are transformed and used for performing some functions without direct involvement of men is called automaton.
- Finite automata are also called a finite state machine.
- A finite state machine is a mathematical model for actual physical process. By considering the possible inputs on which these machines can work, we can analyse their strengths and weaknesses.
- Finite automata are used for solving several common types of computer algorithms. Some of them are :
  1. Design of digital circuits.
  2. String matching.
  3. Communication protocols for information exchange.
  4. Lexical analyser of a typical compiler.

1.4.1 Working of a Finite Automata

Let us consider a T-flip flop.

A T-flip-flop has :

1. One input denoted by T.
2. Two outputs denoted by Q and $\overline{Q}$.
3. It has two distinct states, defined by the logical values of Q. The flip-flop is in q_0 state when Q = 0 and it is in q_1 state when Q = 1.

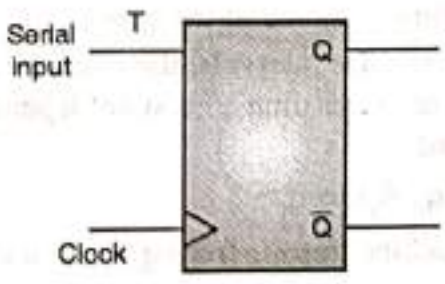


Fig. 1.4.1 : A T-flip flop as a finite automata

Scanned by CamScanner

4. A value of 1 applied to its input changes its state from q_0 to q_1 or from q_1 to q_0.

A T-flip-flop can be used to check whether an input binary number contains an even number of 1's. Let us assume that a binary number 101110011 is applied to input T of the flip-flop. Initial state of the flip-flop is q_0 (i.e. Q = 0)

Input (T)	1	0	1	1	1	0	0	1	1
State (Q)	q_1	q_1	q_0	q_1	q_0	q_0	q_0	q_1	q_0

Fig. 1.4.2 : State transition of T-flip-flop under the input 101110011

From the Fig. 1.4.2, it should be clear that every even number of 1's in input will make the T-flip-flop toggle twice and thus it will be back to q_0. Thus we can conclude the following:

1. If the initial state of flip flop is q_0, it will come back to q_0 if the input number contains even number of 1's.
2. A single T-flip flop can be used as a machine for checking whether a binary number contains an even number of 1's.

The machine starts with initial state q_0 and after feeding the binary number, if the machine is found to be in q_0, the input binary number contains an even number of 1's.

Working of finite automata can be understood with the help of an abstract model of finite automata. It is shown in Fig. 1.4.3.

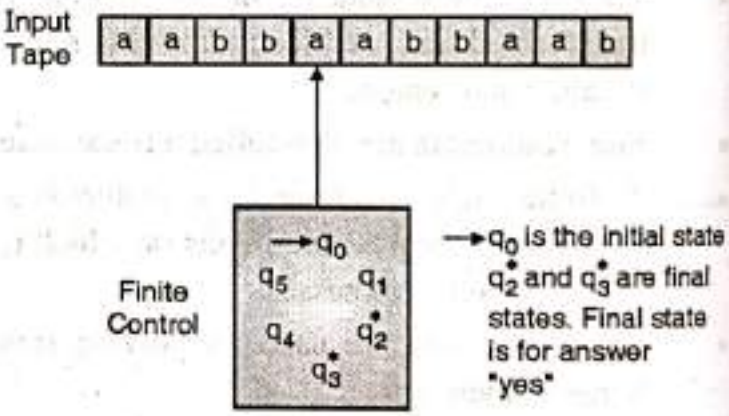


Fig. 1.4.3 : Model of a finite state machine

Operation of the finite automata is given below :

- Input string is fed to the machine through a tape. Tape is divided into squares and each square contains an input symbol.
- The main machine is shown as a box. Machine could be in any of the internal states (q_0, q_1, q_2, q_3, q_4, q_5).
- Initially, the machine is in the starting (initial) state q_0. Reading head is placed at the leftmost square of the tape.
- At regular intervals, the machine reads one symbol from the tape and then enters a new state. Transition to a state depends only on the current state and the input symbol just read.

  $\delta (q_i, A_i) \Rightarrow q_j$

  Machine transits from q_i to q_j on input A_i.
- After reading an input symbol, the reading head moves right to the next square.

Scanned by CamScanner

- This process is repeated again and again, i.e.
  1. A symbol is read.
  2. State of the machine (finite control) changes.
  3. Reading head moves to the right.
- Eventually, the reading head reaches the end of the input string.
- Now, the automaton has to say 'yes' or 'no'. If the machine ends up in one of a set of finial states (q_2, q_3) then the answer is 'yes' otherwise the answer is 'no'.

1.4.2 Some Important Terms

1.4.2.1 Alphabet

An alphabet is a finite, non empty set of symbols. Conventionally, the symbol Σ is used for an alphabet. Common alphabet include :

1. Σ {0, 1}, the binary alphabets.
2. Σ {A, B, ..., Z}, the set of uppercase letters.
3. Σ {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}, the decimal alphabets.
4. Σ {0, 1, 2}, the ternary alphabet.

1.4.2.2 Strings (words)

It is a finite sequence of symbols from the alphabet.

For example,

110110 is a string from binary alphabet.

5920 is a string from decimal alphabet.

'STAMP' is a string from Roman alphabet.

- A string may have no symbol at all, in this case it is known as an **empty string** and is denoted by ε.
- The **length** of a string is number of positions for symbols in it. The standard notation for the length of a string ω is $|\omega|$.

For example,

If $\omega = 01101$

then $|\omega| = 5$

Length of an empty string is zero.

i.e., $|\varepsilon| = 0$.

- The **exponential** notation is used to express the set of all strings of a certain length.

  Σ^k = set of strings of length k, each of whose symbol is in Σ.

Scanned by CamScanner

For example,

if $\Sigma = \{0, 1\}$

then $\Sigma^1 = \{0, 1\}$

$\Sigma^2 = \{00, 01, 10, 11\}$

and $\Sigma^0 = \{\varepsilon\}$

- The **reversal** of string ω is denoted by ω^R.

For example,

If $\omega = xyz$

Then $\omega^R = zyx$.

- A string v is a **substring** of a string ω if and only if there are strings x and y such that,

$\omega = x\ v\ y$, where both x and y could be null

Every string is a substring of itself.

For example,

if ω = 'abcdef' is a string.

then 'cde' is a substring of ω.

- For each string ω, the string ω^i is defined as,

$\omega^0 = \varepsilon$, the empty string

$\omega^{i+1} = \omega^i \cdot \omega$ for each $i \geq 0$ where, i is a natural number.

For example,

if $\omega = abb$

then $\omega^0 = \varepsilon$

$\omega^1 = abb$

$\omega^2 = \omega \cdot \omega = abbabb$

$\omega^3 = \omega^2 \cdot \omega = abbabbabb$

- The set of all strings over an alphabet Σ is denoted by Σ^*.

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \ldots\ldots\ldots\ldots$$

ε — String of length 1 — String of length 2.

For example,

if $\Sigma = \{0, 1\}$

then $\Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, \ldots.\}$

Scanned by CamScanner

1.4.2.3 Languages

A subset of strings over an alphabet Σ is a **language**. In other words, any subset of Σ^* is a language.

If Σ is an alphabet, and $L \subseteq \Sigma^*$, then L is a language over Σ.

For example,

$$L_1 = [\omega \in \{0, 1\}^* \mid \omega \text{ has an equal number of 0's and 1's}]$$

i.e., L_1 is a language over alphabets {0, 1}, having equal number of 0's and 1's.

$$\therefore \quad L_1 = \{\varepsilon, 01, 10, 0011, 0101, 1100, 1010, \ldots\}$$

Thus, a language can be specified by :

$$L = \{\omega \in \Sigma^* \mid \omega \text{ has the given property}\}$$

Some examples of language are

1. A language of all strings, where the string represents a binary number.
   {0, 1, 00, 01, 10, 11, ...}
2. The set of strings of 0's and 1's with an equal number of each.
   {ε, 01, 10, 0011, 0101, 1010, 1100, ...}
3. The set of strings of 0's and 1's, ending in 11.
   {11, 011, 111, 0011, 0111, 1011, ...}
4. Σ^* is a language for any alphabet Σ.
5. ϕ, the empty language, is a language over an alphabet.
6. {ε}, the language consisting of only the empty string, is also a language over any alphabet.

1.4.3 Application of Finite Automata

MU - May 2010

Finite automata are used for solving several common types of computer algorithms. Some of them are :

(i) Design of digital circuit

(ii) String matching

(iii) Communication protocols for information exchange.

(iv) Lexical analysis phase of a compiler.

Finite automata can work as an algorithm for regular language. It can be used for checking whether a string $w \in L$, where L is a regular language.

Scanned by CamScanner

1.5 Finite State Machine (FSM)

MU - Dec. 2010

Finite state machine is a more general form of finite automata. FSM can also generate output. Finite automata that we have discussed so far have a limited capability of either accepting a string or rejecting a string. Acceptance of a string was based on the reachability of a machine from starting state to final state. Finite automata can also be used as an output device.

- Such machines do not have final state/states.
- Machine generates an output on every input. The value of the output is a function of current state and the current input.

Such machines are characterised by two behaviours :

1. State transition function (δ)
2. Output function (λ)

State transition function (δ) is also known as STF.

Output function (λ) is also known as machine function (MTF).

$\delta : \Sigma \times Q \rightarrow Q$

$\lambda : \Sigma \times Q \rightarrow O$ [for Mealy machine]

$\lambda : Q \rightarrow O$ [for Moore machine]

There are two types of automata with outputs :

1. **Mealy machine :** Output is associated with transition.

   $\lambda : \Sigma \times Q \rightarrow O$

   Set of output alphabet O can be different from the set of input alphabet Σ

2. **Moore machine :** Output is associated with state.

   $\lambda : Q \rightarrow O$

1.6 University Questions and Answers

Dec. 2009

Q. 1 Write note on Chomsky Hierarchy. **(Section 1.3)** **(5 Marks)**

May 2010

Q. 2 State applications of Finite Automata in brief. **(Section 1.4.3)** **(3 Marks)**

Scanned by CamScanner

Dec. 2012

Q. 3 What is Finite Automata? **(Section 1.4)** **(2 Marks)**

Q. 4 Explain Chomsky Hierarchy with languages used, forms of productions in grammars and accepting device. **(Section 1.3)** **(5 Marks)**

May 2013

Q. 5 Explain Chomsky Hierarchy for formal languages. **(Section 1.3)** **(5 Marks)**

Q. 6 Define the following term : Unrestricted grammar **(Section 1.3.4)** **(2 Marks)**

May 2014

Q. 7 Write short note on : Chomsky Hierarchy of Grammar **(Section 1.3)** **(5 Marks)**

Dec. 2014

Q. 8 Give Chomsky hierarchy of grammar. **(Section 1.3)** **(5 Marks)**

May 2015

Q. 9 Explain with an example the Chomsky hierarchy. **(Section 1.3)** **(5 Marks)**

Dec. 2016

Q. 10 Write detailed note on : Chomsky Hierarchy. **(Section1.3)** **(10 Marks)**

May 2017

Q. 11 Explain Chomsky Hierarchy. **(Section 1.3)** **(5 Marks)**

Dec. 2017

Q. 12 Explain Chomsky Hierarchy. **(Section 1.3)** **(5 Marks)**

Chapter Ends...

❑❑❑

Scanned by CamScanner

Finite Automata

2.1 Introduction to Finite Automata

MU - Dec. 2012

Finite automaton or finite state machine can be thought of as a severely restricted model of a computer.

- Every computer has central processing unit; it executes a program stored in a memory. This program normally accepts some input and delivers processed result.
- The word Automata is for automation. A system where energy and information are transformed and used for performing some functions without direct involvement of men is called automaton.
- A finite automata are also called a finite state machine.
- A finite state machine is a mathematical model for actual physical process. By considering the possible inputs on which these machines can work, we can analyse their strengths and weaknesses.
- Finite automata are used for solving several common types of computer algorithms. Some of them are :
  1. Design of digital circuits.
  2. String matching.
  3. Communication protocols for information exchange.
  4. Lexical analyser of a typical compiler.

Scanned by CamScanner

2.1.1 Working of a Finite Automata

Let us consider a T-flip flop.

A T-flip-flop has :

1. One input denoted by T.
2. Two outputs denoted by Q and $\overline{Q}$.
3. It has two distinct states, defined by the logical values of Q. The flip-flop is in q_0 state when Q = 0 and it is in q_1 state when Q = 1.

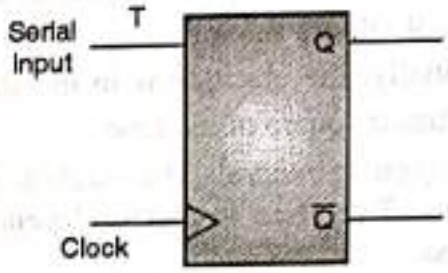


Fig. 2.1.1 : A T-flip flop as a finite automata

4. A value of 1 applied to its input changes its state from q_0 to q_1 or from q_1 to q_0.

A T-flip-flop can be used to check whether an input binary number contains an even number of 1's. Let us assume that a binary number 101110011 is applied to input T of the flip-flop. Initial state of the flip-flop is q_0 (i.e. Q = 0)

Input (T)	1	0	1	1	1	0	0	1	1
State (Q)	q_1	q_1	q_0	q_1	q_0	q_0	q_0	q_1	q_0

Fig. 2.1.2 : State transition of T-flip-flop under the input 101110011

From the Fig. 2.1.2, it should be clear that every even number of 1's in input will make the T-flip-flop toggle twice and thus it will be back to q_0. Thus we can conclude the following :

1. If the initial state of flip flop is q_0, it will come back to q_0 if the input number contains even number of 1's.
2. A single T-flip flop can be used as a machine for checking whether a binary number contains an even number of 1's.

The machine starts with initial state q_0 and after feeding the binary number, if the machine is found to be in q_0, the input binary number contains an even number of 1's.

Working of finite automata can be understood with the help of an abstract model of a finite automata. It is shown in Fig. 2.1.3.

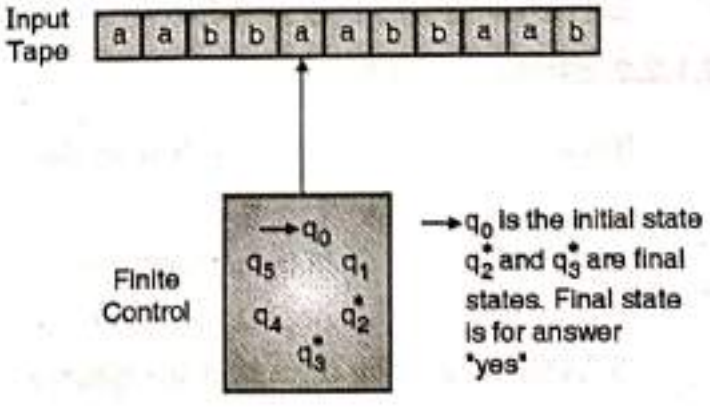


Fig. 2.1.3 : Model of a finite state machine

Scanned by CamScanner

Operation of the finite automata is given below :

- Input string is fed to the machine through a tape. Tape is divided into squares and each square contains an input symbol.
- The main machine is shown as a box. Machine could be in any of the internal states (q_0, q_1, q_2, q_3, q_4, q_5).
- Initially, the machine is in the starting (initial) state q_0. Reading head is placed at the leftmost square of the tape.
- At regular intervals, the machine reads one symbol from the tape and then enters a new state. Transition to a state depends only on the current state and the input symbol just read.

  $\delta (q_i, A_i) \Rightarrow q_j$

  Machine transits from q_i to q_j on input A_i.
- After reading an input symbol, the reading head moves right to the next square.
- This process is repeated again and again, i.e.
  1. A symbol is read.
  2. State of the machine (finite control) changes.
  3. Reading head moves to the right.
- Eventually, the reading head reaches the end of the input string.
- Now, the automaton has to say 'yes' or 'no'. If the machine ends up in one of a set of finial states (q_2, q_3) then the answer is 'yes' otherwise the answer is 'no'.

2.1.2 Some Important Terms

2.1.2.1 Alphabet

An alphabet is a finite, non empty set of symbols. Conventionally, the symbol Σ is used for an alphabet. Common alphabet include :

1. Σ {0, 1}, the binary alphabets.
2. Σ {A, B, ..., Z}, the set of uppercase letters.
3. Σ {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}, the decimal alphabets.
4. Σ {0, 1, 2}, the ternary alphabet.

2.1.2.2 Strings (words)

It is a finite sequence of symbols from the alphabet.

For example :

110110 is a string from binary alphabet.

5920 is a string from decimal alphabet.

'STAMP' is a string from Roman alphabet.

Scanned by CamScanner

- A string may have no symbol at all, in this case it is known as an **empty string** and is denoted by ε.
- The **length** of a string is number of positions for symbols in it. The standard notation for the length of a string ω is $|\omega|$.

  For example,

  If $\omega = 01101$

  then $|\omega| = 5$

  Length of an empty string is zero.

  i.e., $|\varepsilon| = 0$.
- The **exponential** notation is used to express the set of all strings of a certain length.

  Σ^k = set of strings of length k, each of whose symbol is in Σ.

  For example,

  if $\Sigma = \{0, 1\}$

  then $\Sigma^1 = \{0, 1\}$

  $\Sigma^2 = \{00, 01, 10, 11\}$

  and $\Sigma^0 = \{\varepsilon\}$
- The **reversal** of string ω is denoted by ω^R.

  For example,

  If $\omega = xyz$

  Then $\omega^R = zyx$.
- A string v is a **substring** of a string ω if and only if there are strings x and y such that,

  $\omega = x\,v\,y$, where both x and y could be null

  Every string is a substring of itself.

  For example,

  if ω = 'abcdef' is a string.

  then 'cde' is a substring of ω.
- For each string ω, the string ω^i is defined as,

  $\omega^0 = \varepsilon$, the empty string

  $\omega^{i+1} = \omega^i \cdot \omega$ for each $i \geq 0$ where, i is a natural number.

  For example,

  if $\omega = abb$

  then $\omega^0 = \varepsilon$

  $\omega^1 = abb$

  $\omega^2 = \omega \cdot \omega = abbabb$

  $\omega^3 = \omega^2 \cdot \omega = abbabbabb$

Scanned by CamScanner

- The set of all strings over an alphabet Σ is denoted by Σ^*.

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \ldots\ldots\ldots\ldots$$

Σ^0: ε — Σ^1: String of length 1 — Σ^2: String of length 2.

For example,

if $\Sigma = \{0, 1\}$

then $\Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, \ldots.\}$

2.1.2.3 Languages

A subset of strings over an alphabet Σ is a **language**. In other words, any subset of Σ^* is a language.

If Σ is an alphabet, and $L \subseteq \Sigma^*$, then L is a language over Σ.

For example,

$$L_1 = [\omega \in \{0, 1\}^* \mid \omega \text{ has an equal number of 0's and 1's}]$$

i.e., L_1 is a language over alphabets {0, 1}, having equal number of 0's and 1's.

$$\therefore \quad L_1 = \{\varepsilon, 01, 10, 0011, 0101, 1100, 1010, \ldots\}$$

Thus, a language can be specified by :

$$L = \{\omega \in \Sigma^* \mid \omega \text{ has the given property}\}$$

Some examples of language are :

1. A language of all strings, where the string represents a binary number.
   {0, 1, 00, 01, 10, 11, ...}
2. The set of strings of 0's and 1's with an equal number of each.
   {ε, 01, 10, 0011, 0101, 1010, 1100, ...}
3. The set of strings of 0's and 1's, ending in 11.
   {11, 011, 111, 0011, 0111, 1011, ...}
4. Σ^* is a language for any alphabet Σ.
5. ϕ, the empty language, is a language over an alphabet.
6. {ε}, the language consisting of only the empty string, is also a language over any alphabet.

2.1.3 Applications of Finite Automata

MU - May 2010, May 2014

Finite automata are used for solving several common types of computer algorithms. Some of them are :

(i) Design of digital circuit

Scanned by CamScanner

(ii) String matching

(iii) Communication protocols for information exchange.

(iv) Lexical analysis phase of a compiler.

Finite automata can work as an algorithm for regular language. It can be used for checking whether a string $w \in L$, where L is a regular language.

2.2 Deterministic Finite Automata (DFA)

The word "deterministic" refers to the fact that the transition is deterministic. There is only one state to which an automaton can transit from the current state on each input. The word "finite" implies that number of states are finite. A finite automata consists of five parts :

1. A finite set of states, represented as Q.
2. A finite set of alphabet, represented as Σ.
3. An initial state, represented as q_0.
4. A set of accepting states. An accepting state or final state is for 'yes' answer. It is a subset of Q and is represented as F.

   $F \subseteq Q$ [F is subset of Q]
5. A next state function or a transition function. Next state depends on the current state and the current input. It is a function from $Q \times \Sigma$ to Q. It is represented as δ.

2.2.1 Definition of a DFA

MU – Dec. 2006 , May 2010

A deterministic finite automata is a quintuple.

$M = (Q, \Sigma, \delta, q_0, F)$, where

Q is a set of states.

Σ is a set of alphabet.

$q_0 \in Q$ is the initial state,

$F \subseteq Q$ is the set of final states, and δ, the transition function, is a function from $Q \times \Sigma$ to Q.

2.2.2 Representation of a DFA

Let the machine M be a deterministic finite automata.

$$M = \{Q, \Sigma, \delta, q_0, F\}, \text{ where}$$

$$Q = \{q_0, q_1\}$$

$$\Sigma = (0, 1)$$

q_0 is the starting state

$$F = \{q_1\}.$$

Scanned by CamScanner

and δ is the transition function as given below :

$$\delta (q_0,0) \Rightarrow q_0.$$

$$\delta (q_0,1) \Rightarrow q_1$$

$$\delta (q_1,0) \Rightarrow q_1$$

$$\delta (q_1,1) \Rightarrow q_0.$$

The above representation of transition function is not very readable. Conventionally, there are two representations for transition function :

1. State transition table.
2. State transition diagram.

1. State transition table

The Fig. 2.2.1 shows the state transition table of the transition function discussed in this section.

	0	1 ← Input alphabets.
$\rightarrow q_0$	q_0	q_1 → before q_0 indicates it is the starting state
q_1^*	q_1	q_0 * indicates a final state.

↑ states

Fig. 2.2.1 : State transition table

- In state transition table, there is a row for every state $q_i \in Q$.
- In state transition table, there is a column for every alphabet $A_i \in \Sigma$.
- Starting state is marked as '→'.
- Final state is marked as '*'.

2. State transition diagram

The Fig. 2.2.2 shows the state transition diagram of the transition function discussed in this section.

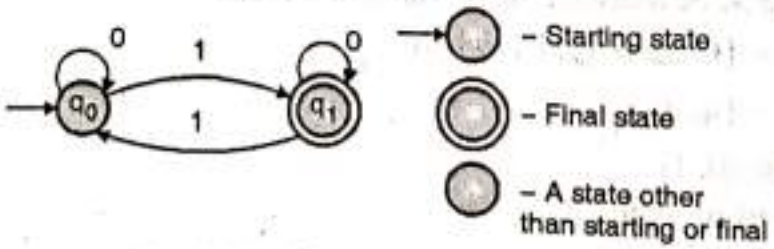


Fig. 2.2.2 : State transition diagram

Scanned by CamScanner

Language of the above DFA

- It is easy to see that machine remain in same state on input 0.
- The machine transitions from q_0 to q_1 and q_1 to q_0, when the input is 1.
- Machine will be in state q_1 (final state), after reading odd number of 1's.

Thus we can conclude that the DFA accepts a string if and only if the number of 1's is odd.

The language L accepted by M, the given DFA, is the set of all strings having odd number of 1's.

Or,

$L(M) = \{\omega \in \{0, 1\}^* \mid \omega \text{ contains odd number of 1's}\}$

Simulating the functioning of the above DFA for a given input

If M is given an input 0010110, its initial configuration is $(q_0, 0010110)$.

$$(q_0, 0010110) \overset{\delta}{\Rightarrow} (q_0, 010110) \text{ [input 0 in state } q_0\text{]}$$
$$\overset{\delta}{\Rightarrow} (q_0, 10110) \text{ [input 0 in state } q_0\text{]}$$
$$\overset{\delta}{\Rightarrow} (q_1, 0110) \text{ [input 1 in state } q_0\text{]}$$
$$\overset{\delta}{\Rightarrow} (q_1, 110) \text{ [input 0 in state } q_1\text{]}$$
$$\overset{\delta}{\Rightarrow} (q_0, 10) \text{ [input 1 in state } q_1\text{]}$$
$$\overset{\delta}{\Rightarrow} (q_1, 0) \text{ [input 1 in state } q_0\text{]}$$
$$\overset{\delta}{\Rightarrow} (q_1, \varepsilon) \text{ [input 0 in state } q_1\text{]}$$

Therefore,

$(q_0, 0010110) \overset{\delta^*}{\Rightarrow} (q_1, \varepsilon)$ and so the input 0010110 is accepted as q_1 is a final state.

2.2.3 Designing a DFA

Designing a DFA is like writing a program. In a program, the current state of a program is given by :

1. Values stored in variables.
2. Current line of execution.

In case of a DFA, variables are not used. Everything is remembered through explicit states. Transition function of a DFA is similar to algorithm. While designing a DFA, one has to concentrate on four issues.

Scanned by CamScanner

These are :

1.	**Number of states**	**2.**	**Transition function**
3.	**Start state**	**4.**	**Final states**

Number of states

Number of states depends on "what must be remembered as input symbols are read by a DFA."

For example,

1. DFA to check whether a binary numbers has even number of 1's :

DFA has to remember whether the number of 1's seen so for is even or odd. It does not have to count number of 1's. Let us look at a binary number given below :

Binary number →	0	1	1	0	1	1	0	0	1	1
Number of 1's →	even	odd	even	even	odd	even	even	even	odd	even

- Machine will have two states :
  - (i) One corresponding to even number of 1's, so far.
  - (ii) One corresponding to odd number of 1's, so far.
- These two states are necessary and sufficient to conclude whether a binary number contains even number of 1's.

2. DFA to check whether a string over alphabets (a, b) contains a substring abb :

DFA has to search for substring abb in input stream. Input string is not stored in a memory variable. Input is made one symbol at a time. As the substring is of length 3, it may be necessary to remember the preceding two symbols.

- Preceding symbol is a. (a is the first symbol in abb).
- Preceding two symbols are ab. (ab constitutes first two symbols of abb).
- DFA has already seen the substring abb in the input string. Once the substring abb is found in input string, this status will not change irrespective of what follows thereafter in the input string.
- Preceding symbols are neither a nor ab, and abb has not come earlier.

The machine will have four states, each standing for one of the four situations.

Transition function

Transition function gives the next-state, depending on :

1. Current state
2. Current input

A transition function is problem specific and it depends on the problem.

Scanned by CamScanner

For example

1. Give transition function for a "DFA to check whether a binary number has even number of 1's".

We have already seen that the corresponding DFA will have two states.

(i) State q_0, indicating even number of 1's seen so far.

(ii) State q_1, indicating odd number of 1's seen so far.

- 0 as next input will have no effect on number of 1's.
- 1 as next input will make the transition from q_0 to q_1 and from q_1 to q_0. Thus, the transition behaviour can be described using the Fig. 2.2.3.

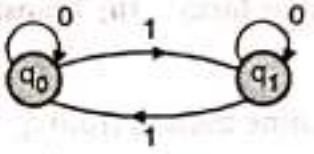
$\delta(q_0,0) \Rightarrow q_0$

$\delta(q_0,1) \Rightarrow q_1$

$\delta(q_1,0) \Rightarrow q_1$

$\delta(q_1,1) \Rightarrow q_0$

	0	1
q_0	q_0	q_1
q_1	q_1	q_0



(a) Tabular form **(b) Transition table** **(c) Transition diagram**

Fig. 2.2.3 : Transition behaviour

2. Give transition function for a "DFA to check whether a string over alphabets (a, b) contains a substring abb".

We have already seen that the corresponding DFA will have four states.

1. State q_1, preceding symbol is a.
2. State q_2, preceding two symbols are ab.
3. State q_3, substring abb has already been seen in input string.
4. State q_0, situations q_1, q_2, or q_3 are not there.

State of the DFA after every input symbol for a sample input data is shown in Fig. 2.2.4.

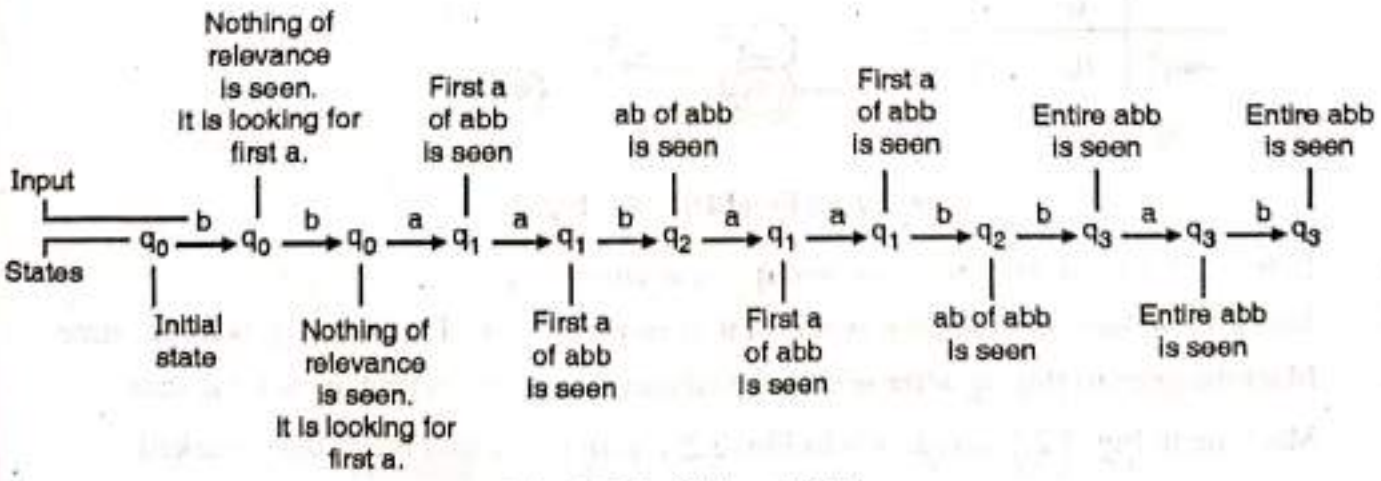


Fig. 2.2.4 : States of DFA

Scanned by CamScanner

Transition behaviour is described using Fig. 2.2.5.

$\delta(q_0,b) \Rightarrow q_0$

$\delta(q_0,a) \Rightarrow q_1$

$\delta(q_1,a) \Rightarrow q_1$

$\delta(q_1,b) \Rightarrow q_2$

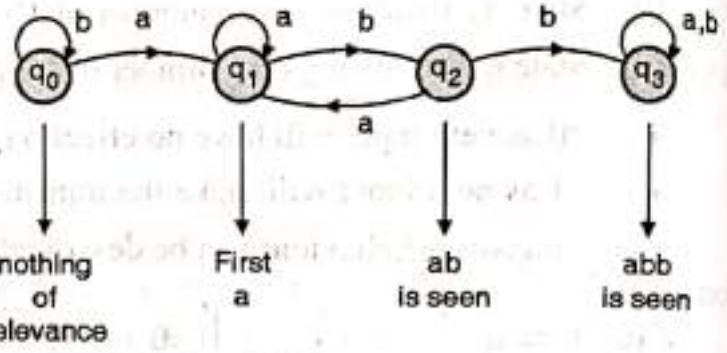
$\delta(q_2,a) \Rightarrow q_1$

$\delta(q_2,b) \Rightarrow q_3$

$\delta(q_3,a) \Rightarrow q_3$

$\delta(q_3,b) \Rightarrow q_3$

	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_2
q_3^*	q_3	q_3



(a) Tabular form **(b) Transition table** **(c) Transition diagram**

Fig. 2.2.5 : Transition behaviour

- Machine transits from q_0 to q_1 on input a, as the first symbol a of abb is seen.
- Machine remains in q_1 state on input a, as the previous two symbols aa will still make the first symbol a of abb.
- Machine transits from q_1 to q_2 on input b, as ab of abb is seen.
- Machine transits from q_2 to q_3 on input b, as entire abb is seen.
- Machine transits from q_2 to q_1 on input a, as the previous three symbols aba will still make the first symbol a of aba.

Starting state and final states :

In Fig. 2.2.3(c), q_0 is both the initial state and final state as :

1. Zero number of 1's is even number of 1's.
2. Machine will have "yes" answer for even number of 1's. q_0 is final state. Machine in Fig. 2.2.3(c) is redrawn in Fig. 2.2.6 with initial and final states marked.

	0	1
$\rightarrow q_0^*$	q_0	q_1
q_1	q_1	q_0

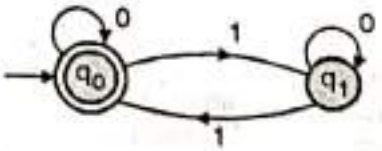


Fig. 2.2.6 : Final DFA for Fig. 2.2.3

In Fig. 2.2.5(c), q_0 is initial state and q_3 is the final state.

1. Initially, we have nothing that is relevant to substring abb. Therefore, q_0 is initial state.
2. Machine goes to state q_3 after seeing the substring abb. Therefore, q_3 is final state.

Machine in Fig. 2.2.5 is redrawn in Fig. 2.2.7 with initial and final states marked.

Scanned by CamScanner

	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_3
q_3^*	q_3	q_3

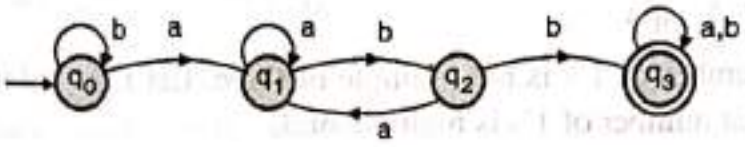


Fig. 2.2.7 : Final DFA for Fig. 2.2.5(c)

2.2.4 Examples on DFA

2.2.4.1 Examples on Counting of Symbols

Example 2.2.1 : Give deterministic finite automata accepting the following language over the alphabet {0, 1}.

(a) Number of 1's is multiple of 3.

(b) Number of 1's not multiple of 3.

Solution :

(a) Number of 1's is multiple of 3.

Number of 1's seen, so far by the machine can be written as :

(i) 3n

(ii) 3n + 1

(iii) 3n + 2

Corresponding to three cases mentioned above, there will be three states.

State q_0 – no. of 1's, so far is 3n

State q_1 – no. of 1's, so far is 3n + 1

State q_2 – no. of 1's, so far is 3n + 2

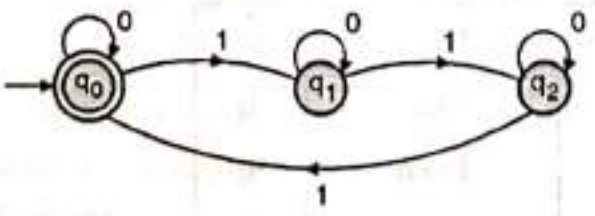
An input of 1 will cause a transition from :

q_0 to q_1, if the machine is in q_0.

q_1 to q_2, if the machine is in q_1.

q_2 to q_0, if the machine is in q_2.

An input of 0 will not cause any transition.



(a) Transition diagram

	0	1
$\rightarrow q_0^*$	q_0	q_1
q_1	q_1	q_2
q_2	q_2	q_0

(b) Transition table

Fig. Ex. 2.2.1 : Final DFA for Example 2.2.1(a)

Scanned by CamScanner

- q_0 is the starting state. Zero number of 1's implies that number of 1's is of the form 3n.
- q_0 is a final state. Machine will be in q_0 if number of 1's seen so far is multiple of 3.

(b) Number of 1's is not multiple of three. Let L be a language over alphabets {0, 1} such that number of 1's is multiple of 3.

Thus,

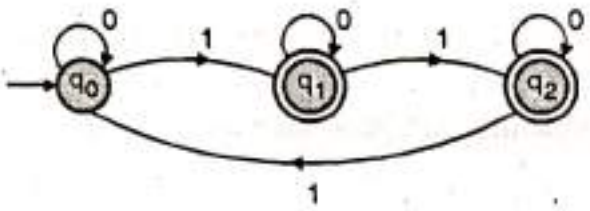
$$L = [\omega \in \{0, 1\}^* \mid \text{no. of 1's in } \omega \text{ is multiple of 3}]$$

Complement of L is given by :

$$L' = [\omega \in \{0, 1\}^* \mid \text{no. of 1's in } \omega \text{ is not multiple of 3}]$$

DFA for L′ can be obtained through following modifications on DFA for L.

1. Every final state of L should become a non-accepting state in L′.
2. Every non-accepting state of L should become a final state in L′.



(c) Transition diagram

	0	1
$\rightarrow q_0$	q_0	q_1
q_1^*	q_1	q_2
q_2^*	q_2	q_0

(d) Transition table

Fig. Ex. 2.2.1 : Final DFA for Example 2.2.1(b)

Example 2.2.2 : Give deterministic finite automata accepting the following languages over the alphabet {0, 1}
(a) Number of 1's is even and number of 0's is even.
(b) Number of 1's is odd and number of 0's is odd. **MU - May 2010**

Solution :

(a) Number of 1's is even and number of 0's is even.

At any instance of time, we will have following cases for number of 0's and number of 1's seen by the machine.

Situations		State
Number of 0's	Number of 1's	
Even	Even	q_0
Even	Odd	q_1
Odd	Even	q_2
Odd	Odd	q_3

Scanned by CamScanner

- An input 0 in state q_0, will make number of 0's odd.

  $\delta (q_0, 0) \Rightarrow q_2$

- An input 1 in state q_0, will make number of 1's odd.

  $\delta (q_0, 1) \Rightarrow q_1$

- An input 0 in state q_1, will make number of 0's odd.

  $\delta (q_1, 0) \Rightarrow q_3$

- An input 1 in state q_1, will make number of 1's even.

  $\delta (q_1, 1) \Rightarrow q_0$

- An input 0 in state q_2, will make number of 0's even.

  $\delta (q_2, 0) \Rightarrow q_0$

- An input 1 in state q_2, will make number of 1's odd.

  $\delta (q_2, 1) \Rightarrow q_3$

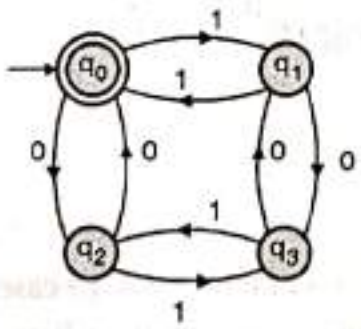
- An input 0 in state q_3, will make number of 0's even.

  $\delta (q_3, 0) \Rightarrow q_1$

- An input 1 in state q_3, will make number of 1's even.

  $\delta (q_3, 0) \Rightarrow q_2$

- q_0 is the starting state. An empty string contains even number of 0's and even number of 1's.
- q_0 is a final state. q_0 stands for even number of 0's and even number of 1's.



(a) Transition diagram

	0	1
$\rightarrow q_0$	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

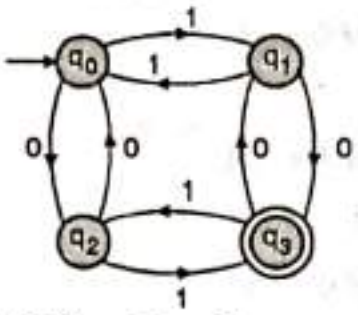
(b) Transition table

Fig. Ex. 2.2.2 : Final DFA for Example 2.2.2(a)

(b) Number of 1's is odd and number of 0's is odd.

In solution of Example 2.2.2(a), the state q_3 stands for odd number of 0's should be declared as final state.

Scanned by CamScanner



(c) Transition diagram

	0	1
$\rightarrow q_0$	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3^*	q_1	q_2

(d) Transition table

Fig. Ex. 2.2.2 : Final DFA for Example 2.2.2(b)

Example 2.2.3 Design a DFA which accepts the odd number 1's and any number of 0's over $\Sigma = \{0, 1\}$.

Solution :

The DFA must keep track of number of 1's in the string already seen by it.

- The number of 1's seen could be even, state q_0.
- The number of 1's seen could be odd, state q_1.

The required DFA is given in Fig. Ex. 2.2.3.

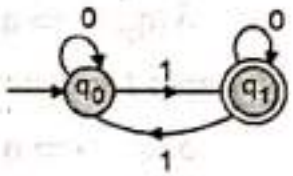


Fig. Ex. 2.2.3

Example 2.2.4 : Design a DFA for a set of strings over alphabet {0, 1} such that the number of 0's is divisible by five, and number of 1's divisible by 3.

Solution : At any instance of time, we will have following cases for number of 0's.

Case 1 – 5n
Case 2 – 5n + 1
Case 3 – 5n + 2
Case 4 – 5n + 3
Case 5 – 5n + 4

Number of 0's should be divisible by 5.

Similarly, there will be three cases for number of 1's.

Case 1 – 3m
Case 2 – 3m + 1
Case 3 – 3m + 2

Number of 1's is divisible by 3.

Thus, depending on number of 0's and 1's there will be $5 \times 3 = 15$ cases.

Let us represent a state of the DFA under consideration as q_{ij}, where i can take a value from 0 to 4 depending on number of 0's seen so far :

q_{0j} – number of 0's is 5n
q_{1j} – number of 0's is 5n + 1
q_{2j} – number of 0's is 5n + 2
q_{3j} – number of 0's is 5n + 3
q_{4j} – number of 0's is 5n + 4

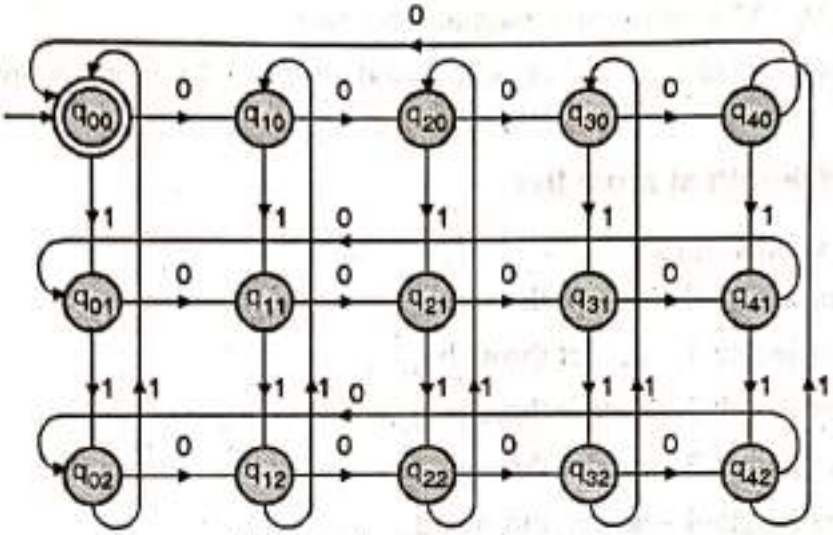
Scanned by CamScanner

Similarly, in state q_{ij} can take value from 0 to 2 depending on number of 1's seen so far.

q_{i0} – number of 1's is 3m

q_{i1} – number of 1's is 3m + 1

q_{i2} – number of 1's is 3m + 2



(a) State transition diagram

	0	1
$\rightarrow q_{00}^*$	q_{10}	q_{01}
q_{01}	q_{11}	q_{02}
q_{02}	q_{12}	q_{00}
q_{10}	q_{20}	q_{11}
q_{11}	q_{21}	q_{12}
q_{12}	q_{22}	q_{10}
q_{20}	q_{30}	q_{21}
q_{21}	q_{31}	q_{22}
q_{22}	q_{32}	q_{20}
q_{30}	q_{40}	q_{31}
q_{31}	q_{11}	q_{32}
q_{32}	q_{42}	q_{30}
q_{40}	q_{00}	q_{41}
q_{41}	q_{01}	q_{42}
q_{42}	q_{02}	q_{40}

(b) State transition table

Fig. Ex. 2.2.4 : Final DFA for Example 2.2.4

Scanned by CamScanner

Example 2.2.5 : Draw DFA for the following language over {0, 1} :

(i) All strings of length at most five

(ii) All strings with exactly two 1's

(iii) All string containing at least two 0's.

(iv) All strings containing at most two 0's.

(v) All strings starting with 1 and length of the string is divisible by 3.

Solution :

(i) All strings of length at most five

These are the valid strings :

String of length 0 – accept through q_0.

String of length 1 – accept through q_1.

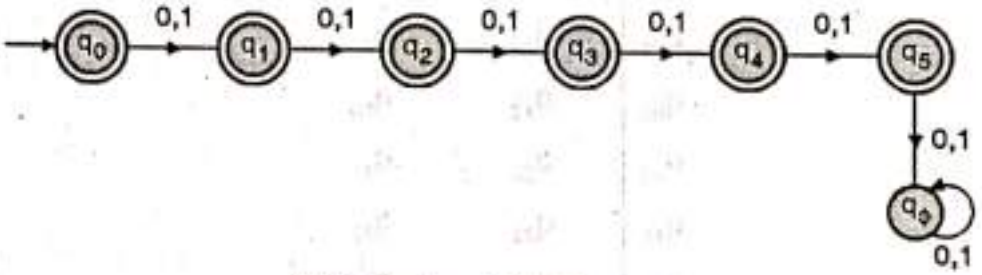
String of length 2 – accept through q_2.

String of length 3 – accept through q_3.

String of length 4 – accept through q_4.

String of length 5 – accept through q_5.

String of length > 5 should be rejected through a dead state or a failure state. A failure state is designated as q_ϕ.



(a) State transition diagram

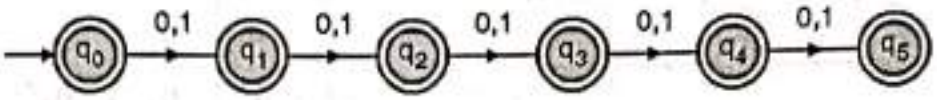
	0	1
$\rightarrow q_0^*$	q_1	q_1
q_1^*	q_2	q_2
q_2^*	q_3	q_3
q_3^*	q_4	q_4
q_4^*	q_5	q_5
q_5	q_ϕ	q_ϕ
q_ϕ	q_ϕ	q_ϕ

(b) State transition table

Fig. Ex. 2.2.5 : Final DFA for Example 2.2.5(i) with explicit failure state

Scanned by CamScanner

It is not necessary to mention the failure/dead state explicitly. If a transition is not mentioned, it is taken as a failure transition. Thus, the solution can be given with out explicit failure state.



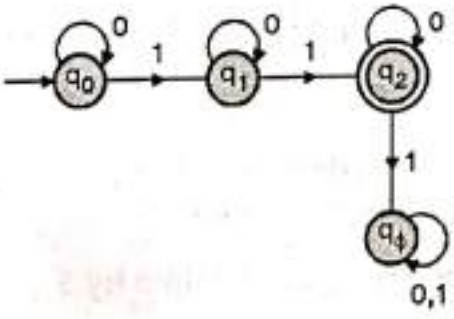
(c) State transition diagram

	0	1
$\rightarrow q_0^*$	q_1	q_1
q_1^*	q_2	q_2
q_2^*	q_3	q_3
q_3^*	q_4	q_4
q_4^*	q_5	q_5
q_5^*	ϕ	ϕ

(d) State transition table

Fig. Ex. 2.2.5 : Final DFA for Example 2.2.5(i) without a failure state or with failure transition omitted.

(ii) All strings with exactly two 1's



	0	1
$\rightarrow q_0$	q_0	q_1
q_1	q_1	q_2
q_2^*	q_2	q_ϕ
q_ϕ	q_ϕ	q_ϕ

(e) State transition diagram **(f) State transition table**

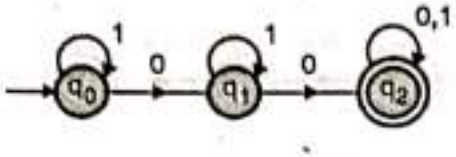
Fig. Ex. 2.2.5 : Final DFA for Example 2.2.5(ii)

- First 1 takes the machine from q_0 to q_1.
- Second 1 takes the machine from q_1 to q_2.

  q_2 is a final state.
- A 1 in q_2 takes the machine to a failure state q_ϕ.

Scanned by CamScanner

(iii) All string containing at least two 0's



(g) State transition diagram

	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_1
q_2^*	q_2	q_2

(h) State transition table

Fig. Ex. 2.2.5 : Final DFA for Example 2.2.5(iii)

- First two 0's will take the machine from q_0 to q_2. Thereafter, the machine remains in q_2 as the string should contain at least two 0's.

(iv) All strings containing at most two 0's

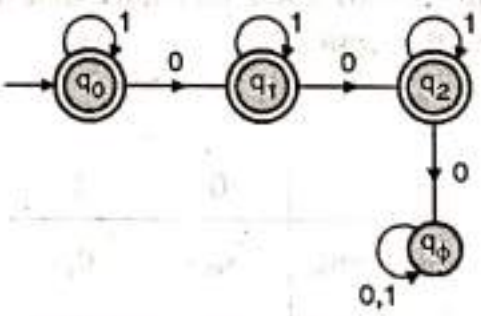
These are the valid strings :

A string not containing 0 – accept through q_0.

A string containing one 0 – accept through q_1.

A string containing two 0's – accept through q_2.

Another occurrence of 0 in state q_2 will cause a transition to dead state.



(i) State transition diagram

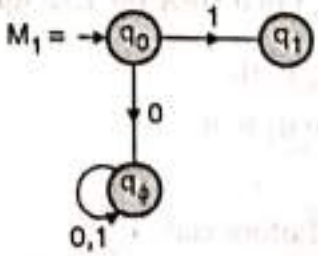
	0	1
$\rightarrow q_0^*$	q_1	q_0
q_1^*	q_2	q_1
q_2^*	q_ϕ	q_2
q_ϕ	q_ϕ	q_ϕ

(j) State transition table

Fig. Ex. 2.2.5 : Final DFA for Example 2.2.5(iv)

(v) All strings starting with 1 and length of the string is divisible by 3 :

Step 1 : A machine M_1, representing a DFA accepting a string starting with 1 is given in Fig. Ex. 2.2.5(k) :



A 0 as the first input takes the machine to a dead state

Fig. Ex. 2.2.5(k)

Scanned by CamScanner

Step 2 : A machine M_2, representing a DFA accepting a strings starting having length divisible by 3 is given in Fig. Ex. 2.2.5(*l*) :

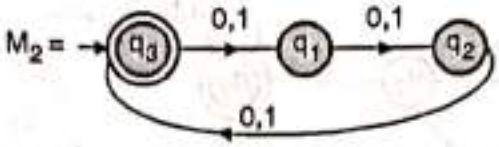
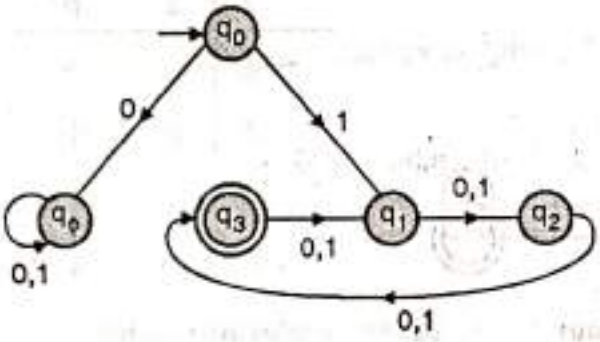


Fig. Ex. 2.2.5 (*l*)

To design the required machine, we can combine M_1 and M_2.

- Merge q_1 of M_1 with q_1 of M_2 .
- Initial input is handled by M_1, subsequent inputs are handled by M_2.



(m) State transition diagram

	a	b
→q_0	q_ϕ	q_1
q_1	q_2	q_2
q_2	q_3	q_3
q_3^*	q_1	q_1
q_ϕ	q_ϕ	q_ϕ

(n) State transition table

Fig. Ex. 2.2.5 : Final DFA for Example 2.2.5(v)

2.2.4.2 Examples on Substring

Example 2.2.6 : Draw DFA for the following language over {a, b} :

(a) All strings starting with abb.

(b) All strings with abb as a substring i.e., abb anywhere in the string.

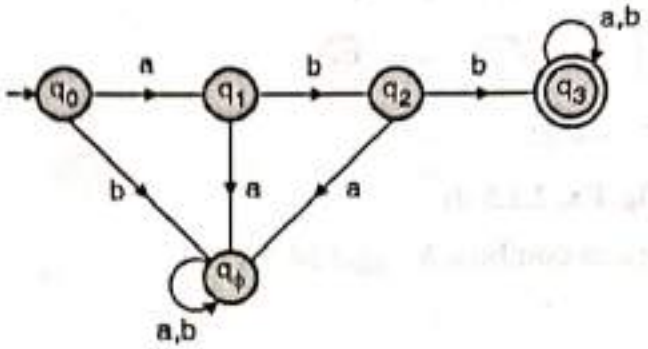
(c) All strings ending in abb. **MU – May 2017**

Solution :

(a) All strings starting with abb

- First input as 'b' will take the machine to a failure state.
- First two inputs as 'aa' will take the machine to a failure state.
- First three inputs as 'aba' will take the machine to a failure state.
- First three inputs as 'abb' will take the machine to a final state.

Scanned by CamScanner

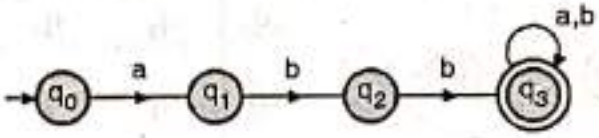


	a	b
$\rightarrow q_0$	q_1	q_ϕ
q_1	q_ϕ	q_2
q_2	q_ϕ	q_3
q_3^*	q_3	q_3
q_ϕ	q_ϕ	q_ϕ

(a) State transition diagram **(b) State transition table**

Fig. Ex. 2.2.6 : Final DFA for Example 2.2.6(a)

A DFA without explicit failure state is given in Fig. Ex. 2.2.6(a).



	a	b
$\rightarrow q_0$	q_1	ϕ
q_1	ϕ	q_2
q_2	ϕ	q_3
q_3^*	q_3	q_3

(c) State transition diagram **(d) State transition table**

Fig. Ex. 2.2.6 : Final DFA for Example 2.2.6(a), without a failure / dead state

(b) All strings with abb as a substring

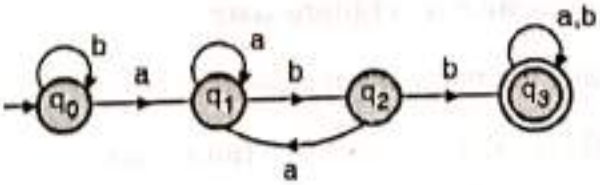
The machine will have fours states :

State q_0 – It is the starting state and indicates that nothing of relevance to complete 'abb' has been seen.

State q_1 – preceding character is 'a' and 'bb' is required to complete 'abb'.

State q_2, – Preceding characters are 'ab' and 'b' is required to complete 'abb.'

State q_3, – Preceding characters are 'abb' and the substring 'abb' has been seen by the machine.



	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_3
q_3^*	q_3	q_3

(e) State transition diagram **(f) State transition table**

Fig. Ex. 2.2.6 : Final DFA for Example 2.2.6(b)

Scanned by CamScanner

q_0 to q_0 on input 'b' :

First character in 'abb' is a.

q_0 to q_1 on input 'a' :

q_1 is for preceding characters as 'a', first character of abb.

q_1 to q_1 on input 'a' :

An input of 'a' in state q_1 will make the preceding two characters as 'aa'. Last 'a' will still constitute the first 'a' of abb.

q_1 to q_2 on input 'b' :

q_2 is for preceding two characters as 'ab' of 'abb'.

q_2 to q_1 on input 'a' :

An input 'a' in q_2 will make the preceding three characters as 'aba'. Out of the three characters 'aba', only the last character 'a' is relevant to 'abb'.

q_2 to q_3 on input b :

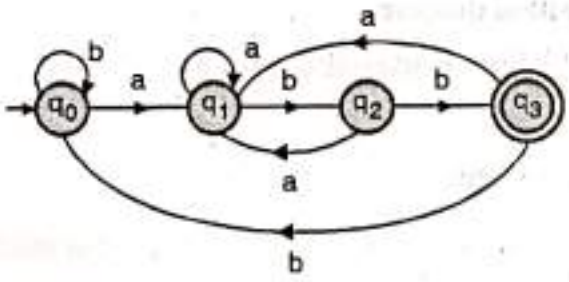
q_3 is for preceding three characters as 'abb'.

q_3 to q_3 on input a or b :

The substring 'abb' has been seen by the machine and a new input will not change this status.

(c) All strings ending in abb

This can be seen as an extention of solution which is given in Example 2.2.6(b) as the substring 'abb' should be at the end of the string. Transitions from q_3 should be modified to handle the condition that the string has to end in 'abb'.



(g) State transition diagram

	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_3
q_3^*	q_1	q_0

(h) State transition table

Fig. Ex. 2.2.6 : Final DFA for Example 2.2.6(c)

q_3 to q_1 on input a :

An input of a in q_3 will make the previous four characters as 'abba'. Out of the four characters as 'abba' only the last character 'a' is relevant to 'abb'.

q_3 to q_0 on input b :

An input of b in q_3 will make the previous four characters 'abbb'. Out of the four characters 'abbb', nothing is relevant to 'abb'.

Scanned by CamScanner

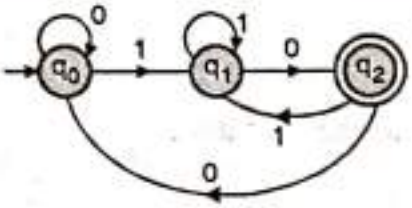
Example 2.2.7 : Design DFA for a language of string 0 and 1 that :

(I) Ending with 10 (II) Ending with 11

(III) Ending with 1

Solution :

I) Ending with 10



	0	1
$\rightarrow q_0$	q_0	q_1
q_1	q_2	q_1
q_2^*	q_0	q_1

(a) State transition diagram

Fig. Ex. 2.2.7 : DFA for string ending with 10

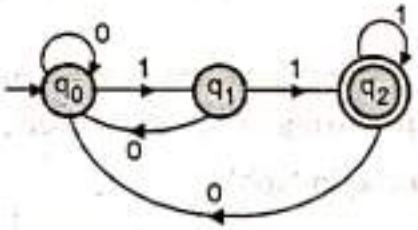
Meaning of various states :

q_0 : Starting state, nothing of sequence 10 is seen.

q_1 : '1' of sequence 10 is seen.

q_2 : Final state for strings ending with 10.

II) Ending with 11



	0	1
$\rightarrow q_0$	q_0	q_1
q_1	q_0	q_2
q_2^*	q_0	q_2

(b) State transition diagram

Fig. Ex. 2.2.7 : DFA for string ending with 11

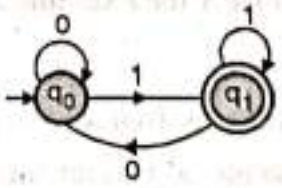
Meaning of various states :

q_0 : Starting state, nothing of sequence 11 is seen.

q_1 : First '1'of sequence '11' is seen

q_2 : Final state of strings ending with '11'

III) Ending with 1



	0	1
$\rightarrow q_0$	q_0	q_1
q_1^*	q_0	q_1

(c) State transition diagram

Fig. Ex. 2.2.7 : DFA for string ending with 1

- The DFA will be in state q_1, whenever the preceding symbol is 1.

Scanned by CamScanner

Example 2.2.8 : Design the DFA which accepts set of strings such that every string containing 00 as a substring but not 000 as a substring. **MU - Dec. 2007**

Solution :

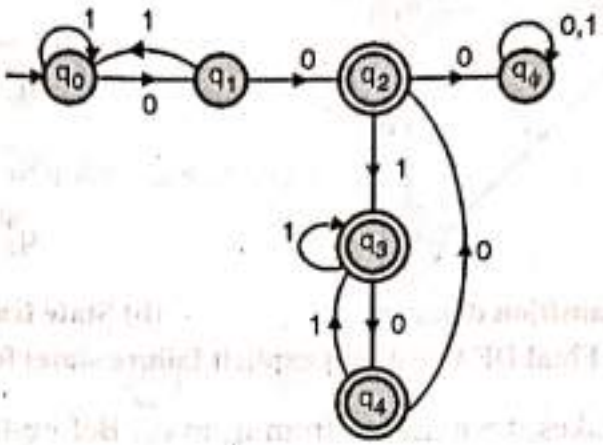


Fig. Ex. 2.2.8 : Final DFA for Example 2.2.8

- A sequence of 00 takes the machine from q_0 to q_2.
- q_2 is a final state.
- A '0' input in q_2 will cause a substring 000.
- An input of 1 in state q_2, causes a transition from q_2 to q_3. If starting from q_3, a substring '000' is detected, machine goes to the failure state q_4.

Example 2.2.9 : Design the DFA for the language, containing strings in which leftmost symbol differ from rightmost symbol. Σ is given by {0, 1}. **MU - May 2008, Dec. 2008**

Solution :

- Machine will end in the final state q_2 if the leftmost symbol is 0 and the rightmost symbol is 1.
- Machine will end in the final state q_4 if the leftmost symbol is 1 and the rightmost symbol is 0.
- A transition from q_0 to q_1 is for '0' as the first symbol.
- A transition from q_0 to q_3 for '1' as the first symbol.

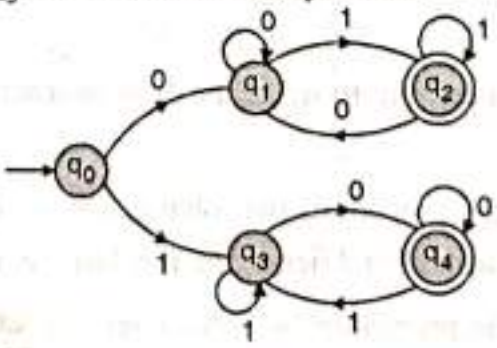
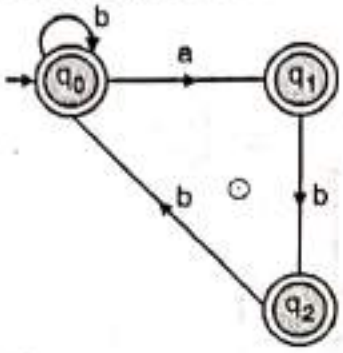


Fig. Ex. 2.2.9 : Final DFA for Example 2.2.9

Scanned by CamScanner

Example 2.2.10 : Design a DFA for set of strings over {a, b} in which there are at least two occurrences of b between any two occurrences of a.

Solution :



	a	b
$\rightarrow q_0^*$	q_1	q_0
q_1^*	ϕ	q_2
q_2^*	ϕ	q_0

(a) State transition diagram **(b) State transition table**

Fig. Ex. 2.2.10 : Final DFA (without explicit failure state) for Example 2.2.10

- An input 'a' in q_0 takes the machine from q_0 to q_1. Before the next 'a' can come, there should be at least two b's taking the machine from q_1 to q_2 and from q_2 to q_0.
- An input 'a' in either q_1 or q_2 causes a failure.
- All the three states are 'accepting states'.

Example 2.2.11 : Design a DFA for set of all strings over {a, b} ending in either ab or ba.

Solution :

Meaning of different states :

$q_0 \rightarrow$ starting state

$q_1 \rightarrow$ a of sequence ab

$q_2 \rightarrow$ ab of sequence ab

$q_3 \rightarrow$ b of sequence ba

$q_4 \rightarrow$ ba of sequence ba

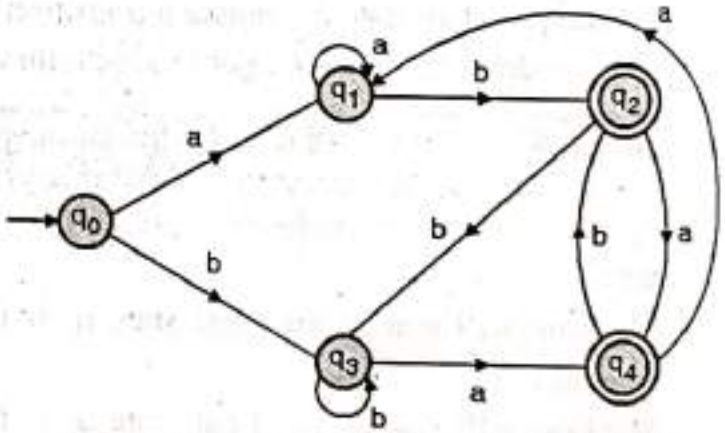


Fig. Ex. 2.2.11 : DFA for Example 2.2.11

Transitions :

- Input a in q_0 takes the machine to q_1 as the first character 'a' of 'ab' is the preceding character.
- Input b in q_0 takes the machine to q_3 as the first character 'b' of 'ba' is the preceding character.
- Input 'a' in q_1 makes the preceding two characters as 'aa'. Out of 'aa', only the last character 'a' is relevant to 'ab' and hence the machine requires in q_1.
- Input 'b' in q_1 makes the preceding two characters as 'ab'. Machine enters the state q_2 which stands for previous two characters as ab.

Scanned by CamScanner

- Input 'a' in q_2 makes the preceding two characters as 'ba'. Machine enters the state q_4 which stands for previous two characters as ba.
- Input b in q_2 makes the preceding two characters as 'bb'. Out of 'bb', only the last character 'b' is relevant to 'ba' and hence the machine enters the state q_3.
- Similar explanation can be given for q_3 and q_4.

Example 2.2.12 : Design an DFA for set of all strings over {a, b} containing both ab and ba as substrings.

Solution :

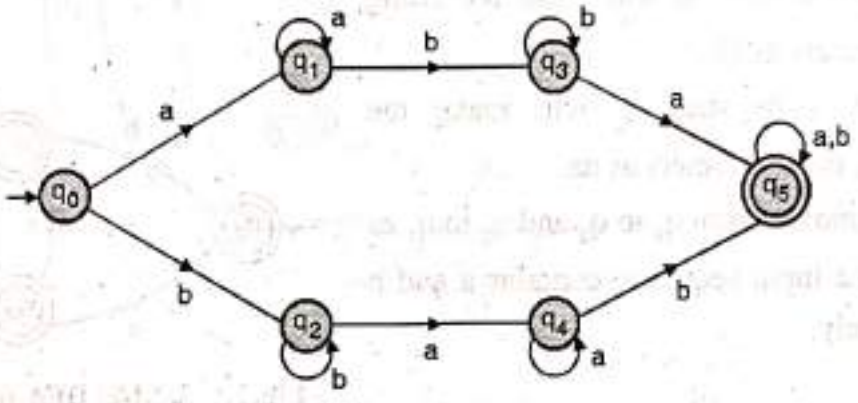


Fig. Ex. 2.2.12 : DFA for Example 2.2.12

- Machine takes the path $q_0 - q_1 - q_3 - q_5$, if the first substring is ab and the next substring is ba.
- Machine takes the path $q_0 - q_2 - q_4 - q_5$, if the first substring is ba and the next substring is ab.

Example 2.2.13 : Design a FA that reads string defined over Σ = {a,b} and accepts only those strings which end up in either 'aa' or 'bb'.

Solution :

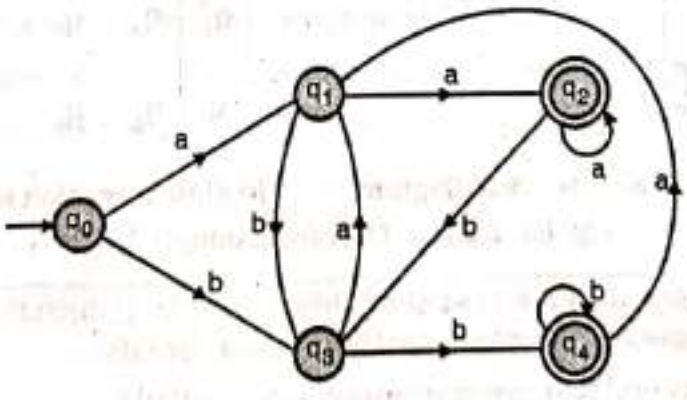


Fig. Ex. 2.2.13

Meaning of different states :

$q_0 \rightarrow$ starting state

Scanned by CamScanner

$q_1 \rightarrow$ a of sequence aa is seen.

$q_2 \rightarrow$ aa of sequence aa is seen.

$q_3 \rightarrow$ b of sequence bb is seen.

$q_4 \rightarrow$ bb of sequence bb is seen.

Example 2.2.14 : Design a DFA for set of all strings over {a, b} containing neither aa nor bb as a substring.

Solution :

- Machine enters the failure state q_ϕ on seeing either aa or bb.
- An input b is state q_1 will make preceding two characters as bb.
- An input a in state q_2 will make the preceding two characters as aa.
- Machine moves from q_1 to q_2 and q_2 to q_1 as long as the input sequence contains a and b alternatively.

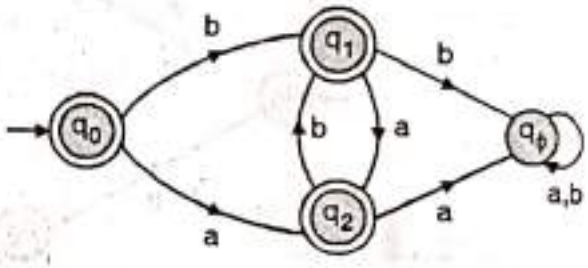
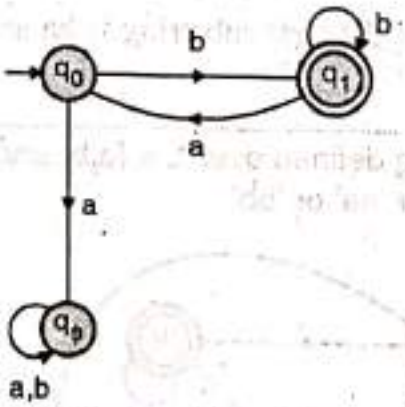


Fig. Ex. 2.2.14 : DFA for Example 2.2.14

Example 2.2.15 : Design a DFA for set of all strings over {a, b} such that each a in ω is immediately preceded and immediately followed by a 'b'.

Solution :



	a	b
$\rightarrow q_0$	q_ϕ	q_1
q_1^*	q_0	q_1
q_ϕ	q_ϕ	q_ϕ

(a) State transition diagram **(b) State transition table**

Fig. Ex. 2.2.15 : DFA for Example 2.2.15

Example 2.2.16 : Design a DFA for set of all strings over {0, 1} such that every block of five consecutive symbols contains at least two 0's. **MU - May 2009**

Solution : To solve this problem, we must maintain a count of :

1. Number of 1's before 0.
2. Number of 1's after 0.

Scanned by CamScanner

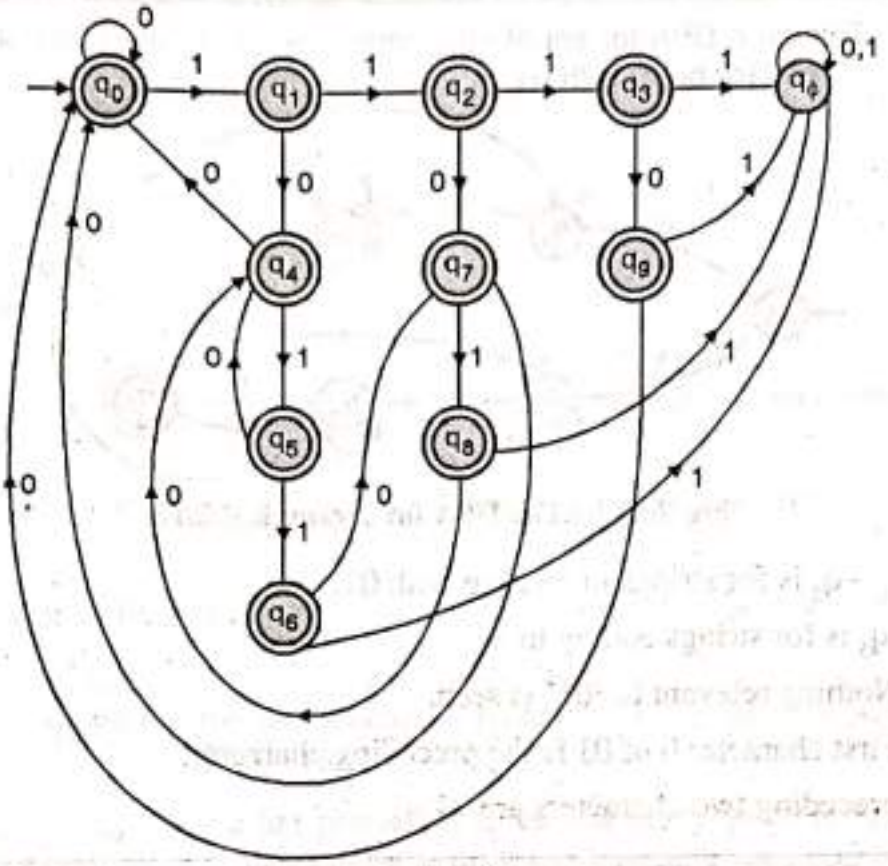


Fig. Ex. 2.2.16 : DFA for Example 2.2.16

If the sum total of number of 1's before 0 and number of 1's after 0 is 4, the machine enters a failure state.

Meaning of various states :

$q_0 \rightarrow$ number of 1's before 0 is zero.

$q_4 \rightarrow$ number of 1's before 0 is one.

$q_7 \rightarrow$ number of 1's before 0 is two.

$q_9 \rightarrow$ number of 1's before 0 is three.

$q_5 \rightarrow$ number of 1's before 0 is one and after zero is 1.

$q_6 \rightarrow$ number of 1's before 0 is one and after zero is 2.

$q_8 \rightarrow$ number of 1's before 0 is two and after zero is 1.

An input 0 in q_4, q_5, q_6, q_7, q_8, q_9 will make number of 1's after 0 as number of 1's before 0.

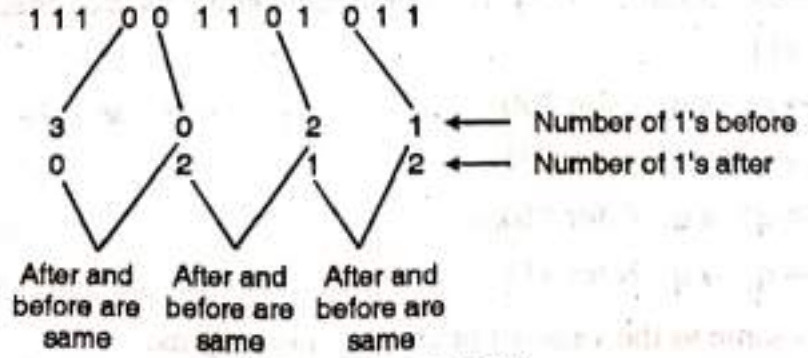


Fig. Ex. 2.2.16(a)

Scanned by CamScanner

Example 2.2.17 : Design a DFA for set of all strings over {0, 1} such that strings either begin or end (or both) with 01.

Solution :

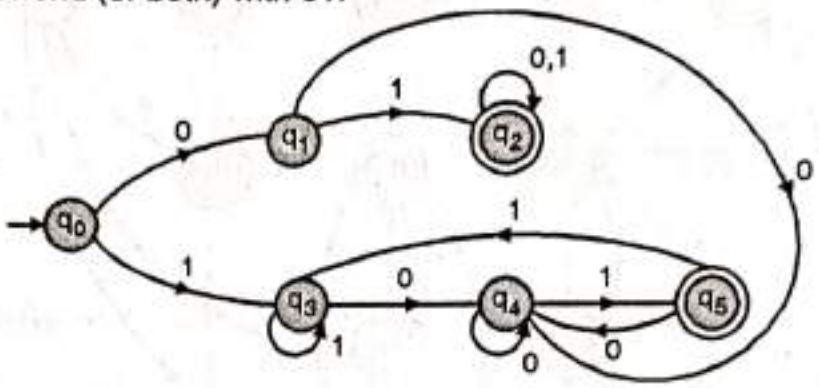


Fig. Ex. 2.2.17 : DFA for Example 2.2.17

- Path $q_0 - q_1 - q_2$ is for strings beginning with 01.
- Final state q_5 is for strings ending in 01.
- State q_3 – Nothing relevant to '01' is seen.
- State q_4 – First character 0 of 01 is the preceding character.
- State q_5 – preceding two characters are 01.

Example 2.2.18 : Design a DFA for set of all strings over {0, 1} such that the third symbol from the right end is 1. **MU - Dec. 2006**

Solution :

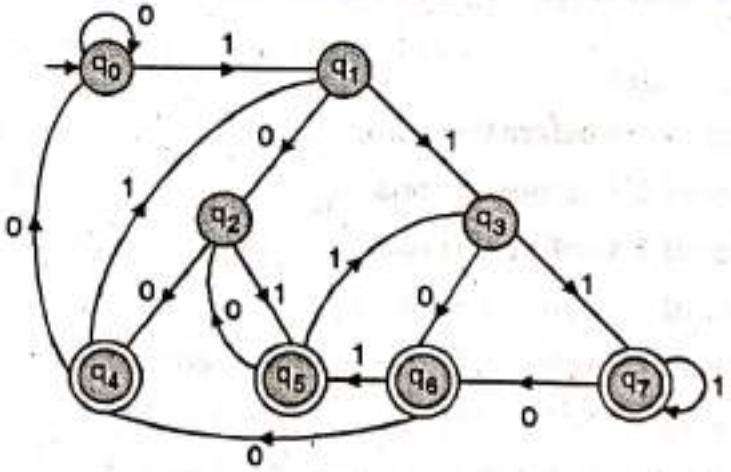


Fig. Ex. 2.2.18 : DFA for Example 2.2.18

Following 4 combinations for the preceding three characters are accepting.

100, 101, 110, 111.

- Path $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_4$ is for 100.
- Path $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_5$ is for 101.
- Path $q_0 \rightarrow q_1 \rightarrow q_3 \rightarrow q_6$ is for 110.
- Path $q_0 \rightarrow q_1 \rightarrow q_3 \rightarrow q_7$ is for 111.

State number is same as the value of preceding three inputs.

Scanned by CamScanner

Transitions :

- An input 0 in q_4 will make the preceding three characters as 000, machine moves to state q_0.
- An input 1 in q_4 will make the preceding three characters as 001, machine moves to state q_1.
- An input 0 in q_5 will make the preceding three characters as 010, machine moves to state q_2.
- An input 1 in q_5 will make the preceding three characters as 011, machine moves to state q_3.
- An input 0 in q_6 will make the preceding three characters as 100, machine moves to state q_4.
- An input 1 in q_6 will make the preceding three characters as 101, machine moves to state q_5.
- An input 0 in q_7 will make the preceding three characters as 110, machine moves to state q_6.
- An input 1 in q_7 will make the preceding three characters as 111, machine moves in state q_7.

Example 2.2.19 : Construct a DFA for set of strings containing either the substring 'aaa' or 'bbb'.

Solution :

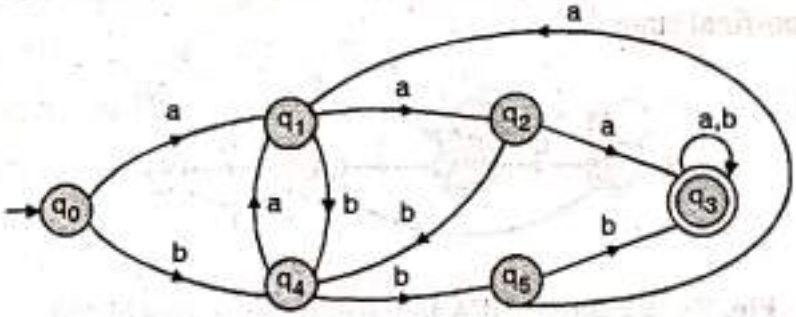


Fig. Ex. 2.2.19 : DFA for Example 2.2.19

Meaning of various states :

$q_0 \rightarrow$ Starting state.

$q_1 \rightarrow$ previous character is a of 'aaa'.

$q_2 \rightarrow$ previous two character are 'aa' of 'aaa'.

$q_4 \rightarrow$ previous character is b of 'bbb'.

$q_5 \rightarrow$ previous two character are 'bb' of 'bbb'.

$q_3 \rightarrow$ substring 'aaa'or 'bbb' is seen.

Scanned by CamScanner

- An input 'b' in q_0, q_1 or q_2 moves the machine to q_4 as b is the first character of the sequence bbb.
- An input 'a' in q_0, q_4 or q_5 moves the machine to q_1 as 'a' is the first character of the sequence aaa.

Example 2.2.20 : Construct a DFA for accepting a set of strings over alphabet {0,1} not ending in 010.

Solution :

Above DFA can be constructed in two steps :

1. DFA for strings ending in 010.
2. By taking complement of DFA derived in step 1; make every final state as non-final state and non-finals state as final state.

Step 1 : DFA for accepting strings ending in 010.

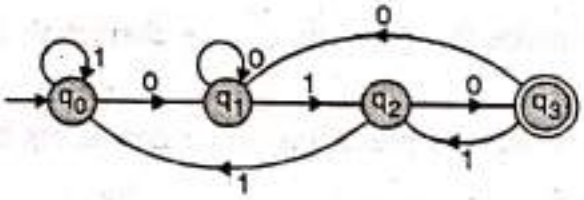


Fig. Ex. 2.2.20(a) : DFA for strings ending in 010

Step 2 : Complementing the DFA by reversing a non-final state to final state and a final state to non-final state.

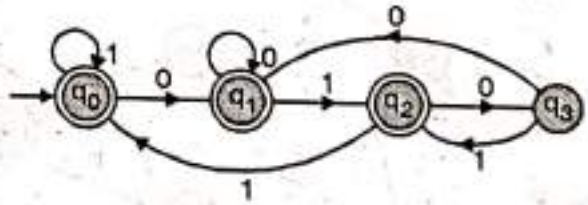


Fig. Ex. 2.2.20(b) : DFA for strings not ending in 010

Example 2.2.21 : Design a DFA that reads strings made up of letters in the word 'CHARIOT' and recognizes these strings that contain the word 'CAT' as a substring.

Solution :

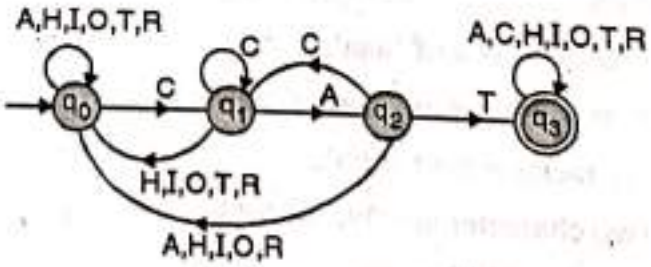


Fig. Ex. 2.2.21(a) :State transition diagram

Scanned by CamScanner

	C	H	A	R	I	O	T
$\rightarrow q_0$	q_1	q_0	q_0	q_0	q_0	q_0	q_0
q_1	q_1	q_0	q_2	q_0	q_0	q_0	q_0
q_2	q_1	q_0	q_0	q_0	q_0	q_0	q_3
q_3^*	q_3	q_3	q_3	q_3	q_3	q_3	q_3

(b) State transition table

Fig. Ex. 2.2.21 : DFA for Example 2.2.21

Meaning of various states :

q_0 : Starting state.

q_1 : First character C of 'CAT' is the preceding character.

q_2 : First two characters CA of 'CAT' are the preceding two characters.

q_3 : entire 'CAT' has been seen.

Example 2.2.22 : Design a FA that reads strings made of letter in the word 'UNIVERSITY' and recognize these strings that contains the word UNITY as substring.

Solution :

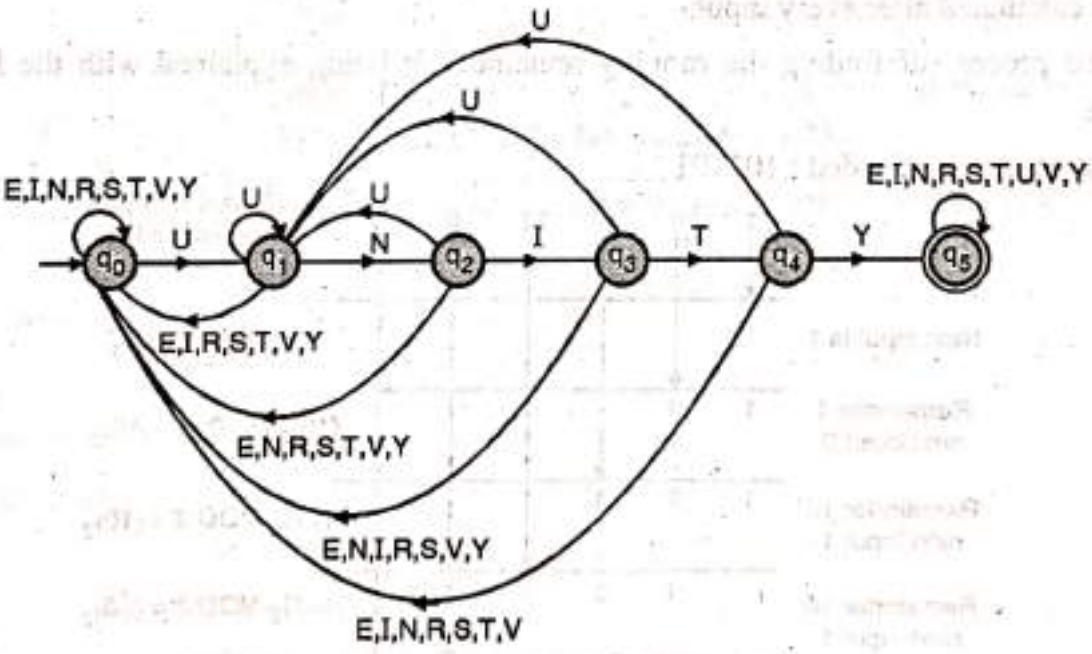


Fig. Ex. 2.2.22

Meaning of various states

q_0 : Starting state

q_1 : First character U of 'UNITY' is the preceding character.

q_2 : First two characters UN of 'UNITY' are the preceding two characters.

q_3 : First three characters UNI of 'UNITY' are the preceding three characters.

Scanned by CamScanner

q_4 : First four characters UNIT of 'UNITY' are the preceding four characters.

q_5 : Entire 'UNITY' has been seen.

2.2.4.3 Examples of Divisibility

Example 2.2.23 : Design a DFA which can accept a binary number divisible by 3
Or
Design of a divisibility – by – 3 – tester for a binary number.
MU - Dec. 2005, May 2014, May 2017

Solution :

A binary number is divisible by 3, if the remainder when divided by 3 will work out to be zero. We must device a mechanism for finding the final remainder.

- We can calculate the running remainder based on previous remainder and the next input.
- The running remainder could be :

  $0 \rightarrow$ associated state, q_0

  $1 \rightarrow$ associated state, q_1

  $2 \rightarrow$ associated state, q_2
- Starting with the most significant bit, input is taken one bit at a time. Running remainder is calculated after every input.

The process of finding the running remainder is being explained with the help of an example.

Number to be divided : 101101

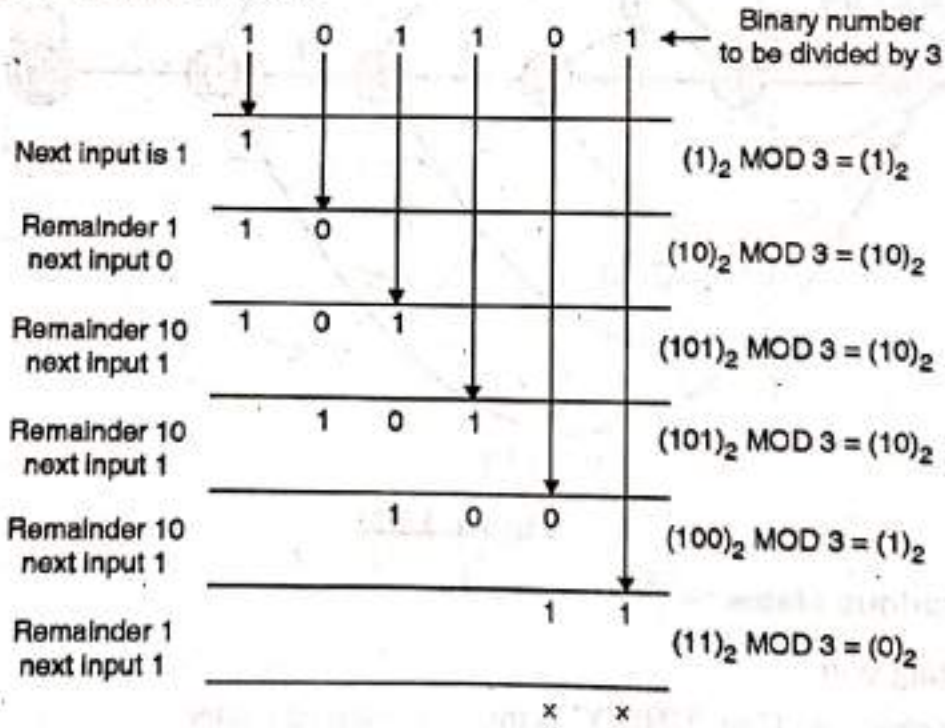
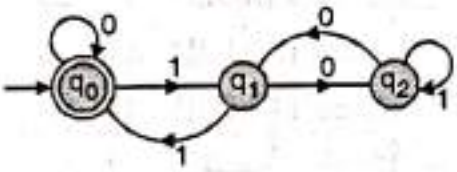


Fig. Ex. 2.2.23(a)

Scanned by CamScanner

The calculation of next remainder is shown below,

Previous remainder	Next input	Calculation of remainder		Next remainder
0 (q_0)	0	00 % 3	⇒	0 (q_0)
0 (q_0)	1	01 % 3	⇒	1 (q_1)
1 (q_1)	0	10 % 3	⇒	10 (q_2)
1 (q_1)	1	11 % 3	⇒	0 (q_0)
10 (q_2)	0	100 % 3	⇒	1 (q_1)
10 (q_2)	1	101 % 3	⇒	10 (q_2)
↑		↑ ↖		↑
Binary		Binary decimal		Binary



(b) State transition diagram

	0	1
$\rightarrow q_0^*$	q_0	q_1
q_1	q_2	q_0
q_2	q_1	q_2

(c) State transition table

Fig. Ex. 2.2.23 : DFA for Example 2.2.23

Example 2.2.24 : Design a DFA which can accept a ternary number divisible by 4.

Solution :

A ternary system has three alphabets.

$$\Sigma = \{0, 1, 2\}$$

Base of a ternary number is 3.

The running remainder could be :

$(0)_3 = 0 \rightarrow$ associated state, q_0

$(1)_3 = 1 \rightarrow$ associated state, q_1

$(2)_3 = 2 \rightarrow$ associated state, q_2

$(10)_3 = 3 \rightarrow$ associated state, q_3

↑ ↑

Ternary Decimal

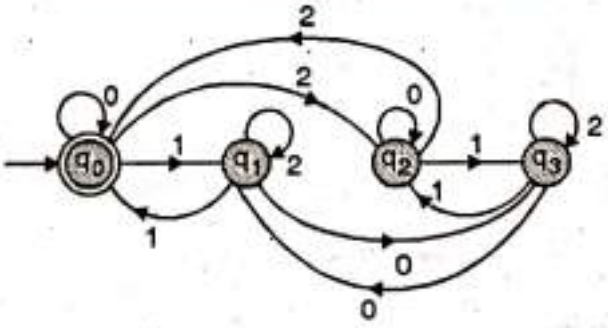
Transition behaviour will depend on the current remainder (current state) and the next input. These two will be used to get the next remainder (next state).

Scanned by CamScanner

Transition behaviour is explained below :

Current remainder	Next input	Calculation of remainder	Remainder
(q_0) 0	0	00 ⇒ 0 MOD 4 ⇒	0 (q_0)
	1	01 ⇒ 1 MOD 4 ⇒	1 (q_1)
	2	02 ⇒ 2 MOD 4 ⇒	2 (q_2)
(q_1) 1	0	10 ⇒ 3 MOD 4 ⇒	3 (q_3)
	1	11 ⇒ 4 MOD 4 ⇒	0 (q_0)
	2	12 ⇒ 5 MOD 4 ⇒	1 (q_1)
(q_2) 2	0	20 ⇒ 6 MOD 4 ⇒	2 (q_2)
	1	21 ⇒ 7 MOD 4 ⇒	3 (q_3)
	2	22 ⇒ 8 MOD 4 ⇒	0 (q_0)
(q_2) 10	0	100 ⇒ 9 MOD 4 ⇒	1 (q_1)
	1	101 ⇒ 10 MOD 4 ⇒	2 (q_2)
	2	102 ⇒ 11 MOD 4 ⇒	3 (q_3)

Ternary — Ternary — Decimal — Decimal



	0	1	2
$\rightarrow q_0^*$	q_0	q_1	q_2
q_1	q_3	q_0	q_1
q_2	q_2	q_3	q_0
q_3	q_1	q_2	q_3

(a) State transition diagram **(b) State transition table**

Fig. Ex. 2.2.24 : DFA for Example 2.2.24

Example 2.2.25 : Design a DFA for a mod 5 tester for ternary input. **MU – Dec. 2017**

Solution :

A ternary system has three alphabets

$$\Sigma = \{0, 1, 2\}$$

Base of a ternary number is 3.

Scanned by CamScanner

The running remainder could be :

$(0)_3 = 0 \rightarrow$ associated state, q_0

$(1)_3 = 1 \rightarrow$ associated state, q_1

$(2)_3 = 2 \rightarrow$ associated state, q_2

$(10)_3 = 3 \rightarrow$ associated state, q_3

$(11)_3 = 4 \rightarrow$ associated state, q_4

↑ ↑

Ternary Decimal

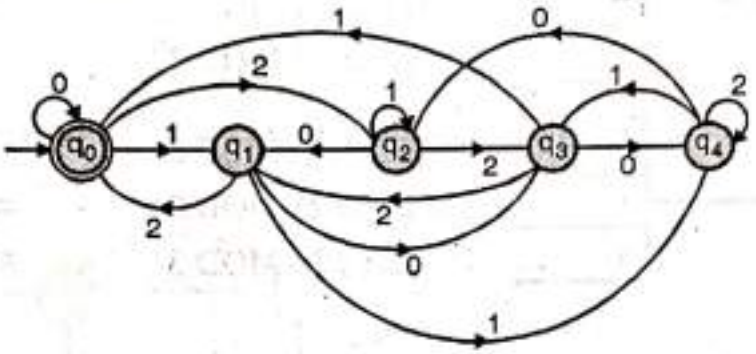


Fig. Ex. 2.2.25

Example 2.2.26 : Design a DFA which can accept a decimal number divisible by 3.

Solution :

A decimal system has 10 alphabets.

$\Sigma = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)$

The running remainder could be

$0 \rightarrow$ associated state, q_0

$1 \rightarrow$ associated state, q_1

$2 \rightarrow$ associated state, q_2

Transition behaviour will depend on the current remainder (current rate) and the next input. These two will be used to get the next remainder (next state).

- Next digit as 0, 3, 6, 9 will be mapped to the same next state.
- Next digit as 1, 4, 7 will be mapped to the same next state
- Next digit as 2, 5, 8 will be mapped to the same next state.

Thus it is sufficient to consider transition for just three inputs :

0 for (0, 3, 6, 9)

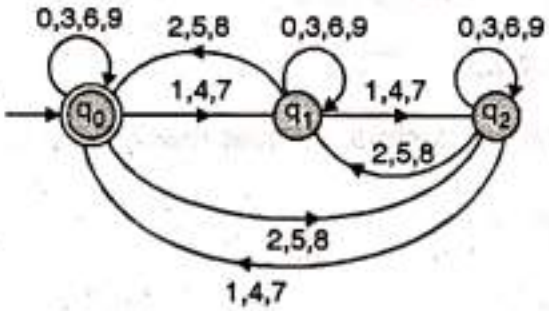
1 for (1, 4, 7)

2 for (2, 5, 8)

Scanned by CamScanner

Transition behaviour is explained below :

Current remainder	Next input	Calculation of remainder		Remainder
0	0	$00 \Rightarrow 0$ MOD 3	$\Rightarrow$	0 (q_0)
	1	$01 \Rightarrow 1$ MOD 3	$\Rightarrow$	1 (q_1)
	2	$02 \Rightarrow 2$ MOD 3	$\Rightarrow$	2 (q_2)
1	0	$10 \Rightarrow 10$ MOD 3	$\Rightarrow$	1 (q_1)
	1	$11 \Rightarrow 11$ MOD 3	$\Rightarrow$	2 (q_2)
	2	$12 \Rightarrow 12$ MOD 3	$\Rightarrow$	0 (q_0)
2	0	$20 \Rightarrow 20$ MOD 3	$\Rightarrow$	2 (q_2)
	1	$21 \Rightarrow 21$ MOD 3	$\Rightarrow$	0 (q_0)
	2	$22 \Rightarrow 22$ MOD 3	$\Rightarrow$	1 (q_1)



(a) State transition diagram

	(0, 3, 6, 9)	(1, 4, 7)	(2, 5, 8)
$\rightarrow q_0^*$	q_0	q_1	q_2
q_1	q_1	q_2	q_0
q_2	q_2	q_0	q_1

(b) State transition table

Fig. Ex. 2.2.26 : DFA for Example 2.2.26

Example 2.2.27 : Design an FSM for divisibility by 3 tester for a unary number.

MU - May 2007

Solution :

Let us assume that $\Sigma = \{0\}$

For unary number to be divisible by 3, the number of 0's should be multiple of 3. The FSM is given in Fig. Ex. 2.2.27

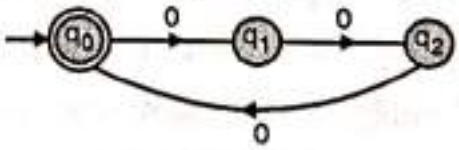


Fig. Ex. 2.2.27

Example 2.2.28 Design a Finite State Machine for divisibility by 5 tester of a given decimal number.

Solution :

A decimal number will be divisible by 5 if the rightmost digit is either '0' or '5'.

Scanned by CamScanner

The required DFA is given in Fig. Ex. 2.2.28.

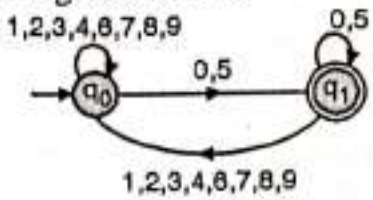


Fig. Ex. 2.2.28

Example 2.2.29 : Design DFA that accepts the following language :

(i) Set of all strings with odd number of 1's followed by even number of 0's $\Sigma = \{0, 1\}$.

(ii) Set of all strings which begin and end with different letters $\Sigma = \{x, y, z\}$.

(iii) Strings ending with 110 or 111. **MU - Dec. 2006, Dec. 2009, Dec. 2010**

Solution :

(i)

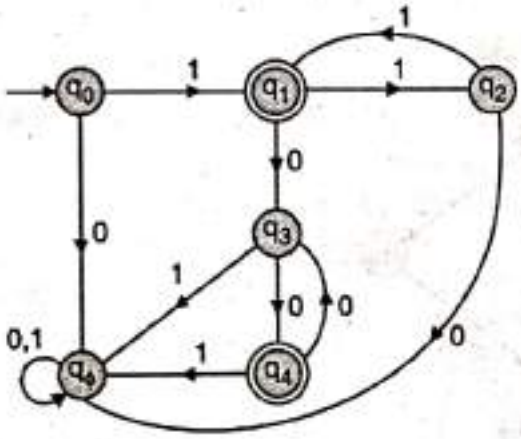


Fig. Ex. 2.2.29(a)

(ii)

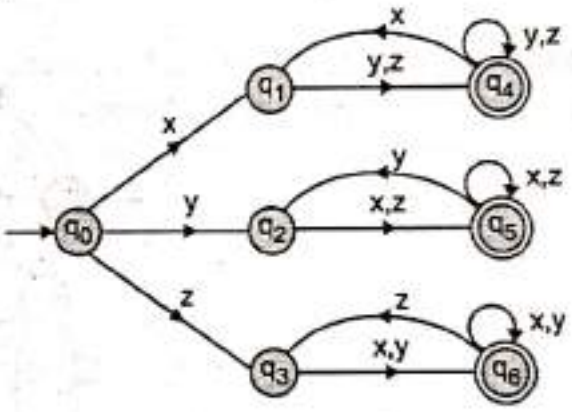


Fig. Ex. 2.2.29(b)

(iii)

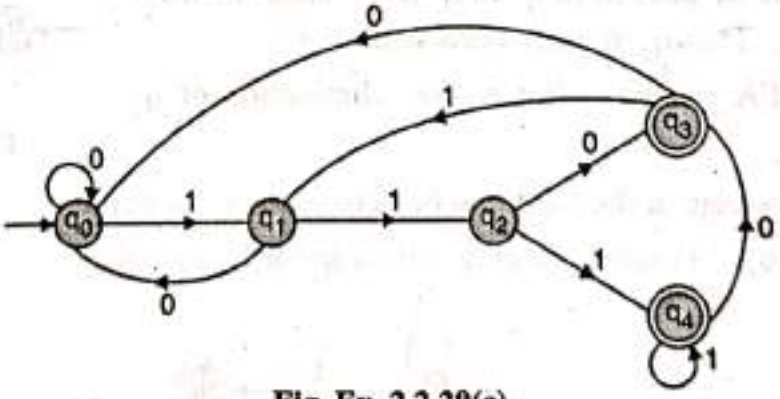


Fig. Ex. 2.2.29(c)

2.2.5 Language of DFA

The language of a DFA $M = \{Q, \Sigma, \delta, q_0, F\}$ is denoted by L (M) and is defined by :

$L(M) = \{\omega \mid \delta^*(q_0, \omega) \text{ is in } F\}$

Scanned by CamScanner

That is, the language of DFA M is the set of strings accepted by M. The language of a DFA is also known as regular language. $\delta^*(q_0, \omega)$ stands for a series of transitions starting from q_0.

Example 2.2.30 : Describe the language accepted by the deterministic finite automata shown in Fig. Ex. 2.2.30.

(i) (ii)

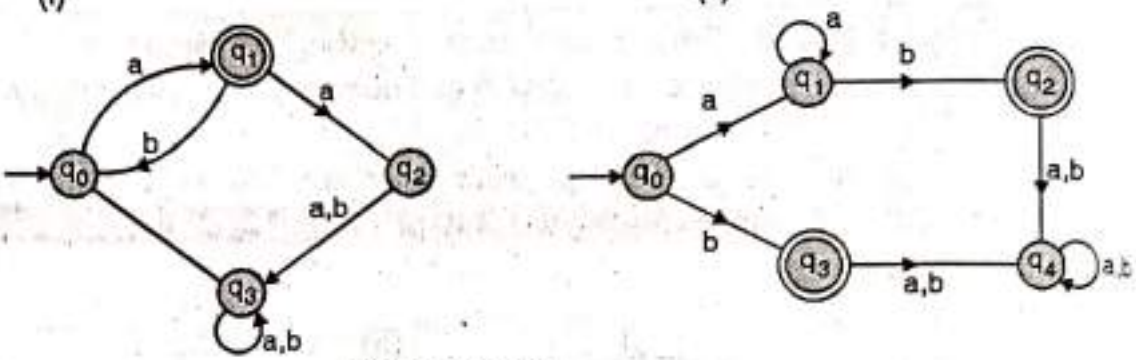


Fig. Ex. 2.2.30

Solution :

(i)

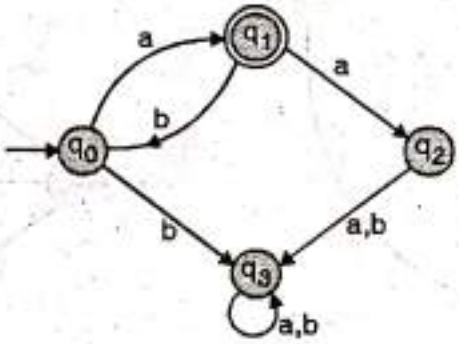


Fig. Ex. 2.2.30(a)

- There is no path from q_2 to q_1 (final state) or from q_3 to q_1. Thus q_2 to q_3 are dead states.
- The DFA can be redrawn after elimination of q_2 and q_3.
- The language of the DFA can be defined as given below :

$L = \{\omega \in \{a, b\}^* \mid \omega$ starts and ends with a and a, b alternate$\}$

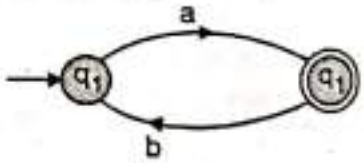


Fig. Ex. 2.2.30(b)

(ii)

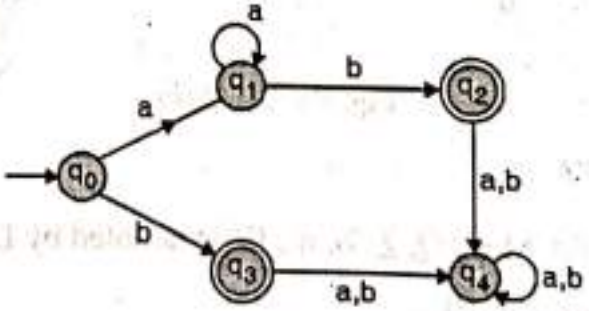


Fig. Ex. 2.2.30(c)

Scanned by CamScanner

- q_4 is dead state. The DFA can be redrawn after elimination of q_4.

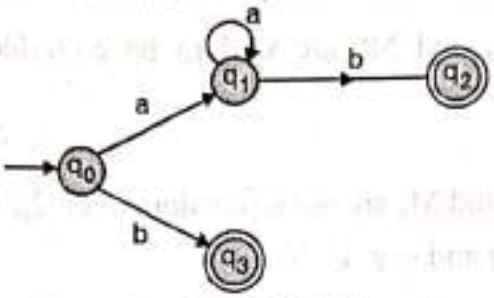


Fig. Ex. 2.2.30(d)

- A string 'b' of length 1 is accepted through the path $q_0 \rightarrow q_3$.
- A string having 1 or more a's followed by a 'b' is accepted through the path.
- $q_0 \rightarrow q_1 \rightarrow q_1 \cdots q_1 \rightarrow q_2$

The language of the DFA can be described as given below :

$L = \{\omega \in \{a, b\}^* \mid \omega$ consists of 0 or more a's followed by a 'b'$\}$.

Example 2.2.31 : Consider the following transition diagram in Fig. Ex. 2.2.31.
Test whether the string 110101 is accepted by the finite automata represented by above transition diagram. Show the entire sequence of states traversed.

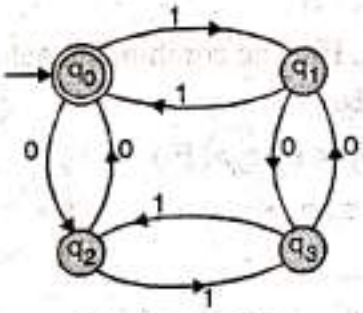


Fig. Ex. 2.2.31

Solution :

$$(q_0, 110101) \overset{\delta}{\Rightarrow} (q_1, 10101)$$
$$\overset{\delta}{\Rightarrow} (q_0, 0101)$$
$$\overset{\delta}{\Rightarrow} (q_2, 101)$$
$$\overset{\delta}{\Rightarrow} (q_3, 01)$$
$$\overset{\delta}{\Rightarrow} (q_1, 1)$$
$$\overset{\delta}{\Rightarrow} (q_0, \varepsilon) \Rightarrow q_0$$

String 110101 will be accepted as q_0 is a final state.

Scanned by CamScanner

2.2.6 Equivalence of DFAs

Two finite automata M_1 and M_2 are said to be equivalent if they accept the same language

i.e., $L(M_1) = L(M_2)$

Two finite automata M_1 and M_2 are not equivalent over Σ, if there exists a ω such that :

$$\omega \in L(M_1) \text{ and } \omega \notin L(M_2)$$

or

$$\omega \notin L(M_1) \text{ and } \omega \in L(M_2)$$

That is, one DFA reaches a final state on application of ω and other reaches a non final state.

Thus, if two DFAs are equivalent, then for every $\omega \in \Sigma^*$:

On application of ω, either both M_1 and M_2 will be in **a final state** or both M_1 and M_2 will be in a **non final state**, simultaneously.

Testing equivalence of two DFAs

Equivalence of two DFA's can be established by constructing a combined DFA for two DFAs M_1 and M_2. Let the combined DFA for M_1 and M_2 be M_3.

where, $M_1 = (S, \Sigma, \delta_1, s_0, F_1)$

and $M_2 = (Q, \Sigma, \delta_2, q_0, F_2)$ The combined machine M_3 can be described in terms of M_1 and M_2.

$$M_3 = (S \times Q, \Sigma, \delta_3, < s_0, q_0 >, F_3)$$

A state of M_3 is of the form $< s_i, q_i >$

Where,

$$s_i \in S \text{ of } M_1 \text{ and } \quad q_i \in Q \text{ of } M_2.$$

The transition function δ_3 is defined as :

$$\delta_3 (< s_i, q_i >, a) = < \delta_1 (s_i, a), \ \delta_2 (q_i, a) >$$

δ_3 is a function from $S \times Q$ to $S \times Q$.

The **algorithm** for generation of transition behaviour δ_3 of M_3 is given below :

1. Add the state $< s_i, q_0 >$ to M_3.
2. for every state $< s_i, q_i >$ in M_3.

```
{
    if(< s_i, q_i > has not been expanded)
    {
      for each alphabet a_i ∈ Σ
      {
          add a transition < δ1 (s_i, a_i), δ2 (q_i, a_i) to M3
```

Scanned by CamScanner

}

}

}

M_1 and M_2 are **equivalent** if :

for every state $< s_i, q_i >$ in M_3

{ both s_i and q_i are final states

or

both s_i and q_i are non-final states

}

M_1 and M_2 are **not equivalent** if :

for any state $< s_i, q_i >$ in M_3

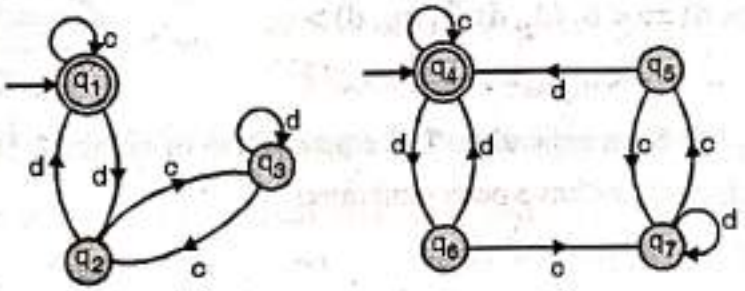
{ s_i is a final states and q_i is a non-final state

or

s_i is a non-final state and q_i is a final state

}

Example 2.2.32 : Show whether the automata M_1 and M_2 in given Fig. Ex. 2.2.32 (a) and (b) are equivalent or not.



(a) M_1 (b) M_2

Fig. Ex. 2.2.32

Solution :

Construction of combined DFA M_3

Step 1 : We start with the combined state $< q_1, q_4 >$,where q_1 is start state of M_1 and q_4 is the start state of M_2. $< q_1, q_4 >$ is expanded and necessary transitions added.

$$\delta_3 (< q_1, q_4 >, c) \Rightarrow < \delta_1 (q_1, c), \delta_2 (q_4, c) >$$

$$\Rightarrow < q_1, q_4 >$$

$$\delta_3 (< q_1, q_4 >, d) \Rightarrow < \delta_1 (q_1, d), \delta_2 (q_4, d) >$$

$$\Rightarrow < q_2, q_6 >$$

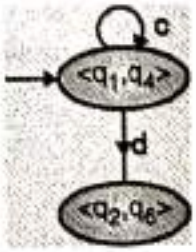


Fig. Ex. 2.2.32(c)

Scanned by CamScanner

Step 2 : State $< q_2, q_6 >$ is selected for expansion

$$\delta_3 (< q_2, q_6 >, c) \Rightarrow < \delta_1 (q_2, c), \delta_2 (q_6, c) >$$
$$\Rightarrow < q_3, q_7 >$$
$$\delta_3 (< q_2, q_6 >, d) \Rightarrow < \delta (q_2, d), \delta_2 (q_6, d) >$$
$$\Rightarrow < q_1, q_4 >$$

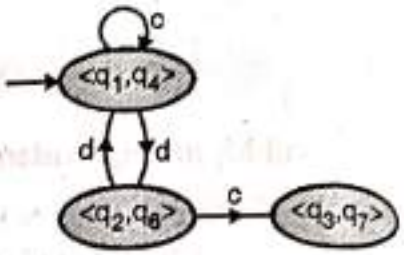


Fig. Ex. 2.2.32(d)

Step 3 : State $< q_3, q_7 >$ is selected for expansion.

$$\delta_3 (< q_3, q_7 >, c) \Rightarrow < \delta (q_3, c), \delta_2 (q_7, c) >$$
$$\Rightarrow < q_2, q_5 >$$
$$\delta_3 (< q_3, q_7 >, d) \Rightarrow < \delta (q_3, d), \delta_2 (q_7, d) >$$
$$\Rightarrow < q_3, q_7 >$$

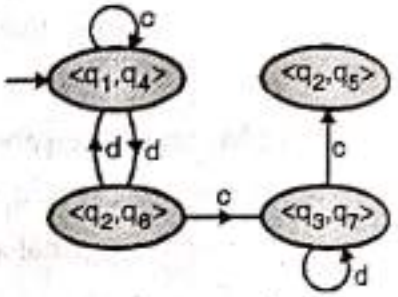


Fig. Ex. 2.2.32(e) M_3

Step 4 : State $< q_2, q_5 >$ is selected for expansion.

$$\delta_3 (< q_2, q_5 >, c) \Rightarrow < \delta_1 (q_2, c), \delta_2 (q_5, c) >$$
$$\Rightarrow < q_3, q_7 >$$
$$\delta_3 (< q_2, q_5 >, d) \Rightarrow < \delta_1 (q_2, d), \delta_2 (q_5, d) >$$
$$\Rightarrow < q_1, q_4 >$$

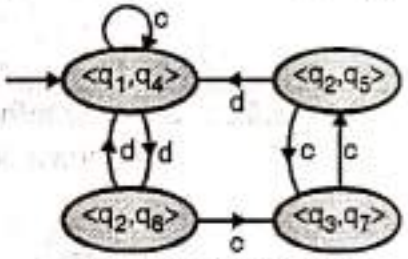


Fig. Ex. 2.2.32(f) M_3

Every state in M_3 has been expanded. The construction of combine DFA is over. During the construction of M_3, four states have been generated.

$< q_1, q_4 >$: both q_1 and q_4 are final states.

$< q_2, q_5 >$: q_2 and q_5 are non final states.

$< q_2, q_6 >$: both are non final states.

$< q_3, q_7 >$: q_3 and q_7 are non final states.

Thus, we can infer that M_1 and M_2 are equivalent.

Example 2.2.33 : Find out whether M_1 and M_2 are equivalent.

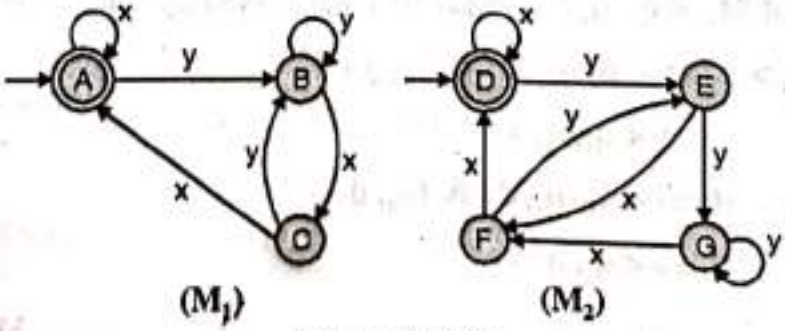


Fig. Ex. 2.2.33

Scanned by CamScanner

Solution :

Two DFAs are equivalent if for any string $\omega \in \Sigma^*$, either the machines will be in a final state or both the machines will be in a non-final state.

This can be established by constructing a combined DFA for M_1 and M_2.

Construction of combined DFA

In the Fig. Ex. 2.2.33(a), following subsets are generated.

1. {A, D} – Both A and D are accepting states.
2. {D, E} – Both D and E are non-accepting states.
3. {C, F} – Both C and F are non-accepting states.
4. {B, G} – Both B and G are non-accepting states.

Hence, machines M_1 and M_2 are equivalent.

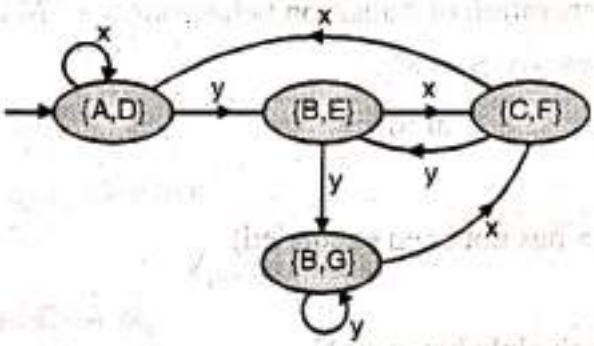


Fig. Ex. 2.2.33(a)

2.2.7 Closure Property of Language Accepted by a DFA

The class of languages accepted by a deterministic finite automata is closed under :

(a) Union (b) Intersection

(c) Difference (d) Complementation.

The closure property can be established by constructing an appropriate finite state automata in each case.

2.2.7.1 Union, Intersection, Difference

Let $M_1 = (S, \Sigma, \delta_1, s_0, F_1)$

and $M_2 = (Q, \Sigma, \delta_2, q_0, F_2)$

be two arbitrary DFAs. To prove the various closure properties, we must show that there is another machine :

M_3 such that $L(M_3) = L(M_1) \cup L(M_2)$

M_4 such that $L(M_4) = L(M_1) \cap L(M_2)$

M_5 such that $L(M_5) = L(M_1) - L(M_2)$

Scanned by CamScanner

We will first construct a combined machine M_c, simulating the transition behaviour of both. M_1 and M_2. Then, we will derive M_3, M_4 or M_5 from M_c.

The combine machine M_c can be described in terms of M_1 and M_2.

$$M_c = (S \times Q, \Sigma, \delta_c, <s_0, q_0>, F_c)$$

A state of M_c is of the form $< s_i, q_i >$ where,

$s_i \in S$ of M_1.

and $q_i \in Q$ of M_2.

The transition function δ_c is defined as :

$$\delta_c (< s_i, q_i >, a) = < \delta_1 (s_i, a), \delta_2 (q_i, a) >$$

δ_c is a function from $S \times Q$ to $S \times Q$.

The **algorithm** for generation of transition behaviour δ_c of M_c is given below :

1. Add the state $< s_0, q_0 >$ to M_c.
2. for every state $< s_i, q_i >$ in M_c.

```
{
    if(< s_i, q_i > has not been expanded)
    {
        for each alphabet a_i ∈ Σ
        {
            add a transition < δ1 (s_i, a_i), δ2(q_i, a_i) > to M_c.
        }
    }
}
```

Deriving M_3 from M_c ($L(M_3) = L(M_1) \cup L(M_2)$)

M_3 can be derived from M_c by selecting a set of states from M_c as a set of final states for M_3.

Set of final states F_3 for $M_3 = \{< s_i, q_i > \in$ states of $M_c \mid s_i \in F_1 \vee q_i \in F_2\}$

where F_1 is set of final states for M_1 and F_2 is set of final states for M_2.

Deriving M_4 form M_c ($L(M_4) = L(M_1) \cap L(M_2)$)

M_4 can be derived from M_c by selecting a set of states M_c as a set of final states for M_4.

Set of final states F_4 for $M_4 = \{< s_i, q_i > \in$ states of $M_c \mid s_i \in F_1 \wedge q_i \in F_2\}$

where, F_1 is set of final states for M_1 and F_2 , is set of final states for M_2.

Scanned by CamScanner

Deriving M_5 form M_c ($L(M_5) = L(M_1) - L(M_2)$)

M_5 can be derived from M_c by selecting a set of states from M_c as a set of final states for M_5.

Set of final states F_5 from $M_5 = \{< s_i, q_j > \in$ states of $M_c \mid s_i \in F_1 \cap q_j \notin F_2\}$

where, F_1 is a set of final states for M_1 and F_2 is a set of final states for M_2.

Example 2.2.34 : Suppose, we have two machines M_1 and M_2.

M_1 : For accepting a set of strings over alphabet {0, 1} ending in 01.

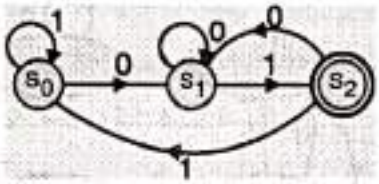
M_2 : For accepting a set of strings over alphabet {0, 1} containing even number of 1's.

Design a DFA for the following :

(i) $L(M_1) \cup L(M_2)$ (ii) $L(M_2) \cap L(M_2)$ (iii) $L(M_1) - L(M_2)$

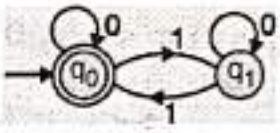
Solution :

$M_1 =$



(a) DFA for strings ending in 01

$M_2 =$



(b) DFA for strings having even number of 1's.

Fig. Ex. 2.2.34

Construction of combined DFA M_c

Step 1 : We start with the state $< s_0, q_0 >$, where s_0 is the initial state of M_1 and q_0 is the initial state of M_2.

$< s_0, q_0 >$ is expanded and necessary transitions added.

$\delta(< s_0, q_0 >, 0) \Rightarrow < \delta(s_0, 0), \delta(q_0, 0) >$

$\Rightarrow < s_1, q_0 >$

$\delta(< s_0, q_0 >, 1) \Rightarrow < \delta(s_0, 1), \delta(q_0, 1) >$

$\Rightarrow < s_0, q_1 >$

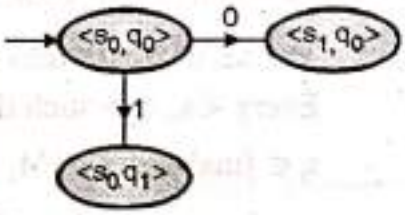


Fig. Ex. 2.2.34(c)

Step 2 : State $< s_1, q_0 >$ and $< s_0, q_1 >$ are selected for expansion.

$\delta(< s_0, q_1 >, 0) \Rightarrow < \delta(s_0, 0), \delta(q_1, 0) >$

$\Rightarrow < s_1, q_1 >$

$\delta(< s_0, q_1 >, 1) \Rightarrow < \delta(s_0, 1), \delta(q_1, 1) >$

$\Rightarrow < s_0, q_0 >$

$\delta(< s_1, q_0 >, 0) \Rightarrow < \delta(s_1, 0), \delta(q_0, 0) >$

$\Rightarrow < s_1, q_0 >$

$\delta(< s_1, q_0 >, 1) \Rightarrow < \delta(s_1, 1), \delta(q_0, d) >$

$\Rightarrow < s_2, q_1 >$

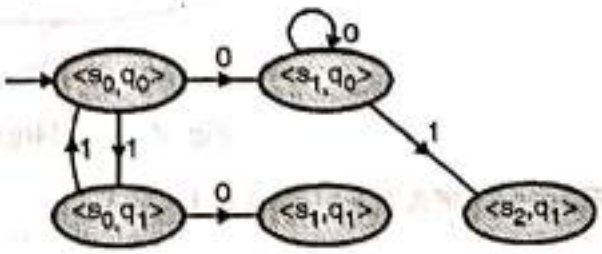


Fig. Ex. 2.2.34(d)

Scanned by CamScanner

Step 3 : State $< s_1, q_1 >$ and $< s_2, q_1 >$ are selected for expansion.

$$\delta (< s_1, q_1 >, 0) \Rightarrow < \delta (s_1\ 0), \delta (q_1\ 0) >$$
$$\Rightarrow < s_1, q_1 >$$
$$\delta (< s_1, q_1 >, 1) \Rightarrow < \delta (s_1, 1), \delta (q_1, 1) >$$
$$\Rightarrow < s_2, q_0 >$$
$$\delta (< s_2, q_1 >, 0) \Rightarrow < \delta (s_2, 0), \delta (q_1, 1) >$$
$$\Rightarrow < s_1, q_1 >$$
$$\delta (< s_2, q_1 >, 1) \Rightarrow < \delta (s_2, 1), \delta (q_1, 1) >$$
$$\Rightarrow < s_0, q_0 >$$

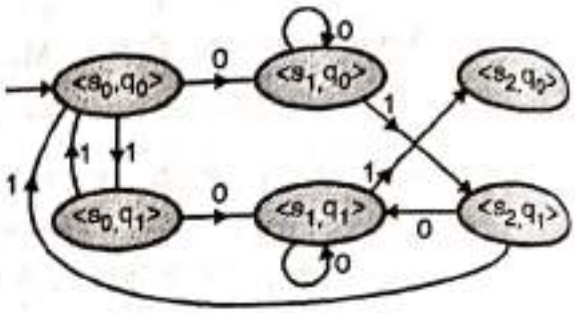


Fig. Ex. 2.2.34(e)

Step 4 : State $< s_2, q_0 >$ is selected for expansion.

$$\delta (< s_2, q_0 >, 0) \Rightarrow < \delta (s_2, 0), \delta (q_0, 1) >$$
$$\Rightarrow < s_1, q_0 >$$
$$\delta (< s_2, q_0 >, 1) \Rightarrow < \delta (s_2, 1), \delta (q_0, 1) >$$
$$\Rightarrow < s_0, q_1 >$$

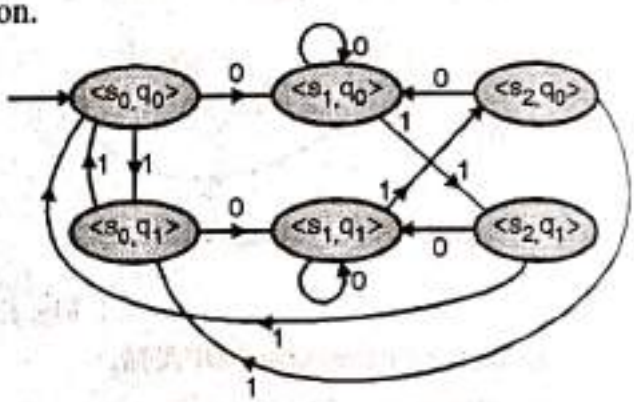


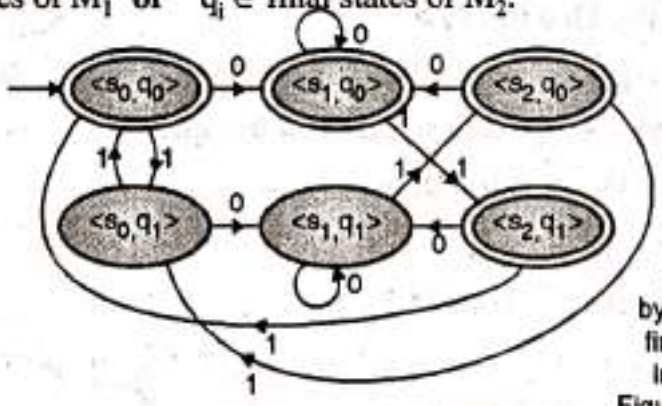
Fig. Ex. 2.2.34(f)

Finding a DFA for $L(M_1) \cup L(M_2)$

The set of final states of the required DFA is given by :

Every $< s_i, q_i >$ such that

$s_i \in$ final states of M_1 **or** $q_i \in$ final states of M_2.



Derived by selecting final states in DFA of Figure 2.2.33(f)

Fig. Ex. 2.2.34(g) : DFA for $L(M_1) \cup L(M_2)$

Finding DFA for $L(M_1) \cap L(M_2)$

The set of final states of the required DFA is given by :

Every $< s_i, q_i >$ such that $s_i \in$ final states of M_1 **and** $q_i \in$ final states of M_2.

Scanned by CamScanner

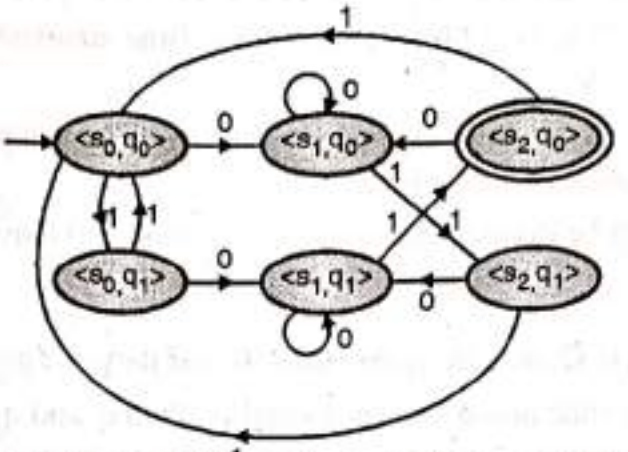


Fig. Ex. 2.2.34(h) : DFA for L (M_1) ∩ L (M_2)

Finding DFA for L (M_1) – L (M_2)

The set of final states of the required DFA is given by :

every $< s_i, q_i >$ such that $s_i \in$ final states of M_1 and $q_i \notin$ final states of M_2.

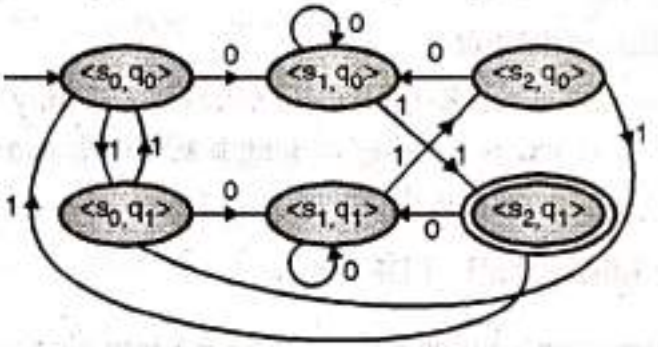


Fig. Ex. 2.2.34(i) : DFA for L (M_1) – L (M_2)

2.2.7.2 Complementation

Let $M_1 = (Q, \Sigma, \delta, q_0, F)$ be an arbitrary DFA. To prove the closure property complementation, we must show that there is another machine M_2 such that,

$$M_2(L) = (M_1(L))'$$

M_2 can be constructed from of M_1 by making the following changes to M_1.

1. Make every final state M_1 of as a non-final state.
2. Make every non-final state of M_1 as a final state.

Machine M_2 can be defined as :

$(Q, \Sigma, \delta, q_0, Q - F)$

2.2.8 Minimization of DFA

It may happen that a DFA contains redundant states. i.e. states whose functions can be handled by other states. Minimization of a DFA is a process of constructing an equivalent DFA with minimum number of states.

Scanned by CamScanner

- It may be possible that for a given DFA some of the states might be equivalent or they are not distinguishable.
- Equivalent states from an equivalent class or group. Every state from an equivalent class exhibit the same transition behaviour.
- States of a DFA can be divided into a group of equivalent classes.

Equivalent states

Two states q_i and $q_j \in Q$ are said to be equivalent if they are not distinguishable

(1) If q_i is an accepting state and q_j is non-accepting, then q_i and q_j are distinguishable.

(2) If q_i is non-accepting and q_j is accepting, then q_i and q_j are distinguishable.

(3) If there is string such that.

$\delta^*(q_i, x) \in F$ and $\delta^*(q_j, x) \notin F$

or

$\delta^*(q_i, x) \notin F$ and $\delta^*(q_j, x) \in F$

then q_i and q_j are distinguishable.

K- equivalence : q_i and q_j are K-equivalent states if and only if no string of length $\leq k$ can distinguish them. If there exists a string of length k, which can distinguish q_i and q_j, the states q_i and q_j are said to be K-distinguishable.

2.2.8.1 Algorithm for Minimization DFA's

Algorithm for minimization divides states of a DFA into blocks such that :

1. States in a block are equivalent.
2. No two states taken from two different blocks are equivalent.

The first step is to partition the states of M into blocks such that all states in a block are 0-equivalent. This is accomplished by placing :

1. Final states into one block.
2. Non-final states into another block.

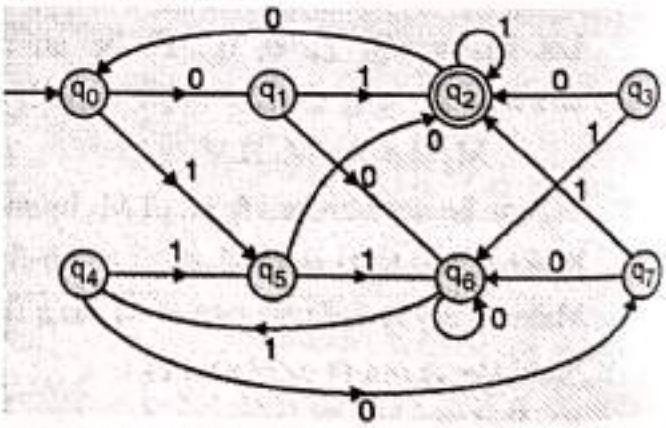


Fig. 2.2.8 : DFA considered for minimization

States of the DFA in Fig. 2.2.8 can be divided into two blocks.

block 1 = set of non-final states

= $(q_0, q_1, q_3, q_4, q_5, q_6, q_7)$

Scanned by CamScanner

block 2 = set of final states.
= (q_2)

∴ 0-equivalence partitioning of Q,

$P_0 = (q_0, q_1, q_3, q_4, q_5, q_6, q_7)\ (q_2)$

- The next step is to obtain the partition P_1 whose blocks consists of the set of states which are 1-equivalent, i.e. equivalent under any input sequence of length 1.

Checking $(q_0, q_1, q_3, q_4, q_5, q_6, q_7)$ for 1-equivalence

Input '0' :

$\delta(q_0, 0) \Rightarrow q_1$ {block 1, q_1 is in block 1}

$\delta(q_1, 0) \Rightarrow q_6$ {block 1, q_6 is in block 1}

$\delta(q_3, 0) \Rightarrow q_2$ {block 2, q_2 is in block 2}

$\delta(q_4, 0) \Rightarrow q_7$ {block 1, q_7 is in block 1}

$\delta(q_5, 0) \Rightarrow q_2$ {block 2, q_2 is in block 2}

$\delta(q_6, 0) \Rightarrow q_6$ {block 1, q_6 is in block 1}

$\delta(q_7, 0) \Rightarrow q_6$ {block 1, q_6 is in block 1}

On input 0, block 1, is successor of q_0, q_1, q_4, q_6, q_7.

On input 0, block 2, is successor of q_3, q_5.

Input '1' :

$\delta(q_0, 1) \Rightarrow q_5$ {block 1}

$\delta(q_1, 1) \Rightarrow q_2$ {block 2}

$\delta(q_3, 1) \Rightarrow q_6$ {block 1}

$\delta(q_4, 1) \Rightarrow q_5$ {block 1}

$\delta(q_5, 1) \Rightarrow q_6$ {block 1}

$\delta(q_6, 1) \Rightarrow q_4$ {block 1}

$\delta(q_7, 1) \Rightarrow q_2$ {block 2}

On input 1, block 1 is successor of q_0, q_3, q_4, q_5, q_6.

On input 1, block 2 is successor or of q_1, q_7.

q_3, q_5 are distinguishable from, q_0, q_1, q_4, q_6, q_7 on input 0.

q_1, q_7 are distinguishable from, q_0, q_3, q_4, q_5, q_6 on input 1.

∴ 1-equivalence partitioning of Q,

$P_1 = (q_3, q_5)$ (q_1, q_7) (q_0, q_4, q_6) (q_2)

↑ ↑ ↑ ↑

block 11 block 12 block 13 block 2

Scanned by CamScanner

- The next step is to obtain the partition P_2 whose blocks consists of the sets of states which are 2-equivalent, i.e. equivalent under any input sequence of length 2.

Checking (q_3, q_5) for equivalence – block 11

$\left.\begin{aligned}\delta(q_3, 0) &= q_2\\ \delta(q_5, 0) &= q_2\end{aligned}\right\}$ mapped to same block (q_2)

$\left.\begin{aligned}\delta(q_3, 1) &= q_6\\ \delta(q_5, 1) &= q_6\end{aligned}\right\}$ mapped to same block (q_0, q_4, q_6)

q_3 and q_5 are 2-equivalent.

Checking (q_1, q_7) for equivalence – block 12

$\left.\begin{aligned}\delta(q_1, 0) &= q_5\\ \delta(q_7, 0) &= q_6\end{aligned}\right\}$ mapped to same block (q_0, q_4, q_6)

$\left.\begin{aligned}\delta(q_1, 1) &= q_2\\ \delta(q_7, 1) &= q_2\end{aligned}\right\}$ mapped to same block (q_2)

q_1 and q_7 are 2-equivalent.

Checking (q_0, q_4, q_6) for equivalence – block 13

$\delta(q_0, 0) = q_1$ [block 12]

$\delta(q_4, 0) = q_7$ [block 12]

$\delta(q_6, 0) = q_6$ [block 13]

$\delta(q_0, 1) = q_5$ [block 11]

$\delta(q_4, 1) = q_5$ [block 11]

$\delta(q_6, 1) = q_4$ [block 13]

q_6 is distinguishable from q_0, q_4 on input 0/1.

$\therefore$ 2-equivalence partitioning of Q_1.

$P_2 = (q_3, q_5)\ (q_1, q_7)\ (q_0, q_4)\ (q_6)\ (q_2)$

- The next step is to obtain the partition P_3 whose blocks consists of the sets of states which are 3-equivalent, i.e. equivalent under any input sequence of length 3.

  It can be seen easily that p_2 can not be portioned further.

  $\therefore P_3 = P_2\ (q_3, q_5),\ (q_1, q_7)\ (q_0, q_4)\ (q_6)\ (q_2)$

- If for some k, $P_k = P_{k+1}$, the process terminates and P_k defines the blocks of equivalent states of the machine.

Scanned by CamScanner

- The transition diagram for minimum state DFA for DFA of Fig. 2.2.8 is given in Fig. 2.2.9. The process of drawing of minimum-state DFA is as below :

1. Start state is $<q_0, q_4>$, since q_0 was start state in Fig. 2.2.8.
2. Only accepting state is $<q_2>$, since q_2 is the only accepting state of Fig. 2.2.8.
3. Since, there is a transition from q_0 to q_1 on input 0 in Fig. 2.2.8, we add a transition from $<q_0, q_4>$ to $<q_1, q_7>$ on input 0 in Fig. 2.2.9.
4. Since, there is a transition from q_0 to q_5 on input 1 in Fig. 2.2.8, we add a transition from $<q_0, q_4>$ to $<q_3, q_5>$ on input 1 in Fig. 2.2.9.
5. Other transitions can be added in a similar fashion.

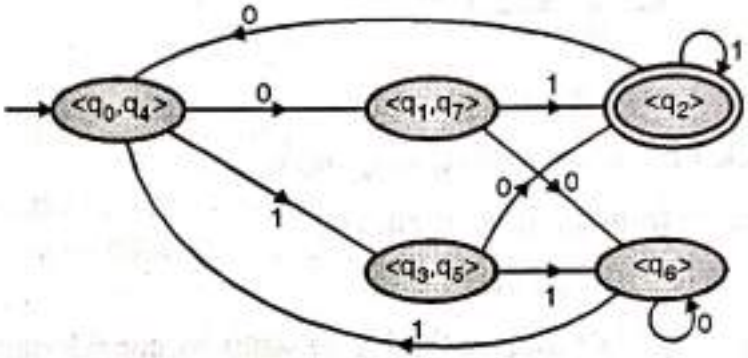


Fig. 2.2.9 : Minimum state DFA equivalent to Fig. 2.2.8

Example 2.2.35 : Construct the minimum state automata equivalent to given DFA.

MU - May 2011

	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_0	q_2
q_2	q_3	q_1
q_3^*	q_3	q_0
q_4	q_3	q_5
q_5	q_6	q_4
q_6	q_5	q_6
q_7	q_6	q_3

Solution :

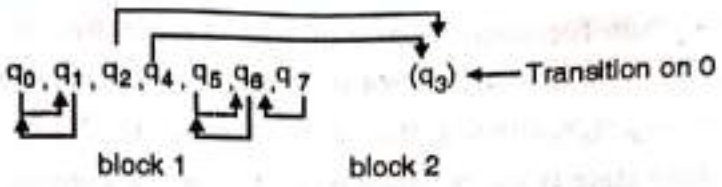
Step 1 : Finding 0-equivalence partitioning of states by putting final and non-final states into independent block.

$P_0 = (q_0, q_1, q_2, q_4, q_5, q_6, q_7)$ (q_3)

block 1 block 2

Step 2 : Finding 1-equivalence partitioning of states by considering transition on '0' and transition on '1'.

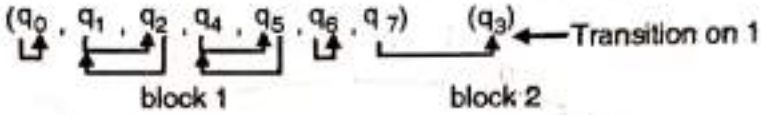
Scanned by CamScanner



On input 0, block 1 is successor of q_0, q_1, q_5, q_6, q_7.

On input 0, block 2 is successor of q_2, q_4.

$\therefore$ q_2, q_4 are distinguishable from q_0, q_1 q_5, q_6, q_7



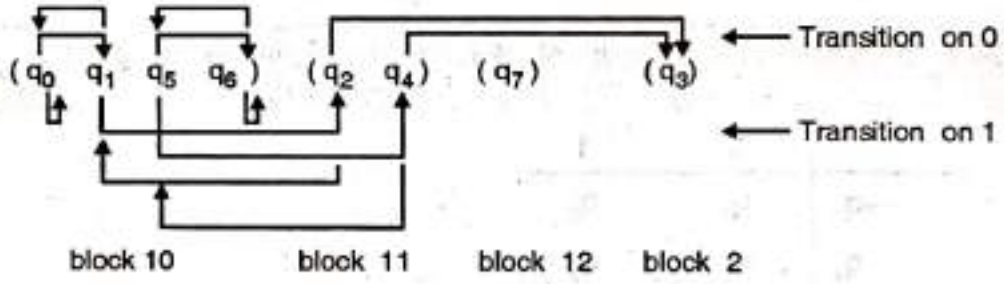
On input 1, block 2 is successor of q_7.

On input 1, block 1 is successor of q_0, q_1 q_2, q_4, q_5 , q_6.

q_7 is distinguishable from q_0, q_1, q_2, q_4, q_5 , q_6.

$P_1 = (q_0, q_1, q_5, q_6)\ (q_2, q_4)\ (q_7)\ (q_3)$

Step 3 : Finding 2-equivalence partitioning of states by considering transition on '0' and transition on '1'.



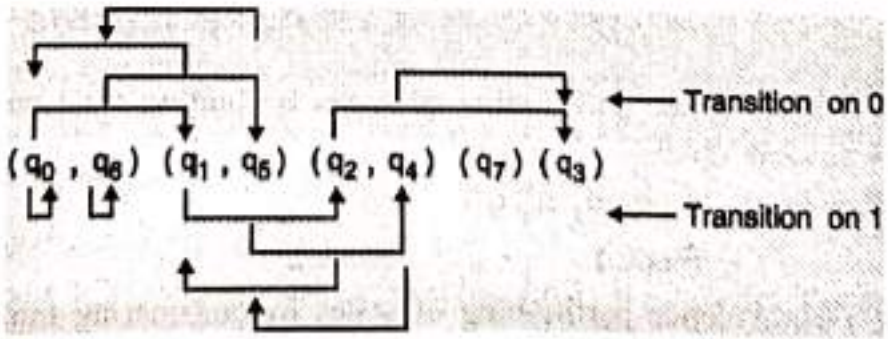
On input 1, block 11 is successor of q_1, q_5.

On input 1, block 10 is successor of q_0, q_6.

q_1, q_5 is distinguishable from q_0, q_6.

$$P_2 = (q_0, q_6)\ (q_1, q_5)\ (q_2, q_4)\ (q_7)\ (q_3)$$

Step 4 : Finding 3-equivalence partitioning of states by considering transition on 0 and 1.



Scanned by CamScanner

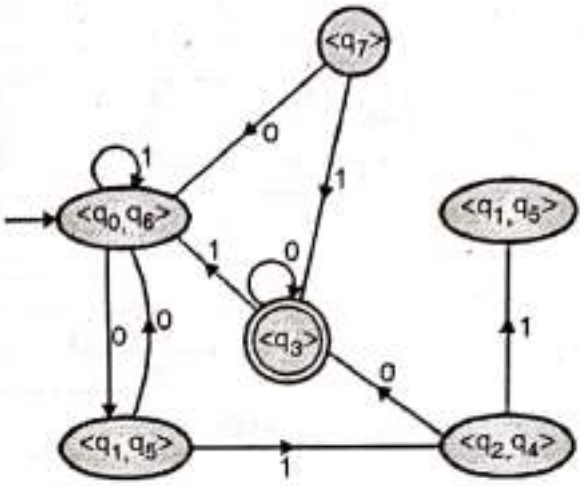
Blocks can not be divided further.

$\therefore P_3 = P_2 = (q_0, q_6)\ (q_1, q_5)\ (q_2, q_4)\ (q_7)\ (q_3)$ which is final set of blocks of equivalent classes.

Step 5 : Construction of minimum state DFA.

	0	1
$\rightarrow(q_0, q_6)$	(q_1, q_5)	(q_0, q_6)
(q_1, q_5)	(q_0, q_6)	(q_2, q_4)
(q_2, q_4)	(q_3)	(q_1, q_5)
$(q_3)^*$	(q_3)	(q_0, q_6)
(q_7)	(q_0, q_6)	(q_3)

(a) State transition diagram for minimum-state DFA



(b) State transition diagram for minimum-state DFA

Fig. Ex. 2.2.35

Example 2.2.36 : Construct the minimum-state DFA equivalent to given DFA.

MU - May 2009

	0	1
$\rightarrow q_0$	q_1	q_4
q_1	q_2	q_5
q_2^*	q_3	q_7
q_3	q_4	q_7
q_4	q_5	q_8
q_5^*	q_6	q_1
q_6	q_7	q_1
q_7	q_8	q_2
q_8^*	q_0	q_4

Solution :

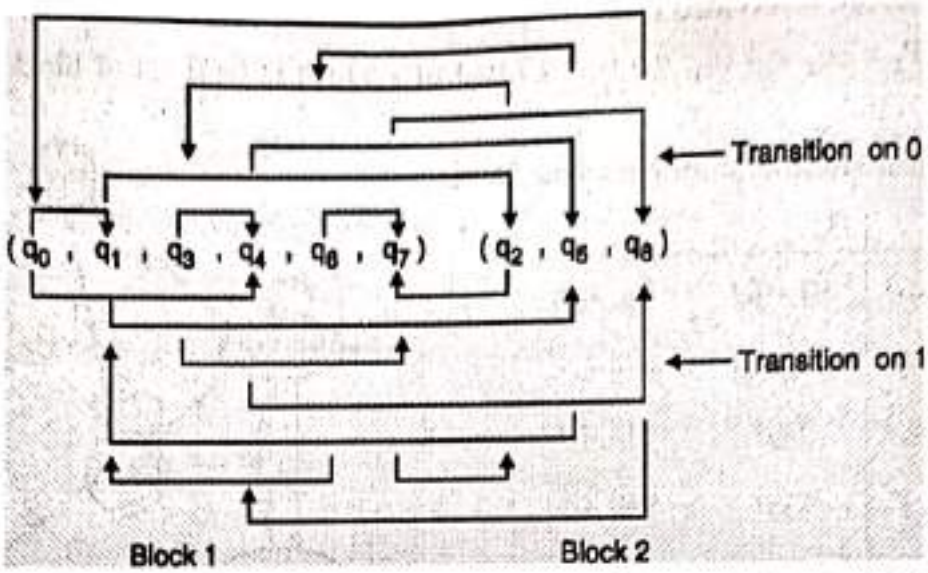
Step 1 : Finding 0-equivalence partitioning of states by putting final and non-final states into independent blocks.

$P_0 = (q_0, q_1, q_3, q_4, q_6, q_7)$ (q_2, q_5, q_8)

block 1 block 2

Step 2 : Finding 1-equivalence partitioning of states by considering transitions on '0' on '1'.

Scanned by CamScanner



On input 0, block 1 is successor of q_0, q_3, q_6.
On input 0, block 2 is successor of q_1, q_4, q_7.
On input 1, block 1 is successor of q_0, q_3, q_6.
On input 1, block 2 is successor of q_1, q_4, q_7.

} block 1

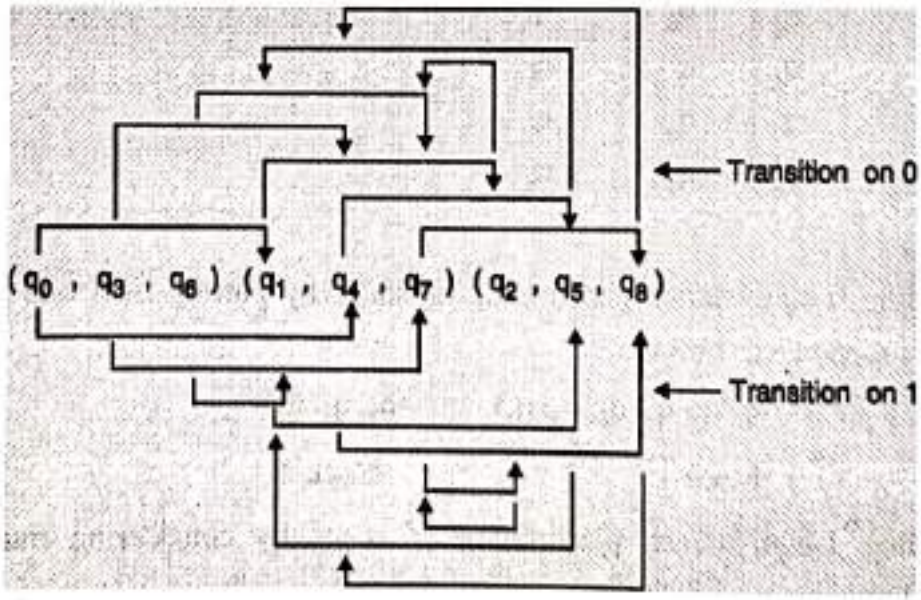
States q_0, q_3, q_6 are distinguishable from q_1, q_4, q_7.

Block 2 is 1-equivalent

$$P_1 = (q_0, q_3, q_6) \quad (q_1, q_4, q_7) \quad (q_2, q_5, q_8)$$

block 11 block 12 block 2

Step 3 : Finding 2-equivalence partitioning of states by considering transition on '0' and transition on '1'.



Blocks can not be divided further.

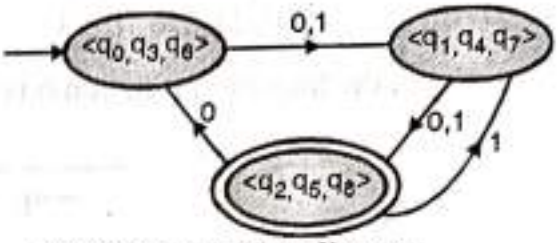
Scanned by CamScanner

$\therefore P_2 = P_1 = (q_0, q_3, q_6)\ (q_1, q_4, q_7)\ (q_2, q_5, q_8)$ which is final set of blocks of equivalent classes.

Step 4 : Construction of minimum state DFA.

	0	1
$\rightarrow(q_0, q_3, q_6)$	(q_1, q_4, q_7)	(q_1, q_4, q_7)
(q_1, q_4, q_7)	(q_2, q_5, q_8)	(q_2, q_5, q_8)
$(q_2, q_5, q_8)^*$	(q_0, q_3, q_6)	(q_1, q_4, q_7)

(a) State transition table for minimum state DFA



(b) State transition diagram for minimum state DFA

Fig. Ex. 2.2.36 : Final DFA for Example 2.2.36

2.3 Non-deterministic Finite Automata

Non-deterministic finite automata can reside in multiple states at the same time. The concept of non-deterministic finite automata is being explained with the help of transition diagram (Fig. 2.3.1).

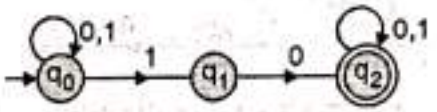


Fig. 2.3.1 : Transition diagram of a nondeterministic finite automata

If the automata is in the state q_0 and the next input symbol is 1, then the next state will be either :

(1) q_0 (2) q_1.

Thus, the move from q_0 on input 1 cannot be determined uniquely. Such machines are called Non-deterministic automata.

- Non-deterministic finite automata can be in several states at any time. Thus, NFA can guess about an input sequence.
- Every NFA can be converted into an equivalent DFA.
- An NFA can be designed with fewer states compare to its deterministic counterpart.
- Due to non-determinism, NFA takes more time to recognize a string.

2.3.1 Definition of NFA

MU - Dec. 2006

A Non-deterministic finite automata is a 5-tuple.

$$M = (Q, \Sigma, \delta, q_0, F)$$

where,

Q = A finite set of states

Σ = A finite set of inputs

δ = A transition function from $Q \times \Sigma$ to the power set of Q i.e.to 2^Q.

Scanned by CamScanner

$q_0 = q_0 \in Q$ is the start / initial state

$F = F \subseteq Q$ is a set of final / accepting states.

The NFA, for Fig. 2.3.1, can be formally represented as,

$(\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$

where, the transition function δ is given below in Fig. 2.3.2.

	0	1
$\rightarrow q_0$	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_2,\}$	ϕ
q_2^*	$\{q_2\}$	$\{q_2\}$

Fig. 2.3.2 : A transition table for NFA in Fig. 2.3.1

2.3.2 Processing of a String by NFA

A string $\omega \in \Sigma^*$ is accepted by NFA M if $\delta^*(q_0, \omega)$ contains a final state.

Let us see how the string 011010 is processed by the NFA given in Fig. 2.3.3.

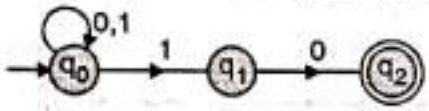


Fig. 2.3.3 : A sample NFA

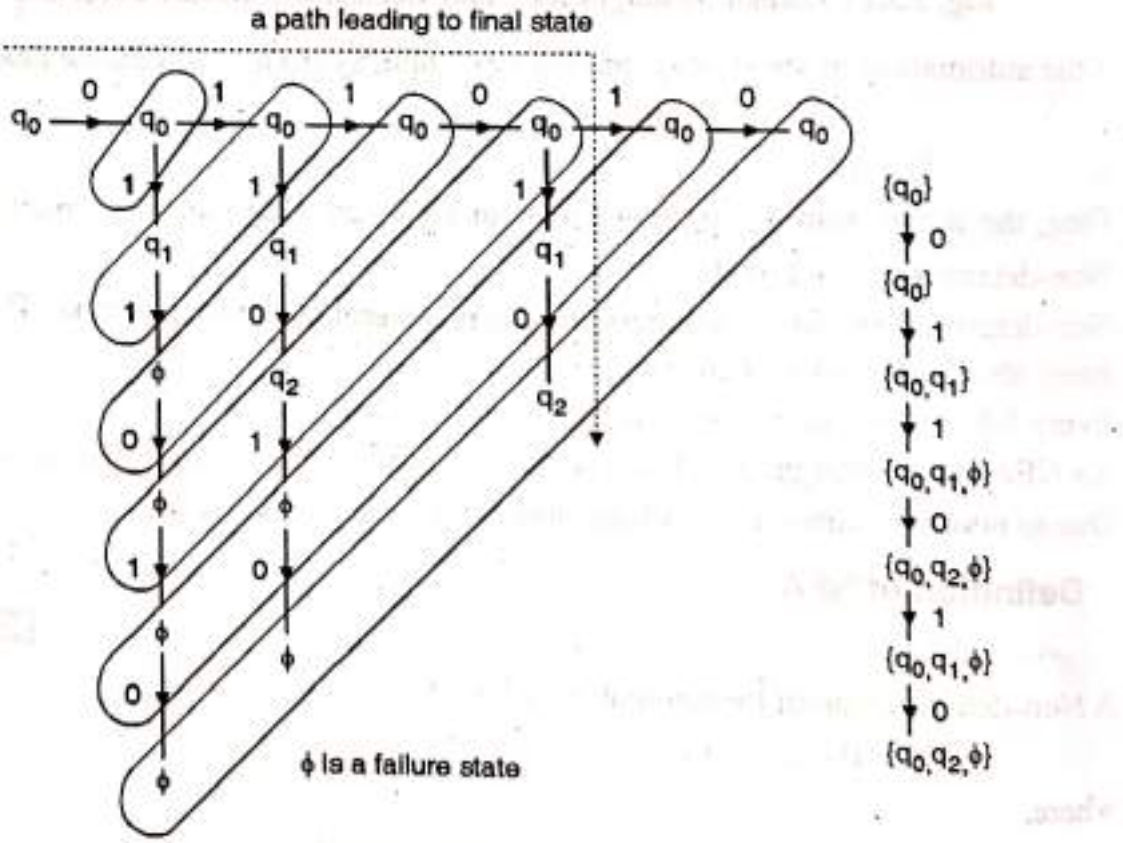


Fig. 2.3.4 : States reached while processing of 011010

- From the state q_0, after input 0, resulting state is q_0.

Scanned by CamScanner

- From the state q_0, after input 1, resulting states are q_0, q_1.

  An input 0 in q_0, generates two paths :

  1. $q_0 \xrightarrow{1} q_0$ 2. $q_0 \xrightarrow{1} q_1$

  On input 011010, the automata can take any of the four paths :

  1. $q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_1 \xrightarrow{1} \phi \xrightarrow{0} \phi \xrightarrow{1} \phi \xrightarrow{0} \phi$

     [Once in ϕ state (dead state) machine will remain in ϕ state]

  2. $q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_2 \xrightarrow{1} \phi \xrightarrow{0} \phi$

  3. $q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_2$

  4. $q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_0$

  Out of the four paths, the path number 3 takes the machine from start state to the final state on input 011010. q_2 is a final state.

  A string is accepted by non-deterministic finite automata, if there exists a path that takes the machine to a final state.

  Thus, 011010 will be accepted by the NFA of Fig. 2.3.3.

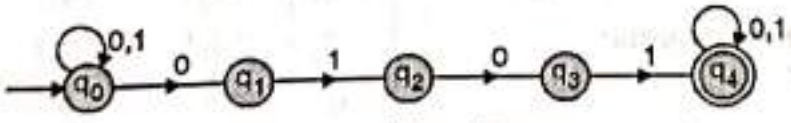
- The language accepted by the above NFA consists of string ending in 10. A string of length n ($|\omega| = n$), ending 10 can be accepted by the above NFA if :
  1. The machine remains in q_0 for first n-2 inputs.
  2. On last two inputs 10, the machine makes the transition as shown below :

     $q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_2$

- To reach the final state, and to remain there, it is necessary that the last two characters of the string should be 10.
- Thus the NFA in Fig. 2.3.3 accepts a string, if and only if it ends in 10.

Example 2.3.1 : Draw a non-deterministic automata to accept strings containing the substring 0101.

Solution :



(a) State transition diagram

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	ϕ	$\{q_2\}$
q_2	$\{q_3\}$	ϕ
q_1	ϕ	$\{q_4\}$
q_4^*	$\{q_4\}$	$\{q_4\}$

(b) State transition table

Fig. Ex. 2.3.1

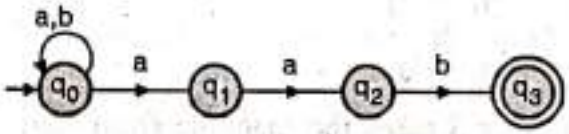
Scanned by CamScanner

- For reaching the final state q_4, from the start state q_0, a substring 0101 is required.
- Any string, containing a substring 0101 will be accepted by the above NFA. The machine can remain in q_0, for the portion of string before 0101. The machine can remain in q_4 for the portion of string after 0101.

Example 2.3.2 : Construct a NFA that accepts any positive number of occurrences of various strings from the following language L given by,

$L = \{x \in \{a,b\} \mid x \text{ ends with aab}\}$

Solution :



	a	b
$\rightarrow q_0$	$\{q_0, q_1\}$	q_0
q_1	q_2	–
q_2	–	q_3
q_3	–	–

Fig. Ex. 2.3.2

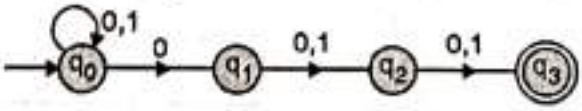
- The language accepted by the NFA shown in Fig. Ex. 2.3.2 consists of string ending in aab. A string of length n, ending in aab can be accepted by the NFA if :
  1. The machine remains in q_0 for first n-3 inputs.
  2. On last three inputs aab, the machine makes the transition as :

$$q_0 \xrightarrow{a} q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_3$$

- To reach the final state, and to remain there, it is necessary that the last two characters of the string should be 10.
- Thus the NFA accepts a string, if and only if it ends in 10.

Example 2.3.3 : Draw a non-deterministic finite automata to accept strings over alphabet {0, 1}, such that the third symbol from the right end is 0.

Solution :



(a) State transition diagram

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\{q_2\}$	$\{q_2\}$
q_2	$\{q_3\}$	$\{q_3\}$
q_3^*	ϕ	ϕ

(b) State transition table

Fig. Ex. 2.3.3 : NFA for Example 2.3.3

- For the first n-3 symbols, the machine can remain in the start state q_0.
- On seeing the third symbol from the right end as 0 (NFA can guess), it makes a move towards the final state q_3.

Scanned by CamScanner

Example 2.3.4 : Design a NFA to recognize the following sets of strings : abc, abd, aacd. Assume that alphabet is {a, b, c, d}. Give the transition table and transition diagram data.

Solution :

Machine can guess and take one of the three paths as shown :

1. Path $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3$ for string abc.
2. Path $q_0 \rightarrow q_4 \rightarrow q_5 \rightarrow q_6$ for string abd.
3. Path $q_0 \rightarrow q_7 \rightarrow q_8 \rightarrow q_9 \rightarrow q_{10}$ for string aacd.

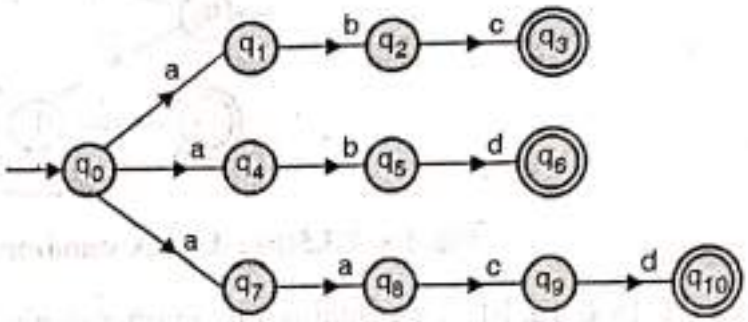


Fig. Ex. 2.3.4 : State diagram of NFA

Table Ex. 2.3.4 : State table for NFA

	a	b	c	d
$\rightarrow q_0$	$\{q_1, q_4, q_7\}$	ϕ	ϕ	ϕ
q_1	ϕ	q_2	ϕ	ϕ
q_2	ϕ	ϕ	q_3	ϕ
q_3^*	ϕ	ϕ	ϕ	ϕ
q_4	ϕ	q_5	ϕ	ϕ
q_5	ϕ	ϕ	ϕ	q_6
q_6^*	ϕ	ϕ	ϕ	ϕ
q_7	q_8	ϕ	ϕ	ϕ
q_8	ϕ	ϕ	q_9	ϕ
q_9	ϕ	ϕ	ϕ	q_{10}
q_{10}^*	ϕ	ϕ	ϕ	ϕ

Example 2.3.5 : Construct the NFA and DFA for the following languages.

(i) L = {x ∈ {a, b, c}* : x contains exactly one b immediately following c}

(ii) L = {x ∈ {0, 1}* : x is starting with 1 and | x | is divisible by 3}

(iii) L = {x ∈ {a, b}* : x contains any number of a's followed by at least one b}

Solution :

(i) L {x ∈ {a, b, c}*: x contains exactly one b immediately following c}

Since, the problem is full deterministic in nature; it will serve no purpose to design an NFA. We can design a DFA and since every DFA is also an NFA, a DFA can stand for both DFA and NFA.

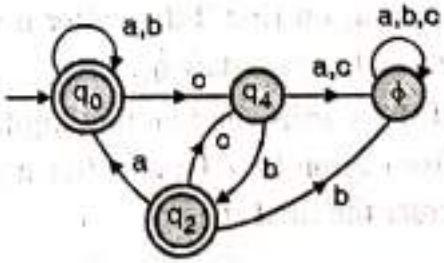


Fig. Ex. 2.3.5(a) : A DFA standing for both DFA and NFA

Scanned by CamScanner

(ii) L {x ∈ {0, 1}* : x is starting with 1 and | x | is divisible by 3}

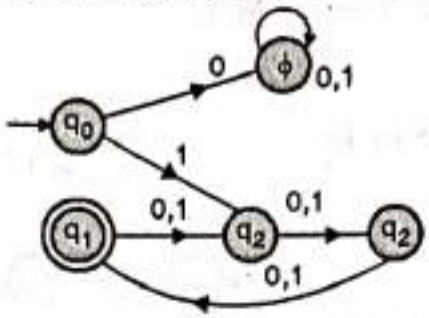


Fig. Ex. 2.3.5(b) : A DFA standing for both DFA and NFA

(iii) L {x ∈ {a, b}* : x contains any number of a's followed by at least one b}

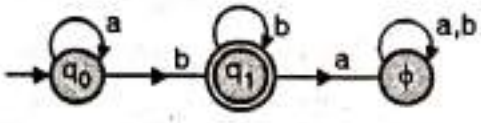
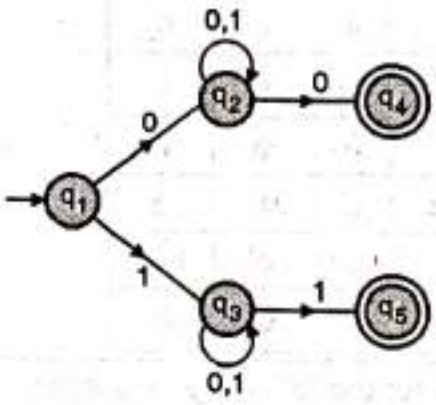


Fig. Ex. 2.3.5(c) : A DFA standing for both DFA and NFA

Note : Technically, a DFA with ϕ transitions omitted may be called as NFA.

Example 2.3.6 : Design a NFA for a binary number where the first and the last digits are same

Solution :



(a) State transition diagram

	0	1
$\rightarrow q_1$	q_2	q_3
q_2	$\{q_2, q_4\}$	q_2
q_3	q_3	$\{q_3, q_4\}$
q_4^*	–	–
q_5^*	–	–

(b) State transition table

Fig. Ex. 2.3.6

- If the starting and ending digits are 0 then NFA will end in q_4 by making a move to q_2 from q_1 on first 0 thereafter it will wait non-deterministically in q_2 and on last 0 it will enter the final state q_4.
- If the starting and ending digits are 1 then NFA will end in q_5 by making a move to q_3 from q_1 on first 1 thereafter it will wait non-deterministically in q_3 and on last 1 it will enter the final state q_5.

Scanned by CamScanner

Example 2.3.7 : An NFA with states 1-5 and input alphabet {a, b} has following transition table.

(a) Draw a transition diagram (b) Calculate $\delta^*(1, ab)$

(c) Calculate $\delta^*(1, abaab)$

q	δ (q, a)	δ (q, b)
1	{1, 2}	{1}
2	{3}	{3}
3	{4}	{4}
4	{5}	ϕ
5	ϕ	{5}

Solution :

(a) Transition diagram is given in Fig. Ex. 2.3.7.

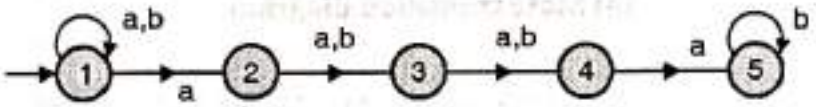


Fig. Ex. 2.3.7 : State transition diagram

(b) $\delta^*(1, ab) = \delta((\delta(1, a)), b)$

$= \delta(\{1, 2\}, b) = \delta(1, b) \cup \delta(2, b)$

$= \{1\} \cup \{3\} = \{1, 3\}$

(c) $\delta^*(1, abaab) = \delta^*(\{1, 2\}, baab)$

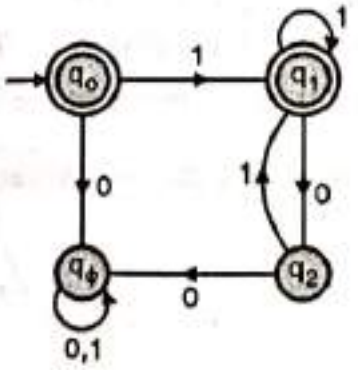
$= \delta^*(\{1, 3\}, aab)$

$= \delta^*(\{1, 2, 4\}, ab) = \delta(\{1, 2, 3, 5\}, b)$

$= \{1, 3, 4, 5\}$

Example 2.3.8 : Construct a FA for accepting L over {0,1} such that each 0 is immediately preceded and immediately followed by 1.

Solution :



	0	1
→ q_0^*	q_ϕ	q_1
q_1^*	q_2	q_1
q_2	q_ϕ	q_1
q_ϕ	q_ϕ	q_ϕ

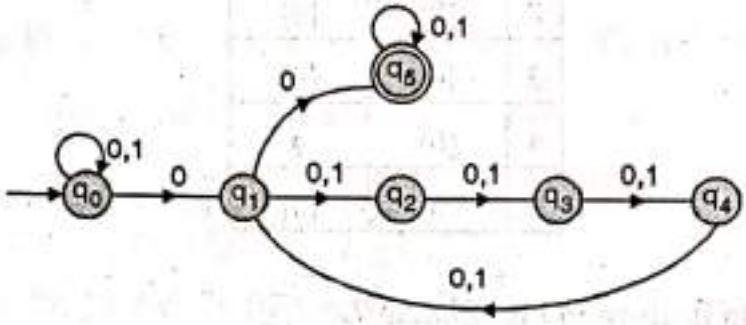
Fig. Ex. 2.3.8

Scanned by CamScanner

- If the starting symbol is 0, string is rejected by entering the failure state q_ϕ.
- A loop through $q_1 \rightarrow q_2 \rightarrow q_1$ ensures that every 0 is preceded and succeeded by 1.
- An input 0 in state q_2 will mean two consecutive 0's. The string is rejected by entering the failure state q_ϕ.

Example 2.3.9 : Construct a NFA that accept the set of strings in (0 + 1)* such that some two 0's are separated by string whose length is 4i, for some i >=0.

Solution :



(a) State transition diagram

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\{q_2, q_5\}$	$\{q_2\}$
q_2	$\{q_3\}$	$\{q_3\}$
q_3	$\{q_4\}$	$\{q_4\}$
q_4	$\{q_1\}$	$\{q_1\}$
q_5^*	$\{q_5\}$	$\{q_5\}$

(b) State transition table

Fig. Ex. 2.3.9 : NFA for Example 2.3.9

- The loop $q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4 \rightarrow q_1$ can absorb a string of length 4i.
- On receiving the 0 of the portion of the string containing 'two 0's separated by string of length 4i, the machine makes a move to q_1 from q_0. Then, it loops i times in $q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4 \rightarrow q_1$. Then it gets a 0 and makes a move from q_1 to q_5.

Example 2.3.10 : Design NFA for the set of strings on the alphabet {0, 1} that start with 01 and end with 10.

Scanned by CamScanner

Solution :

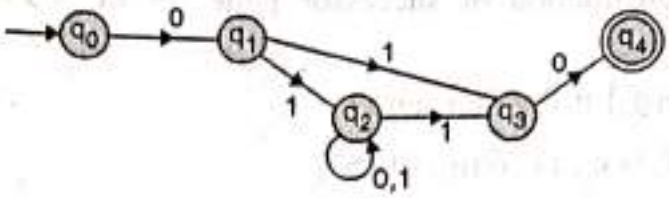


Fig. Ex. 2.3.10 : NFA for Example 2.3.10

- A string 010 (starting with 01 and ending with 10) can be accepted through the path $q_0 \rightarrow q_1 \rightarrow q_3 \rightarrow q_4$.
- For any other string, the portion between starting 01 and ending 10 can be absorbed by the state q_2.

2.3.3 NFA to DFA Conversion

The NFA to DFA conversion is based on subset construction. If a non-deterministic finite state machine consists of three states.

i.e. $Q = (q_0, q_1, q_2)$

The machine could be in any of the following states :

1. ϕ
2. (q_0) , (q_1), (q_2)
3. (q_0, q_1), (q_0, q_2), (q_1, q_2)
4. (q_0, q_1, q_2)

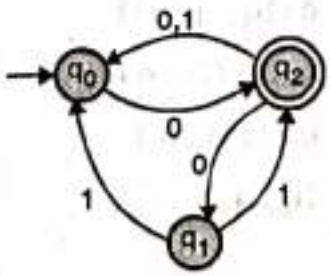
These subsets form a power set on Q, which is given by 2^Q.

The procedure for finding whether a given string is accepted by the NFA, involves :

1. Tracing the various paths followed by the machine for the given string.
2. Finding the set of states reached from the starting state by applying the symbols of the string.
3. If the set of states obtained in step (2), contains a final state, the given string is accepted by the NFA.

The above procedure can be applied to any string by constructing a successor table.

$\delta =$



$\delta =$

	0	1
$\rightarrow q_0$	$\{q_2\}$	ϕ
q_1	ϕ	$\{q_0, q_2\}$
q_2^*	$\{q_0, q_1\}$	$\{q_0\}$

Fig. 2.3.5 : NFA to be converted to DFA

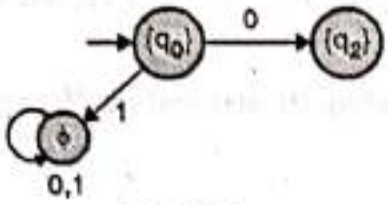
Scanned by CamScanner

Algorithm for construction of successor table for the NFA of Fig. 2.3.5 is being explained, stepwise :

Step 1 : Starting state $\{q_0\}$ is the first subset.

0 – successor of q_0 is q_2, i.e. $\delta(q_0, 0) \Rightarrow q_2$.

1 – successor of q_0 is ϕ, i.e. $\delta(q_0, 1) \Rightarrow \phi$.



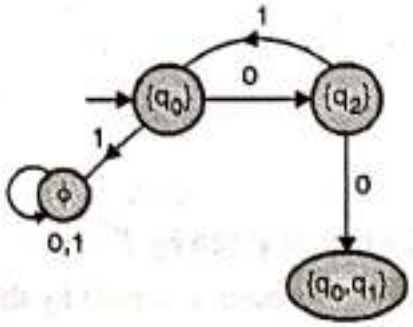
	0	1
→{q0}	{q2}	ϕ

Fig. 2.3.6

Step 2 : A new subset $\{q_2\}$ is generated. Successor of the subset $\{q_2\}$ are generated.

0 – successor of $\{q_2\}$ are $\{q_0, q_1\}$.

1 – successor of $\{q_2\}$ are $\{q_0\}$.



	0	1
→{q0}	{q2}	ϕ
{q2}	{q0, q1}	{q0}

Fig. 2.3.7

Step 3 : A new subset $\{q_0, q_1\}$ is generated. Successors of the subset $\{q_0, q_1\}$ are generated.

$$\begin{aligned} 0 - \text{successor of } \{q_0, q_1\} &= \delta(\{q_0, q_1\}, 0) \\ &= \delta(q_0, 0) \cup \delta(q_1, 0) \\ &= \{q_2\} \cup \phi = \{q_2\} \\ 1 - \text{successor of } \{q_0, q_1\} &= \delta(\{q_0, q_1\}, 1) \\ &= \delta(q_0, 1) \cup \delta(q_1, 1) \\ &= \phi \cup \{q_0, q_2\} \\ &= \{q_0, q_2\} \end{aligned}$$

Scanned by CamScanner

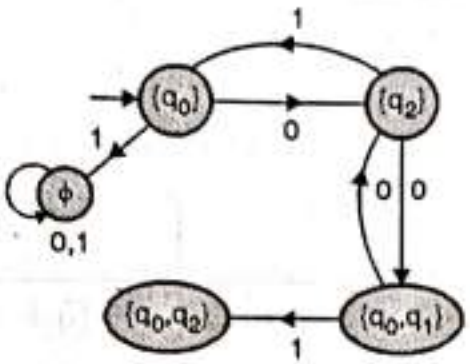


Fig. 2.3.8

	0	1
→{q0}	{q2}	ϕ
{q2}	{q0, q1}	{q0}
{q0, q1}	{q2}	{q0, q2}

Step 4 : A new subset $\{q_0, q_2\}$ is generated successors of the subset $\{q_0, q_2\}$ are generated.

$$0 - \text{successor of } \{q_0, q_1\} = \delta(\{q_0, q_2\}, 0)$$
$$= \delta(q_0, 0) \cup \delta(q_2, 0)$$
$$= \{q_2\} \cup \{q_2, q_1\} = \{q_0, q_1, q_2\}$$
$$1 - \text{successor of } \{q_0, q_2\} = \delta(\{q_0, q_2\}, 1)$$
$$= \delta(q_0, 1) \cup \delta(q_2, 1) = \phi \cup \{q_0\} = \{q_0\}$$

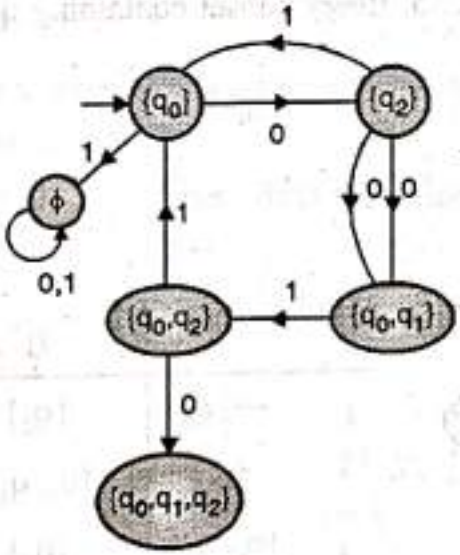


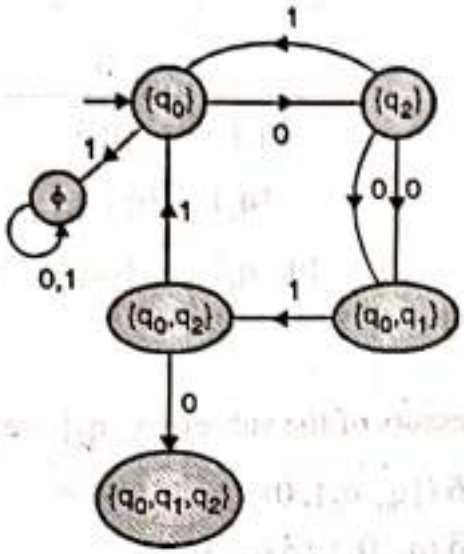
Fig. 2.3.9

	0	1
→{q0}	{q2}	ϕ
{q2}	{q0, q1}	{q0}
{q0, q1}	{q2}	{q0 , q2}
{q0 , q2}	{q0 , q1, q2}	{q0}

Step 5 : A new subset $\{q_0, q_1, q_2\}$ is generated successors of the subset $\{q_0, q_1, q_2\}$ are generated.

$$0 - \text{successor of } \{q_0, q_1, q_2\} = \delta(\{q_0, q_1, q_2\}, 0)$$
$$= \delta(\{q_0, q_1\}\ 0) \cup \delta(q_2, 0)$$
$$= \{q_2\} \cup \{q_0, q_1\} = \{q_0, q_1, q_2\}$$
$$1 - \text{successor of } \{q_0, q_1, q_2\} = \delta(\{q_0, q_1, q_2\}, 1)$$
$$= \delta(\{q_0, q_1\}, 1) \cup \delta(q_2, 1)$$
$$= \{q_0, q_2\} \cup \{q_0\} = \{q_0, q_2\}$$

Scanned by CamScanner

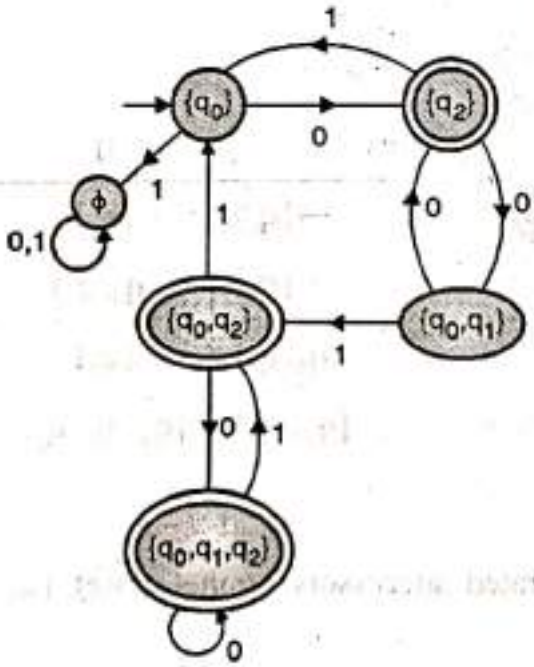


	0	1
$\rightarrow\{q_0\}$	$\{q_2\}$	ϕ
$\{q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_2\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_0\}$

Fig. 2.3.10

Since, a new subset is not generated the process of subset generation stops.

Step 6 : q_2 is the final state in NFA of Fig. 2.3.5. Every subset containing q_2 should be takes as a final state.



	0	1
$\rightarrow\{q_0\}$	$\{q_2\}$	ϕ
$\{q_2\}^*$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_2\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}^*$	$\{q_0, q_1, q_2\}$	$\{q_0\}$
$\{q_0, q_1, q_2\}^*$	$\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$

Fig. 2.3.11 : Final DFA after marking of accepting states

Theorem 2.1 : A language L is accepted by some NFA if and only if it is accepted by some DFA.

OR

For every NFA, there exists an equivalent DFA. MU - Dec. 2014

Scanned by CamScanner

Proof :

This theorem has two parts :

1. If L is accepted by a DFA M_2, then L is accepted by some NFA M_1.
2. If L is accepted by an NFA M_1, then L is accepted by some DFA M_2.

First part can be proved trivially. Determinism is a case of non-determinism. Thus a DFA is also an NFA.

Second part of the theorem is proved below :

Construct M_2 from M_1 using subset generation algorithm as explained earlier. We can prove the theorem using induction on the length of ω.

Base case : Let $\omega = \varepsilon$ with $|\omega| = 0$, where $|\omega|$ is length of ω.

Starting state for both NFA and DFA are taken as q_0. When $\omega = \varepsilon$, both DFA and NFA will be in q_0. Hence, the base case is proved.

Assumption : Let us assume that both NFA and DFA are equivalent for every string of length n. We must show that the machines M_1 (NFA) and M_2 (DFA) are equivalent for strings of length (n + 1).

Let $\omega_{n+1} = \omega_n a$, where ω_n is a string of length n and ω_{n+1} is a string of length (n + 1). 'a' is an arbitrary alphabet from Σ.

$\delta_2(q_2, \omega_n) = \delta_2(q_0, \omega_n)$, where δ_2 is transition function of DFA (M_2) and δ , is transition function of NFA (M_1).

If the subset reached by NFA is given by

$\{p_1, p_2, \cdots p_k\}$

$$\text{then,} \quad \delta_2(q_0, \omega_{n+1}) = \bigcup_{i=1}^{k} \delta_2(p_i, a) \qquad \ldots(i)$$

$$\text{or } \delta_2(\{p_1, p_2, \cdots p_k\}, a) = \bigcup_{i=1}^{k} \delta_1(p_i, a) \qquad \ldots(ii)$$

$$\text{also,} \quad \delta_2(q_0, \omega_n) = \{p_1, p_2, \cdots p_k\} \qquad \ldots(iii)$$

from (i), (ii) and (iii) we get,

$$\begin{aligned} \delta_2(q_0, \omega_{n+1}) &= \delta_2(\delta_2(q_0, \omega_n), a) \\ &= \delta_2(\{p_1, p_2, \cdots p_k\}, a) \\ &= \bigcup_{i=1}^{k} \delta_1(p_i, a) = \delta_1(q_0, s_{n+1}) \end{aligned}$$

Thus, the result is true for $|\omega| = n + 1$, hence it is always true.

Scanned by CamScanner

Example 2.3.11 : Convert to a DFA the following NFA :

	0	1
→p	{p, q}	{p}
q	{r}	{r}
r	{s}	ϕ
s*	{s}	{s}

Solution :

Table Ex. 2.3.11 : State transition table of DFA

	0	1	
→{p}	{p, q}	{p}	A new subset {p, q} is generated.
{p, q}	{p, q, r}	{p, r}	Two new subsets {p, q, r} and {p, r} are generated
{p, q, r}	{p, q, r, s}	{p, r}	A new subset {p, q, r, s} is generated
{p, r}	{p, q, s}	{p}	A new subset {p, q, s} is generated
{p, q, r, s}*	{p, q, r, s}	{p, r, s}	A new subset {p, r, s} is generated
{p, r, s}*	{p, q, s}	{p, s}	A new subset {p, s} is generated
{p, s}*	{p, q, s}	{p, s}	

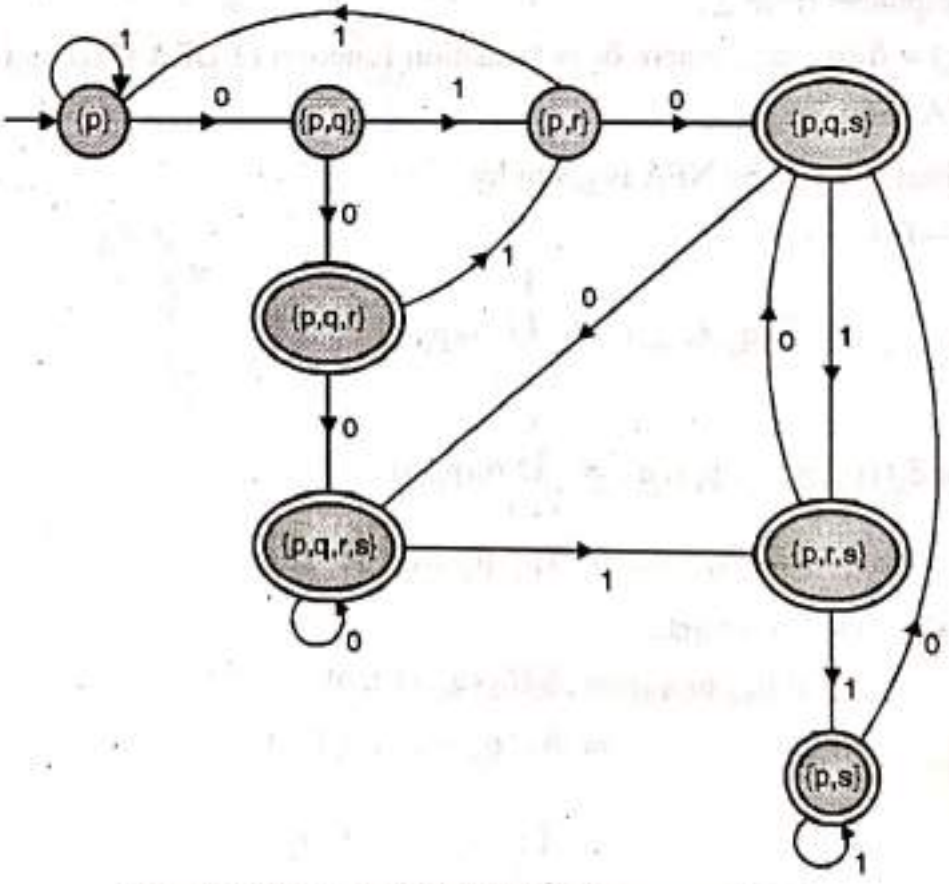


Fig. Ex. 2.3.11 : State transition diagram of DFA

Scanned by CamScanner

- Subsets are generated recursively.
- Every subset is selected for generation of successor subsets.
- Every subset containing s (final state of the given NFA) should be marked as a final state.

Example 2.3.12 : Convert to a DFA the following NFA.

	0	1
→ {p}	{q, s}	{q}
q*	{r}	{q, r}
r	{s}	{p}
s*	ϕ	{p}

Solution :

Step 1 : {p} is taken as the first subset.

0-Successor of {p} = δ ({p}, 0) = {q, s}

1-Successor of {p} = δ ({p}, 1) = {q}

	0	1
→{p}	{q, s}	{q}

Step 2 : Two new subsets {q, s} and {q} have been generated. Successors of {q, s} and {q} are calculated.

$$\delta(\{q, s\}, 0) = \delta(q, 0) \cup \delta(s, 0) = \{r\} \cup \phi = \{r\}$$

$$\delta(\{q, s\}, 1) = \delta(q, 1) \cup \delta(s, 1) = \{q, r\} \cup \{p\} = \{p, q, r\}$$

$$\delta(\{q\}, 0) = \{r\}$$

$$\delta(\{q\}, 1) = \{q, r\}$$

	0	1
→{p}	{q, s}	{q}
{q, s}	{r}	{p, q, r}
{q}	{r}	{q, r}

Step 3 : Three new subsets {r}, {p, q, r} and {q, r} are generated.

Their successors are calculated.

$$\delta(\{r\}, 0) = \{s\}$$

$$\delta(\{r\}, 1) = \{p\}$$

$$\delta(\{p, q, r\}, 0) = \delta(p, 0) \cup \delta(q, 0) \cup \delta(r, 0)$$

Scanned by CamScanner

$$= \{q, s\} \cup \{r\} \cup \{s\}$$
$$= \{q, r, s\}$$
$$\delta(\{p, q, r\}, 1) = \delta(p, 1) \cup \delta(q, 1) \cup \delta(r, 1)$$
$$= \{q\} \cup \{q, r\} \cup \{p\}$$
$$= \{p, q, r\}$$
$$\delta(\{q, r\}, 0) = \delta(q, 0) \cup \delta(r, 0)$$
$$= \{r\} \cup \{s\}$$
$$= \{r, s\}$$
$$\delta(\{q, r\}, 1) = \delta(q, 1) \cup \delta(r, 1)$$
$$= \{q, r\} \cup \{p\}$$
$$= \{p, q, r\}$$

	0	1
→{p}	{q, s}	{q}
{q, s}	{r}	{p, q, r}
{q}	{r}	{q, r}
{r}	{s}	{p}
{q, r}	{r, s}	{p, q, r}
{p, q, r}	{q, r, s}	{p, q, r}

Step 4 : Three new subsets {s}, {r, s} and {q, r, s} are generated. Their successors are calculated.

$$\delta(\{s\}, 0) = \phi$$
$$\delta(\{s\}, 1) = p$$
$$\delta(\{r, s\}, 0) = \{s\}$$
$$\delta(\{r, s\}, 1) = \{p\}$$
$$\delta(\{q, r, s\}, 0) = \{r, s\}$$
$$\delta(\{q, r, s\}, 1) = \{p, q, r\}$$

	0	1
→{p}	{q, s}	{q}
{q, s}	{r}	{p, q, r}
{q}	{r}	{q, r}
{r}	{s}	{p}

Scanned by CamScanner

{q, r}	{r, s}	{p, q, r}
{p, q, r}	{q, r, s}	{p, q, r}
{s}	ϕ	{p}
{r, s}	{s}	{p}
{q, r, s}	{r, s}	{p, q, r}

Step 5 : Every subset containing q or s is declared as a final state. Final DFA is shown in Fig. Ex. 2.3.12.

Table Ex. 2.3.12 : State transition table of DFA

	0	1
→{p}	{q, s}	{q}
{q, s}*	{r}	{p, q, r}
{q}*	{r}	{q, r}
{r}	{s}	{p}
{q, r}*	{r, s}	{p, q, r}
{p, q, r}*	{q, r, s}	{p, q, r}
{s}*	ϕ	{p}
{r, s}*	{s}	{p}
{q, r, s}*	{r, s}	{p, q, r}

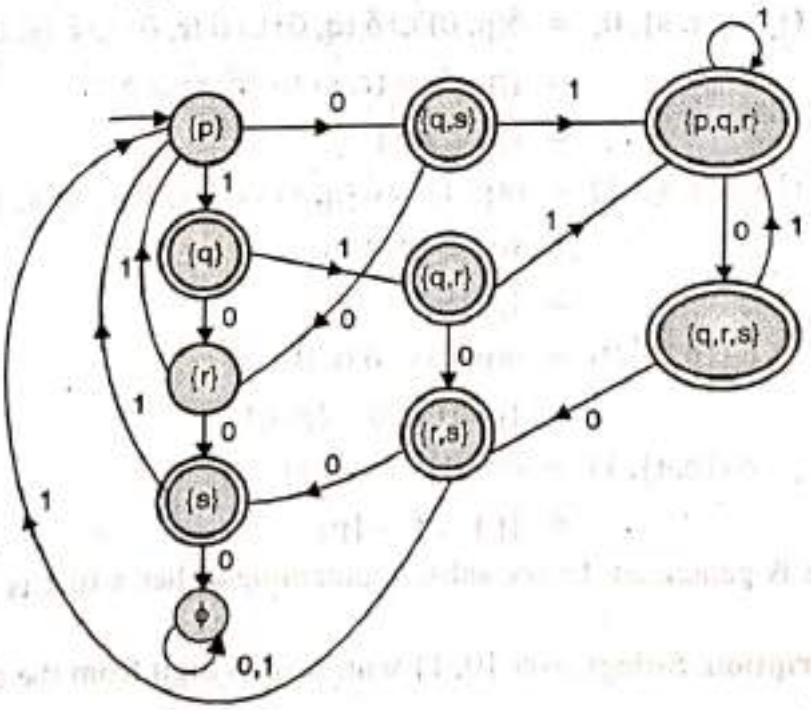


Fig. Ex. 2.3.12 : State transition diagram of the DFA

Scanned by CamScanner

Example 2.3.13 : Convert the following NFA to a DFA and informally describe the language it accepts. **MU - Dec. 2011**

	0	1
→p	{p, q}	{p}
q	{r, s}	{t}
r	{p, r}	{t}
s*	ϕ	ϕ
t*	ϕ	ϕ

Solution :

Step 1 : {p} is taken as the first subset.

$$0\text{-Successor of } \{p\} = \delta(\{p\}, 0) = \{p, q\}$$

$$1\text{-Successor of } \{p\} = \delta(\{p\}, 1) = \{p\}$$

Step 2 : The new subsets {p, q} is generated. Successors of {p, q} are calculated.

$$\begin{aligned}\delta(\{p, q\}, 0) &= \delta(p, 0) \cup \delta(q, 0)\\ &= \{p, q\} \cup \{r, s\}\\ &= \{p, q, r, s\}\end{aligned}$$

$$\begin{aligned}\delta(\{p, q\}, 1) &= \delta(p, 1) \cup \delta(q, 1) = \{p\} \cup \{t\}\\ &= \{p, t\}\end{aligned}$$

Step 3 : Two new subsets {p, q, r, s} and {p, t} are generated. Their successors are calculated.

$$\begin{aligned}\delta(\{p, q, r, s\}, 0) &= \delta(p, 0) \cup \delta(q, 0) \cup \delta(r, 0) \cup \delta(s, 0)\\ &= \{p, q\} \cup \{r, s\} \cup \{p, r\} \cup \phi\\ &= \{p, q, r, s\}\end{aligned}$$

$$\begin{aligned}\delta(\{p, q, r, s\}, 1) &= \delta(p, 1) \cup \delta(q, 1) \cup \delta(r, 1) \cup \delta(s, 1)\\ &= \{q\} \cup \{t\} \cup \{t\} \cup \phi\\ &= \{p, t\}\end{aligned}$$

$$\begin{aligned}\delta(\{p, t\}, 0) &= \delta(p, 0) \cup \delta(t, 0)\\ &= \{p, q\} \cup \phi = \{p, q\}\end{aligned}$$

$$\begin{aligned}\delta(\{p, t\}, 1) &= \delta(p, 1) \cup \delta(t, 1)\\ &= \{p\} \cup \phi = \{p\}\end{aligned}$$

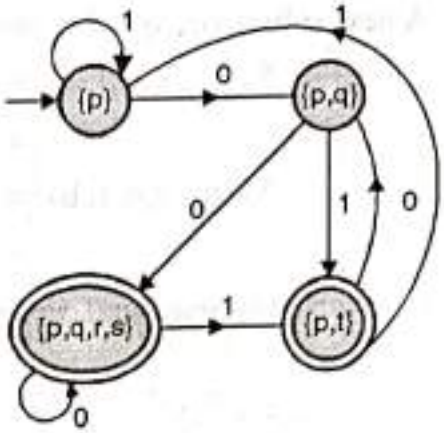
No, new subset is generated. Every subset containing either s or t is marked as a final state.

Informal Description: Strings over {0, 1} with second digit from the end is 0.

Scanned by CamScanner

	0	1
→{p}	{p, q}	{p}
{p, q}	{p, q, r, s}	{p, t}
{p, q, r, s}*	{p, q, r, s}	{p, t}
{p, t}*	{p, q}	{p}

(a) State table



(b) State diagram

Fig. Ex. 2.3.13 : Final DFA for Example 2.3.13

Example 2.3.14 : Construct a NFA that accepts a set of all strings over {a, b} ending in aba. Use this NFA to construct DFA accepting the same set of strings. **PU - May 2014**

Solution :

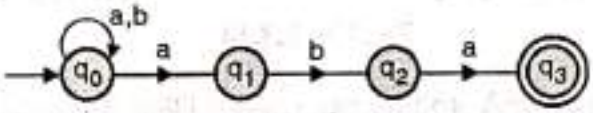


Fig. Ex. 2.3.14(a) : Non-deterministic finite automata for Example 2.3.14

Non-determinism should be utilized to full extent while designing an NFA. A string of length n, ending in aba can be recognized by the NFA given in Fig. Ex. 2.3.14(a). First n-3 characters can be absorbed by the state q_0 by making a guess. On guessing the last three characters as aba, the machine can make a transition from q_0 to q_3.

NFA to DFA conversion :

Step 1 : $\{q_0\}$ is taken as first subset

$$\text{a-successor of } \{q_0\} = \delta(q_0, a) = \{q_0, q_1\}$$

$$\text{b-successor of } \{q_0\} = \delta(q_0, b) = \{q_0\}$$

Step 2 : A new subset $\{q_0, q_1\}$ is generated. Successors of $\{q_0, q_1\}$ are calculated.

$$\delta(\{q_0, q_1\}, a) = \delta(q_0, a) \cup \delta(q_1, a)$$
$$= \{q_0, q_1\} \cup \phi = \{q_0, q_1\}$$
$$\delta(\{q_0, q_1\}, b) = \delta(q_0, b) \cup \delta(q_1, b)$$
$$= \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$$

Step 3 : A new subset $\{q_0, q_2\}$ is generated. Successors of $\{q_0, q_2\}$ are calculated.

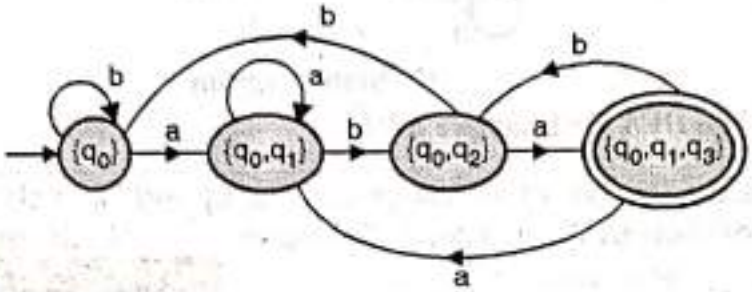
$$\delta(\{q_0, q_2\}, a) = \delta(q_0, a) \cup \delta(q_2, a)$$
$$= \{q_0, q_1\} \cup \{q_3\}$$
$$= \{q_0, q_1, q_3\}$$
$$\delta(\{q_0, q_2\}, b) = \delta(q_0, b) \cup \delta(q_2, b) = \{q_0\} \cup \phi = \{q_0\}$$

Scanned by CamScanner

Step 4 : A new subset $\{q_0, q_1q_3\}$ is generated. Successors of $\{q_0, q_1q_3\}$ are calculated.

$$\delta(\{q_0, q_1q_3\}, a) = \delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_3, a)$$
$$= \{q_0, q_1\} \cup \phi \cup \phi = \{q_0, q_1\}$$
$$\delta(\{q_0, q_1q_3\}, b) = \delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_3, b)$$
$$= \{q_0\} \cup \{q_2\} \cup \phi = \{q_0, q_2\}$$

No, new subset is generated. Every subset containing q_3 is marked as a final state.



(b) State diagram of the DFA

	a	b
$\rightarrow\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1, q_3\}$	$\{q_0\}$
$\{q_0, q_1, q_3\}^*$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

(c) State table of the DFA

Fig. Ex. 2.3.14

Example 2.3.15 : Construct a NFA and its equivalent DFA for accepting a language defined over $\Sigma = \{0,1\}$ such that each string has two consecutive zeroes followed by 1.

Solution :

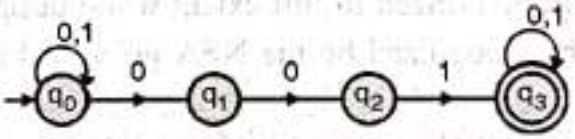


Fig. Ex. 2.3.15

A string of length n, having two consecutive zeroes followed by 1 can be recognized by the NFA. Symbols before 001 can be absorbed by the state q_0 by making a guess. On guessing 001, the machine can make a transition from q_0 to q_3.

NFA to DFA conversion :

	0	1	
$\rightarrow\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$	A new subset $\{q_0, q_1\}$ is generated
$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	$\{q_0\}$	A new subset $\{q_0, q_1, q_2\}$ is generated
$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_3\}$	A new subset $\{q_0, q_3\}$ is generated
$\{q_0, q_3\}^*$	$\{q_0, q_1, q_3\}$	$\{q_0, q_3\}$	A new subset $\{q_0, q_1, q_3\}$ is generated
$\{q_0, q_1, q_3\}^*$	$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_3\}$	A new subset $\{q_0, q_1, q_2, q_3\}$ is generated
$\{q_0, q_1, q_2, q_3\}^*$	$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_3\}$	

Transition behaviour of $\{q_0, q_3\}$, $\{q_0, q_1, q_3\}$ and $\{q_0, q_1, q_2, q_3\}$ is identical. Therefore these three states can be merged into a single state with resultant DFA as shown in Fig. Ex. 2.3.15(a) :

Scanned by CamScanner

		0	1
$\rightarrow\{q_0\}$	as $\rightarrow$A	B	A
$\{q_0, q_1\}$	as B	C	A
$\{q_0, q_1, q_2\}$	as C	C	D
$\{q_0, q_3\}$	as D*	D	D

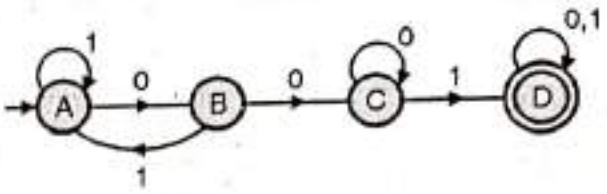


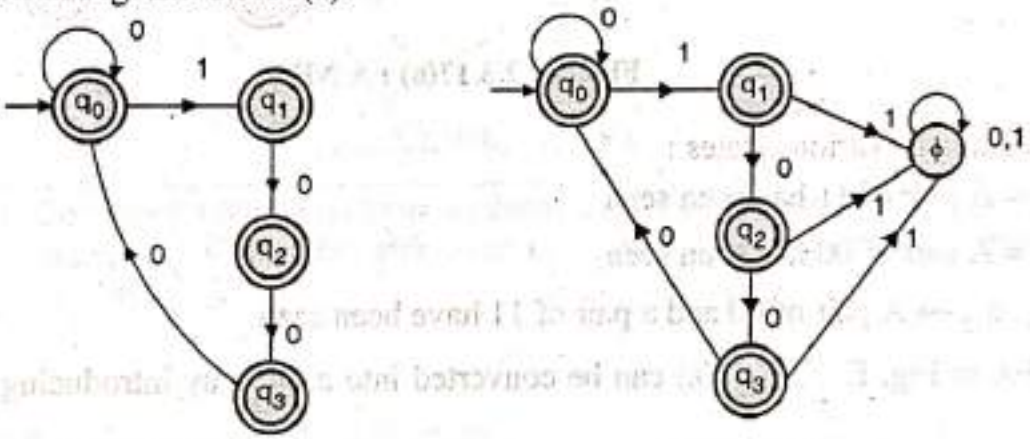
Fig. Ex. 2.3.15(a)

Example 2.3.16 : Construct an NFA an then equivalent DFA accepting strings over {0, 1}, whose every block of 4 consecutive symbols, contain at least 3 zeros.

Solution :

This problem is fully deterministic in nature. Technically, a DFA with ϕ transitions not mentioned can be called a NFA.

NFA is given in Fig. Ex. 2.3.16(a).



(a) A NFA (b) A DFA

Fig. Ex. 2.3.16

From Fig. Ex. 2.3.16(a), it is clear that between any two 1's there will be at least three 0's. Thus every block of 4 consecutive symbols will contain at least three 0's. Every state is a final state as the failure cases have not been shown.

Above NFA can be converted into a DFA by introducing the dead state. DFA is given in Fig. Ex. 2.3.16(b).

Example 2.3.17 : Construct a NFA and then equivalent DFA accepting strings over {0, 1}, which accepts the set of all strings of zeros (i.e. 0's) and ones (i.e. 1's) with at most one pair of consecutive zeros and at most one pair of consecutive ones (i.e.1's).

Solution :

This problem is fully deterministic in nature. Technically, a DFA with ϕ transitions not mentioned can be called as NFA.

Scanned by CamScanner

NFA is given in Fig. Ex. 2.3.17(a).

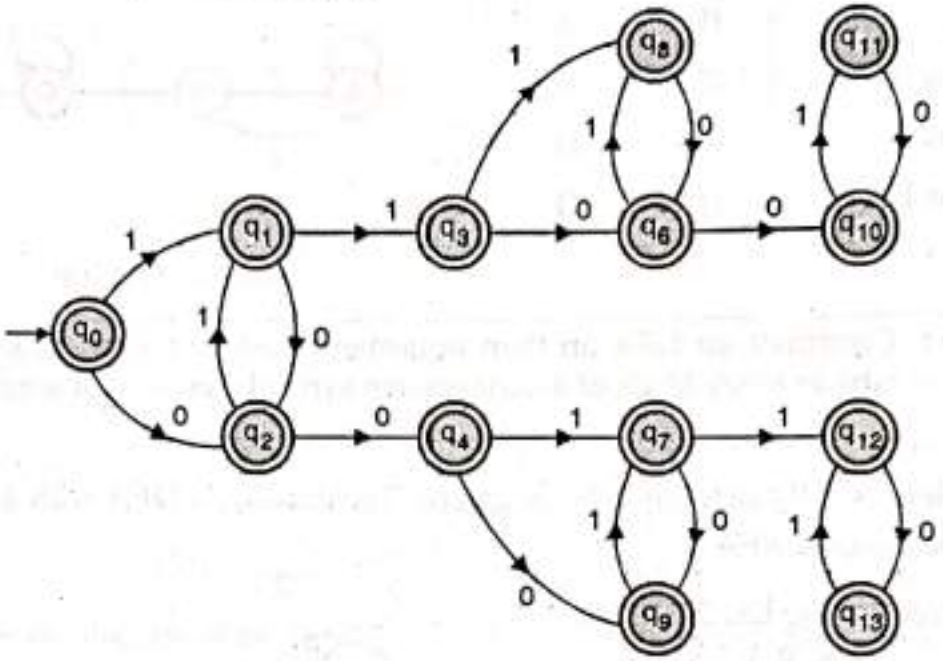


Fig. Ex. 2.3.17(a) : A NFA

Meaning of various states :

q_3 = A pair of 11 has been seen

q_4 = A pair of 00 has been seen

q_{10}, q_{12} → A pair of 00 and a pair of 11 have been seen

NFA in Fig. Ex. 2.3.17(a) can be converted into a DFA by introducing an explicit dead state.

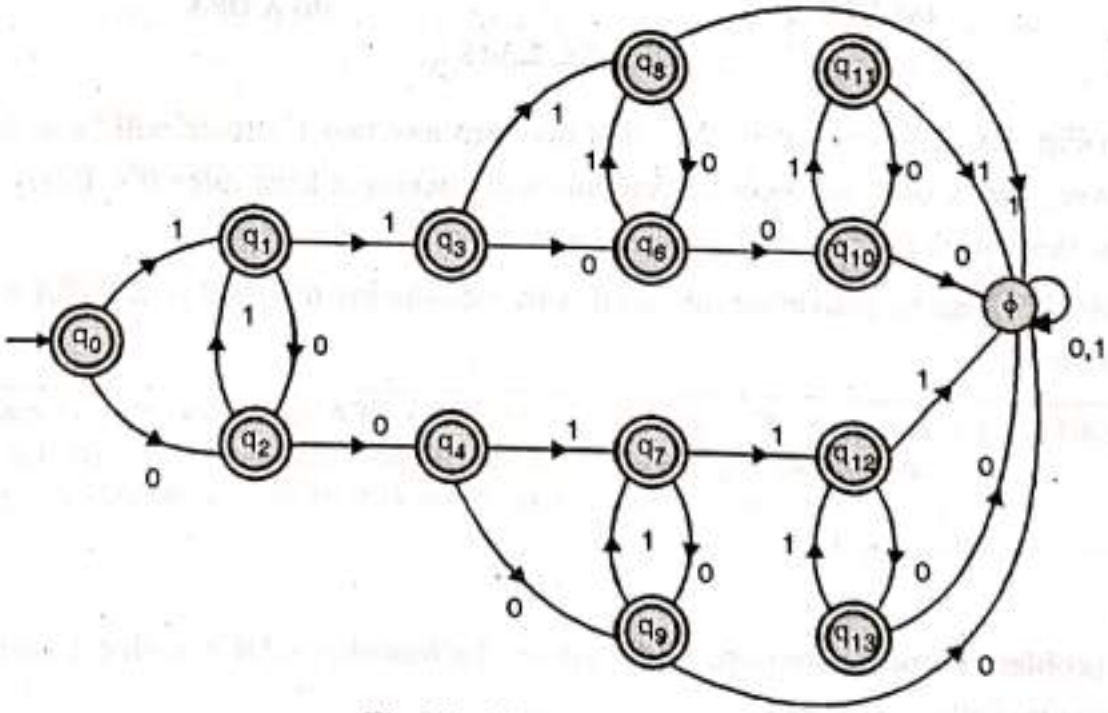


Fig. Ex. 2.3.17(b) : A DFA

Scanned by CamScanner

Example 2.3.18 : Construct an NFA and then equivalent DFA accepting string over {0, 1}, for accepting all possible strings of zero and ones not containing 101 as substring.

Solution : This problem is fully deterministic in nature as technically, a DFA with ϕ transitions not mentioned can be called as NFA.

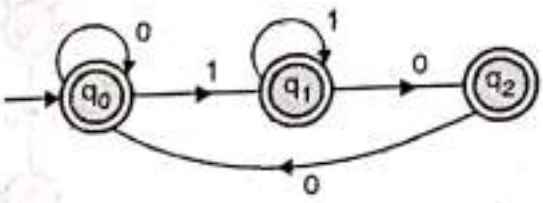


Fig. Ex. 2.3.18(a) : A NFA

- State q_1 is for previous character as 1.
- State q_2 is for previous two characters as 10.
- An input of 1 in state q_2 will cause the machine to enter a failure state.
- NFA can be converted into a DFA by introducing an explicit failure state. The equivalent DFA is given in Fig. Ex. 2.3.18(b).

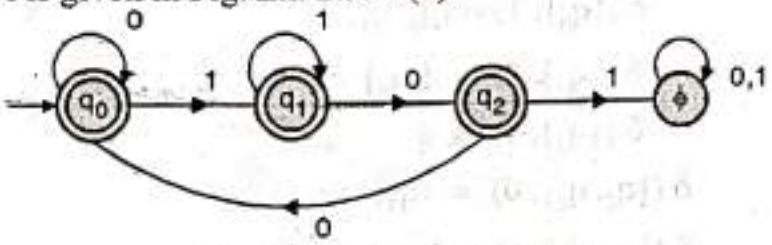


Fig. Ex. 2.3.18(b) : A DFA

Example 2.3.19 : Construct a NFA and then equivalent DFA accepting strings over {0, 1}, for accepting all possible strings of zeros and ones which do not contain 011 as substring.

Solution :

This problem is fully deterministic in nature. Technically, a DFA with ϕ transitions not mentioned can be called as NFA.

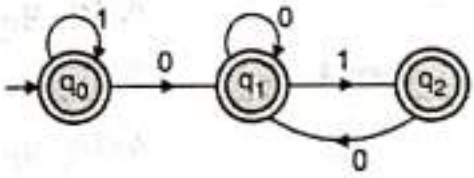


Fig. Ex. 2.3.19(a) : A NFA

- State q_1 is for previous character as 0.
- State q_2, is for previous two characters as 01.
- An input of 1 in state q_2 will cause the machine to enter a failure state.
- NFA can be converted into a DFA by introducing an explicit failure state.

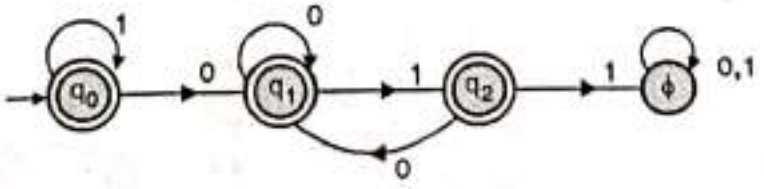


Fig. Ex. 2.3.19(b) : An equivalent DFA

Scanned by CamScanner

Example 2.3.20 : Construct a DFA equivalent to the NDFA shown in Fig. Ex. 2.3.20 :

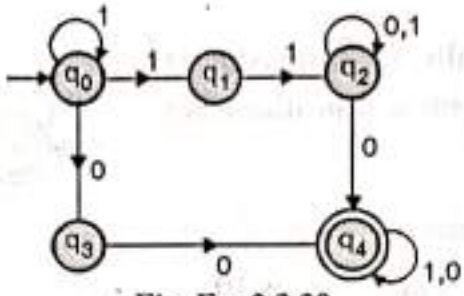


Fig. Ex. 2.3.20

Solution : Construction of subsets :

Step 1 : $\delta(\{q_0\}, 0) = \{q_3\}$

$\delta(\{q_0\}, 1) = \{q_0, q_1\}$

Step 2 : $\delta(\{q_3\}, 0) = \{q_4\}$

$\delta(\{q_3\}, 1) = \phi$

$\delta(\{q_0, q_1\}, 0) = \{q_3\}$

$\delta(\{q_0, q_1\}, 1) = \{q_0, q_1, q_2\}$

Step 3 : $\delta(\{q_4\}, 1) = \{q_4\}$

$\delta(\{q_4\}, 1) = \{q_4\}$

$\delta(\{q_0, q_1, q_2\}, 0) = \{q_3, q_2, q_4\} = \{q_2, q_3, q_4\}$

$\delta(\{q_0, q_1, q_2\}, 1) = \{q_0, q_1, q_2\}$

Step 4 : $\delta(\{q_2, q_3, q_4\}, 0) = \{q_2, q_4\}$

$\delta(\{q_2, q_3, q_4\}, 1) = \{q_2, q_4\}$

Step 5 : $\delta(\{q_2, q_4\}, 0) = \{q_2, q_4\}$

$\delta(\{q_2, q_4\}, 1) = \{q_2, q_4\}$

DFA is given in Fig. Ex. 2.3.20(a).

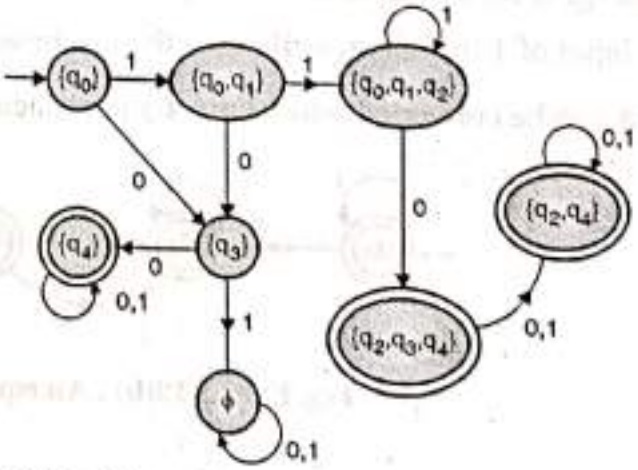


Fig. Ex. 2.3.20(a) : State diagram of the DFA for Example 2.3.20

Scanned by CamScanner

Example 2.3.21 : Let m = ({q_1, q_3, q_4}, {0, 1}, δ, { q_1}, { q_3}) is NFA where δ is given as :

$\delta(q_1, 0) = \{q_2, q_3\}$

$\delta(q_1, 1) = \{q_1\}$

$\delta(q_2, 0) = \{q_1, q_2\}$

$\delta(q_2, 1) = \{q_1, q_2\}$

$\delta(q_3, 0) = \{q_2\}$

$\delta(q_3, 1) = \{q_1, q_2\}$

Construct the transition diagram corresponding to NFA and find and draw its equivalent DFA, show all intermediate steps also.

Solution :

Transition diagram is given in Fig. Ex. 2.3.21(a).

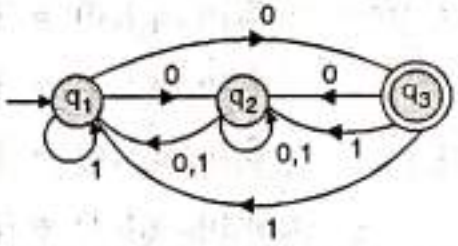


Fig. Ex. 2.3.21(a) : Transition of the given NFA

Subset construction :

Step 1 : $\delta(\{q_1\}, 0) = \{q_2, q_3\}$

$\delta(\{q_1\}, 1) = \{q_1\}$

Step 2 : $\delta(\{q_2, q_3\}, 0) = \{q_1, q_2\}$

$\delta(\{q_2, q_3\}, 1) = \{q_1, q_2\}$

Step 3 : $\delta(\{q_1, q_2\}, 0) = \{q_1, q_2, q_3\}$

$\delta(\{q_1, q_2\}, 1) = \{q_1, q_2\}$

Step 4 : $\delta(\{q_1, q_2, q_3\}, 0) = \{q_1, q_2, q_3\}$

$\delta(\{q_1, q_2, q_3\}, 1) = \{q_1, q_2\}$

Equivalent DFA is given in Fig. Ex. 2.3.21(b).

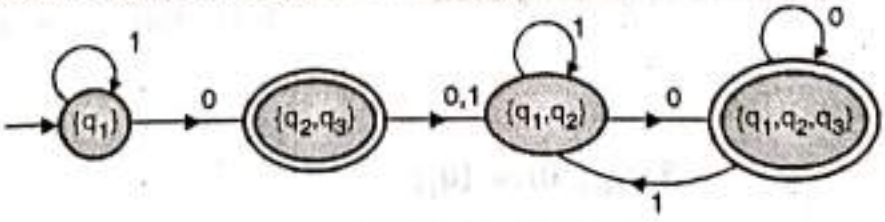


Fig. Ex. 2.3.21(b) : Equivalent DFA

Scanned by CamScanner

Example 2.3.22 : M ($\{q_1, q_2, q_3\}$, $\{0, 1\}$, δ, q_1, $\{q_3\}$) is a non-deterministic finite automaton where δ is given by.

$\delta(q_1, 0) = \{q_2, q_3\}$ $\quad$ $\delta(q_1, 1) = \{q_1\}$

$\delta(q_2, 0) = \{q_1, q_2\}$ $\quad$ $\delta(q_2, 0) = \phi$

$\delta(q_3, 0) = \{q_2\}$ $\quad$ $\delta(q_3, 1) = \{q_1, q_2\}$

Construct an equivalent DFA.

Solution :

Subset construction :

Step 1 : $\delta(\{q_1\}, 0) = \{q_2, q_3\}$

$\delta(\{q_1\}, 1) = \{q_1\}$

Step 2 : $\delta(\{q_2, q_3\}, 0) = \{q_1, q_2\}$

$\delta(\{q_2, q_3\}, 1) = \{q_1, q_2\}$

Step 3 : $\delta(\{q_1, q_2\}, 0) = \{q_1, q_2, q_3\}$

$\delta(\{q_1, q_2\}, 1) = \{q_1\}$

$\delta(\{q_1, q_2, q_3\}, 0) = \{q_1, q_2, q_3\}$

$\delta(\{q_1, q_2, q_3\}, 1) = \{q_1, q_2\}$

DFA is given in Fig. Ex. 2.3.22.

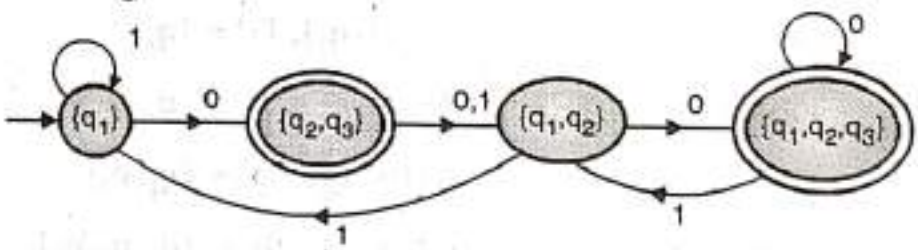


Fig. Ex. 2.3.22 : Equivalent DFA

Example 2.3.23 : Let M = ($\{q_0, q_1\}$, $\{0, 1\}$, δ, q_0, $\{q_1\}$) be an NFA with :

$\delta(q_0, 0) = \{q_1\}$

$\delta(q_0, 1) = \{q_0, q_1\}$

$\delta(q_1, 1) = \{q_0, q_1\}$

Find the corresponding DFA.

Solution :

Subset construction :

Step 1 : $\delta(\{q_0\}, 0) = \{q_1\}$

$\delta(\{q_0\}, 1) = \{q_0, q_1\}$

Scanned by CamScanner

Step 2 : $\delta(\{q_1\}, 0) = \phi$

$\delta(\{q_1\}, 1) = \{q_0, q_1\}$

Step 3 : $\delta(\{q_0, q_1\}\ 0) = \{q_1\}$

$\delta(\{q_0, q_1\}, 1) = \{q_0, q_1\}$

An equivalent DFA is given in Fig. Ex. 2.3.23.

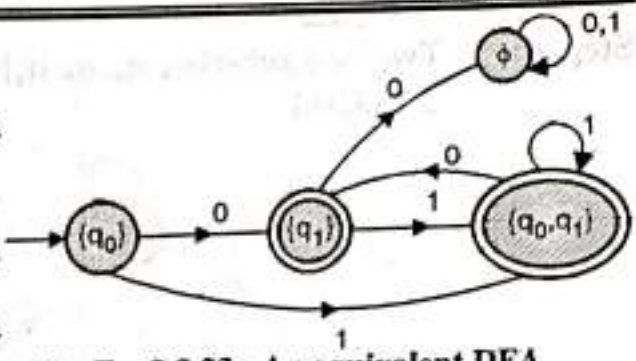


Fig. Ex. 2.3.23 : An equivalent DFA

Example 2.3.24 : Convert NFA shown in Fig. Ex. 2.3.24 to its corresponding DFA.

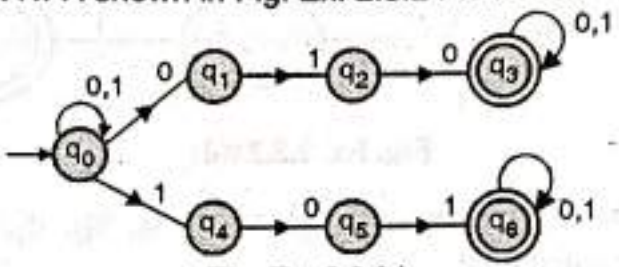


Fig. Ex. 2.3.24

Solution :

Step 1 : q_3 and q_6 have identical NFA and they can be merged into a single state q_3.

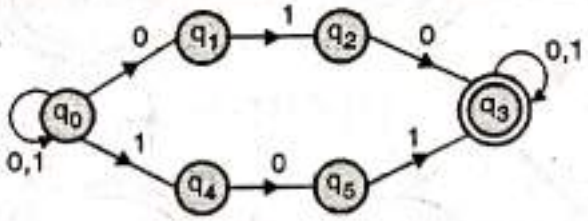


Fig. Ex. 2.3.24(a)

Step 2 :

We start with the subset $\{q_0\}$

0-successor of $\{q_0\} = \{q_0, q_1\}$

1-successor of $\{q_1\} = \{q_0, q_4\}$

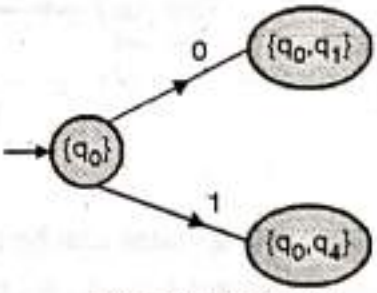


Fig. Ex. 2.3.24(b)

Step 3 : Two new subsets $\{q_0, q_1\}$ and $\{q_0, q_4\}$ are generated. Their successors are calculated.

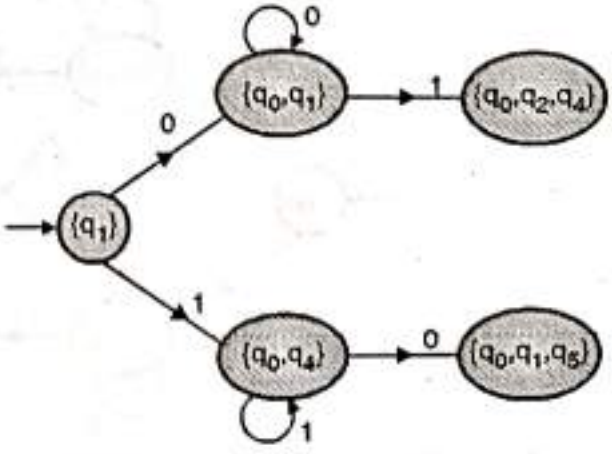


Fig. Ex. 2.3.24(c)

Scanned by CamScanner

Step 4 : Two new subsets { q_0, q_2, q_4} and { q_0, q_1, q_5} are generated. Their successors are calculated.

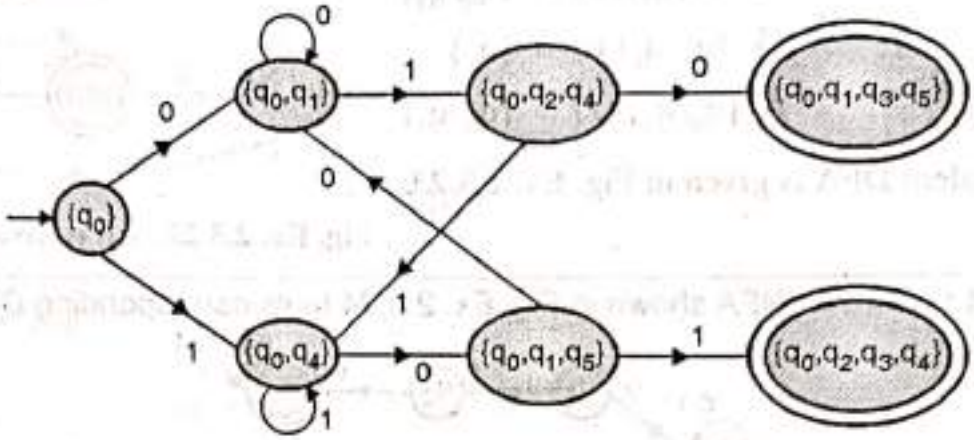


Fig. Ex. 2.3.24(d)

Step 5 : Two new subsets { q_0, q_1, q_3, q_5} and { q_0, q_2, q_3, q_4} are generated. Their successors are calculated.

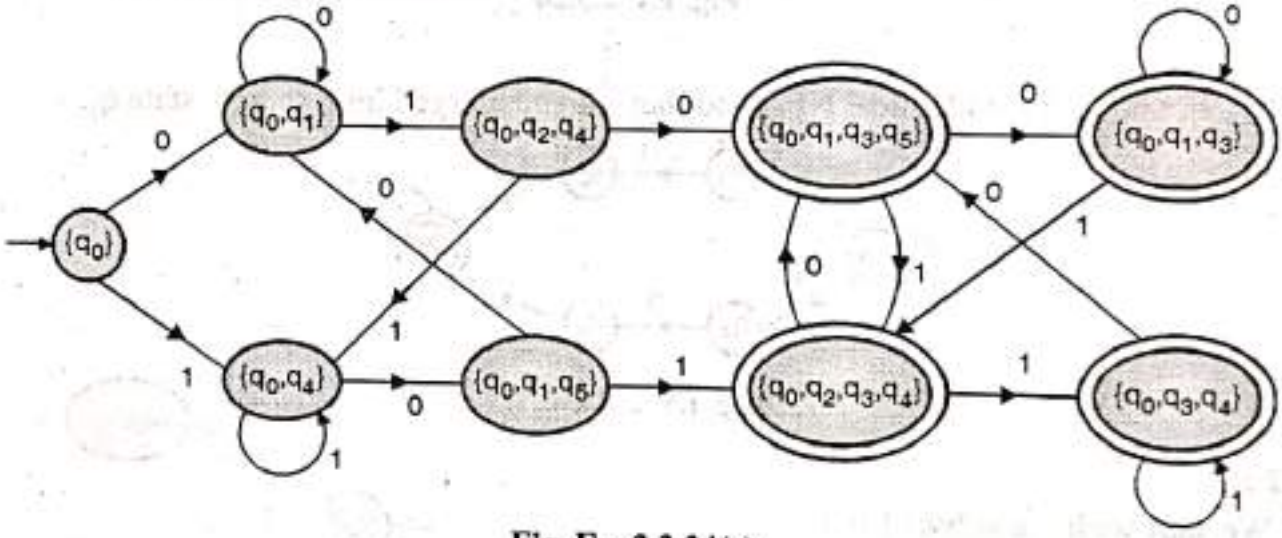


Fig. Ex. 2.3.24(e)

Step 6 : Following states can be combined into a single state.

These states are { q_0, q_1, q_3, q_5}, { q_0, q_1, q_3}, { q_0, q_2, q_3, q_4}, { q_0, q_3, q_4}

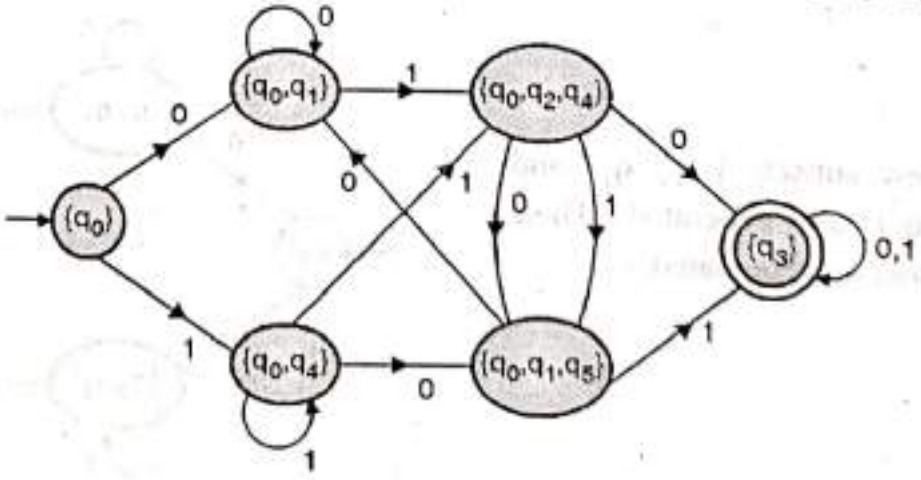


Fig. Ex. 2.3.24(f) : Final DFA

Scanned by CamScanner

Example 2.3.25 : Convert NFA shown in Fig. Ex. 2.3.25 to its corresponding DFA.

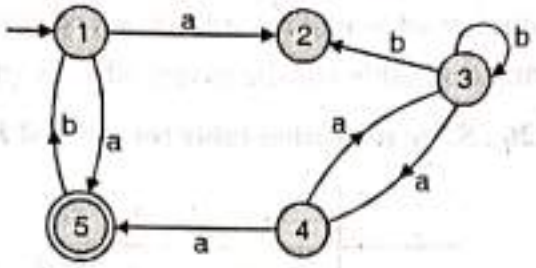


Fig. Ex. 2.3.25

Solution :

Step 1 : Starting state {1} is taken as the first subset. Its successors are calculated.

$$\delta(\{1\}, a) = \{2, 5\}$$

$$\delta(\{1\}, b) = \phi$$

Step 2 : A new subset {2, 5} is generated. Its successors are calculated.

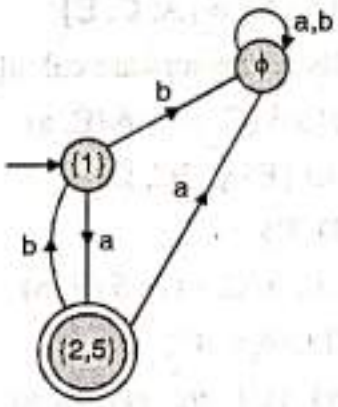
$$\delta(\{2, 5\}, a) = \delta(2, a) \cup \delta(5, a)$$

$$= \phi \cup \phi = \phi$$

$$\delta(\{2, 5\}, b) = \delta(2, b) \cup \delta(5, b)$$

$$= \phi \cup \{1\} = \{1\}$$

A subset containing 5 is marked as a final state. The equivalent DFA is given in Fig. Ex. 2.3.25(a).



	a	b
→{1}	{2, 5}	ϕ
{2, 5}*	ϕ	{1}
ϕ	ϕ	ϕ

Fig. Ex. 2.3.25(a) : DFA for Example 2.3.25

Example 2.3.26 : Convert the NFA shown in Fig. Ex. 2.3.26 to DFA.

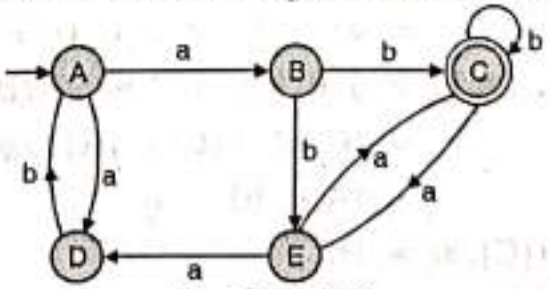


Fig. Ex. 2.3.26

Scanned by CamScanner

Solution :

In some cases where number of states is large, it is convenient to derive subsets using a state transition table. State transition table for the given NFA is shown in Table Ex. 2.3.26(a).

Table Ex. 2.3.26 : State transition table for NFA of Example 2.3.26

	a	b
→A	{B, D}	ϕ
B	ϕ	{C, E}
C*	{E}	{C}
D	ϕ	{A}
E	{C, D}	ϕ

Step 1 : Starting state {A} is taken as the first subset. Its successors are calculated.

$$\delta(\{A\}, a) = \{B, D\}$$
$$\delta(\{A\}, b) = \phi$$

Step 2 : A new subset {B, D} is generated. Its successors are calculated

$$\delta(\{B, D\}, a) = \delta(B, a) \cup \delta(D, a)$$
$$= \phi \cup \phi = \phi$$
$$\delta(\{B, D\}, b) = \delta(B, b) \cup \delta(D, b)$$
$$= \{C, E\} \cup \{A\} = \{A, C, E\}$$

Step 3 : A new subset {A, C, E} is generated. Its successors are calculated.

$$\delta(\{A, C, E\}, a) = \delta(A, a) \cup \delta(C, a) \cup \delta(E, a)$$
$$= \{B, D\} \cup \{E\} \cup \{C, D\}$$
$$= \{B, C, D, E\}$$
$$\delta(\{A, C, E\}, b) = \delta(A, b) \cup \delta(C, b) \cup \delta(E, b)$$
$$= \phi \cup \{C\} \cup \phi = \{C\}$$

Step 4 : Two new subsets {B, C, D, E} and {C} are generated. Their successors are calculated.

$$\delta(\{B, C, D, E\}, a) = \delta(B, a) \cup \delta(C, a) \cup \delta(D, a) \cup \delta(E, a)$$
$$= \phi \cup \{E\} \cup \phi \cup \{C, D\} = \{C, D, E\}$$
$$\delta(\{B, C, D, E\}, b) = \delta(B, b) \cup \delta(C, b) \cup \delta(D, b) \cup \delta(E, b)$$
$$= \{C, E\} \cup \{C\} \cup \{A\} \cup \phi$$
$$= \{A, C, E\}$$
$$\delta(\{C\}, a) = \{E\}$$
$$\delta(\{C\}, b) = \{C\}$$

Scanned by CamScanner

Step 5 : Two new subsets {C, D, E} and {E} are generated. Their successors are calculated.

$$\delta(\{C, D, E\}, a) = \{E\} \cup \phi \cup \{C, D\} = \{C, D, E\}$$
$$\delta(\{C, D, E\}, b) = \{C\} \cup \{A\} \cup \phi$$
$$= \{A, C\}$$
$$\delta(\{E\}, a) = \{C, D\}$$
$$\delta(\{E\}, b) = \phi$$

Step 6 : Two new subsets {A, C} and {C, D} are generated. Their successors are calculated.

$$\delta(\{C, D\}, a) = \{E\} \cup \phi = \{E\}$$
$$\delta(\{C, D\}, b) = \{C\} \cup \{A\} = \{A, C\}$$
$$\delta(\{A, C\}, a) = \{B, D\} \cup \{E\} = \{B, D, E\}$$
$$\delta(\{A, C\}, b) = \phi \cup \{C\} = \{C\}$$

Step 7 : A new subset {B, D, E} is generated. Its successors are calculated.

$$\delta(\{B, D, E\}, a) = \phi \cup \phi \cup \{C, D\} = \{C, D\}$$
$$\delta(\{B, D, E\}, b) = \{C, E\} \cup \{A\} \cup \phi$$
$$= \{A, C, E\}$$

Every subset containing C is marked as final state. Final DFA is given in Fig. Ex. 2.3.26(a).

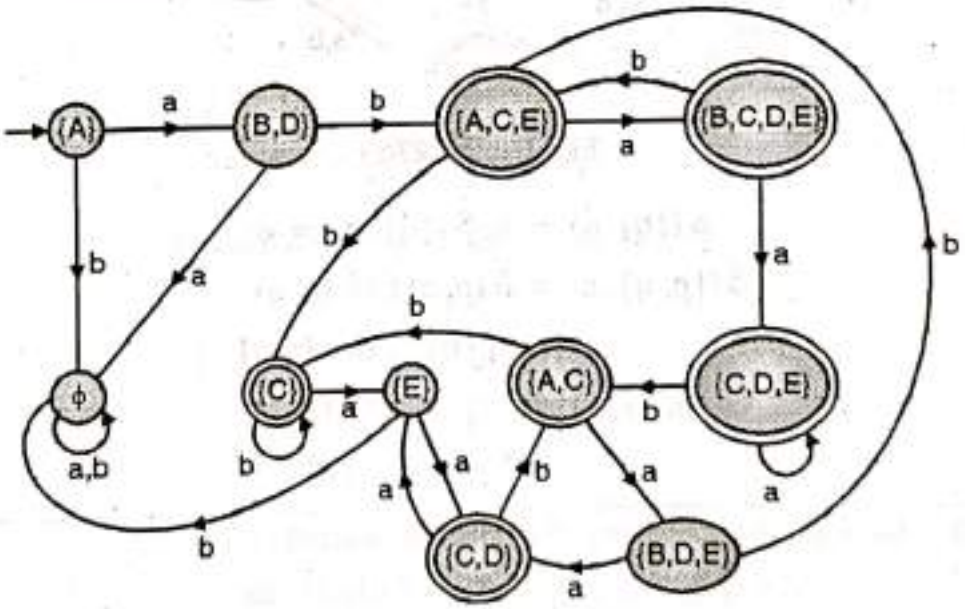


Fig. Ex. 2.3.26(a) : Final DFA for Example 2.3.26

Example 2.3.27 Convert the following NFA to its equivalent DFA.

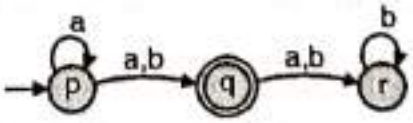


Fig. Ex. 2.3.27

Scanned by CamScanner

Solution :

The state r is is a dead state and it can be removed.

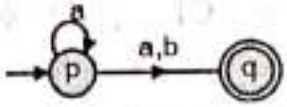


Fig. Ex. 2.3.27(a)

NFA to DFA :

Step 1 : Writing transition from the state p.

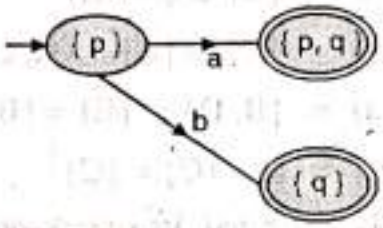


Fig. Ex. 2.3.27(b)

Step 2 : Writing transitions from the states {p, q} and {q}.

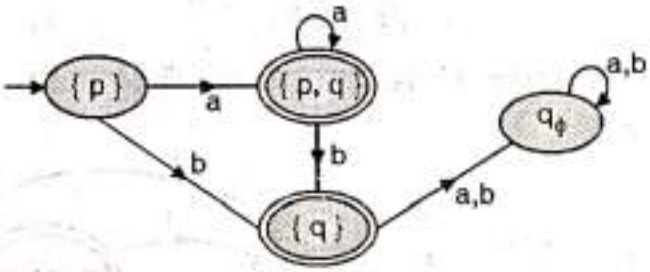


Fig. Ex. 2.3.27(c)

$$\delta(\{q\}, a) = \phi, \delta(\{q\}, b) = \phi$$

$$\delta(\{p, q\}, a) = \delta(p, a) \cup \delta(q, a)$$

$$= \{p, q\} \cup \phi = \{p,q\}$$

$$\delta(\{p, q\}, b) = \delta(p, b) \cup \delta(q, b)$$

$$= \{q\} \cup \phi = \{q\}$$

Example 2.3.28 : Convert the following NFA into equivalent DFA

$M = (\{ q_0, q_1\}, \{ 0, 1\}, \delta, q_0, \{q_1\})$ where

$\delta(q_0, 0) = \{q_0, q_1\}$, $\delta(q_0, 1) = \{q_1\}$, $\delta(q_1, 0) = \phi$

$\delta(q_1, 1) = \{q_0, q_1\}$

Solution : The state transition table for the above NFA is as given below :

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_1\}$
q_1^*	–	$\{q_0, q_1\}$

Scanned by CamScanner

Step 1 : Transitions from the starting state q_0.

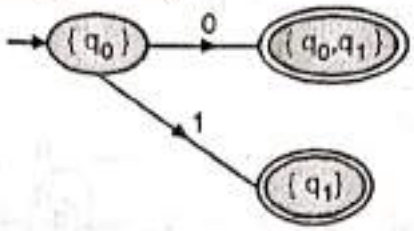


Fig. Ex. 2.3.28

Step 2 : Transitions from q_1.

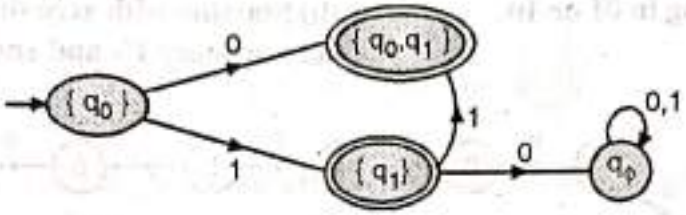


Fig. Ex. 2.3.28(a)

Step 3 : Transitions from $\{q_0, q_1\}$

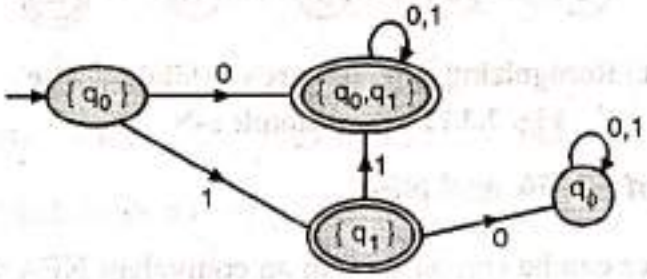


Fig. Ex. 2.3.28(b)

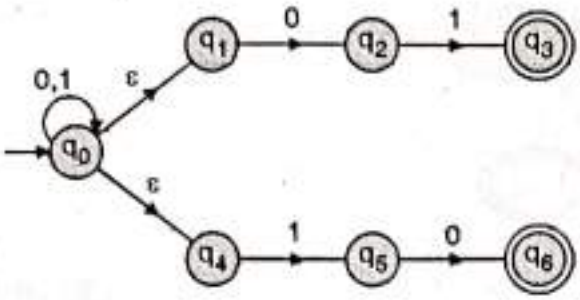
$$\delta(\{q_0, q_1\}, 0) = \delta(q_0, 0) \cup \delta(q_1, 0)$$
$$= \{q_0, q_1\} \cup \phi = \{q_0, q_1\}$$
$$\delta(\{q_0, q_1\}, 1) = \delta(q_0, 1) \cup \delta(q_1, 1)$$
$$= \{q_1\} \cup \{q_0, q_1\} = \{q_0, q_1\}$$

2.3.4 NFA with ε-Transitions

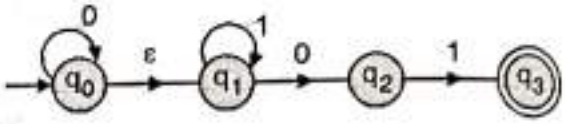
ε stands for a null symbol. A ε transition allows transition on ε (or no input), the empty string. This implies that a machine can make a transition without any input.

- When finding the string described by a path containing arc with label ε, the ε symbols are discarded.
- The use of ε-transition may simplify a transition graph by reducing number of labelled arcs.

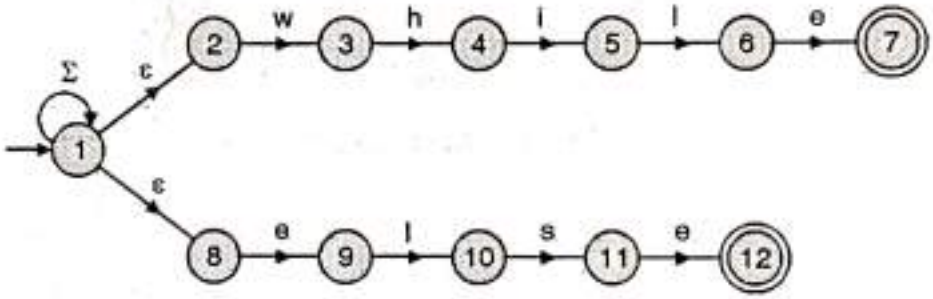
Scanned by CamScanner



(a) Strings ending in 01 or 10



(b) Starting with zero or more 0's followed by zero or more 1's and ending in 01



(c) Recognizing reverse words 'while' or 'else'

Fig. 2.3.12 : Few sample ε-NFAs

2.3.4.1 Equivalence of ε-NFA and NFA

An NFA with ε-move can be converted into an equivalent NFA without ε-move. We can always find the equivalence between systems with ε-move and without ε-move. This can be understood with the help of an example.

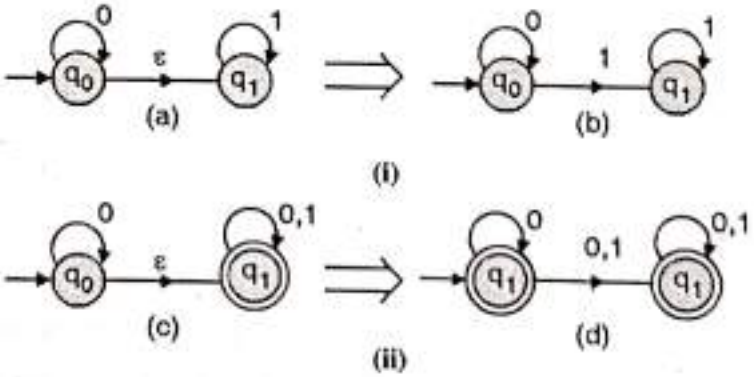


Fig. 2.3.13 : NFA with ε-move converted to NFA without ε-move

If there is an ε-move between any two states q_i and q_j, then ε-move can be removed as follows :

(1) All moves from q_j should also originate from q_i. For example : If there is move from q_j to q_k , then we must add a move on q_i to q_k.

(2) If q_j is a final state then make q_i as a final state.

Scanned by CamScanner

Example 2.3.29 : Use direct method to find an equivalent NFA without ε-move for the NFA given below :

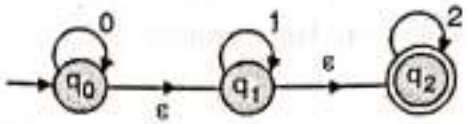


Fig. Ex. 2.3.29 : NFA with ε-move

Solution :

Step 1 : Transitions from q_2 are also duplicated on q_1. q_1 is made a final state.

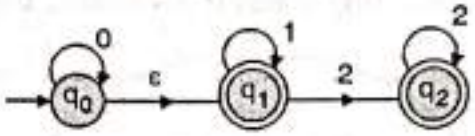


Fig. Ex. 2.3.29(a) : ε-move between q_1 and q_2 is removed

Step 2 : Transition from q_1 are also duplicated on q_0. q_0 is made a final state.

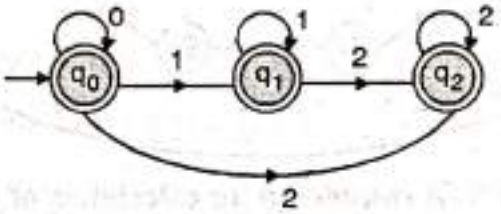


Fig. Ex. 2.3.29(b) : Final NFA without ε-move after removing ε-move between q_0 and q_1

2.3.4.2 The Formal Notation for an ε-NFA

A non deterministic finite automaton M with ε-transition is given by :

Where, Q = A finite set of states

Σ = is an alphabet

q_0 = $q_0 \in Q$ is the initial/start state.

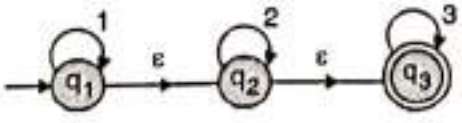
F = $F \subseteq Q$ is a set of final/accepting states

δ = A transition function from $Q \times (\Sigma \cup \{\varepsilon\})$ to the power set of Q i.e. to 2^Q.

The transition function δ also includes transitions on ε. The NFA given in Fig. 2.3.14(a) can be formally represented as :

$$M = (\{q_1, q_2, q_3\}, \{1, 2, 3\}, \delta, q_1, \{q_3\})$$

where δ is defined by the transition table/diagram in Fig. 2.3.14(b).



(a)

	ε	1	2	3
$\rightarrow q_1$	$\{q_2\}$	$\{q_1\}$	ϕ	ϕ
q_2	$\{q_3\}$	ϕ	$\{q_2\}$	ϕ
q_3^*	ϕ	ϕ	ϕ	$\{q_3\}$

(b)

Fig. 2.3.14 : An NFA used for formal notation

Scanned by CamScanner

2.3.4.3 ε-Closures

ε-closures of a state q_i is the set of states including q_i where q_i can reach by any number of ε-moves of the given non-deterministic finite automata.

ε-closure of q_i :

- ε-closure of a state q_i, includes q_i.
- Set of states reachable from q_i on ε-move.
- Set of states reachable from existing states in ε-closure, using ε-move, and so on.

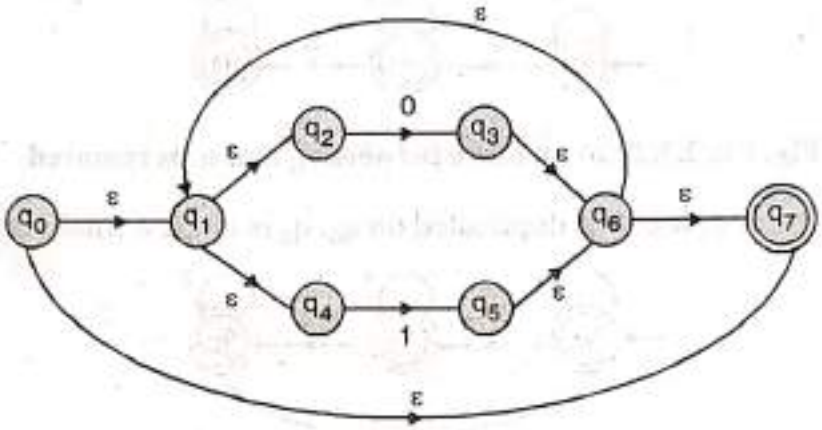


Fig. 2.3.15 : NFA considered for calculation of ε-closures

ε-closures of various states in Fig. 2.3.15 are given below :

ε-closure of $q_0 = \{q_0, q_1, q_2, q_4, q_7\}$

There are ε-moves from q_0 to q_1,
q_0 to q_7,
q_1 to q_2,
q_1 to q_4.

ε-closures of $q_1 = \{q_1, q_2, q_4\}$

There are ε-moves from q_1 to q_4,
q_1 to q_2.

ε-closure of $q_2 = \{q_2\}$

There is no ε-move from q_2.

ε-closure of $q_3 = \{q_3, q_6, q_7, q_1, q_2, q_4\}$

There are ε-moves from q_3 to q_6,
q_6 to q_7,
q_6 to q_1,
q_1 to q_2,
q_1 to q_4.

ε-closure of $q_4 = \{q_4\}$

Scanned by CamScanner

There is no ε-move from q_4.

ε-closure of $q_5 = \{q_5, q_6, q_7, q_1, q_2, q_4\}$

There are ε-moves from q_5 to q_6,

q_6 to q_7,

q_6 to q_1,

q_1 to q_2,

q_1 to q_4.

ε-closure of $q_6 = \{q_7, q_1, q_2, q_4\}$

There are ε-moves from q_6 to q_7,

q_6 to q_1,

q_1 to q_2,

q_1 to q_4.

ε-closure of $q_7 = \{q_7\}$

There is no ε-move from q_7.

ε-closure of various states are summarised below :

State		ε-closure
q_0	—	$\{q_0, q_1, q_2, q_4, q_7\}$
q_1	—	$\{q_1, q_2, q_4\}$
q_2	—	$\{q_2\}$
q_3	—	$\{q_3, q_6, q_7, q_1, q_2, q_4\}$
q_4	—	$\{q_4\}$
q_5	—	$\{q_5, q_6, q_7, q_1, q_2, q_4\}$
q_6	—	$\{q_7, q_1, q_2, q_4\}$
q_7	—	$\{q_7\}$

Example 2.3.30 : A transition table is given for another with NULL with seven states.

q	δ(q, a)	δ(q, b)	δ(q, ϕ)
1	{5}	ϕ	{4}
2	{1}	ϕ	ϕ
3	{ϕ}	{2}	ϕ
4	ϕ	{7}	{3}
5	ϕ	ϕ	{1}
6	ϕ	{5}	{4}
7	{6}	ϕ	ϕ

(a) Draw a transition diagram. (b) Calculate δ*(1, ba)

Scanned by CamScanner

Solution :

(a) Transition diagram is in Fig. Ex. 2.3.30(a).

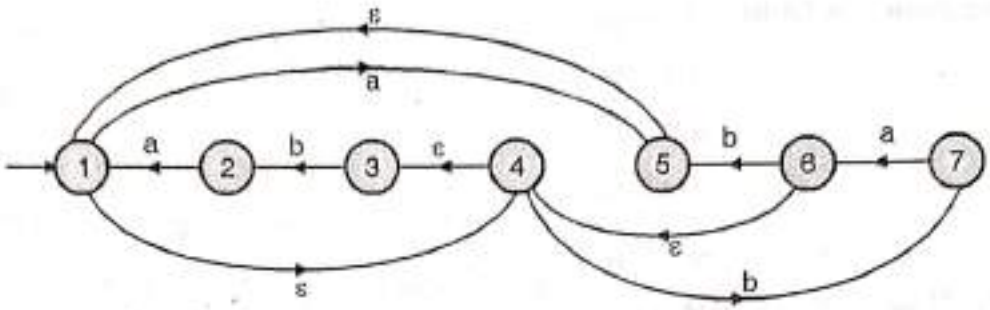


Fig. Ex. 2.3.30(a) : State transition diagram

(b) Calculate δ*(1,ba)

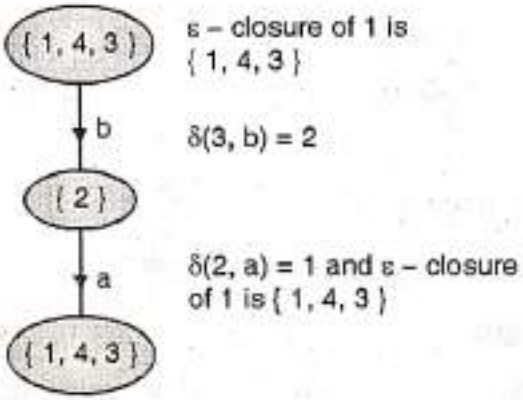


Fig. Ex. 2.3.30(b)

$\therefore \delta^*(1,ba) = \{1,4,3\}$

2.3.4.4 ε-NFA to DFA

ε-NFA to DFA conversion is also based on subset construction as in case of NFA to DFA. ε-closure is calculated for every state in the subset. That is the subset is extended by ε-closure of every state in the subset.

Let us compute the extended transition (δ_ε) for state q_2 on input 0 for the ε-NFA in Fig. 2.3.15.

$$\delta_\varepsilon(q_2,0) = \varepsilon\text{-closure}(q_3) = \{q_3, q_6, q_7, q_1, q_2, q_4\}$$

Example 2.3.31 : Find an equivalent DFA for the ε-NFA given in Fig. Ex. 2.3.31.

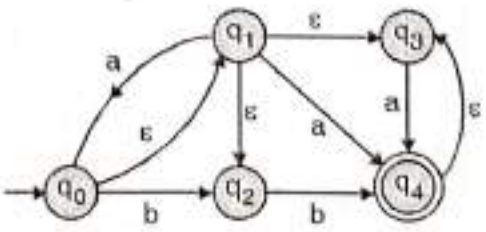


Fig. Ex. 2.3.31

Scanned by CamScanner

Solution :

Step 1 : ε-closure of states.

State	ε-closure	Comment
q_0	$\{q_0, q_1, q_2, q_3\}$	There are ε-moves from q_0 to q_1, q_1 to q_2, q_1 to q_3.
q_1	$\{q_1, q_2, q_3\}$	There are ε-moves from q_1 to q_2, q_1 to q_3
q_2	$\{q_2\}$	There is no ε-move from q_2.
q_3	$\{q_3\}$	There is no ε-move from q_3.
q_4	$\{q_4, q_3\}$	There is an ε-move from q_4 to q_3.

Step 2 : ε-closure of the initial state q_0 is taken as the first subset.

$$\varepsilon\text{-closure}(q_0) = \{q_0, q_1, q_2, q_3\}$$

$$\text{a-successor of } \{q_0, q_1, q_2, q_3\} = \varepsilon\text{-closure}(\delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a) \cup \delta(q_3, a))$$
$$= \varepsilon\text{-closure}(\phi \cup \{q_0, q_4\} \cup \phi \cup \{q_4\})$$
$$= \varepsilon\text{-closure}(\{q_0, q_4\}) = \varepsilon\text{-closure}\{q_0\} \cup \varepsilon\text{-closure}(q_4)$$
$$= \{q_0, q_1, q_2, q_3\} \cup \{q_3, q_4\} = \{q_0, q_1, q_2, q_3, q_4\}$$

$$\text{b-successor of } \{q_0, q_1, q_2, q_3\} = \varepsilon\text{-closure}(\delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b) \cup \delta(q_3, b))$$
$$= \varepsilon\text{-closure}(\{q_2\} \cup \phi\ \{q_4\} \cup \phi)$$
$$= \varepsilon\text{-closure}(\{q_2, q_4\}) = \varepsilon\text{-closure}(q_2) \cup \varepsilon\text{-closure}(q_4)$$
$$= \{q_2\} \cup \{q_3, q_4\} = \{q_2, q_3, q_4\}$$

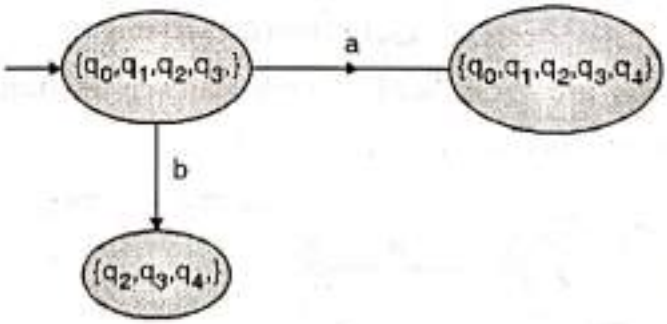


Fig. Ex. 2.3.31(a)

Step 3 : Two new subsets $\{q_2, q_3, q_4\}$ and $\{q_0, q_1, q_2, q_3, q_4\}$ have been generated. Their successors are calculated.

$$\text{a-successor of } \{q_2, q_3, q_4\} = \varepsilon\text{-closure}(\delta(q_2, a) \cup \delta(q_3, a) \cup (q_4, a)$$
$$= \varepsilon\text{-closure}(\phi \cup \{q_4\} \cup \phi) = \varepsilon\text{-closure}(q_3, q_4)$$
$$= \varepsilon\text{-closure}(q_3) \cup \varepsilon\text{-closure}(q_4)$$

Scanned by CamScanner

$= \{q_3\} \cup \{q_3, q_4\} = \{q_3, q_4\}$

b-successor of $\{q_2, q_3, q_4\}$ $= \varepsilon\text{-closure}(\delta(q_2, b) \cup \delta(q_3, b) \cup (q_4, b))$

$= \varepsilon\text{-closure}(\{q_4\} \cup \phi \cup \phi)$

$= \varepsilon\text{-closure}(q_4) = \{q_3, q_4\}$

a-successor of $\{q_0, q_1, q_2, q_3, q_4\}$ = a-successor of $\{q_0, q_1, q_2, q_3\} \cup$ a-successor $\{q_4\}$

$= \{q_0, q_1, q_2, q_3, q_4\} \cup \phi = \{q_0, q_1, q_2, q_3, q_4\}$

b-successor of $\{q_0, q_1, q_2, q_3, q_4\}$ = b-successor of $\{q_0, q_1, q_2, q_3\} \cup$ b-successor of $\{q_4\}$

$= \{q_2, q_3, q_4\} \cup \phi = \{q_2, q_3, q_4\}$

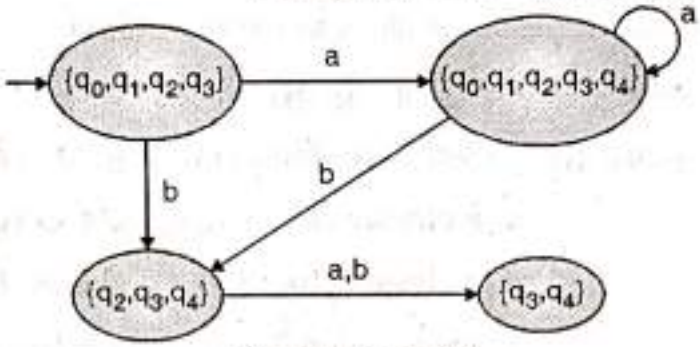


Fig. Ex. 2.3.31(b)

Step 4 : A new subset $\{q_3, q_4\}$ has been generated. Successors of $\{q_3, q_4\}$ are calculated.

a-successor of $\{q_3, q_4\}$ $= \varepsilon\text{-closure}(\delta(q_3, a) \cup \delta(q_4, b))$

$= \varepsilon\text{-closure}(\{q_4\} \cup \phi) = \varepsilon\text{-closure}(q_4) = \{q_3, q_4\}$

b-successor of $\{q_3, q_4\}$ $= \varepsilon\text{-closure}(\delta(q_3, b) \cup \delta(q_4, b))$

$= \varepsilon\text{-closure}(\phi \cup \phi) = \phi$

No further subsets are generated. Every subset containing q_4 (final state in Fig. Ex. 2.3.31) is marked as final state.

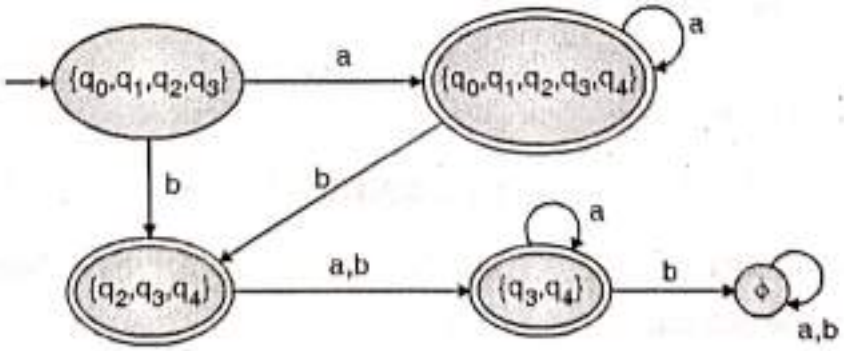


Fig. Ex. 2.3.31(c) : Final DFA

Scanned by CamScanner

Example 2.3.32 : Consider the following ε-NFA

	ε	a	b	c
→p	ϕ	{p}	{q}	{r}
q	{p}	{q}	{r}	ϕ
r*	{q}	{r}	ϕ	{p}

(a) Compute the ε-closure of each state.

(b) Convert the automaton to a DFA.

Solution :

(a) ε-closure of states.

State	ε-closure	Comment
p	{p}	There is no ε-move from p
q	{p, q}	There is an ε-move from q to p.
r	{p, q, r}	There are ε-moves from r to q, q to p.

(b) Conversion of NFA to DFA

Step 1 : ε-closure of the initial state p is taken as the first subset

$$\varepsilon\text{-closure}(p) = \{p\}$$

$$\text{a-successor of } \{p\} = \varepsilon\text{-closure}(\delta(p, a))$$
$$= \varepsilon\text{-closure}(p) = p$$

$$\text{b-successor of } \{p\} = \varepsilon\text{-closure}(\delta(p, b))$$
$$= \varepsilon\text{-closure}(q)$$
$$= \{p, q\}$$

$$\text{c-successor of } \{p\} = \varepsilon\text{-closure}(\delta(p, c))$$
$$= \varepsilon\text{-closure}(r) = \{p, q, r\}$$

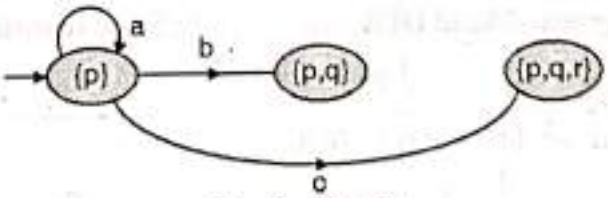


Fig. Ex. 2.3.32

Step 2 : Two new subsets {p, q} and {p, q, r} are generated. Their successors are calculated.

$$\text{a-successor of } \{p, q\} = \varepsilon\text{-closure}(\delta(p, a) \cup \delta(q, a))$$
$$= \varepsilon\text{-closure}(\{p\} \cup \{q\})$$
$$= \varepsilon\text{-closure}(\{p, q\})$$

Scanned by CamScanner

$$= \varepsilon\text{-closure}(p) \cup \varepsilon\text{-closure}(q)$$

$$= \{p\} \cup \{p, q\} = \{p, q\}$$

$$\text{b-successor of } \{p, q\} = \varepsilon\text{-closure}(\delta(p, b) \cup \delta(q, b))$$

$$= \varepsilon\text{-closure}(\{q\} \cup \{r\})$$

$$= \varepsilon\text{-closure}(q) \cup \varepsilon\text{-closure}(r)$$

$$= \{p, q\} \cup \{p, q, r\} = \{p, q, r\}$$

$$\text{c-successor } \{p, q\} = \varepsilon\text{-closure}(\delta(p, c) \cup (q, c))$$

$$= \varepsilon\text{-closure}(\{r\} \cup \phi) = \{p, q, r\}$$

$$\text{a-successor of } \{p, q, r\} = \text{a-successor of } \{p, q\} \cup \text{a-successor of } \{r\}$$

$$= \{p, q\} \cup \varepsilon\text{-closure}(r)$$

$$= \{p, q\} \cup \{p, q, r\} = \{p, q, r\}$$

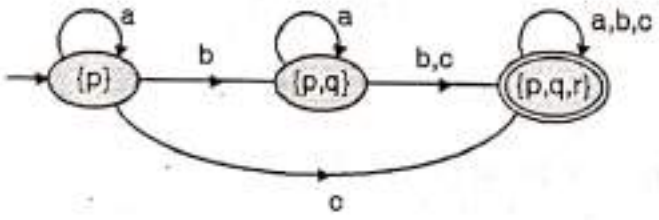
$$\text{b-successor of } \{p, q, r\} = \text{b-successor of } \{p, q\} \cup \text{b-successor of } \{r\}$$

$$= \{p, q, r\}$$

$$\text{c-successor of } \{p, q, r\} = \text{c-successor } \{p, q\} \cup \text{c-successor of } \{r\}$$

$$= \{p, q, r\} \cup \varepsilon\text{-closure}(p) = \{p, q, r\}$$

No new subsets are generated. Each subset containing r (final state in given (ε-NFA) is marked as final state.



(a) State transition diagram of final DFA

	a	b	c
→{p}	{p}	{p, q}	{p, q, r}
{p, q}	{p, q}	{p, q, r}	{p, q, r}
{p, q, r}*	{p, q, r}	{p, q, r}	{p, q, r}

(b) State transition table of final DFA

Fig. Ex. 2.3.32

Example 2.3.33 : Consider the following ε-NFA.

	ε	a	b	c
→p	{q, r}	ϕ	{q}	{r}
q	ϕ	{p}	{r}	{p, q}
r*	ϕ	ϕ	ϕ	ϕ

(a) Compute the ε-closure of each state.
(b) Convert the automation of a DFA.

Scanned by CamScanner

Solution :

(a) ε-closure of states

State	ε-closure	Comment
p	{p, q, r}	There is an ε-move from p to {q, r}
q	{q}	There is no ε-move from q.
r	{r}	There is no ε-move from r.

(b) Conversion of NFA to DFA

Step 1 : ε-closure of the initial state p is taken as the first subset.

$$\varepsilon\text{-closure (p)} = \{p, q, r\}$$

Successor of {p, q, r} are given by :

$$\begin{aligned}
\text{a-successor of } \{p, q, r\} &= \varepsilon\text{-closure } (\delta(p, q, r\}, a)) \\
&= \varepsilon\text{-closure } (\delta(p, a) \cup \delta(q, a) \cup \delta(r, a)) \\
&= \varepsilon\text{-closure } (\phi \cup \{p\} \cup \phi) \\
&= \varepsilon\text{-closure } (\{p\}) \\
&= \{p, q, r\} \\
\text{b-successor of } \{p, q, r\} &= \varepsilon\text{-closure } [\delta(\{p, q, r,\}, b)] \\
&= \varepsilon\text{-closure } (\delta(p, b) \cup \delta(q, b) \cup \delta(r, b)) \\
&= \varepsilon\text{-closure } (\{q\} \cup \{r\} \cup \phi) \\
&= \varepsilon\text{-closure } (\{q, r\}) \\
&= \varepsilon\text{-closure } (q) \cup \varepsilon\text{-closure } (r) \\
&= \{q\} \cup \{r\} \\
&= \{q, r\} \\
\text{c-successors of } \{p, q, r\} &= \varepsilon\text{-closure } (\delta(\{p, q, r\}, c)) \\
&= \varepsilon\text{-closure } (\delta(p, c) \cup (q, c) \cup \delta(r, c)) \\
&= \varepsilon\text{-closure } (\{r\} \cup \{p, q\} \cup \phi) \\
&= \varepsilon\text{-closure } (\{p, q, r\}) \\
&= \varepsilon\text{-closure } (p) \cup \varepsilon\text{-closure } (q) \cup \varepsilon\text{-closure } (r) \\
&= \{p, q, r\} \cup \{q\} \cup \{r\} = \{p, q, r\}
\end{aligned}$$

Step 2 : A new subset {q, r} is generated and its successors are calculated.

$$\begin{aligned}
\text{a successor of } \{q, r\} &= \varepsilon\text{-closure } (\delta\{q, r\}, a)) \\
&= \varepsilon\text{-closure } (\delta(q, a) \cup \delta(r, a))
\end{aligned}$$

Scanned by CamScanner

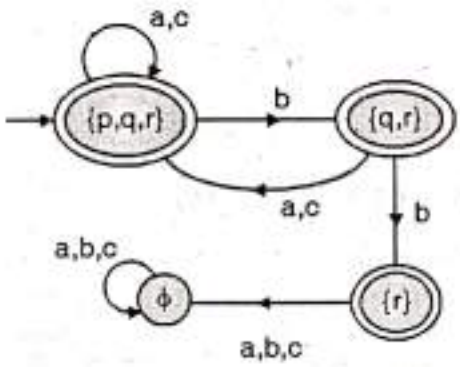
$$= \varepsilon\text{-closure}(\{p\} \cup \phi)$$
$$= \varepsilon\text{-closure}(p)$$
$$= \{p, q, r\}$$

b-successors of $\{q, r\} = \varepsilon\text{-closure}(\delta(\{q, r\}, b))$
$$= \varepsilon\text{-closure}(\delta(q, b) \cup \delta(r, b))$$
$$= \varepsilon\text{-closure}(\{r\} \cup \phi)$$
$$= \varepsilon\text{-closure}(\{r\}) = \{r\}$$

c-successor of $\{q, r\} = \varepsilon\text{-closure}(\delta(\{q, r\}, c))$
$$= \varepsilon\text{-closure}(\delta(q, c) \cup \delta(r, c))$$
$$= \varepsilon\text{-closure}(\{p, q\} \cup \phi)$$
$$= \{p, q, r\}$$

Step 3 : A new subset {r} is generated successor of {r} are ϕ.

Every subset containing r is marked as a final state. Final DFA is given in Fig. Ex. 2.3.33.



(a) State diagram

	a	b	c
→{p, q, r}*	{p, q, r}	{q, r}	{p, q, r}
{q, r}*	{p, q, r}	{r}	{p, q, r}
{r}*	ϕ	ϕ	ϕ
ϕ	ϕ	ϕ	ϕ

(b) State table

Fig. Ex. 2.3.33 : Final DFA for Example 2.3.33

Example 2.3.34 : Design ε-NFA for the following languages. Try to use ε-transitions to simplify the design

(a) The set of strings consisting of zero or more a's followed by zero or more b's, followed by zero or more c's.

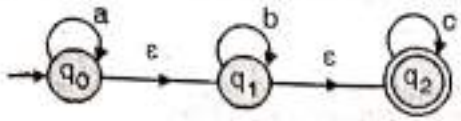
(b) The set of strings that consist of either 01 repeated zero or more times or 010 repeated zero or more times.

(c) The set of strings of 0's and 1's such that at least one of the last ten positions is 1.

Solution :

(a) The set of strings consisting of zero or more a's followed by zero or more b's, followed by zero or more c's.

Scanned by CamScanner



	ε	a	b	c
→q_0	$\{q_1\}$	$\{q_0\}$	ϕ	ϕ
q_1	$\{q_2\}$	ϕ	$\{q_1\}$	ϕ
q_2^*	ϕ	ϕ	ϕ	$\{q_2\}$

Fig. Ex. 2.3.34 : ε-NFA for Example 2.3.34(a)

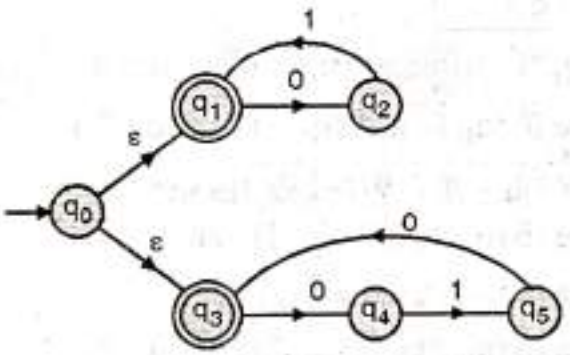
Meaning of various states :

$q_0 \rightarrow q_0$ will generate zero or more a's

$q_1 \rightarrow q_1$ will generate zero or more b's

$q_2 \rightarrow q_2$ will generate zero or more c's

(b) The set of strings that consist of either 01 repeated zero or more times or 010 repeated zero or more times.



(a) State diagram

	ε	0	1
→q_0	$\{q_1, q_3\}$	ϕ	ϕ
q_1^*	ϕ	$\{q_2\}$	ϕ
q_2	ϕ	ϕ	$\{q_1\}$
q_3^*	ϕ	$\{q_4\}$	ϕ
q_4	ϕ	ϕ	$\{q_5\}$
q_5	ϕ	$\{q_3\}$	ϕ

(b) State table

Fig. Ex. 2.3.34 : ε-NFA for Example 2.3.34(b)

- Path $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_1 \ldots q_2 \rightarrow q_1$ is for generation of zero or more representation of 01.
- Path $q_0 \rightarrow q_3 \rightarrow q_4 \rightarrow q_5 \ldots \rightarrow q_3$ is for generation of zero or more representation of 010.

(c) The set of strings of 0's and 1's such that at least one of the last ten positions is 1.

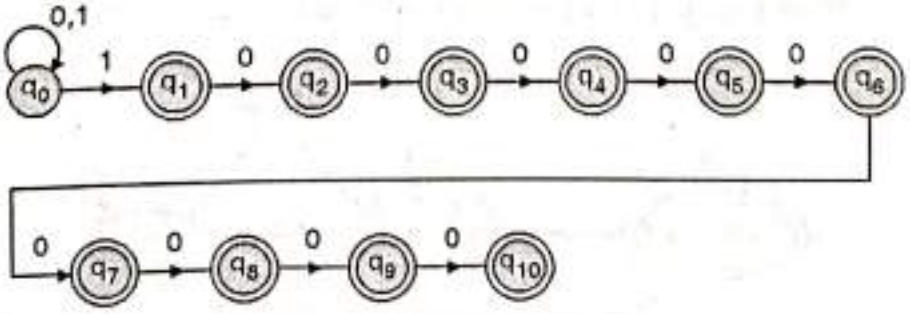


Fig. Ex. 2.3.34(c) : NFA for Example 2.3.34(c)

Scanned by CamScanner

- State q_1 is for accepting strings with last character is 1.
- Similarly, states q_2 to q_{10} are for accepting strings whose second position to 10th position from the end contain 1.

Example 2.3.35 : Convert the following NFA-ε to its equivalent DFA.

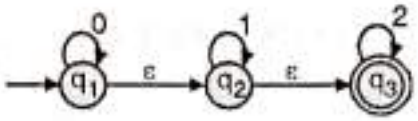


Fig. Ex. 2.3.35

Solution :

Step 1 : Finding ε-closure

State	ε-closure
q_1	$\{ q_1, q_2, q_3 \}$
q_2	$\{ q_2, q_3 \}$
q_3	$\{ q_3 \}$

Step 2 : Initial state is taken as the ε-closure of q_1. Transitions are written from it.

$$\delta(\{q_1, q_2, q_3\}, 0) = \varepsilon\text{-closure}(\delta(q_1, 0) \cup \delta(q_2, 0) \cup \delta(q_3, 0))$$
$$= \varepsilon\text{-closure}(q_1 \cup \phi \cup \phi) = \{ q_1, q_2, q_3 \}$$
$$\delta(\{q_1, q_2, q_3\}, 1) = \varepsilon\text{-closure}(\delta(q_1, 1) \cup \delta(q_2, 1) \cup \delta(q_3, 1))$$
$$= \varepsilon\text{-closure}(\phi \cup q_2 \cup \phi) = \{ q_2, q_3 \}$$
$$\delta(\{q_1, q_2, q_3\}, 2) = \varepsilon\text{-closure}(\delta(q_1, 2) \cup \delta(q_2, 2) \cup \delta(q_3, 2))$$
$$= \varepsilon\text{-closure}(\phi \cup \phi \cup q_3) = \{ q_3 \}$$

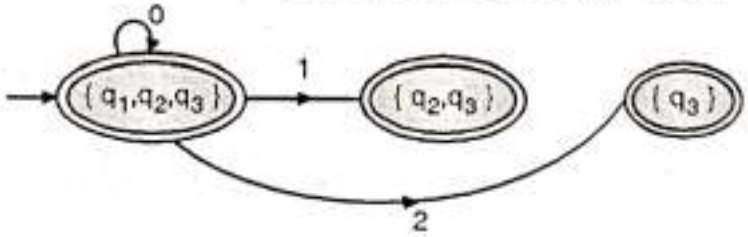


Fig. Ex. 2.3.35(a)

Step 3 : Transitions from $\{ q_2, q_3 \}$ and $\{ q_3 \}$ are calculated.

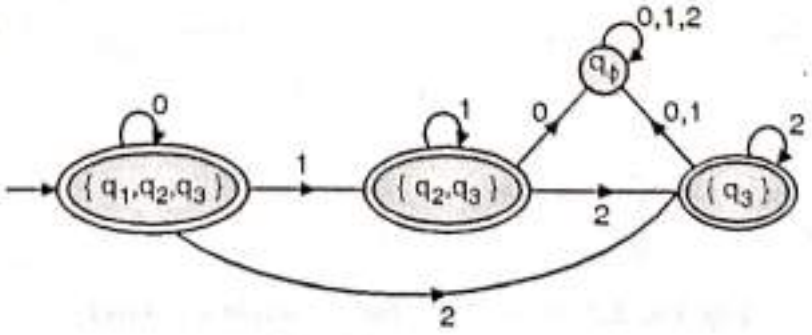


Fig. Ex. 2.3.35(b)

Scanned by CamScanner

Example 2.3.36 : Design an FSM to accept those strings having 101 or 110 as substring.

Solution : The required FSM is given below.

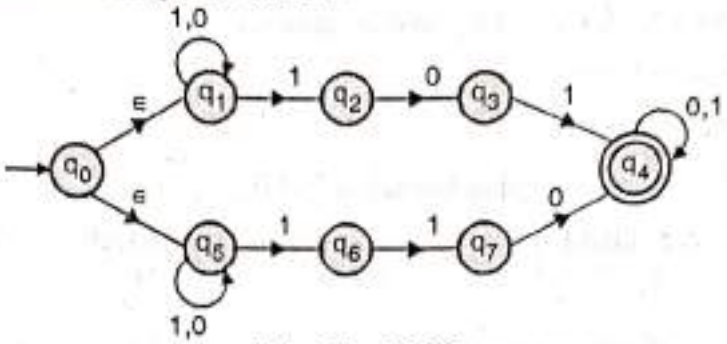


Fig. Ex. 2.3.36

- The path from $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4$ is for the substring 101.
- The path from $q_0 \rightarrow q_5 \rightarrow q_6 \rightarrow q_7 \rightarrow q_4$ is for the substring 110.

2.3.5 Difference between NFA and DFA

MU - May 2007, Dec. 2009, May 2011, May 2014, May 2015, May 2016, May 2017, Dec. 2017

Parameter	NFA	DFA
Transition	Non-deterministic.	Deterministic
No. of states.	NFA has fewer number of states.	More, if NFA contains Q states then the corresponding DFA will have $\leq 2^Q$ states.
Power	NFA is as powerful as a DFA	DFA is as powerful as an NFA
Design	Easy to design due to non-determinism.	Relatively, more difficult to design as transitions are deterministic.
Acceptance	It is difficult to find whether $w \in L$ as there are several paths. Backtracking is required to explore several parallel paths.	It is easy to find whether $w \in L$ as transitions are deterministic.

2.4 Finite Automata as Output Devices

MU - Dec. 2010

Finite automata that we have discussed so far have a limited capability of either accepting a string or rejecting a string. Acceptance of a string was based on the reachability of a machine from starting state to final state. Finite automata can also be used as an output device.

- Such machines do no have final state/states.

Scanned by CamScanner

- Machine generates an output on every input. The value of the output is a function of current state and the current input.

  Such machines are characterised by two behaviours :

  1. State transition function (δ)
  2. Output function (λ)

  State transition function (δ) is also known as STF.

  Output function (λ) is also known as machine function (MAF).

  $\delta : \Sigma \times Q \rightarrow Q$

  $\lambda : \Sigma \times Q \rightarrow O$ [for Mealy machine]

  $\lambda : Q \rightarrow O$ [for Moore machine]

There are two types of automata with outputs :

1. Mealy machine : Output is associated with transition.

   $\lambda : \Sigma \times Q \rightarrow O$

   Set of output alphabet O can be different from the set of input alphabet Σ

2. Moore machine : Output is associated with state.

   $\lambda : Q \rightarrow O$

2.4.1 A Sample Mealy Machine

MU - Dec. 2005, Dec. 2010, Dec. 2017

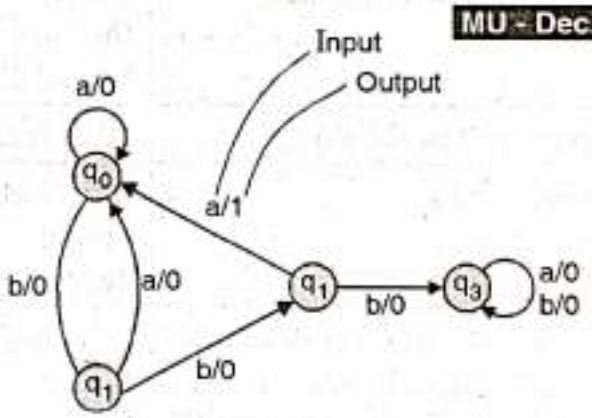


Fig. 2.4.1 : State diagram of a Mealy machine

State transition function (δ) (or STF) :

	a	b
→q_0	q_0	q_1
q_1	q_0	q_2
q_2	q_0	q_3
q_3	q_3	q_3

Fig. 2.4.2 : State transition function for Mealy machine of Fig. 2.4.1

Scanned by CamScanner

Output function (λ) (or MAF) :

	a	b
$\rightarrow q_0$	0	0
q_1	0	0
q_2	1	0
q_3	0	0

Fig. 2.4.3 : Output function for mealy machine of Fig. 2.4.1

State table for both δ and λ (both STF and MAF) :

	A	b
$\rightarrow q_0$	$q_0/0$	$q_1/0$
q_1	$q_0/0$	$q_2/0$
q_2	$q_0/1$	$q_3/0$
q_3	$q_3/0$	$q_3/0$

Output

Next state

Fig. 2.4.4 : State table depicting both transition and output behavior of mealy machine of Fig. 2.4.1

- An arc from state q_i in a mealy machine is associated with :
  1. Input alphabet $\in \Sigma$
  2. An output alphabet $\in$ O.
- An arc marked as 'a/0' in Fig. 2.4.1 implies that :
  1. a is in input
  2. 0 is an output.
- State transition behavior and output behavior of a mealy machine can be shown separately as in Fig. 2.4.2 and 2.4.3; or they can be combined together as in Fig. 2.4.4.

2.4.2 Formal Definition of a Mealy Machine

MU - Dec. 2005, Dec. 2017

A mealy machine M is defined as :

$$M = (Q, \Sigma, O, \delta, \lambda, q_0)$$

Where,

Q = A finite set of states.

Σ = A finite set of input alphabet

Scanned by CamScanner

O = A finite set of output alphabet

δ = A transition function $\Sigma \times Q \rightarrow Q$

λ = An output function $\Sigma \times Q \rightarrow O$

q_0 = $q_0 \in Q$ is an initial state.

2.4.3 A Sample Moore Machine

MU- Dec. 2017

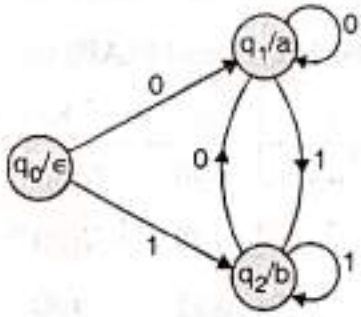


Fig. 2.4.5 : State diagram of a Moore machine

State transition function (δ) :

	0	1
$\rightarrow q_0$	q_1	q_2
q_1	q_1	q_2
q_2	q_1	q_2

Fig. 2.4.6 : State transition function for Moore machine of Fig. 2.4.5

Output function (O)

State	Output
$\rightarrow q_0$	ϵ
q_1	a
q_2	b

Fig. 2.4.7 : Output function for Moore machine of Fig. 2.4.5

State table for both δ and λ :

State	Input		Output
	0	1	
$\rightarrow q_0$	q_1	q_2	ϵ
q_1	q_1	q_2	a
q_2	q_1	q_2	b

Fig. 2.4.8 : State table depicting both transition and output behavior of Moore machine of Fig. 2.4.5

- Output in a Moore machine is associated with a state and not with transition.

Scanned by CamScanner

2.4.4 Formal Definition of a Moore Machine

MU- Dec. 2017

A Moore machine M is defined as :

$$M = \{Q, \Sigma, O, \delta, \lambda, q_0)$$

where,

Q = A finite set of states

Σ = A finite set of input alphabet

O = A finite set of output alphabet

δ = A transition function $\Sigma \times Q \rightarrow Q$

λ = An output function $Q \rightarrow O$

q_0 = $q_0 \in Q$ is an initial state.

Example 2.4.1 : Give Mealy and Moore machine for the following :
From input Σ^*, where $\Sigma = (0, 1, 2)$ print the residue modulo 5 of the input treated as ternary (base 3).

MU - May 2006, Dec. 2015

Solution :

(a) Mealy machine :

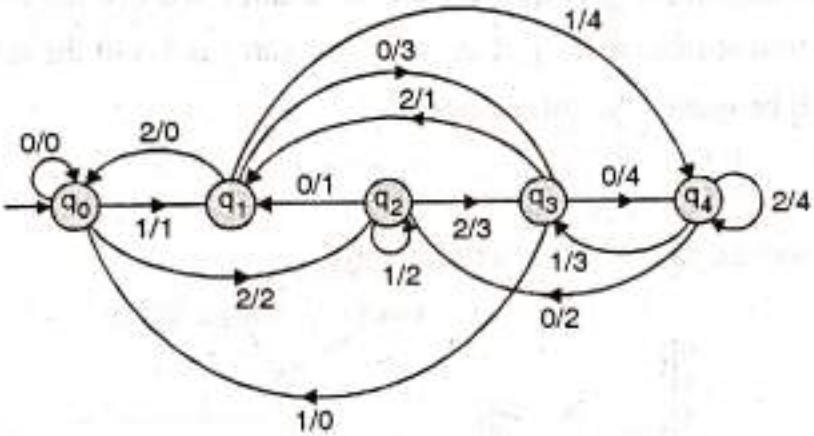


Fig. Ex. 2.4.1(a) : Mealy machine

Meaning of various states is :

q_0 – Running remainder is 0

q_1 – Running remainder is 1

q_2 – Running remainder is 2.

q_3 – Running remainder is $3 = (10)_3$

q_4 – Running remainder is $4 = (11)_3$

In decimal system

In ternary system

Scanned by CamScanner

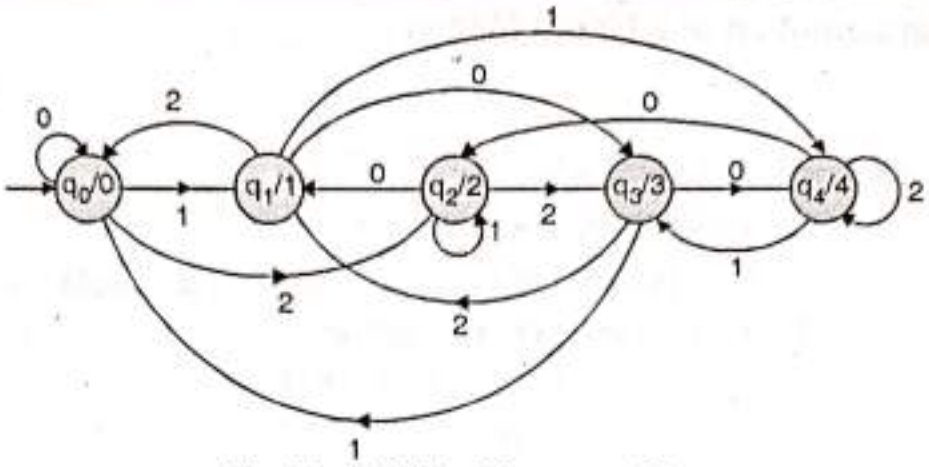


Fig. Ex. 2.4.1(b) : Moore machine

Example 2.4.2 : Draw the transition matrix and transition graph for a binary adder.

Solution : The machine will have two states :

q_0 – corresponding to previous carry as 0

q_1 – corresponding to previous carry as 1.

- Machine will transit from q_0 to q_1 if the previous carry is 0 and the next carry is 1.
- Machine will transit from q_1 to q_0 if the previous carry is 1 and the next carry is 0.
- Next input will be one of the followings :

$\binom{0}{0}, \binom{0}{1}, \binom{1}{0}, \binom{1}{1}$

- Sum of carry will be calculated in a usual way.

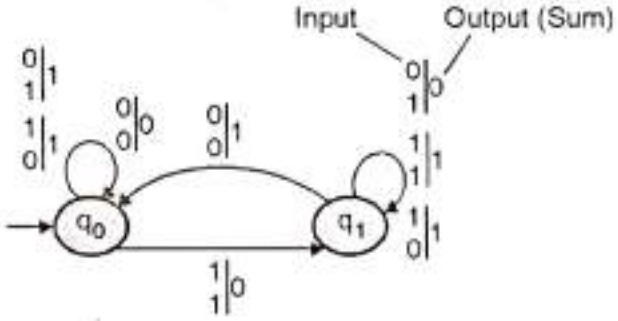


Fig. Ex. 2.4.2(a) : Transition graph

	$\binom{0}{0}$	$\binom{0}{1}$	$\binom{1}{0}$	$\binom{1}{1}$	
q_0	$q_0, 0$	$q_0, 1$	$q_0, 1$	$q_1, 0$	← carry is 0
q_1	$q_0, 1$	$q_1, 0$	$q_1, 0$	$q_1, 1$	← carry is 1

Output

Next state

Fig. Ex. 2.4.2(b) : Transition matrix

Scanned by CamScanner

Example 2.4.3 : Show MAF, STF, Transition graph and Transition matrix for a divisibility by 3 tester assuming ternary input.

Solution :

A divisibility by 3 tester will have three states. These states will be based on running remainder. The next remainder will be a function of current remainder and the next input. A ternary system has three alphabets.

$\Sigma = \{0, 1, 2\}$

q_0 = current remainder is 0

q_1 = current remainder is 1

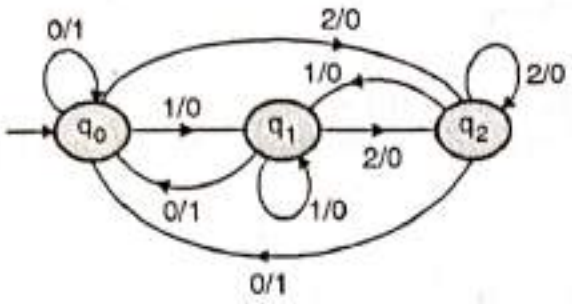
q_2 = current remainder is 2.

Next remainder as a function of current remainder (state) and the next input is given below.

Current state (current remainder)	Next input		Next remainder (next state)
0 (q_0)	0	00 mod 10 = 0	q_0
	1	01 mod 10 = 1	q_1
	2	02 mod 10 = 2	q_2
		$(10)_3 = (3)_{10}$	
1 (q_1)	0	10 mod 10 = 0	q_0
	1	11 mod 10 = 1	q_1
	2	12 mod 10 = 2	q_2
2 (q_2)	0	20 mod 10 = 0	q_0
	1	21 mod 10 = 1	q_1
	2	22 mod 10 = 2	q_2

Machine will give an output 1 if the current remainder is 0, output will be 0 otherwise.

Scanned by CamScanner



(a) State transition diagram

Current state	Input		
	0	1	2
$\rightarrow q_0$	$q_0/1$	$q_1/0$	$q_2/0$
q_1	$q_0/1$	$q_1/0$	$q_2/0$
q_2	$q_0/1$	$q_1/0$	$q_2/0$

(b) State transition table

	0	1	2
$\rightarrow q_0$	1	0	0
q_1	1	0	0
q_2	1	0	0

(c) MAF

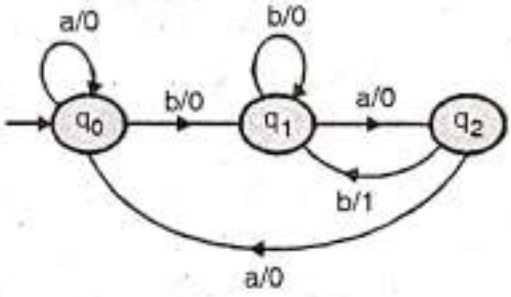
	0	1	2
$\rightarrow q_0$	q_0	q_1	q_2
q_1	q_0	q_1	q_2
q_2	q_0	q_1	q_2

(d) STF

Fig. Ex. 2.4.3

Example 2.4.4 : Design a Mealy machine that gives an output of 1 if the input string ends in bab.

Solution :



	a	b
$\rightarrow q_0$	q_0, 0	q_1, 0
q_1	q_2, 0	q_1, 0
q_2	q_0, 0	q_1, 1

(a) State diagram (b) State table

Fig. Ex. 2.4.4

Meaning of various states :

q_0 – Start state

q_1 – Previous character is b

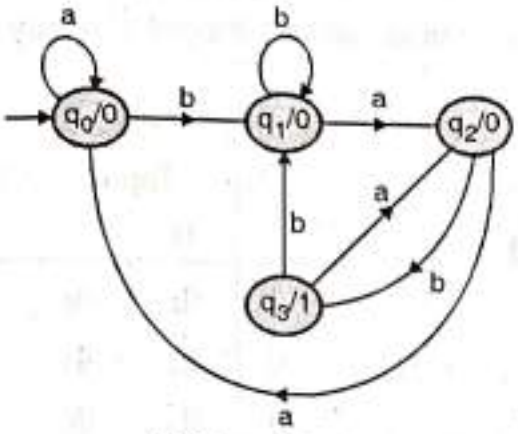
q_2 – Preceding two character are ba

A transition from q_2 to q_1 will make the preceding three characters as bab and hence the output 1.

Example 2.4.5 : Design a Moore machine that gives an output of 1 if the input string ends in bab.

Scanned by CamScanner

Solution :



(a) State diagram

Present state	Input a	Input b	Output
$\rightarrow q_0$	q_0	q_1	0
q_1	q_2	q_1	0
q_2	q_0	q_3	0
q_3	q_2	q_1	1

(b) State table

Fig. Ex. 2.4.5 : Moore machine for the Example 2.4.5

Meaning of various states :

q_0 – Start state

q_1 – Previous character is b

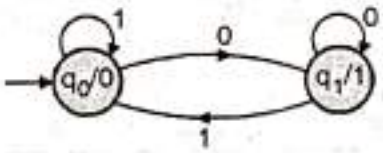
q_2 – Preceding two character are ab

q_3 – Previous character is b

A transition from q_2 to q_3 will make the preceding three characters as bab hence the output 1.

Example 2.4.6 : Design a Moore machine for the 1's complement of binary number.

Solution :



(a)

	0	1	Output
$\rightarrow q_0$	q_1	q_0	0
q_1	q_1	q_0	1

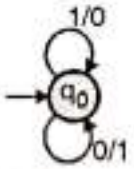
(b)

Fig. Ex. 2.4.6

- Next input as 0, sends the machine to state q_1 with output of 1. Complement of 0 is 1.
- Next input as 1, sends the machine to state q_0 with output of 0. Complement of 1 is 0.

Example 2.4.7 : Design a Mealy machine to generate 1's complement of a given binary number.

Solution :



(a)

	0	1
$\rightarrow q_0$	q_0, 1	q_0,0

(b)

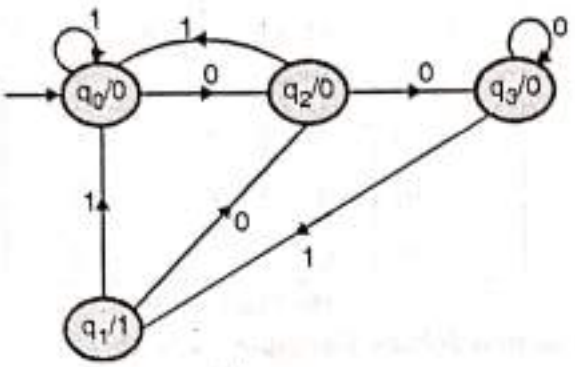
Fig. Ex. 2.4.7

- Every input is simply complemented in state q_0.

Scanned by CamScanner

Example 2.4.8 : Design a Moore machine for generating output 1 if input of binary sequence 1 is preceded with exactly two zeros.

Solution :



(a)

	Input		Output
	0	1	
$\rightarrow q_0$	q_2	q_0	0
q_1	q_2	q_0	1
q_2	q_3	q_0	0
q_3	q_3	q_1	0

(b)

Fig. Ex. 2.4.8

Meaning of various states :

q_0 – initial state, associated with output 0

q_2 – preceding symbol is 0 of sequence 001, associated with output 0.

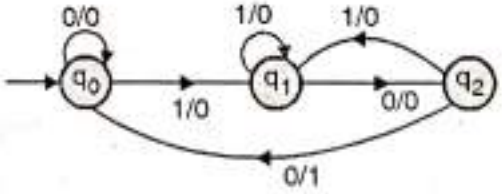
q_3 – preceding two symbols as 00 of sequence 001, associated with output 0.

q_1 – preceding three symbols as 001 of sequence 001, associated with output 1.

Example 2.4.9 : Design a mealy machine for a binary input sequence such that if the sequence ends with 100 the output is 1 otherwise output is 0.

MU - Dec. 2006, May 2008, Dec. 2008

Solution :



(a) State diagram

	0	1
$\rightarrow q_0$	q_0, 0	q_1, 0
q_1	q_2, 0	q_1, 0
q_2	q_0, 1	q_1, 0

(b) State table

Fig. Ex. 2.4.9

Meaning of various states :

q_0 – start state

q_1 – previous symbol is 1

q_2 – preceding two symbols are 10

A transition from q_2 to q_0 will make the preceding three symbol as 100 and hence the output 1.

Example 2.4.10 : Design a mealy machine for incrementing the value of any binary number by one. The output should also be a binary number with value one more than the given number.

Scanned by CamScanner

Solution :

When a binary number is incremented then trailing 1's become 0's and the first zero immediately before trailing 1's become 1.

For example, if we add 1 to 0101111 then the incremented value will be 0110000.

$$\begin{array}{r} 0\ 1\ 0\ 1\ 1\ 1\ 1 \\ +\qquad\qquad\quad 1 \\ \hline 0\ 1\ 1\ 0\ 0\ 0\ 0 \end{array}$$

The mealy machine is shown in Fig. Ex. 2.4.10.

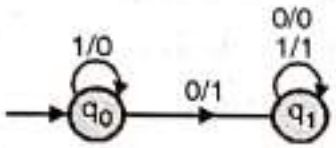


Fig. Ex. 2.4.10

Input should be applied, starting from the rightmost symbol.

	0	1
$\rightarrow q_0$	$q_1, 1$	$q_0, 0$
q_1	$q_1, 0$	$q_1, 1$

Example 2.4.11 : Design a Moore machine for the following problem :
Input alphabet = {0, 1, 2}
Output alphabet = {a, b, c}
If the sum of 3 consecutive number is 0, 1, 4; output a.
If the sum is 2, 3; output b.
If the sum is 5, 6; output c. **MU - May 2007**

Solution :

Sum of three consecutive symbols will depend on :

1. Preceding two symbols
2. Next input.

Preceding two symbols could be :

Symbols		State
00	—	q_{00}
01	—	q_{01}
02	—	q_{02}
10	—	q_{10}
11	—	q_{11}
12	—	q_{12}

Scanned by CamScanner

Symbols		State
20	—	q_{20}
21	—	q_{21}
22	—	q_{22}

The next state and the output is given below :

1. An input 0 in state q_{00} will make sum as 0 and next state as q_{00}
2. An input 1 in state q_{00} will make sum as 1 and next state as q_{01}
3. An input 2 in state q_{00} will make sum as 2 and next state as q_{02}
4. An input 0 in state q_{01} will make sum as 1 and next state as q_{10}
5. An input 1 in state q_{01} will make sum as 2 and next state as q_{11}
6. An input 2 in state q_{01} will make sum as 3 and next state as q_{12}

:

:

:

25. An input 0 in state q_{22} will make sum as 4 and next state as q_{20}
26. An input 1 in state q_{22} will make sum as 5 and next state as q_{21}
27. An input 2 in state q_{22} will make sum as 6 and next state as q_{22}

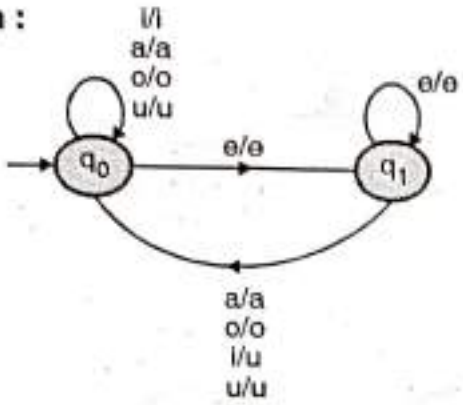
State transition table,

	0	1	2
$\rightarrow q_{00}$	q_{00}/a	q_{01}/a	q_{02}/b
q_{01}	q_{10}/a	q_{11}/b	q_{12}/b
q_{02}	q_{20}/b	q_{21}/b	q_{22}/a
q_{10}	q_{00}/a	q_{01}/b	q_{02}/b
q_{11}	q_{10}/b	q_{11}/b	q_{12}/a
q_{12}	q_{20}/b	q_{21}/a	q_{22}/c
q_{20}	q_{00}/b	q_{01}/b	q_{02}/a
q_{21}	q_{10}/b	q_{11}/a	q_{12}/c
q_{22}	q_{20}/a	q_{21}/c	q_{22}/c

Scanned by CamScanner

Example 2.4.12 : Design a Mealy machine that will read sequences made up of vowels of English language, it will give output in same sequences, except in cases where a 'i' follows 'e', it will be changed to 'u'.

Solution :



(a) State diagram

Current state	Next state output				
	a	e	i	o	u
$\rightarrow q_0$	q_0, a	q_1, e	q_0, i	q_0, o	q_0, u
q_1	q_0, a	q_1, e	q_0, u	q_0, o	q_0, u

(b) State table

Fig. Ex. 2.4.12

- Machine must convert every occurrence of ei into eu. In other cases, output will be same as input.
- State q_1 implies that the preceding character is e.
- An input i in stat q_1 will generate an output u.

Example 2.4.13 : Give the Mealy and Moore machine for the following processes. "For input from (0 + 1)*, if inputs ends in 101, output x; if input ends in 110, output y; otherwise output z". **MU – May 2009, May 2010**

Solution :

(a) Design of Mealy machine :

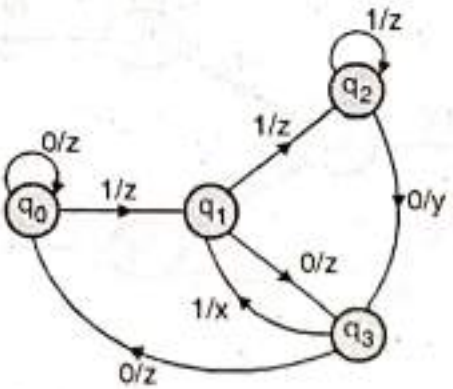


Fig. Ex. 2.4.13(a) : State diagram of Mealy machine

- The mealy machine must be designed to remember the preceding two symbols particularly '11' and '10'.
- The state q_2 is for preceding two symbols as '11'.
- The state q_3 is for preceding two symbols as '10'.

Scanned by CamScanner

- An input of 0 in state q_2 will mean a sequence of 110 (hence the output y), and it will make preceding two symbols as 10 (q_3 stands for preceding two symbols as 10).
- Similar argument can be given for the state q_3.

(b) Design of Moore machine :

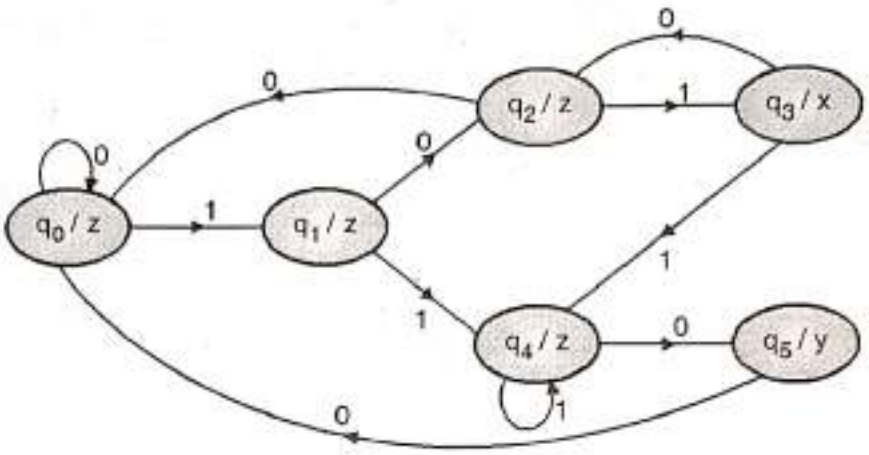


Fig. Ex. 2.4.13(b) : State diagram of the Moore machine

Example 2.4.14 : Design a Moore and Mealy machine for a binary input sequence such that if it has a substring 110 the machine outputs A, if it has a substring 101 machine outputs B, otherwise outputs C.

Solution :

(a) Design of Mealy machine :

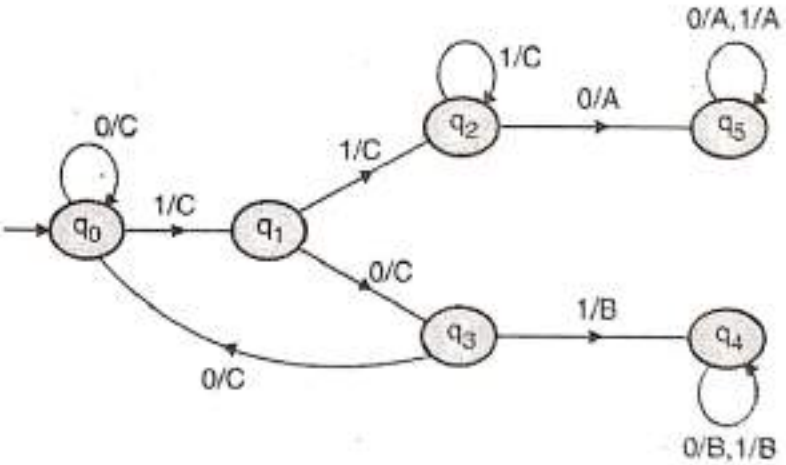


Fig. Ex. 2.4.14(a) : State diagram of the Mealy machine

Meaning of various states :

q_0 – Start state

q_1 – Preceding character is 1

q_2 – Preceding two characters are 11 and it needs a 0 to complete the sequence 110

q_3 – Preceding two characters are 10 and it needs a 1 to complete the sequence 101.

Scanned by CamScanner

(b) Design of Moore machine :

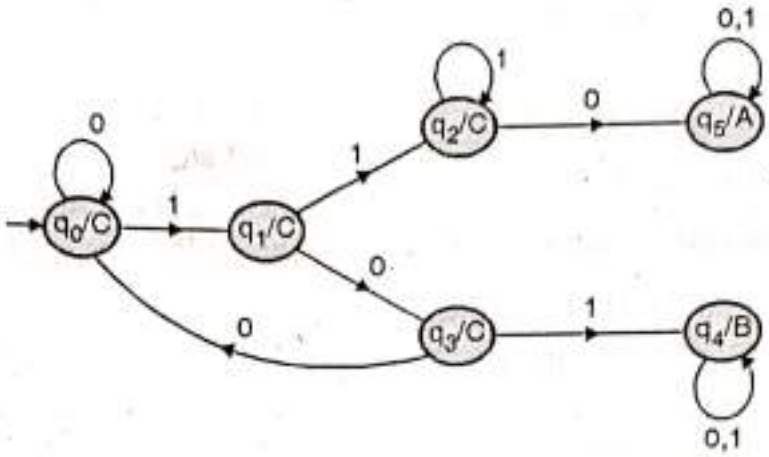


Fig. Ex. 2.4.14(b) : State diagram of the Moore machine

Example 2.4.15 : Design finite state machine that compares two binary numbers to determine whether they are equal and which of two is larger.

MU - May 2006, Dec. 2007

Solution :

Let the two numbers are A and B.

where, $A = a_n, a_{n-1} \ldots a_1$

$B = b_n, b_{n-1} \ldots b_1$

Numbers are entered bitwise, starting from the least significant digit. If there is a mismatch at the i^{th} bit position then we can conclude the following.

$A > B$ if $a_i > b_i$

$A < B$ if $a_i < b_i$

Machine generates the following output :

$A > B \quad \ldots \quad +$

$A < B \quad \ldots \quad -$

$A = B \quad \ldots \quad 0$

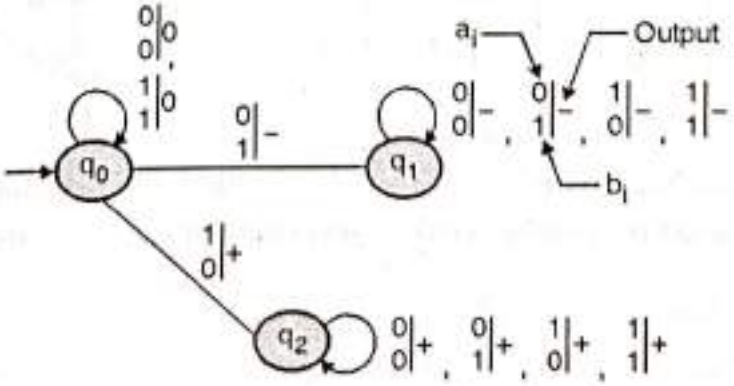


Fig. Ex. 2.4.15 : State diagram of the Mealy machine

Scanned by CamScanner

Example 2.4.16 : Design a Moore Machine for checking divisibility by 3 of a given decimal number (residue of 3).

Solution :

The decimal system has 10 alphabet.

$$\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

The running remained when a given decimal number is divided by 3 could be :

0 → associated state is q_0.

1 → associated state is q_1

2 → associated state is q_2.

The required Moore machine is given Fig. Ex. 2.4.16.

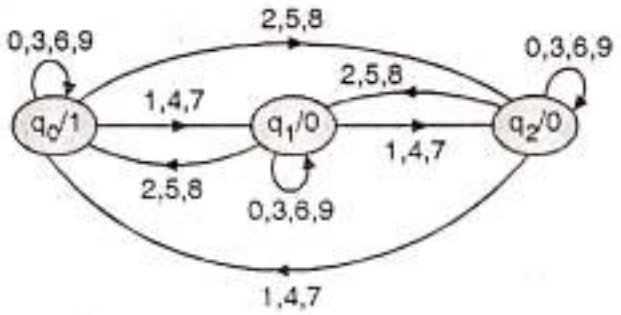


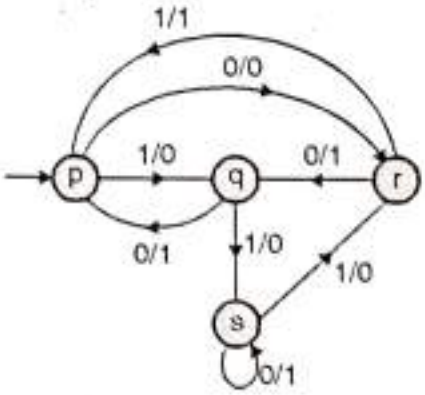
Fig. Ex. 2.4.16

2.4.5 Conversion of a Mealy Machine into a Moore Machine :

A Mealy machine can be transformed into an equivalent Moore machine so that both the machines show the same output behaviour for any input sequence. Conversion process is being explained with the help of a Mealy machine shown in Fig. 2.4.9.

Present state	(Next state, output)	
	Input 0	Input 1
→p	r, 0	q, 0
q	p, 1	s, 0
r	q, 1	p, 1
s	s, 1	r, 0

(a) State table



(b) State diagram

Fig. 2.4.9 : Mealy machine considered for conversion into an equivalent Moore machine

Step 1 : Splitting of states :

It may be necessary to split some of the states of a Mealy machine to get an equivalent Moore machine. Splitting is based on incoming lines into a state and the associated outputs. We must split a state to handle different output values associated with incoming lines.

Scanned by CamScanner

- There are two incoming lines into the state p.
  (i) From q to p on input 0, giving an output of 1.
  (ii) From r to p on input 1, giving an output of 1.
  In every case output is 1 and hence splitting is not required.
- There are two incoming lines into the state q.
  (i) From p to q on input 1, giving an output of 0.
  (ii) From r to q on input 0, giving an output of 1.
  Since there are two different outputs, we must split q into two different states q_0 with output
  0 and q_1 with output 1.

$$q \xrightarrow{\text{split}} \begin{matrix} q_0/0 - \text{output } 0 \\ q_1/1 - \text{output } 1 \end{matrix}$$

- There are two incoming lines into the state r.
  i) From p to r on input 0, giving an output of 0.
  ii) From s to r on input 1, giving an output of 0.
  In every case output is 0 and hence splitting is not required.
- There are two incoming lines into the states S.
  (i) From q to s an input 1, giving an output of 0.
  (ii) From s to s on input 0, giving an output of 1.

Since there are two different outputs, we must split s into two different states. s_0 with output 0 and s_1 with output 1.

$$s \xrightarrow{\text{split}} \begin{matrix} s_0/0 - \text{output } 0 \\ s_1/1 - \text{output } 1 \end{matrix}$$

Step 2 : Drawing of an equivalent Moore machine.

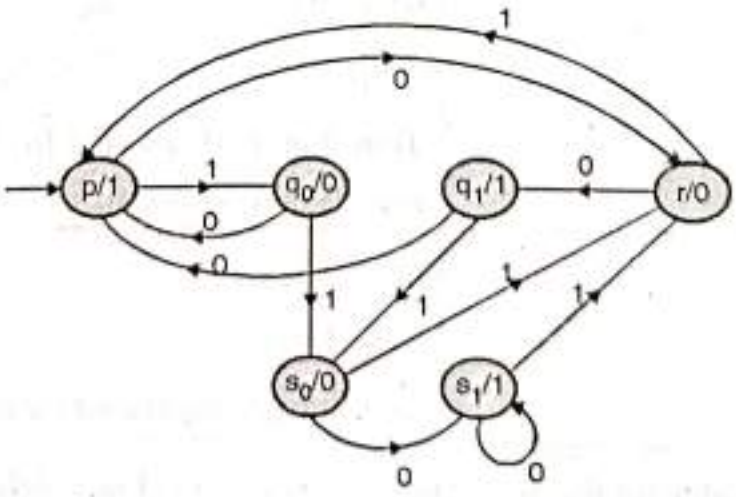


Fig. 2.4.9(c) : State diagram of Moore machine

Scanned by CamScanner

- Every transition into q with output 0 is directed to q_0.
- Every transition into q with output 1 is directed to q_1.
- Every transition into s with output 0 is directed to s_0.
- Every transition into s with output 1 is directed to s_1.
- Every transition from q is duplicated both from q_0 and q_1.
- Every transition from s is duplicated both from s_0 and s_1.

Transitions for Mealy Machine	Equivalent transitions for Moore Machine
δ (p, 0) = r, 0	δ (p, 0) = r
δ (p, 1) = q, 0	δ (p, 1) = q_0 q_0 is associated with output 0
δ (q, 0) = p, 1	δ (q_0, 0) = p δ (q_1, 0) = p Transition is duplicated for both q_0 and q_1
δ (q, 1) = s, 0	δ (q_0, 1) = s_0 δ (q_1, 1) = s_0 Transition is duplicated for both q_0 and q_1. s_0 is associated with output 0.
δ (r, 0) = q, 1	δ (r, 0) = q_1 q_1 is associated with output
δ (r, 1) = p, 1	δ (r, 1) = p
δ (s, 0) = s, 1	δ (s_0, 0) = s_1 δ (s_1, 0) = s_1 Transition is duplicated for both s_0 and s_1. s_1 is associated with output 1.
δ (s, 1) = r, 0	δ (s_0, 1) = r δ (s_1, 1) = r Transition is duplicated for both s_0 and s_1.

State transition table for the Moore machine is given in Fig. 2.4.9(d).

Scanned by CamScanner

Present state	Next state		Output
	0	1	
→p	r	q_0	1
q_0	p	s_0	0
q_1	p	s_0	1
r	q_1	p	0
s_0	s_1	r	0
s_1	s_1	r	1

Fig. 2.4.9(d) : State transition table

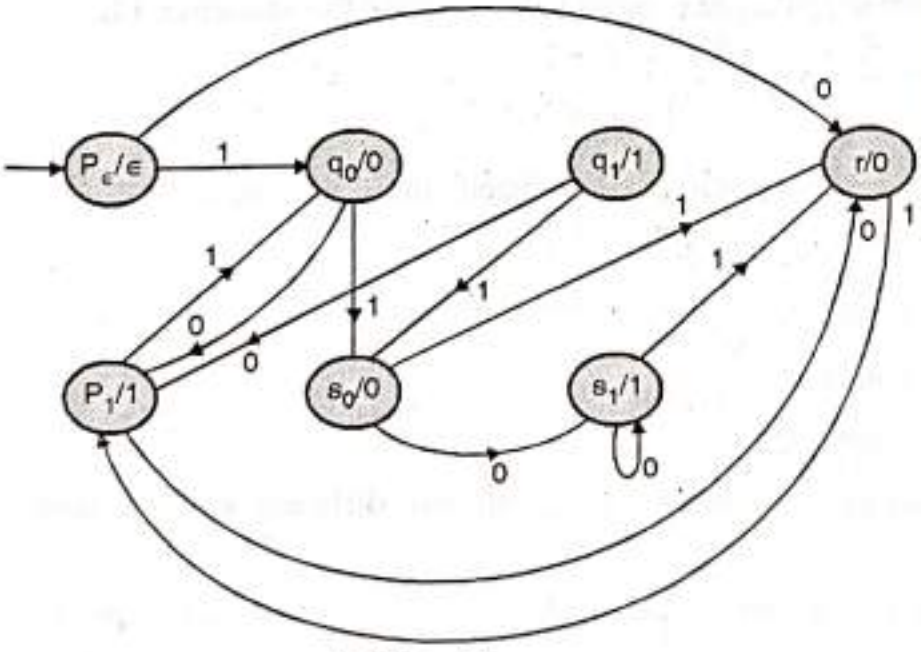
Step 3 : Moore machine of Fig. 2.4.9(c) produces an output 1 without any input. Thus, the Moore machine gives an output 1 on a zero length sequence. But the Mealy machine will produce no output for a zero length sequence.

To handle the above problem, we must split the start state p into two states :

p_ϵ – Giving no output

p_1 – Giving an output 1.

Final Moore machine is drawn in Fig. 2.4.9(e).



(e) State diagram

Present state	Next state		Output
	0	1	
→p_ϵ	r	q_0	ε
p_1	r	q_0	1
q_0	p_1	s_0	0
q_1	p_1	s_0	1
r	q_1	p_1	0
s_0	s_1	r	0
s_1	s_1	r	1

(f) State table

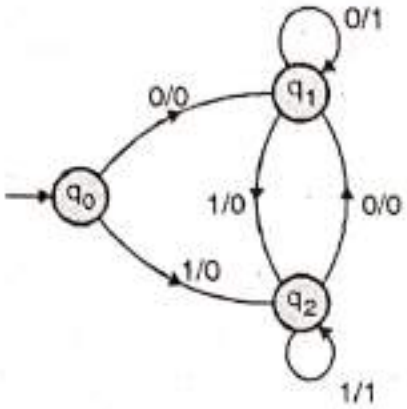
Fig. 2.4.9 : Final Moore machine

Scanned by CamScanner

Example 2.4.17 : Construct a Mealy machine that accept strings ending in '00' and '11'. Convert the same to a Moore machine.

Solution :

(i) Construction of Mealy machine.



(a) State transition diagram of the Mealy machine

Current state	(Next state, Output)	
	0	1
$\rightarrow q_0$	q_1, 0	q_2, 0
q_1	q_1, 1	q_2, 0
q_2	q_1, 0	q_2, 1

(b) State transition table of the Mealy machine

Fig. Ex. 2.4.17

The Mealy machine is shown in the Fig. Ex. 2.4.17. Interpretation of various states is given below.

q_0 = Initial state

q_1 = Preceding input is 0, 0 as next input will complete the sequence 00.

q_2 = Preceding input is 1, 1 as next input will complete the sequence 11.

(ii) Conversion to a Moore machine :

Step 1 : Splitting of states

(i) There is no incoming line into the state q_0. Output associated with q_0 will be ε.

(ii) There are three incoming lines into the state q_1.

(a) q_0 to q_1 on input 0, with output 0.

(b) q_1 to q_1 on input 0, with output 1.

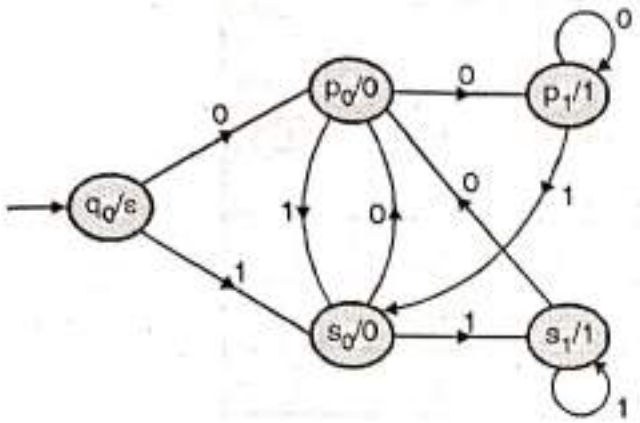
(c) q_2 to q_1 on input 0, with output 0.

Since there are two different outputs, we must split q_0 into two different states p_0 with output 0 and p_1 with output 1.

(iii) Similarly, we must split q_2 into two different states s_0 with output 0 and s_1 with output 1.

Scanned by CamScanner

Step 2 : Drawing of an equivalent Moore machine.



(c) State transition diagram of the Moore machine

Current state	Input		Output
	0	1	
q_0	p_0	s_0	ε
p_0	p_1	s_0	0
p_1	p_1	s_0	1
s_0	p_0	s_1	0
s_1	p_0	s_1	1

(d) State transition table of the Moore machine

Fig. Ex. 2.4.17

Transition in Mealy machine	Corresponding transitions in Moore machine
$\delta(q_0, 0) = q_1, 0$	$\delta(q_0, 0) = p_0$, p_0 is associated with output 0.
$\delta(q_0, 1) = q_2, 0$	$\delta(q_0, 1) = s_0$, s_0 is associated with output 0.
$\delta(q_1, 1) = q_1, 1$	$\delta(p_0, 0) = p_1$ $\delta(p_1, 0) = p_1$ Transition is duplicated for both p_0 and p_1. p_1 is associated with output 1
$\delta(q_1, 1) = q_2, 0$	$\delta(p_0, 1) = s_0$ $\delta(p_1, 1) = s_0$ Transition is duplicated for both p_0 and p_1. s_0 is associated with output 0.
$\delta(q_2, 0) = q_1, 0$	$\delta(s_0, 0) = p_0$ $\delta(s, 0) = p_0$
	Transition is duplicated for both s_0 and s_1. p_0 is associated with output 0.
$\delta(q_2, 1) = q_2, 1$	$\delta(s_0, 1) = s_1$ $\delta(s_1, 1) = s_1$ Transition is duplicated for both s_0 and s_1. s_1 is associated with output 1.

Scanned by CamScanner

Example 2.4.18 : Consider the following Mealy machine. Convert it to Moore.

Present state	Next state			
	a = 0		a = 1	
	State	Output	State	Output
$\rightarrow q_1$	q_1	1	q_2	0
q_2	q_4	1	q_4	1
q_3	q_2	1	q_3	1
q_4	q_3	0	q_1	1

Solution :

Step 1 : Splitting of states.

(i) Incoming lines into q_1 always give an output 1.

(ii) Incoming lines into q_2 give both 0 and 1 as output.

We must split q_2 into two different states p_0 with output 0 and p_1 with output 1.

(iii) Incoming lines into q_3 give both 0 and 1 as output.

We must split q_3 into two different states so with output 0 and s_1 with output 1.

(iv) Incoming lines into q_4 always give an output 1.

Step 2 : Drawing of an equivalent Moore machine.

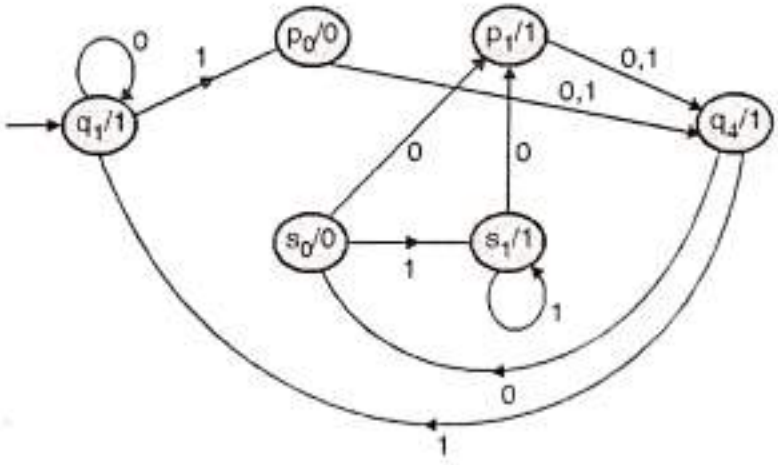


Fig. Ex. 2.4.18(a) : Moore machine

Step 3 : Moore machine of Fig. Ex. 2.4.18(a) produces an output of 1 without any input. But the Mealy machine given in Example 2.4.18 does not produce any output for a zero-length input sequence.

To handle the above problem, we must split the start state q_1 into two states :

r_ε – Giving no output

r_1 – Giving an output 1.

Scanned by CamScanner

Final Moore machine is drawn in Fig. Ex. 2.4.18(b) and 2.4.18(c).

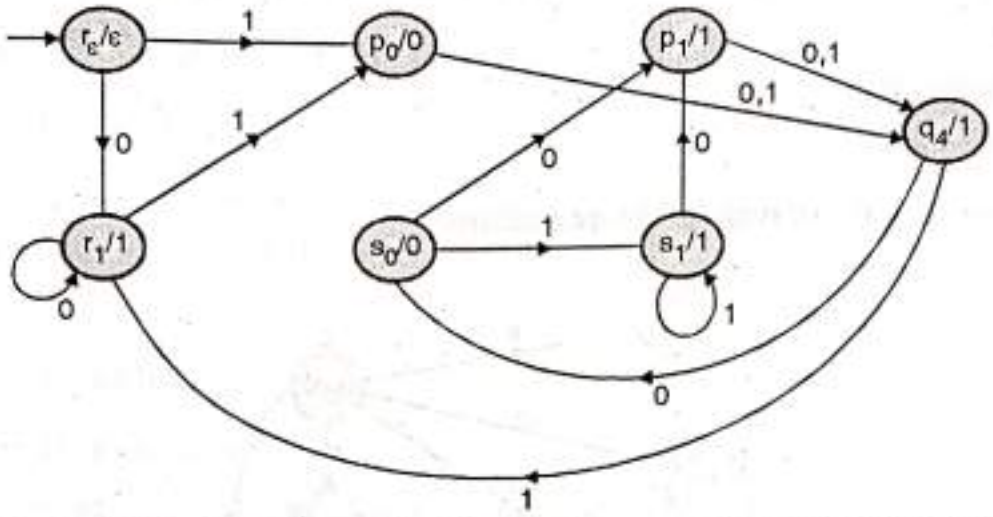


Fig. Ex. 2.4.18(b) : State transition diagram for an equivalent Moore machine

Current state	0	1	Output
$\rightarrow r_\varepsilon$	r_1	p_0	ε
r_1	r_1	p_0	1
p_0	q_4	q_4	0
p_1	q_4	q_4	1
s_0	p_1	s_1	0
s_1	p_1	s_1	1

Fig. Ex. 2.4.18(c) : State transition table for an equivalent Moore machine

Example 2.4.19 : Convert the Mealy machine shown in Fig. Ex.2.4.19 to Moore machine.

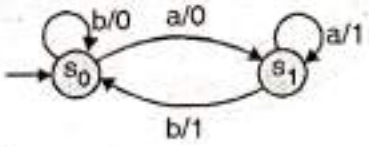


Fig. Ex. 2.4.19

Solution :

Step 1 : Splitting of states.

(i) Incoming lines into s_0 give both 0 and 1 as output. In addition, s_0 in an initial state and hence an ε-output condition should also be considered. s_0 is split into three states :

p_ε with output ε,

p_0 with output 0,

p_1 with output 1.

Scanned by CamScanner

(ii) Incoming lines into s_1 give both 0 and 1 as output. We must split s_1 into two different states.

q_0 with output 0,

q_1 with output 1.

Step 2 : Drawing of an equivalent Moore machine.

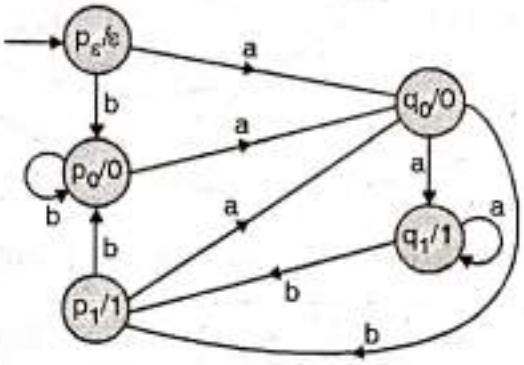


Fig. Ex. 2.4.19(a) : Moore machine for Example 2.4.19

- Transitions for s_0 are duplicated for p_ε, p_0 and p_1.
- Transitions for s_1 are duplicated for q_0 and q_1.

Example 2.4.20 : Convert the Mealy machine shown in Fig. Ex.2.4.20 to its equivalent Moore machine.

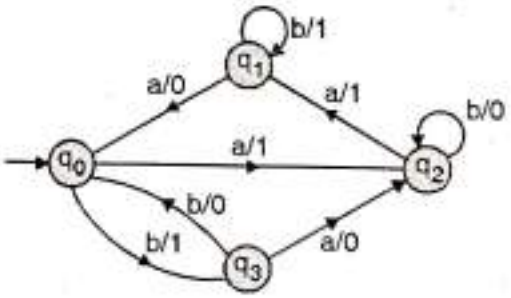


Fig. Ex. 2.4.20

Solution :

Step 1 : Splitting of states.

(i) q_0 is associated with three different outputs (on the basis of incoming lines) ε, 0, 1. We must split q_0 into three different states :

A_ε with output ε_1 .... (q_0 is a start state)

A_0 with output 0,

A_1 with output 1.

Scanned by CamScanner

(ii) q_1 is associated with two different outputs 0 and 1.

We must split q_1 into two different states :

B_0 with output 0,

B_1 with output 1.

(iii) q_2 is associated with two different outputs 0 and 1.

We must split q_2 into two different states :

C_0 with output 0,

C_1 with output 1.

(iv) q_3 is associated with two different outputs 0 and 1. We must split q_3 into two different states :

D_0 with output 0,

D_1 with output 1.

Step 2 : Drawing of an equivalent Moore machine.

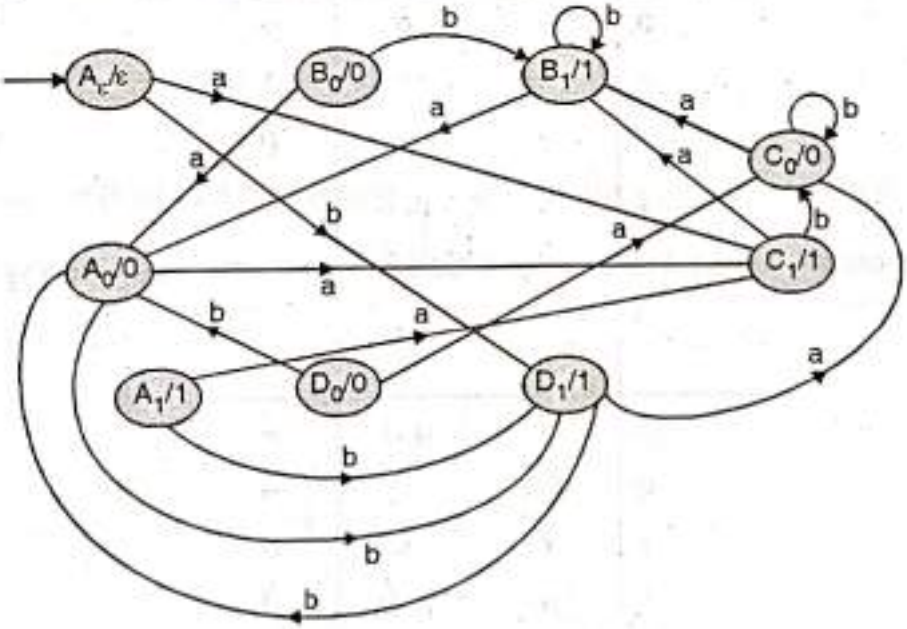


Fig. Ex. 2.4.20(a) : Mealy machine for Example 2.4.20

2.4.6 Conversion of a Moore Machine into a Mealy Machine :

MU - Dec. 2012, May 2013

A Moore machine can be transformed to a corresponding Mealy machine in two steps. These steps are :

1. Construction of a trivial Mealy machine -

   By moving output associated with a state to transitions entering into that state.

Scanned by CamScanner

2. Minimization of the trivial Mealy machine obtained in step 1.

These two steps are being explained with the help of a Moore machine shown in Fig. 2.4.10(a).

Present state	Next state		Output
	0	1	
→p	s	q	0
q	q	r	1
r	r	s	0
s	s	p	0

Fig. 2.4.10(a) : Moore machine considered for conversion

Step 1 : Construction of trivial Mealy machine.

(i) Moving the output associated p, which is 0, to transitions entering into state p, we get :

	0	1	
→p	s	q	–
q	q	r	1
r	r	s	0
s	s	p, 0	0

(ii) Moving the output associated with q, which is 1 to transitions entering into state q, we get :

	0	1	
→p	s	q, 1	–
q	q, 1	r	–
r	r	s	0
s	s	p, 0	0

(iii) Moving the output associated with r, which is 0 to transitions entering into the state r, we get :

	0	1	
→p	s	q, 1	–
q	q, 1	r, 0	–
r	r, 0	s	–
s	s	p, 0	0

Scanned by CamScanner

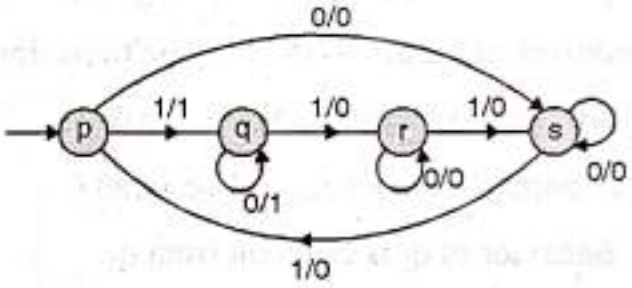
(iv) Moving the output associated with s, which is 0 to transitions entering the state s, we get :

	0	1
→p	s, 0	q, 1
q	q, 1	r, 0
r	r, 0	s, 0
s	s, 0	p, 0

Fig. 2.4.10(b) : A trivial Mealy machine

Step 2 : The trivial Mealy machine obtained in Fig. 2.4.10(b) can not be minimized further. Final Mealy machine is given in Fig. 2.4.10(c) and 2.4.10(d).

	0	1
→p	s, 0	q, 1
q	q, 1	r, 0
r	r, 0	s, 0
s	s, 0	p, 0



(c) Transition table for final Mealy machine **(d) Transition diagram for final Mealy machine**

Fig. 2.4.10

Example 2.4.21 : Change the given Moore machine into Mealy machine.

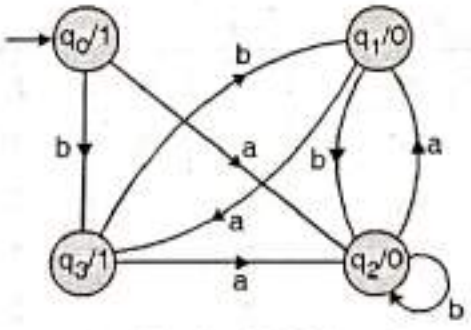
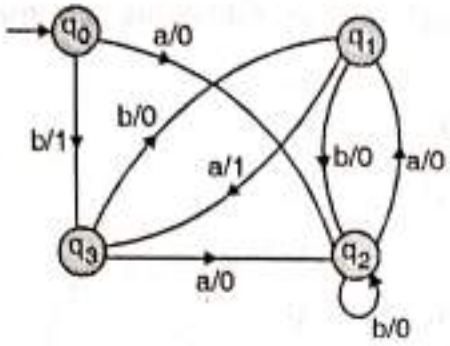


Fig. Ex. 2.4.21

Solution :

Step 1 : Construction of a trivial Mealy machine by moving output associated with a state to transitions entering that state, we get :

Scanned by CamScanner



(a)

	a	b
→q_0	q_2, 0	q_3, 1
q_1	q_3, 1	q_2, 0
q_2	q_1, 0	q_2, 0
q_3	q_2, 0	q_1, 0

(b)

Fig. Ex. 2.4.21

Step 2 : Minimization of Mealy machine obtained in Fig. Ex. 2.4.20(a).

(i) 1-equivalent partitions p_1 based on outputs

$$P_1 = (q_0)\,(q_1)\,(q_2, q_3)$$

(ii) 2-equivalent partitions P_2 based on transition,

q_2 is mapped to block (q_1) on input 0.

q_3 is mapped to block (q_2, q_3) on input 0.

$\therefore$ Behavior of q_2 is different from q_3.

$$P_2 = (q_0)\,(q_1)\,(q_2)\,(q_3)$$

Mealy machine given in Fig. Ex. 2.4.20(a) is the final Mealy machine.

Example 2.4.22 : Change the given Moore machine into Mealy machine.

Present state	Next state		Output
	0	1	
→p_0	r	q_0	ε
p_1	r	q_0	1
q_0	p_1	s_0	0
q_1	p_1	s_0	1
r	q_1	p_1	0
s_0	s_1	r	0
s_1	s_1	r	1

Solution :

Step 1 : Construction of a trivial Mealy machine by moving output associated with a state to transitions entering that state, we get :

Scanned by CamScanner

Present state	(Next state, Output)	
	0	1
$\rightarrow p_0$	r, 0	q_0, 0
p_1	r, 0	q_0, 0
q_0	p_1, 1	s_0, 0
q_1	p_1, 1	s_0, 0
r	q_1, 1	p_1, 1
s_0	s_1, 1	r, 0
s_1	s_1, 1	r, 0

Fig. Ex. 2.4.22(a) : A trivial Mealy machine

Step 2 : Minimization of the trivial Mealy machine obtained in Fig. Ex. 2.4.22(a).

(i) 1-equivalent partitions P_1 based on outputs.

$$P_1 = (p_0, p_1)\ (q_0, q_1, s_0, s_1)\ (r)$$

(ii) 2-equivalent partitions p_2 based on transitions.

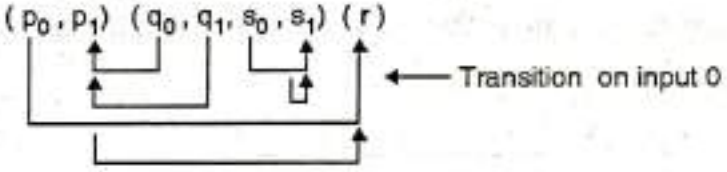


Fig. Ex. 2.4.22(b)

On input 0, q_0 and q_1 are mapped to block (p_0, p_1)

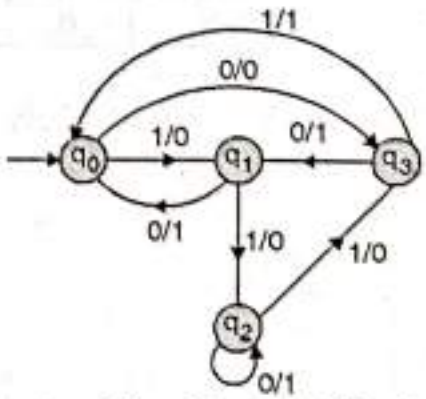
On input 0, s_0 and s_1 are mapped to block (q_0, q_1, s_0, s_1)

$\therefore$ Behavior of q_0, q_1 is different from s_0, s_1.

$$P_2 = (p_0, p_1)\ (q_0, q_1)\ (s_0, s_1)\ (r)$$

Further partitioning is not possible. The minimum state Mealy machine is given in Fig. Ex. 2.4.22(c) and 2.4.22(d)

Present state	(Next state, Output)	
	0	1
$(p_0, p_1) = \rightarrow q_0$	q_3, 0	q_1, 0
$(q_0, q_1) = q_1$	q_0, 1	q_2, 0
$(s_0, s_1) = q_2$	q_2, 1	q_3, 0
$(r) = q_3$	q_1, 1	q_0, 1



(c) State transition table of Mealy machine **(d) State transition diagram of Mealy machine**

Fig. Ex. 2.4.22

Scanned by CamScanner

Example 2.4.23 : Consider the Moore machine described by the transition table given below. Construct corresponding Mealy machine.

Present state	Next State		Output
	a = 0	a = 0	
$\rightarrow q_1$	q_1	q_2	0
q_2	q_1	q_3	0
q_3	q_1	q_3	1

Solution :

Step 1 : Construction of a trivial Mealy machine by moving output associated with a state to transitions entering that state, we get :

	0	1
$\rightarrow q_1$	q_1,0	q_2,0
q_2	q_1,0	q_3,1
q_3	q_1,0	q_3,1

Step 2 : Minimization of the trivial Mealy machine obtained in step1.

States q_2 and q_3 are equivalent as they have the same output and transition behaviour.

The minimum state machine is given below :

	0	1
$\rightarrow q_1$	q_1,0	q_2,0
$(q_2, q_3) = q_2$	q_1,0	q_2,1

Example 2.4.24 : Convert the following Moore machine to Mealy machine.

State	Input		Output
	a	b	
q_0	q_1	q_3	1
q_1	q_3	q_1	0
q_2	q_0	q_3	0
q_3	q_3	q_2	1

Solution :

Step 1 : Construction of trivial mealy machine by moving output associated with states on incoming transitions.

State	Input	
	a	b
q_0	q_1/0	q_3/1
q_1	q_3/1	q_1/0
q_2	q_0/1	q_3/1
q_3	q_3/1	q_2/0

Scanned by CamScanner

Step 2 : Minimization of trivial mealy machine.

State are grouped on the basis of output.

$(q_0)\ (q_1, q_3)\ (q_2)$

The transition behaviour of q_1 and q_3 are different under the input 'b'. Hence the states q_1 and q_3 are distinguishable.

∴ The final mealy machine is same as the trivial mealy machine and it is given Fig. Ex. 2.4.24.

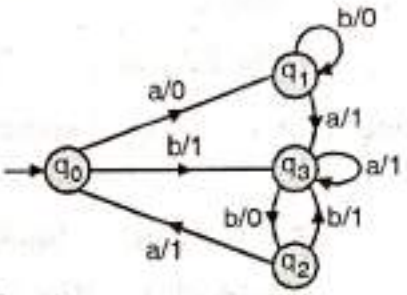


Fig. Ex. 2.4.24

2.5 Minimization of a Mealy Machine :

Minimization of a Mealy machine is almost similar to minimization of a DFA (Discussed in Chapter 3).

- In a DFA, initial grouping is based on accepting and non-accepting states.
- In a Mealy machine, initial grouping is based on outputs.

Current state	(Next state, Output)	
	0	1
$\rightarrow q_0$	$q_4, 0$	$q_3, 1$
q_1	$q_5, 0$	$q_3, 0$
q_2	$q_4, 0$	$q_1, 1$
q_3	$q_5, 0$	$q_1, 0$
q_4	$q_2, 0$	$q_5, 1$
q_5	$q_1, 0$	$q_2, 0$

Fig. 2.5.1 : Mealy machine being considered for minimization

Minimization process is being explained step wise for the Mealy machine given in Fig. 2.5.1.

Step 1 : Finding 1-equivalent partitions p_1 based on outputs.

$$P_1 = (q_0, q_2, q_4)\ (q_1, q_3, q_5)$$

q_0, q_2, q_4 have the same output behaviour i.e. 0, 1.

q_1, q_3, q_5 have the same output behaviour i.e. 0, 0.

Scanned by CamScanner

Step 2 : Finding 2-equivalent partitions P_2 based on transitions.

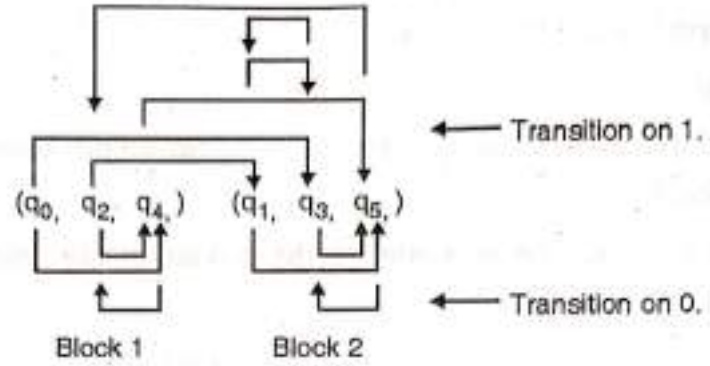


Fig. 2.5.1(a)

Behaviour of q_5 is different from q_1 and q_3. 1-successor of q_5 is block 1. 1-successor of q_1 and q_3 is block 2.

$$P_2 = (q_0, q_2, q_4) \quad (q_1, q_3) \quad (q_5)$$

Block 1 Block 2 Block 3

Step 3 : Finding 3-equivalent partitions P_3 based on transitions.

1-successor of q_0 and q_2 is the block (q_1, q_3)

1-successor of q_4 is the block (q_5).

Behaviour of q_4 is different from q_0 and q_2.

$P_3 = (q_0, q_2)\ (q_4)\ (q_1, q_3)\ (q_5)$

Further division is not possible. The minimum machine is drawn in Fig. 2.5.1(b) and 2.5.1(c).

Present state	(Next state, Output)	
	0	1
(q_0, q_2) as p_0	p_1, 0	p_2, 1
(q_4) as p_1	p_0, 0	p_3, 1
(q_1, q_3) as p_2	p_3, 0	p_2, 0
(q_5) as p_3	p_2, 0	p_0, 0

Fig. 2.5.1(b) : State table for minimum state Mealy machine

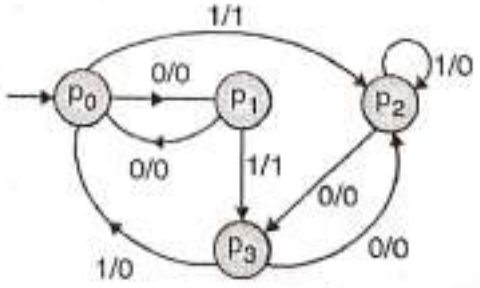


Fig. 2.5.1(c) : State diagram for minimum state Mealy machine

Scanned by CamScanner

Example 2.5.1 : Find the equivalent partition and corresponding reduced machine in standard form, for the following machine –

PS	NS, Z	
	X = 0	X = 1
A	F, 0	B, 1
B	G, 0	A, 1
C	B, 0	C, 1
D	C, 0	B, 1
E	D, 0	A, 1
F	E, 1	F, 1
G	E, 1	G, 1

PS = Present state
NS = Next state
Z = Output
X = I/P

Solution :

Step 1 : 1-equivalent partitions p_1 based on outputs

$$P_1 = (A, B, C, D, E)\ (F, G)$$

Step 2 : 2-equivalent partitions P_2 based on transitions.

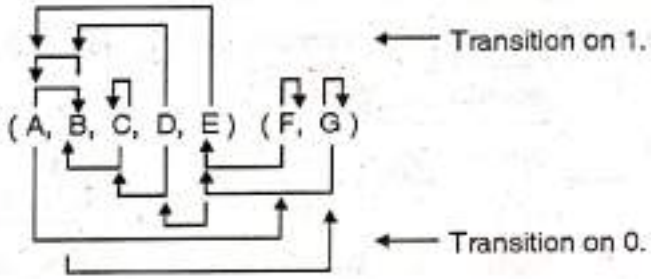


Fig. Ex. 2.5.1(a)

On input 0, A, B are mapped to block (F, G)

On output 0, C, D and E are mapped to block (A, B, C, D, E)

∴ Behavior of A, B is different from C, D, E

$$P_2 = (A, B)\ (C, D, E)\ (F, G)$$

Step 3 : 3-equivalent partitions based on transition.

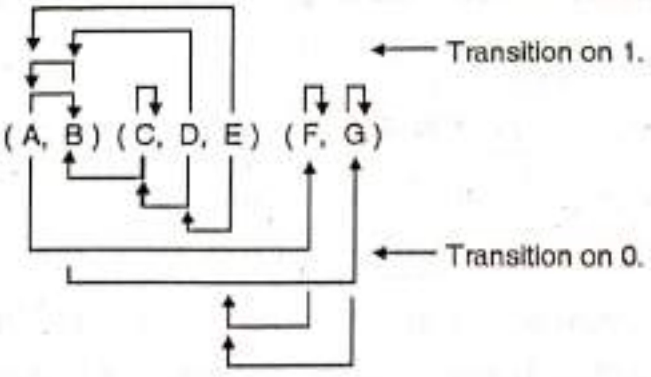


Fig. Ex. 2.5.1(b)

Scanned by CamScanner

Behaviour of C is different from D, E.

$$p_3 = (A, B)\ (C)\ (D, E)\ (F, G)$$

Step 4 : 4-equivalent partitions based on transition.

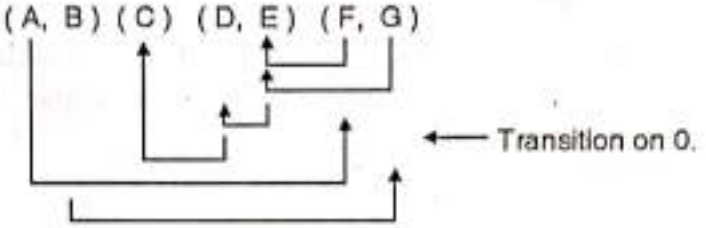


Fig. Ex. 2.5.1(c)

On input 0, D is mapped to block (C)

On input 0, E is mapped to block (D, E)

∴ Behaviour of D is different from E.

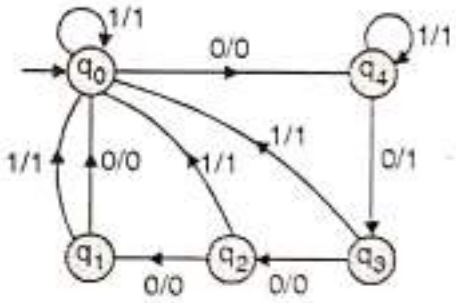
$$P_4 = (A, B)\ (C)\ (D)\ (E)\ (F, G)$$

Further division is not possible.

The minimum state Mealy machine is drawn in Fig. Ex. 2.5.1(e).

Current state	(Next state, Output)	
	0	1
(A, B) = q_0	q_4, 0	q_0, 1
(C) = q_1	q_0, 0	q_1, 1
(D) = q_2	q_1, 0	q_0, 1
(E) = q_3	q_2, 0	q_0, 1
(F, G) = q_4	q_3, 1	q_4, 1

(d) State transition table



(e) State transition diagram

Fig. Ex. 2.5.1 : Minimum state Mealy machine for Example 2.5.1

2.5.1 Difference between Moore and Mealy Machines :

MU - Dec. 2006, Dec. 2008, May 2011, May 2012, Dec. 2012, May 2013, May 2014

Sr. No.	Mealy machine	Moore machine
1.	Output is associated with transition	Output is associated with state.
2.	There is an equivalent Moore machine for every Mealy machine.	There is an equivalent Mealy machine for every Moore machine.
3.	A mealy machine can be implemented with a fewer number of states.	A Moore machine has often a larger number of states compare to the corresponding Mealy machine.

Scanned by CamScanner

Sr. No.	Mealy machine	Moore machine
4.	The transition function is given by $\Sigma \times Q \rightarrow Q$. The output function is given by, $\Sigma \times Q \rightarrow O$	The transition function is given by $\Sigma \times Q \rightarrow Q$. The output function is given by, $Q \rightarrow O$
5.	A mealy machine is seldom implemented either using circuit or program.	It is easy to implement a Moore machine using circuit or program.

2.6 University Questions and Answers :

Dec. 2005

Q. 1 Design a divisibility by four tester FSM for Binary numbers.
(Similar to Example 2.2.23) **(5 Marks)**

Q. 2 Write short note on Mealy machine. **(Sections 2.4.1 and 2.4.2)** **(5 Marks)**

May 2006

Q. 3 Design the finite state machine that compares two binary numbers to determine whether they are equal and which of the two is larger. **(Example 2.4.15)** **(8 Marks)**

Q. 4 Given the Moore and Mealy machine for the following :

For the input from Σ^*, where Σ = { 0, 1, 2 }, print the residue modulo 5 of the input treated as ternary. **(Example 2.4.1)** **(10 Marks)**

Dec. 2006

Q. 5 Define formally a deterministic finite Automata DFA and Nondeterministic Finite Automata. **(Sections 2.2.1 and 2.3.1)** **(9 Marks)**

Q. 6 Give NFA and minimized DFA for the language over {0, 1} having all strings such that the third symbol form the right end is zero.
(Similar to Example 2.2.18) (Note : Take 0 as 1 and 1 as 0) **(12 Marks)**

Q. 7 Design DFA that accepts the following language :
(i) Set of all strings with odd number of 1's followed by even number of 0's $\Sigma = \{0, 1\}$.
(ii) Set of all strings which begin and end with different letters $\Sigma = \{x, y, z\}$.
(Example 2.2.29) **(10 Marks)**

Q. 8 Compare Mealy and Moore machine. Design a Mealy machine to convert each occurrence of substring abb by aba. $\Sigma = \{a, b\}$
(Section 2.5.1, Similar to Example 4.5.9) **(10 Marks)**

Scanned by CamScanner

May 2007

Q. 9 Distinguish between NFA and DFA. **(Section 2.3.5)** **(5 Marks)**

Q. 10 Design a FSM for testing divisibility by three. **(Example 2.2.27)** **(5 Marks)**

Q. 11 Design a Moore machine to change all vowels to '$'. **(Similar to Example 2.4.11)** **(5 Marks)**

Dec. 2007

Q. 12 Design the DFA which accepts set of strings such that every string containing 00 as a substring but not 000 as a substring. **(Example 2.2.8)** **(8 Marks)**

Q. 13 Design FSM that compares two binary numbers to determine whether they are equal and which of the two are larger. **(Example 2.4.15)** **(10 Marks)**

May 2008

Q. 14 Convert the following ε-NFA to minimized DFA.

	ε	a	b	c
→p	ϕ	{p}	{q}	{r}
q	{p}	{q}	{r}	ϕ
r*	{q}	{r}	ϕ	{p}

(a) Compute the ε-closure of each state.

(b) Convert the automation to a DFA. **(Example 2.3.32)** **(10 Marks)**

Q. 15 Design the DFA for the language, contains strings in which leftmost symbol differ from rightmost symbol. Σ is given by {0, 1}. **(Example 2.2.9)** **(10 Marks)**

Q. 16 Design Moore machine to convert each occurrence of substring 1000 by 1001. **(Similar to Example 2.4.9)** **(10 Marks)**

Dec. 2008

Q. 17 Design the DFA for the language, contains strings in which leftmost symbol differ from rightmost symbol. Σ is given by {0, 1}. **(Example 2.2.9)** **(10 Marks)**

Q. 18 Compare and contrast Moore and Mealy machine. Design a Mealy machine to convert each occurrence of substring abb by aba $\Sigma = \{ a, b \}$. **(section 2.5.1, Similar to Example 2.4.9)** **(10 Marks)**

May 2009

Q. 19 Give DFA accepting the following languages over the character (0, 1)

Scanned by CamScanner

(i) The set of all strings with three consecutive zeroes

(ii) The set of all strings ending in 00. **(Similar to Example 2.2.16)** **(4 Marks)**

Q. 20 Design a Mealy machine that accepts an input from $(0 + 1)^*$, if the input ends in 101, output A; if the input ends in 110, output B, otherwise C. **(Similar to Example 2.4.13)** **(5 Marks)**

Q. 21 Write down minimization algorithm and using it find out the minimum state finite automation equivalent to the transition table given below :

	0	1
→A	B	E
B	C	F
C	D	H
D	E	H
E	F	I
F*	G	B
G	H	B
H	I	C
I*	A	E

(Similar to Example 2.2.36, Refer q_0 as A, q_1 as B, q_2 as C, q_3 as D and so on) **(10 Marks)**

Dec. 2009

Q. 22 Distinguish between NFA and DFA. **(Section 2.3.5)** **(4 Marks)**

Q. 23 Construct DFA accepting the following language : Strings ending with 110 or 111. **(Example 2.2.29)** **(5 Marks)**

May 2010

Q. 24 Define DFA and state applications of Finite Automata in brief. **(Sections 2.2.1 and 2.1.3)** **(5 Marks)**

Q. 25 Obtain a grammar to generate the language $L = \{ 0^n 1^{2n} \mid n \geq 0) \}$. **(5 Marks)**

Ans. :

Productions for the required language are as follows.

$P = \{S \rightarrow 0\,S11 \mid \varepsilon\}$

CFG for the above language is $(\{S\}, \{0, 1\}, P, S)$

Scanned by CamScanner

Q. 26 Obtain DFA to accept strings of a's and b's with even no .of a's and even no. of b's. **(Example 2.2.2) (Note : Take 0 as a and 1 as b)** **(5 Marks)**

Q. 27 Give Mealy and Moore machine for input from $\sum = \{ 0, 1 \}$, if the input ends in 101, output should be x, if input ends in 110, output should be y, otherwise output should be z. **(Example 2.4.13)** **(10 Marks)**

Dec. 2010

Q. 28 Design a DFA to accept

(i) a set of all strings with odd number of one's followed by even number of zeros

(ii) a set of all strings with which begin and end with different letters $\Sigma = \{x, y, z\}$

(Example 2.2.29) **(10 Marks)**

Q. 29 Explain different types of machines and state at least one application of each **(Sections 2.4 and 2.4.1)** **(10 Marks)**

May 2011

Q. 30 Differentiate between-

(i) NFA and DFA **(Section 2.3.5)** **(5 Marks)**

(ii) Moore and Mealy Machines. **(Section 2.5.1)**

Q. 31 Find a minimum state finite automata equivalent to the following automata

	0	1
→a	b	a
b	a	c
c	d	b
*d	d	a
e	d	f
f	g	e
g	f	g
h	h	d

(Similar to Example 2.2.35) (Note : Consider q_0 as a, q_1 as b and so on) **(10 Marks)**

Scanned by CamScanner

Dec. 2011

Q. 32 Convert the following NFA to a DFA **(Example 2.3.13)** **(5 Marks)**

	0	1
→ p	{p, q}	(p)
q	{r, s}	{t}
r	{p, r}	{t}
*s	ϕ	ϕ
*t	ϕ	ϕ

May 2012

Q. 33 Compare and Contrast Moore and Mealy Machine. **(Section 2.5.1)** **(5 Marks)**

Dec. 2012

Q. 34 Differentiate Moore and Mealy machine. **(Section 2.5.1)** **(5 Marks)**

Q. 35 What is finite automation ? Give the finite automation M accepting (a,b)*(baaa). **(Section 2.1)** **(5 Marks)**

Ans. :

The R.E. = (a, b)* (baaa), represents strings ending in baaa.

The FA is given below

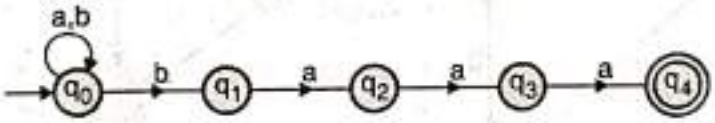


Fig. 1-Q. 35

Q. 36 Differentiate between Moore and Mealy machine with proper example and usage carry out conversion of Moore M/C to Mealy M/C. **(Sections 2.5.1 and 2.4.6)** **(10 Marks)**

May 2013

Q. 37 Differentiate between Moore and Mealy machine with proper example and usage carry out conversion of Moore M/C to Mealy M/C. **(Sections 2.4.6 and 2.5.1)** **(10 Marks)**

May 2014

Q. 38 Differentiate between NFA and DFA. **(Section 2.3.5)** **(5 Marks)**

Q. 39 Give applications of Finite Automata. **(Section 2.1.3)** **(2 Marks)**

Scanned by CamScanner

Q. 40 Compare and Contrast Moore and Mealy Machine. Construct Moore Machine to find out the residue-modulo-3 for binary numbers.

(Section 2.5.1 and Example 2.2.23) **(10 Marks)**

Ans. :

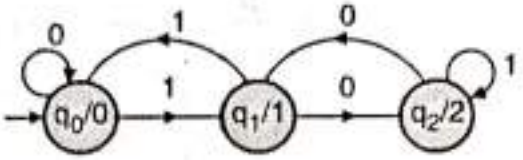


Fig. 1-Q. 40

(For details, please refer to Example 2.2.3)

Q. 41 Construct NFA that accepts a set of all strings over {a, b} ending with "abb" Convert this NFA to Equivalent DFA. **(Similar to Example 2.3.14)** **(10 Marks)**

Dec. 2014

Q. 42 State and prove equivalence of NFA and DFA. **(Theorem 2.1)** **(5 Marks)**

Q. 43 Design a DFA to accept strings over the alphabet set {a, b} that begin with 'aa' but not end with 'aa'. **(10 Marks)**

Ans. : [**Hint :** Refer as Example 2.2.6(A)]

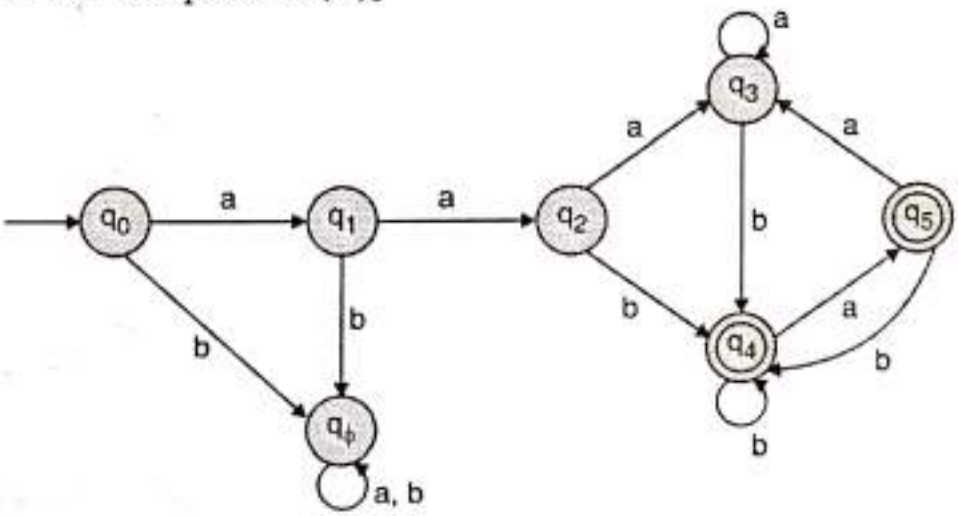


Fig. 1-Q. 43

- A string not starting with aa will reach the dead state q_ϕ.
- A string starting with aa will reach the state q_2.
- A string starting with aa and not ending in aa will be either in q_4 or q_5.

The DFA is given by,

$$M = (\{q_0, q_1, q_2, q_3, q_4, q_5, q_\phi\}, \{a, b\}, \delta, q_0, \{q_4, q_5\})$$

Scanned by CamScanner

Q. 44 Design a MOORE and MEALY machine to decrement a binary number. **(10 Marks)**

Ans. : [Hint : Refer as Example 2.4.1(A)]

One can decrement a binary by adding 11...1 (all 1's is 2's complement of 1) to the given number. The addition should start from the least significant digit.

Mealy machine :

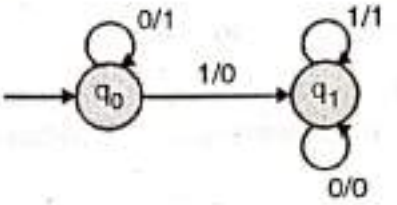


Fig. 1-Q. 44

.(q_0 – Previous carry as 0, q_1 – Previous carry as 1)

i.e., all trailing 0's are written as 1 and the first 1 is written as 0.

Moore machine :

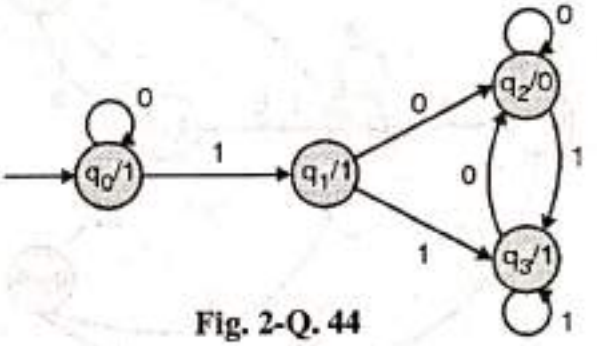


Fig. 2-Q. 44

May 2015

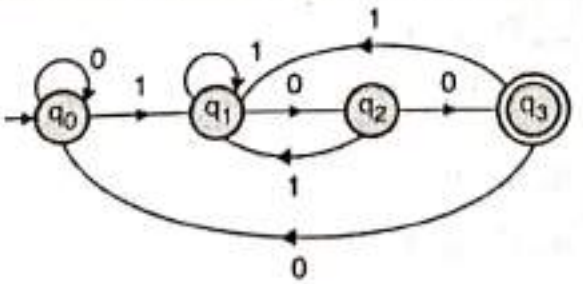
Q. 45 Differentiate between NFA and DFA. **(Section 2.3.5)** **(5 Marks)**

Q. 46 Design minimized DFA for accepting strings ending with 100 over alphabet (0,1). **(10 Marks)**

Ans. : [Hint : Refer as Example 2.2.3(A)]

All strings ending in 100 :

The substring '100' should be at the end of the string. Transitions from q_3 should be modified to handle the condition that the string has to end in '100'.



	1	0
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_3
q_3^*	q_1	q_0

(a) State transition diagram **(b) State transition table**

Fig. 1-Q. 46

Scanned by CamScanner

q_3 to q_1 on input 1 :

An input of 1 in q_3 will make the previous four characters as '1001'. Out of the four characters as '1001' only the last character '1' is relevant to '100'.

q_3 to q_0 on input 0 :

An input of 0 in q_3 will make the previous four characters '1000'. Out of the four characters '1000', nothing is relevant to '100'.

Q. 47 Design Moore Machine to generate output A if string is ending with abb, B if string ending with aba and C otherwise over alphabet (a, b). and convert it to mealy machine. **(10 Marks)**

Ans. : [Hint : Refer as Example 2.4.9(A)]

Design of Moore machine

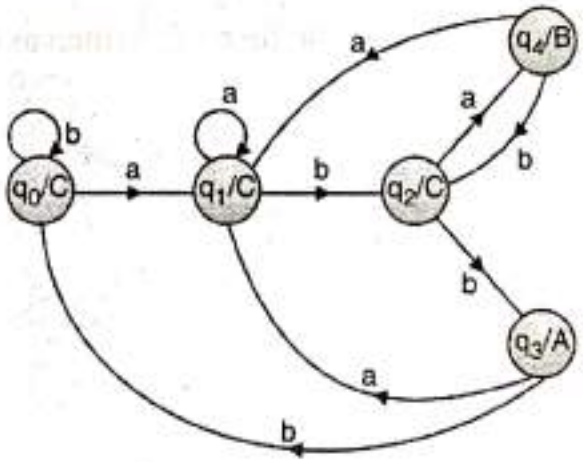


Fig. 1-Q. 47

Conversion into Mealy machine :

Step 1 : Construction of a trivial Mealy machine by moving output associated with a state to transition entering into that state.

	a	b
q_0	q_1, C	q_0, C
q_1	q_1, C	q_2, C
q_2	q_1, B	q_3, A
q_3	q_1, C	q_0, C
q_4	q_1, C	q_2, C

Step 2 : Minimization

The two states q_1 and q_4 can be merged into a single state, say q_1.

Scanned by CamScanner

	a	b
q_0	q_1, C	q_0, C
q_1	q_1, C	q_2, C
q_2	q_1, B	q_3, A
q_3	q_1, C	q_0, C

The two state q_0, q_3 can be merged into a single state, say q_0.

	a	b
q_0	q_1, C	q_0, C
q_1	q_1, C	q_2, C
q_2	q_1, B	q_0, A

The final Mealy machine is

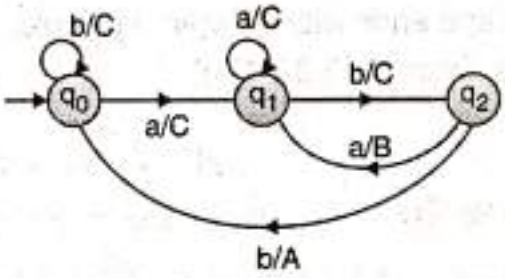


Fig. 2-Q. 47

Dec. 2015

Q. 48 Convert following ∈-NFA to NFA without ∈. **(5 Marks)**

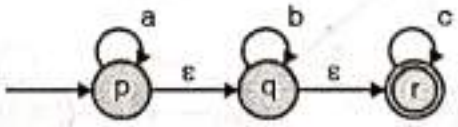


Fig. 1-Q. 48

Ans. : [Hint : Refer Example as Example 2.3.37]

To convert ∈-NFA to NFA without ∈

Step 1 : To remove ∈ transition from q state to r state, we do following

(a) Duplicate transitions of r state on q state

(b) Since r is the final state, we make q as well as the final state.

Scanned by CamScanner

Step 2 : To remove ∈ transition from p state to q state do following :

(a) Duplicate the transitions of q state on p state

(b) Since q is a final state we make p as well as the final state.

Thus, the NFA is :

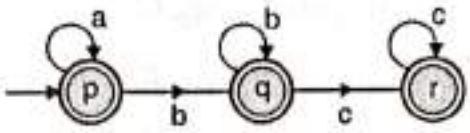


Fig. 2-Q. 48

Since all 3 states in the NFA are final states, we can merge all 3 states

∴ NFA – without ∈ is

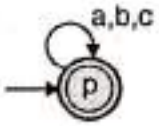


Fig. 3-Q. 48

Q. 49 Design the DFA to accept the language containing all the strings over $\sum = \{a, b, c\}$ that starts and ends with different symbols. **(10 Marks)**

Ans. : [Hint : Refer Example as Example 2.2.22(A)**]**

$M = \{Q, \Sigma, \delta, q_0, F\}$

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7\}$

$\Sigma = \{a, b, c\}$

q_0 = initial state

$F = \{q_3, q_5, q_7\}$

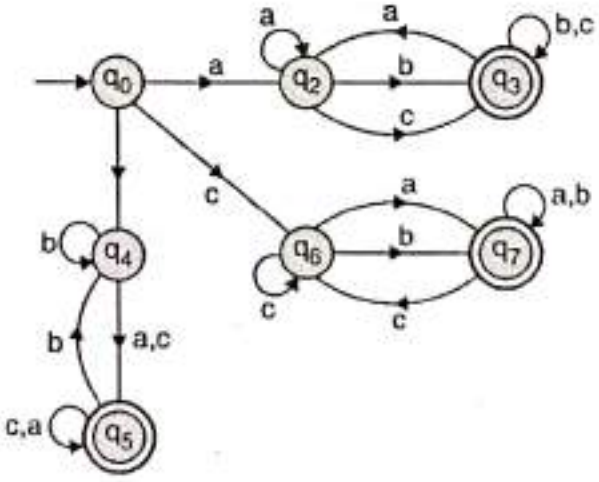


Fig. 1-Q. 49 : DFA

δ = Transitions are :

$\delta(q_0, a) \Rightarrow q_2$ $\quad$ $\delta(q_6, c) \Rightarrow q_6$

$\delta(q_0, b) \Rightarrow q_4$ $\quad$ $\delta(q_3, a) \Rightarrow q_2$

Scanned by CamScanner

$\delta(q_0, c) \Rightarrow q_6$
$\delta(q_2, a) \Rightarrow q_2$
$\delta(q_2, b) \Rightarrow q_3$
$\delta(q_2, c) \Rightarrow q_3$
$\delta(q_4, a) \Rightarrow q_5$
$\delta(q_4, c) \Rightarrow q_5$
$\delta(q_4, b) \Rightarrow q_4$
$\delta(q_6, a) \Rightarrow q_7$
$\delta(q_6, b) \Rightarrow q_7$
$\delta(q_3, b) \Rightarrow q_3$
$\delta(q_3, c) \Rightarrow q_3$
$\delta(q_5, a) \Rightarrow q_5$
$\delta(q_5, b) \Rightarrow q_4$
$\delta(q_5, c) \Rightarrow q_5$
$\delta(q_7, a) \Rightarrow q_7$
$\delta(q_7, b) \Rightarrow q_7$
$\delta(q_7, c) \Rightarrow q_6$

Q. 50 Design Moore Machine for the input from (0, 1, 2) * which print the residue modulo 5 of the input treated as ternary number. **(Example 2.4.1)** **(10 Marks)**

Q. 51 Convert the following grammar into finite automata. **(5 Marks)**

$S \rightarrow aX \mid bY \mid a \mid b$
$X \rightarrow aS \mid bY \mid b$
$Y \rightarrow aX \mid bS$

Ans. : [Hint : Refer Example as Example 2.2.22(B)]

The above grammar can be converted to FA as follows :

For every non terminating symbol we consider it as a different state

$M = \{Q, \Sigma, \delta, S, F\}$
$Q = \{S, X, Y\}$
$\Sigma = \{a, b\}$
$S =$ initial state
$F = \{X, Y\}$

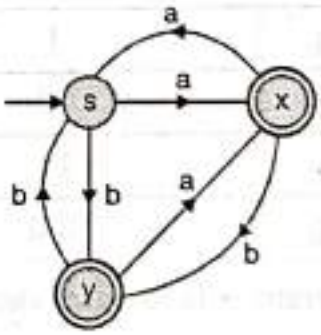


Fig. 1-Q. 51 : FA

δ : Transition functions are :

$\delta(S, a) \Rightarrow X$
$\delta(S, b) \Rightarrow Y$
$\delta(X, a) \Rightarrow S$
$\delta(X, b) \Rightarrow Y$
$\delta(Y, a) \Rightarrow X$
$\delta(Y, b) \Rightarrow S$

Scanned by CamScanner

May 2016

Q. 52 Differentiate between NFA and DFA. **(Section 2.3.5)** **(5 Marks)**

Q. 53 Design the DFA to accept all the binary strings over $\Sigma = \{0, 1\}$ that are beginning with 1 and having its decimal value multiple of 5. **(10 Marks)**

Ans. : [Hint : Refer Example as Example 2.2.22(C)**]**

Running remained is maintained through the states q_0, q_1, q_2, q_3, q_4. If the number start with 0, it is rejected.

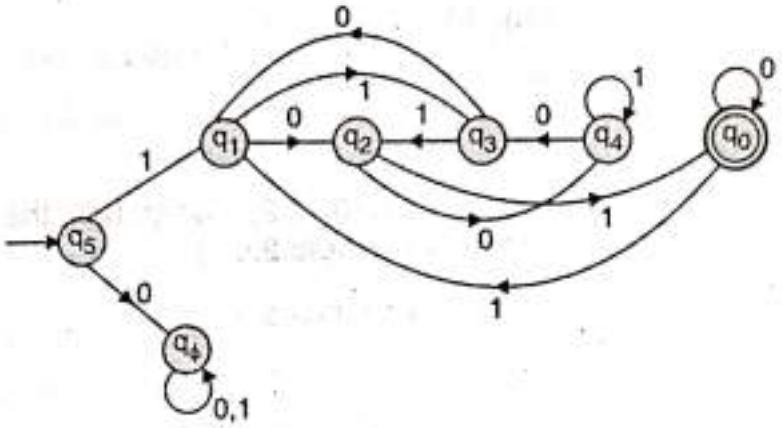


Fig. 1-Q. 53 : DFA

Reminder calculation for finding the next state

State	Binary value of the state	$\delta(q_i, 0)$	$\delta(q_i, 1)$
q_0	0	$00 \div 5 = 0(q_0)$	$01 \div 5 = 1\ (q_1)$
q_1	1	$10 \div 5 = 2\ (q_2)$	$11 \div 5 = 3\ (q_3)$
q_2	10	$100 \div 5 = 4\ (q_4)$	$101 \div 5 = 0\ (q_0)$
q_3	11	$110 \div 5 = 1\ (q_1)$	$111 \div 5 = 2\ (q_2)$
q_4	100	$1000 \div 5 = 3\ (q_3)$	$1001 \div 5 = 4\ (q_4)$

The operator ÷ is for reminder.

Q. 54 Design mealy machine to find out 2's complement of a binary number. **(10 Marks)**

Ans. : [Hint : Refer Example as Example 2.4.7(A)**]**

2's complement of a binary number

2's complement of a binary number can be found by not changing bits from right end till the first '1' and then complementing remaining bits. For example, the 2's complement of a binary number 0101101000 is calculated as given below :

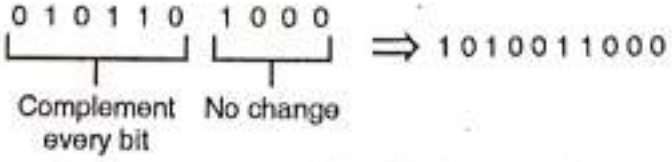


Fig. 1-Q. 54

Scanned by CamScanner

The required mealy machine is given below.

The input is entered from right to left.

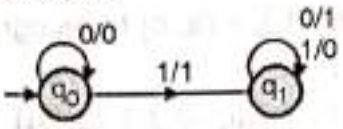


Fig. 2-Q. 54

Q. 55 Convert the following NFA to an equivalent DFA **(10 Marks)**

State	a	b	∈
→ q_0	$\{q_0, q_1\}$	$\{q_1\}$	$\{\}$
q_1	$\{q_2\}$	$\{q_1, q_2\}$	$\{\}$
*q_2	$\{q_0\}$	$\{q_2\}$	$\{q_1\}$

Ans. :

[Hint : Refer Example as Example 2.3.28(A)]

The transition graph of the given NFA is :

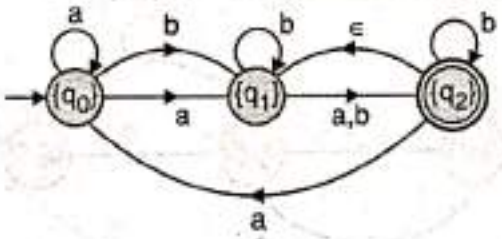


Fig. 1-Q. 55

∈-closure of states :

$q_0 \rightarrow (q_0)$

$q_1 \rightarrow (q_1)$

$q_2 \rightarrow (q_1, q_2)$

NFA to DFA using direct method.

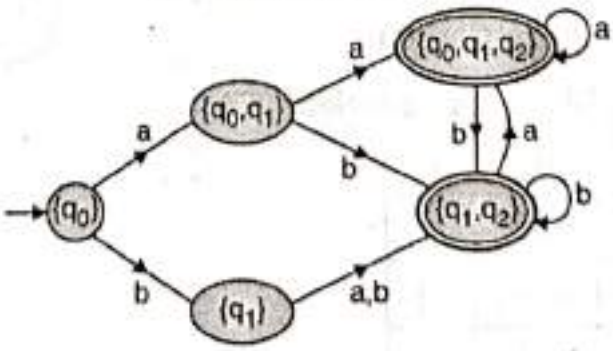


Fig. 2-Q. 55

Scanned by CamScanner

Dec. 2016

Q. 56 Design a DFA over an alphabet $\Sigma = \{a, b\}$ to recognize a language in which every 'a' is followed by 'b' **(5 Marks)**

Ans. : [Hint : Please refer example as Example 2.2.22(D)**]**

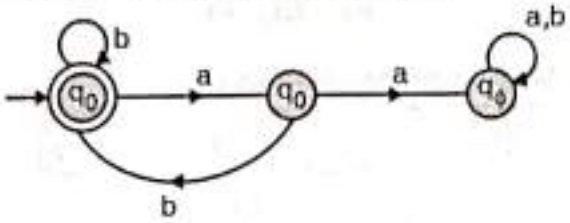


Fig. 1-Q. 56

- If 'a' is followed by 'a' then the machine enters the failure state q_ϕ
- A 'b' immediately after 'a' takes the machine to the accepting state q_0

Q. 57 Design a mealy machine to determine the residue mod 3 of a binary number. **(5 Marks)**

Ans. : [Hint : Please refer Example as Example 2.4.20(A)**]**

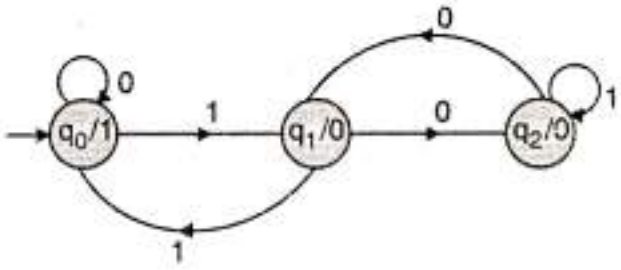


Fig 1-Q. 57

- State q_0 is for the running reminder as 0.
- State q_1 is for the running reminder as 1.
- State q_2 is for the running reminder as 2.

Output 1 indicates divisibility by 3

Output 0 indicate that the number is not divisible by 3.

$\therefore$ Required R.E. = $(0 + 1\ (1 + 01)^*\ 00)^*$

Q. 58 Convert the following NFA to an equivalent DFA **(10 Marks)**

State	a	b	∈
$\rightarrow q_0$	$\{q_0, q_1\}$	q_1	$\{\}$
q_1	$\{q_2\}$	$\{q_1, q_2\}$	$\{\}$
$^*q_2$	$\{q_0\}$	$\{q_2\}$	$\{q_1\}$

Scanned by CamScanner

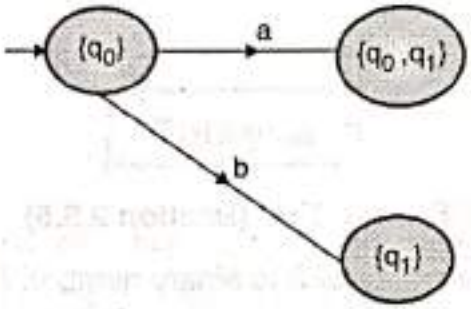
Ans. : [Hint : Please refer example as Example 2.3.28(A)]

∈ - closure of states

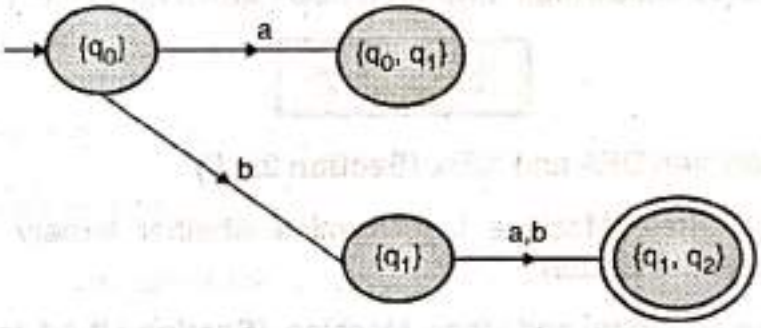
State	∈ - closure
q_0	$\{q_0\}$
q_1	$\{q_1\}$
q_2	$\{q_1, q_2\}$

Constructing DFA using the direct method

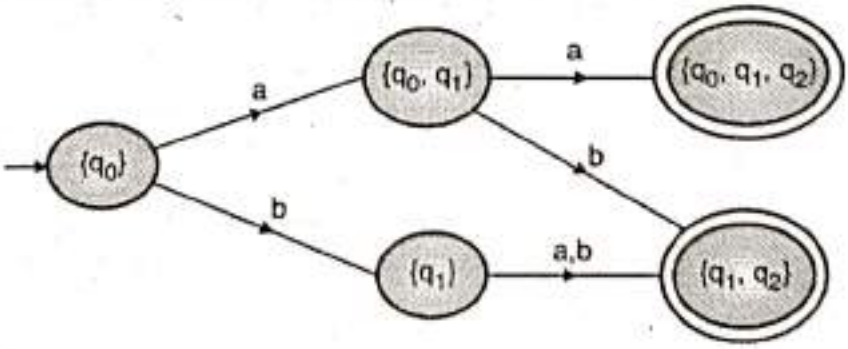
Step 1 : Transitions for the state $\{q_0\}$



Step 2 : Writing transitions for the state $\{q_1\}$

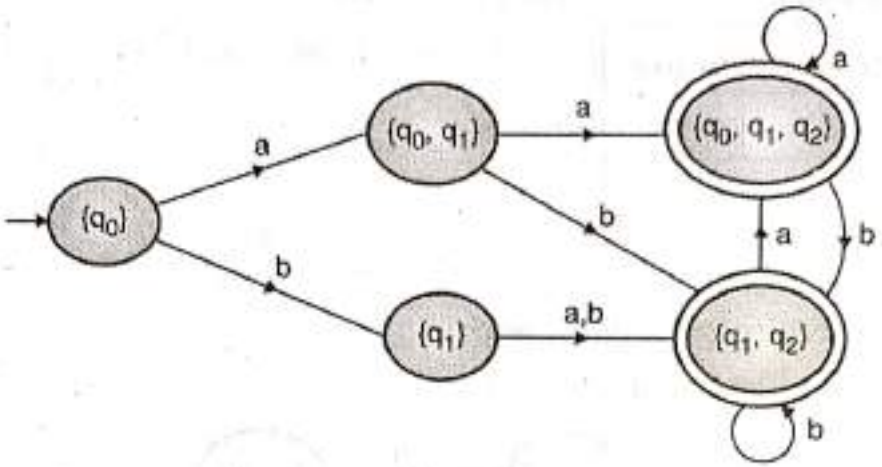


Step 3 : Writing transitions for the state $\{q_0, q_1\}$



Scanned by CamScanner

Step 4 : Writing transitions for the states $\{q_1, q_2\}$ and $\{q_0, q_1, q_2\}$



May 2017

Q. 59 Differentiate between NFA and DFA. **(Section 2.3.5)** **(5 Marks)**

Q. 60 Design FA to check divisibility by 3 to binary number. **(Example 2.2.23)** **(10 Marks)**

Q. 61 Design minimized DFA for accepting strings ending with 100 over alphabet (0, 1). **(Example 2.2.6(c), All strings ending in abb (take 0 as a and 1 as b))** **(10 Marks)**

Dec. 2017

Q. 62 Differentiate between DFA and NFA. **(Section 2.3.5)** **(5 Marks)**

Q. 63 Design a Finite State Machine to determine whether ternary number (base 3) is divisible 5. **(Example 2.2.25)** **(10 Marks)**

Q. 64 Write short note on Moore and Mealy Machine. **(Sections 2.4.1 to 2.4.4)** **(5 Marks)**

Chapter Ends...

❑❑❑

Scanned by CamScanner

CHAPTER 3

Module 2

Regular Expressions and Languages

3.1 Introduction

MU - Dec. 2010

The set of strings accepted by finite automata is known as regular language. This language can also be described in a compact form using a set of operators.

These operators are :

1) + , union operator

2) . , concatenation operator

3) * , star or closure operator.

An expression written using the set of operators (+, · , *) and describing a regular language is known as regular expression. Regular expressions for some basic automata are given in Fig. 3.1.1.

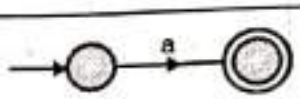
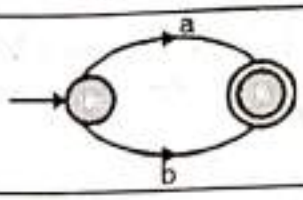
Automata	Language	Regular expression
	{ε}	R.E. = ε
	{a}	R.E. = a
	{a, b}	R.E = a + b

Fig. 3.1.1 contd...

Scanned by CamScanner

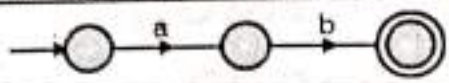
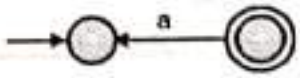
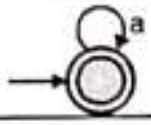
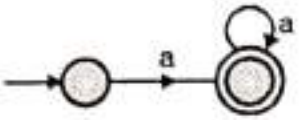
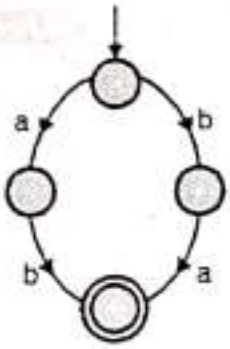
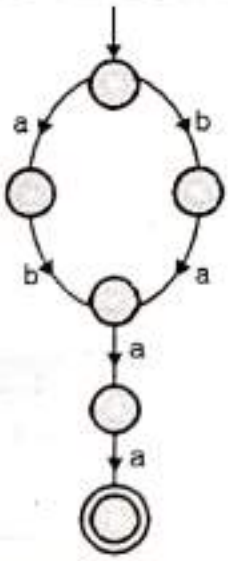
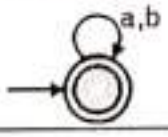
Automata	Language	Regular expression
	{ab}	R.E. = a · b or simply ab
	ϕ	R.E. = ϕ
	{ε, a, aa, aaa, ...}	R.E. = a^*
	{a, aa, aaa, ...}	R.E. = aa^* or a^+
	{ab, ba}	R.E. = ab + ba
	{abaa, baaa}	R.E. = (ab + ba) aa
	{ε, a, b, aa, ab, ba, bb, ...}	R.E. = $(a + b)^*$

Fig. 3.1.1 : Examples on regular expression

If R_1 and R_2 are regular expressions then :

$R_1 + R_2$ is also regular,

$R_1 \cdot R_2$ is also regular,

Scanned by CamScanner

R_1^* is also regular,

R_2^* is also regular,

R_1^+ is also regular.

[R_1^+ stands for one or more occurrences of R_1]

- 0* stands for a language in which a word contains zero or more 0's.
- (0 + 1)* stands for a language in which a word ω contains any combination of 0's and 1's and $|\omega| \geq 0$.

Example 3.1.1 : Write regular expression for the following languages.

(a) The set {1010}
(b) The set {10, 1010}
(c) The set {ε, 10, 01}
(d) The set {ε, 0, 00, 000, ...}
(e) The set {0, 00, 000, ...}
(f) The set of strings over alphabet {0, 1} starting with 0.
(g) The set of strings over alphabet {0, 1} ending in 1.
(h) The set of strings over alphabet {a, b} starting with a and ending in b.
(i) The set of strings recognized by $(a + b)^3$

Solution :

(a) Regular expression,

R.E = 1010 stands for the set {1010}

(b) Regular expression,

R.E. = 10 + 1010 represent the set {10, 1010}

(c) The set {ε, 10, 01} is represented by the regular expression,

R.E. = ε + 10 + 01

(d) The set {ε, 0, 00, 000, ...} is represented by the regular expression,

R.E. = 0*

(e) The set {0, 00, 000, ...} is represented by the regular expression,

R.E. = 0^+

R.E. = 00*

00* represents 0{ε, 0, 00, 000, ...} = {0, 00, 000, ...}

(f) The set of string starting with 0 is given by 0 followed by any combination of 0, 1.

R.E. = 0(0 + 1)*

(g) The regular expression for a set over alphabet {0, 1} ending in 1 is given by :

R.E. = (0 + 1)*1

Scanned by CamScanner

(h) The regular expression for a set of strings over alphabet {a, b} starting with a and ending in b is given by,

$$R.E. = a(a + b)^*b$$

(i) $(a + b)^3$ stands for $(a + b)(a + b)(a + b) = aaa + aab + aba + abb + baa + bab + bba + bbb$.

3.2 Finite Automata Representing a Regular Expression

We can always construct an ε-NFA for recognizing a set of strings represented by a regular expression.

ε-NFA can be converted directly into a DFA

OR

ε-NFA can be converted into an NFA without ε-moves which can be converted into a DFA.

A regular expression can be converted to an ε-NFA through recursive application of the operations +, · , * on alphabet Σ, ϕ and ε.

Recursive nature of regular expressions :

A regular expression $R = (1 + 0(10)^*)^*$ can be written as $R = P^*$, where $P = 1 + 0(10)^*$

P can be written as $P_1 + P_2$, where $P_1 = 1$ and $P_2 = 0(10)^*$.

P_2 can be written as $P_3 \cdot P_4$, where P_3 is 0 and P_4 is $(10)^*$.

P_4 can be written as P_5^*, where P_5 is 10.

P_5 can be written as $P_6 \cdot P_7$, where P_6 is 1 and P_7 is 0.

From the recursive nature of regular expression, we can conclude that :

If FAs for two regular expressions R_1 and R_2 are given and if we can construct composite FA for

(1) $R_1 + R_2$

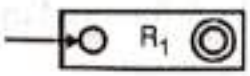
(2) $R_1 \cdot R_2$

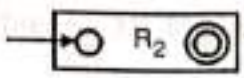
(3) R_1^*

Then we can construct finite state automata (FA) for any regular expression.

3.2.1 Composite Finite State Automata

Let us assume the existence of FAs for regular expressions R_1 and R_2.

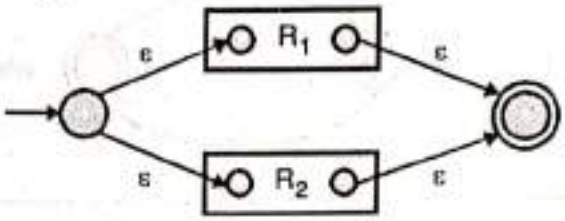
FA for R_1 is given by 

FA for R_2 is given by 

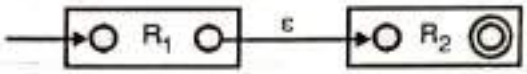
Scanned by CamScanner

Each FA is assumed to have one final state.

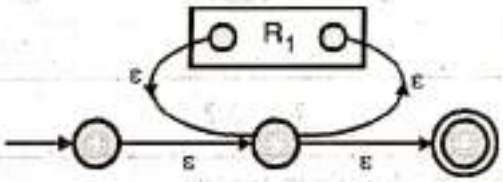
1. FA for $R_1 + R_2$ is given by



2. FA for $R_1 \cdot R_2$ is given by

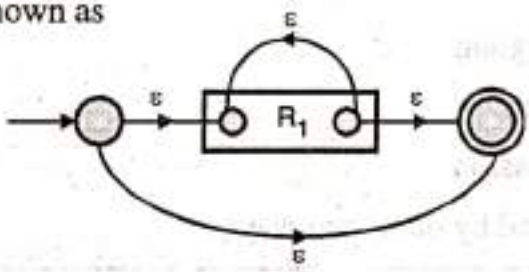


3. FA for R_1^* is given by



OR

FA for R_1^* can also be shown as



Finite automata for some primitive strings are shown in Fig. 3.2.1.

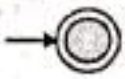
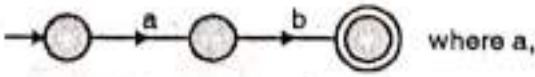
String	Finite Automata
ε	
ϕ	where $a_i \in \Sigma$
a	where $a \in \Sigma$
ab	 where $a,b \in \Sigma$

Fig. 3.2.1 contd...

Scanned by CamScanner

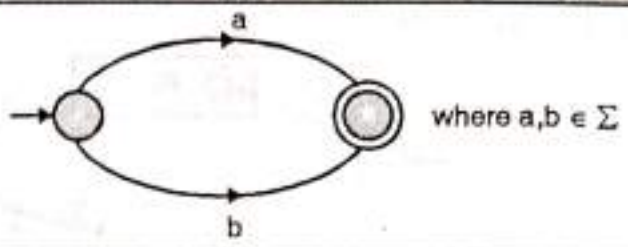
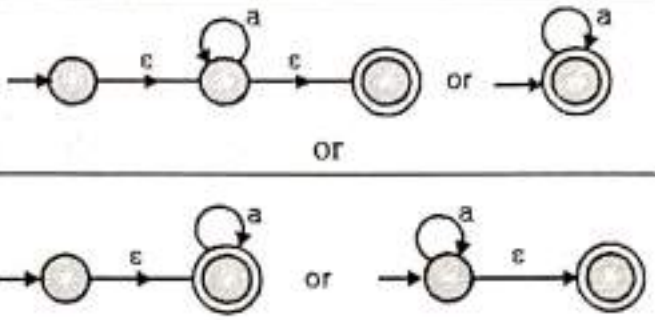
String	Finite Automata
a + b	
a*	

Fig. 3.2.1

Example 3.2.1 : Write a note on Kleen's theorem.

Solution : Kleen's theorem says that if a language can be defined by any one of these three ways :

(i) Transition Diagram

(ii) Transition Table

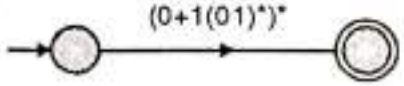
(iii) Regular Expression

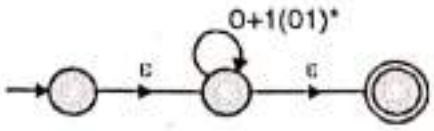
Then, it can be defined by other two ways.

Example 3.2.2 : Construct a finite automata for the regular expression

$$R = (0 + 1(01)^*)^*$$

Solution :

Step 1 : 

Step 2 : 

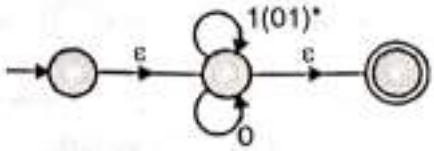
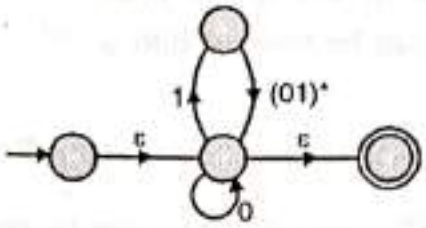
Step 3 : 

Fig. Ex. 3.2.2 cont...

Scanned by CamScanner

Step 4 :



Step 5 :

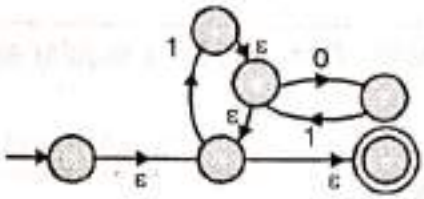
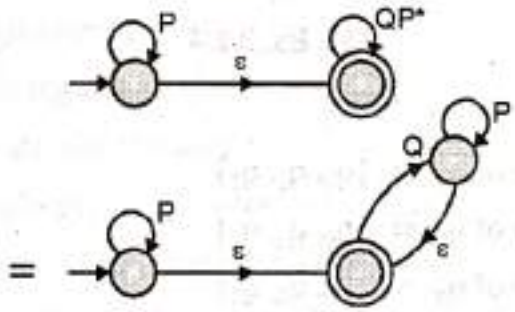


Fig. Ex. 3.2.2

Example 3.2.3 : Show that $P^*(QP^*)^* = (P + Q)^*$

Solution :

A finite automata recognizing $P^*(QP^*)^*$ is given by :



Naming the states, we get the Fig. Ex. 3.2.3.

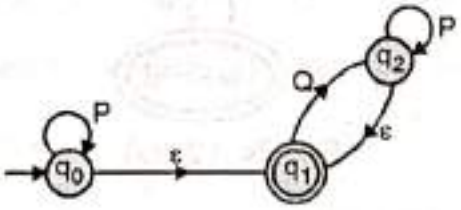


Fig. Ex. 3.2.3 : A finite automata for $P^*(QP^*)^*$

ε-move from q_0 to q_1 can be removed by of

1. Duplicating transitions of q_1 on q_0
2. Making q_0 as a final state.

ε-move from q_2 to q_1 can be removed by

1. Duplicating transitions of q_1 on q_2
2. Making q_2 as a final state.

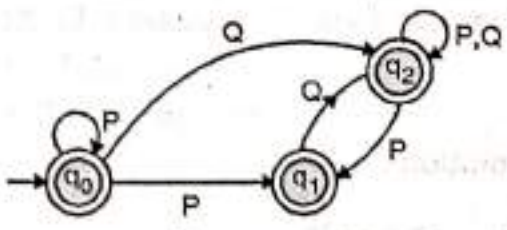


Fig. Ex. 3.2.3(a) : An equivalent finite automata without ε-moves

Scanned by CamScanner

Since all the three states q_0, q_1 and q_2 are final states (in Fig. Ex. 3.2.3(a)), they can be merged into a single state.

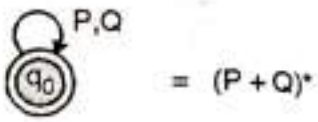


Fig. Ex. 3.2.3(b) : An equivalent DFA

Thus both expressions $P^*(QP^*)^*$ and $(P + Q)^*$ can be recognized by equivalent DFA and hence we can infer that $P^*(QP^*)^* = (P + Q)^*$

Example 3.2.4 : Prove or disprove the following for a regular expression.

$$(r^*s^*)^* = (r + s)^*$$

Solution :

Step 1 : ε-NFA for $(r^*s^*)^*$

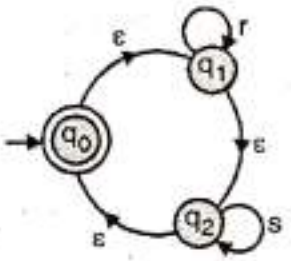


Fig. Ex. 3.2.4

Step 2 : ε-NFA to DFA

$$\varepsilon\text{-closure of } q_0 = \{q_0, q_1, q_2\}$$
$$\varepsilon\text{-closure of } q_1 = \{q_0, q_1, q_2\}$$
$$\varepsilon\text{-closure of } q_2 = \{q_0, q_1, q_2\}$$

The equivalent DFA is given shown in Fig. Ex. 3.2.4(a).

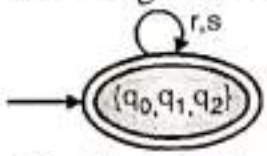


Fig. Ex. 3.2.4(a)

Step 3 : The regular expression for the DFA

$$= (r + s)^*$$
$$= \text{R.H.S.}$$

Example 3.2.5 : Describe in English the language represented by following expressions :

(i) $(a + ab)^*$ (ii) $(a + b)^*a\,(a + b)^*$

(iii) $(a^*ab^*ab^*) + b^*$ (iv) $a^+b^*c^*$

Solution :

(i) $(a + ab)^*$:

→ The language is "Every b is preceded by an a".

Scanned by CamScanner

(ii) $(a + b)^* a (a + b)^*$:

→ The language is "The string contains at least one a".

(iii) $(a^*ab^*ab^*) + b^*$:

→ The language is "The string contains at least two a's or a string of only b's including null string".

(iv) $a^+b^*c^*$:

→ The language is "One or more a's followed by any number of b's followed by one or more number of c's".

3.3 Determination of Regular Expression

MU - Dec. 2010, May 2013, Dec. 2017

Formal definition :

A regular expression over alphabet

$q = (a_1, a_2, \ldots a_n)$ is defined recursively as follows :

1. An empty set ϕ is a regular expression.
2. A null string ε is a regular expression.
3. An alphabet $a_i \in \Sigma$ is a regular expression.
4. If r_1 and r_2 are regular expression then :
   (a) Their concatenation $r_1 \cdot r_2$ is a regular expression.
   (b) Their union $r_1 + r_2$ is a regular expression.
   (c) Closure of r_1, i.e. r_1^* is a regular expression.
5. A regular expression is generated through a finite number of applications of above rules.

3.3.1 Language Generated by a Regular Expression

If r is a regular expression then the language generated by r is given by L(r).

Example :

If $r_1 = 0 + 1$ then $L(r_1) = \{0, 1\}$

If $r_2 = 0^*$ then $L(r_2) = \{\varepsilon, 0, 00, 000, \ldots\}$

If $r_3 = (0 + 1)^*$ then $L(r_3) = \{\varepsilon, 0, 1, 00, 01, 10, 11, \ldots\}$

3.3.2 Basic Properties of Regular Expressions

Some of the basic identities for regular expressions are given below. P, Q and R are regular expressions.

$$\phi + R = R \quad \ldots(3.1)$$

$$\phi \cdot R = R \cdot \phi = \phi \quad \ldots(3.2)$$

Scanned by CamScanner

$$\varepsilon \cdot R = R \cdot \varepsilon = R \quad \ldots(3.3)$$

$$\varepsilon^* = \varepsilon \quad \ldots(3.4)$$

$$\phi^* = \varepsilon \quad \ldots(3.5)$$

$$R + R = R \quad \ldots(3.6)$$

$$PQ + PR = P(Q + R) \quad \ldots(3.7)$$

$$QP + RP = (Q + R)P \quad \ldots(3.8)$$

$$R^*R^* = R^* \quad \ldots(3.9)$$

$$RR^* = R^*R \quad \ldots(3.10)$$

$$(R^*)^* = R^* \quad \ldots(3.11)$$

$$\varepsilon + RR^* = R^* \quad \ldots(3.12)$$

$$(PQ)^*P = P(QP)^* \quad \ldots(3.13)$$

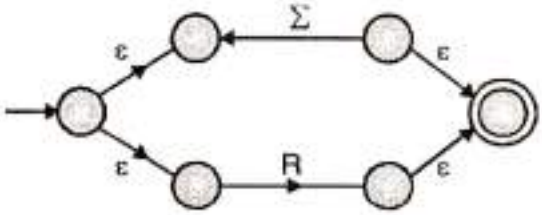
$$(P + Q)^* = (P^*Q^*)^* = (P^* + Q^*)^* \quad \ldots(3.14)$$

$$(P^*Q)^* = \varepsilon + (P + Q)^*Q \quad \ldots(3.15)$$

Proof for basic properties :

1. $\phi + R = R$

   Machine for $\phi + R$ is given by

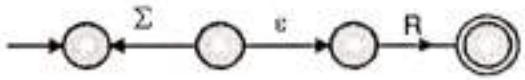


There is only one path from the start state to the final state, recognizing $\varepsilon \cdot R \cdot \varepsilon = R$.

Hence, $\phi + R = R$

2. $\phi \cdot R = \phi$

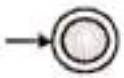
   A composite machine for $\phi \cdot R$ is given by



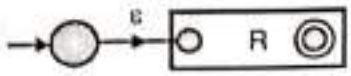
There is no way of reaching the final state from the start state.

$\therefore \quad \phi \cdot R = \phi$

3. $\varepsilon \cdot R = R$

   FA for ε is given by 

   FA for R is given by

   Composite FA for $\varepsilon \cdot R$ is given by 

Scanned by CamScanner

This machine accepts R and only R.

$$\therefore \quad \varepsilon \cdot R = R.$$

4. $\varepsilon + RR^* = R^*$

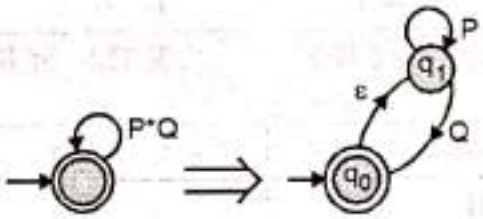
$$\begin{aligned} \text{L.H.S.} &= \varepsilon + RR^* \\ &= \varepsilon + R\{\varepsilon + R + RR + RRR + \ldots\} \\ &= \varepsilon + R + RR + RRR + \ldots \\ &= R^* = \text{R.H.S.} \end{aligned}$$

5. $(PQ)^*P = P(QP)^*$

$$\begin{aligned} \text{L.H.S.} &= (PQ)^* P \\ &= \{\varepsilon + PQ + (PQ)^2 + (PQ)^3 + \ldots\} P \\ &= \{\varepsilon + PQ + PQPQ + PQPQPQ + \ldots\} P \\ &= \{P + PQP + PQPQP + \ldots\} \\ &= P\{\varepsilon + QP + QPQP + \ldots\} \\ &= P(QP)^* = \text{R.H.S.} \end{aligned}$$

6. $(P^*Q)^* = \varepsilon + (P + Q)^*Q$

Finite automata for $(P^*Q)^*$ is given by :



ε-move can be removed by duplicating transitions of q_1 on q_0.

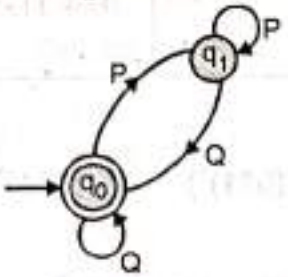
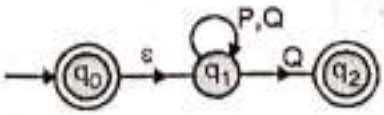


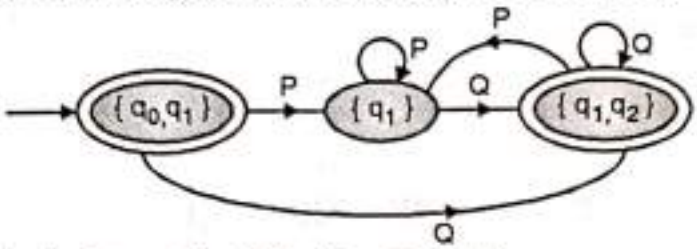
Fig. 3.3.1 : FA for $(P^*Q)^*$

Finite automata for $\varepsilon + (P + Q)^*Q$ is given by



Scanned by CamScanner

An equivalent DFA is being constructed using direct method.



Behaviour of both $\{q_0, q_1\}$ and $\{q_1, q_2\}$ are identical and they can be merged into a single state say q_0.

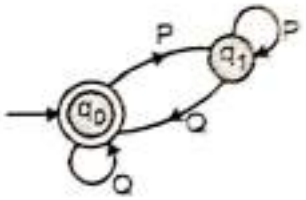


Fig. 3.3.2 : FA for $\varepsilon + (P + Q)^*Q$

DFAs for $(P + Q)^*$ given in Fig. 3.3.1 and $\varepsilon + (P + Q)^*Q$ given in Fig. 3.3.2 are same.

$$\therefore (P^*Q)^* = \varepsilon + (P + Q)^*Q.$$

Example 3.3.1 : Give the examples of sets that demonstrates the following inequality. Here r_1, r_2, r_3 are regular expression :

(1) $r_1 + \varepsilon \neq r_1$ (2) $r_1 \cdot r_2 \neq r_2 \cdot r_1$

(3) $r_1 \cdot r_1 \neq r_1$ (4) $r_1 + (r_2 \cdot r_3) \neq (r_1 + r_2) \cdot (r_1 \cdot r_3)$

Solution : Let us make the following assumptions :

$r_1 = 0$ $r_2 = 1$ $r_3 = 1$

Sr. No.	R.E.	L.H.S. of R.E.	R.H.S. of R.E.	Remark
1.	$r_1 + \varepsilon \neq r_1$	$0 + \varepsilon$	0	L.H.S. ≠ R.H.S.
2.	$r_1 \cdot r_1 \neq r_1$	0.0	0	L.H.S. ≠ R.H.S.
3.	$r_1 \cdot r_2 \neq r_2 \cdot r_1$	0.1	1.0	L.H.S. ≠ R.H.S.
4.	$r_1 (r_2 \cdot r_3) \neq (r_1 + r_2) (r_1 \cdot r_3)$	0 + 10	$(0 + 1) \cdot 00 = 000 + 100$	L.H.S. ≠ R.H.S.

Example 3.3.2 : Prove that

$$\varepsilon + 1^*(011)^*(1^*(011)^*)^* = (1 + 011)^*$$

Solution :

$$\begin{aligned}
\text{L.H.S.} &= \varepsilon + 1^*(011)^*(1^*(011)^*)^* \\
&= \varepsilon + RR^*, \text{ where R is taken as } 1^*(011)^* \\
&= R^* \qquad \ldots \text{[By Equation (3.12)]} \\
&= (1^*(011)^*)^* \\
&= (1 + 011)^* \qquad \ldots \text{[By Equation (3.14)]} \\
&= \text{R.H.S.}
\end{aligned}$$

Scanned by CamScanner

Example 3.3.3 : Prove that
$(1 + 00^*1) + (1 + 00^*1)(0 + 10^*1)^*(0 + 10^*1) = 0^*1(0 + 10^*1)^*$

MU - May 2006, Dec. 2007

Solution :

$$\begin{aligned}
\text{L.H.S.} &= (1 + 00^*1) + (1 + 00^*1)(0 + 10^*1)^*(0 + 10^*1) \\
&= (1 + 00^*1)[\varepsilon + (0 + 10^*1)^*(0 + 10^*1)] && \text{... [By Equation (3.7)]} \\
&= (1 + 00^*1)(0 + 10^*1)^* && \text{... [By Equation (3.12)]} \\
&= [(\varepsilon + 00^*)1](0 + 10^*1)^* && \text{...[By Equation (3.8)]} \\
&= 0^*1(0 + 10^*1)^* && \text{... [By Equation (3.12)]} \\
&= \text{R.H.S.}
\end{aligned}$$

Example 3.3.4 : Prove or disprove the given regular expression $(rs + r)^* = r(sr + r)^*$

Solution :

ε and rs are in $(rs + r)^*$

but ε and rs $\notin$ $r(sr + r)^*$

Hence, $(rs + r)^* \neq r(sr + r)^*$

Example 3.3.5 : If s = {aa, b} write all the strings in s^* which are having length 4 or less, also say the following is true or false.

(i) $(s^+)^* = (s^*)^*$ (ii) $(s^+)^+ = s^+$ (iii) $(s^*)^+ = (s^+)^*$

Solution : We have to find all strings of length 4 or less in $(aa + b)^*$

(i) String of length 0 = {ε}

(ii) String of length 1 = {b}

(iii) String of length 2 = {aa, bb}, taking either 'aa' one time or b two times.

(iv) String of length 3 = {bbb, baa, aab}, taking either b 3 times or b and aa one time each with permutations.

(v) String of length 4 = {aaaa, bbbb, bbaa, aabb, baab},

taking aa two times,

taking b four times,

taking aa one time and b two times with permutations.

∴ Required strings are {ε, b, aa, bb, bbb, baa, aab, aaaa, bbbb, bbaa, aabb, baab}

(i) $(s^+)^* = (s^*)^*$ – **True**, both $(s^+)^*$ and $(s^*)^*$ contain ε

(ii) $(s^+)^+ = s^+$ – **True**, both $(s^+)^+$ and s^+ do not contain ε

(iii) $(s^*)^+ = (s^+)^*$ – **True**, both $(s^*)^+$ and $(s^+)^*$ contain ε.

Scanned by CamScanner

Example 3.3.6 : Find all possible regular expression $L \subseteq (a, b)^*$

(a) the set of all strings ending in b.
(b) the set of all strings ending in ba.
(c) the set of all strings ending neither in b nor in ba.
(d) the set of all strings ending in ab
(e) the set of all strings ending neither in ab nor ba.

Solution :

(a) The set of all strings ending in b.

$$R.E. = (a + b)^*b$$

(b) The set of all strings ending in ba.

$$R.E. = (a + b)^*ba$$

(c) The set of all strings ending neither in b nor in ba.

$$R.E. = (a + b)^*aa + a + \varepsilon$$

(d) The set of all strings ending in ab

$$R.E. = (a + b)^*ab$$

(e) The set of all strings ending neither in ab nor ba.

$$R.E. = \varepsilon + a + b + (a + b)^*(aa + bb)$$

Example 3.3.7 : Find a regular expression corresponding to each of the following subsets of $\{0, 1\}^*$

(1) The language of all strings containing exactly two 0's.
(2) The language of all strings containing at least two 0's.
(3) The language of all strings that do not end with 01.

Solution :

(1) Strings containing exactly two 0's

$$R.E. = 1^*01^*01^*$$

(2) Strings containing at least two 0's

$$R.E. = 1^*01^*0\,(1 + 0)^*$$

(3) Strings not ending in 01.

$$R.E. = (1 + 0)^*(00 + 11 + 10)$$

Example 3.3.8 : Construct NFA with ε-moves for R.E. = 0.1[$(10^* + 111)^* + 0$]*1. Convert it to DFA using direct method of conversion from NFA with ε-moves to DFA.

Solution : The ε-NFA is being constructed recursively as shown in Fig. Ex. 3.3.8.

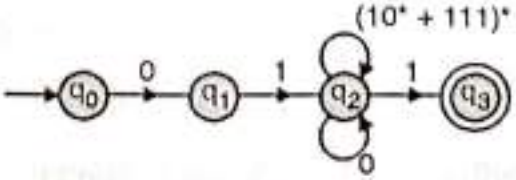


Fig. Ex. 3.3.8 Contd...

Scanned by CamScanner

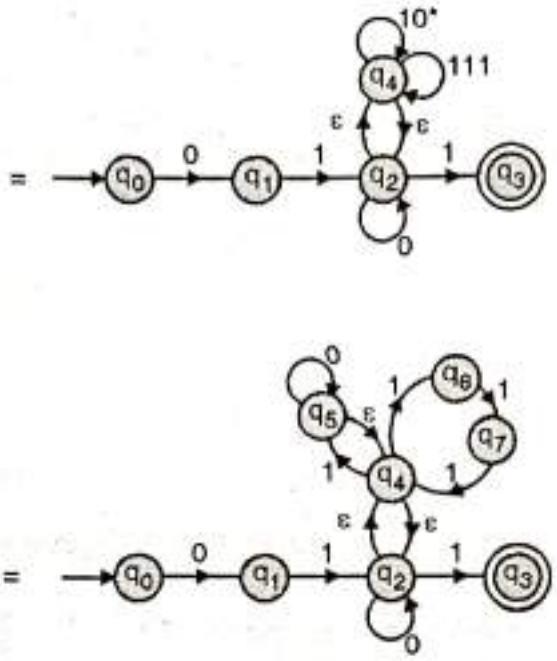


Fig. Ex. 3.3.8 : ε-NFA for Example 3.3.8

DFA is being constructed from ε-NFA. It is shown in Fig. Ex. 3.3.8(a).

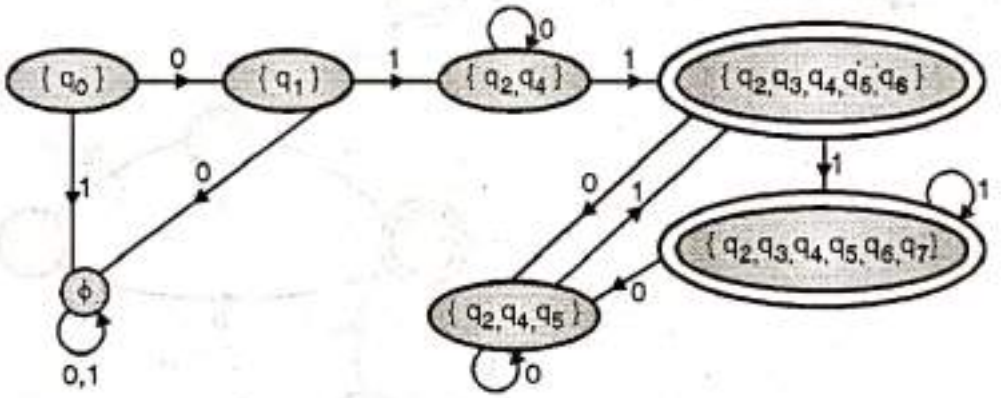


Fig. Ex. 3.3.8(a) : DFA for Example 3.3.8

Example 3.3.9 : Construct an NFA to accept the language represented by a*b*c*. Convert the NFA to its equivalent DFA.

Solution :

Step 1 : RE to NFA

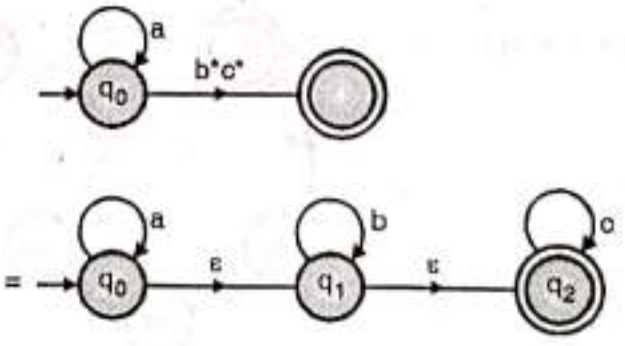


Fig. Ex. 3.3.9(a)

Scanned by CamScanner

Step 2 : NFA to DFA

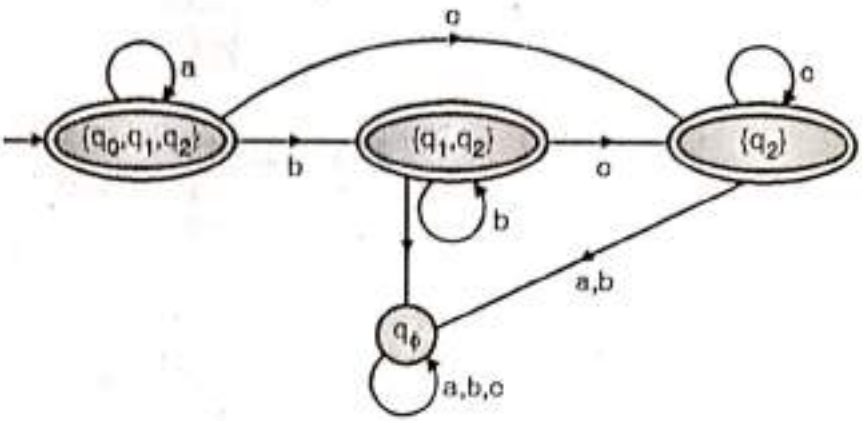
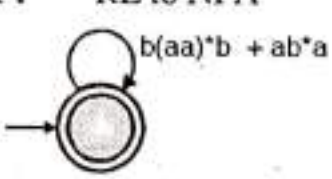
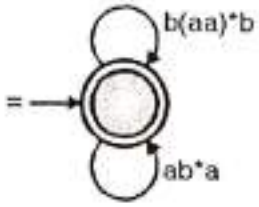


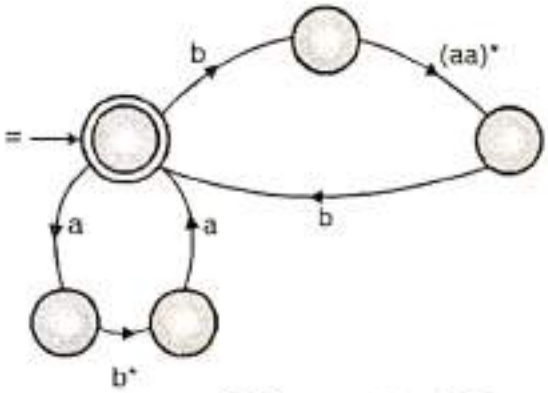
Fig. Ex. 3.3.9(b)

Example 3.3.10 : Construct an NFA with ε-moves for the regular expression (b(aa)*b + ab*a)*
Convert this NFA to its equivalent DFA. **MU - May 2006, Dec. 2011**

Solution : Step 1 : RE to NFA







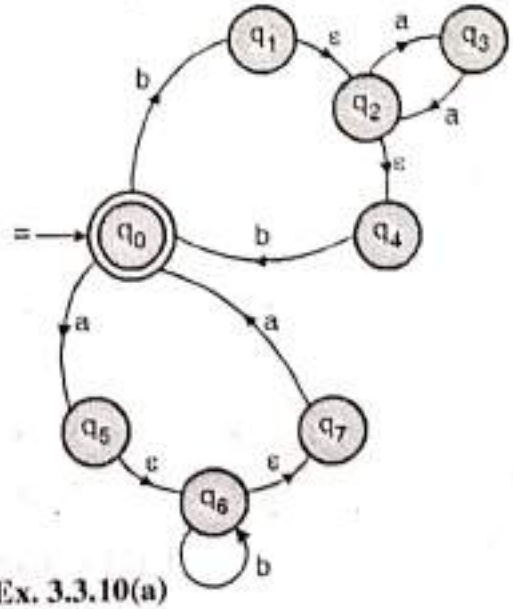


Fig. Ex. 3.3.10(a)

Scanned by CamScanner

Step 2 : NFA to DFA

ε-closure of states :

State	ε-closure
q_0	$\{q_0\}$
q_1	$\{q_1, q_2, q_4\}$
q_2	$\{q_2, q_4\}$
q_3	$\{q_3\}$
q_4	$\{q_4\}$
q_5	$\{q_5, q_6, q_7\}$
q_6	$\{q_6, q_7\}$
q_7	$\{q_7\}$

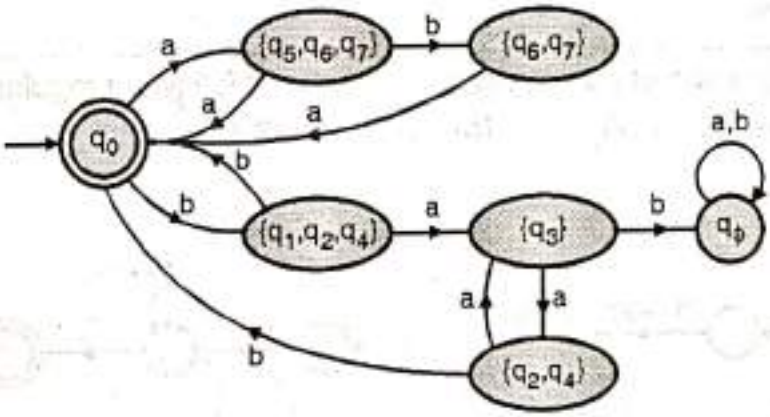


Fig. Ex. 3.3.10(b)

Example 3.3.11 : Which of the following are true ? Explain.

(i) baa ∈ $a^*b^*a^*b^*$ (iii) $a^*b^* \cap b^*c^* = \phi$

(ii) $b^*a^* \cap a^*b^* = a^* \cup b^*$ (iv) abcd ∈ $(a\,(cd)^*b)^*$

Solution :

(i) baa ∈ $a^*b^*a^*b^*$

$$a^*b^*a^*b^* = (\varepsilon + a + aa + \ldots)(\varepsilon + b + \ldots)(\varepsilon + a + aa + \ldots)(\varepsilon + b + bb + \ldots)$$

By taking ε-from the first set,
b-from the second set
aa from the third set
and ε from the fourth set

We can say that, baa ∈ $a^*b^*a^*b^*$. Answer is **True.**

(ii) $b^*a^* \cap a^*b^* = a^* \cup b^*$

$$b^*a^* = (\varepsilon + bb^*)(\varepsilon + aa^*)$$ ...[From Equation (3.12)]

Scanned by CamScanner

$$= \varepsilon + aa^* + bb^* + bb^*aa^*$$
$$= (\varepsilon + aa^*) + (\varepsilon + bb^*) + bb^*aa^*$$
$$= a^* + b^* + bb^*aa^* \quad \ldots(Q11.1)$$

Similarly, $a^*b^* = a^* + b^* + aa^*bb^*$ ...(Q11.2)

Taking intersection of equations (Q11.1) and (Q11.2)

$$b^*a^* \cap a^*b^* = (a^* + b^* + bb^*aa^*) \cap (a^* + b^* + aa^* bb^*)$$
$$= a^* + b^* = a^* \cup b^*$$

Answer is **True**.

(iii) $a^*b^* \cap b^*c^* = \phi$

Both a^*b^* and b^*c^* contain the common string b^*.

Answer is **False**.

(iv) $abcd \in (a(cd)^*b)^*$

A string belonging to $(a(cd)^*b)^*$ can not end in cd.

Answer is **False**.

Example 3.3.12 : For each of the following draw DFA of following regular expressions :

(a) $(11 + 00)^*$ (b) $(111 + 100)^*0$

Solution :

(a) DFA for $(11 + 00)^*$

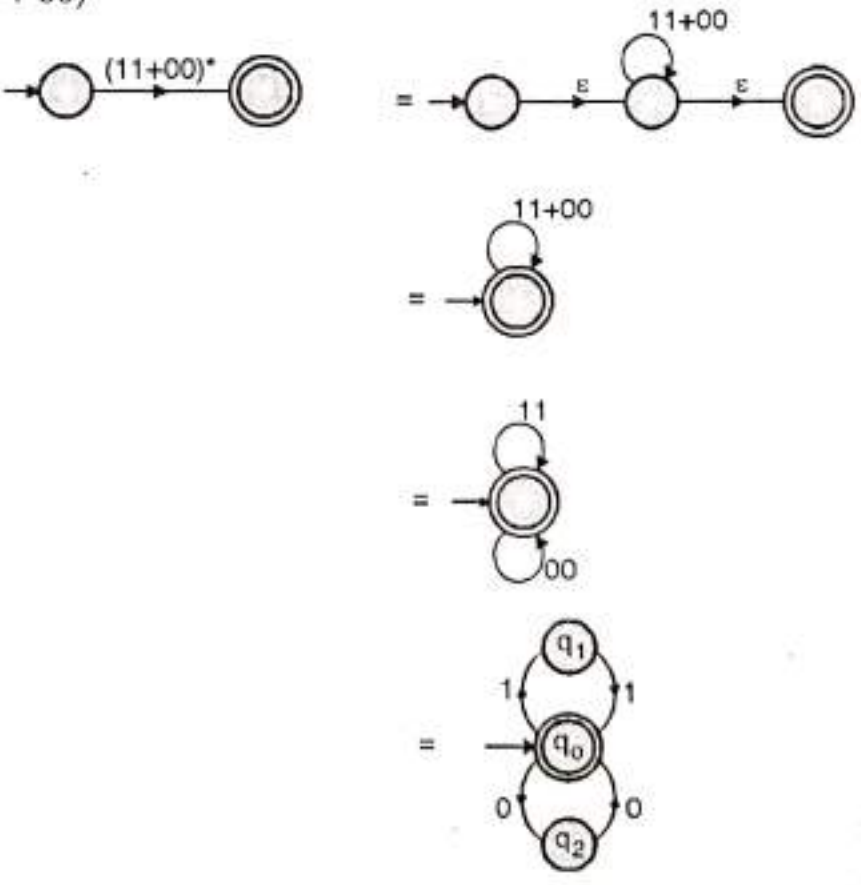


Fig. Ex. 3.3.12(a)

Scanned by CamScanner

(b) DFA for (111 + 100)*0

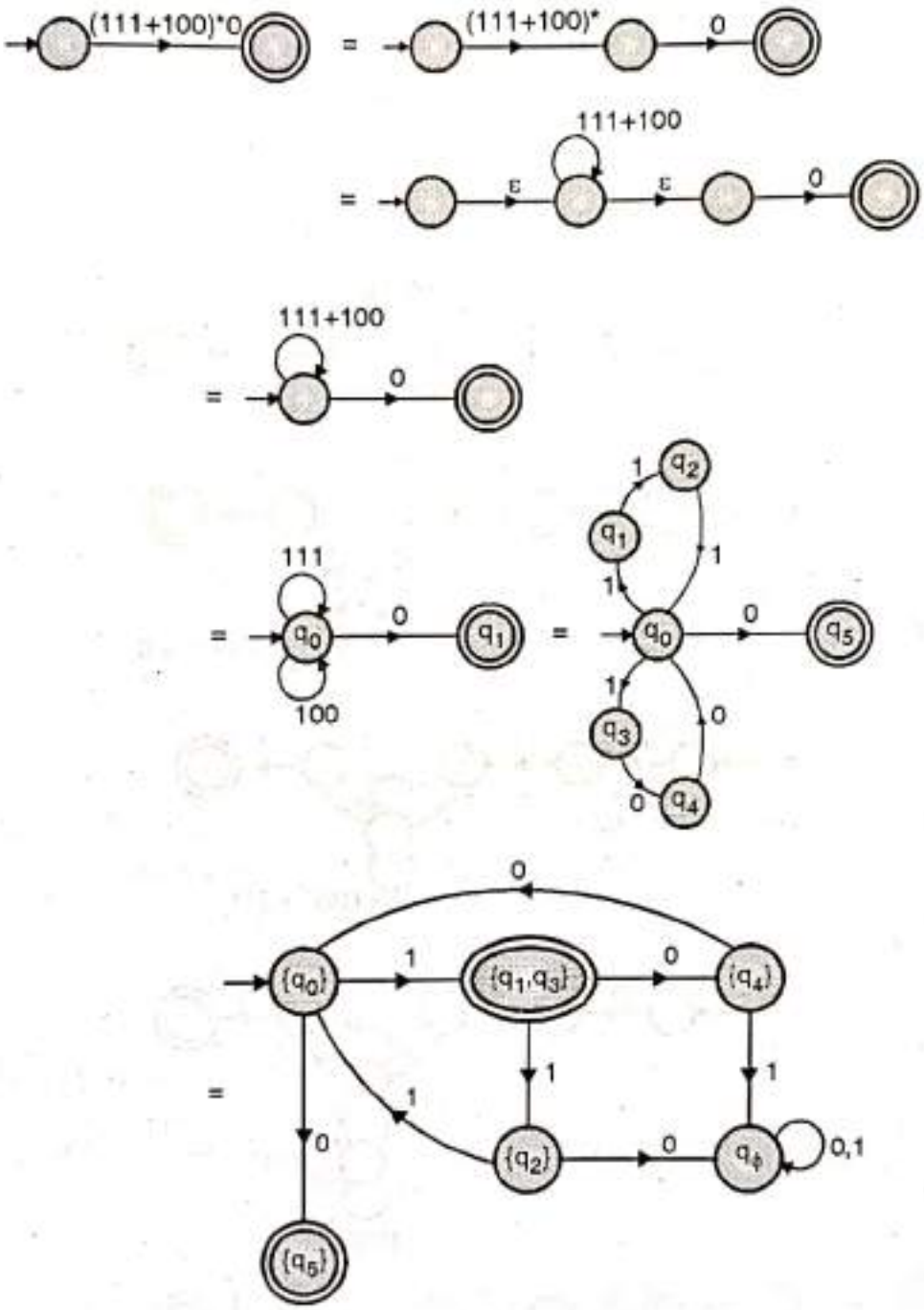


Fig. Ex. 3.3.12(b)

Example 3.3.13 : Construct finite automata equivalent to the following regular sets.

(a) $10 + (0 + 11)0^*1$

(b) $01[((10)^* + 111)^* + 0]^*1$

(c) $[00 + 11 + (01 + 10) (00 + 11)^*(01 + 10)]^*$ **MU - Dec. 2009**

Solution :

(a) FA for $10 + (0 + 11) 0^*1$

Scanned by CamScanner

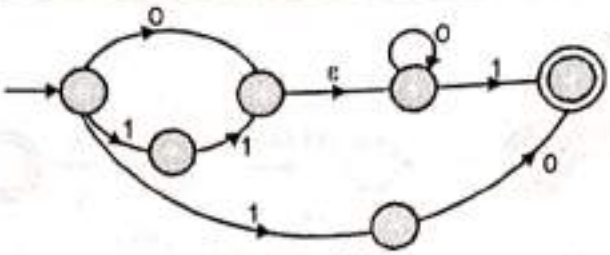


Fig. Ex. 3.3.13

(b) FA for $01[((10)^* + 111)^* + 0]^*1$

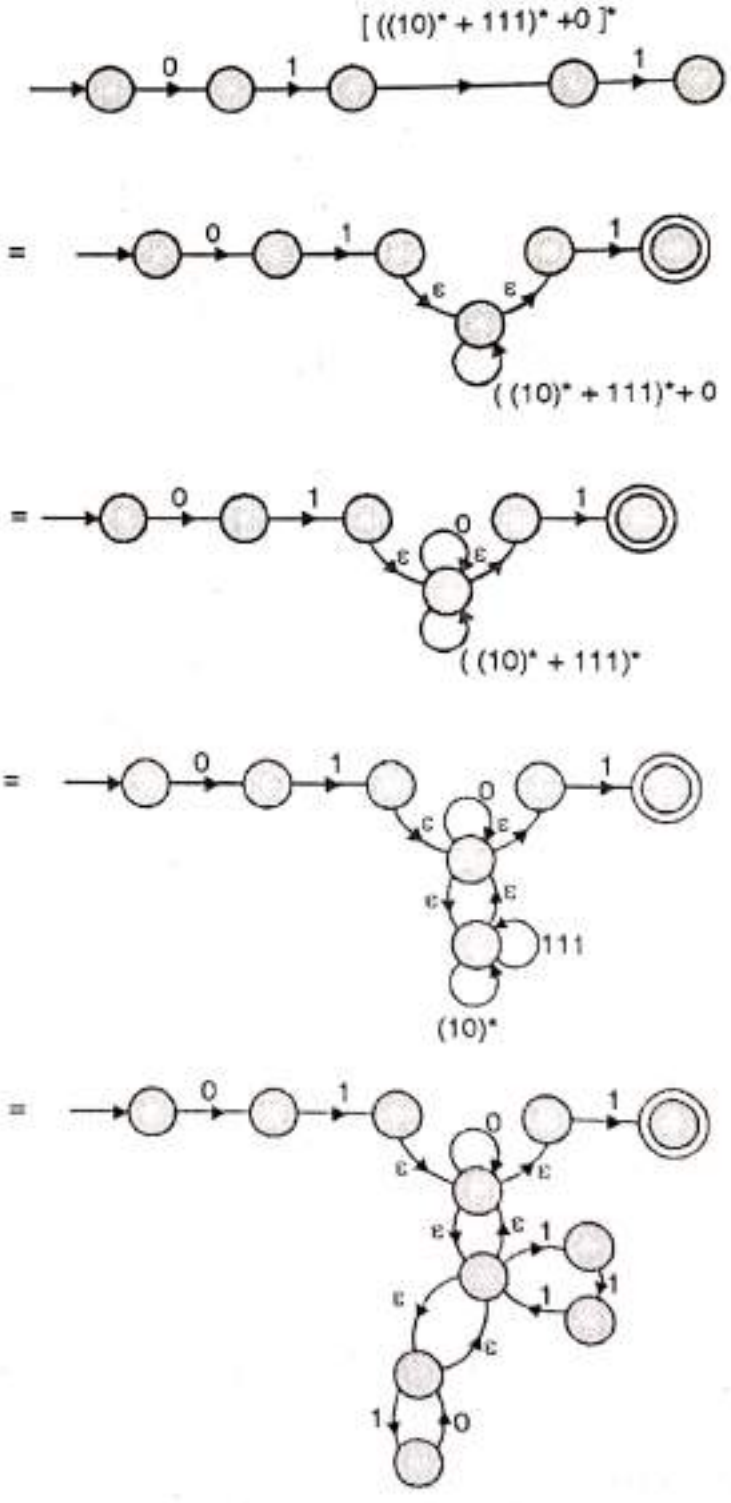


Fig. Ex. 3.3.13(a)

Scanned by CamScanner

(c) FA for $[00 + 11 + (01 + 10)(00 + 11)^*(01 + 10)]^*$

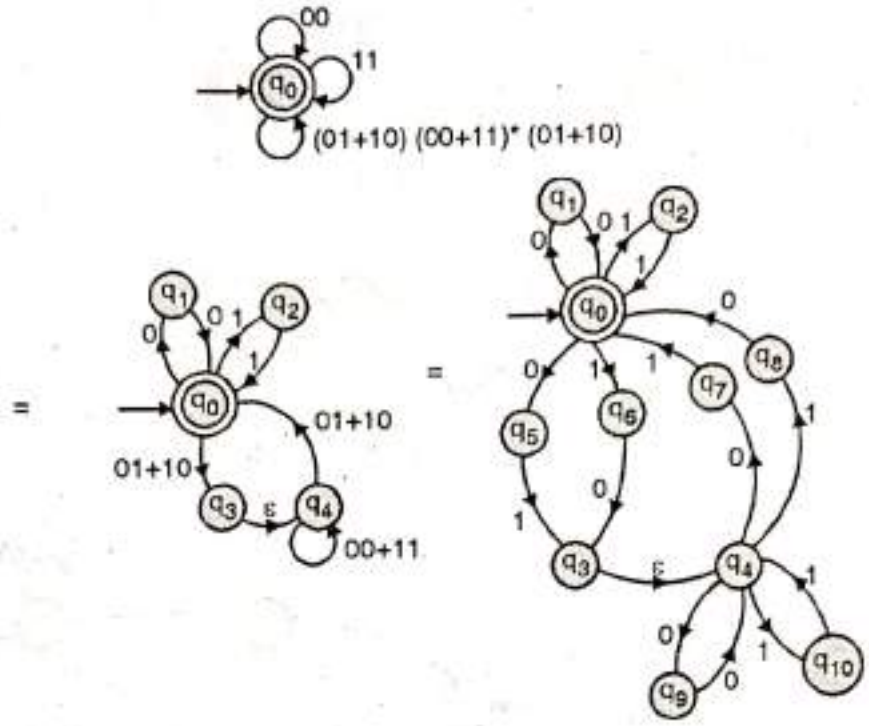


Fig. Ex. 3.3.13(b)

Example 3.3.14 : For each of the following regular expression, draw a finite automata recognizing the corresponding language.

1. $(1 + 10 + 110)^*0$
2. $1(01 + 10)^* + 0(11 + 10)^*$
3. $(010 + 00)^*(10)$
4. $1(1 + 10)^* + 10(0 + 01)^*$

Solution :

1. F.A. for $(1 + 10 + 110)^*0$

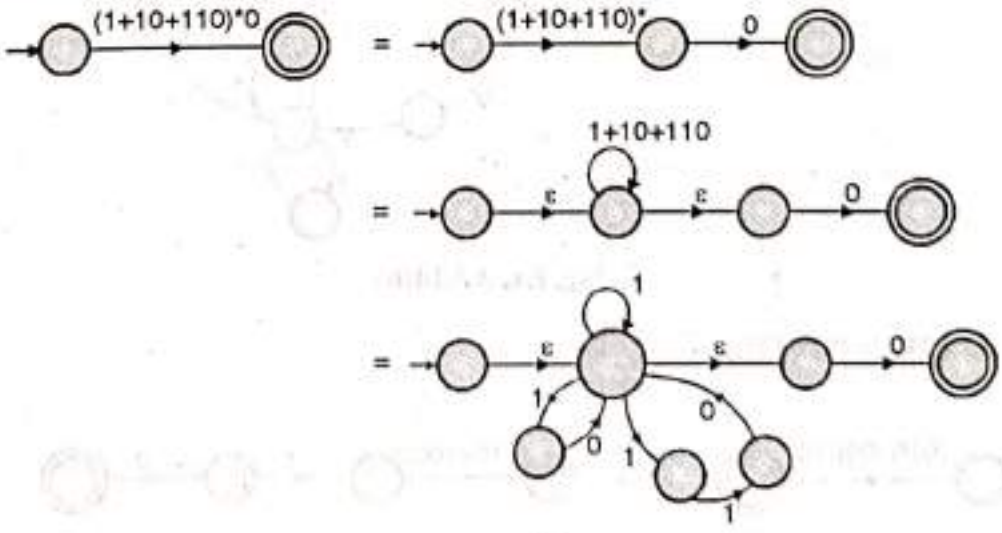


Fig. Ex. 3.3.14(a)

Scanned by CamScanner

2. F.A. for $1(01 + 10)^* + 0(11 + 10)^*$

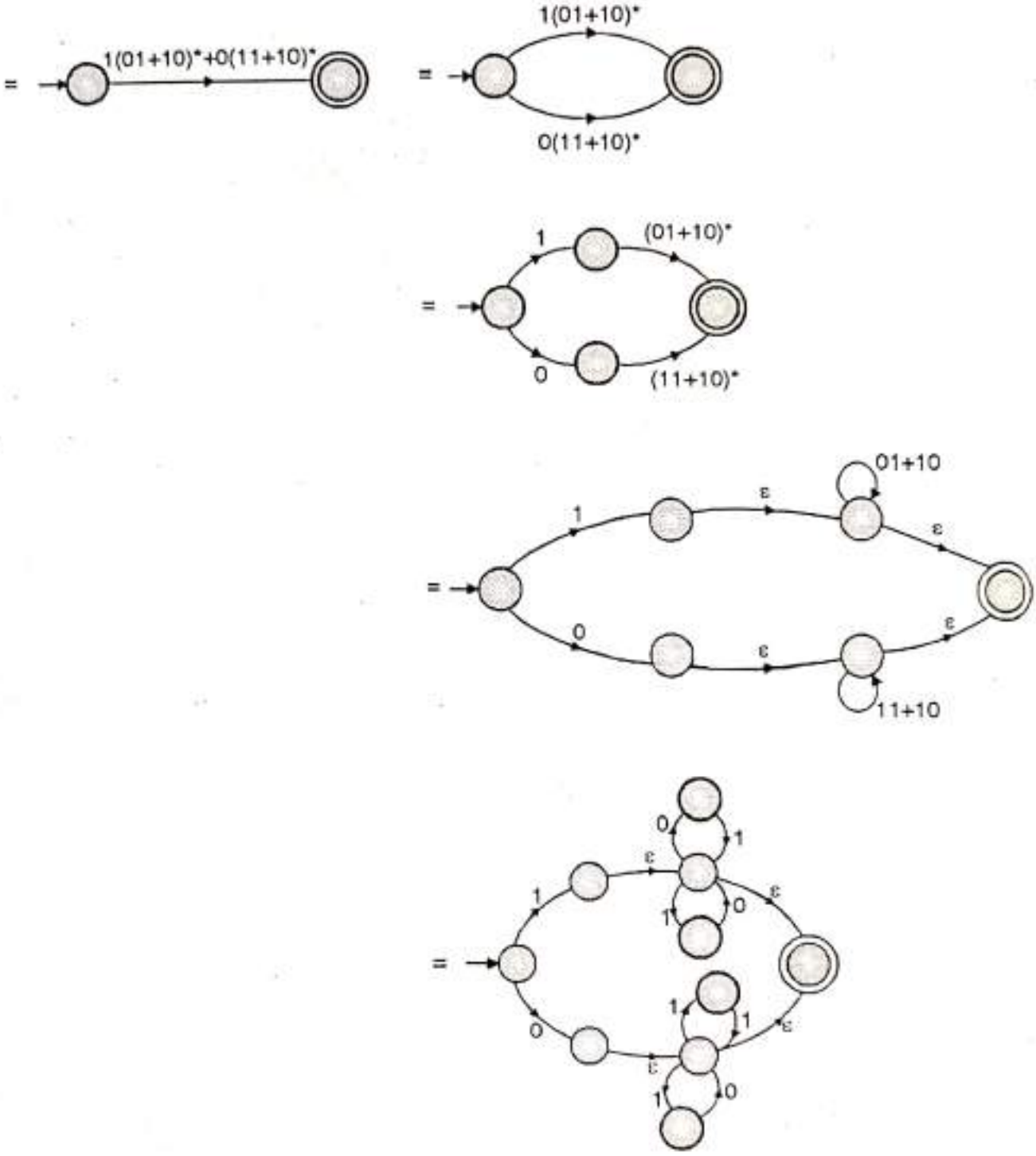


Fig. Ex. 3.3.14(b)

3. F.A. for $(010 + 00)^*(10)$

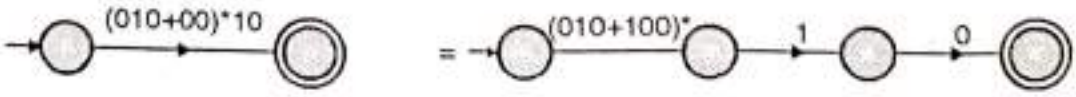


Fig. Ex. 3.3.14(c) contd....

Scanned by CamScanner

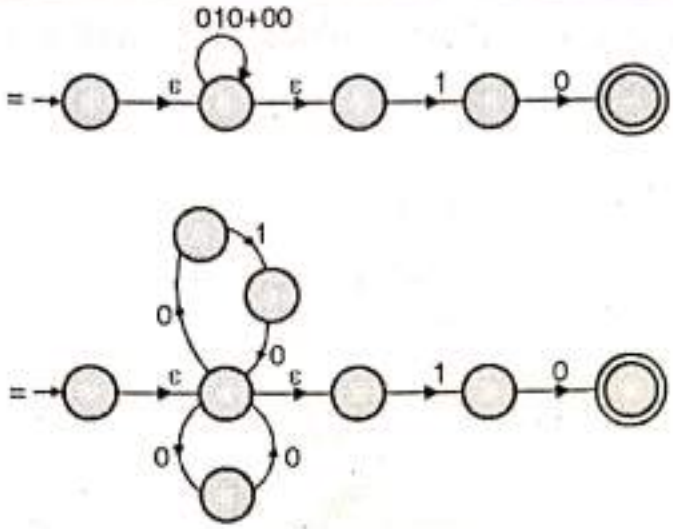


Fig. Ex. 3.3.14(c)

4. F.A. for $1(1 + 10)^* + 10(0 + 01)^*$

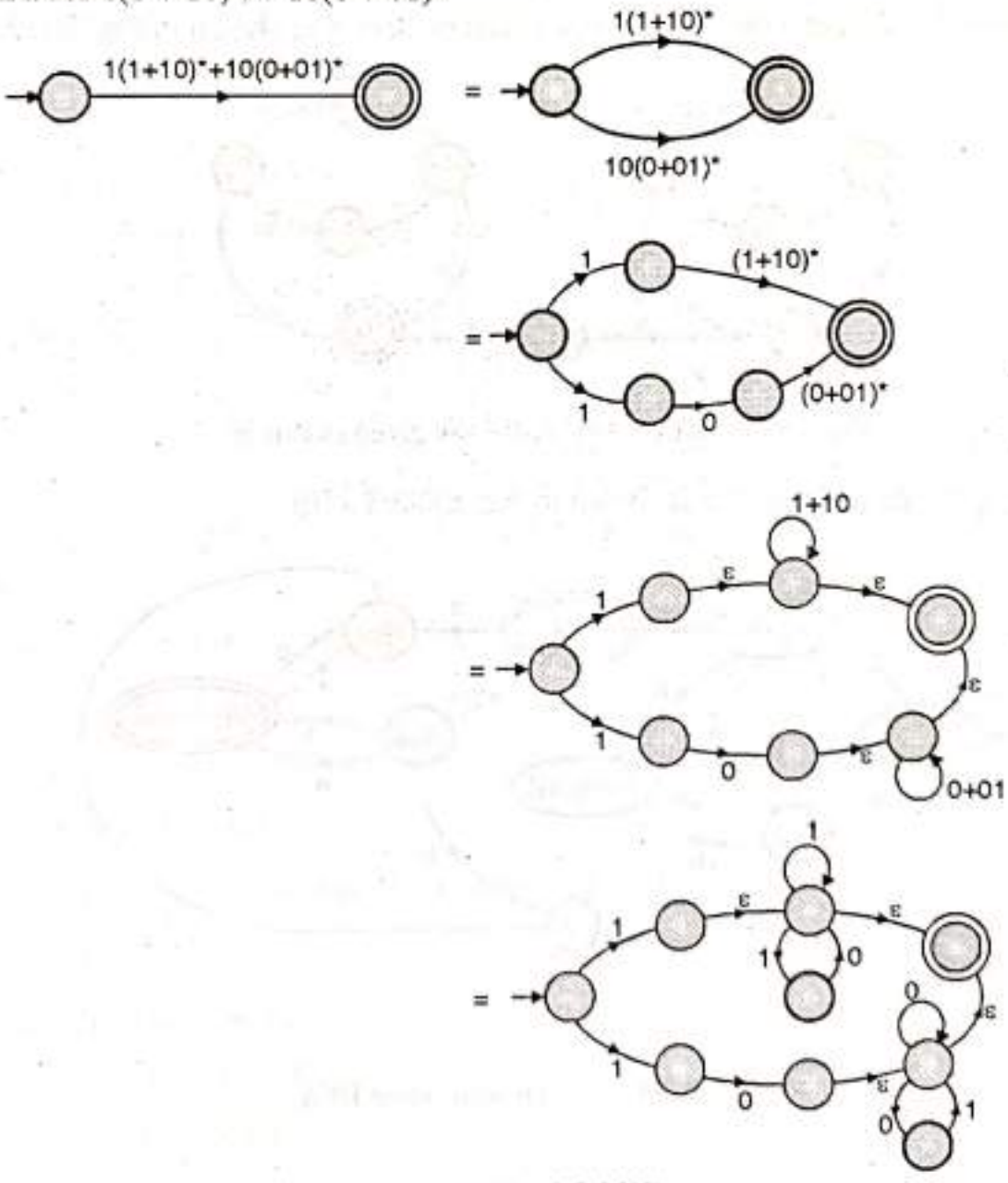


Fig. Ex. 3.3.14(d)

Scanned by CamScanner

Example 3.3.15 : Convert the following R.E. to DFA. (R.E. to NFA with ε-moves to DFA)

$(ab + ba)^*aa(ab + ba)^*$

Solution :

The ε-NFA for $(ab + ba)^*$ is given by

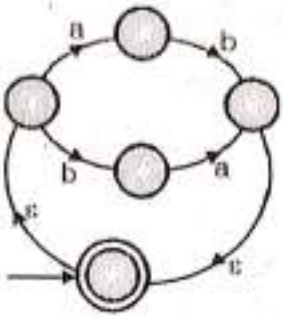


Fig. Ex. 3.3.15

Thus the ε-NFA for $(ab + ba)^*aa\ (ab + ba)^*$ can be drawn as shown in Fig. Ex. 3.3.15(a)

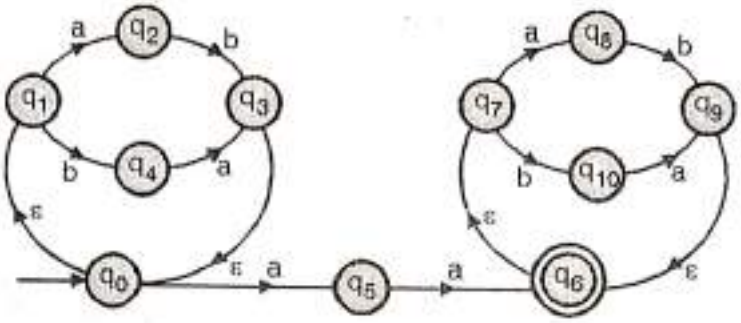


Fig. Ex. 3.3.15(a) : ε-NFA for the given example

DFA using the direct approach is shown in Fig. Ex. 3.3.15(b).

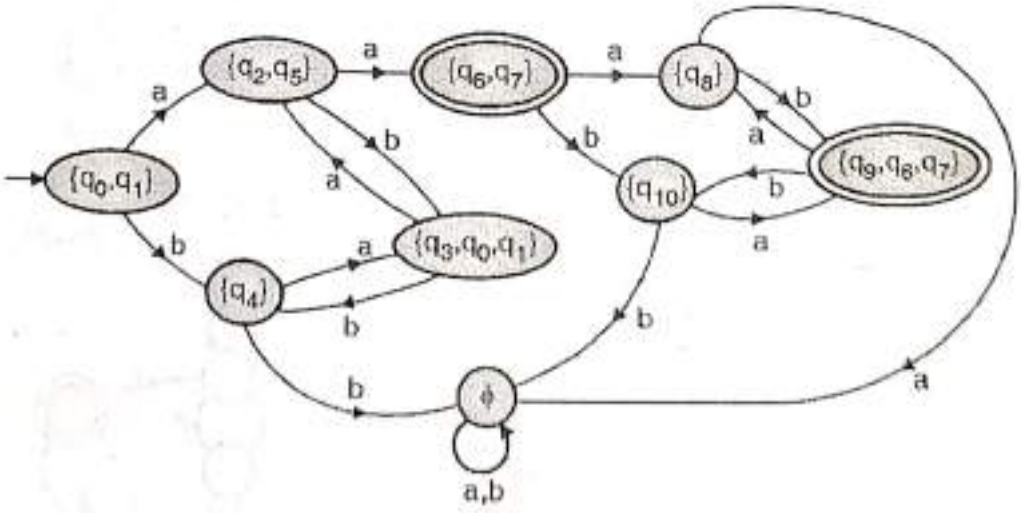


Fig. Ex. 3.3.15(b) : An equivalent DFA

Scanned by CamScanner

Example 3.3.16 : What language is represented by Regular expression.(((a*a)b) ∪ b)

Solution : The regular set (((a*a)b) ∪ b) can be written as

$$\text{R.E.} = (a^*a)b + b = (a^*a + \varepsilon)b$$
$$= a^*b \qquad \text{[From Equation (3.12)]}$$

Thus the language represented by the regular expression is "zero or more a's followed by a 'b' ".

Example 3.3.17 : Consider the two regular expression

$r = 0^* + 1^*$, $s = 01^* + 10^* + 1^*0 + (0^*1)^*$

a) Find the string corresponding to r but not to s.

b) Find the string corresponding to s but not to r.

c) Find the string corresponding to both r and s.

Solution :

a) Find the string corresponding to r but not to s.

00 is in r but not in s

b) Find the string corresponding to s but not to r.

01 is in s but not in r

c) Find the string corresponding to both r and s.

11 is both in r and s.

Example 3.3.18 : Write RE for the following languages

(i) The set of all string over {0, 1} without length two.

(ii) $L = \{a^n b^m \mid (n + m) \text{ is even}\}$

(iii) $L = \{\omega \in (a, b)^* \mid (\text{number of a's in } \omega) \bmod 3 = 0\}$

(iv) $L = \{a^n b^m \mid n >= 4, m <= 3\}$ **MU - May 2006, Dec. 2007, May 2008**

Solution :

(i) The set of all strings over {0, 1} without length two.

$\varepsilon + (0 + 1) + (0 + 1)(0 + 1)(0 + 1)(0 + 1)^*$

(ii) $L = \{a^n b^m \mid (n + m) \text{ is even}\}$

$((aa)^*ab + bb)(bb)^*$

(iii) $L = \{\omega \in (a, b)^* \mid (\text{number of a's in } \omega) \bmod 3 = 0\}$

$(b + ab^*ab^*a)^*$

(iv) $L = \{a^n b^m \mid n >= 4, m <= 3\}$

$aaaaa^*[\varepsilon + b + bb + bbb]$

Example 3.3.19 : Construct an NFA with ε-moves for the RE $10(0 + 01 + 0110)^*$ **MU - May 2009**

Scanned by CamScanner

Solution :

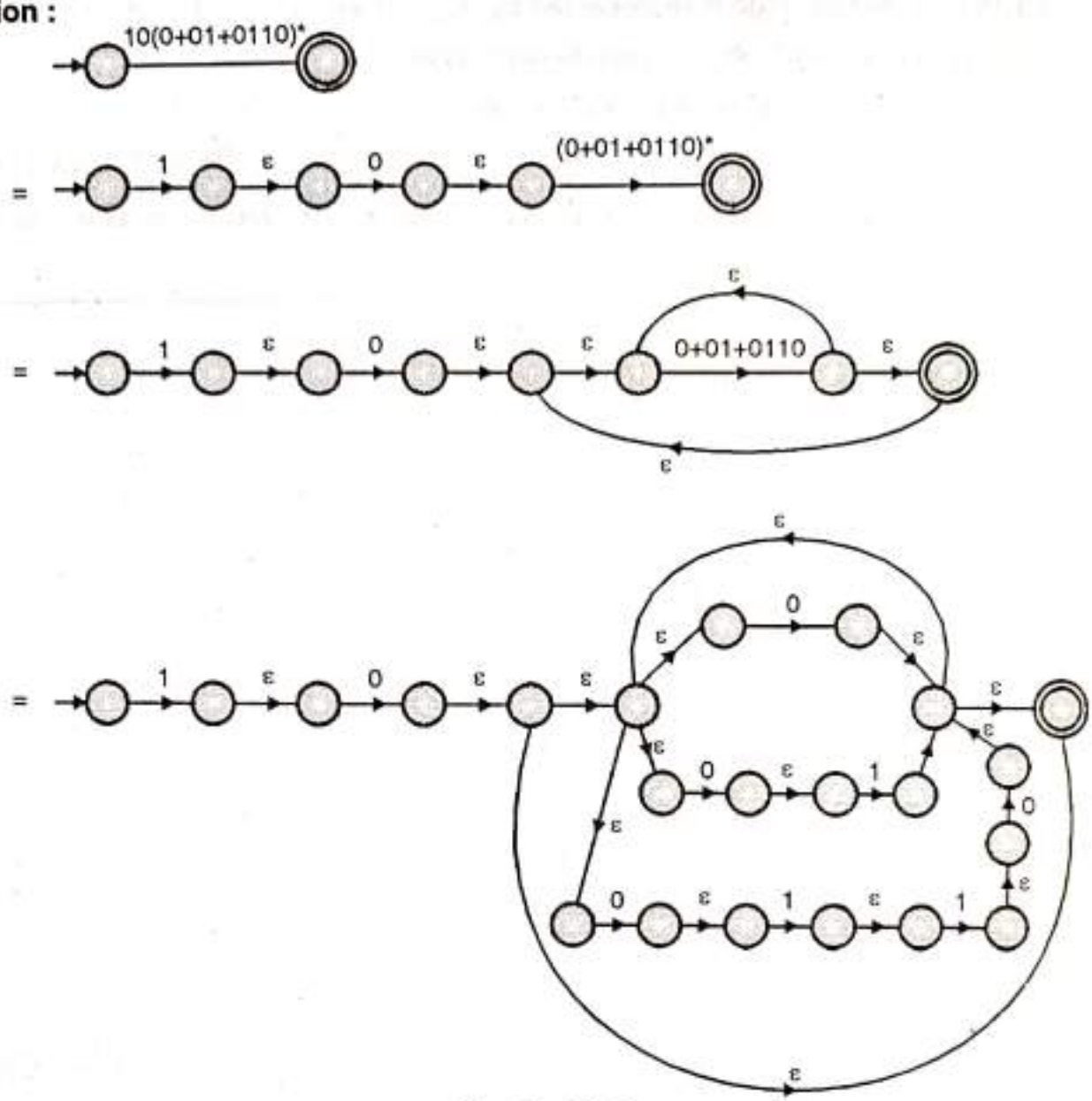


Fig. Ex. 3.3.19

Example 3.3.20 : Construct NFA from following RE and convert into minimized DFA.

(i) $(0 + 1)^*(00 + 11)$ (ii) $(01^* + 1)$

(iii) $(a + b)^*aba$ **MU - May 2007, Dec. 2008, Dec. 2009, May 2010**

Solution :

(i) $(0 + 1)^*(00 + 11)$

RE to NFA

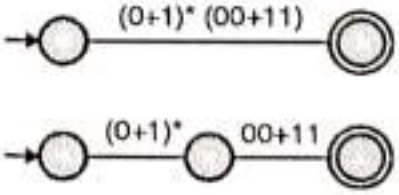


Fig. Ex. 3.3.20(a) : RE to NFA contd...

Scanned by CamScanner

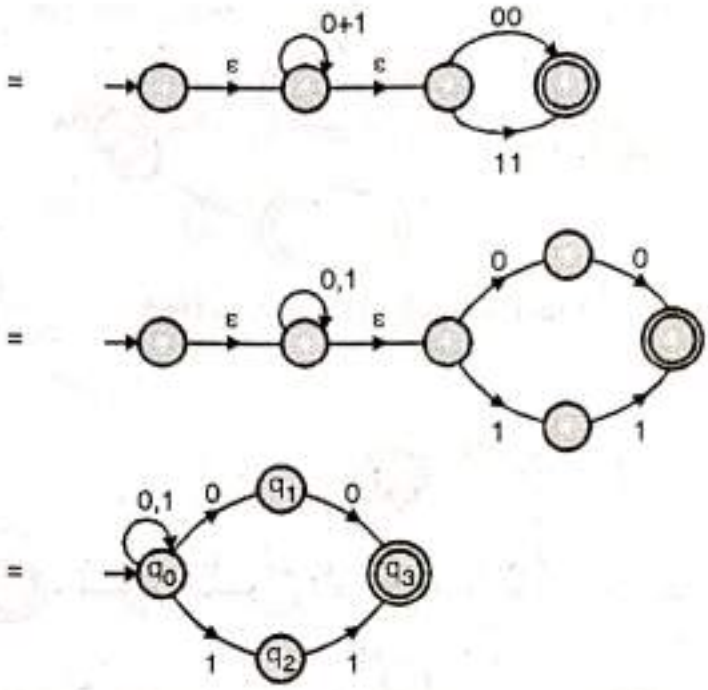


Fig. Ex. 3.3.20(a) : RE to NFA

NFA to DFA

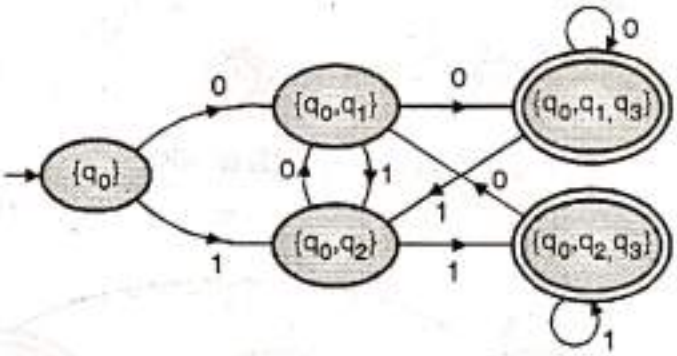


Fig. Ex. 3.3.20(b) : NFA to DFA

(ii) $(01^* + 1)$

RE to NFA

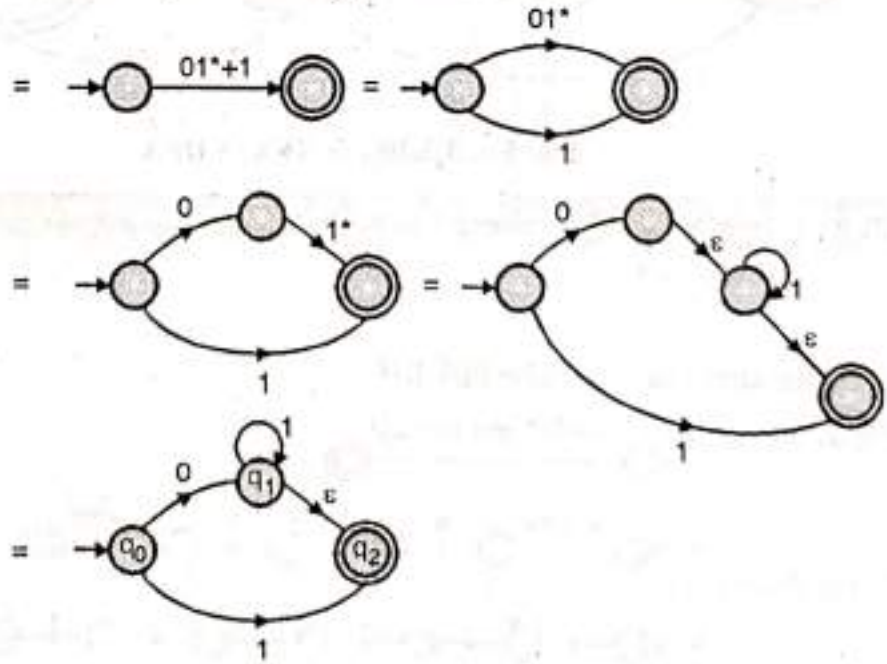


Fig. Ex. 3.3.20(c) : RE to NFA

Scanned by CamScanner

NFA to DFA

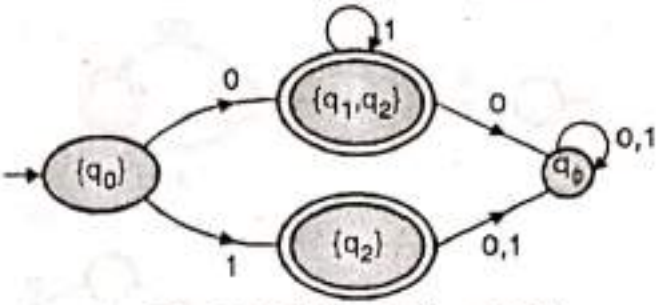


Fig. Ex. 3.3.20(d) : NFA to DFA

(iii) $(a + b)^*$ aba

RE to NFA

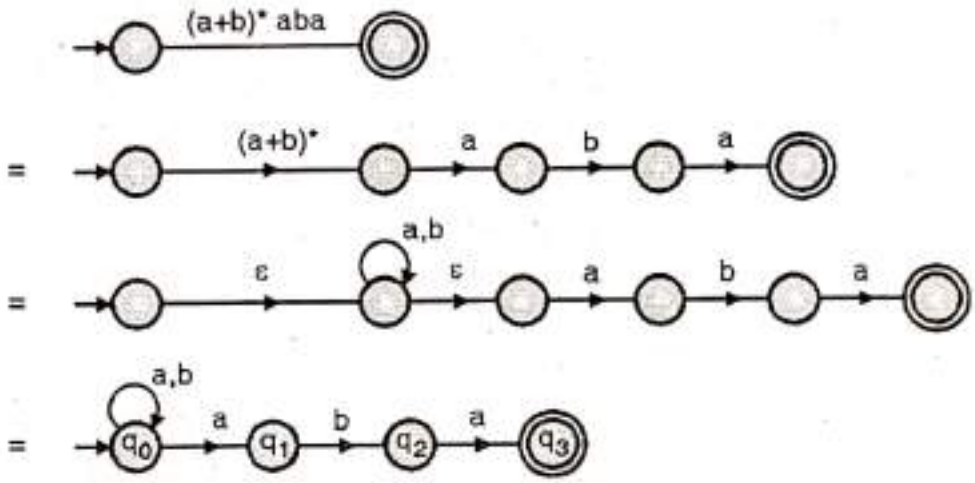


Fig. Ex. 3.3.20(e) : RE to NFA

NFA to DFA

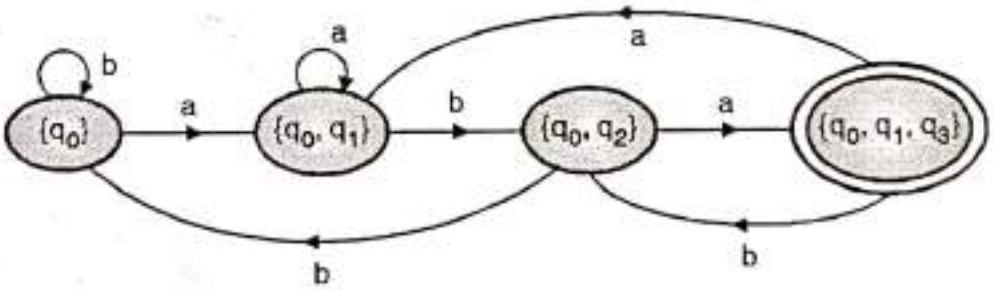


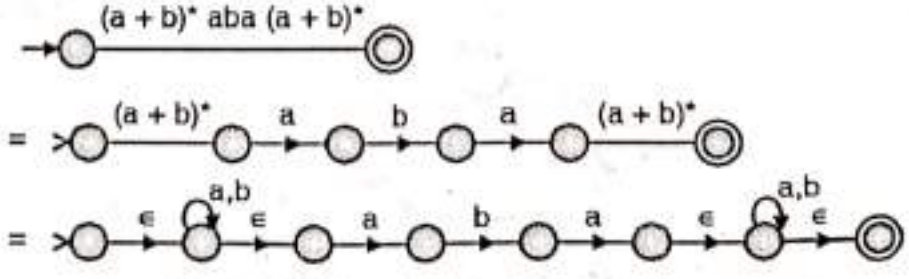
Fig. Ex. 3.3.20(f) : NFA to DFA

Example 3.3.21 : Design a DFA corresponding to the regular expression (a+b) * aba(a+b)* **MU - Dec. 2010**

Solution :

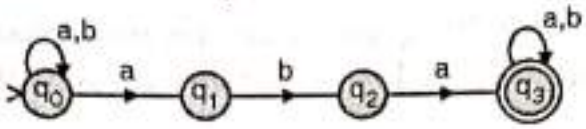
Give regular expression : $(a + b)^*$ aba $(a + b)^*$.

Step 1 : RE to NFA

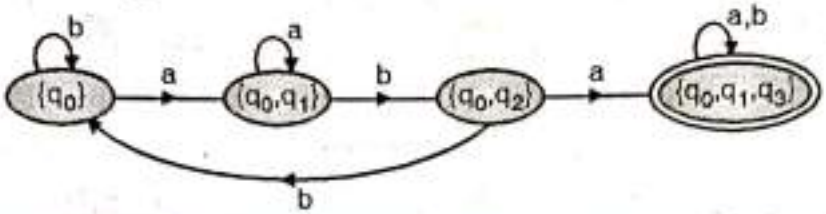


Scanned by CamScanner

ϵ-moves can be eliminated as the adjacent nodes do not contain a loop.



Step 2 : NFA to DFA using the direct method



Example 3.3.22 : Construct an NFA with epsilon transition for the following RE. $(00 + 11)^* (10)^*$ **MU - May 2014**

Solution :

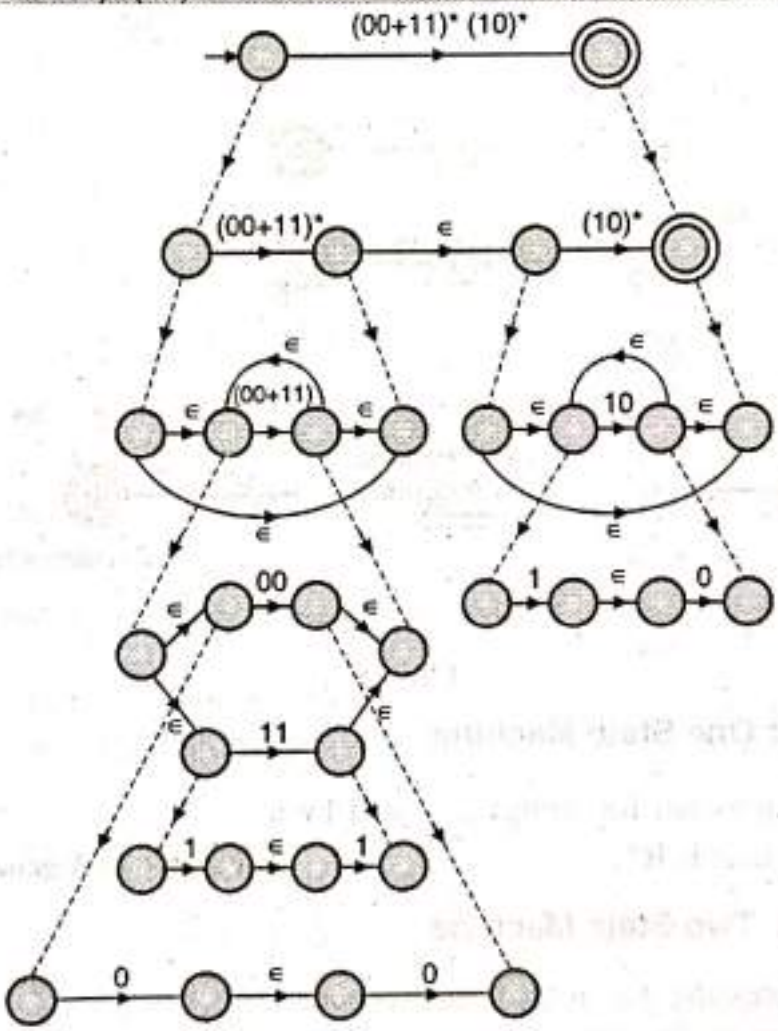


Fig. Ex. 3.3.22

3.4 DFA to Regular Expression

There are two popular approaches for constructing a regular expression from FA.

(1) Through state/loop elimination

(2) Arden's theorem.

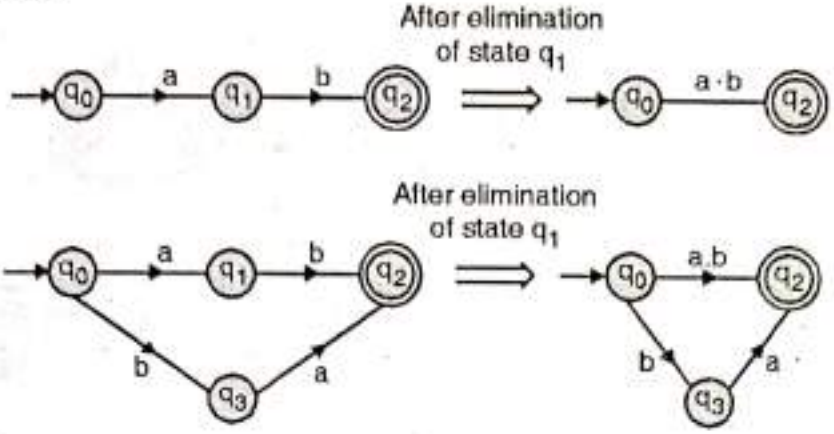
Scanned by CamScanner

3.4.1 State/Loop Elimination Process

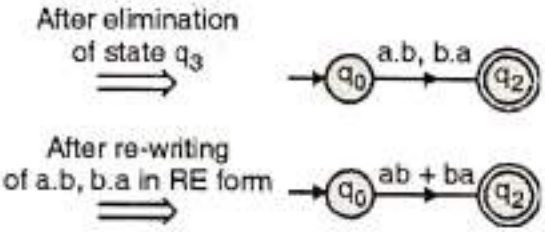
Every state $q_i \notin F$ (set of final states) can be eliminated if q_i is not a start state. Elimination of a state involves writing of regular expression as label on arcs.

For example,

1.



2.



3.

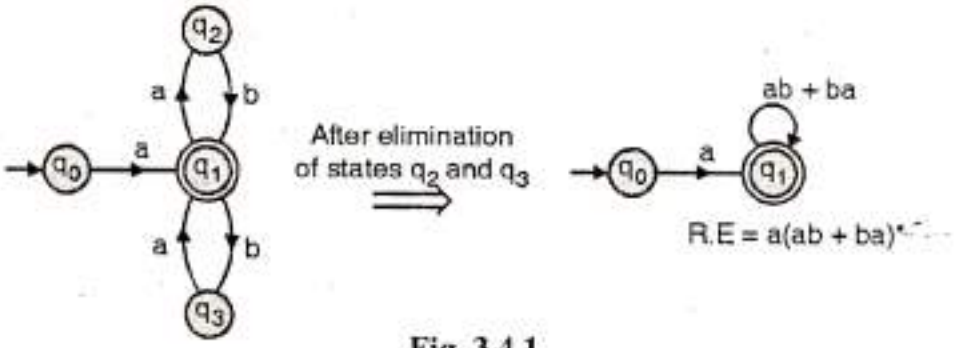


Fig. 3.4.1

3.4.1.1 A Generic One State Machine

The regular expression for strings accepted by a one state generic machine is **R***.

Fig. 3.4.2 : A generic one-state machine

3.4.1.2 A Generic Two State Machine

A regular expression for a two sate generic machine can be written easily after elimination of loop between q_0 and q_1.

- The effect of the loop can be moved either to state q_0 or q_1.
- Even after elimination of loop between q_0 and q_1, a path from start state (q_0) to final state (q_1) should be maintained.

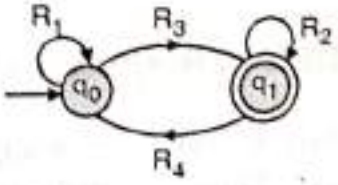


Fig. 3.4.3 : A generic two-state machine

Scanned by CamScanner

Machine after moving the effect of loop to state q_0 :

Starting from the start state q_0, one can come back to q_0 either on R_1,

or $R_3R_2^*R_4$

The equivalent R.E. for machine shown in Fig. 3.4.4 can be written easily. It is given by

$$\text{R.E.} = (R_1 + R_3 R_2^* R_4)^* R_3 R_2^*$$

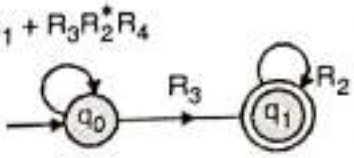


Fig. 3.4.4 : Effect of loop is moved to state q_0

Machine after moving the effect of loop to state q_1 :

Starting from the state q_1, one can come back to q_1 either on R_2.

or $R_4R_1^*R_3$

The equivalent R.E. for machine shown in Fig. 3.4.5 can be written easily. It is given by :

$$\text{R.E.} = R_1^* R_3 (R_2 + R_4 R_1^* R_3)^*$$

Fig. 3.4.5 : Effect of loop is moved to state q_1

Example 3.4.1 : Find a regular expression for the given two state generic machine.

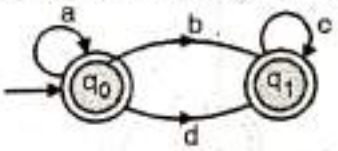


Fig. Ex. 3.4.1 : A two state generic machine

Solution :

Neither state q_0 nor state q_1 can be eliminated as both are final states. Effect of the loop between q_0 and q_1 can be moved either to state q_0 or state q_1.

Moving the effect of loop between q_0 and q_1 on state q_0, we get a machine as shown in Fig. Ex. 3.4.1(a).

The above machine has two final states. The machine can be divided into two machines, first with q_0 as final state and second with q_1 as final state. Machine after division is shown in Fig. Ex. 3.4.1(b).

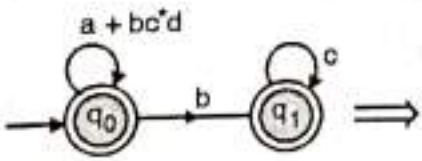


Fig. Ex. 3.4.1(a)

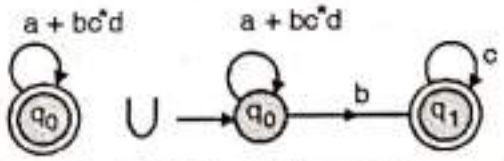


Fig. Ex. 3.4.1(b) : Union of two machines

R.E. corresponding to final state $q_0 = (a + bc^*d)^*$

R.E. corresponding to final state $q_1 = (a + bc^*d)^* bc^*$

The final, regular expression will be obtained by taking union of R.E. for state q_0 and q_1.

$$\therefore \text{Final R.E.} = (a + bc^*d)^* + (a + bc^*d)^* bc^*$$

$$= (a + bc^*d)^* [\varepsilon + bc^*]$$

Scanned by CamScanner

Example 3.4.2 : Convert the automata shown in Fig. Ex. 3.4.2 to RE

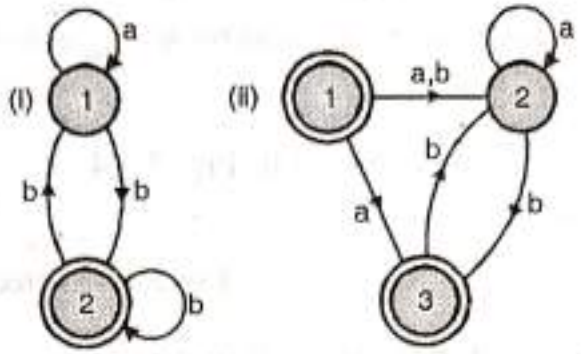


Fig. Ex. 3.4.2

Solution :

(i) Machine is redrawn after the effect of loop between state 1 and 2 is transferred to state 1.

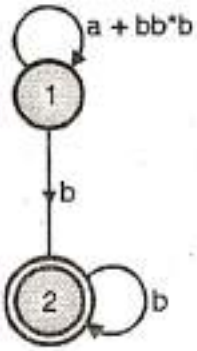


Fig. Ex. 3.4.2(a)

$\therefore$ R.E. $= (a + bb^*b)^*b^*b^*$

(ii) Machine is redrawn after the effect of loop between state 2 and 3 is transferred to state 2.

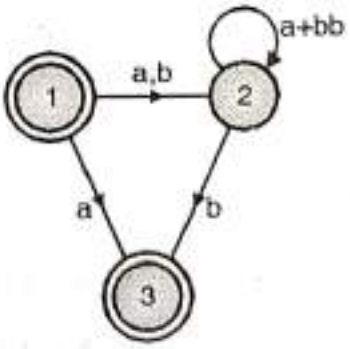


Fig. Ex. 3.4.2(b)

$$\text{R.E.} = \varepsilon + \underbrace{(a + b)(a + bb)^*b + a}$$

for state 1 for state 3

Scanned by CamScanner

Example 3.4.3 : Find R.E. corresponding to T.G. shown in Fig. Ex. 3.4.3.

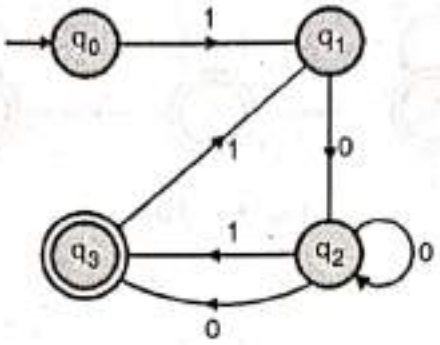


Fig. Ex. 3.4.3

Solution :

Reducing the given T.G.

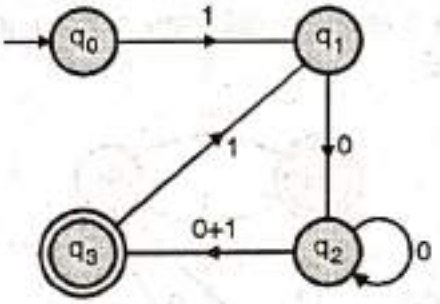


Fig. Ex. 3.4.3(a)

Now, eliminating the state q_2, we get,

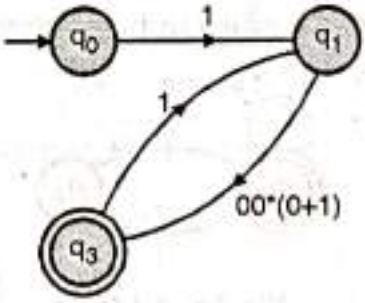


Fig. Ex. 3.4.3(b)

Machine is redrawn after the effect of loop between q_1 and q_3 is transferred to q_1.

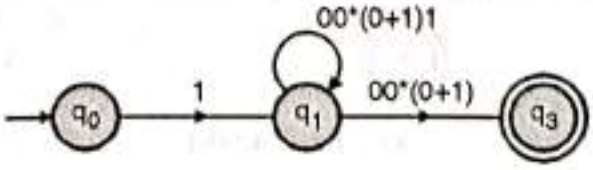


Fig. Ex. 3.4.3(c)

$\therefore$ Final R.E. = $1[00^*(0 + 1)1]^* 00^* (0 + 1)$

Scanned by CamScanner

Example 3.4.4 : Find R.E. for shown in Fig. Ex. 3.4.4.

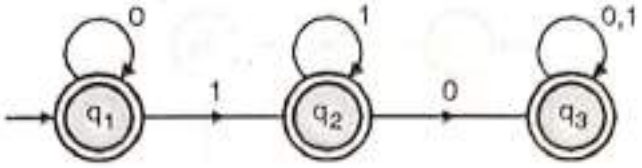


Fig. Ex. 3.4.4

Solution :

R.E. for state $q_1 = 0^*$

R.E. for state $q_2 = 0^*11^*$

R.E. for state $q_3 = 0^*11^*0(0 + 1)^*$

$\therefore$ Final RE $= 0^* + 0^*11^* + 0^*11^*0(0 + 1)^*$

Example 3.4.5 : Consider the transition diagram shown in Fig. Ex. 3.4.5. Convert it to equivalent RE.

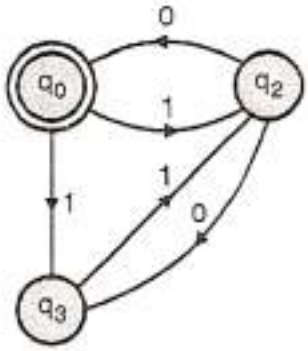


Fig. Ex. 3.4.5

Solution : Machine in redrawn after the effect of loop between q_0, q_3, q_2 is transferred to q_0.

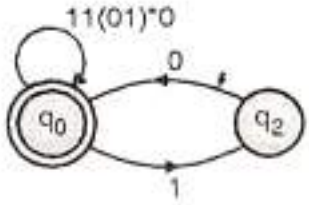


Fig. Ex. 3.4.5(a)

Now, removing the loop between q_0 and q_2,

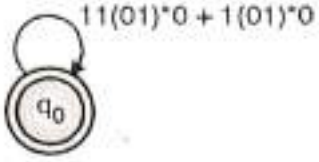


Fig. Ex. 3.4.5(b)

$\therefore$ Final RE $= [11(01)^*0 + 1(01)^*0]^*$

Scanned by CamScanner

Example 3.4.6 : Find a regular expression for the given three state machine shown in Fig. Ex. 3.4.6.

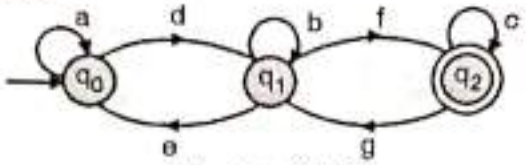


Fig. Ex. 3.4.6

Solution : The state q_1 is at the crossing point of three loops :

1. q_1 to q_1 on input b.
2. Loop between q_1 and q_0 on input ea*d
3. Loop between q_1 and q_2 on input fc*g.

The effect of these three loops can be transferred to state q_1; the machine after this effect is shown in Fig. Ex. 3.4.6(a).

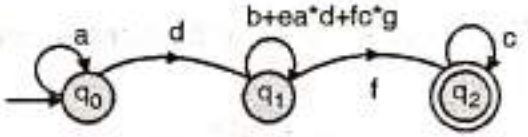


Fig. Ex. 3.4.6(a) : Moving the effect of loops on state q_1

The state q_1 can be eliminated.

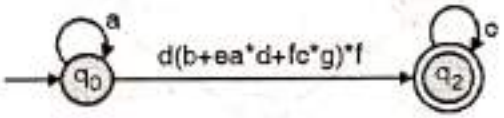


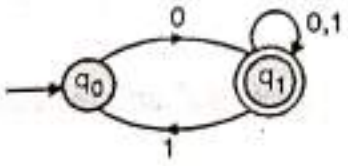
Fig. Ex. 3.4.6(b) : After elimination of state q_1

Now, the regular expression can be written as,

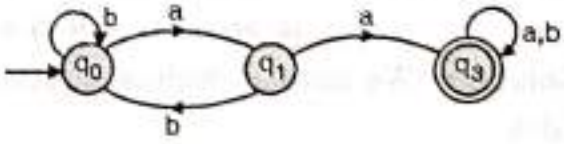
$$R.E. = a^*d(b + ea^*d + fc^*g)^*fc^*$$

Example 3.4.7 : Find the regular expression for the DFA's, shown in Fig. Ex. 3.4.7.

(i)



(ii)



(iii)

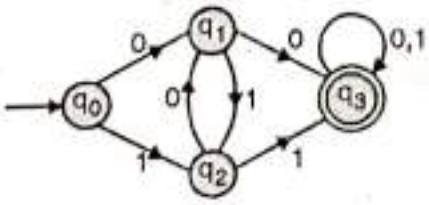


Fig. Ex. 3.4.7

Scanned by CamScanner

Solution :

(i) Machine is redrawn after the effect of loop between q_0 and q_1 is transferred to q_0.

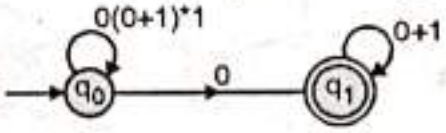


Fig. Ex. 3.4.7(a)

$$\text{R.E.} = (0(0+1)^*1)^*0(0+1)^*$$

(ii) Machine is re-drawn after the effect of loop between q_0 and q_1 is transferred to q_0

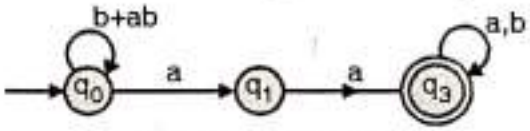


Fig. Ex. 3.4.7(b)

$$\text{R.E.} = (b + ab)^*aa(a + b)^*$$

(iii) Machine is redrawn after the effect of loop between q_1 and q_2 is transferred to both q_1 and q_2.

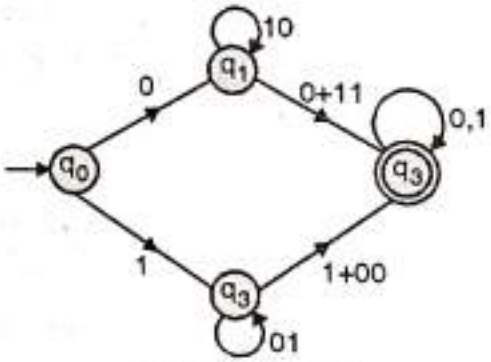


Fig. Ex. 3.4.7(c)

There are two paths from q_1 and q_3 [0 + 11]. There are two paths from q_2 to q_3 [1 + 00].

$$\text{R.E.} = [0(10)^*(0 + 11) + 1(01)^* (1 + 00)](0 + 1)^*$$

Example 3.4.8 : Prove the identity given below :

$$(a^*ab + ba)^*a^* = (a + ab + ba)^*$$

Solution : We can start with construction of NFA for $(a + ab + ba)^*$ and then convert it into DFA.

Step 1 : $(a + ab + ba)^*$ to NFA. An equivalent NFA is given in Fig. Ex. 3.4.8.

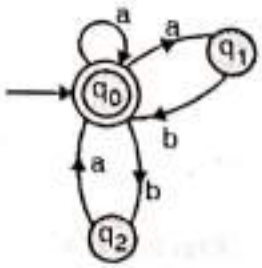


Fig. Ex. 3.4.8 : An equivalent NFA

Scanned by CamScanner

Step 2 : NFA to DFA. An equivalent DFA is given in Fig. Ex. 3.4.8(a).

	a	b
$\rightarrow\{q_0\}^*$	$\{q_0, q_1\}$	$\{q_2\}$
$\{q_2\}$	$\{q_0\}$	ϕ
$\{q_0, q_1\}^*$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}^*$	$\{q_0, q_1\}$	$\{q_2\}$

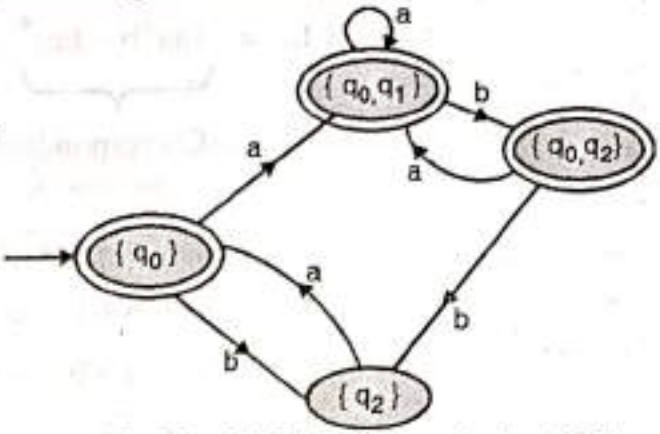


Fig. Ex. 3.4.8(a) : An equivalent DFA

Step 3 : Minimization of DFA obtained from Fig. Ex. 3.4.8(a).

Behaviour of $\{q_0\}$ and $\{q_0, q_2\}$ are identical as their a-successors and b-successors are same. Merging the two states $\{q_0\}$ and $\{q_0, q_2\}$, we get, the minimum state DFA as shown in Fig. Ex. 3.4.8(b).

	a	b
$\{q_0, q_2\} = \rightarrow A^*$	B	C
$\{q_0, q_1\} = B^*$	B	A
$\{q_2\} = C$	A	ϕ

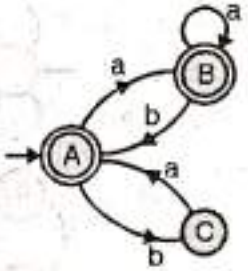


Fig. Ex. 3.4.8(b) : An equivalent minimal DFA

Step 4 : Writing of regular expression for DFA derived from Fig. Ex. 3.4.8(b).

- There are two loops on A.
  1. Loop through $A \rightarrow C \rightarrow A$ on input ba
  2. Loop through $A \rightarrow B \rightarrow B \rightarrow \ldots. \rightarrow B \rightarrow A$ on input aa*b
  3. There are two final states.

An equivalent machine without a loop is shown in Fig. Ex. 3.4.8(c).

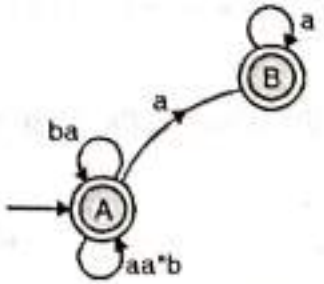


Fig. Ex. 3.4.8(c) : An equivalent machine without a loop

Scanned by CamScanner

Regular expression for the machine shown in Fig. Ex. 3.4.8(c) is given by :

$$\text{R.E.} = \underbrace{(aa^*b + ba)^*}_{\text{Corresponding to state A}} + \underbrace{(aa^*b + ba)^*aa^*}_{\text{Corresponding to state B}}$$

$$= (aa^*b + ba)^*[\varepsilon + aa^*]$$

$$= (aa^*b + ba)^*a^* \quad \ldots \text{[By Equation (3.12)]}$$

$$= (a^*ab + ba)^*a^* \quad \ldots \text{[By Equation (3.10)]}$$

$$= \text{L.H.S.}$$

Example 3.4.9 : Prove or disprove the following for regular expression r, s and t.

(a) $(rs + r)^*r = r(sr + r)^*$ (b) $s(rs + s)^*r = rr^*s(rr^*s)^*r$

(c) $(r + s)^* = r^* + s^*$

Solution :

(a) $(rs + r)^*r = r(sr + r)^*$

This can be proved by drawing a F.A. for $(rs + r)^*r$

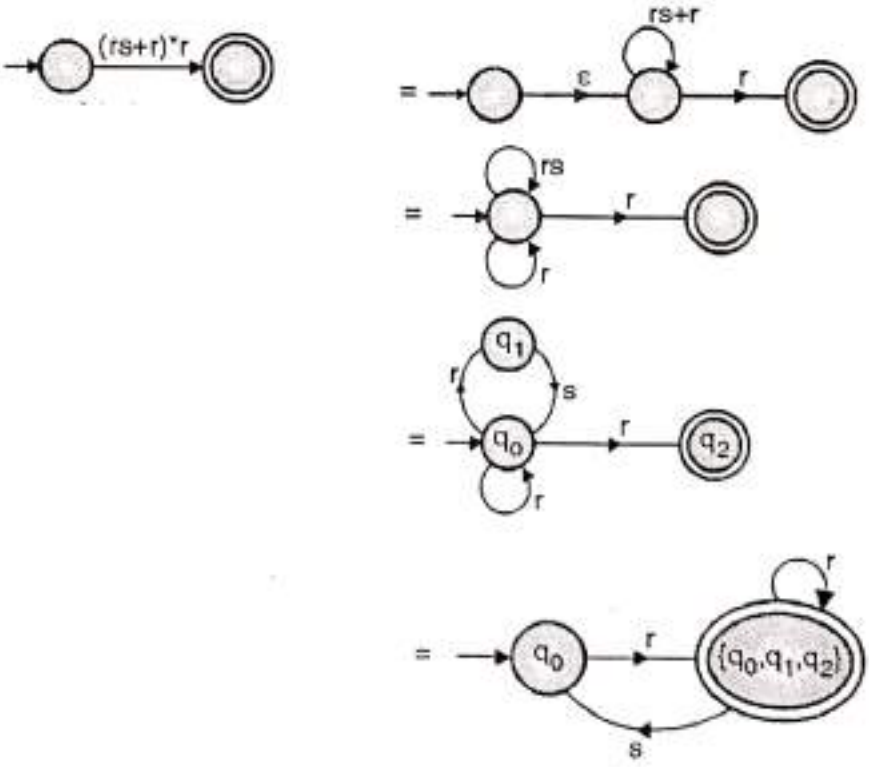


Fig. Ex. 3.4.9

Moving the effect of the loop on $\{q_0, q_1, q_2\}$, we get,

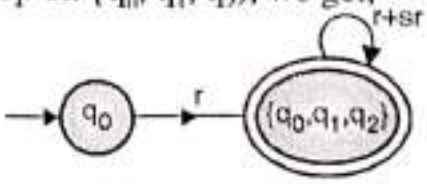


Fig. Ex. 3.4.9(a)

Scanned by CamScanner

The regular expression for the above FA is given by $r(sr + r)^* = R.H.S.$

Hence, it is proved.

(b) $s(rs + s)^*r = rr^*s\ (rr^*s)^*r$

L.H.S. is starting with s, whereas R.H.S. starts with r. Hence, disproved.

(c) $(r + s)^* = r^* + s^*$

The expression rs belongs to $(r + s)^*$ but it does not belong to $r^* + s^*$, hence disproved.

Example 3.4.10 : Find the regular expression corresponding to each of the following subset of {0, 1}*

a) The language of all strings not containing the substring 000.
b) The language of all strings that do not contain the substring 110.
c) The language of all strings containing both 101 and 010 as substring.

Solution :

(a) The language of all strings not containing the substring 000.

R.E. can be written by drawing an equivalent FA and then writing a regular expression for it.

DFA for strings not containing 000 is given by,

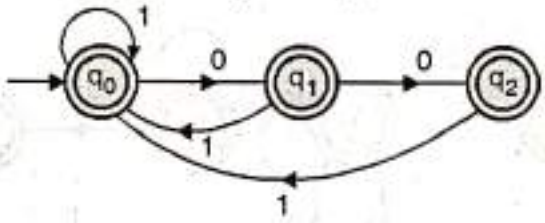


Fig. Ex. 3.4.10

Moving the effect of loops on state q_0 we get,

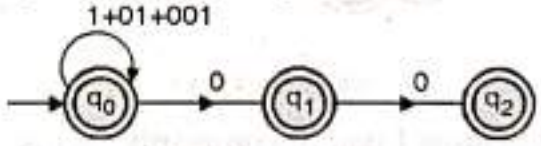


Fig. Ex. 3.4.10(a)

$\therefore$ R.E. $= (1 + 01 + 001)^*(\varepsilon + 0 + 00)$

(b) The language of all strings that do not contain the substring 110.

R.E. can be written by drawing an equivalent FA and then writing a regular expression for it.

DFA for strings not containing 110 is given by,

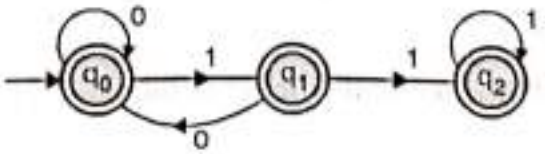


Fig. Ex. 3.4.10(b)

Scanned by CamScanner

Moving the effect of loop on state q_0, we get,

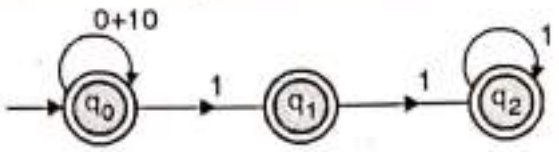


Fig. Ex. 3.4.10(c)

$$\therefore \quad \text{R.E.} = (0+10)^* + (0+10)^*1 + (0+10)^*111^*$$

(c) The language of all strings containing both 101 and 010 as substring.

$$\therefore \quad \text{R.E.} = (0+1)^*101(0+1)^*010(0+1)^* + (0+1)^*010(0+1)^*101(0+1)^*$$

Example 3.4.11 : Write RE for the following $\Sigma = \{0,1\}$

(i) At most 1 pair of consecutive '0' or 1 pair of consecutive '1'.

(ii) Without any double letter.

Solution :

(i) At most 1 pair of consecutive '0' or 1 pair of consecutive '1':

A finite automata with at most 1 pair of consecutive 0's is shown in Fig. Ex. 3.4.11.

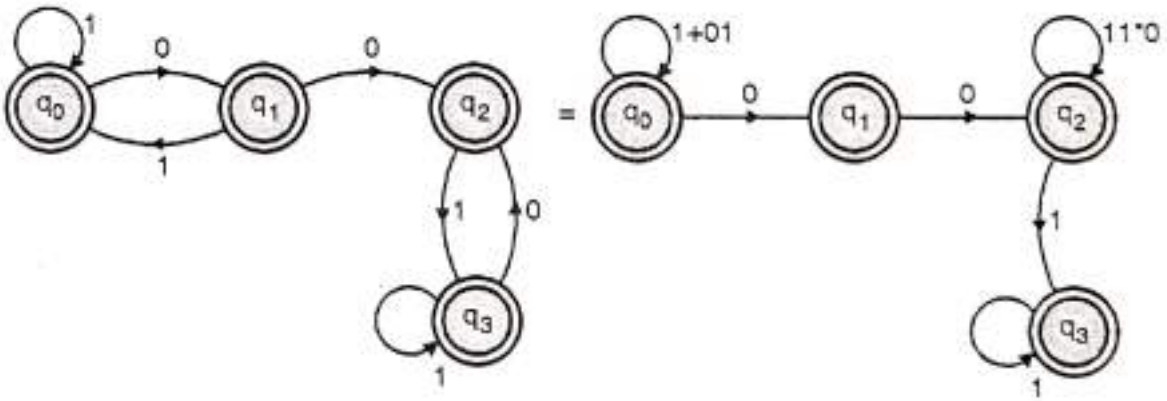


Fig. Ex. 3.4.11

∴ R.E. for strings with at most 1 pair of consecutive 0's is

$(1+01)^* [\varepsilon + 0 + 00\,(11^*0)^* + 00\,(11^*0)^*\ 11^*]$

∴ R.E. for strings with at most 1 pair 1's is

$(0+10)^* [\varepsilon + 1 + 11\,(00^*1)^* + 11(00^*1)^*00^*]$

∴ Required R.E. is:

$(1+01)^* [\varepsilon + 0 + 00\,(11^*0)^* + 00\,(11^*0)^*\ 11^*] + (0+10)^*$

$[\varepsilon + 1 + 11\,(00^*1)^* + 11\,(00^*1)^*00^*]$

(ii) Without any double letter :

A Finite Automata without any double letter is shown in Fig. Ex. 3.4.11(a).

Scanned by CamScanner

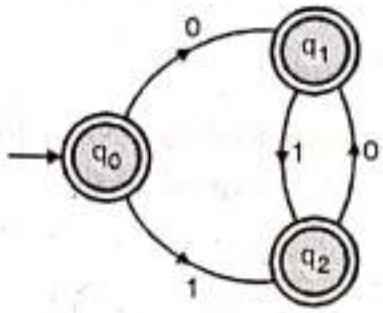


Fig. Ex. 3.4.11(a)

$\therefore$ R.E. $= \varepsilon + 0(10)^* [\varepsilon + 1] + 1 (01)^* [\varepsilon + 0]$

Example 3.4.12 : Write RE for the following languages :

(i) The language of all strings that contains at least one occurrence of each symbol in $\Sigma = \{0,1\}$

(ii) $\Sigma = \{a, b\}$ such that ab is not a substring of any strings.

Solution :

(i) The FA for language of all strings containing at least one occurrence of each symbol in $\Sigma = \{0, 1\}$ is shown in Fig. Ex. 3.4.12 :

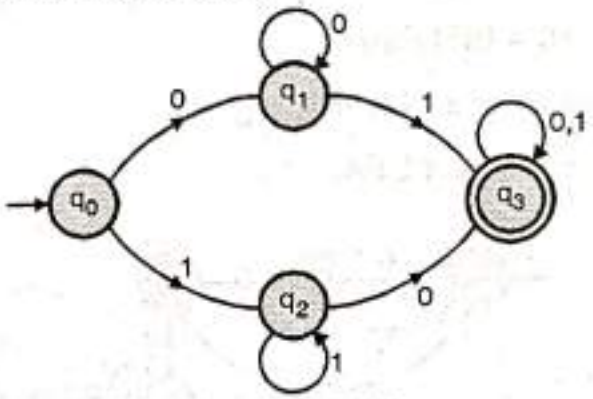


Fig. Ex. 3.4.12

The RE can be written from the DFA.

R.E. $= 00^*1(0 + 1)^* + 11^*0 (0 + 1)^*$

(ii) The DFA for language of all strings not containing 'ab' as a substring is shown in Fig. Ex. 3.4.12(a) :

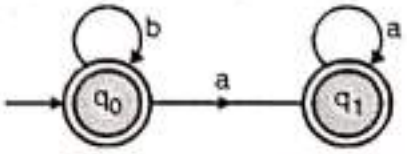


Fig. Ex. 3.4.12(a)

$\therefore$ R.E. $= b^* + b^*aa^*$

Example 3.4.13 : Write a R.E. for the following

(i) $\Sigma = (0, 1)$ odd number of 1's in strings

(ii) $\Sigma = \{0, 1\}$ Triple 0 must never appear in strings.

(iii) Identities in C language.

Scanned by CamScanner

Solution :

(i) $\Sigma = (0, 1)$ odd number of 1's in strings.

In some cases it is more convenient to draw a DFA for the given language and then an equivalent regular expression can be written for it.

Step 1 : DFA for the given language

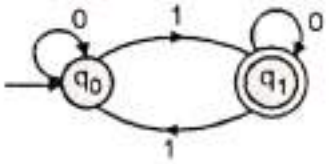


Fig. Ex. 3.4.13(a) : DFA for strings having odd number of 1's

An equivalent FA with effect of loop between q_0 and q_1 moved to q_0.

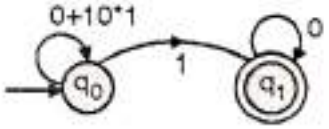


Fig. Ex. 3.4.13(b) : An equivalent machine of DFA given in Fig. Ex. 3.4.13(a)

Step 2 : Writing of R.E. from Fig. Ex. 3.4.13(b).

$$\textbf{R.E.} = (0 + 10^*1)^*10^*$$

(ii) $\Sigma = \{0, 1\}$ Triple 0 must never appear in string.

Step 1 : Drawing an equivalent DFA.

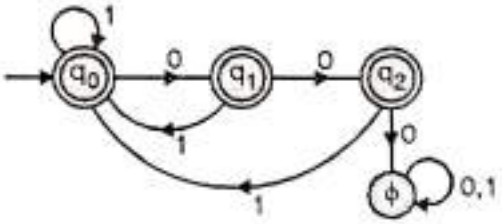


Fig. Ex. 3.4.13(c) : DFA for strings not having 000

An equivalent FA with effect of loops moved to q_0 is shown in Fig. Ex. 3.4.13(d).

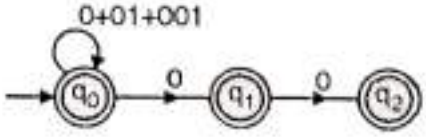


Fig. Ex. 3.4.13(d) : An equivalent FA for DFA of Fig. Ex. 3.4.13(c)

Step 2 : Writing of R.E.

$$\textbf{R.E.} = (1 + 01 + 001)^*(\varepsilon + 0 + 00)$$

(iii) Identifiers in C language.

$$\textbf{R.E.} = (A + B + \ldots + Z + a + b + z)\,(A + B \ldots + Z + a + \ldots z + 0 + \ldots 9)^*$$

Scanned by CamScanner

Example 3.4.14 : Represent the following using regular expressions.

(i) Σ = {a, b, c} the language such that "any number of a's followed by any member of c's"

(ii) If L (r) = {0, 2, 201, 21, 011, 211, 0111, ...} then what is r ?

(iii) If L (r) = {00, 010, 0110, 01110, ...} then what is r ?

(iv) Languages defined over Σ = {a, b} has to have the strings beginning with 'a' and not have two consecutive a's. Write the regular expression for the same.

Solution :

(i) R.E. = a^*c^*

(ii) $L(r) = \{0, 2, 01, 21, 011, 211, 0111, \ldots\}$

$r = 0 + 2 + 01 + 21 + 011 + 211 + 0111 + \ldots$

$= 0(\varepsilon + 1 + 11 + 111 + \ldots) + 2(\varepsilon + 1 + 11 + \ldots) = 01^* + 21^*$

$\mathbf{r = (0 + 2)1^*}$

(iii) $L(r) = \{00, 010, 0110, 01110, \ldots\}$

R.E. = 0(1)*0

(iv) DFA for the given language can be drawn as shown in Fig. Ex. 3.4.14(a).

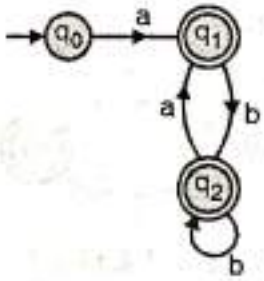


Fig. Ex. 3.4.14(a)

Moving the loop between q_1 and q_2 and q_1 we get :

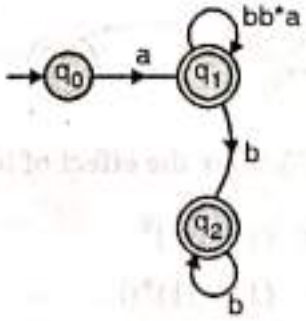


Fig. Ex. 3.4.14(b)

R.E. corresponding to q_1 = $a(bb^*a)^*$

R.E. corresponding to q_2 = $a(bb^*a)^*bb^*$

$\therefore$ Combined R.E. = $a(bb^*a)^* + a(bb^*a)^*bb^*$

$= \mathbf{a(bb^*a^*) + [\varepsilon + bb^*]}$

Scanned by CamScanner

Example 3.4.15 : Let $\Sigma = \{0, 1\}$, construct NFA and hence regular expression for each of the following :

(a) $L_1 = \{\omega \in \Sigma^* \mid \omega$ has at least one pair of consecutive 0's$\}$

(b) $L_2 = \{\omega \in \Sigma^* \mid \omega$ has no pair of consecutive zero$\}$ **MU - May 2006**

(c) $L_3 = \{\omega \in \Sigma^* \mid \omega$ is a string with either 1 preceding a 0 or 0 preceding 1$\}$

(d) $L_4 = \{\omega \in \Sigma^* \mid \omega$ consists of an even number of 0's followed by odd number of 1's$\}$. **MU - May 2014**

Solution :

(a) $L_1 = \{\omega \in \Sigma^* \mid \omega$ has at least one pair of consecutive 0's$\}$

The equivalent FA is shown in Fig. Ex. 3.4.15(a).

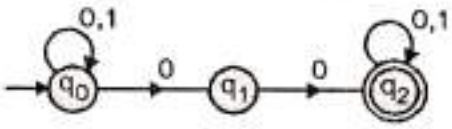


Fig. Ex. 3.4.15(a)

R.E. = $(0 + 1)^*00\ (0 + 1)^*$

(b) $L_2 = \{\omega \in \Sigma^* \mid \omega$ has no pair of consecutive zero$\}$

The equivalent FA is shown in Fig. Ex. 3.4.15(b)

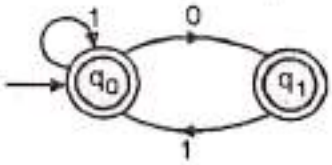


Fig. Ex. 3.4.15(b)

The effect of loop between q_0 and q_1 can be move to q_0. A new machine is obtained and shown in Fig. Ex. 3.4.15(c).

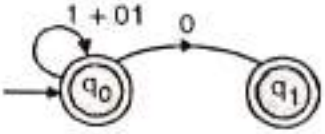


Fig. Ex. 3.4.15(c) : FA after the effect of loop moved to q_0

R.E. for state q_0 = $(1 + 01)^*$

R.E. for state q_1 = $(1 + 01)^*0$

$\therefore$ The final R.E. = $(1 + 01)^* + (1 + 01)^*0 = \mathbf{(1 + 01)^*[\varepsilon + 0]}$

(c) $L_3 = \{\omega \in \Sigma^* \mid \omega$ is a string with either 1 preceding a 0 or 0 preceding 1$\}$

The equivalent FA is shown in Fig. Ex. 3.4.15(d).

Scanned by CamScanner

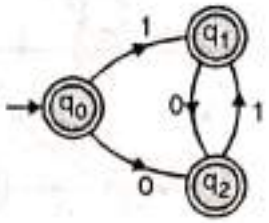


Fig. Ex. 3.4.15(d)

The effect of loop between q_1 and q_2 can be removed by moving its effect both on q_1 and q_2. A new machine is shown in Fig. Ex. 3.4.15(e).

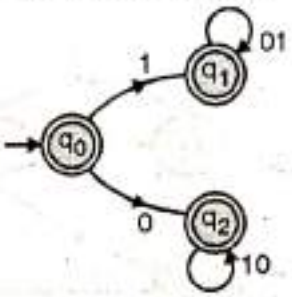


Fig. Ex. 3.4.15(e) : FA after elimination of loop

$$\therefore \quad \text{R.E} = \varepsilon + 1(01)^* + 0(10)^*$$

(d) $L_4 = \{\omega \in \Sigma^* \mid \omega$ consists of an even number of 0's followed by an odd number of 1's.$\}$

The equivalent FA is shown in Fig. Ex. 3.4.15(f)

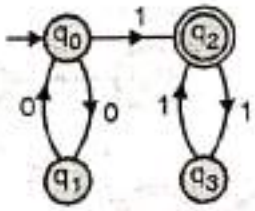


Fig. Ex. 3.4.15(f)

The effect of loop between q_0 and q_1, between q_2 and q_3 are removed and FA is redrawn as shown in Fig. Ex. 3.4.15(g).

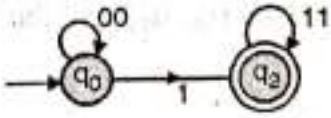


Fig. Ex. 3.4.15(g)

$$\therefore \quad \text{R.E.} = (00)^*1(11)^*$$

Example 3.4.16 : Prove the formula $(00^*1)^*1 = 1 + 0(0 + 10)^*11$

Solution :

The formula can be proved by drawing an equivalent DFA.

DFA for $(00^*1)^*1$

Scanned by CamScanner

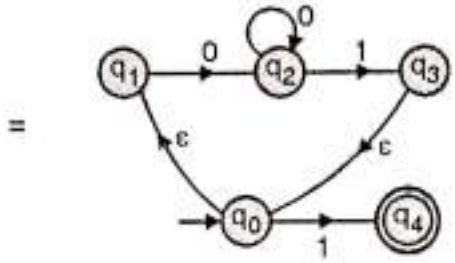


Fig. Ex. 3.4.16(a) : An equivalent ε-NFA

DFA corresponding to ε-NFA of Fig. Ex. 3.4.16(b) is given in Fig. Ex. 3.4.16(b).

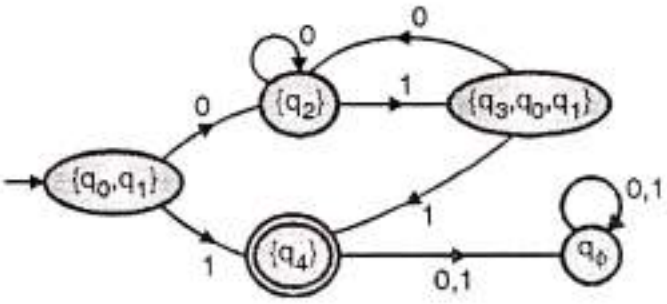


Fig. Ex. 3.4.16(b) : An equivalent DFA

FA after moving the effect of loop between $\{q_2\}$ and $\{q_3, q_0, q_1\}$ is redrawn as shown in Fig. Ex. 3.4.16(c).

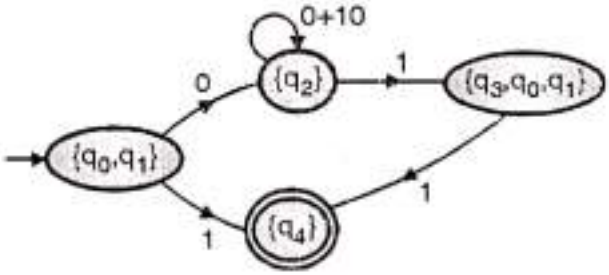


Fig. Ex. 3.4.16(c) : Affect of loop is moved to $\{q_2\}$

There are two paths between the start $\{q_0, q_1\}$ and the final state $\{q_4\}$

1. First path is represented by

   R.E. = 1

2. Second path is represented by

   R.E. = $0(0 + 10)^*11$

$\therefore$ The equivalent regular expression for FA is shown in Fig. Ex. 3.4.16(c) is,

R.E. = $0(0 + 10)^*11 + 1$

Hence the formula is proved.

Scanned by CamScanner

Example 3.4.17 : Obtain a regular expression such that L (R) = { W | W ε { 0, 1 }* } with atleast three consecutive zeros.

Solution :

$$R.E. = (0 + 1)^*000\ (0 + 1)^*$$

3.4.2 Arden's Theorem

MU – Dec. 2010

Let P, Q and R be regular expressions on Σ. Then, if P does not contain ε, the equation

$$R = Q + RP \qquad \ldots(3.16)$$

has a unique solution given by,

$$R = QP^* \qquad \ldots(3.17)$$

Proof : Let us verify whether QP* is a solution to the equation.

$$R = Q + RP$$

On substitution of QP* for R in Equation (3.16),

$$\begin{aligned} R = Q + RP &= Q + QP^*P \\ &= Q^*(\varepsilon + P^*P) \\ &= QP^* \end{aligned}$$

Thus the Equation (3.16) is satisfied by R = QP*. We still do not know whether QP* is a unique solution or not.

To establish uniqueness, we expand

$$\begin{aligned} R &= Q + RP \\ &= Q + (Q + RP)\,P \\ &= Q + QP + RP^2 \\ &= Q + QP + (Q + RP)\,P^2 \\ &= Q + QP + QP^2 + RP^3 \\ &\ \ \vdots \\ &\ \ \vdots \\ &= Q + QP + QP^2 + \ldots + QP^i + RP^{i+1} \\ &= Q\,(\varepsilon + P + P^2 + \ldots + P^i) + RP^{i+1} \end{aligned} \qquad \ldots(3.18)$$

here, i is any arbitrary integer.

Let us take a string ω ∈ R | length of ω = k.

Substituting k for i in Equation (3.18)

$$R = Q\,(\varepsilon + P + P^2 + \ldots + P^k) + RP^{k+1}$$

Since P does not contain ε, w is not contained in RP^{k+1} as the shortest string generated by RP^{k+1} will have a length of k + 1.

Since ω is contained in R, it must be contained in $Q(\varepsilon + P + P^2 + \ldots + P^k)$.

Conversely, if ω is contained in QP* then for some integer k it must be in $Q\{\varepsilon + P + P^2 + \ldots + P^k\}$, and hence in R = Q + RP.

Scanned by CamScanner

3.4.2.1 Application of Arden's Theorem

A finite automata can be represented using a system of equations :

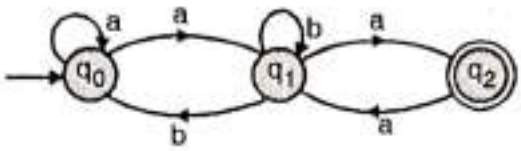


Fig. 3.4.6 : A transition diagram to be converted into a set of equations

Let us consider the FA of Fig. 3.4.6. The set of strings accepted by this FA will be described by a path.

Starting from q_0 and ending q_2.

- A path from q_0 to q_2 will be reached through q_1.
- These strings will end in 'a' with a prefix generated during coming to q_1.
- If the set of strings generated by coming to q_1 from q_0 is $\mathbf{q_1}$ and the set of strings generated by coming to q_2 from q_0 is $\mathbf{q_2}$.

The set $\mathbf{q_2}$ can be written as

$$\mathbf{q_2} = \mathbf{q_1}\, a \qquad \ldots(3.19)$$

Similarly, the state q_1 can be reached from

(1) q_2 on a,

(2) q_1 on b,

(3) q_0 on a.

Thus $\mathbf{q_1}$ can be expressed in terms of $\mathbf{q_0}$ and $\mathbf{q_2}$.

$$\mathbf{q_1} = \mathbf{q_0}\, a + \mathbf{q_1}\, b + \mathbf{q_2}\, a \qquad \ldots(3.20)$$

Similarly, the state q_0 can be reached from :

(1) q_0 on a,

(2) q_1 on b,

(3) Without any input as q_0 is a start state.

Thus $\mathbf{q_0}$ can be expressed in terms of $\mathbf{q_0}$, $\mathbf{q_1}$ and ε.

$$\mathbf{q_0} = \mathbf{q_0}\, a + \mathbf{q_1}\, b + \varepsilon \qquad \ldots(3.21)$$

- These three Equations (3.19, 3.20, 3.21) can be solved with the help of Arden's theorem.
- The set represented by $\mathbf{q_2}$ will be the solution for the FA of Fig. 3.4.6. q_2 is a final state.

Solving of equations using Arden's theorem :

1. Substituting for $\mathbf{q_2}$ in Equation (3.20). $\mathbf{q_2}$ is given by Equation (3.19)

$$\mathbf{q_1} = \mathbf{q_0}\, a + \mathbf{q_1}\, b + \mathbf{q_1}\, aa$$

Scanned by CamScanner

$$= q_0 a + q_1 (b + aa) \quad \ldots(3.22)$$

Equation (3.22) is of the form

$$R = Q + RP \text{ with } Q = q_0 a,$$
$$R = q_1 \text{ and } P = (b + aa)$$

and its solution is given by,

$$R = QP^*$$

Thus, $$q_1 = q_0 a (b + aa)^* \quad \ldots(3.23)$$

2. Substituting the value of q_1 from Equation (3.23) into Equation (3.21)

$$q_0 = q_0 a + q_0 a (b + aa)^* b + \varepsilon$$
$$= \varepsilon + q_0 (a + a (b + aa)^* b) \quad \ldots(3.24)$$

The Equation (3.24) is of the form

$$R = Q + RP \text{ from } Q = \varepsilon, R = q_0 \text{ and } P = (a + a (b + aa^*)^* b)$$

and its solution is given by,

$$R = QP^*$$

Thus, $$q_0 = \varepsilon \cdot (a + a (b + aa)^* b)^*$$
$$= (a + a (b + aa)^* b)^* \quad \ldots(3.25)$$

3. Substituting the value of q_0 from Equation (3.25) into the Equation (3.23).

$$q_1 = (a + a(b + aa)^*)^* a(b + aa)^* \quad \ldots(3.26)$$

4. Substituting the value of q_1 from Equation (3.26), into the Equation (3.19).

$$q_2 = (a + a(b + aa)^*)^* a(b + aa)^* a$$

Thus the R.E. representing the FA of Fig. 3.25 is given by :

$$\text{R.E.} = (a + a(b + aa)^*)^* a(b + aa)^* a$$

To use Arden's theorem, the FA should not contain ε-moves.

Example 3.4.18 : Prove that the following F.A. accepts strings with equal number of 0's and 1's; such that each prefix has at most one more 0 than 1's or at most one more 1 than 0's.

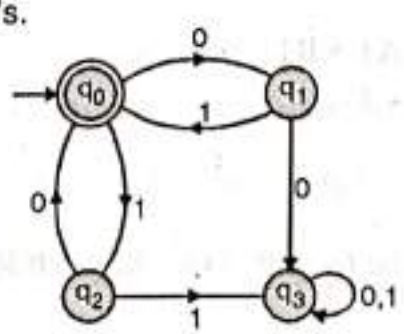


Fig. Ex. 3.4.18

Scanned by CamScanner

Solution : The state q_3 is a dead state. It is not required to equate for state q_3.

Set of equations for regular sets represented by the states q_0, q_1 and q_2 are given below.

$$\mathbf{q_0} = \varepsilon + \mathbf{q_1}\cdot 1 + \mathbf{q_2}\cdot 0 \quad \ldots(Q18.1)$$

$$\mathbf{q_1} = \mathbf{q_0}\cdot 0 \quad \ldots(Q18.2)$$

$$\mathbf{q_2} = \mathbf{q_0}\cdot 1 \quad \ldots(Q18.3)$$

Substituting values of $\mathbf{q_1}$ and $\mathbf{q_2}$ from Equation (Q18.2) and (Q18.3) in Equation (Q18.1)

$$\mathbf{q_0} = \varepsilon + \mathbf{q_0}01 + \mathbf{q_0}10$$

$$= \varepsilon + \mathbf{q_0}(01 + 10) \quad \ldots(Q18.4)$$

The Equation (Q18.4) is of the form $R = Q + RP$ with $Q = \varepsilon$, $R = \mathbf{q_0}$ and $P = (01 + 10)$ and its solution is given by $R = QP^*$

$$\therefore \mathbf{q_0} = \varepsilon\cdot(01 + 10)^* = (01 + 10)^*$$

Observations :

1. The regular set accepted by the given FA will have equal number of 0's and 1's as the R.E. = $(01 + 10)^*$ implies 01 or 10 coming zero or more times.
2. The prefix of regular set accepted by the given FA is
   (a) The set represented by $\mathbf{q_1}$ which is $(01 + 10)^*0$. Number of 0's is one more than 1's.
   (b) The set represented by $\mathbf{q_2}$ which is $(01 + 10)^*1$. Number of 1's is one more than 0's.

Example 3.4.19 : Consider the following transition diagram, convert it to an equivalent regular expression using Arden's theorem.

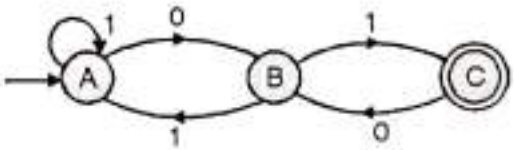


Fig. Ex. 3.4.19

Solution : The set of equations for regular sets represented by the states A, B and C are given below :

$$\mathbf{A} = \varepsilon + \mathbf{A}1 + \mathbf{B}1 \quad \ldots(1)$$

$$\mathbf{B} = \mathbf{A}0 + \mathbf{C}0 \quad \ldots(2)$$

$$\mathbf{C} = \mathbf{B}1 \quad \ldots(3)$$

Rewriting Equation (1),

$\mathbf{A} = (\varepsilon + \mathbf{B}1) + \mathbf{A}1$, it is of the from $R = Q + RP$ with $R = \mathbf{A}$; $Q = (\varepsilon + \mathbf{B}1)$ and $P = 1$ and its solution is given by $R = QP^*$

$$\therefore \mathbf{A} = (\varepsilon + \mathbf{B}1)1^* \quad \ldots(4)$$

Scanned by CamScanner

Substituting, the value of **A** from Equation (4) and the value of **C** from Equation (3) in Equation (2).

$$\begin{aligned} \mathbf{B} &= (\varepsilon + \mathbf{B}1)1^*0 + \mathbf{B}10 \\ &= 1^*0 + \mathbf{B}11^*0 + \mathbf{B}10 \\ &= 1^*0 + \mathbf{B}(11^*0 + 10) \end{aligned}$$

From Arden's theorem,

$$\mathbf{B} = 1^*0(11^*0 + 10)^* \quad \ldots(5)$$

Substituting the value of **B** from Equation (5) in Equation (3).

$$\mathbf{C} = 1^*0\,(11^*0 + 10)^*1$$

Since state C is a final state, the regular expression represented by **C** describes the FA given in the Example 3.4.18.

$$\therefore \textbf{ R.E. } = \mathbf{1^*0(11^*0 + 10)^*1}$$

Example 3.4.20 : Find the regular expression for the set strings recognized by the FA shown in Fig. Ex. 3.4.20. Use Arden's theorem.

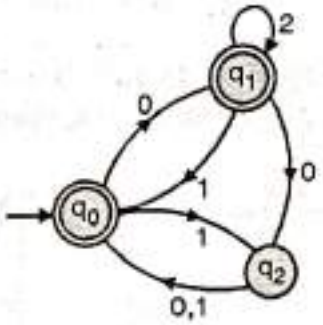


Fig. Ex. 3.4.20

Solution :

The set of equations for regular sets represented by the state q_0, q_1 and q_2 are given below.

$$\mathbf{q_0} = \varepsilon + \mathbf{q_1}1 + \mathbf{q_2}(0 + 1) \quad \ldots(1)$$

$$\mathbf{q_1} = \mathbf{q_1}2 + \mathbf{q_0}0 \quad \ldots(2)$$

$$\mathbf{q_2} = \mathbf{q_1}0 + \mathbf{q_0}1 \quad \ldots(3)$$

Substituting $\mathbf{q_2}$ from Equation (3) in Equation (1)

$$\begin{aligned} q_0 &= \varepsilon + \mathbf{q_1}1 + (\mathbf{q_1}0 + \mathbf{q_0}1)\,(0 + 1) \\ &= \varepsilon + \mathbf{q_1}1 + \mathbf{q_1}00 + \mathbf{q_1}01 + \mathbf{q_0}1\,(0 + 1) \\ &= \varepsilon + \mathbf{q_1}(1 + 00 + 01) + \mathbf{q_0}1\,(0 + 1) \end{aligned}$$

From Arden's theorem,

$$\mathbf{q_0} = [\varepsilon + \mathbf{q_1}(1 + 00 + 01)]\,(1(0 + 1))^* \quad \ldots(4)$$

Scanned by CamScanner

Putting the value of q_0 from Equation (4) in Equation (2),

$$q_1 = q_1[2 + (\varepsilon + q_1(1 + 00 + 01)] (1(0 + 1))^*0$$
$$= (1(0 + 1))^*0 + q_1[2 + (1 + 00 + 01) (1(0 + 1))^*0]$$

From Arden's theorem,

$$q_1 = (1 + (0 + 1))^*0[2 + (1 + 00 + 01) (1(0 + 1))^*0]^* \quad ...(5)$$

Putting the value of q_1 from Equation (5) in Equation (4)

$$q_0 = [\varepsilon + (1 (0 + 1))^*0[2 + (1 + 00 + 01)$$
$$(1(0 + 1))^*0]^*(1 + 00 + 01)] (1(0 + 1))^* \quad ...(6)$$

The R.E. describing the machine is given by,

$$\text{R.E.} = q_0 + q_1 \quad [\text{both } q_0 \text{ and } q_1 \text{ are final states}]$$
$$= [\varepsilon + (1 (0 + 1))^*0[2 + (1 + 00 + 01) (1(0 + 1))^*0]^*$$
$$(1 + 00 + 01)] (1(0 + 1))^* + (1(0 + 1))^*$$
$$0 [2 + (1 + 00 + 01) (1(0 + 1))^*0]$$

Example 3.4.21 : State Arden's theorem and use it to construct a regular expression corresponding to the following automata **MU - Dec. 2010**

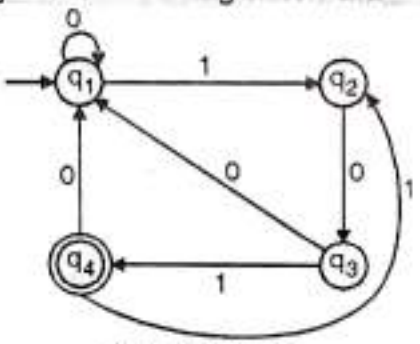


Fig. Ex. 3.4.21

Solution :

Step 1 : Expressing the automata as a set of equations.

$$q_1 = \in + q_1 0 + q_4 0 + q_3 0 \quad ...(1)$$
$$q_2 = q_1 1 + q_2 1 + q_4 1 \quad ...(2)$$
$$q_3 = q_2 0 \quad ...(3)$$
$$q_4 = q_3 1 \quad ...(4)$$

Step 2 : Solving of equations

Putting the values of q_3 and q_4 (from Equations (3) and (4)) in Equations (1) and (2).

$$q_1 = \in + q_1 0 + q_3 10 + q_3 0$$
$$= \in + q_1 0 + q_2 010 + q_2 00$$

Scanned by CamScanner

$$= \in + q_1 0 + q_2 (010 + 00) \quad \ldots(5)$$

$$q_2 = q_1 1 + q_2 1 + q_3 11$$

$$= q_1 1 + q_2 1 + q_2 011$$

$$= q_1 1 + q_2 (1 + 011) \quad \ldots(6)$$

From Equation (6)

$$q_2 = q_1 1 (1 + 011)^* \quad \ldots(7)$$

From Equations (5) and (7)

$$q_1 = \in + q_1 0 + q_1 1 (1 + 011)^* (010 + 00)$$

$$= \in + q_1 [0 + 1 (1 + 011)^* (010 + 00)]$$

$$\therefore \quad q_1 = [0 + 1 (1 + 011)^* (010 + 00)]^* \quad \ldots(8)$$

From equations (7) and (8)

$$q_2 = [0 + 1 (1 + 011)^* (010 + 00)]^* 1 (1 + 011)^* \quad \ldots(9)$$

Now,

$$q_4 = q_3 1 = q_2 01$$

$$= [0 + 1 (1 + 011)^* (0101 + 00)]^* 1 (1 + 011)^* 01$$

Which is the required RE.

3.5 FA Limitations

1. FA cannot handle the following types of languages.
   a) Context Free Language
   b) Context Sensitive Language
   c) Recursively enumerable language
2. FA cannot be used for computation.
3. FA is not allowed to modify its own input. A turing machine can modify its own input.
4. FA does not have memory to store variables. Therefore, it cannot be used for solving of general problems.
5. FA can be used for a subset of language, which is regular.

3.6 Pumping Lemma for Regular Language

MU - Dec. 2005, Dec. 2007, May 2009, Dec. 2009, May 2013, Dec. 2016

Some languages are regular. There are other languages which are not regular. One can neither express a non-regular language using regular expression nor design finite automata for it.

Scanned by CamScanner

- Pumping lemma gives a necessary condition for an input string to belong to a regular set.
- Pumping lemma does not give sufficient condition for a language to be regular.
- Pumping lemma should not be used to establish that a given language is regular.
- **Pumping lemma should be used to establish that a given language is not regular.**
- The pumping lemma uses the pigeonhole principle which states that if n pigeons are placed into less than n holes, some holes have to have more than one pigeon in it. Similarly, a string of length ≥ n when recognized by a FA with n states will see some states repeating.

3.6.1 Definition of Pumping Lemma

MU - Dec. 2007, May 2009, Dec. 2009, May 2010, Dec. 2011, Dec. 2012, May 2014, Dec. 2016, Dec. 2017

Let L be a regular language and M = (Q, Σ, δ, q_0, F) be a finite automata with n-states. Language L is accepted by m. Let ω ∈ L and | ω| ≥ n, then ω can be written as xyz, where

(i) | y | > 0

(ii) | xy | ≤ n

(iii) $xy^i z \in L$ for all i ≥ 0 here y^i denotes that y is repeated or pumped i times.

3.6.2 Interpretation of Pumping Lemma

MU - Dec. 2009

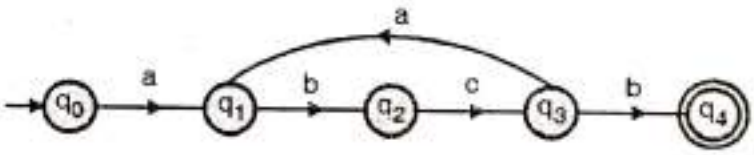


Fig. 3.6.1 : FA considered for interpretation of pumping lemma

Let us consider the FA of Fig. 3.6.1

No. of states = 5 (q_0 to q_4)

Let us take a string ω with | ω | ≥ 5, recognized by the FA.

ω = abcabcb

To recognize the string ω = abcabcb, the machine will transit through various states as shown in Fig. 3.6.2.

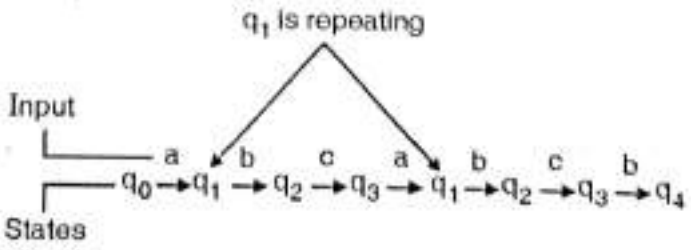


Fig. 3.6.2 : Transitions of FA on input abcabcb

Scanned by CamScanner

As the input abcabcb takes the machine through the loop $q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_1$, this loop can repeat any number of times. In terms of abcabcb, we can say that if abcabcb is accepted by FA then every string in a(bca)*bcb will be accepted by the FA of Fig. 3.6.1. The portion bca is input during the loop.

$q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4$.

Thus, if abcabcb is accepted by the FA then abcabcb can be written as xyz, with

$x = a$

$y = bca$

$z = bcb$.

- Length of abcabcb is $\geq n$
- xy^iz for every $i \geq 0$ or $a(bca)^ibcb$ for every $i \geq 0$ will be accepted by the FA of Fig. 3.6.1.

3.6.3 Proof of Pumping Lemma

MU - Dec. 2007, May 2009, May 2010, May 2012, Dec. 2016

1. Suppose L is a regular language.
2. Suppose M be a DFA with n states, such that DFA accepts the given regular language L.
   i.e. L = L(M), Language of M is same as the given regular language.
3. Let us consider a string $\omega \in L \mid |\omega| \geq n$.
   String ω can be written as
   $a_1 a_2 a_3 \ldots a_m$ with $m \geq n$.
4. Let us assume that states of M are given by $q_0, q_1, q_2, \ldots, q_{n-1}$ with q_0 as a starting state and q_{n-1} as a final state.
5. Let us assume that after feeding the first i characters of the word $\omega = a_1 a_2 \ldots a_m$, machine will be in a state r_i.
   $\delta^*(q_0, a_1 a_2, \ldots a_i) = r_i$
6. As the word ω is fed through the machine M, the machine will go through the various states as :

   $q_0 \quad r_1 \quad r_2 \quad \ldots r_{m-1} \quad q_{n-1}$ ← state

   $a_1 \quad a_2 \quad \ldots a_{m-1} \quad a_m$ ← input

   Since the length of ω, $|\omega| \geq n$ it is not possible for the machine to move through distinct states.
7. Let us assume that r_i and r_j are same. There is a loop from r_i to r_j.
8. The string ω can be divided into three parts.
   1. x = Portion before loop = $a_1 a_2 \ldots a_i$
   2. y = Portion of loop = $a_{i+1} a_{i+2} \ldots a_j$
   3. z = Portion of loop = $a_{j+1} a_{j+2} \ldots a_m$

Scanned by CamScanner

9. Since y is a portion relating to loop, it can repeat any number of times. The same is shown in Fig. 3.6.3.

$$x = a_1 a_2 \dots a_i \qquad y = a_{i+1} \dots a_j$$

$$z = a_{j+1} \dots a_m$$

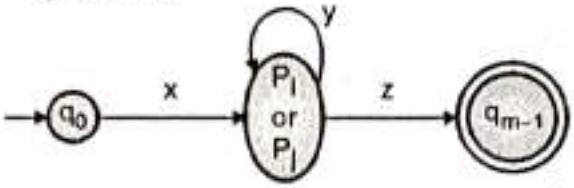


Fig. 3.6.3 : Behaviours of machine M on input xy^iz for $i \geq 0$

10. From the Fig. 3.6.3 it is clear that if $xyz \in L$ then xy^iz will also be accepted by the machine for every $i \geq 0$

3.6.4 Application of Pumping Lemma

- Pumping lemma should be used to prove that given set is not regular.
- Pumping lemma should be applied to a problem in steps as given below.

Step 1 : To prove that L is not regular, we proceed with assumption that L is regular and it is accepted by a FA with n states.

Step 2 : Now, we choose a string $\omega \in L$ such that $|\omega| \geq n$.

— which ω to select is very important and it is the key to solution using pumping lemma.

Step 3 : ω is written as xyz,

With $|xy| \leq n$

and $|y| > 0$

Now we must find an i such that $xy^iz \notin L$. This will contradict the assumption that L is regular and hence we can conclude that L is not regular.

— which i to select such that $xy^iz \notin L$ is very important.

Example 3.6.1 : Show that the language $L = \{a^n b^n\}$ is not regular.

MU - Dec. 2006, May 2010, Dec. 2010, Dec. 2012, May 2013, May 2014, Dec. 2016, May 2017, Dec. 2017

Solution :

Step 1 : Let us assume that L is regular and L is accepted by a FA with n states.

Step 2 : Let us choose a string

$$\omega = a^n b^n$$

$$|\omega| = 2n \geq n$$

Scanned by CamScanner

Let us write w as xyz, with

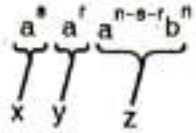
$$|y| > 0$$

and $|xy| \leq n$

Since, $|xy| \leq n$, y must be of the form $a^r \mid r > 0$

Since, $|xy| \leq n$, x must be the form a^s.

Now, $a^n b^n$ can be written as



Step 3 : Let us check whether xy^iz for i = 2 belongs to L.

$$\begin{aligned} xy^2z &= a^s(a^r)^2a^{n-s-r}b^n \\ &= a^sa^{2r}a^{n-s-r}b^n \\ &= a^{s+2r+n-s-r}b^n \\ &= a^{n+r}b^n \end{aligned}$$

Since $r > 0$, number of a's in $a^{n+r}b^n$ is greater than number of b's. Therefore, $xy^2z \notin L$. Hence by contradiction we can say that the given language is not regular.

Example 3.6.2 : Using pumping lemma for regular sets. Prove that the language. $L = \{a^m b^n \mid m > n\}$ is not regular. **MU - Dec. 2011**

Solution :

Step 1 : Let us assume that L is regular and L is accepted by a FA with n states.

Step 2 : Let us choose a string $w = a^pb^q$ such that

$$p + q > n \quad \text{and} \quad p = q + 1$$

Let us write w as xyz with

$$|y| > 0$$

and $|xy| \leq n$

y could take any of the given forms :

1. a^i .... case I
2. a^ib^j ... case II
3. b^j ... case III

Step 3 : We want to find i so that $xy^iz \notin L$ and we must consider all the three cases.

Case I :

We can take i = 0 with $xyz = xz = a^{p-i}b^q$

Since p = q + 1 and i > 0 therefore p – i is not greater than q.

$$xy^0z \notin L$$

Scanned by CamScanner

Case II :

xy^2z for i = 2 is given by

$a^{p-i}(a^i b^j)^2 b^{q-j}$

$$= a^{p-i} a^i b^j a^i b^j b^{q-j}$$
$$= a^p b^j a^i b^q \notin L$$

Case III :

$$xy^nz = a^p (b^j)^n b^{q-j}$$
$$= a^p b^{q-j+nj}$$
$$= a^p b^{q+(n-1)j}$$

Since $p = q + 1$, $n > 1$ and $j \geq 1$, it is clear that p is not greater than $q + (n - 1)j$ and hence $xy^nz \notin L$.

Thus in all the three cases we get a contradiction. Therefore, L is not regular.

Example 3.6.3 : Using pumping lemma for regular sets. Prove that the language $L = \{0^i 1^{2i} \mid i > 0\}$ is not regular. **MU - May 2011**

Solution :

Step 1 : Let us assume that L is regular and L is accepted by a FA with n states.

Step 2 : Let us choose a string $\omega = a^n b^{2n}$

$$|\omega| = 3^n \geq n$$

Let us write w as xyz, with

$$|y| > 0$$

and $|xy| \leq n$

Since $|xy| \leq n$, y must be of the form $a^r \mid r > 0$.

Since $|xy| \leq n$, x must be of the form a^s.

Now, $\omega = a^n b^{2n}$ can be written as

$$\underbrace{a^s}_{x}\ \underbrace{a^r}_{y}\ \underbrace{a^{n-s-r}\ b^{2n}}_{z}$$

Step 3 : Let us check whether xy^iz for i = 2 belongs to L.

$$xy^2z = a^s (a^r)^2 a^{n-s-r} b^{2n}$$
$$= a^s a^{2r} a^{n-s-r} b^{2n}$$
$$= a^{s+2r+n-s-r} b^{2n}$$
$$= a^{n+r} b^{2n}$$

Since $r > 0$, $a^{n+r} b^{2n}$ is not of the form $a^i b^{2i}$. Therefore, $xy^2z \notin L$. Hence by contraction, we can say that the given language is not regular.

Scanned by CamScanner

Example 3.6.4 : Using pumping lemma for regular sets prove that the language.

$L = \{0^m 1^n 0^{m+n} \mid m \geq 1 \text{ and } n \geq 1\}$ is not regular.

Solution :

Step 1 : Let us assume that L is regular and L is accepted by a FA with n states.

Step 2 : Let us choose a string.

$$\omega = 0^n 1^m 0^{m+n}$$

$$|\omega| = 2(m+n) \geq n$$

Let us write ω as xyz with

$$|y| > 0$$

and $|xy| \leq n$.

Since $|xy| \leq n$, y must be of the form $0^r \mid r > 0$

Since $|xy| \leq n$, x must be of the form 0^s.

Now,

$\omega = 0^n 1^m 0^{m+n}$ can be written as

$$\underbrace{0^s}_{x}\ \underbrace{0^r}_{y}\ \underbrace{0^{n-r-s}1^m 0^{m+n}}_{z}$$

Step 3 : Let us check whether xy^iz for $i = 2$ belongs to L.

$$xy^2z = 0^s 0^{2r} 0^{n-r-s} 1^m 0^{m+n}$$

$$= 0^{n+r} 1^m 0^{m+n}$$

Since $r > 0$, $0^{n+r} 1^m 0^{m+n}$ is not of the form

$$0^i 1^j 0^{i+j}$$

Therefore, $xy^2z \notin L$. Hence by contradiction, we can say that given language is not regular.

Example 3.6.5 : Using pumping lemma for regular sets, prove that the language $L = \{\omega\omega^R \mid \omega \in \{0, 1\}^*\}$ is not regular. **MU - May 2010**

Solution :

Step 1 : Let us assume that L is regular and L is accepted by a FA with n states.

Step 2 : Let us choose a string

$$\omega = \underbrace{a^n b}_{\omega}\ \underbrace{ba^n}_{\omega^R} \leftarrow \text{from } \omega\omega^R$$

$$|\omega| = 2n + 2 \geq n$$

Scanned by CamScanner

Let us write w as xyz with

$$|y| > 0$$

and $|xy| \leq n$

Since $|xy| \leq n$, x must be of the form a^s.

Since $|xy| \leq n$, y must be of the form $a^r \mid r > 0$.

Now,

$$\omega = a^n bba^n = \underbrace{a^s}_{x}\ \underbrace{a^r}_{y}\ \underbrace{a^{n-s-r}bba^n}_{z}$$

Step 3 : Let us check whether $xy^i z$ for i = 2 belongs to L.

$$xy^2z = a^s a^{2r} a^{n-s-r} bba^n = a^{n+r} bba^n$$

Since $r > 0$, $a^{n+r}bba^n$ is not of the form $\omega\omega^R$ as the strings starts with (n + r) a's but ends in (n) a's. Therefore, $xy^2z \notin L$. Hence by contradiction, we can say that the given language is not regular.

Example 3.6.6 : Using pumping lemma for regular sets. Prove that the language $L = \{ \omega\omega \mid \omega \in \{0, 1\}^* \}$ is not regular. **MU - Dec. 2006, Dec. 2010**

Solution :

Step 1 : Let us assume that the given language is regular and L is accepted by a FA with n states.

Step 2 : Let us choose a string

$$\omega = \underbrace{a^n b}_{\omega}\ \underbrace{a^n b}_{\omega} \longleftarrow \text{from } \omega\omega$$

$$|\omega| = 2n + 2 \geq n$$

Let us write ω as xyz with

$$|y| > 0$$

and $|xy| \leq n$

Since $|xy| \leq n$, x must be of the form a^s.

Since $|xy| \leq n$, y must be of the form $a^r \mid r > 0$.

Now,

$$\omega = a^n ba^n b = \underbrace{a^s}_{x}\ \underbrace{a^r}_{y}\ \underbrace{a^{n-s-r}ba^n b}_{z}$$

Step 3 : Let us check whether $xy^i z$ for i = 2 belongs to L.

$$xy^2z = a^s a^{2r} a^{n-s-r} ba^n b$$

Scanned by CamScanner

$$= a^{n+r}ba^nb$$

Since $r > 0$, $a^{n+r}ba^nb$ is not of the form $\omega\omega^R$ as the number of a's in the first half is $n + r$ and in the second half is n.

Therefore, $xy^2z \notin L$. Hence by contradiction, the given language is not regular.

Example 3.6.7 : Use pumping Lemma to show whether or not the following language is regular : $L = \{\omega\omega\omega \mid \omega \in \{a,b\}^*\}$

Solution :

Step 1 : Let us assume that L is regular and L is accepted by a Finite Automata with n states.

Step 2 : Let us chose a string

$$\omega = a^nba^nba^nb$$

$$|\omega| = 3n + 3 \geq n$$

Let us write ω as xyz, with

$$|y| > 0$$

and $|xy| \leq n$

Since, $|xy| \leq n$, y must be of the form $a^r \mid r > 0$

Since, $|xy| \leq n$, x must be of the form a^s.

Now, $a^nba^nba^nb$ can be written as

$$\underbrace{a^s}_{x}\ \underbrace{a^r}_{y}\ \underbrace{a^{n-s-r}ba^nba^nb}_{z}$$

Step 3 : Let us check whether xy^iz for $i = 0$ belongs to L.

$$xy^0z = a^s a^{n-s-r} ba^nba^nb = a^{n-r}ba^nba^nb$$

Since $r > 0$, $a^{n-r}ba^nba^nb$ is not of the form $\omega\omega\omega$. Hence by contradiction, we can say that the given language is not regular.

Example 3.6.8 : Using pumping lemma for regular sets, prove that the language, $L = \{0^n \mid n \text{ is a prime}\}$ is not regular.

MU - Dec. 2007, Dec. 2009, Dec. 2015, May 2016

Solution :

Step 1 : Let us assume that the given language is regular and L is accepted by a FA with n states.

Step 2 : Let us choose a string $\omega = a^p$, where p is a prime and $p > n$.

$$|\omega| = |a^p| = p \geq n$$

Let us write w as xyz with

$$|y| > 0$$

and $|xy| \leq n$

Scanned by CamScanner

We can assume that $y = a^m$ for $m > 0$.

Step 3 : Length of xy^iz can be written as given below :

$$|xy^iz| = |xyz| + |y^{i-1}| = p + (i-1)\,m$$

as $$|y| = |a^m| = m$$

Let us check whether $P(i-1)\,m$ is a prime for every i.

For $i = p + 1$, $p + (i-1)\,m = P + P_m = P(1+m)$.

$P(1+m)$ is not a prime as it has two factors p and $(1+m)$ and

$$|p| > 1,$$

$$|1+m| > 1$$

So $xy^{p+1}z \notin L$. Hence by contradiction the given language is not regular.

Example 3.6.9 : Using pumping lemma for regular sets, prove that the language, $L = \{a^{i^2} \mid i \geq 1\}$ is not regular.

Solution :

Step 1 : Let us assume that the given language L is regular and L is accepted by a FA with n states.

Step 2 : Let us choose a string $\omega = a^{n^2}$.

$$|\omega| = |a^{n^2}| = n^2 \geq n$$

Let us write ω as xyz with

$$|y| > 0$$

and $$|xy| \leq n$$

Step 3 : We will try to prove that xy^2z is not of the form a^{i^2} by showing that $|xy^2z|$ lies between the square of two consecutive natural numbers.

i.e. $n^2 < |xy^2z| < (n+1)^2$

A number lying between the square of two consecutive numbers can never be of the form i^2.

Let us find the length xy^2z

$$|xy^2z| = |xyz| + |y|$$

$$= n^2 + (>0)$$ as the length of $y > 0$ and length of xyz is n^2.

$$\therefore |xy^2z| > n^2 \quad \ldots(1)$$

Again, $$|xy^2z| = |xyz| + |y|$$

Since the length of $|y| \leq n$ as $|xy| \leq n$ we can say that

$$|xy^2z| \leq n^2 + n$$

or, $$|xy^2z| < n^2 + n + n + 1$$ [n + 1 is added on the right of the inequality]

or, $$|xy^2z| < (n+1)^2 \quad \ldots(2)$$

Scanned by CamScanner

From the two inequalities (1) and (2)

$$n^2 < |xy^2z| < (n+1)^2.$$

Thus $xy^2z \notin L$. Hence by contradiction, the given language is not regular.

Example 3.6.10 : Show that the language $L\{a^nba^n \mid n > 0\}$ is not regular.

MU - Dec. 2009, Dec. 2011

Solution :

Step 1 : Let us assume that L is regular and L is accepted by an FA with n states.

Step 2 : Let us choose a string

$$\omega = a^nba^n$$
$$|\omega| = 2n + 1 \geq n$$

Let us write ω as xyz, with

$$|y| > 0$$
$$\text{and } |xy| \leq n$$

Since, $|xy| \leq n$, y must be of the form $a^r \mid r > 0$

Since, $|xy| \leq n$, x must be of the form a^s.

Now, a^nb^n can be written as :

$$a^s a^r a^{n-s-r} ba^n$$

Step 3 : Let us check whether xy^iz for i = 0 belongs to L.

$$xy^0z = a^s (a^r)^0 a^{n-s-r} b\, a^n$$
$$= a^{n-r} b\, a^n$$

Since, r > 0 the string $a^{n-r}ba^n \notin L$.

Hence by contradiction we can say that the given language is not regular.

Example 3.6.11 : Prove $L = \{(ab)^n a^k : n > k, k \geq 0\}$ is not regular.

MU - May 2006, May 2008

Solution :

Step 1 : Let us assume that L is regular and L is accepted by an FA with n states.

Step 2 : Let us choose a string

$$\omega = (ab)^{n+1}a^n$$
$$|\omega| = 2(n+1) + n = 3n + 2 \geq n$$

Let us write ω as xyz, with

$$|y| > 0$$
$$\text{and } |xy| \leq n$$

The string xy will contain a maximum of n symbols from $(ab)^n$.

Step 3 : In the string xy^iz with i = 0, at least one 'a' or atleast one 'b' will be erased from $(ab)^{n+1}$ of $(ab)^{n+1}a^n$. This will lead to one of the following situations :

Scanned by CamScanner

1. Number of a's in $(ab)^n$ is equal to number of a's in a^k of $(ab)^n a^k$.
2. xy^0z will not be of the form $(ab)^n a^k$.

Therefore, $xy^0z \in L$.

Hence, this is proved by contradiction.

Example 3.6.12 : Is $L = \{a^{2n} \mid n \geq 1\}$ regular ?

Solution :

The given language is regular as we can draw an equivalent FA.

[From Kleene's theorem]

The FA is given in Fig. Ex. 3.6.12.

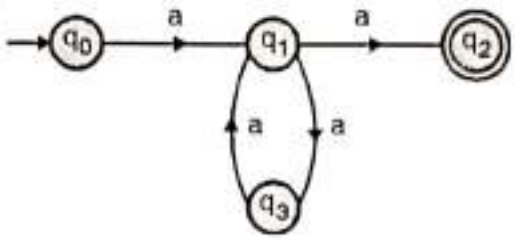


Fig. Ex. 3.6.12 : FA for a^{2n}

Example 3.6.13 : Are the following sets regular ? (Using pumping lemma) Prove the same :

$a^n b^{2m} \mid m,n > 0$ **MU - May 2007**

Solution :

(a) $a^n b^{2m}$ with m, n > 0 is regular. The corresponding FA is shown in Fig. Ex. 3.6.13 :

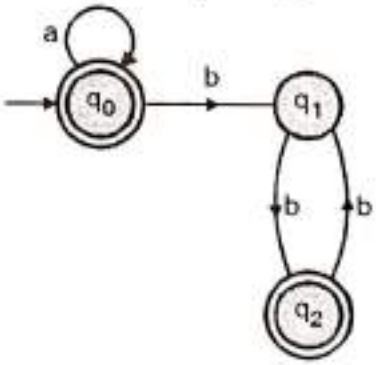


Fig. Ex. 3.6.13 : FA for $a^n b^{2m}$

3.7 Closure Properties of Regular Language

MU - Dec. 2006, May 2013, Dec. 2014

If an operation on regular languages generates a regular language then we say that the class of regular languages is closed under the above operation. Some of the important closure properties for regular languages are given below.

1. Union
2. Difference
3. Concatenation

Scanned by CamScanner

4. Intersection
5. Complementation
6. Kleene star
7. Transpose or reversal.

3.7.1 Regular Language is Closed under Union

Let $M_1 = (S, \Sigma, \delta_1, s_0, F)$ and

$M_2 = (Q, \Sigma, \delta_2, q_0, G)$ be two given automata.

To prove the closure property; we must show that there is another machine M_3 which accepts every string accepted by either M_1 or M_2 and no other string. The construction M_3 is quite simple as shown in Fig. 3.7.1.

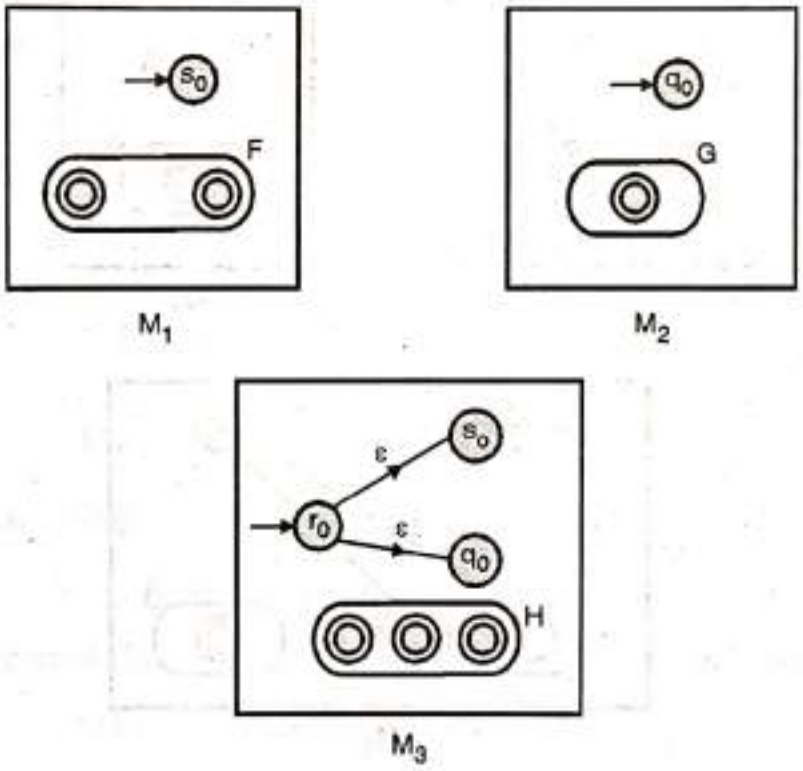


Fig. 3.7.1 : M_3 is constructed such the $L(M_3) = L(M_1) \cup L(M_2)$

Machine M_3 is constructed to accept $L(M_1) \cup L(M_2)$.

$M_3 = (R, \Sigma, \delta_3, r_0, H)$ where r_0 is a new start state. Two ε-moves, one from r_0 to s_0 and another from r_0 to q_0 are added.

$$R = S \cup Q \cup \{r_0\}$$

$$H = F \cup G$$

$$\delta_3 = \delta_1 \cup \delta_2 \cup \{(r_0, \varepsilon, s_0), (r_0, \varepsilon, q_0)\}$$

Scanned by CamScanner

Machine M_3 can non-deterministically choose either M_1 or M_2. Therefore,

$$L(M_3) = L(M_1) \cup L(M_2)$$

3.7.2 Regular Language is Closed under Concatenation

Let $M_1 = (S, \Sigma, \delta_1, s_0, F)$

and $M_2 = (Q, \Sigma, \delta_2, q_0, G)$ be two given automata.

To prove that closure property under concatenation, we must show that there is another machine M_3 such that $L(M_3) = L(M_1) \cdot L(M_2)$. The construction of M_3 is shown in Fig. 3.7.2.

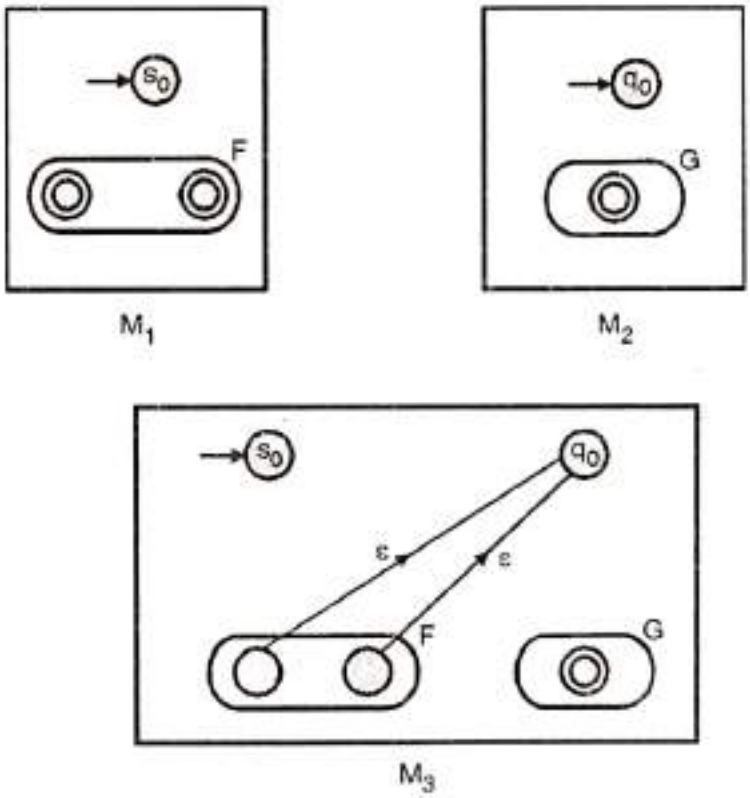


Fig. 3.7.2 : M_3 is constructed such that $L(M_3) = L(M_1) \cdot L(M_2)$

M_3 is constructed by adding ε-move from every final state of M_1 to start state of M_2.

Machine M_3 is given by :

$M_3 = (R, \Sigma, \delta_3, s_0, G)$ where

$\delta_3 = \delta_1 \cup \delta_2 \cup \{\varepsilon\text{-move from every final state of } M_1 \text{ to start state of } M_2\}$

Machine M_3 recognizes $L(M_1) \cdot L(M_2)$ by going non-deterministically from the final state of M_1 to start state of M_2.

Scanned by CamScanner

3.7.3 Regular Language is Closed under Kleene Star

Let $M_1 = (Q, \Sigma, \delta, q_0, F)$ be the given automata. We can construct a non-deterministic finite automata M_2 such that $L(M_2) = L(M_1)^*$. The construction of M_2 from M_1 is shown in Fig. 3.7.3.

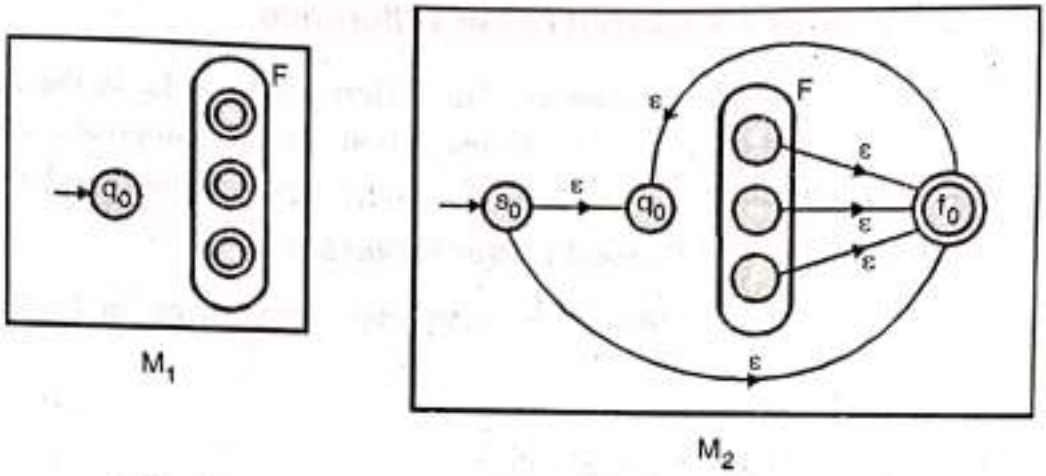


Fig. 3.7.3 : M_2 is constructed such that $L(M_2) = L(M_1)^*$

M_2 is constructed as given below :

(a) A new start state s_0 is added with an ε-move from s_0 to q_0.

(b) A new final state f_0 is added with ε-moves from every state of F to f_0. An ε-move is added from s_0 to f_0 as ε is a member of $L(M_1)^*$.

$$\text{Machine} \quad M_2 = (Q \cup \{s_0, f_0\}, \Sigma, \delta, s_0, \{f_0\})$$

Machine can accept a string $\in L(M_1)$ and resume back from the start state q_0 through the ε-move from f_0 to q_0. Thus accepting $L(M_1)^*$.

3.7.4 Regular Language is Closed under Complementation

Let $M = \{Q, \Sigma, \delta, q_0, F)$ be the given automata. To prove the closure property under complementation, we must show that there is another machine $\overline{M}$ which accepts $L(\overline{M})$ where

$$\underbrace{\overline{L(M)}}_{\text{Given machine}} = \underbrace{L(\overline{M})}_{\text{Machine after complementation}} = \Sigma^* - L(M)$$

If M is a deterministic finite automata then $\overline{M}$ can be constructed by interchanging final and non final states of M.

$$\therefore \quad \overline{M} = (Q, \Sigma, \delta, q_0, Q - F)$$

3.7.5 Regular Language is Closed under Intersection

If L_1 and L_2 are two regular languages, then

Scanned by CamScanner

$$L_1 \cap L_2 = ((L_1 \cap L_2)')' = (\bar{L}_1 \cup \bar{L}_2)' = \Sigma^* - [(\Sigma^* - L_1) \cup (\Sigma^* - L_2)]$$

Closeness under intersection follows directly from closeness under union and complementation.

3.7.6 Regular Languages are Closed under Difference

Let L_1 and L_2 are two regular languages. The difference $L_1 - L_2$ is the set of strings that are in language L_1 but not in L_2. Construction of a composite automata for $L(M_1) - L(M_2)$ is explained in Chapter 2. Thus regular languages are closed under difference.

3.7.7 Regular Languages are Closed under Reversal

Reversal of a language L is obtained by reversing every string in L. Reversal of a language L is represented by L^R.

For example,

if L = {aab, abb, aaa}, then

L^R = {baa, bba, aaa}

Let $M_1 = (Q, \Sigma, \delta, q_0, F)$ be the given automata. To prove the closure property under reversal, we must show that there is another machine M_2 which accepts $L(M_1)^R$.

or $L(M_2) = L(M_1)^R$

M_2 can be constructed from M_1 by :

1. By reversing every transition in M_1.
2. Start state of M_1 is made the only final state.
3. A new start state s_0 is added with ε-move to every final state of M_1.

Example 3.7.1 : Construct a FA accepting every string over alphabet {0, 1} ending in 01 or 10. Find the transpose of the machine or create a reverse machine.

Solution :

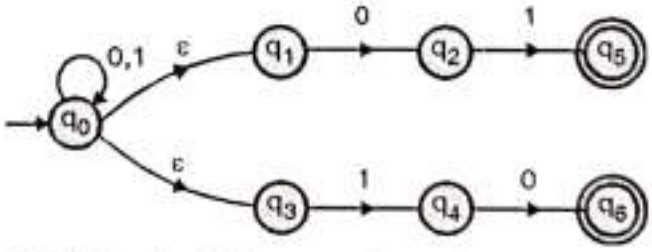


Fig. Ex. 3.7.1(a) : An NFA accepting strings ending in 01 or 10

If L is a language representing strings ending in 01 or 10. A machine for L^R is shown in Fig. Ex. 3.7.1(b).

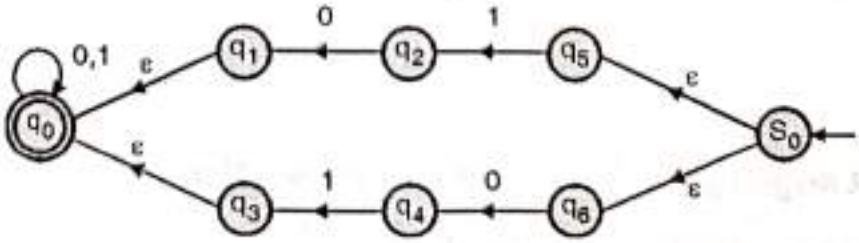


Fig. Ex. 3.7.1(b) : An NFA accepting L^R

Scanned by CamScanner

Example 3.7.2 : Explain your answer in each of the following :

(i) Every subset of a regular language is regular.

(ii) Every regular language has a regular proper subset.

Solution :

(i) Every subset of a regular language is regular.

A subset of a regular language need not be regular. Let us consider a language

$$L_1 = \{a^n b^n \mid n \geq 1\}$$

We know from the application of pumping lemma that L_1 is not regular. Language L_1 is subset of language described by $(a + b)^*$.

Thus a subset of a regular language need not be regular.

(ii) Every regular language has a regular proper subset.

(1) If the language is as empty language then it will have no proper subset. Hence the above argument is not true for an empty language.

if $L = \phi$ then it has no proper subset

(2) If L is $\{\varepsilon\}$ then ϕ is its proper subset. A language $L = \phi$ is a regular language.

(3) For any other language, let us consider a word $\omega_1 \in L$ where ω_1 is given by $a_1 a_2 \ldots a_n$, we can always construct a finite automata recognizing a language L_1 where $L_1 = \{\omega_1\}$. Thus the language has a proper subset.

Example 3.7.3 : For each statement below, decide whether it is true or false, if it is true, prove it, all parts refer to language over the alphabet {a, b}

(a) if $L_1 \subseteq L_2$ and L_1 is not regular, then L_2 is not regular

(b) if $L_1 \subseteq L_2$ and L_2 is not regular, then L_1 is not regular.

(c) if L_1 and L_2 are non regular then $L_1 \cup L_2$ is non regular.

Solution :

Let us assume that following languages.

$L_a = \{a^n b^n \mid n \geq 1\}$, L_a is not regular.

$L_b = \{\omega \mid \omega \in \{a, b\}^*\}$ L_b is regular.

$L_c = \{ab\}$, L_c is regular.

(a) If $L_1 \subseteq L_2$ and L_1 is not regular, then L_2 is not regular.

Answer = **False.**

$L_a \subseteq L_b$, L_a is not regular but L_b is regular.

(b) If $L_1 \subseteq L_2$ and L_2 is not regular, then L_1 is not regular.

Answer = **False.**

$L_c \subseteq L_a$, L_a is not regular but L_c is regular.

Scanned by CamScanner

(c) If L_1 and L_2 are non-regular then $L_1 \cup L_2$ is non-regular.

Answer = **True**.

We will not be able construct a FA for $L_1 \cup L_2$ if L_1 and L_2 are non-regular.

Example 3.7.4 : Let L be any subset of 0*. Prove that L* is regular.

Solution :

It is given that L is any subset of 0*.

DFA of L is given by :

Fig. Ex. 3.7.4 : DFA for L

L* can be considered to be regular if we can construct an FA for L*. FA for L* is given in Fig. Ex. 3.7.4(a).

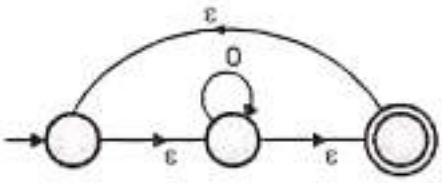


Fig. Ex. 3.7.4(a) : FA for L*

Thus, L* is regular

Example 3.7.5 : Let L be a language. It is clear from the definition that $L^+ \subseteq L^*$. Under what circumstances are they equal ?

Solution :

Let us assume that L is a language with ε belonging to it.

$$L^+ = L\,L^* = (\varepsilon + L_1)\,L^*$$

where L_1 is a language obtained from L after deleting ε from it.

$$\therefore \quad L^+ = \varepsilon\,L^* + L_1\,L^* = L^* + L_1\,L^*$$

$$= L^*, \text{ as } L_1\,L^* \subseteq L^*$$

Thus, L^+ will be equal to L* if ε is a member of L.

Example 3.7.6 : Are the following True or False. Support your answer with proofs or counter examples.

(i) If $L_1 \cup L_2$ is regular and L_1 is regular then L_2 is regular.

(ii) If $L_1 \cdot L_2$ is regular and L_1 is regular then L_2 is regular.

(iii) If L^* is regular, then L is regular.

Scanned by CamScanner

Solution :

(i) If $L_1 \cup L_2$ is regular and L_1 is regular then L_2 is regular.

– We can always construct a combined

FA for $M(L_1 \cup L_2) - M(L_1)$

$M(L_2)$ is same as $M(L_1 \cup L_2) - M(L_1)$

$\therefore L_2$ is regular (True)

(ii) If $L_1 \cdot L_2$ is regular and L_1 is regular then L_2 is regular.

– True [If L_2 is not regular then $L_1 \cdot L_2$ can not be regular]

(iii) If L^* is regular, then L is regular.

–. True

Existence of an FA for L^* will trivially imply existence of an FA for L.

Example 3.7.7 : If L is regular, prove L^T is also regular.

Solution : Let us assume L = L(M) for some FA = (Q,Σ,δ,q_0,F)

For simplicity, we can assume that F contains only one state q_n.

$$\therefore F = \{ F_n \}$$

$$\text{The set of states } Q = \{ q_0, q_1, \ldots, q_n \}$$

The Finite Automate M^T accepting L^T can be derived from the given Finite Automata M. Steps for the same are given below :

1. q_n will become the start state in M^T.
2. q_0 will become the final state in M^T.
3. For every transition $\delta(q_i,a) = q_j$ (where $a \in \Sigma$), a transition $\delta(q_j, a) = q_i$ is added to M^T.

- It is clear that M^T is FA and the language of M^T must be regular.
- If ω is accepted by M, i.e. $\delta^* (q_0,\omega) = q_n$ then ω^T will be accepted by M^T by tracing the reverse path. Therefore, $\delta^*(q_n,\omega) = q_0$.

3.8 Decision Properties of Regular Languages

MU - Dec. 2009, Dec. 2010, May 2017

There are a number of decision problems related to finite automata and regular languages. We have algorithms to answer various questions concerning regular languages.

A problem that has a decision algorithm is called decidable. A decision algorithm terminates with yes or no answer. Some of the decision problems are given below :

1. Is a regular set empty ? – Emptiness property.

Scanned by CamScanner

2. Whether a finite automata accepts a finite number of strings ? – Finiteness property.
3. Whether a finite automata accepts an infinite number of strings ? – Infiniteness property.

In addition to above decision problems, we can formulate a number of other decision problems. Some of them are :

1. Given a regular expression R and a string ω, does ω belong to L(R) ?
2. Given two FAs M_1 and M_2, is $L(M_1) = L(M_2)$?
3. Given two FAs M_1 and M_2, is $L(M_1)$ subset of $L(M_2)$?
4. Given an FA M, is M a minimum state FA accepting L(M) ?

Decision Algorithm for emptiness :

- Finite automata will fail to accept any string if it does not have a final state.
- Finite automata will fail to accept a string if none of its accepting states is reachable from the initial state.
- We can determine the emptiness of language accepted by an FA by calculating Q_k, the set of states that can be reached from q_0 by using strings of length k or less.

$$Q_k = \begin{Bmatrix} \{q_0\} & \text{if } k = 0 \\ \{Q_{k-1} \cup \{\delta(q, a)\} \mid q \in Q_{k-1} \text{ and } a \in \Sigma\} & \text{if } k > 0 \end{Bmatrix}$$

- We can go on computing the Q_k for each $k \geq 0$ until one of the two cases arise :
  1. Q_k contains a final state.
     The language is not empty.
  2. $Q_k = Q_{k-1}$
     The language is empty as the final states are not reachable from q_0.

Decision algorithm for finiteness / infiniteness :

The set of strings accepted by a finite automata M with n states is finite if and only if the finite automata accepts only strings of length less than n.

The set of strings accepted by a finite automata M with n states is infinite if and only if it accepts some string ω such that $n \leq |\omega| < 2n$.

From the pumping lemma we know :

1. If ω with length of $\omega \geq n$ is accepted by M then ω can be written as xyz.
2. For every, i xy^iz will be accepted by M.

We can always design an algorithm to generate all strings over Σ with length between n and 2n. If any of these strings is accepted by M then L(M) is infinite else L(M) is finite.

Scanned by CamScanner

3.9 Myhill - Nerode Theorem

MU - Dec. 2005, May 2006, Dec. 2006, May 2007, May 2008, Dec. 2008, Dec. 2012, May 2013

Given a language L, two strings x and y are said to be in the same class if for all possible strings z either both xz and yz are in L or both are not.

The Myhill-Nerode theorem says :

1. A language L divides the set of all possible strings into mutually exclusive classes.
2. If L is regular, the number of classes created by L is finite.
3. If the number of classes L creates is finite, then L is regular.

In finite automata, each state can be thought of as creating a class of strings. Two strings are said to be in the same class if they both trace a path from starting state q_0 to some state q_i (say).

- Number of strings is infinite.
- Number of states in an FA is finite.

Many strings when applied to the FA will end up in the same state. Each state of FA can stand for a class of strings.

3.10 Applications of RE

MU - Dec. 2012, May 2014, May 2015, Dec. 2017

3.10.1 R.E. in Unix

The UNIX regular expression lets us specify a group of characters using a pair of square brackets [ ]. The rules for character classes are :

1. **[ab]** Stand for a + b
2. **[0 – 9]** Stand for a digit from 0 to 9
3. **[A – Z]** Stands for an upper-case letter
4. **[a – z]** Stands for a lower-case letter
5. **[0 – 9A-Za – z]** Stands for a letter or a digit.

The **grep** utility in UNIX, scans a file for the occurrence of a pattern and displays those lines in which the given pattern is found.

For example :

```
$ grep president emp.txt
```

Scanned by CamScanner

It will list those lines from the file emp.txt which has the pattern "president". The pattern in grep command can be specified using regular expression.

6. * matches zero or more occurrences of previous character.
7. • matches a single character.
8. **[^ pqr]** Matches a single character which is not a p, q or r.
9. **^ pat** Matches pattern **pat** at the beginning of a line
10. **pat $** Matches pattern at end of line.

Example :

(a) The regular expression **[aA] g [ar] [ar] wal** stands for either "Agarwal" or 'agrawal".

(b) **g*** stands for zero or more occurrences of g.

(c) **$grep "A . * thakur" emp.txt** will look for a pattern starting with **A.** and ending with **thakur** in the file emp.txt.

3.10.2 Lexical Analysis :

Lexical analysis is an important phase of a compiler. The lexical analyser scans the source program and converts it into a steam of tokens. A token is a string of consecutive symbol defining an entity.

For example a C statement **x = y + z** has the following tokens :

x	–	An identifier
=	–	Assignment operator
y	=	An identifier
+	–	Arithmetic operator +
z	–	An identifier

Keywords, identifiers and operators are common examples of tokens.

The UNIX utility **lex** can be used for writing of a lexical analysis program. Input to lex is a set of regular expressions for each type of token and output of **lex** is a C program for lexical analysis.

3.11 University Questions and Answers

Dec. 2005

Q. 1 Explain how can pumping lemma be used to test if a language is regular or not. **(Section 3.6)** **(10 Marks)**

Q. 2 Write short note on Myhill-Nerode theorem. **(Section 3.9)** **(5 Marks)**

Scanned by CamScanner

May 2006

Q. 3 Convert the following regular expression to NFA with ε transitions.
$R = (1(00)^*1 + 01^*0)^*$ **(Example 3.3.10)** **(4 Marks)**
(Note : Take 'a' as '0' and 'b' as '1').

Q. 4 Show that
$(1 + 00^*1) + (1 + 00^*1)(0 + 10^*1)^*(0 + 10^*1) = 0^*1(0 + 10^*1)^*$
(Example 3.3.3) **(4 Marks)**

Q. 5 Write RE for the following languages
(i) $\Sigma = \{0, 1\}$ containing all possible combination of 0's and 1's but not having two consecutive 0's. **(Example 3.4.15(b))**
(ii) The set of all string over {0, 1} without length two. **(Example 3.3.18(i))** **(6 Marks)**

Q. 6 Prove $L = \{ (ab)^n a^k : n > k, k \geq 0 \}$ is not regular. **(Example 3.6.11)** **(5 Marks)**

Q. 7 Write short note on Myhill-Nerode theorem. **(Section 3.9)** **(4 Marks)**

Dec. 2006

Q. 8 Using pumping lemma prove that the given language is not regular.
(i) $L_1 = \{\omega\omega \mid \omega \in \{0, 1\}^*\}$ **(Example 3.6.6)**
(ii) $L_2 = \{0^i 1^i \mid i \geq 1\}$ **(Example 3.6.1)** **(10 Marks)**
(Note : Take 'a' as '0' and 'b' as '1').

Q. 9 Write short notes on closure properties of regular language and Myhill-Nerode's theorem. **(Sections 3.7 and 3.9)** **(10 Marks)**

May 2007

Q. 10 Design a NFA to accept $(a + b)^*aba$ convert it to a reduced DFA.
(Example 3.3.20(iii)) **(10 Marks)**

Q. 11 Use pumping lemma to show that the following is regular.
$L = \{ x \mid x = 0^n 1^{2m}, n > 0, m > 0\}$ **(Example 3.6.13)** **(8 Marks)**
(Note : Take '0' as 'a' and '1' as 'b').

Q. 12 Write short note on Myhill-Nerode theorem. **(Section 3.9)** **(5 Marks)**

Dec. 2007

Q. 13 Show that $0(0 + 1)^* + (0 + 1)^*00(0 + 1) = (1^*0)^*(01^*)^*$
(Similar to Example 3.3.3) **(4 Marks)**

Scanned by CamScanner

Q. 14 Write RE for following languages.

(i) $L = \{a^n b^m : (n + m) \text{ is even}\}$

(ii) $L = \{\omega \in (a, b)^* : (\text{number of a's in } \omega) \bmod 3 = 0\}$

(Example 3.3.18(II), (III)) **(5 Marks)**

Q. 15 State and prove pumping lemma for regular languages and prove that L_{pr} consisting of all strings of 1's whose length is prime is not regular.

(Sections 3.6, 3.6.1 and 3.6.3, Similar to Example 3.6.8) **(10 Marks)**

May 2008

Q. 16 Write regular expression for the following languages.

(i) $L = \{a^n b^m \mid n >= 4, m <= 3\}$

(ii) $L = \{\omega \mid |\omega| \bmod 3 = 0, \omega \in \{a, b\}^*\}$ **(Example 3.3.18(iii), (iv))** **(5 Marks)**

Q. 17 Prove $L = \{ (ab)^n a^k \mid n > k, k >= 0 \}$ is not regular. **(Example 3.6.11)** **(5 Marks)**

Q. 18 Write short note on Myhill-Nerode theorem. **(Section 3.9)** **(5 Marks)**

Dec. 2008

Q. 19 Construct a NFA for the RE $(01^* + 1)$ and convert it to DFA.

(Example 3.3.20(ii)) **(10 Marks)**

Q. 20 Write short note on Myhill-Nerode theorem. **(Section 3.9)** **(5 Marks)**

May 2009

Q. 21 Construct an NFA with ε-moves for the regular expression $10(0 + 01 + 0110)^*$

(Example 3.3.19) **(6 Marks)**

Q. 22 State the pumping lemma for regular language. **(Sections 3.6, 3.6.1, 3.6.3)** **(5 Marks)**

Dec. 2009

Q. 23 Design Finite Automata from $1(011)^*0$ RE. **(Similar to Example 3.3.13)** **(5 Marks)**

Q. 24 Construct NFA from $(0 + 1)^*(00 + 11)$ and convert into minimized DFA.

(Example 3.3.20(I)) **(10 Marks)**

Q. 25 State and explain pumping lemma for regular languages. Using pumping lemma, show that the following languages are not regular. **(Sections 3.6, 3.6.1 and 3.6.2)**

(i) $L = \{1^n \mid n \text{ is prime number}\}$ **(Example 3.6.8)**

(ii) $L = \{0^n 10^n \mid n > 0\}$ **(Example 3.6.10)** **(10 Marks)**

(Note : Take '0' as 'a' and '1' as 'b')

Q. 26 Explain decision properties for regular languages. **(Section 3.8)** **(10 Marks)**

Scanned by CamScanner

May 2010

Q. 27 Obtain a regular expression such that L (R) = { ω | ω^R ε { 0, 1 }* } with atleast three consecutive zeros. **(Example 3.6.5)** **(5 Marks)**

Q. 28 Construct reduced DFA for Language represented by
(ab/ba)* aa (ab/ba)* **(Similar to Example 3.3.20)** **(10 Marks)**

Q. 29 State and Prove Pumping Lemma theorem for regular languages.
(Sections 3.6.1 and 3.6.3) **(5 Marks)**

Q. 30 Using Pumping Lemma theorem, show that following grammars are not regular:-
(i) $L = \{ a^n b^n \mid n \geq 0 \}$ **(Example 3.6.1)** **(5 Marks)**
(ii) $L = \{ \omega\omega^R \mid \omega \varepsilon (0,1)^* \}$ **(Example 3.1.5)** **(5 Marks)**

Dec. 2010

Q. 31 What is a regular expression? Give formal definition of a regular expression. Design a DFA corresponding to the regular expression (a+b) * aba(a+b)*
(Sections 3.1, 3.3 and Example 3.3.21) **(10 Marks)**

Q. 32 State Arden's theorem and use it to construct a regular expression corresponding to the following automata

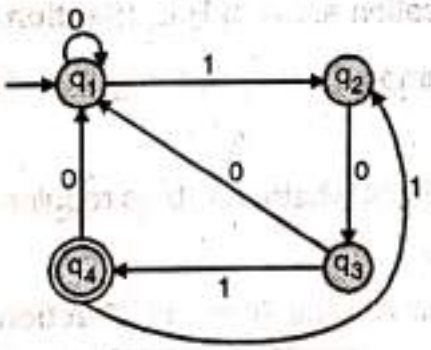


Fig. Q. 32

(Section 3.4.2 and Example 3.4.21) **(10 Marks)**

Q. 33 Using pumping lemma prove that the following languages are not regular
(i) L1 = {$\omega\omega$ | ω ε {0,1)* } **(Example 3.6.6)** **(5 Marks)**
(ii) L2 = { 0^i1^i | i > = 1} **(Example 3.6.1)** **(5 Marks)**

Q. 34 List and explain decision properties of regular languages, Explain the test for checking emptiness of a regular language. **(Section 3.8)** **(10 Marks)**

Scanned by CamScanner

May 2011

Q. 35 Give and explain the formal statement of Pumping Lemma for regular languages and use it to prove that the following language is not regular
$L = a^n b^{2n} \mid n > 0\}$ **(Similar to Example 3.6.3)** **(10 Marks)**

Dec. 2011

Q. 36 Convert the following regular expression to NFA with ϵ-transitions :
$R = (1\ (00)^*\ 1 + 01^*\ 0)^*$ **(Example 3.3.10) (Note : 0 as a and 1 as b)** **(5 Marks)**

Q. 37 Give the statement of Pumping Lemma for regular Languages.
(Section 3.6.1) **(2 Marks)**

Q. 38 Using pumping Lemma, show that following grammars are not regular : **(10 Marks)**
(i) $L = \{a^n\ ba^n \mid n \geq 0\}$ **(Example 3.6.10)**
(ii) $L = \{o^i\ 1^j \mid i \geq j\}$ **(Similar to Example 3.6.2)**

May 2012

Q. 39 State and prove the Pumping Lemma for Regular Language. **(5 Marks)**
(Section 3.6.3)

Dec. 2012

Q. 40 Write short note on application areas of R.E. **(Section 3.10)** **(5 Marks)**

Q. 41 Give the statement for pumping Lemma for regular languages.
(Section 3.6.1) **(2 Marks)**

Q. 42 Using pumping lemma, check whether $a^n\ b^n$ is regular or not. **(10 Marks)**
(Example 3.6.1)

Q. 43 Write short note on : Myhill Nerode Theorem. **(Section 3.9)** **(5 Marks)**

May 2013

Q. 44 Give the formal definition of RE and design a DFA corresponding to the regular expression.
$(a + b)^*\ aba\ (a + b)^*$ **(Section 3.3)** **(5 Marks)**

Ans. :

The language associated with the R.E. $(a + b)^*aba(a + b)^*$ = strings with "aba" as substring.

Scanned by CamScanner

DFA for strings with aba as substring.

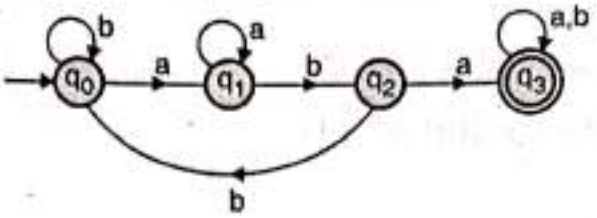


Fig. Q. 44

Q. 45 Define the following term : Pumping lemma. **(Section 3.6)** **(3 Marks)**

Q. 46 Give the closure properties of regular languages. **(Section 3.7)** **(5 Marks)**

Q. 47 What is Myhill-Nerode theorem ? Explain necessary of it. **(Section 3.9)** **(5 Marks)**

Q. 48 Using pumping lemma prove that the following language is regular or not $L = \{a^n b^n \mid n >= 1\}$ **(Example 3.6.1)** **(5 Marks)**

May 2014

Q. 49 Give applications of Regular Expression. **(Section 3.10)** **(3 Marks)**

Q. 50 Give formal definition of Regular expression. Give R.E. for following :

(i) Set of all strings over {1, 0} that end with 1 and has no substring 00.

(ii) Set of all strings over {1, 0} with even number of 1's followed by odd number of 0's. **(Similar to Example 3.4.15(d))** **(5 Marks)**

Ans. :

(i) $(1 + 01)^* (1 + 01)$

Q. 51 Give and explain the formal statement of Pumping Lemma for regular languages and use it to prove that the following language is not regular :

$L = \{a^n b^n \mid n >= 1\}$

(Section 3.6.1 and Similar to Example 3.6.1) **(10 Marks)**

Q. 52 Construct an NFA with epsilon transition for the following RE.

$(00 + 11)^* (10)^*$

(Example 3.3.22) **(5 Marks)**

Dec. 2014

Q. 53 State and explain any 5 closure properties of regular languages. **(Section 3.7)** **(5 Marks)**

Scanned by CamScanner

Q. 54 Convert $(0 + \epsilon)\ (10)^*\ (\epsilon + 1)$ into NFA with ϵ-moves and hence obtain a DFA. **(10 Marks)**

Ans. : [Hint : Refer Example as Example 3.3.8(A)**]**

Step 1 : RE to NFA for $(0 + \epsilon)\ (10)^*\ (\epsilon + 1)$

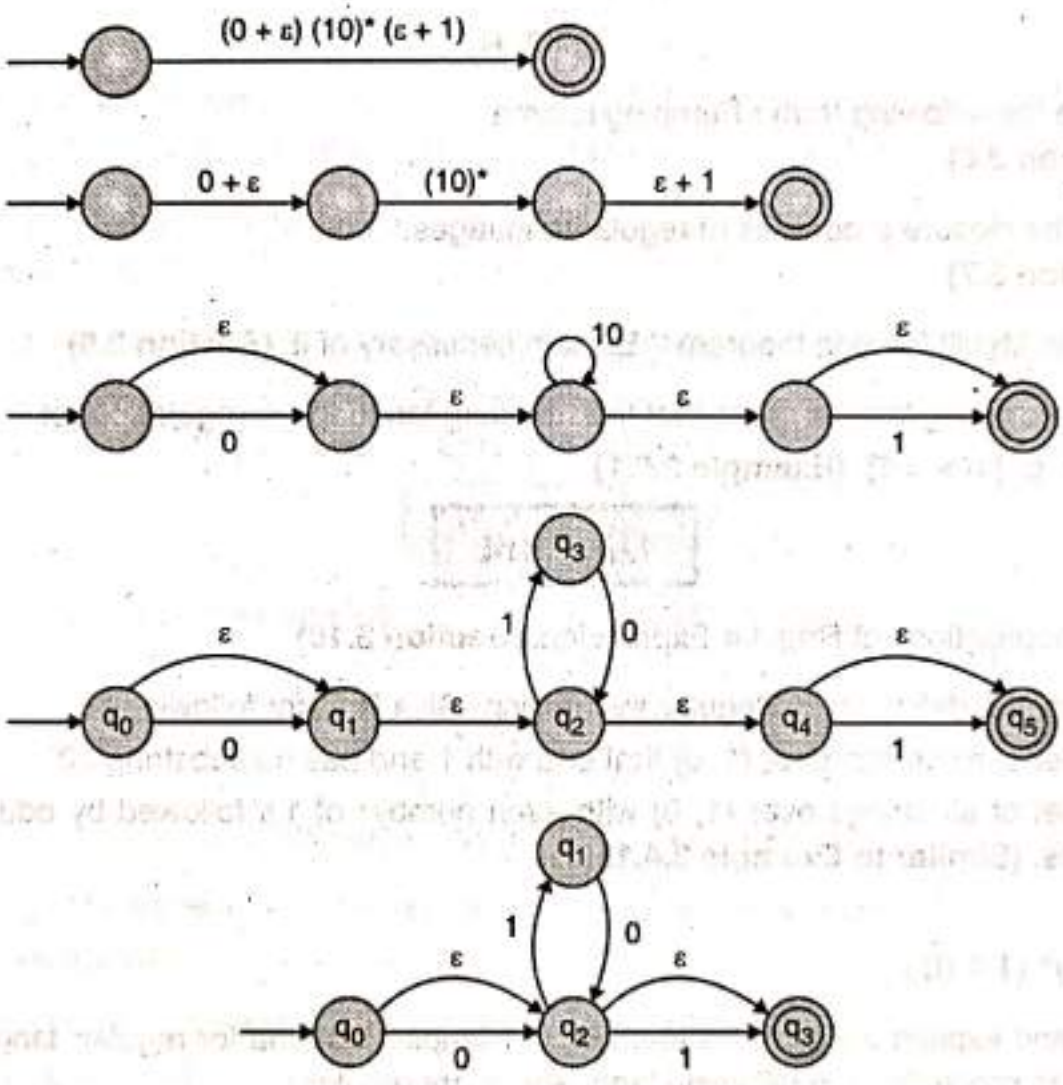


Fig. 1-Q. 54

(Note : States have been removed.)

Step 2 : ϵ-NFA to DFA

ϵ-closure of states

$q_0 \rightarrow \{q_0, q_2, q_3\}$, $q_1 \rightarrow \{q_1\}$

$q_2 \rightarrow \{q_2, q_3\}$, $q_3 \rightarrow \{q_3\}$

The DFA using the direct method is given below.

Scanned by CamScanner

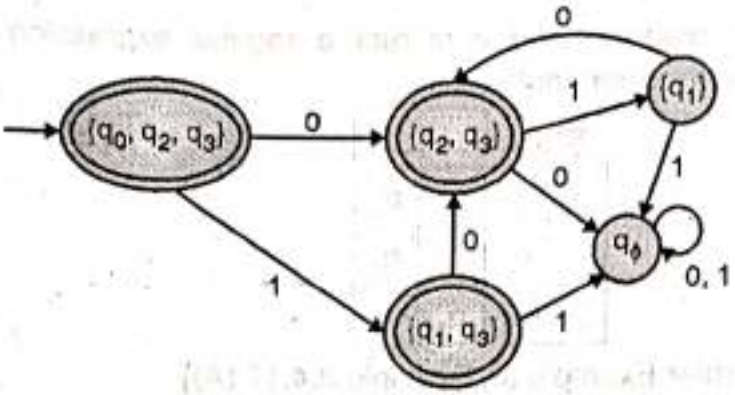


Fig. 2-Q. 54

May 2015

Q. 55 Write short note on : Applications of regular expression. **(Section 3.10)** **(5 Marks)**

Q. 56 Convert $(0 + \varepsilon)\ (10)^*(\varepsilon + 1)$ into NFA with ε-moves and obtain DFA **(Q. 2(b) of Dec. 2014)** **(10 Marks)**

Dec. 2015

Q. 57 Prove that Language $L = \{0^n \mid n \text{ for } n = 0, 1, 2, ...\}$ is not regular. **(Example 3.6.8)** **(5 Marks)**

Q. 58 Write regular expression for the following languages.

(i) Language containing all the strings in which every pair of adjacent a's appears before any pair of adjacent b's over the alphabet $\Sigma = \{a, b\}$.

(ii) Language containing all the strings in which all possible combination of a's and b's is present but strings does not have two consecutive a's, over the alphabet $\Sigma\ \{a, b\}$. **(5 Marks)**

Ans. : [Hint : Refer Example as Example 3.4.15(e)**]**

(i) Regular expression for given language

$$\{\{\{a \mid b \mid a^* \mid a^+ b \mid ba^+\} \cdot (aabb)^+ \{a^+ b \mid a^+\}\}^* \mid \{a \mid b \mid a^* \mid a^+ b \mid b a^+\} \cdot \{aabb\}^+ \mid \{a \mid b \mid a^* \mid a^+ b \mid ba^+\}\}$$

May 2016

Q. 59 Show that language $L = \{0^l \mid l \text{ is prime number}\}$ is not regular. **(Example 3.6.8)** **(5 Marks)**

Dec. 2016

Q. 60 State and explain pumping lemma for regular languages. Using pumping lemma prove that the language $L = \{0^n 1^n \mid n \geq 0\}$ is not regular. **(Sections 3.6 to 3.6.3, Example 3.6.1)** **(10 Marks)**

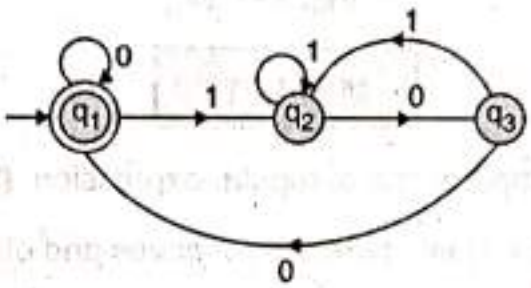
Scanned by CamScanner

Q. 61 Draw a state diagram and construct a regular expression corresponding to the following state transition table. **(10 Marks)**

State	0	1
$\rightarrow$ *q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_1	q_2

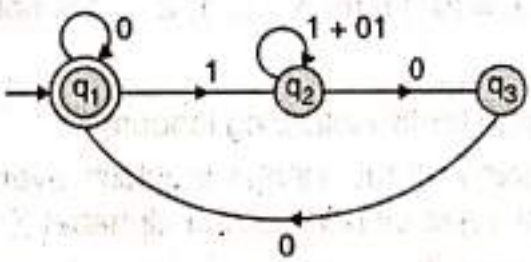
Ans. : [Hint : Please refer Example as Example 3.4.17 (A)]

State diagram

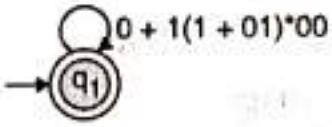


R.E. form state diagram

Step 1 : Removing loop between q_2 and q_3 we get



Step 2 : Removing the main loop, we get



Q. 62 State and explain decision properties of regular languages. **(Section 3.8)** **(10 Marks)**

May 2017

Q. 63 Using Pumping Lemma prove that following language is not regular : $L\{0^m 1^{m+1} \mid m > 0\}$. **(Example 3.6.1)** **(10 Marks)**

Q. 64 Construct NFA for given regular expressions : **(10 Marks)**

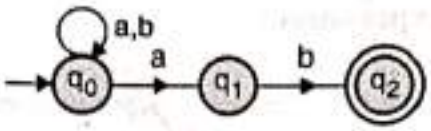
(i) (a + b)*ab (ii) aa(a + b)*b

(iii) (aba) (a + b)* (iv)(ab/ba)*|(aa/bb)*

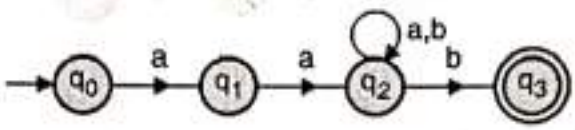
Scanned by CamScanner

Ans. : [Hint : Refer Example as Example 3.2.3(A)]

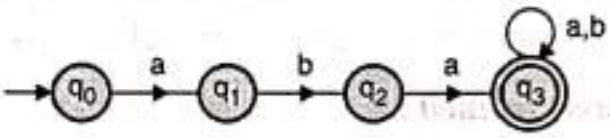
(i) (a + b)*ab : NFA



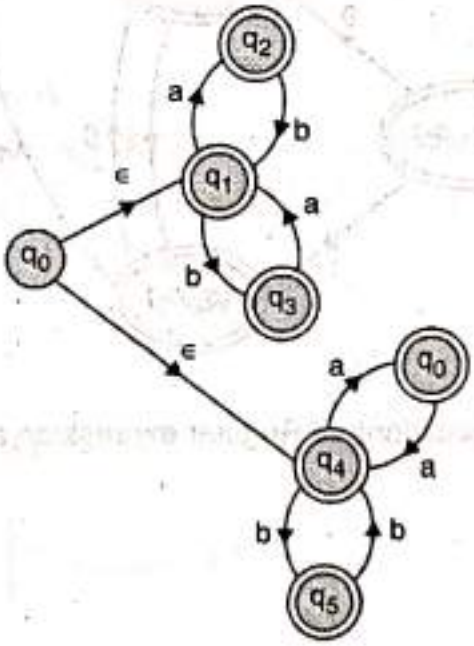
(ii) aa(a + b)*b : NFA



(iii) (aba) (a + b)* : NFA



(iv) (ab|ba)*|(aa|bb)* : NFA



Dec. 2017

Q. 65 Define Regular Expression. Design R.E. for strings ending in consecutive 1's over $\Sigma = \{0, 1\}$. **(Section 3.3)** **(5 Marks)**

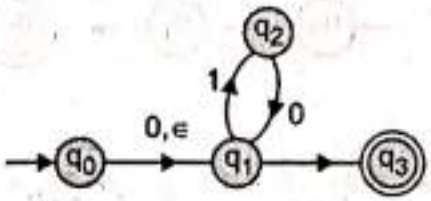
Q. 66 Give and explain formal definition of Pumping Lemma for Regular Language and prove that following language is not regular.
$L = \{ a^n b^n \mid n >= 1 \}$ **(Section 3.6.1 and Example 3.6.1)** **(10 Marks)**

Scanned by CamScanner

Q. 67 Convert (0 + ε) (10)*(ε + 1) into NFA with ε-moves and obtain DFA. **(10 Marks)**

Ans. : [Hint : Refer Example as Example 3.2.4(A)]

Step 1 : NFA for the given expression :



Step 2 : ∈-closure of states :

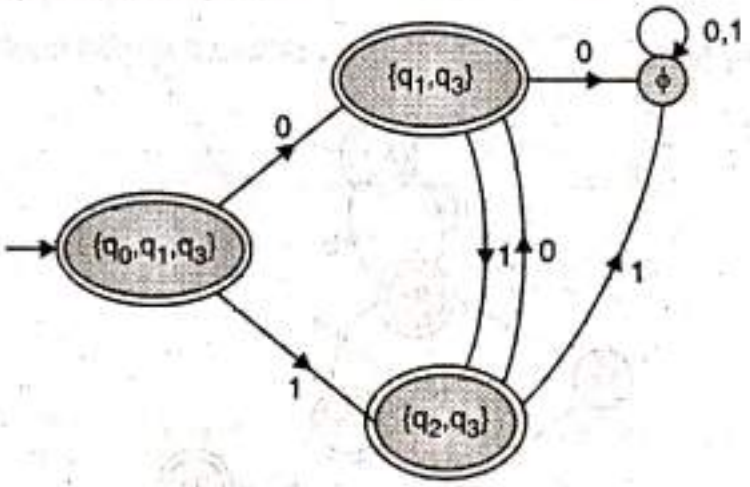
$q_0 \rightarrow \{ q_0, q_1, q_3 \}$

$q_1 \rightarrow \{ q_1, q_3 \}$

$q_2 \rightarrow \{ q_2 \}$

$q_3 \rightarrow \{ q_3 \}$

Step 3 : DFA using direct method :



Q. 68 Write short note on Applications of Regular expression and Finite automata. **(Section 3.10)** **(5 Marks)**

Chapter Ends...

□□□

Scanned by CamScanner

Regular Grammar (RG)

4.1 Regular Grammar

The language accepted by finite automata can be described using a set of productions known as regular grammar. The productions of a regular grammar are of the following form :

$A \rightarrow a$

$A \rightarrow aB$

$A \rightarrow Ba$

$A \rightarrow \varepsilon$

Where, $a \in T$ and $A, B \in V$.

A language generated by a regular grammar is known as regular language.

A regular grammar could be written in two forms :

1. Right-linear form
2. Left-linear form.

Right-linear form :

A right linear regular grammar will have production of the given form.

$A \rightarrow a$

$A \rightarrow aB$

$A \rightarrow \varepsilon$

Variable B in $A \rightarrow aB$ is the second symbol on the right.

Left-linear form :

A left linear regular grammar will have productions of the following form :

$A \rightarrow a$

$A \rightarrow Ba$

$A \rightarrow \varepsilon$

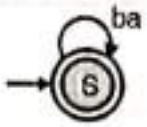
Variable B in $A \rightarrow Ba$ is the first symbol on the right.

Scanned by CamScanner

Example 4.1.1 : Construct right linear grammar and left linear grammar for the language $(ba)^*$. **MU - Dec. 2006**

Solution :

Transition system for (ba)* is given by :



We can write left linear grammar and the right linear grammar form the transition systems.

Right linear grammar :

$$S \rightarrow baS \mid \varepsilon$$

Left linear grammar :

$$S \rightarrow Sba \mid \varepsilon$$

4.2 DFA to Right-Linear Regular Grammar

Every DFA can be described using a set of production using the following steps :

Let the DFA, $M = (Q, \Sigma, \delta, q_0, F)$

Let the corresponding right linear grammar

$$G = (V, T, P, S)$$

1. Rename $q_0 \in Q$ as $S \in V$, relating start state of M with starting symbol of G.
2. Rename states of Q as A, B, C, D,.....

   Where, A, B, C, D,....$\in V$
3. Creating a set of productions P.

   3.1 If $q_0 \in F$ then add a production

   $S \rightarrow \varepsilon$ to P.

   3.2 For every transition of the form,

add a production $B \rightarrow aC$, where C is a non-accepting state.

Scanned by CamScanner

3.3 For every transition of the form

add two productions $B \rightarrow aC$, $B \rightarrow a$, where C is an accepting state.

Example 4.2.1 : Give a right linear grammar for the DFA shown in Fig. Ex. 4.2.1.

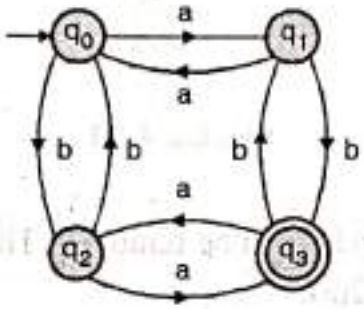


Fig. Ex. 4.2.1

Solution :

Step 1 : Renaming of states, we get the DFA shown in Fig. Ex. 4.2.1(a).

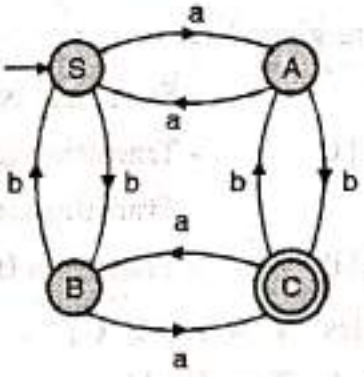


Fig. Ex. 4.2.1(a)

Step 2 : Set of productions are given by :

$$P = \begin{cases} S \rightarrow aA \mid bB \\ A \rightarrow aS \mid bC \mid b \\ B \rightarrow bs \mid aC \mid a \\ C \rightarrow aB \mid bA \end{cases}$$

where,

The set of variables $V = \{S, A, B, C\}$

The set of terminals $T = \{a, b\}$

The start symbol = S

Scanned by CamScanner

Example 4.2.2 : Give a right linear grammar for the DFA shown in Fig. Ex. 4.2.2.

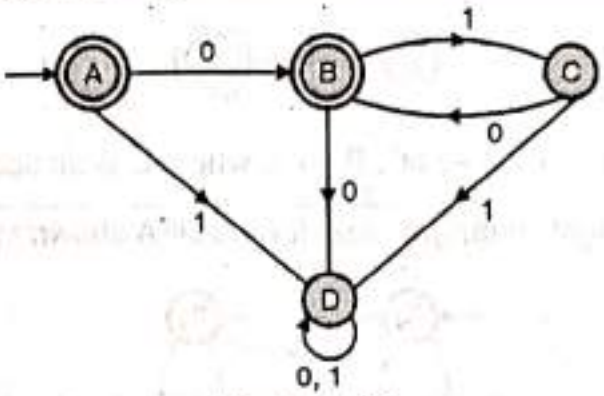


Fig. Ex. 4.2.2

Solution :

Step 1 : State D is a dead state and it can be removed. The FA after deletion of dead state D is shown in Fig. Ex. 4.2.2(a).

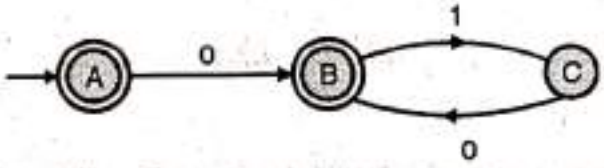


Fig. Ex. 4.2.2(a)

Step 2 : The set of productions are given below.

$$P = \begin{cases} A \rightarrow \varepsilon, & \text{- Start state is a final state} \\ A \rightarrow 0B \mid 0 & \text{- Transition from A to B on 0} \\ B \rightarrow 1C & \text{- Transition from B to C on 1} \\ C \rightarrow 0B \mid 0 & \text{- Transition from C to B on 0} \end{cases}$$

where, Set of variables V = {A, B, C}

Set of terminals T = {0, 1}

Start symbol = A.

4.2.1 Right Linear Grammar to DFA

Every right linear grammar can be represented using a DFA

- A production of the form

  $A \rightarrow aB$

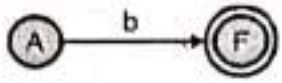
  Will generate a transition for the DFA

- A production of the form $A \rightarrow aB \mid a$ will generate a transition provided every transition entering B terminates in B.

- A Production of the form $A \rightarrow \varepsilon$ will make A a final state.

Scanned by CamScanner

- An independent production of the form A → b, will generate a transition



Where, F is a new state and it should be a final state.

Example 4.2.3 : Convert the following right-linear grammar to an equivalent DFA.

$S \rightarrow bB$
$B \rightarrow bC$
$B \rightarrow aB$
$C \rightarrow a$
$B \rightarrow b$

Solution :

Re-writing the production we get

$S \rightarrow bB$
$B \rightarrow bC \mid b$
$B \rightarrow aB$
$C \rightarrow a$

Step 1 : Adding transitions corresponding to every production, we get

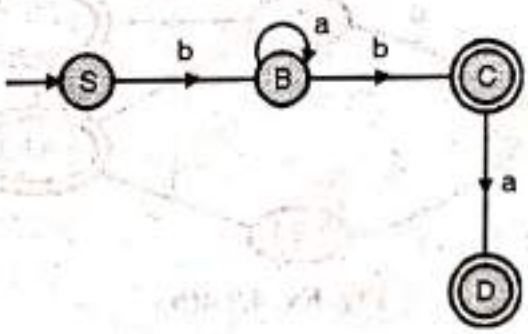


Fig. Ex. 4.2.3(a)

Step 2 : Adding a state E to handle ϕ-transitions, we get the final DFA

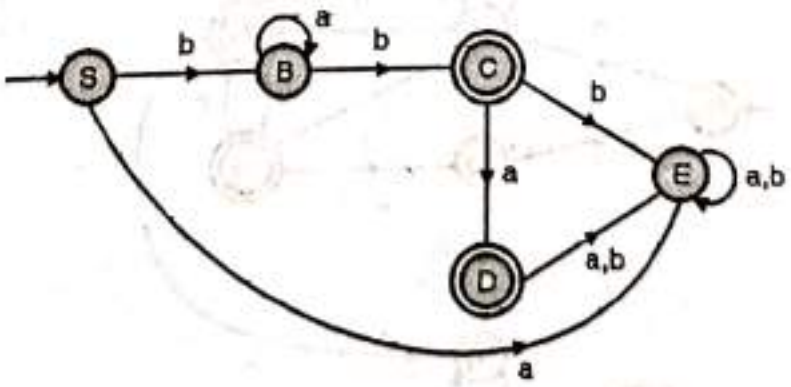


Fig. Ex. 4.2.3(b) : Final DFA

Scanned by CamScanner

Example 4.2.4 : Convert following RG to DFA

$S \rightarrow 0A \mid 1B$, $A \rightarrow 0C \mid 1A \mid 0$,

$B \rightarrow 1B \mid 1A \mid 1$, $C \rightarrow 0 \mid 0A$.

Solution : A new final state F is being introduced to handle productions like,

$A \rightarrow 0, B \rightarrow 1, C \rightarrow 0.$

Step 1 : Adding transitions corresponding to every production, we get the FA shown in Fig. Ex. 4.2.4(a).

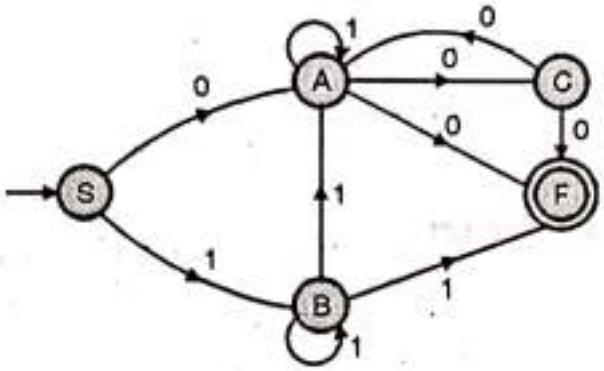


Fig. Ex. 4.2.4(a)

Step 2 : Drawing an equivalent DFA, we get

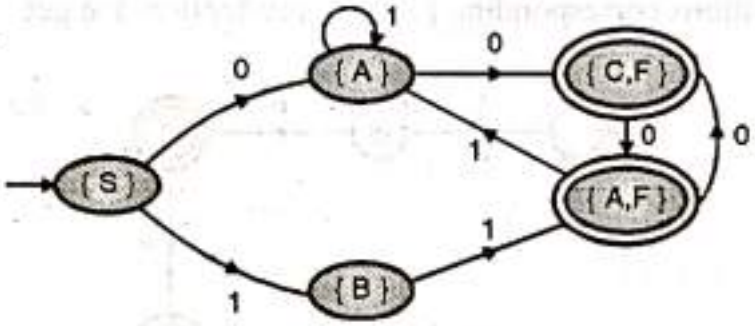


Fig. Ex. 4.2.4(b)

Step 3 : States {S}, {A}, {B}, {C,F}, and {A, F} are renamed as q_0, q_1, q_2, q_3,q_4 and a dead state q_ϕ is introduced to handle ϕ – transitions. The resulting DFA is shown in Fig. Ex.4.2.4(c) :

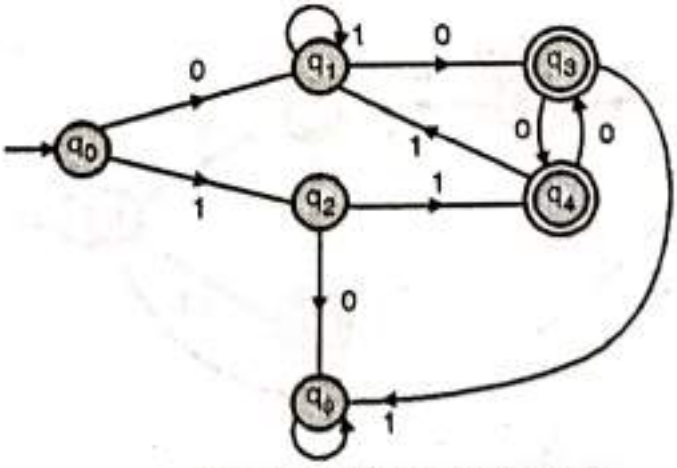


Fig. Ex. 4.2.4(c) : Final DFA

Scanned by CamScanner

Example 4.2.5 : Final the equivalent DFA accepting the regular language defined by the right linear grammar given as :

$S \rightarrow aA \mid bB,$
$A \rightarrow aA \mid bc \mid a$
$B \rightarrow aB \mid b$
$C \rightarrow bB$

MU - May 2009

Solution :

A new final state F is being introduced to handle productions like,

$A \rightarrow a, \quad B \rightarrow b$

Step 1 : Adding transitions corresponding to every production, we get the FA shown in Fig. Ex. 4.2.5(a).

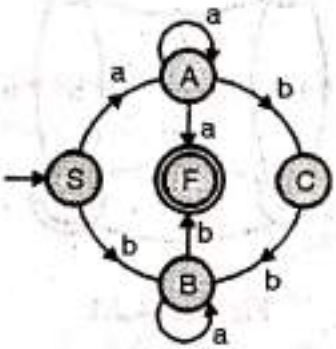


Fig. Ex. 4.2.5(a)

Step 2 : Drawing an equivalent DFA, we get :

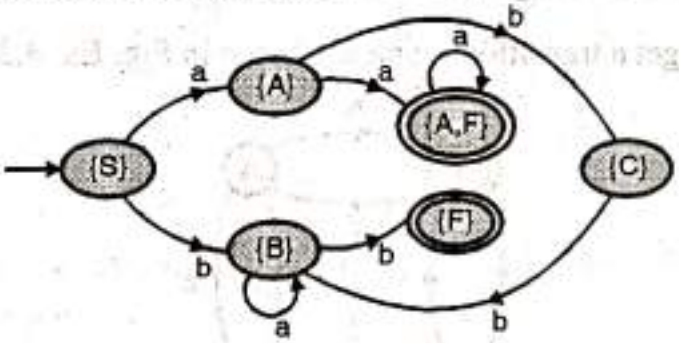


Fig. Ex. 4.2.5(b)

Step 3 : A dead state is added to handle ϕ–transition. The resulting DFA is shown in Fig. Ex. 4.2.5(c).

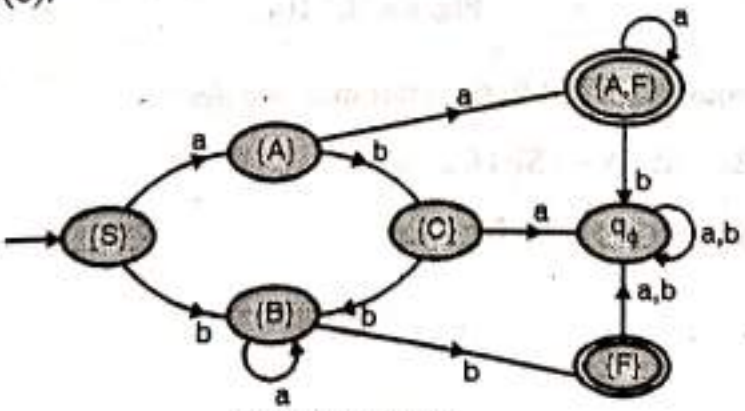


Fig. Ex. 4.2.5(c)

Scanned by CamScanner

4.3 DFA to Left-Linear Grammar

Following steps are required to write a left linear grammar corresponding to a DFA.

1. Interchange starting state and the final state.
2. Reverse the direction of all the transitions.
3. Write the grammar from the transition graph in left-linear form.

Example 4.3.1 : Give a left linear grammar for the DFA shown in Fig. Ex. 4.3.1.

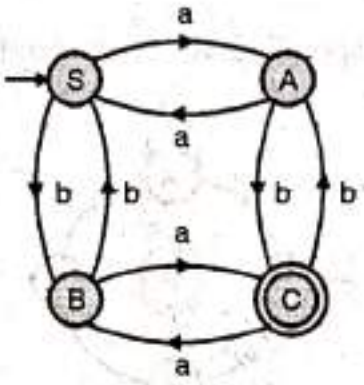


Fig. Ex. 4.3.1

Solution :

Step 1 : Interchanging the starting state and the final state and reversing the direction of transitions, we get a transition graph as shown in Fig. Ex. 4.3.1(a).

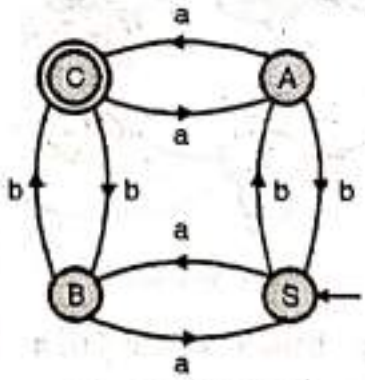


Fig. Ex. 4.3.1(a)

Step 2 : Writing an equivalent left linear grammar, we get :

$S \rightarrow Ba \mid Ab, A \rightarrow Sb \mid Ca \mid a$

$B \rightarrow Sa \mid Cb \mid b, C \rightarrow Bb \mid Aa.$

Scanned by CamScanner

Example 4.3.2 : Give a left linear grammar for the DFA shown in Fig. Ex. 4.3.2.

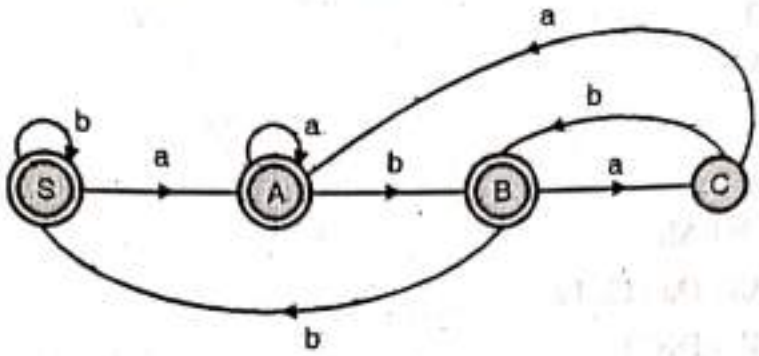


Fig. Ex. 4.3.2

Solution :

Step 1 : The given DFA contains three final states S A, and B. We can draw an equivalent DFA with a single final state D by adding ε - transitions from S to D, A to D and B to D.

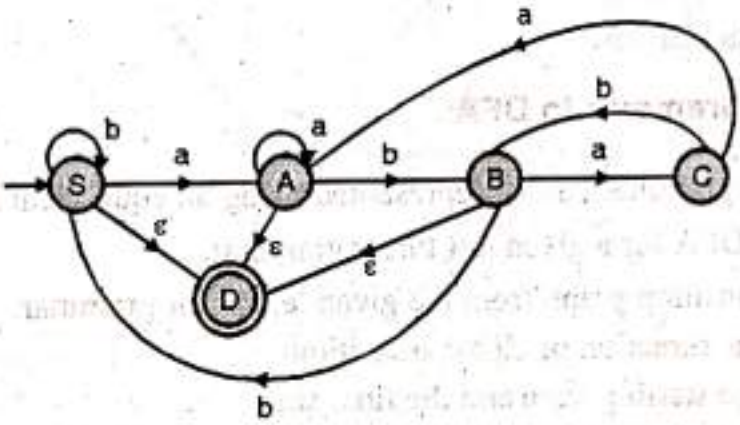


Fig. Ex. 4.3.2(a)

Step 2 : Interchanging the starting state and the final state and reversing direction of transitions, we get

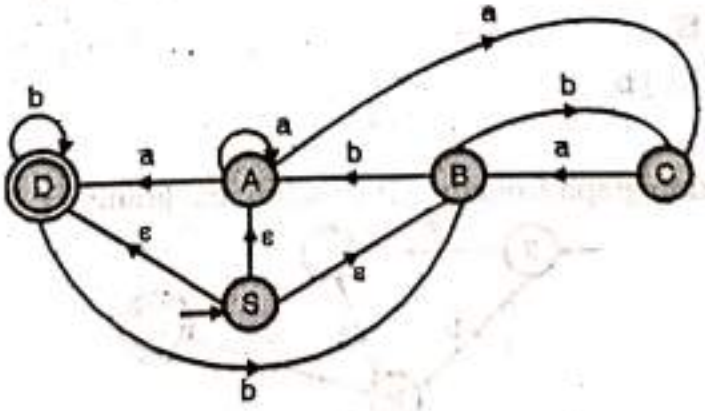


Fig. Ex. 4.3.2(b)

Scanned by CamScanner

Step 3 : Writing an equivalent left linear grammar we get :

$S \rightarrow D$
$S \rightarrow A$
$S \rightarrow B$
$C \rightarrow Ba$
$B \rightarrow Cb \mid Ab$
$A \rightarrow Aa \mid Da \mid Ca \mid a$
$D \rightarrow Bb \mid Db \mid b$

Step 4 : Removing unit productions, the resulting productions are :

$S \rightarrow Cb \mid Ab \mid Aa \mid Da \mid Ca \mid a \mid Bb \mid Db \mid b$
$C \rightarrow Ba$
$B \rightarrow Cb \mid Ab$
$A \rightarrow Aa \mid Da \mid Ca \mid a$
$D \rightarrow Ba \mid Db \mid b$

4.3.1 Left Linear Grammar to DFA

Every left linear grammar can be represented using an equivalent DFA. Following steps are required to draw a DFA for a given left linear grammar.

1. Draw a transition graph from the given left linear grammar.
2. Reverse the direction of all the transitions.
3. Interchange starting state and the final state.
4. Carry out conversion from FA to DFA.

Example 4.3.3 : Construct DFA accepting the regular language generated by the left linear grammar given below.

$S \rightarrow Ca \mid Bb$
$C \rightarrow Bb$
$B \rightarrow Ba \mid b$

Solution :

Step 1 : Draw a transition graph from the given left linear grammar.

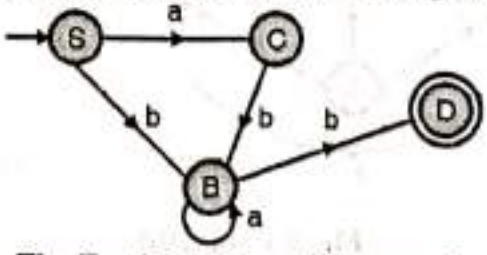


Fig. Ex. 4.3.3 : Transition graph

D is an accepting state. It is added for the production $B \rightarrow b$.

Scanned by CamScanner

Step 2 : Reverse the direction of transitions and interchange starting state and the final state.

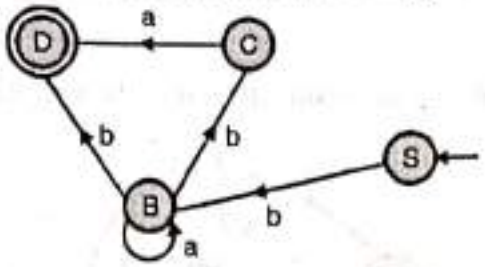


Fig. Ex. 4.3.3(a)

Step 3 : Conversion from FA to DFA

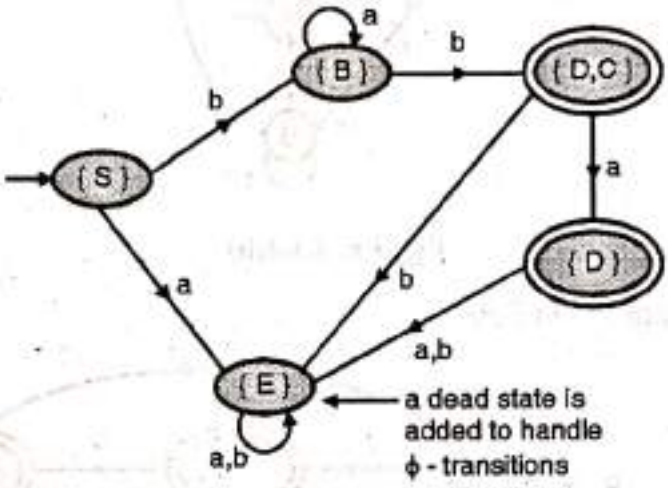


Fig. Ex. 4.3.3(b) : DFA

Example 4.3.4 : Construct DFA accepting the language generated by the left linear grammar given below.

$S \rightarrow B1 \mid A0 \mid C0$

$B \rightarrow B1 \mid 1$

$A \rightarrow A1 \mid B1 \mid C0 \mid 0$

$C \rightarrow A0$

Solution :

Step 1 : Draw a transition graph from the given left linear grammar

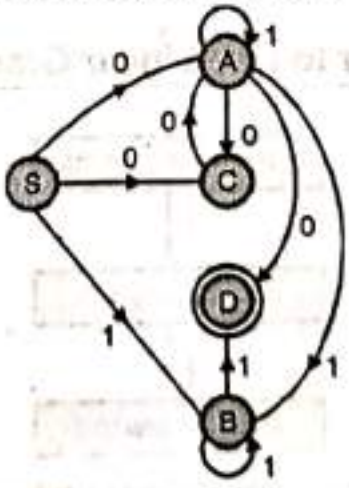


Fig. Ex. 4.3.4(a)

Scanned by CamScanner

State D is a final state. It is added to handle the following transitions :

$B \rightarrow 1, A \rightarrow 0.$

Step 2 : Reverse the direction of transitions and interchange starting state and the final state.

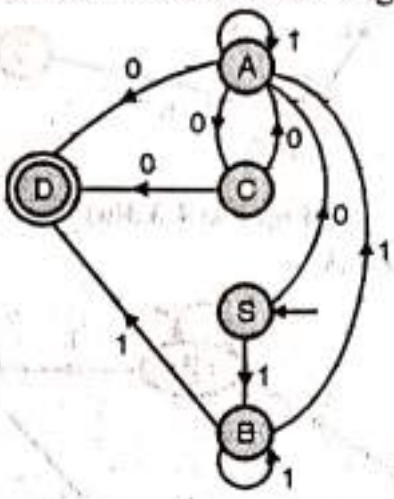


Fig. Ex. 4.3.4(b)

Step 3 : Conversion from FA to DFA.

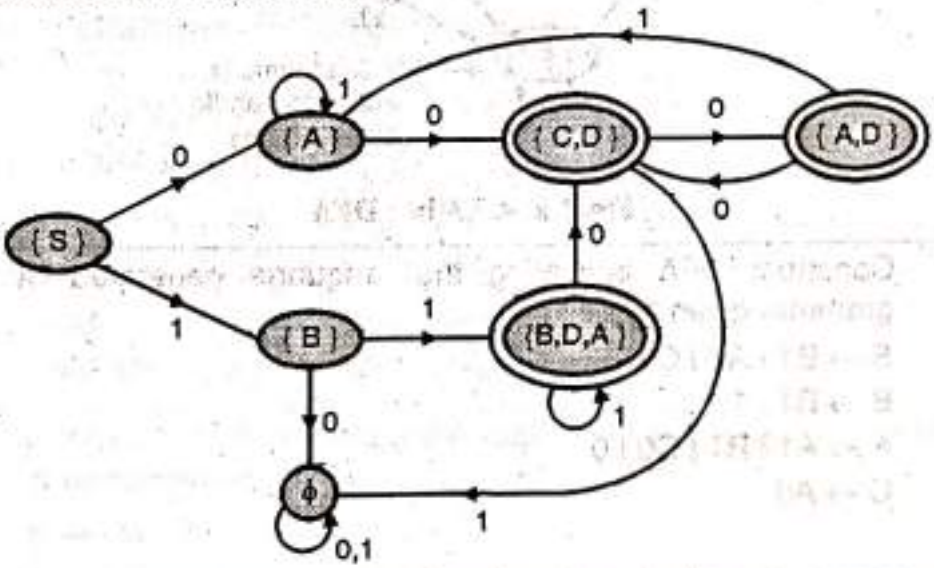


Fig. Ex. 4.3.4(c) : DFA

4.4 Right Linear Grammar to Left Linear Grammar

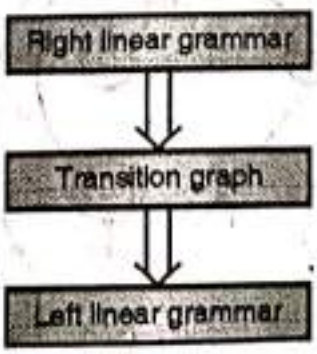


Fig. 4.4.1 : From right linear to left linear grammar

Scanned by CamScanner

Every right linear grammar can be represented by an equivalent left linear grammar. Conversion process involves drawing of an intermediate transition graph. Following steps are required :

1. Represent the right linear grammar using a transition graph. Mark the final state as
2. Interchange the start state and the final state.
3. Reverse the direction of all transitions.
4. Write left linear grammar from the transitions graph.

Example 4.4.1 : Convert the following right linear grammar to an equivalent left-linear grammar.

$S \rightarrow bB \mid b$
$B \rightarrow bC$
$B \rightarrow aB$
$C \rightarrow a$
$B \rightarrow b$

Solution :

Step 1 : Conversion of right linear grammar to transition system.

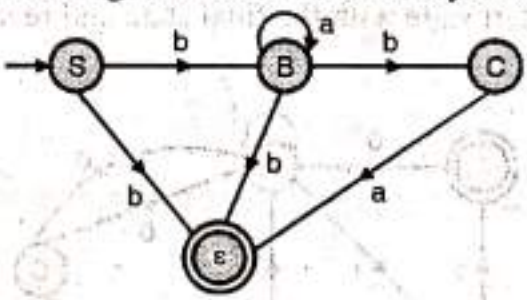


Fig. Ex. 4.4.1(a)

Step 2 : Interchanging the start state with the final state and receiving direction of transitions, we get :

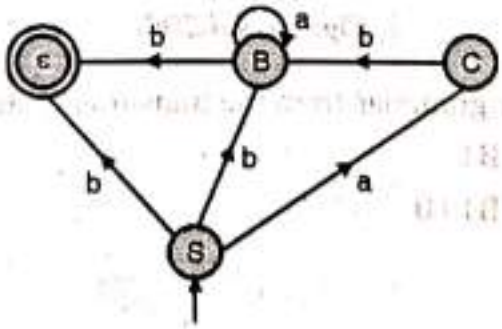


Fig. Ex. 4.4.1(b)

Step 3 : Writing of left linear grammar from the transition system, we get :

$S \rightarrow b \mid Bb \mid Ca$
$B \rightarrow Ba \mid b$
$C \rightarrow Bb$

Scanned by CamScanner

Example 4.4.2 : Write an equivalent left linear grammar from the given right linear grammar.

$S \rightarrow 0A \mid 1B$

$A \rightarrow 0C \mid 1A \mid 0$

$B \rightarrow 1B \mid 1A \mid 1$

$C \rightarrow 0 \mid 0A$

Solution :

Step 1 : Transition system for the given right linear grammar is as shown in Fig. Ex. 4.4.2(a).

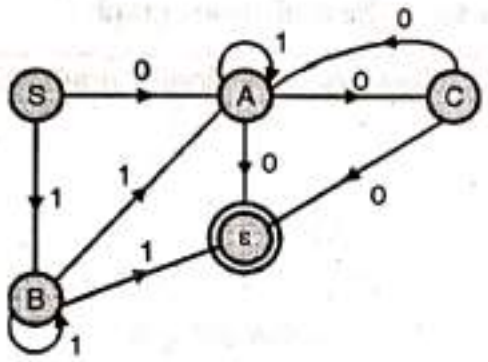


Fig. Ex. 4.4.2(a) : Transition graph

Step 2 : Interchanging the start state with the final state and reversing direction of transitions, we get

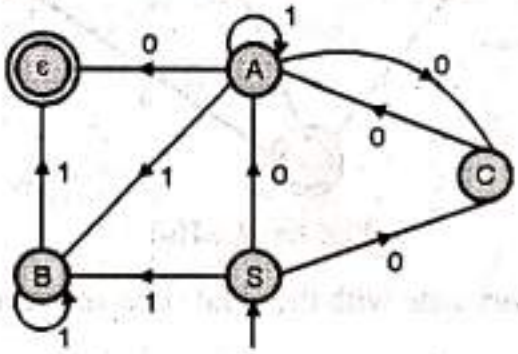


Fig. Ex. 4.4.2(b)

Step 3 : Writing of left linear grammar from the transition system, we get :

$S \rightarrow C0 \mid A0 \mid B1$

$A \rightarrow A1 \mid C0 \mid B1 \mid 0$

$B \rightarrow B1 \mid 1$

$C \rightarrow A0.$

Example 4.4.3 : For right linear grammar given below, obtain an equivalent left linear grammar.

$S \rightarrow 10A \mid 01$

$A \rightarrow 00A \mid 1$

Scanned by CamScanner

Solution :

Step 1 : Transition system from the given right linear grammar is as shown in Fig. Ex. 4.4.3(a).

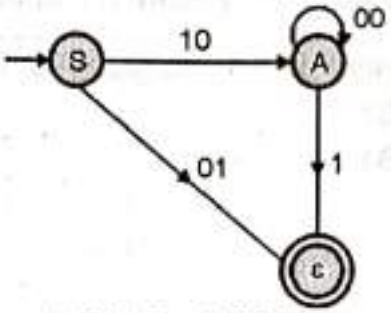


Fig. Ex. 4.4.3(a)

Step 2 : Interchanging the start state with the final state and reversing direction of transitions, we get :

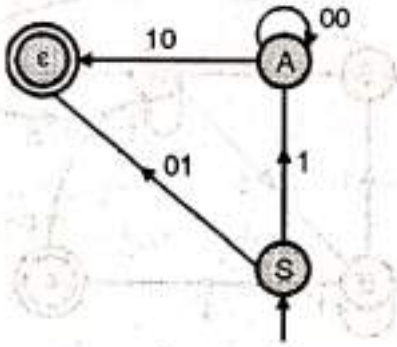


Fig. Ex. 4.4.3(b)

Step 3 : Writing of left linear grammar from the transition system, we get

$S \rightarrow A1 \mid 01$

$A \rightarrow A00 \mid 10$

4.5 Left Linear Grammar to Right Linear Grammar

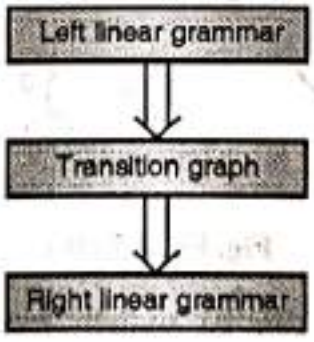


Fig. 4.5.1 : From left linear grammar to right linear grammar

Every left linear grammar can be represented by an equivalent right linear grammar. The conversion process involves drawing of an intermediate transition graph. Following steps are required.

1. Represent the left linear grammar using a transition graph. Mark the final state as

Scanned by CamScanner

2. Interchange the start state and the final state.
3. Reverse the direction of all transitions.
4. Write right - linear grammar from the transition graph.

Example 4.5.1 : Write an equivalent right linear grammar from the given left-linear grammar.

$S \rightarrow C0 \mid A0 \mid B1$
$A \rightarrow A1 \mid C0 \mid B1 \mid 0$
$B \rightarrow B1 \mid 1$
$C \rightarrow A0$

Solution :

Step 1 : Transition system for the left - linear grammar is as shown in Fig. Ex. 4.5.1(a) :

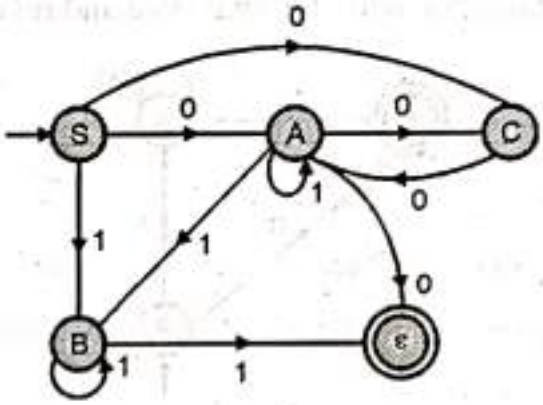


Fig. Ex. 4.5.1(a)

Step 2 : Interchanging the start state and the final state and changing direction of all transitions, we get :

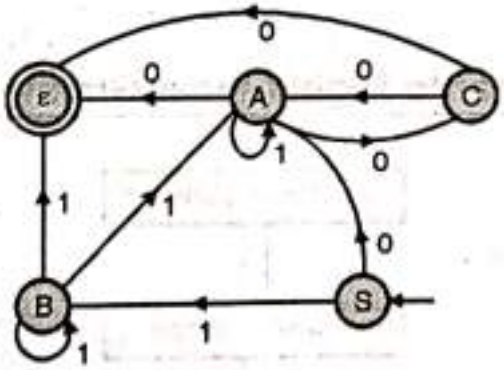


Fig. Ex. 4.5.1(b)

Step 3 : A right linear grammar can be written from the transition system. A set of productions is given below :

$S \rightarrow 1B \mid 0A$
$A \rightarrow 0C \mid 1A \mid 0$
$B \rightarrow 1B \mid 1A \mid 1$
$C \rightarrow 0C \mid 0$

Scanned by CamScanner

Example 4.5.2 : Construct the right linear grammar corresponding to the regular expression

$R = (0+1)1^*(1 + (01)^*)$

Solution :

Step 1 : The given expression can be represented using a transition system as shown in Fig. Ex. 4.5.2.

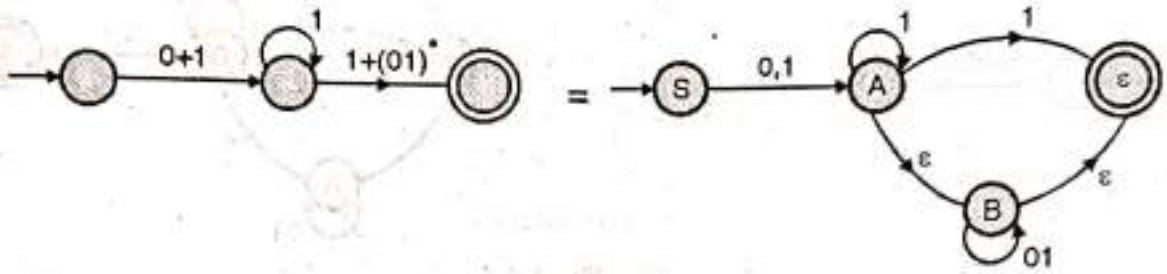


Fig. Ex. 4.5.2

Step 2 : Writing of right linear grammar from the transition system, we get :

$S \rightarrow 0A \mid 1A,\ A \rightarrow 1A \mid 1 \mid B,\ B \rightarrow 01B \mid \varepsilon$

The ε - transition. $B \rightarrow \varepsilon$, makes both A and B nullable. The ε–transitions, $B \rightarrow \varepsilon$ is removed and the resulting productions are given below.

$S \rightarrow 0A \mid 1A \mid 0 \mid 1$

$A \rightarrow 1A \mid 1$

$B \rightarrow 01B \mid 01$

Example 4.5.3 : Write an equivalent right recursive grammar for the given left recursive grammar : $S \rightarrow S10 \mid 0$

Solution :

Step 1 : Transition system for the left-linear grammar is shown in Fig. Ex. 4.5.3(a) :

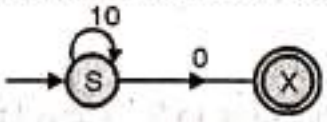


Fig. Ex. 4.5.3(a)

Step 2 : Interchanging the start state and the final state and changing direction of all transitions, we get :

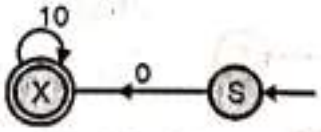


Fig. Ex. 4.5.3(b)

Step 3 : A right recursive grammar can be written from the transition system. A set of productions is given below :

$S \rightarrow 0X \mid 0$

$X \rightarrow 10X \mid 10$

Scanned by CamScanner

Example 4.5.4 : Construct the right linear grammar corresponding to the regular expression $R = (1 + (01)^*)\ 1^*\ (0 + 1)$

Solution :

The R.E. $(1 + (01)^*)\ 1^*(0 + 1)$ can be represented using a transition system as shown in Fig. Ex. 4.5.4.

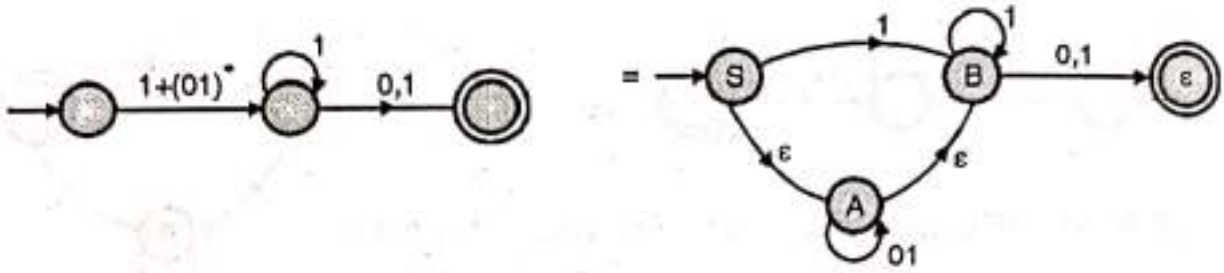


Fig. Ex. 4.5.4

Writing of right linear grammar from the transition system, we get :

$S \rightarrow 1B|A$

$A \rightarrow 01A|B$

$B \rightarrow 1B|0C|1C|0|1$

Removing unit productions $S \rightarrow A$ and $A \rightarrow B$:

$S \rightarrow 1B|01A|1B|0C|1C|0|1$

$A \rightarrow 01A|1B|0C|1C|0|1$

$B \rightarrow 1B|0C|1C|0|1$

Example 4.5.5 : Draw NFA accepting the language generated by grammar with productions :

$S \rightarrow abA \mid bB \mid aba$

$A \rightarrow b \mid aB \mid bA$

$B \rightarrow aB \mid aA$

Solution :

Step 1 : Transitions system corresponding to $S \rightarrow abA \mid bB \mid aba$ is given by

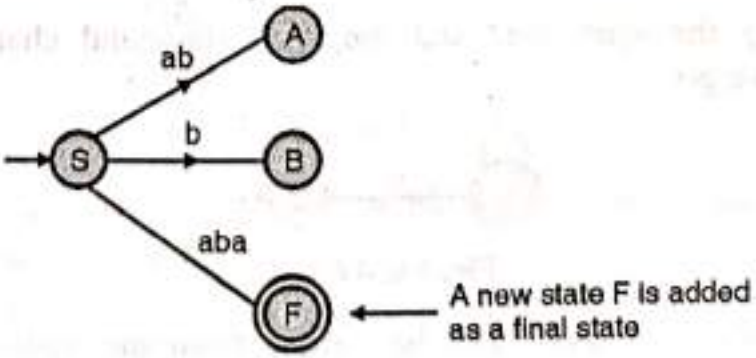


Fig. Ex. 4.5.5(a) : Transition graph

Scanned by CamScanner

Step 2 : Transitions for $A \rightarrow b \mid aB \mid bA$ and $B \rightarrow aB \mid aA$ are added to the transition graph

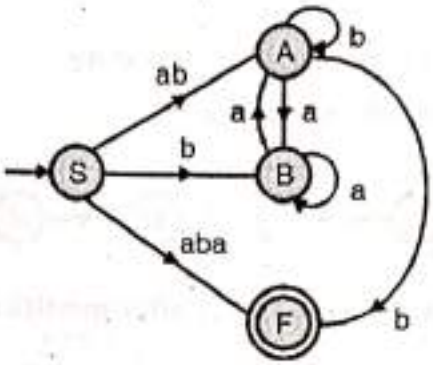


Fig. Ex. 4.5.5(b)

Step 3 : Transitions on ab and aba are expanded with resulting NFA as shown in Fig. Ex. 4.5.5(c).

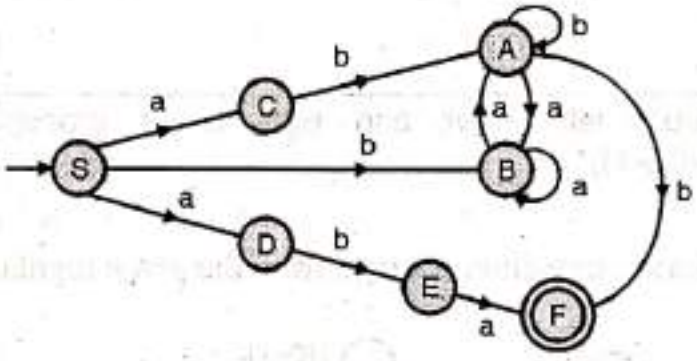


Fig. Ex. 4.5.5(c)

Example 4.5.6 : Construct right linear and left linear grammar for the language $L = \{a^n b^m, n \geq 2, m \geq 3\}$

Solution :

Step 1 : A DFA for $L = \{a^n b^m \mid n \geq 2, m \geq 3\}$ is shown in Fig. Ex. 4.5.6(a) :

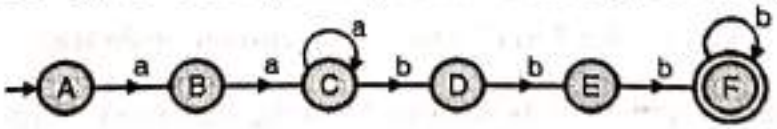


Fig. Ex. 4.5.6(a)

Step 2 : Right linear grammar can be written from the DFA.

$A \rightarrow aB$

$B \rightarrow aC$

$C \rightarrow aC \mid bD$

$D \rightarrow bE$

$E \rightarrow bF \mid b$

$F \rightarrow bF \mid b$

Scanned by CamScanner

Step 3 : Left linear grammar can be written after making the following modification in transition graph.

1. Interchange start state and the final state
2. Reverse direction of transitions.

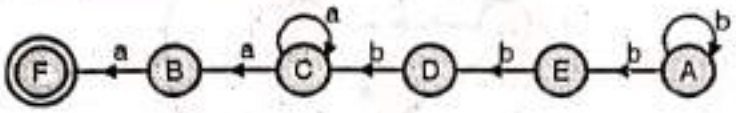


Fig. Ex. 4.5.6(b) : DFA after modifications

Step 4 : Left linear grammar can be written from the transition graph.

$A \rightarrow Ab \mid Eb$

$E \rightarrow Db$

$D \rightarrow Cb$

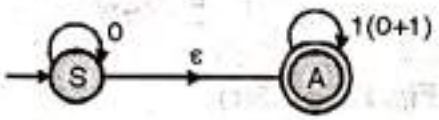
$C \rightarrow Ca \mid Ba$

$B \rightarrow a$

Example 4.5.7 : Construct left linear and right linear grammar for the language $0^* (1 (0 + 1))^*$

Solution :

Step1 : We will first draw a transition diagram from the given regular expression.



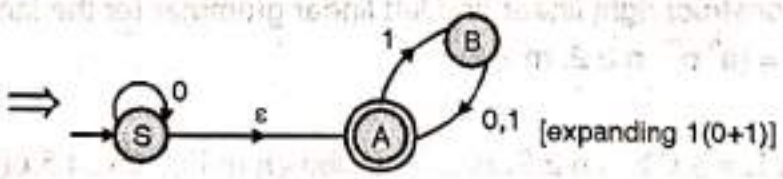


Fig. Ex. 4.5.7(a) : Transition graph for given RE

Step 2 : A right linear grammar can be written from the transition graph.

$S \rightarrow 0S \mid A \mid \varepsilon$

$A \rightarrow 1B$

$B \rightarrow 0A \mid 1A \mid 0 \mid 1$

Removing the unit production, $S \rightarrow A$, the resulting grammar is given below :

$S \rightarrow 0S \mid 1B \mid \varepsilon$

$A \rightarrow 1B$

$B \rightarrow 0A \mid 1A \mid 0 \mid 1$

Scanned by CamScanner

Step 3 : Left linear grammar can be written after making the following modifications :

1. Interchange start state and the final state
2. Reverse direction of transitions.

The resulting transition diagram with above modifications is shown in Fig. Ex. 4.5.7(b).

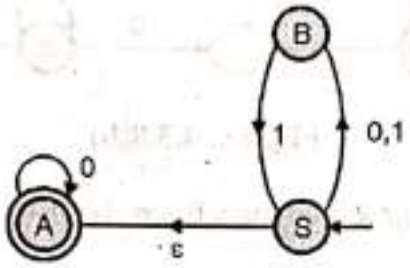


Fig. Ex. 4.5.7(b)

Step 4 : A left linear grammar can be written from the transition diagram

$S \rightarrow B0 \mid B1 \mid A \mid \varepsilon$

$B \rightarrow S1$

$A \rightarrow A0 \mid 0$

Removing the unit production, $S \rightarrow A$, the resulting grammar is given below.

$S \rightarrow B0 \mid B1 \mid A0 \mid 0 \mid \varepsilon$

$B \rightarrow S1$

$A \rightarrow A0 \mid 0$

Example 4.5.8 : Construct left linear and right linear grammar for the language $(0 + 1)^*00(0 + 1)^*$

Solution :

Step1 : We will first draw a transition diagram from the given regular expression.

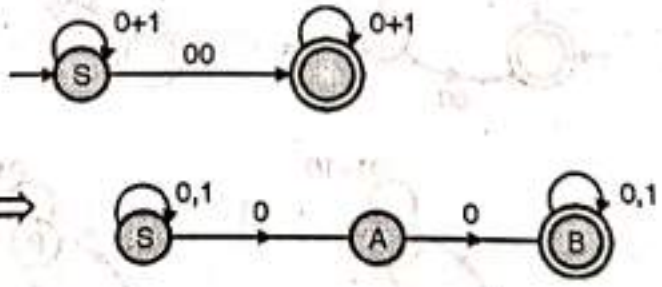


Fig. Ex. 4.5.8(a) : Transition graph for RE

Step 2 : Right linear grammar, from the transition graph can be written as given below :

$S \rightarrow 0S \mid 1S \mid 0A$

$A \rightarrow 0B \mid 0$

$B \rightarrow 0B \mid 1B \mid 0 \mid 1$

Right linear grammar

Scanned by CamScanner

Step 3 : Left linear grammar can be written after making the following modifications :

1. Interchange start state and the final state
2. Reverse direction of transitions.

Resulting transition diagram with above modifications is shown in Fig. Ex. 4.5.8(b) :

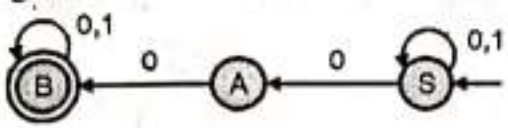


Fig. Ex. 4.5.8(b)

Step 4 : A left linear grammar can be written from the transition graph. It is given below :

$S \rightarrow S0 \mid S1 \mid A0$

$A \rightarrow B0 \mid 0$

$B \rightarrow B0 \mid B1 \mid 1 \mid 0$

Example 4.5.9 : Construct left linear and right linear grammar for the regular expression. $((01 + 10)^*11)^*00)^*$ **MU - May 2009**

Solution :

The given expression can be represented using a transition system as shown below :

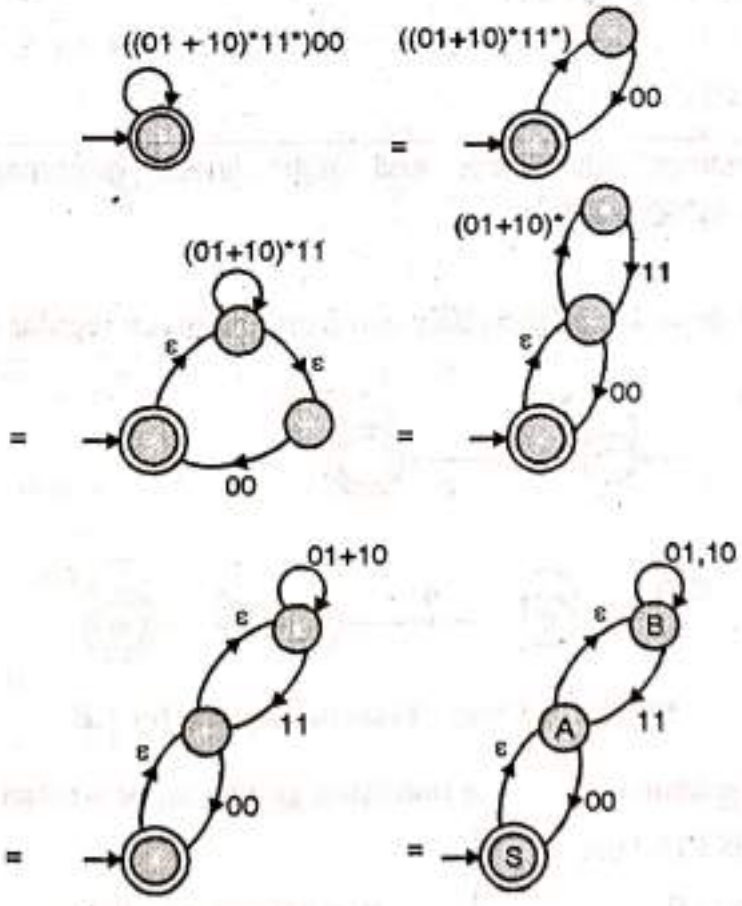


Fig. Ex. 4.5.9(a)

Scanned by CamScanner

Removing ε – transitions, we get :

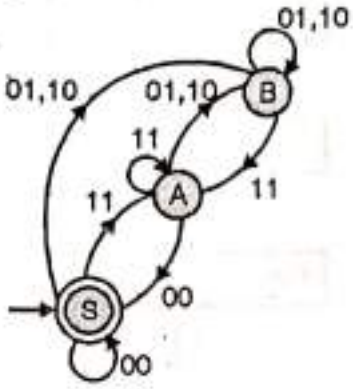


Fig. Ex. 4.5.9(b)

Writing of right linear grammar we get,

$S \rightarrow 00S \mid 11A \mid 01B \mid 10B \mid \varepsilon$

$A \rightarrow 11A \mid 01B \mid 10B \mid 00S$

$B \rightarrow 01B \mid 10B \mid 11A$

For writing of left linear grammar, we interchange the start state and the final state and change direction of all transitions. The resulting transition system is given by :

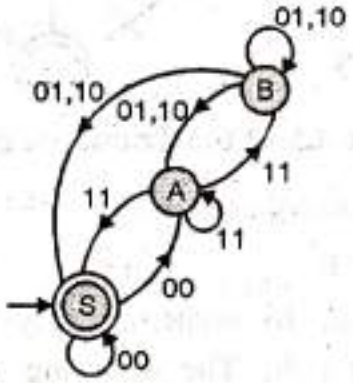


Fig. Ex. 4.5.9(c)

Writing of left linear grammar we get,

$S \rightarrow S00 \mid A00 \mid \varepsilon$

$A \rightarrow A11 \mid B11 \mid S11$

$B \rightarrow B01 \mid 10B \mid S01 \mid S10 \mid A01 \mid A10$

Example 4.5.10 : Describe the language generated by the following grammar.

$S \rightarrow bS \mid aA \mid \varepsilon$

$A \rightarrow aA \mid bB \mid b$

$B \rightarrow bS$

Solution :

A language can be understood easily from a regular expression. Thus the above problem should be solved as shown in Fig. Ex. 4.5.10 :

Scanned by CamScanner

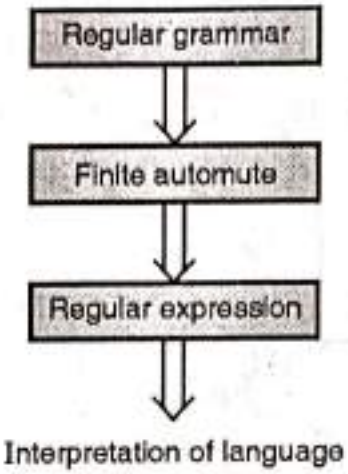


Fig. Ex. 4.5.10(a)

Step 1 : We will first draw a transition diagram from the given regular grammar.

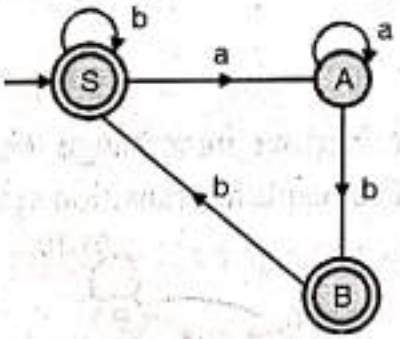


Fig. Ex. 4.5.10(b) : Transition graph

S is a final state, as $S \rightarrow \varepsilon$

B is a final state, as $A \rightarrow bB \mid b$

Step 2 : A regular expression can be written easily by moving the effect of loop $S \rightarrow A \rightarrow B \rightarrow S$ on S. The resulting transition diagram is shown in Fig. Ex. 4.5.10(c).

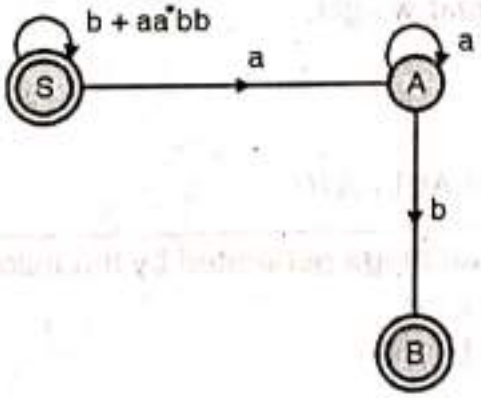


Fig. Ex. 4.5.10(c)

$\therefore$ R.E.= $(a + aa^*bb)^*[\varepsilon + aa^*b]$

Scanned by CamScanner

Language : A string of length 0 or more ending in either ab or b's but never containing aba as a substring.

Example 4.5.11 : Describe the language generated by the following grammar.

$S \rightarrow aA \mid bC \mid b$

$A \rightarrow aS \mid bB$

$B \rightarrow aC \mid bA \mid a$

$C \rightarrow aB \mid bS$

Solution :

A language can be understood easily from a regular expression or finite automata.

Step 1 : We will first draw a transition diagram from the given regular grammar.

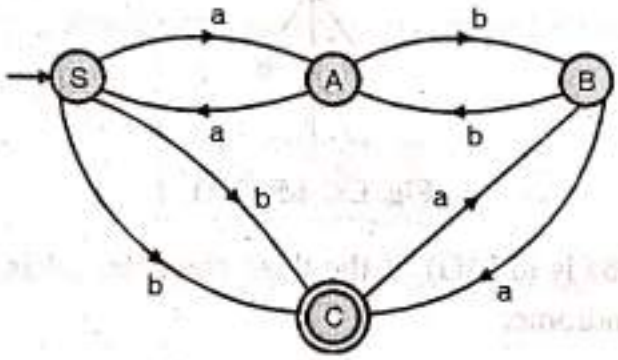


Fig. Ex. 4.5.11 : Transition graph for RE

Language generated : From the transition diagram, it clear that any path from S to C will involve :

1. Even number of a's
2. Odd number of b's

Thus the language is,

$L = \{\omega \in \{a, b\}^* \mid \omega$ contains even number of a's and odd umber of b's$\}$.

Example 4.5.12 : Describe the language generated by each of these grammar and justify your answer with the example string derive from the grammar of the productions given below.

(1) $S \rightarrow aSa|bSb|aAb|bAa$

$A \rightarrow aAa|bAb|a|b|\varepsilon$

(2) $S \rightarrow bT|aT|\varepsilon$

$T \rightarrow aS|bS$

Scanned by CamScanner

Solution :

(1) It generates a string in which there is an 'a' and if it is rewritten as 'b' then the string will become a palindrome as shown in Fig. Ex. 4.5.12(a).

Fig. Ex. 4.5.12(a)

The string abaabbabba is in L(G). If the third character 'a' is made b then the modified string abbabbabba is a palindrome.

(2) $S \rightarrow bT|aT|\varepsilon$

$A \rightarrow as|bs$

The FA representing the grammar is shown in Fig. Ex. 4.5.12(b).

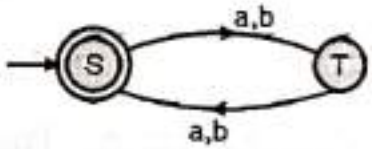


Fig. Ex. 4.5.12(b)

Thus the grammar generates the following language :

$$L(G) = \{\omega \in \{a,b\}^* \mid |\omega| \text{ is even}\}$$

Example 4.5.13 : Find the CFL associated with CFG

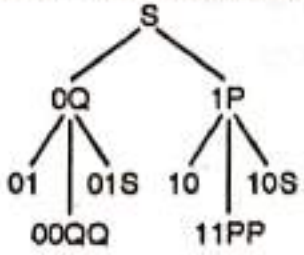
$S \rightarrow 0Q \mid 1P$

$P \rightarrow 0 \mid 0S \mid 1PP$

$Q \rightarrow 1 \mid 1S \mid 0QQ$

Solution :

This is a case of indirect recursive. S is related back to itself through P or Q.



Scanned by CamScanner

There are the following relations :

$S \rightarrow 01S \mid 10S \mid 01 \mid 10 \mid 00QQ \mid 11PP$

- Strings generated by S have equal number of 0's and 1's.
- P generates those strings, where number of 0's is one more than number of 1's.
- Q generates those strings, where number of 1's is one more than them number of 0's.

∴ CFL associated with the given CFG

$= \{w \in \{0, 1\}^* \mid w \text{ contain equal number of 0's and 1's}\}$.

4.6 University Questions and Answers

Dec. 2006

Q. 1 Construct right linear grammar and left linear grammar for the language (ba)*.

(Example 4.1.1) **(10 Marks)**

May 2009

Q. 2 Find the equivalent DFA accepting the regular language defined by the right linear grammar given as :

$S \rightarrow aA \mid bB$

$A \rightarrow a A \mid bC \mid a$

$B \rightarrow a B \mid b$

$C \rightarrow bB$ **(Example 4.2.5)** **(7 Marks)**

Q. 3 Construct left linear and right linear grammar for the regular expression.

$(((01 + 10)^*11)^*00)^*$. **(Example 4.5.9)** **(6 Marks)**

Chapter Ends...

❑❑❑

Scanned by CamScanner

Context Free Grammars (CFG)

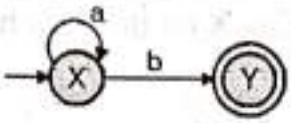
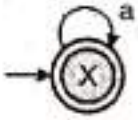
5.1 An Example to Explain Grammar

- We have seen that every finite automata M accepts a language L, which is represented by L(M).
- We have seen that a regular language can be described by a regular expression.
- We have seen that there are several languages which are not regular.
  1. $L1 = \{a^p \mid p \text{ is a prime}\}$ is not regular.
  2. $L2 = \{a^n b^n \mid n \geq 0\}$ is not regular.
  3. $L3 = \{a^{i^2} \mid i \geq 1\}$ is not regular.
  4. $L4 = \{ww \mid w \in \{a, b\}^*\}$ is not regular.
  5. $L5 = \{ww^R \mid w \in \{a, b\}^*\}$ is not regular.
- We have seen two ways for representing a language :
  1. Using finite automata.
  2. Using a regular expression.
- If a language is non-regular, it cannot be represented either using a FA or using a regular expression. Hence there was a need for representing such languages.
- Grammar is another approach for representing a language.
  1. In this approach, a language is represented using a set of equations.
  2. Equations are recursive in nature.
  3. Finite automata have a set of states. A context free grammar has a set of variables.
  4. Finite automata are defined over an alphabet. Similarly, a grammar is defined over a set of terminals.

Scanned by CamScanner

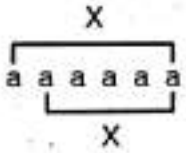
5. Finite automata have a set of transitions; a grammar has a set of equations (productions).

Table 5.1.1 : A set of DFA represented using an equivalent set of productions

Sr. No.	DFA	Equivalent grammar
1.		$X \rightarrow a$
2.		$X \rightarrow aX$ $X \rightarrow b$
3.		$X \rightarrow aX$ $X \rightarrow \varepsilon$

- The production $X \rightarrow a$, should read as X produces a or X derives a or X gives a.
- The production $X \rightarrow aX$ is a way of defining a string containing one or more a's.

If X stands for a set of strings containing one or more a's then a can be expressed in terms of X. The concept is shown below in Fig. 5.1.1.



A string of 6 a's can be generated by X
A string of 5 a's can be generated by X.

Fig. 5.1.1 : Recursive definition of $X \rightarrow aX$

A string of 6 a's = a followed by a string of 5 a's.

Or $X = aX$

Every recursive definition has a base case or termination case.

Let us consider the DFA given below in Fig. 5.1.2.

Fig. 5.1.2 : DFA taken as an example

The language of the DFA is given below,

$$L = \{\varepsilon, a, aa, aaa, ...\}$$

Scanned by CamScanner

Now, let us consider the grammar given below and see if it can generate the language generated by the DFA of Fig. 5.1.2.

$$X \rightarrow aX \quad \ldots(4\text{-}1)$$

$$X \rightarrow \varepsilon \quad \ldots(4\text{-}2)$$

- The empty string ε can be generated by the production $X \rightarrow \varepsilon$
- A string "a" can be generated as shown in Fig. 5.1.3.

First by writing X as aX and then by replacing X on the right hand side with an ε.

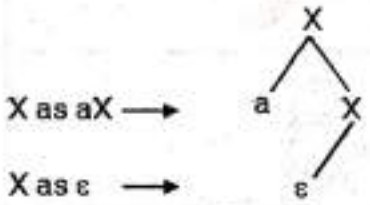


Fig. 5.1.3

- A string "aa" can be generated as shown in Fig. 5.1.4.

First by writing X as aX, secondly by replacing X on RHS with aX and finally replacing X on RHS with ε.

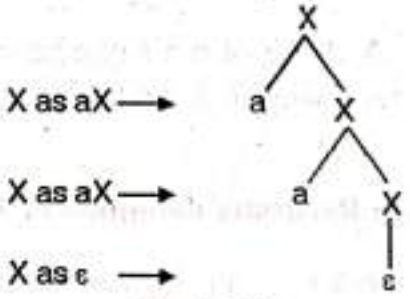


Fig. 5.1.4

- Thus the language generated by the set of productions $\{X \rightarrow aX, X \rightarrow \varepsilon\}$ is

$$L = \{\varepsilon, a, aa, \ldots\}$$

The production $X \rightarrow \varepsilon$, is for termination of recursion.

A non regular language can be expressed using a set of productions :

Let us consider a language

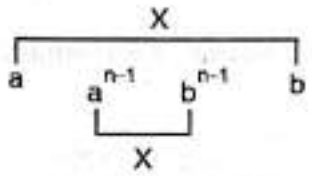
$$L = \{0^n 1^n \mid n \geq 1\}$$

Scanned by CamScanner

We know from pumping lemma that the above language is not regular; but this language can be represented using a set of productions. It will be worthwhile to understand the recursive nature of the above language.

1. A string $0, 1 \in L$ [Base case or termination case]
2. $a^n b^n$ can be written as $a(a^{n-1}b^{n-1})b$

If, $a^n b^n$ is generated by X then $a^{n-1}b^{n-1}$ can also be generated by X.



Now, we can say that,

$$X \rightarrow aXb$$

i.e. If we have a string of the form $a^n b^n$, after removing the first a and the last b, the string remains in the form $a^n b^n$.

Thus the language,

$L = \{a^n b^n \mid n \geq 1\}$ can be represented using

1. $X \rightarrow aXb$ ...(4-3)
2. $X \rightarrow ab$ ...(4-4)

The production $X \rightarrow aXb$ can recursively generate a string $a^n b^n$ and the second production $X \rightarrow ab$ is used for termination of recursion.

Let us try to generate a string $a^3 b^3$ using the two productions (4-3) and (4-4).

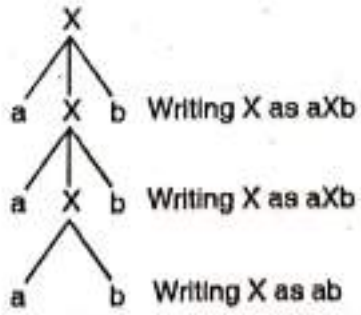


Fig. 5.1.5 : Generation of $a^3 b^3$

Conclusion : Grammar can be used to represent both.

1. Regular languages
2. Non-regular languages

Scanned by CamScanner

5.2 Context Free Grammar

A context free grammar G is a quadruple (V, T, P, S),

where,

V is a set of variables.

T is a set of terminals.

P is a set of productions.

S is a special variable called the start symbol $S \in V$.

A production is of the form

$V_i \rightarrow \alpha_i$ where $V_i \in V$ and α_i is a string of terminals and variables.

5.2.1 Notations

1. Terminals are denoted by lower case letters a, b, c … or digits 0, 1, 2 …. etc.
2. Non-terminals (variables) are denoted by capital letters A, B, …, V, W, X …
3. A string of terminals or a word $w \in L$ is represented using u, v, w, x, y, z.
4. A sentential form is a string of terminals and variables and it is denoted by α, β, γ etc.

CFG explained through an example :

Let us consider English sentences of the form :

1. Mohan eats. 2. Soham plays. 3. Ram reads.

The first word of in the above sentences is a noun and the second word is a verb. A sentence of the above form can be written as

<sentence> → <noun> <verb>

Here, noun can be replaced with Mohan, Soham or Ram and <verb> can be replaced with reads, plays or eats. We can write :

<noun> → Mohan

<noun> → Soham

<noun> → Ram

<verb> → eats

<verb> → plays

<verb> → reads

In the above example :

1. <sentence>, <noun> and <verb> are variables or non-terminals.
2. Mohan, Soham, Ram, eats, plays and reads are terminals.
3. The variable <sentence> is the start symbol as a sentence will be formed using the start symbol <sentence>.

Scanned by CamScanner

Formally, the grammar can be written as :

G = (V, T, P, S), where

V = {<sentence>, <noun>, <verb>}

T = {Mohan, Soham, Ram, eats, plays, reads}

P = {<sentence> → <noun> <verb>,
<noun> → Mohan,
<noun> → Soham,
<noun> → Ram,
<verb> → eats,
<verb> → plays,
<verb> → reads
}

S = <sentence>

Several productions of <noun> and <verb> can be merged together and the set of productions can be re-written as :

P = {<sentence> → <noun> <verb>,
<noun> → Mohan | Soham | Ram,
<verb> → eats | plays | reads
}

5.2.2 The Language of a Grammar

Every grammar generates a language. A word of a language is generated by applying productions a finite number of times. Derivation of a string should start from the start symbol and the final string should consist of terminals.

If G is a grammar with start symbol S and set of terminals T, then the language of G is the set

$$L(G) = \left\{ w \mid w \in T^* \text{ and } S \overset{*}{\underset{G}{\Rightarrow}} w \right\}$$

If the production rule is applied once, then we write $\alpha \underset{G}{\Rightarrow} \beta$. When it is applied a number of times then we write $\alpha \overset{*}{\underset{G}{\Rightarrow}} \beta$

Derivations are represented either in the

1. Sentential form OR 2. Parse tree form

Scanned by CamScanner

5.2.2.1 Sentential Form

Let us consider a grammar given below :

$S \rightarrow A1B$ (Production 4-5)

$A \rightarrow 0A \mid \varepsilon$ (Production 4-6)

$B \rightarrow 0B \mid 1B \mid \varepsilon$ (Production 4-7)

where, G is given by (V, T, P, S)

with, $V = \{S, A, B\}$

$T = \{0, 1\}$

$P = \{$Productions 4-5, 4-6 and 4-7$\}$

$S =$ Start symbol

Let us try to generate the string 00101 from the given grammar.

$S \rightarrow A1B$ [Starting production]

$\rightarrow 0A1B$ [Using the production $A \rightarrow 0A$]

$\rightarrow 00A1B$ [Using the production $A \rightarrow 0A$]

$\rightarrow 001B$ [Using the production $A \rightarrow \varepsilon$]

$\rightarrow 0010B$ [Using the production $B \rightarrow 0B$]

$\rightarrow 00101B$ [Using the production $B \rightarrow 1B$]

$\rightarrow 00101$ [Using the production $B \rightarrow \varepsilon$]

Thus the string $00101 \in L(G)$.

In sentential form, derivation starts from the start symbol through a finite application of productions.

A string α derived so far consists of terminals and non-terminals.

$$S \underset{G}{\overset{*}{\Rightarrow}} \alpha \mid \alpha \in (V \cup T)^*$$

- A final string consists of terminals.
- In left sentential form, leftmost symbol is picked up for expansion.
- In right sentential form, rightmost symbol is picked up for expansion.
- A string can be derived in many ways. But we restrict ourselves to :
  1. Leftmost derivation.
  2. Rightmost derivation.

In leftmost derivation the leftmost variable of α (sentential form) is picked for expansion.

In rightmost derivation the rightmost variable of α (sentential form) is picked for expansion.

Scanned by CamScanner

Example 5.2.1 : For the grammar given below

$S \rightarrow A1B$

$A \rightarrow 0A \mid \varepsilon$

$B \rightarrow 0B \mid 1B \mid \varepsilon$

Give leftmost and rightmost derivation of the string 1001.

Solution :

(i) Leftmost derivation of 1001 (Leftmost variable is picked up for expansion.)

$S \rightarrow A1B$	[Derivation starts from the start symbol]
$\rightarrow 1B$	[Using the production $A \rightarrow \varepsilon$]
$\rightarrow 10B$	[Using the production $B \rightarrow 0B$]
$\rightarrow 100B$	[Using the production $B \rightarrow 0B$]
$\rightarrow 1001B$	[Using the production $B \rightarrow 1B$]
$\rightarrow 1001$	[Using the production $B \rightarrow \varepsilon$]

(ii) Rightmost derivation of 1001 (Rightmost variable is picked up for expansion)

$S \rightarrow A1B$	[Derivation starts from the start symbol]
$\rightarrow A10B$	[Using the production $B \rightarrow 0B$]
$\rightarrow A100B$	[Using the production $B \rightarrow 0B$]
$\rightarrow A1001B$	[Using the production $B \rightarrow 1B$]
$\rightarrow A1001$	[Using the production $B \rightarrow \varepsilon$]
$\rightarrow 1001$	[Using the production $A \rightarrow \varepsilon$]

5.2.2.2 Parse Tree

A set of derivations applied to generate a word can be represented using a tree. Such a tree is known as a parse tree. A parse tree representation gives us a better understanding of :

1. Recursion 2. Grouping of symbols

A parse tree is constructed with the following condition :

1. Root of the tree is represented by start symbol.
2. Each interior mode is represented by a variable belonging to V.
3. Each leaf mode is represented by a terminal or ε.

A string generated by a parse tree is seen from left to right.

Example 5.2.2 : For the grammar given below

$S \rightarrow A1B$

$A \rightarrow 0A \mid \varepsilon$

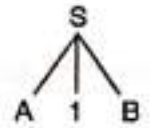
$B \rightarrow 0B \mid 1B \mid \varepsilon$

Give parse tree for leftmost and rightmost derivation of the string 1001.

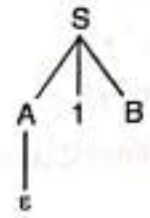
Scanned by CamScanner

Solution :

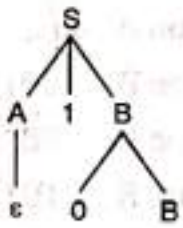
(i) Parse tree for leftmost derivation of 1001.



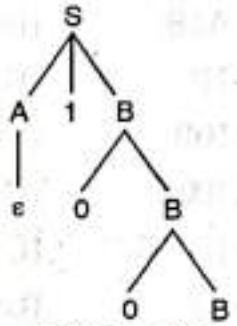
(a) S → A 1 B



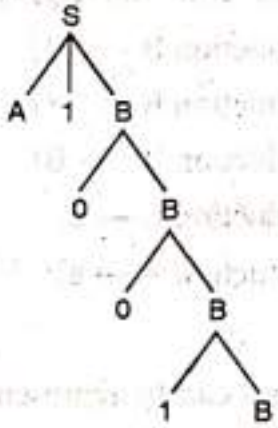
(b) A → ε



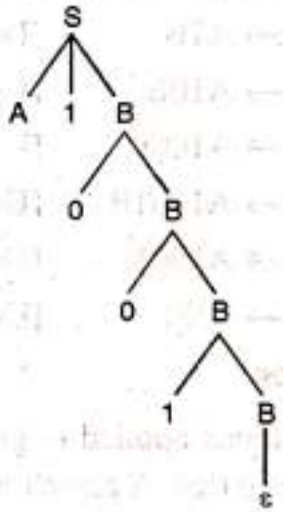
(c) B → 0B



(d) B → 0B



(e) B → 1B

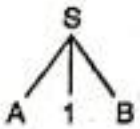


(f) B → ε

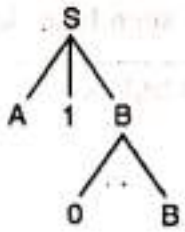
Fig. Ex. 5.2.2

Leftmost derivation of 1001 is shown by series of Figs. Ex. 5.2.2(a) to (f).

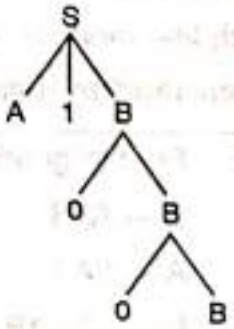
(ii) Parse tree for rightmost derivation of 1001.



(g) S → A 1 B



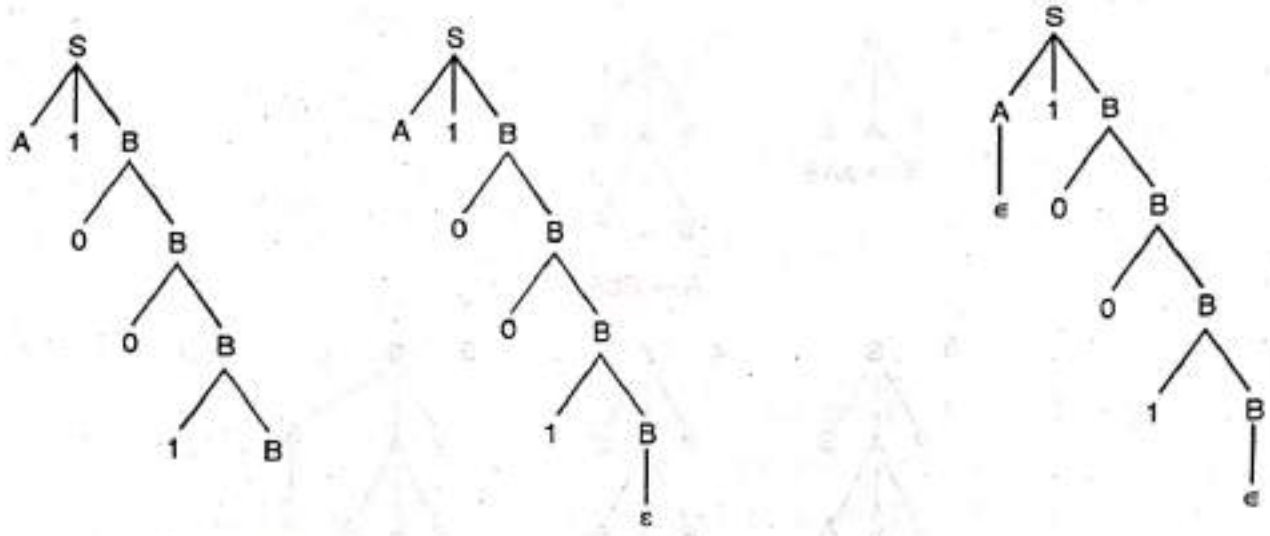
(h) B → 0B



(i) B → 0B

Fig. Ex. 5.2.2

Scanned by CamScanner



(j) $B \rightarrow 1B$ (k) $B \rightarrow \varepsilon$ Fig. (*l*) : $A \rightarrow \varepsilon$

Fig. Ex. 5.2.2

Rightmost derivation of 1001 is shown by a series of Figs. Ex. 5.2.2(g) to (*l*).

It may be noted that the final parse trees in Fig. Ex. 5.2.2(f) and (*l*) are identical.

Example 5.2.3 : For the grammar given below S → 0S1 | 01

Give derivation of 000111.

Solution :

1. Derivation in sentential form

$S \rightarrow 0S1$ [Using production $S \rightarrow 0S1$]

$S \rightarrow 00S11$ [Using production $S \rightarrow 0S1$]

$S \rightarrow 000111$ [Using production $S \rightarrow 01$]

2. Derivation using parse tree.

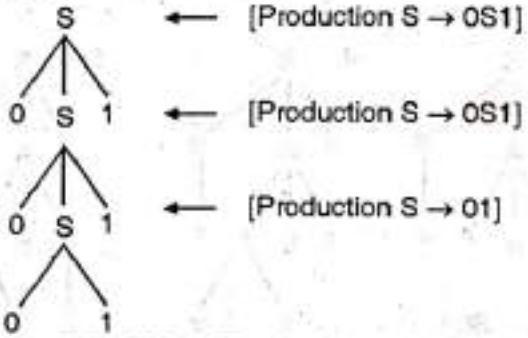


Fig. Ex. 5.2.3 : Derivation of 000111

Example 5.2.4 : Let G = (V, T, P, S) be the CFG having following set of productions. Derive the string "aabbaa" using leftmost derivation and rightmost derivation.
S → aAS | a, A → SbA | SS | ba **MU - May 2009**

Solution :

(i) **Leftmost derivation :**

Leftmost derivation of aabbaa is being shown with the help of the parse tree.

Scanned by CamScanner

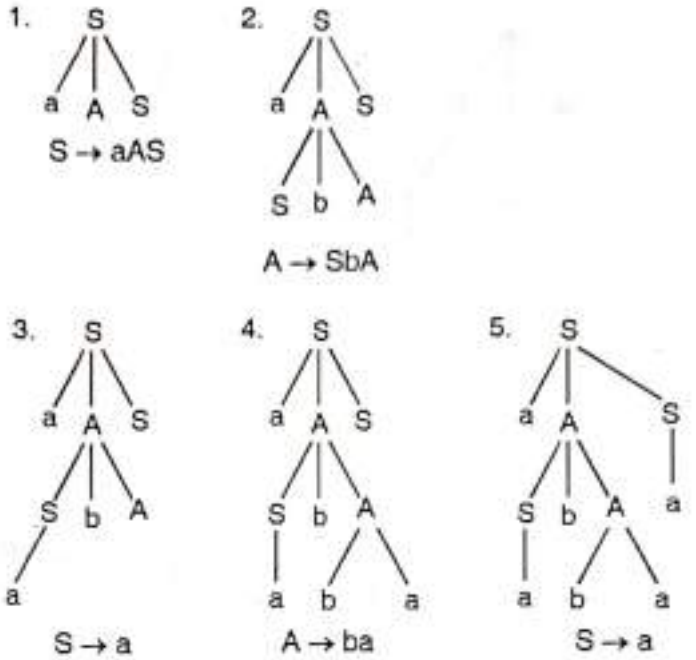


Fig. Ex. 5.2.4(a)

$S \rightarrow aAS \rightarrow aSbAS \rightarrow aabAS \rightarrow aabbas \rightarrow aabbaa$

(ii) Rightmost derivation :

Rightmost derivation of aabbaa is being shown with the help of the parse tree.

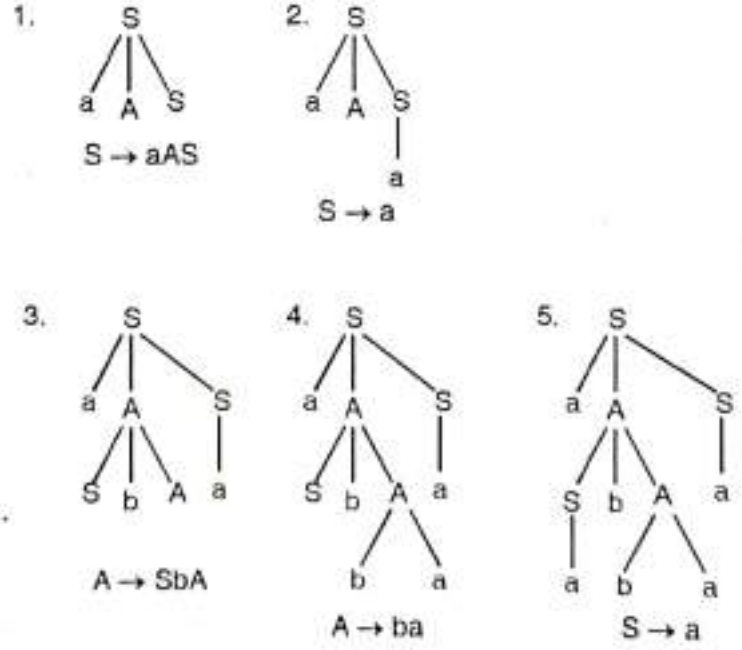


Fig. Ex. 5.2.4(b)

$S \rightarrow aAS \rightarrow aAa \rightarrow aSbAa \rightarrow aSbbaa \rightarrow aabbaa$

Example 5.2.5 : For the grammar given below

$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow (E) \mid a \mid b$

Give the derivation of $(a + b) * a + b$.

Scanned by CamScanner

Solution : For the grammar given in Example 5.2.5,

$$\text{Set of variables } V = \{E, T, F\}$$

$$\text{Set of terminals } \Sigma = \{+, *, (,), a, b\}$$

$$\text{Set of productions } P = \left\{ \begin{array}{l} E \rightarrow E + T \mid T, \\ T \rightarrow T * F \mid F, \\ F \rightarrow (E) \mid a \mid b \end{array} \right\}$$

$$\text{Start symbol} = E$$

(i) Derivation in sentential form

$E \rightarrow E + T$ [Using production $E \rightarrow E + T$]

$\rightarrow T + T$ [Using production $E \rightarrow T$]

$\rightarrow T * F + T$ [Using production $T \rightarrow T * F$]

$\rightarrow F * F + T$ [Using production $T \rightarrow F$]

$\rightarrow (E) * F + T$ [Using production $F \rightarrow (E)$]

$\rightarrow (E + T) * F + T$ [Using production $E \rightarrow E + T$]

$\rightarrow (T + T) * F + T$ [Using production $E \rightarrow T$]

$\rightarrow (F + F) * F + F$ [Using production $T \rightarrow F$]

$\rightarrow (a + b) * a + b$ [Using production $F \rightarrow a \mid b$]

(ii) Derivation using parse tree :

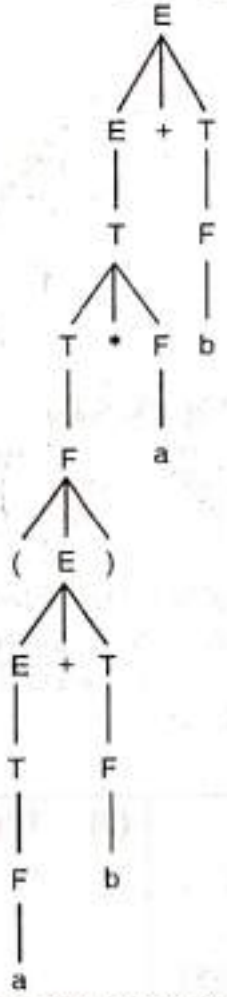


Fig. Ex. 5.2.5 : Parse tree for (a + b) * a + b

Scanned by CamScanner

Example 5.2.6 : Consider the grammar :

$S \rightarrow 0B \mid 1A$

$A \rightarrow 0 \mid 0S \mid 1AA$

$B \rightarrow 1 \mid 1S \mid 0BB$

For the string 00110101 find the following :

(i) Leftmost derivation (ii) Rightmost derivation

(iii) Parse tree.

MU - Dec. 2011

Solution :

(i) Leftmost derivation :

$S \rightarrow 0B$ [Production $S \rightarrow 0B$]

$\rightarrow 00BB$ [Production $B \rightarrow 0BB$]

$\rightarrow 001B$ [Production $B \rightarrow 1$]

$\rightarrow 0011S$ [Production $B \rightarrow 1S$]

$\rightarrow 00110B$ [Production $S \rightarrow 0B$]

$\rightarrow 001101S$ [Production $B \rightarrow 1S$]

$\rightarrow 0011010B$ [Production $S \rightarrow 0B$]

$\rightarrow 00110101$ [Production $B \rightarrow 1$]

(ii) Rightmost derivation :

$S \rightarrow 0B$ [Production $S \rightarrow 0B$]

$\rightarrow 00BB$ [Production $B \rightarrow 0BB$]

$\rightarrow 00B1S$ [Production $B \rightarrow 1S$]

$\rightarrow 00B10B$ [Production $S \rightarrow 0B$]

$\rightarrow 00B101S$ [Production $B \rightarrow 1S$]

$\rightarrow 00B1010B$ [Production $S \rightarrow 0B$]

$\rightarrow 00B10101$ [Production $B \rightarrow 1$]

$\rightarrow 00110101$ [Production $B \rightarrow 1$]

(iii) Parse tree

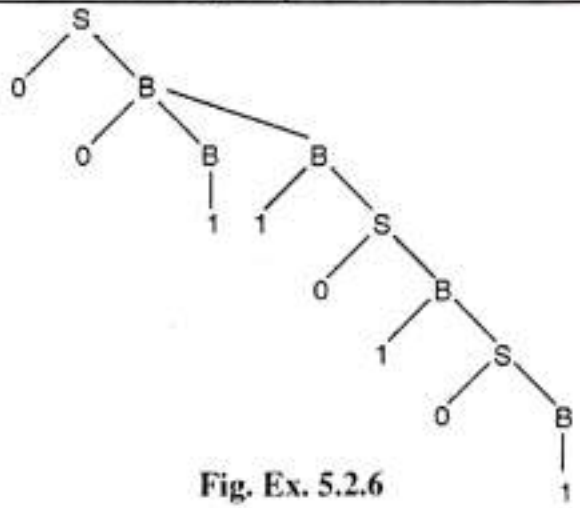


Fig. Ex. 5.2.6

Example 5.2.7 : Consider the following grammar :

$S \rightarrow iCtS \mid iCtSeS \mid a$

$C \rightarrow b$

For the String 'ibtibtaea' find the following :

(i) Leftmost derivation (ii) Rightmost derivation

(iii) Parse Tree (iv) Check if the above grammar is Ambiguous

PU - May 2014

Solution :

(i) Leftmost derivation :

$S \rightarrow iCtS$ [using $S \rightarrow iCtS$]

$\rightarrow ibtS$ [using $C \rightarrow b$]

$\rightarrow ibtiCtSeS$ [using $S \rightarrow iCtSeS$]

(ii) Rightmost derivation :

$S \rightarrow iCtS$ [using $S \rightarrow iCtS$]

$\rightarrow iCtiCtSeS$ [using $S \rightarrow iCtSeS$]

$\rightarrow iCtiCtSea$ [using $S \rightarrow a$]

Scanned by CamScanner

(i) Leftmost derivation :		(ii) Rightmost derivation :	
→ ibtibtSeS	[using C → b]	→ iCteCtaea	[using S → a]
→ ibtibtaeS	[using S → a]	→ iCtebtaea	[using C → b]
→ ibtibtaea	[using S → a]	→ ibtebtaea	[using C → b]

(iii) Parse Tree :

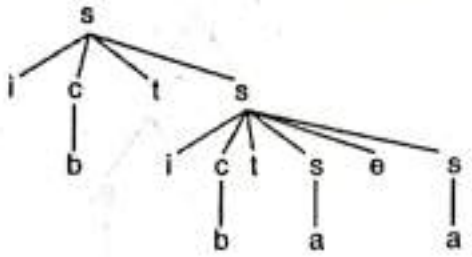


Fig. Ex. 5.2.7

5.2.3 Writing Grammar for a Language

MU - Dec. 2005, Dec. 2008

Productions for an infinite language are written recursively. A production is called recursive if its left side variable occurs on its right side.

For example :

(1) S → aS is recursive

(2) S → b | aA

A→ c | bS is indirectly recursive

as, S → aA → abS [S gives A and A gives S]

A → bS → baA [A gives S and S gives A]

Thus a recursive grammar can be written either using recursive productions or using indirectly recursive productions. Grammar for some basic languages is given below.

(1) Language L = {ε, a, aa, ...}

The productions required to generate the above language are :

S → aS

S → ε

The production, S → ε is taken as ε is a member of L.

The production, S → aS, can generate one or more a's.

Example : Generation of the string aaaa.

Sentential form :

S → aS	[Using production S → aS]
→ aaS	[Using production S → aS]
→ aaaS	[Using production S → aS]
→ aaaaS	[Using production S → aS]
→ aaaa	[Using production S → ε]

Scanned by CamScanner

Parse tree :

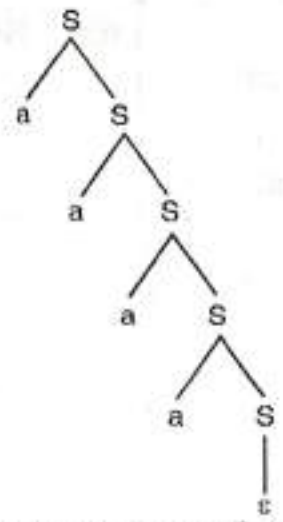


Fig. 5.2.1 : Parse tree for aaaa

(2) Language $L = \{a, aa, aaa, \ldots\}$

The productions required to generate the above language are :

$$S \rightarrow aS$$
$$S \rightarrow a$$

The production $S \rightarrow aS$ can generate one or more a's. The production, $S \rightarrow a$ is for termination of recursion.

Example : Generation of the string aaaa

Sentential form :

$S \rightarrow aS$ [Using production $S \rightarrow aS$]
$\rightarrow aaS$ [Using production $S \rightarrow aS$]
$\rightarrow aaaS$ [Using production $S \rightarrow aS$]
$\rightarrow aaaa$ [Using production $S \rightarrow aS$]

Parse tree :

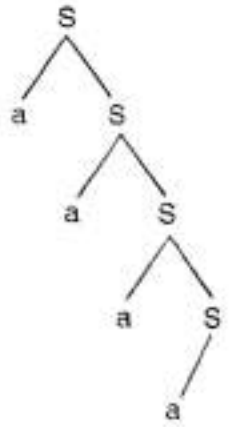


Fig. 5.2.2 : Parse tree for aaaa

(3) Language $L = \{b, ab, aab, aaab, \ldots\}$

The productions required to generate the above languages are :

$$S \rightarrow aS$$
$$S \rightarrow b$$

The production $S \rightarrow aS$ can generate one or more a's. The production, $S \rightarrow b$ is for termination of recursion and thus generating zero or more a's followed by ab.

Example : Generation of the string aaab.

Sentential form :

$S \rightarrow aS$ [Using production $S \rightarrow aS$]
$\rightarrow aaS$ [Using production $S \rightarrow aS$]

Scanned by CamScanner

$\rightarrow$ aaaS [Using production S $\rightarrow$ aS]

$\rightarrow$ aaab [Using production S $\rightarrow$ b]

Parse tree :

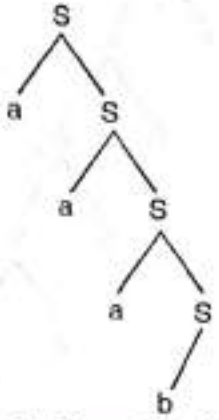


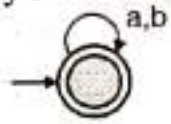
Fig. 5.2.3 : Parse tree for aaab

(4) Language $L = \{w \in \{a, b\}^*\}$

The above language can also be written as

$L = \{\varepsilon, a, b, aa, ab, ba, bb, \ldots\}$

Thus any string that can be derived from the alphabet {a, b} belongs to L. This language can also be generated by :

The DFA M = 

or by the regular expression $(a + b)^*$.

The productions required to generate the above language are :

$S \rightarrow aS$

$S \rightarrow bS$

$S \rightarrow \varepsilon$

The two productions S $\rightarrow$ aS, S $\rightarrow$ bS can generate any number of a's or b's with the next character as either a or b. The production S $\rightarrow \varepsilon$ is for termination of recursion. Since ε is also a member of L, S $\rightarrow \varepsilon$ is taken for termination of recursion.

Example : Generation of the string abba

Sentential form :

S $\rightarrow$ aS [Using production S $\rightarrow$ aS]

$\rightarrow$ abS [Using production S $\rightarrow$ bS]

$\rightarrow$ abbS [Using production S $\rightarrow$ bS]

$\rightarrow$ abbaS [Using production S $\rightarrow$ aS]

$\rightarrow$ abba [Using production S $\rightarrow \varepsilon$]

Scanned by CamScanner

Parse tree :

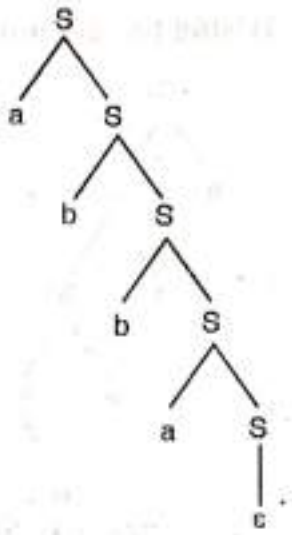


Fig. 5.2.4 : Parse tree for abba

(5) Language $L = \{\varepsilon, ab, aabb, \ldots, a^n b^n\}$

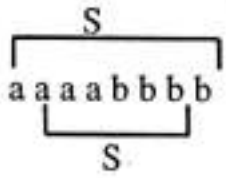
The above language can also be described as a language of words in which every word consists of any number of a's followed by equal number of b's.

The productions required to generate the above language are :

$S \rightarrow aSb$

$S \rightarrow \varepsilon$

The recursion involved in the above language is explained below.



Both the strings a^4b^4 and a^3b^3 are generated by S. a^4b^4 can be written as aa^3b^3b and thus we have the recursive relation $S \rightarrow aSb$.

The production, $S \rightarrow aSb$ can generate any number of a's followed by equal number of b's. The production $S \rightarrow \varepsilon$ is for termination of recursion. Since ε is a member of the given language, $S \rightarrow \varepsilon$ is taken for termination of recursion.

Example : Generation of the string aaabbb.

Sentential form :

$S \rightarrow aSb$ [Using production $S \rightarrow aSb$]

$\rightarrow aaSbb$ [Using production $S \rightarrow aSb$]

$\rightarrow aaaSbbb$ [Using production $S \rightarrow aSb$]

$\rightarrow aaabbb$ [Using production $S \rightarrow \varepsilon$]

Scanned by CamScanner

Parse tree :

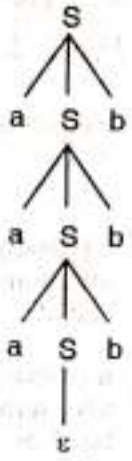


Fig. 5.2.5 : Parse tree for aaabbb or a^3b^3

(6) Language $L = \{ab, aabb, \ldots, a^nb^n\}$

The productions required to generate the above language are :

$S \rightarrow aSb$

$S \rightarrow ab$

The production, $S \rightarrow aSb$ can generate a string of the form a^nb^n. The production $S \rightarrow ab$ is for termination of recursion.

Example : Generation of the string aaabbb

Sentential form :

$S \rightarrow aSb$		[Using production $S \rightarrow aSb$]
$\rightarrow aaSbb$		[Using production $S \rightarrow aSb$]
$\rightarrow aaabbb$		[Using production $S \rightarrow ab$]

Parse tree :

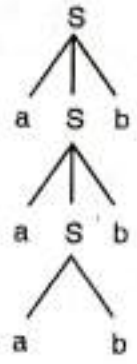


Fig. 5.2.6 : Parse tree for aaabbb or a^3b^3

(7) Language $L = \{w \in (a, b)^* \mid w \text{ is a palindrome of odd length}\}$

- String 'a' is palindrome of odd length.
- String 'b' is a palindrome of odd length

Scanned by CamScanner

- String abababa is palindrome. The palindrome nature of abababa can be understood as given shown in Fig. 5.2.7.

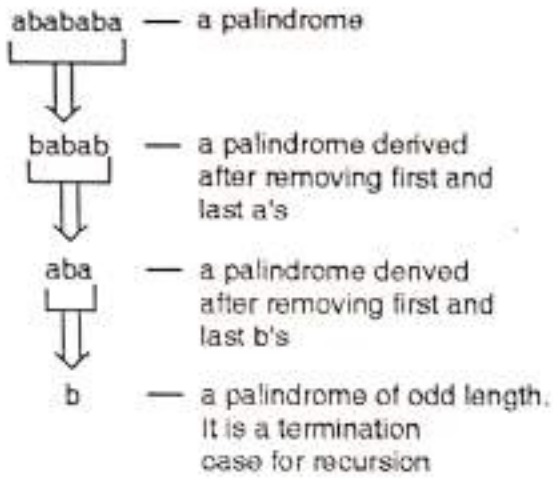


Fig. 5.2.7

Thus a palindrome can be generated recursively using the two productions :

$S \rightarrow aSa$

$S \rightarrow bSb$

Since, we are considering odd length palindromes, S should terminate in either 'a' or 'b' and the productions are :

$S \rightarrow a, S \rightarrow b$

Thus the productions required to generate the above language are :

$S \rightarrow aSa$

$S \rightarrow bSb$

$S \rightarrow a$

$S \rightarrow b$

Or, in short, it can be written as

$S \rightarrow aSa \mid bSb \mid a \mid b$

Example : Generation of the string abababa.

Sentential form :

$S \rightarrow aSa$ [Using production $S \rightarrow aSa$]

$\rightarrow abSba$ [Using production $S \rightarrow bSb$]

$\rightarrow abaSaba$ [Using production $S \rightarrow aSa$]

$\rightarrow abababa$ [Using production $S \rightarrow b$]

Scanned by CamScanner

Parse tree :

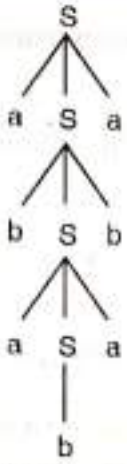


Fig. 5.2.8 : Parse tree for abababa

(8) Language $L = \{w \in \{a, b\}^* \mid w \text{ is an even length palindrome with } |w| > 0\}$

The productions required to generate the above language are :

$S \rightarrow aSa$

$S \rightarrow bSb$

$S \rightarrow bb$

$S \rightarrow aa$

The productions $S \rightarrow aa$, $S \rightarrow bb$ are for termination of recursion. It may be noted that smallest strings generated by the above grammar are aa and bb. The two productions $S \rightarrow aSa$ and $S \rightarrow bSb$ can generate a palindrome of arbitrary length.

Example : Generation of the string ababbaba

Sentential form :

$S \rightarrow aSa$		[Using production $S \rightarrow aSa$]
$\rightarrow abSba$		[Using production $S \rightarrow bSb$]
$\rightarrow abaSaba$		[Using production $S \rightarrow aSa$]
$\rightarrow ababbaba$		[Using production $S \rightarrow bb$]

Parse tree or derivation tree :

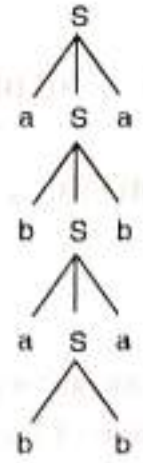


Fig. 5.2.9 : Derivation tree for ababbaba

Scanned by CamScanner

(9) Language $L = \{w \in (a, b)^* \mid w \text{ is a palindrome of either even length or odd length with } |w| > 0\}$

The productions required to generate the above language are :

$S \rightarrow aSa \mid bSb \mid aa \mid bb \mid a \mid b$

- $S \rightarrow aa$ or $S \rightarrow bb$ are for termination of recursion for even length palindromes.
- $S \rightarrow a$ and $S \rightarrow b$ are for termination of recursion for odd length palindromes.
- $S \rightarrow aSa$ or $S \rightarrow bSb$ can generate a palindrome of an arbitrary length.

Example 5.2.7 : Let G be the grammar. Find the leftmost derivation, rightmost derivation and parse tree for the string 00110101.

G : $S \rightarrow 0B \mid 1A$
$A \rightarrow 0 \mid 0S \mid 1AA$
$B \rightarrow 1 \mid 1S \mid 0BB$

MU - Dec. 2010

Solution :

Parse tree for the string 00110101

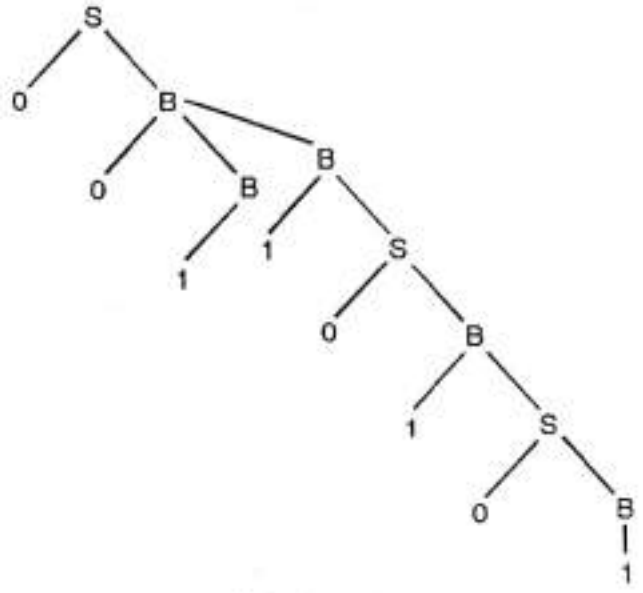


Fig. Ex. 5.2.7

Leftmost derivation :

$S \rightarrow 0B \rightarrow 00BB \rightarrow 001B \rightarrow 0011S \rightarrow 00110B \rightarrow 001101S \rightarrow 0011010B \rightarrow 00110101$

Rightmost derivation :

$S \rightarrow 0B \rightarrow 00BB \rightarrow 00B1S \rightarrow 00B10B \rightarrow 00B101S \rightarrow 00B1010B \rightarrow 00B10101 \rightarrow 00110101$

5.2.3.1 Union Rule for Grammar

If a language L_1 is generated by a grammar with start symbol S_1 and L_2 is generated by a grammar with start symbol S_2 then the union of the languages $L_1 \cup L_2$ can be generated with start symbol S, where

Scanned by CamScanner

$$S \rightarrow S_1 \mid S_2$$

Example : Let the languages L_1 and L_2 are given as below :

$$L_1 = \{a^n \mid n > 0\}$$

$$L_2 = \{b^n \mid n > 0\}$$

Productions for L_1 are :

$$S_1 \rightarrow aS \mid a$$

Productions for L_2 are :

$$S_2 \rightarrow bS \mid b$$

Then the productions for $L = L_1 \cup L_2$ can be written as :

$$S \rightarrow S_1 \mid S_2$$

$$S_1 \rightarrow aS \mid a$$

$$S_2 \rightarrow bS \mid b$$

5.2.3.2 Concatenation Rule for Grammar

If a language L_1 is generated by a grammar with start symbol S_1 and L_2 is generated by a grammar with start symbol S_2 then the concatenation (product) of the languages $L_1 \cdot L_2$ can be generated with start symbol S, where

$$S \rightarrow S_1 S_2$$

Example : Let the languages L_1 and L_2 are given as below :

$$L_1 = \{a^n \mid n > 0\}$$

$$L_2 = \{b^n \mid n > 0\}$$

Productions for L_1 are :

$$S_1 = aS \mid a$$

Productions for L_2 are :

$$S_2 \rightarrow bS \mid b$$

Then the productions for $L = L_1 \cdot L_2$ can be written as :

$$S \rightarrow S_1 S_2$$

$$S_1 \rightarrow aS \mid a$$

$$S_2 \rightarrow bS \mid b$$

Example 5.2.8 : Give a context free grammar for the following language.

$$0(0 + 1)^*01(0 + 1)^*1$$

Solution :

The $(0 + 1)^*$ can be generated by the following productions.

$$S_1 \rightarrow 0\,S_1 \mid 1\,S_1 \mid \varepsilon$$

Scanned by CamScanner

Using the product rule, we can write a set of productions for the language

$0(0 + 1)^*01(0 + 1)^*1$

$$P = \{ \; S \rightarrow 0S_1 01 S_1 1$$
$$S_1 \rightarrow 0S_1 | 1S_1 | \varepsilon$$
$$\}$$

Where,

Set of variables V = $\{S, S_1\}$

Set of terminals T = $\{0, 1\}$

Start symbol = S

Example 5.2.9 : Construct the context free grammar corresponding to the regular expression.

$R = (0 + 1)1^*(1 + (01)^*)$

Solution :

Grammar for (0 + 1) is given by

$S_1 \rightarrow 0 | 1$ [union rule]

Grammar for 1^* is given by

$S_2 \rightarrow 1S_2 | \varepsilon$

Grammar for $(1 + (01)^*)$ is given by

$S_3 \rightarrow 1 | S_4$ where S_4 stands for $(01)^*$

$S_4 \rightarrow 01S_4 | \varepsilon$

Using the concatenation rule, we can write a set of productions for

$R = (0 + 1)1^*(1 + (01)^*)$

$$P = \{ \quad S \rightarrow S_1 S_2 S_3$$
$$S_1 \rightarrow 0 | 1$$
$$S_2 \rightarrow 1S_2 | \varepsilon$$
$$S_3 \rightarrow 1 | S_4$$
$$S_4 \rightarrow 01S_4 | \varepsilon$$
$$\}$$

where,

Set of variables V = $\{S_1, S_2, S_3, S_4\}$

Set of terminals T = $\{0, 1\}$

Start symbol = S

Scanned by CamScanner

Example 5.2.10 : Give the CFG for $L = \{a^i b^j , i \le j \le 2i, i \ge 1\}$

Solution :

A string of L has to meet the following requirements

- A finite number of a's followed by a finite number of b's.
- Number of b's should be at least as many as number of a's.
- Number of b's should not be more than twice the number of a's.

The set of productions for the above language are given by :

$P = \{$

$S \rightarrow aSb$ [Number of b's = Number of a's]

$S \rightarrow aSbb$ [Number of b's = 2 × Number of a's]

$S \rightarrow ab \mid abb$ [Termination cases]

$\}$

where, Set of variables $V = \{S\}$

Set of terminals $T = \{a, b\}$

Start symbol = S.

Example 5.2.11 : Give CFG for

$L = \{a^i b^j c^q ; i + j = q; i, j \ge 1\}$

Solution :

A string of the form $a^i b^j c^q$ with $i + j = q$ can be written as

$a^i b^j c^{i+j}$

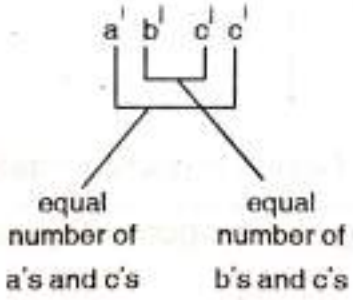


Fig. Ex. 5.2.11

A string of the form $a^i c^j$ can be generated with the help of the following productions :

$S \rightarrow aSc$

$S \rightarrow X$

Recursion is terminated by the production $S \rightarrow X$, where X gives a string of the form $b^j c^j$.

$X \rightarrow bXc$

$X \rightarrow bc$

Scanned by CamScanner

Thus, the set of productions for the above language are given by :

$$P = \{ \quad S \rightarrow aSc \mid X$$
$$X \rightarrow bXc \mid bc$$
$$\}$$

where, Set of variables V = {S, X}
Set of terminals T = {a, b}
Start symbol = S.

Working of the above grammar can be understood with the help of the following example.

Example : Generation of the string aabbbccccc

Parse tree :

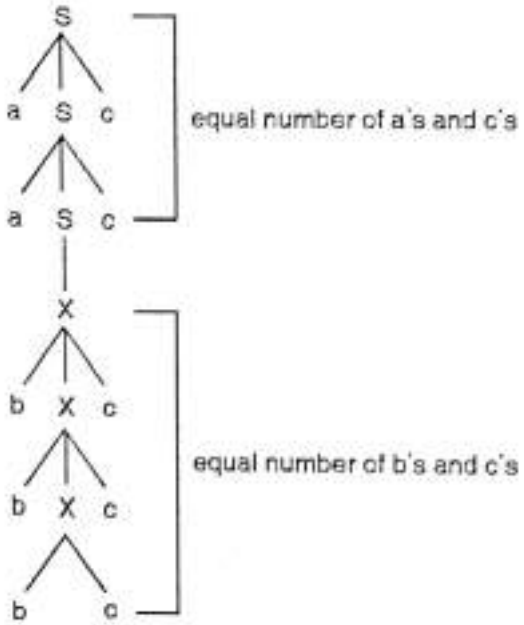


Fig. Ex. 5.2.11(a) : Parse tree for aabbbccccc

Example 5.2.12 : Find context free grammars generating each of these languages

(1) $L_1 = \{a^i b^j c^k \mid i = j + k\}$

(2) $L_2 = \{a^i b^j c^k \mid j = i + k\}$

(3) $L_3 = \{a^i b^j c^k \mid i = j \text{ or } j = k\}$

Solution :

(1) $L_1 = \{a^i b^j c^k \mid i = j + k\}$

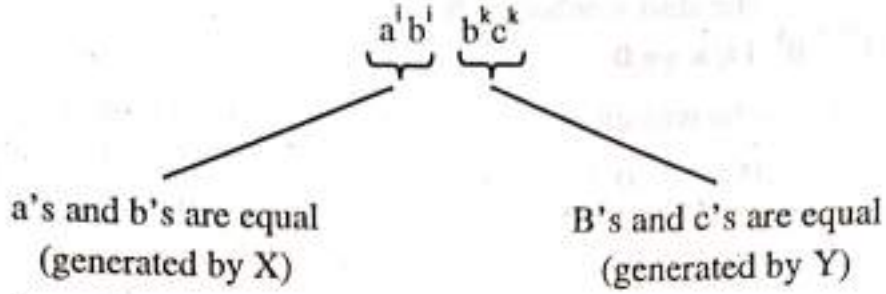
Outer a's should be matched with c's then the remaining a's should be matched with b's.

$$S \rightarrow aSc \mid aXc$$
$$X \rightarrow aXb \mid ab$$

(2) $L_2 = \{a^i b^j c^k \mid j = i + k\}$

Scanned by CamScanner

The string $a^i b^j c^k$ can be written as



$S \rightarrow XY$

$X \rightarrow aXb \mid ab$

$Y \rightarrow bYc \mid bc$

(3) $L_3 = \{a^i b^j c^k \mid i = j \text{ or } j = k\}$

$S \rightarrow AB$	[i = j]
$S \rightarrow CD$	[j = k]
$A \rightarrow aAb \mid ab$	[equal number of a's and b's
$B \rightarrow CB \mid c$	followed by some c's]
$C \rightarrow aC \mid a$	[Some a's followed by equal
$D \rightarrow b\,DC \mid bc$	number of b's and c's]

Example 5.2.13 : Give the context free grammar for the following languages.

(a) $(011 + 1)^* (01)^*$

(b) $0^i i^{i+k} 0^k$ where i, k >= 0

Solution :

(a) CFG for $(011 + 1)^* (01)^*$:

Let us assume that $(011 + 1)^*$ is generated by the variable X.

$\therefore X \rightarrow 011X \mid 1X \mid \varepsilon$

Let us assume that $(01)^*$ is generated by the variable Y.

$\therefore \quad Y \rightarrow 01Y \mid \varepsilon$

Using the concatenation rule, the set of productions for $(011 + 1)^* (01)^*$ can be written as :

$P = \{ \; S \rightarrow XY$

$X \rightarrow 011X \mid 1X \mid \varepsilon$

$Y \rightarrow 01Y \mid \varepsilon$

$\}$

where,

The set of variable V = {S, X, Y}

Scanned by CamScanner

The set of terminals T = {0, 1}

The start symbol = S

(b) CFG for $0^i 1^{i+k} 0^k$ | i, k >= 0

The language L can be written as :

$$L = \{\underbrace{0^i 1^i}_{X} \; \underbrace{1^k 0^k}_{Y}\}$$

Let us assume that 0^i1^i is generated by the variable X.

$\therefore X = 0X1 | \varepsilon$

Let us assume that 1^k0^k is generated by the variable Y.

$\therefore Y = 1Y1|\varepsilon$

The set of productions P is given by :

P = { $S \rightarrow XY$

$X \rightarrow 0X1|\varepsilon$

$Y \rightarrow 1Y0|\varepsilon$

}

Where,

The set of variable V = {S, X, Y}

The set of terminals T = {0, 1}

The start symbol = S

Example 5.2.14 : Show that the languages,

(1) $L_1 = \{a^i b^i c^j \mid i,j >= 1\}$ and

(2) $L_2 = \{a^i b^j c^j \mid i,j >= 1\}$ are context free languages.

Solution : These languages can be shown to CFL by writing the required productions in each case.

Productions for $L_1 = \{a^i b^i c^j \mid i, j >= 1\}$:

$S \rightarrow XY$

$X \rightarrow aXb|ab$ [X generates $a^i b^i$ and

$Y \rightarrow cY|c$ Y generates c^j]

Productions for $L_2 = \{a^i b^j c^j \mid i, j >= 1\}$:

$S \rightarrow XY$ [X generates a^i and

$X \rightarrow aX|a$ Y generates $b^j c^j$]

$Y \rightarrow bYc|bc$

Scanned by CamScanner

Example 5.2.15 : Give CFG for $L = \{0^i 1^j 0^k \mid j > i + k\}$

Solution :

A string of the form $0^i 1^j 0^k$ with $j > i + k$ can be written as

$$0^i 1^{i+k+p} 0^k \text{ with } P > 0$$

$$= \underbrace{0^i 1^i}_{X} \underbrace{1^p}_{Y} \underbrace{1^k 0^k}_{Z}$$

A string $0^i 1^i 1^p 1^k 0^k$ can be represented as concatenation of three strings.

1. $0^i 1^i$ generated by X.
2. 1^p with $P > 0$, generated by Y
3. $1^k 0^k$ generated by Z.

Thus, the set of productions for the above language are given by :

$$P = \{ S \to XYZ$$
$$X \to 0X1 \mid \varepsilon$$
$$Y \to 1Y \mid 1$$
$$Z \to 1Z0 \mid \varepsilon$$
$$\}$$

where, Set of variables V = {X, Y, Z}
Set of terminals T = {0, 1}
Start symbol = S

Example 5.2.16 : Give CFG for matching parenthesis.

Solution :

- A production of the form

  $S \to (S)$

  can generate nested parenthesis.

- A production of the form

  $S \to SS$

  will allow parenthesis to grow sideways.

Nesting : (((0)))

Side ways : (()) (())

S S

Thus, the set of productions for the above language are given by :

$$P = \{S \to (S) \mid SS \mid \varepsilon\}$$

Where, Set of variables V = {S}
Set of terminals T = {(,)}
Start symbol = S

Scanned by CamScanner

Example 5.2.17 : Give CFG for all strings with at least two 0's, $\Sigma = \{0, 1\}$ **MU - Dec. 2006**

Solution :

The regular expression for the above language is given by :

$$RE = 1^*01^*0(0 + 1)^*$$

Let us assume that 1^* is generated by the variable X.

$$\therefore X \rightarrow 1X \mid \varepsilon$$

Let us assume that $(0 + 1)^*$ is generated by the variable Y.

$$\therefore Y \rightarrow 0Y \mid 1Y \mid \varepsilon$$

Using the concatenation rule, the set of productions for $1^*01^*0(0 + 1)^*$ can be written as :

$$P = \{ \quad S \rightarrow X0X0Y$$
$$X \rightarrow 1X \mid \varepsilon$$
$$Y \rightarrow 0Y \mid 1Y \mid \varepsilon$$
$$\}$$

Where, Set of variables V = {S, X, Y}
Set of terminals T = {0, 1}
Start symbol = S.

Example 5.2.18 : Give CFG for set of odd length strings in $\{0, 1\}^*$ with middle symbol '1'.

Solution :

A string generated by the above language could be shown as given below :

$$\underbrace{\text{String of length n from } \{0, 1\}^*}\ 1\ \underbrace{\text{String of length n from } \{0, 1\}^*}$$

The hidden recursion can be understood from the fact that deletion of first and last character will not change the basic nature of the string.

i.e. If S is a string of the given form then S are :

0S0, 0S1, 1S0, 1S1.

Thus, the set of productions for the above language is given by :

$$P = \{$$
$$S \rightarrow 0S0 \mid 0S1 \mid 1S0 \mid 1S1 \mid 1$$
$$\}$$

The production $S \rightarrow 1$, terminates the recursion with 1 as a middle character.

where, Set of variables V = {S}
Set of terminals T = {0, 1}
Start symbol = S.

Example 5.2.19 : Give CFG for set of even length strings in {a, b, c, d}* with two middle symbol equal.

Scanned by CamScanner

Solution :

A string generated by the above language could be shown as given below :

$$\underbrace{\text{String of length n from } \{a, b, c, d\}^*}\ \begin{bmatrix} aa \\ bb \\ cc \\ dd \end{bmatrix}\ \underbrace{\text{String of length n from } \{a, b, c, d\}^*}$$

The hidden recursion can be understood from the fact that deletion of first and last character will not change the basic nature of the string.

i.e., if S is a string of the given form then S are :

aSa, aSb, aSc, aSd, bSa, bSb, bSc, bSd, cSa, cSb, cSc, cSd, dSa, dSb, dSc, dSd.

Thus, the set of productions for the above language is given by :

$$\begin{aligned} P = \{ S \rightarrow\ & aSa \mid aSb \mid aSc \mid aSd \mid \\ & bSa \mid bSb \mid bSc \mid bSd \mid \\ & cSa \mid cSb \mid cSc \mid cSd \mid \\ & dSa \mid dSb \mid dSc \mid dSd \mid \\ & aa \mid bb \mid cc \mid dd \\ \} & \end{aligned}$$

Where, Set of variables V = {S}

Set of terminals T = {a, b, c, d}

Start symbol = S

Example 5.2.20 : Give CFG for $L = \{a^m b^n c^p d^q \mid m + n = p + q\}$

Solution :

There could be three possibilities on number of a's and number of d's.

Case I : $m > q$, $a^m b^n c^p d^q$ can be written as

$$a^q \underbrace{a^{m-q} b^n c^p}_{X} d^q$$

Where, $a^{m-q} b^n c^p$ is generated by the variable X (say).

The string X is of the form $a^i b^j c^{i+j} = a^i \underbrace{b^j c^j}_{Z} c^i$

Productions for X can be written as given below :

$$X \rightarrow aXc \mid Z$$

$$Z \rightarrow bZc \mid \varepsilon$$

Case II : $m < q$, $a^m b^n c^p d^q$ can be written as

$$a^m \underbrace{b^n c^p d^{q-m}}_{Y} d^m$$

Where, $b^n c^p d^{q-m}$ is generated by the variable Y (say).

Scanned by CamScanner

The string Y is of the form $b^{i+j}c^i d^j = b^j b^i c^i d^j$

Z

Productions for Y can be written as given below :

$$Y \rightarrow bYd \mid Z$$

$$Z \rightarrow bZc \mid \varepsilon$$

Case III : $m = q$, $a^m b^n c^p d^q$ can be written as

$a^m \; b^n \; c^n \; d^m$

Z

Z(say)

Where, $b^n c^n$ is generated by Z (say)

$$Z = bZc \mid \varepsilon$$

Thus the set of productions for the above language are given by :

$$P = \{ \quad S \rightarrow aSd \mid X \mid Y$$
$$X \rightarrow aXc \mid Z$$
$$Y \rightarrow bYd \mid Z$$
$$Z \rightarrow bZc \mid \varepsilon$$
$$\}$$

Where, Set of variables V = {S, X, Y, Z}
Set of terminals T = {a, b, c, d}
Start symbol = S.

Example 5.2.21 : Give CFG for $L = \{a^n b^m c^n \mid m, n >= 1\}$

Solution : The production $S \rightarrow aSc$ can generate equal number of a's and b's.

Thus the set of productions for the given language is :

$$P = \{ \quad S \rightarrow aSc \mid X$$
$$X \rightarrow bX \mid b$$
$$\}$$

The production $S \rightarrow X$, terminates the recursion. The variable X can generate one or more b's.

where, Set of variables V = {S, X}
Set of terminals T = {a, b}
Start symbol = S.

Example 5.2.22 : Give CFG for L = {x | x contains equal number of a's and b's}.

Solution :

Let us assume that there are three variables S, A and B.

where,

S generates a string with equal number of a's and b's.

A generates a string in which number of a's = 1 + number of b's.

Scanned by CamScanner

B generates a string in which number of b's = 1 + number of a's.

We can write the set of production using indirect recursion by relating the three variables S, A and B.

- The relation between S and B is given by $S \rightarrow aB$,

  B represents a string in which number of b's is one more than a's. If we pretend an 'a' to 'B', both a's and b's will become equal.
- Similarly, the relation between S and A is given by

  $S \rightarrow bA$
- A is related to S by

  $A \rightarrow aS$, removing an 'a' from a string represented by A will render both a's and b's equal.
- A is related to A by

  $A \rightarrow bAA$, one b and two A's on the right hand side will mean number of a's one more than number of b's.
- Similarly,

  $B \rightarrow bS$

  and $B \rightarrow aBB$

Thus the set of productions for the above language are given by :

$$P = \{ \quad S \rightarrow aB \mid bA$$
$$A \rightarrow aS \mid bAA \mid a$$
$$B \rightarrow bS \mid aBB \mid b$$
$$\}$$

where, Set of variables V = {S, A, B}

Set of terminals T = {a, b}

Start symbol = S.

The behaviour of productions can be understood with the help of the following example.

Example : Generation of the string aababb

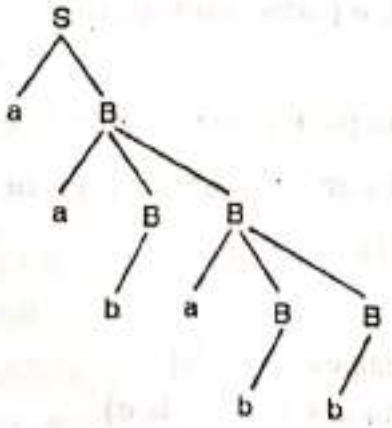


Fig. Ex. 5.2.22 : Parse tree for aababb

Scanned by CamScanner

Example 5.2.23 : Give CFG for strings in ab*

Solution : Let us say that b* is generated by variable B.

$$\therefore B \rightarrow bB \mid \varepsilon$$

The set of productions for the above language can be written using the rule of concatenations.

$$P = \{ \quad S \rightarrow aB$$
$$B \rightarrow bB \mid \varepsilon$$
$$\}$$

Where, Set of variables V = {S, B}
Set of terminals T = {a, b}
Start symbol = S.

Example 5.2.24 : Give CFG for strings in a*b*.

Solution :

Let us assume that a* and b* are generated by variable A and B respectively.

$$A \rightarrow aA \mid \varepsilon$$
$$B \rightarrow bB \mid \varepsilon$$

The set of productions for the above language can be written using the rule of concatenation.

$$P = \{ \quad S \rightarrow AB$$
$$A \rightarrow aA \mid \varepsilon$$
$$B \rightarrow bB \mid \varepsilon$$
$$\}$$

Where, Set of variables V = {S, A, B}
Set of terminals T = {a, b}
Start symbol = S.

Example 5.2.25 : Give CFG for $L = \{ wcw^T \mid w \in \{a, b\}^* \}$

Solution :

L is a language of odd length palindromes with middle character as c.

The set of productions for the above language is given by :

$$S \rightarrow aSa \mid bSb \mid c$$

Where,

Set of variables V = {S}
Set of terminals T = {a, b, c}
Start symbol = S.

Scanned by CamScanner

Example 5.2.26 : Give CFG for $L = \{a^n b^m \mid n \neq m\}$.

Solution : A string belonging to L can have either of the two forms given below :

1. $a^i a^j b^i$ where $j > 0$

or 2. $a^i b^j b^i$ where $j > 0$

Let us assume that a^j is generation by A and b^j is generated by B.

The set of productions for the above language is given by :

$$P = \{ \quad S \rightarrow aSb \mid A \mid B$$
$$A \rightarrow aA \mid a$$
$$B \rightarrow bB \mid b$$
$$\}$$

Where, Set of variables $V = \{S, A, B\}$

Set of terminals $T = \{a, b\}$

Start symbol = S.

Example 5.2.27 : Give CFG for the language $L = \{a^n b^m c^k \mid n = m \text{ or } m \leq k\}$

Solution : The language L is union of two languages L_1 and L_2.

where, $L_1 = \{a^n b^m c^k \mid n = m\} = \{a^n b^n c^k\}$

$L_2 = \{a^n b^m c^k \mid m \leq k\} = \{a^n b^m c^{m+p} \mid p \geq 0\}$

$= \{a^n b^m c^m c^p \mid p \geq 0\}$

The productions for languages $L_1 = \{a^n b^n c^k\}$ are given as :

$S_1 =$ AB, where A generates $a^n b^n$ and B generates c^k.

$A = aAb \mid \varepsilon$

$B = cB \mid \varepsilon$

The productions for language $\{L_2 = a^n b^m c^m c^p \mid p \geq 0\}$ are given as :

$S_2 =$ XYB, where X generates a^n and Y generates $b^m c^m$.

$X \rightarrow aX \mid \varepsilon$

$Y \rightarrow bYc \mid \varepsilon$

Now, we can write productions for the given language L using the rule of union.

$L = L_1 \cup L_2$

$P = \{$

$S \rightarrow S_1 \mid S_2$

$S_1 \rightarrow AB$

$A \rightarrow aAb \mid \varepsilon$

$B \rightarrow cB \mid \varepsilon$

$S_2 \rightarrow XYB$

Scanned by CamScanner

$X \rightarrow aX \mid \varepsilon$

$Y \rightarrow bYc \mid \varepsilon$

}

where, The set of variables V = {S, S_1, S_2, A, B, X, Y}

The set of terminals T = {a, b}

The start symbol = S.

Example 5.2.28 : Give CFG for the language $L = \{0^m 1^n 0^{m+n} \mid m, n >= 0\}$

Solution : The language L can be written as :

$$L = \{0^m 1^n 0^{m+n}\} = \{ \underbrace{0^m}_{X} \underbrace{1^n}_{Y} \underbrace{0^n 0^m}_{X} \}$$

Let us assume that 0^m is generated by a variable X and $1^n 0^n$ is generated by a variable Y.

The set of productions for the given language is given by :

P = { $S \rightarrow XYX$

$X \rightarrow 0X \mid \varepsilon$

$Y \rightarrow 1Y0 \mid \varepsilon$

}

where, The set of variables V = {S, X, Y}

The set of terminals T = {0, 1}

The start symbol = S.

Example 5.2.29 : Give CFG for the language

$L = \{0^a 1^b 2^c \mid a - c = b\}$

Solution :

The string $0^a 1^b 2^c$ with a – c = b

Can be written as $0^{c+b} 1^b 2^c$

$= 0^c \; 0^b \; 1^b \; 2^c$

The set of productions for the given language is :

P = { $S \rightarrow 0S2 \mid X$ [X generates a string of the form $0^b 1^b$]

$X \rightarrow 0X1 \mid \varepsilon$

}

Where, Set of variables V = {S, X}

Set of terminals T = {0, 1}

Start symbol = S.

Scanned by CamScanner

Example 5.2.30 : Given the context free grammar for the following languages :

(a) $L = \{a^n b^{2n} \mid n > 1\}$ (b) $L = \{a^m b^n \mid n > m\}$

Solution :

(a) $L = \{a^n b^{2n} \mid n > 1\}$

$S \rightarrow aSbb \mid aabbbb$

(b) $L = \{a^m b^n \mid n > m\}$

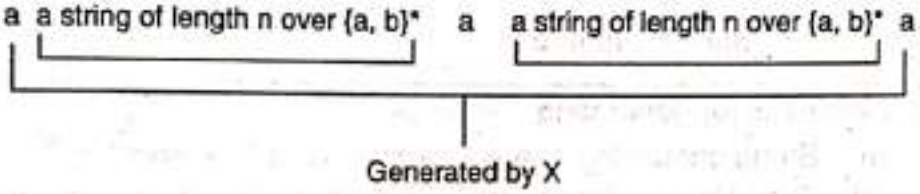
$S \rightarrow aSb \mid B$

$B \rightarrow bB \mid b$

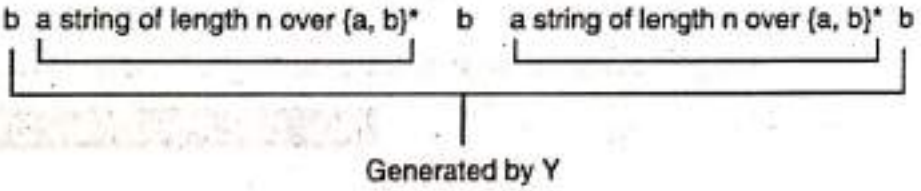
Example 5.2.31 : Give CFG for the following language : The set of odd length strings in $\{a, b\}^*$ whose first, middle and last symbols are all same.

Solution : A string belonging to the language will have any of the two forms, given below.

Form 1 :



Form 2 :



The set of productions for the given language is :

$P = \{ \quad S \rightarrow aXa \mid bYb$

$X \rightarrow aXa \mid aXb \mid bXa \mid bXb \mid a$

$Y \rightarrow aYa \mid aYb \mid bYa \mid bYb \mid b$

$\}$

Where, Set of variables $V = \{S, X, Y\}$

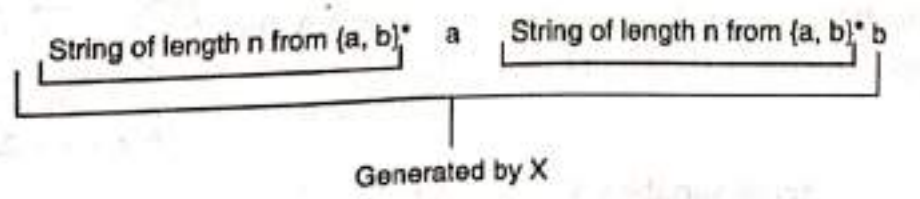
Set of terminals $T = \{a, b\}$

Start symbol $= S$.

Example 5.2.32 : Write a CFG for the following languages

$L = \{ua\omega b \mid u, \omega \text{ are in } \{a, b\}^* \text{ and } |u| = |\omega|\}$

Solution : A string generated by the above language could be shown as given below :



Scanned by CamScanner

The hidden recursion in X can be understood from the fact that deletion of first and last character will not change the basic nature of X.

∴ X is a string of the given form then X are :

$$X \rightarrow aXa \mid aXb \mid bXa \mid bXb$$

Thus, the set of productions for the above language is given by,

$$P = \{ \quad S \rightarrow Xb$$
$$X \rightarrow aXa \mid aXb \mid bXa \mid bXb \mid a$$
$$\}$$

The production $X \rightarrow a$, termjnates the recursion with a as the middle character of X.

Set of variables V = {S, X}

Set of terminals T = {a, b}

Start symbol = S

Example 5.2.33 : Find CFG for generating

(i) String containing alternate sequence of 0's and 1's, $\Sigma = \{0, 1\}$

(ii) The string containing no consecutive 'b's but 'a's can be consecutive.

(iii) The set of all string over alphabet {a, b} with exactly twice as many a's as b's.

(iv) Language having number of a's greater than number of b's.

MU - Dec. 2006, May 2009, Dec. 2009

Solution :

(i) String containing alternate sequence of 0's and 1's, $\Sigma = \{0, 1\}$

Since, any binary number will satisfy the condition of alternate sequence of 0's and 1's, the language $L = (0 + 1)^*$

The set of productions are :

$$S \rightarrow 0S \mid 1S \mid \varepsilon$$

$$\therefore \text{ CFG } G = (\{S\}, \{0, 1\}, \{S \rightarrow bS \mid 1S \mid \in\}, S)$$

(ii) The string containing no consecutive b's but a's can be consecutive.

The set of productions for the given language L are :

$$P = \{$$
$$S \rightarrow aS \mid bX \mid b \mid \varepsilon$$
$$X \rightarrow aSla$$
$$\}$$

These production can easily be written from the FA for the above language. The FA is shown in Fig. Ex. 5.2.33.

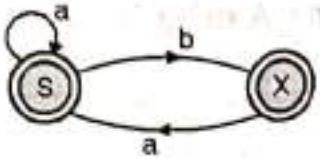


Fig. Ex. 5.2.33

Set of variables V = {S, X}

Scanned by CamScanner

Set of terminals T = {a, b}

Start symbol = S

(iii) The set of all strings over alphabet {a, b} with exactly twice as many a's as b's.

The CFG G = (V, T, P, S)

Where V = {S}

T = {a, b}

P = {S → aSaSbS | aSbSaS | bSaSaS | ε}

S = Start symbol

(iv) Language having number of a's greater than number of b's.

The set of productions for the grammar are given by :

P = {

S → SaS | aSS | SSa | a | aX | Xa

X → aB | bA

A → aX | bAA | a

B → bX | aBB | b

}

- The variable X generates a string having equal number of a's and b's. Group of excess a's over b's are generated by S-productions.

Where

Set of variables V = {S,X,A,B}

Set of terminals T = {a,b}

Start symbol = S

Example 5.2.34 : Write CFG for the language

Σ = {a, b} number of a's is a multiple of 3.

Solution : We can design a DFA for strings in which number of a's is multiple of 3.

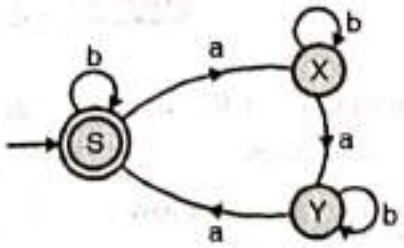


Fig. Ex. 5.2.34

The set of productions can be written from the DFA :

S → bS | aX | ε

Scanned by CamScanner

$X \rightarrow bX \mid aY$

$Y \rightarrow aS \mid bY$

Example 5.2.35 : Consider the following grammar :

$S \rightarrow aSa \mid bSb \mid aa \mid bb$

Show that

1. L(G) has no strings of odd length.
2. Any string in L(G) is of length 2n, $n > 0$
3. The number of strings of length 2n is 2^n.

Solution :

1. Each production from ($S \rightarrow aSa \mid bSb \mid aa \mid bb$) has two terminals on the right side. Terminals are always generated in pair. Therefore, L(G) has no strings of odd length.
2. Smallest string generated by S is either aa or bb with length 2. The language of the given grammar is a set of even length palindromes. Therefore, any string in L(G) is of length 2n, $n > 0$.
3. Let us take a string of length 2n (say S_{2n})

   $S_{2n} = aS_{2n-2}\,a \mid b\,S_{2n-2}\,b$

   Thus, we can write a recurrence relation for S_{2n}

   $S_{2n} = 2 \times S_{2n-2} = 2^2\,S_{2n-y} = 2^{n-1} \times S_2 = 2^n$

Example 5.2.36 : Obtain a grammar to generate the language $L = \{ 0^n 1^{2n} \mid n \geq 0) \}$. **MU - May 2010**

Solution :

Productions for the required language are as follows.

$P = \{S \rightarrow 0\,S11 \mid \varepsilon\}$

CFG for the above language is ({S}, {0, 1}, P, S)

5.3 Ambiguous Grammar

MU - May 2007, Dec. 2008, May 2009, Dec. 2011, May 2012, Dec. 2012, May 2013

A grammar is said to be ambiguous if the language generated by the grammar contains some string that has two different parse trees.

Example : Let us consider the grammar given below :

$E \rightarrow E + E \mid a \mid b$ ...(Grammar 5.8)

A string a + b + a is generated by the given grammar.

Scanned by CamScanner

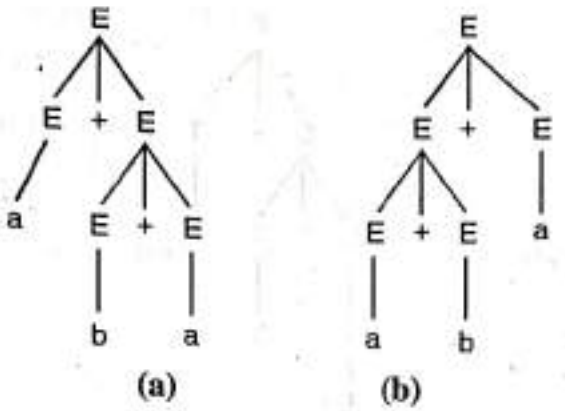


Fig. 5.3.1 : Parse trees considered for ambiguity

- The grammar 5.8, generates a + b + a in two different ways. The two derivations are shown in Fig. 5.3.1(a) and (b).
- The first derivation (Fig. 5.3.1(a)) says that (b + a) is evaluated first and then the evaluated value is added to a. Thus the right side + operator gets a precedence over the left side + operator. The expression a + b + a is treated as (a + (b + a)).
- The second derivation (Fig. 5.3.1(b)) says that a + b is evaluated first and then the evaluated value is added to a. Thus the left side + operator will get precedence over right side + operator. The expression a + b + a is treated as ((a + b) + a).

Removing ambiguity : There is no general rule for removing ambiguity from CFG. Removing ambiguity from a grammar involves rewriting of grammar so that there is only one derivation tree for every string belonging to L(G) i.e., language generated by grammar G.

Ambiguity from the grammar

$$E \rightarrow E + E \mid a \mid b$$

can be removed by strictly assigning higher precedence to left side + operator over right side + operator. This will mean evaluation of an expression of the form a + b + a,, from left to right. This property can be incorporated in the grammar itself by suitably modifying the grammar.

- Parse tree of Fig. 5.3.1(b) is based on left to right evaluation.
- Left to right evaluation in Grammar (5.8) can be enforced by introducing one more variable T. Variable T cannot be broken by + operator.
- An unambiguous grammar for the Grammar (5.8) is given in Grammar (5.9)

$$E \rightarrow E + T \mid T$$
$$T \rightarrow a \mid b$$

...(Grammar 5.9)

- A production of the form E → E + T provides a binding. E + T, implies that E must be evaluated first before an atomic T can be added to it. E can be broken down in E + T but T cannot be broken further. This ensures higher precedence to left side + operator over right side + operator.
- Parse tree in Fig. 5.3.2 is based on unambiguous Grammar (5.9), for the string a + b + a.

Scanned by CamScanner

Fig. 5.3.2 : A parse tree for a + b + a using an unambiguous grammar

In general,

1. Sometimes we can remove ambiguity from a grammar by hand.
2. There is no algorithm for either deletion of ambiguity or for removing ambiguity.
3. Some context free languages are inherently ambiguous. They have only ambiguous CFG.

An example of inherently ambiguous language is

$$L = L_1 \cup L_2$$

Where,

$$L_1 = \{a^n b^m c^m d^n \mid n, m > 0\}$$

$$L_2 = \{a^n b^n c^m d^m \mid n, m > 0\}$$

Example 5.3.1 : Consider the grammar

$E \rightarrow E + E \mid E * E \mid (E) \mid I$

$I \rightarrow a \mid b$

a) Show that the grammar is ambiguous.

b) Remove ambiguity.

Solution : There are two problems in the given grammar

a) Both + operator and * operator are at the same precedence level.

b) An expression can be evaluated either left to right or right to left.

These problems can be removed by introducing more variables, each representing expressions that share a level of "binding strength."

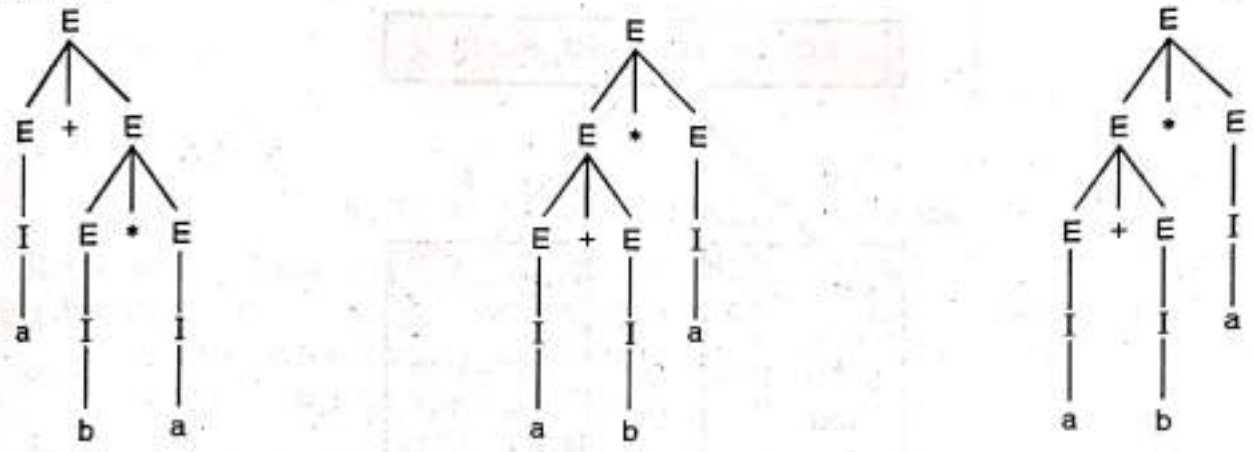
- A variable T is an expression that cannot be broken by + operator. Let us call it a term. A term can be expressed as product of one or more factors.
- A variable F is a factor that cannot be broken by + operator or * operator. A factor is
  - (a) Identifiers
  - (b) Literals
  - (c) Parenthesized expressions

Scanned by CamScanner

An unambiguous grammar is given below :

$$\left.\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid a \mid b \end{aligned}\right\} \quad \text{(Grammar 5.10)}$$

A parse tree for a + b * a using both ambiguous and unambiguous grammar is shown in Fig. Ex. 5.3.1.



(a) Using ambiguous grammar **(b) Using ambiguous grammar** **(c) Using unambiguous grammar**

Fig. Ex. 5.3.1 : Parse trees for a + b * a

Fig. Ex. 5.3.1(a) and Ex. 5.3.1(b) are using the ambiguous grammar.

Fig. Ex. 5.3.1(c) is using an unambiguous grammar.

The grammar given in the example is an ambiguous grammar as there are two different parse trees for the expression a + b * a (shown in Fig. 5.22(a) and 5.22(b)).

Example 5.3.2 : Give an ambiguous grammar for if-then-else statement and then re-write an equivalent unambiguous grammar.

Solution :

The dangling else is a well known problem in programming language. If statement in C-language can be written in any of the two forms :

```
Form A : if (x > y)
           a = a + 1 ;
Form B : if (x > y)
           a = a + 1
         else
           a = a - 1;
```

Scanned by CamScanner

We can substitute the condition x > y with C and S for a = a + 1. With these substitutions, if statement in two forms can be written as :

if	C then S
if	C then S_1 else S_2

For the case of understanding, a reserve word 'then' has been introduced.

A nested if statement can be written with two interpretations. It is shown in Fig. Ex. 5.3.2.

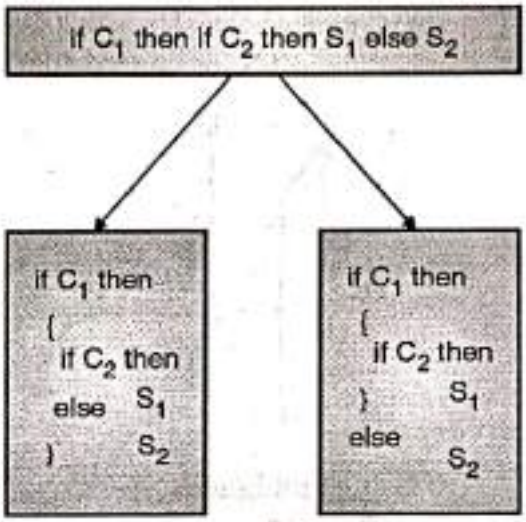


Fig. Ex. 5.3.2 : Two interpretations of a nested if-statement

- A nested if statement can have two interpretations.

Example 5.3.3 : An ambiguous grammar for if statement may have dangling else problem.

Solution :

An unambiguous grammar should attach the dangling else with inner if-statement.

An ambiguous grammar for if-statement is given below :

$S \rightarrow$ if C then S
| if C then S else S
| a [termination of a statement by 'a']
$C \rightarrow c_1 \mid c_2$

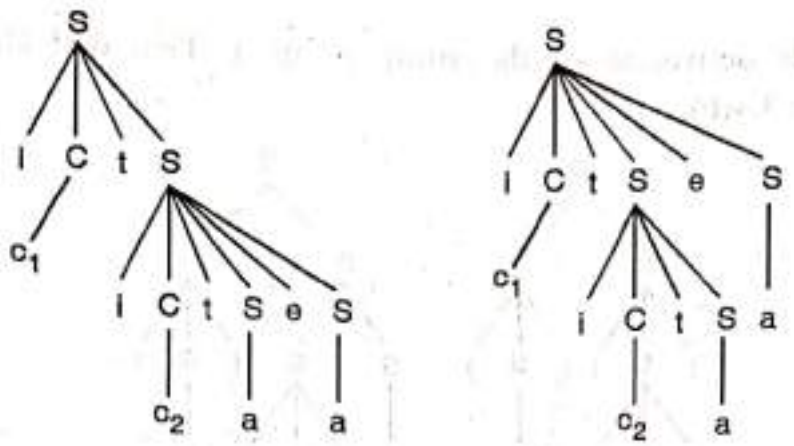
The above grammar can be written in short as :

$S \rightarrow iCtS \mid iCtSeS \mid a$
$C \rightarrow c_1 \mid c_2$
– (Grammar 5.11)

Where, i stands for if,
t stands for then,
and e stands for else

Example : Grammar (5.11) can be shown to be ambiguous by drawing two different derivation trees for the string ic_1tic_2taea. Two different derivations are shown in the Fig. Ex. 5.3.3(a),(b).

Scanned by CamScanner



(a) (b)

Fig. Ex. 5.3.3 : Two derivations for ic_1tic_2taea

Removing ambiguity : The ambiguity from the grammar (5.11) can be removed by binding dangling else with inner if – statement. We will introduce two more variables.

U → For if - statement without else part.

M → For if - statement with else part.

An unambiguous grammar is given below :

$S \rightarrow U \mid M$
$M \rightarrow iCtMeM \mid a$
$U \rightarrow iCtS \mid iCtMeU$
$C \rightarrow c_1 \mid c_2$

– (Grammar 5.12)

The working of grammar (5.12) can be understood with the help of derivation tree for ic_1tic_2taea. It is shown in Fig. Ex. 5.3.3(c).

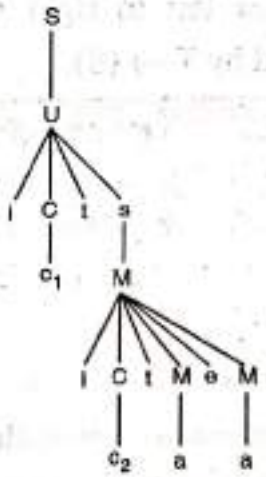


Fig. Ex. 5.3.3(c) : Derivation tree for ic_1tic_2taea

- Outer if-statement is without else part.
- Inner if-statement is with else part.

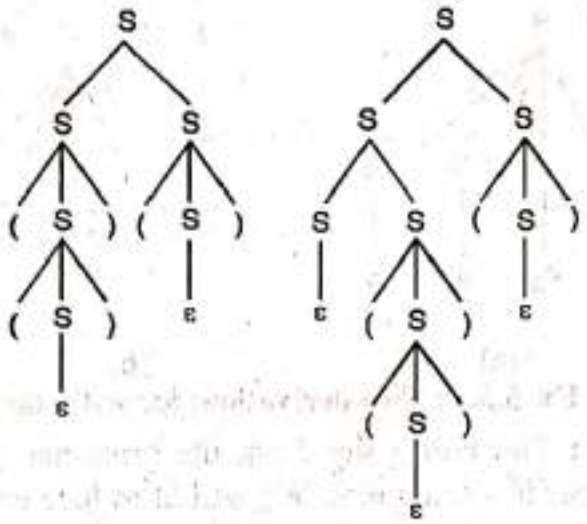
Example 5.3.4 : Show that the CFG given below. Which generates all strings of balanced parentheses is ambiguous. Give an equivalent unambiguous grammar.

$S \rightarrow SS \mid (S) \mid \varepsilon$

Scanned by CamScanner

Solution :

Let us consider derivation of the string (( ))( ). Two derivations are shown in Fig. Ex. 5.3.4(a) and 5.3.4(b).



(a) (b)

Fig. Ex. 5.3.4 : Two different derivations for (( ))( ).

Since, there are two different derivation for the string (( ))( ) the grammar is ambiguous.

An unambiguous grammar is given by

$$S \rightarrow ST \mid T$$

$$T \rightarrow (S) \mid ()$$

- Sideways parentheses can be generated by $S \rightarrow ST$, where T is atomic as far as sideways generation is concerned. It forces left to right generation of balanced parenthesis. Nesting of parenthesis is generated by $T \rightarrow (S)$.

Example 5.3.5 : Write an unambiguous CFG for arithmetic expressions with operators : +, *, /, ^, unary minus and operand a, b, c, d, e, f.
Also, if should be possible to generate brackets with your grammar.
Derive (a + b) ^ d / e + (– f) from your grammar. **MU - Dec. 2005**

Solution :

An unambiguous grammar is given below.

$E \rightarrow E + T \mid T$ [+ has lowest priority with L → R associativity]

$T \rightarrow T * F \mid T / F \mid F$ [* and / has higher priority over + with L → R associativity]

$F \rightarrow F \wedge G \mid G$ [^ has higher priority over * and / with L → R associativity]

$G \rightarrow - H \mid H$ [unary – has the highest priority]

$H \rightarrow a \mid b \mid c \mid d \mid e \mid f \mid (E)$ [to handle brackets and identifiers]

Derivation tree for (a + b) ^ d / e + (– f)

Scanned by CamScanner

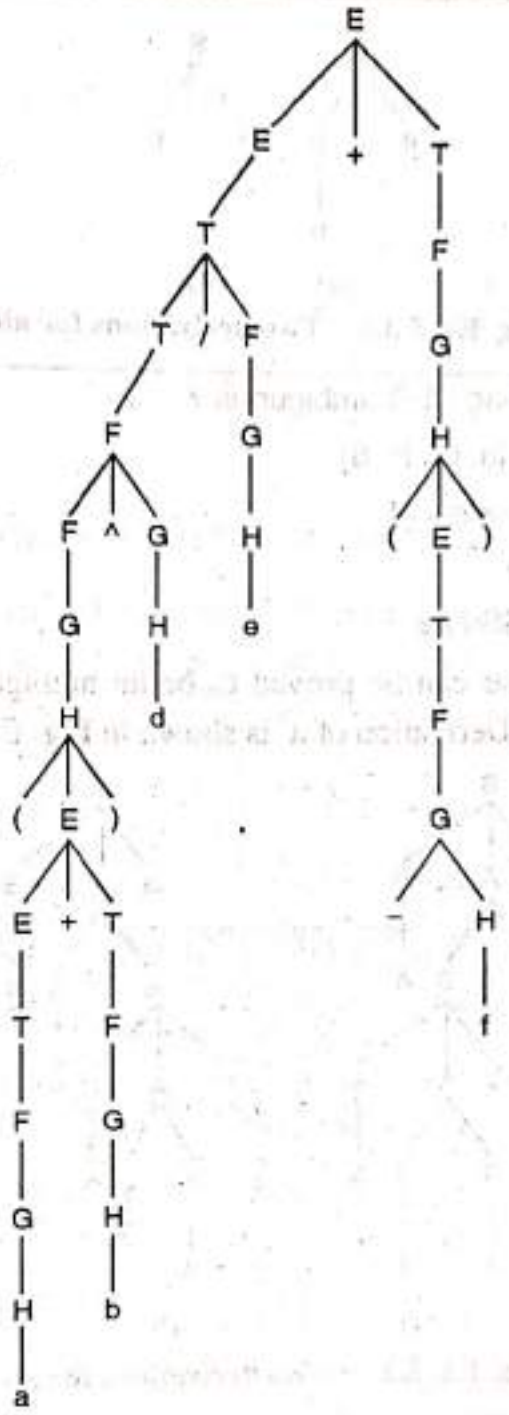


Fig. Ex. 5.3.5 : Derivation tree for (a + b) ^ d / e + (– f)

Example 5.3.6 : Is the following CFG ambiguous ?

$S \rightarrow aB \mid ab$

$A \rightarrow aAB \mid a$

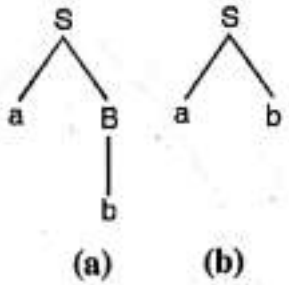
$B \rightarrow ABb \mid b$

If so, show multiple derivation trees for the same string.

Solution :

The given CFG is ambiguous as we can have two different derivation trees for the string ab. Derivation trees are shown in Fig. Ex. 5.3.6.

Scanned by CamScanner



(a) (b)

Fig. Ex. 5.3.6 : Two derivations for ab

Example 5.3.7 : Is the following CFG ambiguous ?

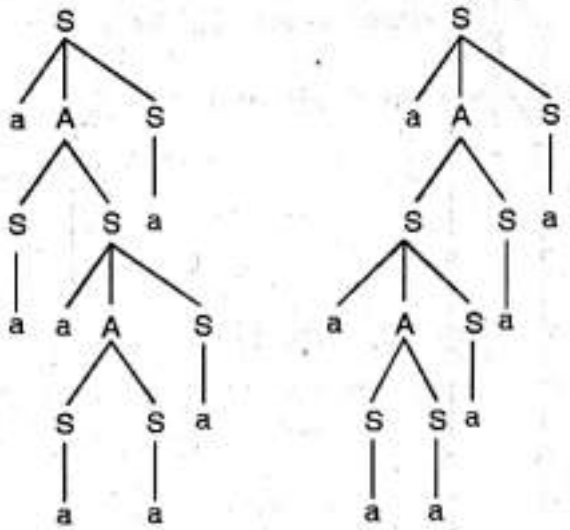
$G = (\{S, A\}, \{a, b\}, P, S)$

where, P consists of

$S \rightarrow aAS \mid a$

$A \rightarrow SbA \mid SS \mid ba$

Solution : The above grammar can be proved to be an ambiguous grammar by deriving a string a^7 in two different ways. Derivation of a^7 is shown in Fig. Ex. 5.3.7.



(a) (b)

Fig. Ex. 5.3.7 : Two derivations for a^7

Example 5.3.8 : Consider the grammar having productions :

$S \rightarrow aS \mid \varepsilon$

$S \rightarrow aSbS$

The grammar is ambiguous.

(i) Show in particular that the string aab has two parse trees.

(ii) Find an unambiguous grammar for the same.

Scanned by CamScanner

Solution :

(i) Two parse trees for aab are given in Fig. Ex. 5.3.8.

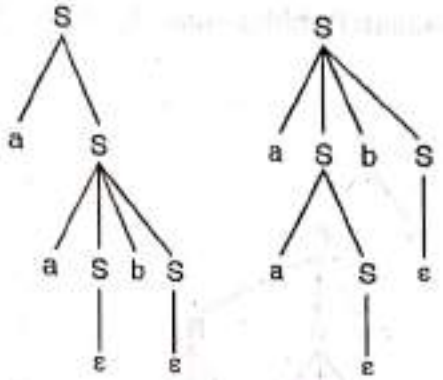


Fig. Ex. 5.3.8 : Two derivations for aab

(ii) The set of productions resemble productions for if-statement. Thus the ambiguity can be removed in a similar way through introduction of two variables U and M with resulting productions is :

$$S \rightarrow U \mid M$$

$$U \rightarrow aS \mid aMbU$$

$$M \rightarrow aMbM \mid \varepsilon$$

Example 5.3.9 : Let G be the grammar
$S \rightarrow aB \mid bA$
$A \rightarrow a \mid aS \mid bAA$
$B \rightarrow b \mid bS \mid aBB$
Find
(i) Left most derivation
(ii) Right most derivation
(iii) Parse Tree
(iv) Is the grammar unambiguous ?
For given strings (A) aaabbabbba (B) bbaaabbaba (C) 00110101

MU - Dec. 2009, Dec. 2012, May 2013

Solution :

(A) For string "aaabbabbba"

It will be worthwhile to draw the parse tree and from the parse tree, one can easily write left most and right most derivation.

(i) Left most derivation :

$S \rightarrow aB \rightarrow aaBB \rightarrow aaaBBB \rightarrow aaabBB$

$\rightarrow aaabbB \rightarrow aaabbaBB \rightarrow aaabbabB \rightarrow aaabbabbS$

$\rightarrow aaabbabbbA \rightarrow aaabbabbba$

Scanned by CamScanner

(ii) Right most derivation :

$S \rightarrow aB \rightarrow aaBB \rightarrow aaBaBB \rightarrow aaBaBbS \rightarrow aaBaBbbA$

$\rightarrow aaBaBbba \rightarrow aaBabbba \rightarrow aaaBBabbba \rightarrow aaaBbabbba \rightarrow aaabbabbba$

(iii) Parse tree :

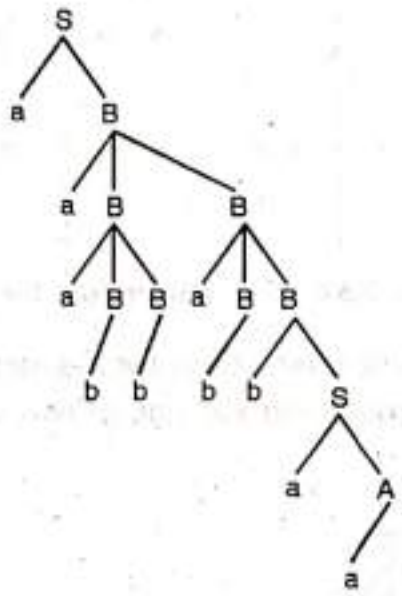
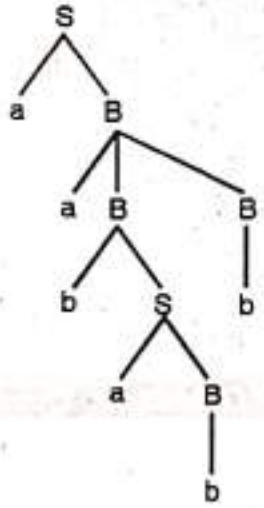
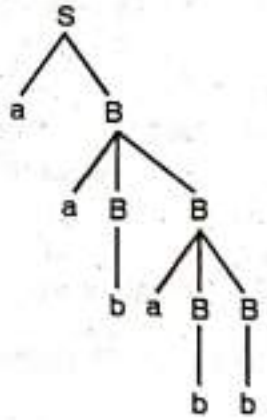


Fig. Ex. 5.3.9

(iv) The grammar is ambiguous as we can draw two parse trees for aababb :



(a)



(b)

Fig. Ex. 5.3.9

(B) For string "bbaaabbaba"

(i) Leftmost derivation

$S \rightarrow bA \rightarrow bbAA \rightarrow bbaA \rightarrow bbaaS$

$\rightarrow bbaaaB \rightarrow bbaaabs \rightarrow bbaaabbA$

$\rightarrow bbaaabbas \rightarrow bbaaabbabA \rightarrow bbaaabbaba$

Scanned by CamScanner

(ii) Rightmost derivation

$S \rightarrow bA \rightarrow bbAA \rightarrow bbAaS \rightarrow bbAaaB$

$\rightarrow bbAaabS \rightarrow bbAaabbA \rightarrow bbAaabbaS$

$\rightarrow bbAaabbabA \rightarrow bbAaabbaba \rightarrow bbaaabbaba$

(iii) Parse tree for bbaaabbaba

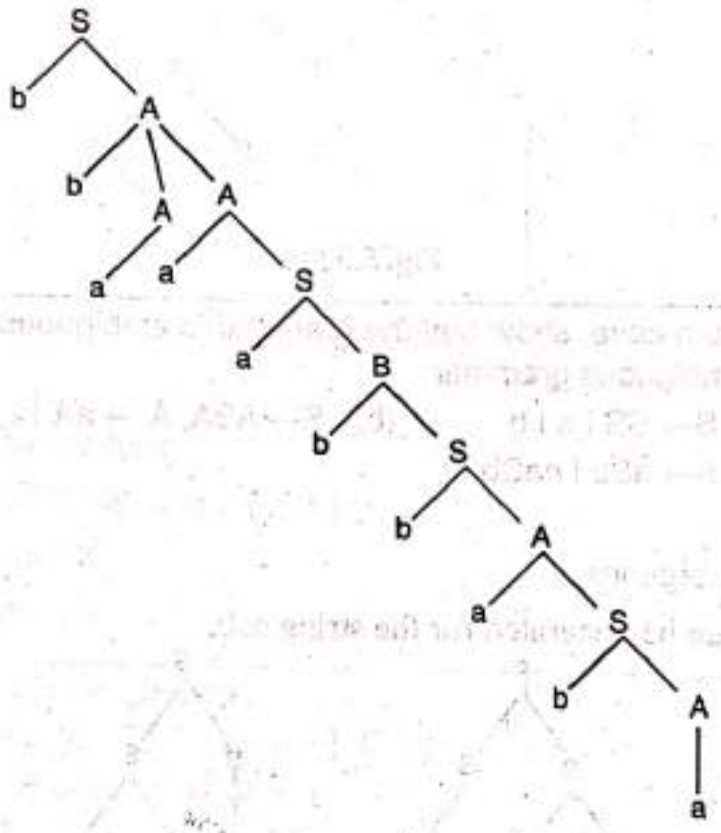


Fig. Ex. 5.3.9 (c)

(C) For the string 00110101

(i) Leftmost derivation

$S \rightarrow 0BB \rightarrow 00BB \rightarrow 001B \rightarrow 0011S$

$\rightarrow 00110 \quad B \rightarrow 001101S \rightarrow 0011010B$

$\rightarrow 00110101$

(ii) Rightmost derivation

$S \rightarrow 0B \rightarrow 00BB \rightarrow 00B1S \rightarrow 00B10B$

$\rightarrow 00B101S \rightarrow 00B1010B \rightarrow 00B10101$

$\rightarrow 001110101$

Scanned by CamScanner

(iii) Parse tree

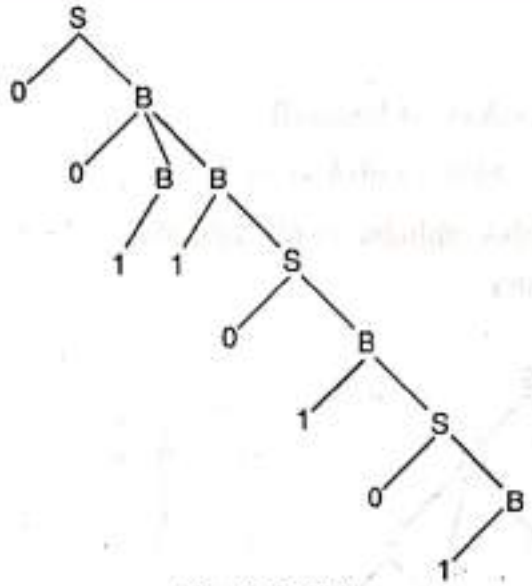


Fig. 5.3.9(d)

Example 5.3.10 : In each case, show that the grammar is ambiguous, and find the equivalent unambiguous grammar

(a) $S \rightarrow SS \mid a \mid b$ (b) $S \rightarrow ABA,\ A \rightarrow aA \mid \varepsilon,\ B \rightarrow bB \mid \varepsilon$

(c) $S \rightarrow aSb \mid aaSb \mid \varepsilon$

Solution :

(a) $S \rightarrow SS \mid a \mid b$ is ambiguous

Two parse trees can be generated for the string aab.

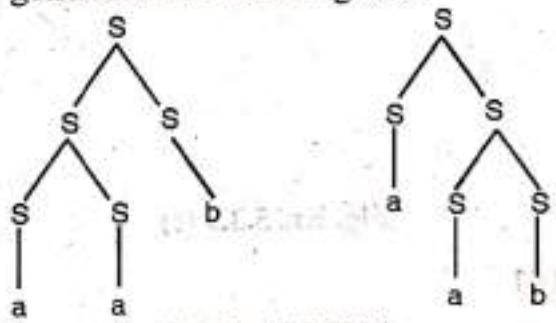


Fig. Ex. 5.3.10(a)

Unambiguous grammar :

$S \rightarrow aS \mid bS \mid aa \mid ab \mid ba \mid bb$

(b) $S \rightarrow ABA,\ A \rightarrow aA \mid \varepsilon,\ B \rightarrow bB \mid \varepsilon$

Two parse trees can be generated for the string aa

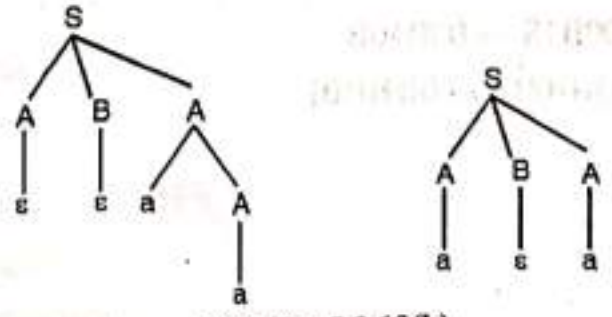


Fig. Ex. 5.3.10(b)

Scanned by CamScanner

Unambiguous grammar :

$S \rightarrow aS \mid bX\varepsilon$ [String of only a's will be generated by $S \rightarrow aS \mid \varepsilon$]

$X \rightarrow bX \mid y$

$Y \rightarrow aY \mid \varepsilon$

(c) $S \rightarrow aSb \mid aaSb \mid \varepsilon$ is ambiguous

We can draw two production trees for aaabb

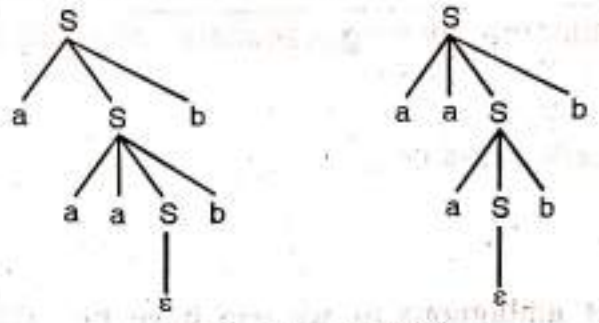


Fig. Ex. 5.3.10(c)

Unambiguous grammar :

Given language is of the form

$L = \{a^i b^j \mid i \geq j \text{ and } i, j \geq 0\}$

$S \rightarrow aSb \mid X$

$X \rightarrow aX \mid \varepsilon$

Example 5.3.11 : Consider the grammar

$G = \{V = \{E, F\}, T = \{a, b, -\}, E, P\}$

where P consists of rules :

$E \rightarrow F - E, F \rightarrow a, E \rightarrow E - F, F \rightarrow b, E \rightarrow F$

(a) Show that G is ambiguous (b) Remove the ambiguity.

Solution : The production rules can be re-written as given below :

$E \rightarrow F - E \mid E - F \mid F$

$F \rightarrow a \mid b$

(a) The grammar can be shown to be an ambiguous grammar by drawing two different derivation trees for the string a – b.

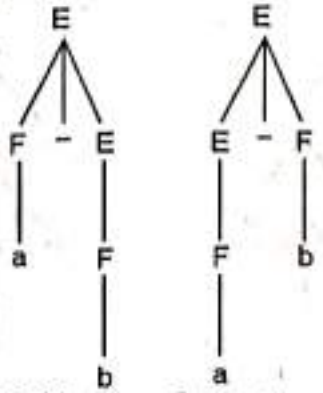


Fig. Ex. 5.3.11 : Two derivations for a – b

Scanned by CamScanner

(b) The production $E \rightarrow F - E$, makes the evaluation from right to left which is normally not allowed.

Thus the ambiguity can be removed by deleting the product the production $E \rightarrow F - E$.

The resulting unambiguous grammar is :

$$E \rightarrow E - F \mid F$$
$$F \rightarrow a \mid b$$

Example 5.3.12 : Test whether the following grammars are ambiguous.

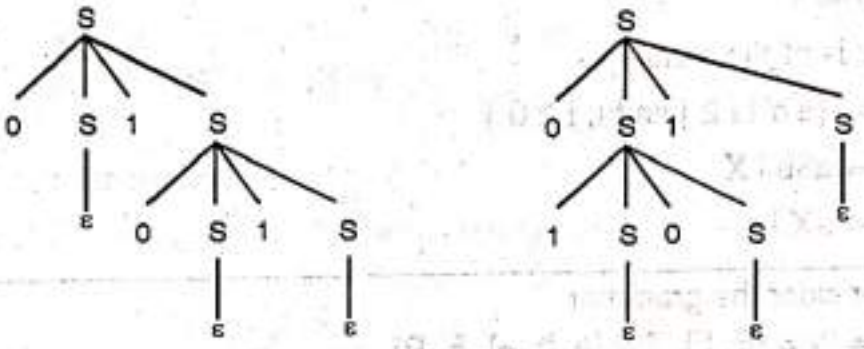
1. $S \rightarrow 0S1S \mid 1S0S \mid \varepsilon$
2. $S \rightarrow AA$
   $A \rightarrow aAb \mid bAa \mid \varepsilon$

MU - May 2010

Solution :

1. $S \rightarrow 0S1S \mid 1S0S \mid \varepsilon$

The given grammar is ambiguous as we can have two different parse trees for the string 0101.

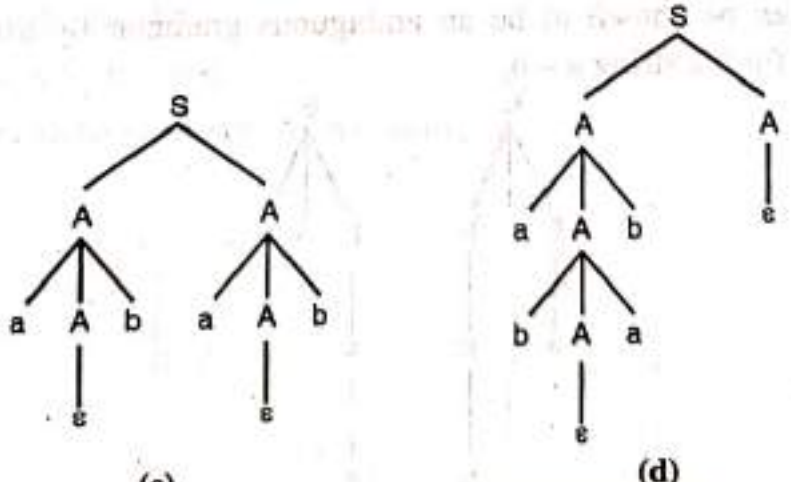


(a) (b)

Fig. Ex. 5.3.12

2. $S \rightarrow AA$
   $A \rightarrow aAb \mid bAa \mid \varepsilon$

The given grammar is ambiguous. We can derive two different parse trees for the string abab.



(c) (d)

Fig. Ex. 5.3.12

Scanned by CamScanner

5.4 Simplification of CFG

A grammar written in a simple form is easy to analyze. Certain restrictions are imposed on simplified grammar. Simplification of CFG involves transforming CFG into an equivalent form that satisfies certain restrictions on its form. A CFG can be simplified by eliminating :

1. Useless symbols.
2. ε – productions.
3. Unit productions.

5.4.1 Elimination of Useless Symbols

A grammar may contain symbols and productions which are not useful for derivation of strings. Two types of symbols are useless :

1. Non - generating symbols.
2. Non reachable symbol.

5.4.1.1 Non-generating Symbols

A symbol $X \in V$ (Set of variables) is a generating symbol if $X \overset{*}{\underset{G}{\Rightarrow}} w$,

where, $w \in T^*$ i.e. every variable must generate a string of terminals.

Example : Grammar

$$\begin{aligned} S &\rightarrow Aa \mid Bb \mid a \mid b \\ A &\rightarrow Aa \mid a \\ B &\rightarrow bB \end{aligned}$$ ...Grammar (4-14)

Requires simplification as the symbol B is non-generating. Only production for B is

$$B \rightarrow bB$$

and it cannot generate a string of terminals. The Grammar (4-14) can be simplified by deleting every production containing the useless symbol B. A simplified grammar is given in (4-15).

$$\begin{aligned} S &\rightarrow Aa \mid a \mid b \\ A &\rightarrow Aa \mid a \end{aligned}$$ ...Grammar (4-15)

A grammar containing a non-generating symbol V_i should be simplified by deleting every production containing the non-generating symbol V_i.

Finding non-generating symbols :

There are two rules for finding a set of generating symbols for the given grammar.

1. Every symbol in T (terminal) is generating.

Scanned by CamScanner

2. If there is a production $A \rightarrow \alpha$ and every symbol in α is generating, then A is generating.
Where, $\alpha \in (V + T)^*$.
A symbol not in a set of generating symbols is said to be non-generating.

Example 5.4.1 : Find non-generating symbols in the grammar given below

$S \rightarrow AB \mid CA$

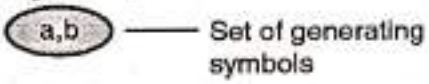
$B \rightarrow BC \mid AB$

$A \rightarrow a$

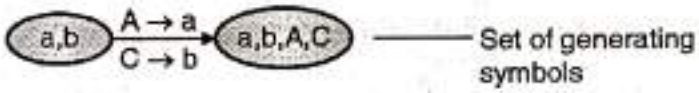
$C \rightarrow aB \mid b.$

Solution :

Step 1 : Every terminal is generating

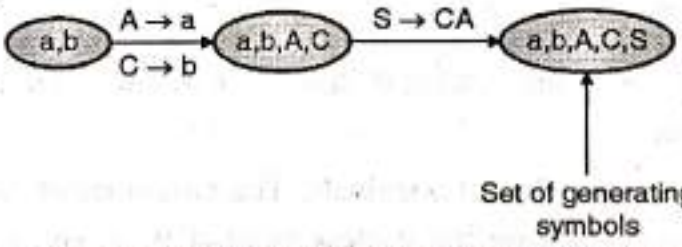


Step 2 : A and C are generating as $A \rightarrow a$ and $C \rightarrow b$.



Two symbols A and C are added to the set of generating symbols using the Rule 2, which says if there is a production of the form $A \rightarrow \alpha$ and every symbol in α is generating then A is generating.

Step 3 : Symbol **S** is generating as there is a production $S \rightarrow CA$ with both C and A as generating.



Thus the symbol **B** is a non-generating symbol as it does not belong to the set of generating symbols.

Grammar in Example 5.4.1 can be simplified by deleting productions with the useless symbol B. Simplified grammar is given below :

$S \rightarrow CA$

$A \rightarrow a$

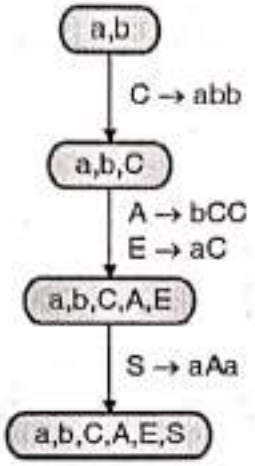
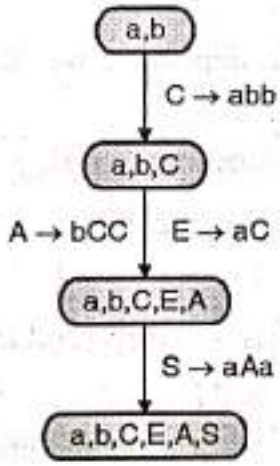
$C \rightarrow b$

Example 5.4.2 : Eliminate non-generating symbols from the grammar.

1. $P = \{S \rightarrow aAa, A \rightarrow Sb \mid bCC, C \rightarrow abb, E \rightarrow aC\}$
2. $P = \{S \rightarrow aAa, A \rightarrow Sb \mid bCC \mid DaA, C \rightarrow abb \mid DD, E \rightarrow aC, D \rightarrow aDA\}$

Scanned by CamScanner

Solution :

Sr. No.	Grammar	Generating symbols	Non-generating symbols	Simplified grammar
1.	S → aAa, A → Sb | bCC, C → abb, E → aC		Every symbol is generating	Same as the original grammar S → aAa, A → Sb | bCC C → abb E → aC
2.	S → aAa A → Sb | bCC | DaA C → abb | DD E → aC D → aDa		Symbol D is non-generating	S → aAa A → Sb | bCC C → abb E → aC

5.4.1.2 Non-reachable Symbols

A symbol X is reachable if it can be reached from the start symbol S.

i.e. if $S \overset{*}{\underset{G}{\Rightarrow}} \alpha$ and α contains a variable X then X is reachable.

A grammar containing a non-reachable symbol V_i should be simplified by deleting every production containing the non-reachable symbol V_i.

Finding non-reachable symbols :

Non-reachable symbols can be located with the help of a dependently graph. A variable X is said to be dependent on S if there is a production

$$S \rightarrow \alpha_1 \times \alpha_2$$

The dependency graph is shown in Fig. 5.4.1.

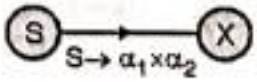


Fig. 5.4.1 : Depending Graph

Scanned by CamScanner

- We must draw a dependency graph for all productions.
- If there is no path from the start symbol S to a variable X, then X is non-reachable.

Example 5.4.3 : Eliminate non-reachable symbols from the given grammar.
$P = \{S \rightarrow aAa, A \rightarrow Sb \mid bCC, C \rightarrow abb, E \rightarrow aC\}$

Solution :

Step 1 : Drawing of dependency graph :

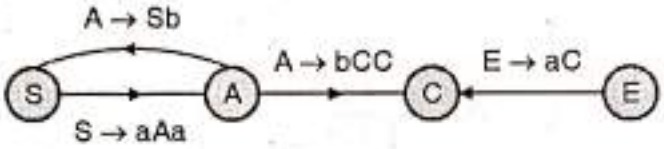


Fig. Ex. 5.4.3 : Dependency Graph

- From the production $S \rightarrow aAa$, A is dependent on S. It is shown using a directed edge from S to A.
- From the production $A \rightarrow bCC$, C is dependent on A. From the production $A \rightarrow Sb$, S is dependent of A.
- From the production $E \rightarrow aC$, C is dependent on E.

There is no path from S to E, E is non-reachable.

Step 2 : Simplification of grammar :

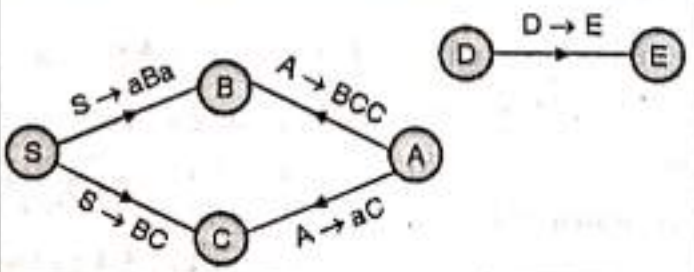
Grammar can be simplified by deleting every production containing the non-reachable symbol E. Simplified grammar is given below :

$$P_1 = \{S \rightarrow aAa, A \rightarrow Sb \mid bCC, C \rightarrow abb\}$$

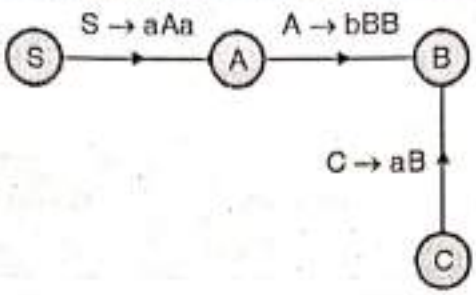
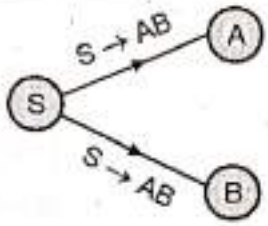
Example 5.4.4 : Eliminate non-reachable symbols from the grammar :

1. $P = \{S \rightarrow aBa \mid BC, A \rightarrow aC \mid BCC, C \rightarrow a, B \rightarrow bCC, D \rightarrow E, E \rightarrow d\}$
2. $P = \{S \rightarrow aAa, A \rightarrow bBB, B \rightarrow ab, C \rightarrow aB\}$
3. $P = \{S \rightarrow aS \mid AB, A \rightarrow bA, B \rightarrow AA\}$

Solution :

Grammar	Dependency graph	Non-reachable symbols	Reduced grammar
1. $S \rightarrow aBa \mid BC$ $A \rightarrow aC \mid BCC$ $C \rightarrow a, B \rightarrow bCC$ $D \rightarrow E, E \rightarrow d$		A, D and E are non-reachable	$S \rightarrow aBa \mid BC$ $C \rightarrow a$ $B \rightarrow bCC$

Scanned by CamScanner

Grammar	Dependency graph	Non-reachable symbols	Reduced grammar
2. S → aAa A → bBB B → ab C → aB		Symbol C is non-reachable	S → aAa A → bBB B → ab
3. S → aS | AB A → bA ; B → AA		Every symbol is reachable	Same as the original grammar

5.4.2 Elimination of ε-productions

A productions of the form A → ε, is called a null production or ε-production. For every context free grammar G with ε-productions, we can find a context-free grammar G_1 having no ε-productions such that

$$L(G_1) = L(G) - \{\varepsilon\}$$

The procedure for finding G_1 (equivalent grammar without ε-productions) is as follows :

Step 1 : Find nullable variables

Step 2 : Addition of productions with nullable variables removed

Step 3 : Remove ε-productions.

Step 1 : Find nullable variables : A symbol X is nullable if

1) There is a production of the form X → ε.
2) If there is a production of the form X → α and all symbols of α are nullable.

Example : In the grammar with productions S → ABA, A → aA | ε, B → bB | ε

A is nullable as there is a production A → ε

B is nullable as there is a production B → ε

S is nullable as there is a production S → ABA

With both A and B as nullable.

∴ The set of nullable symbols = {S, A, B}

Step 2 : Addition of productions with nullable variables removed.

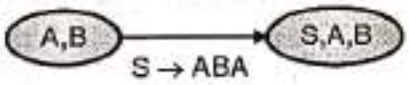
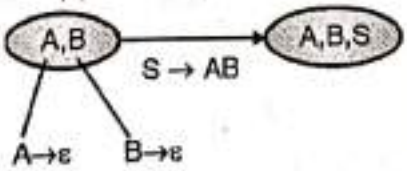
An ε-production has an effect. This effect must be added as productions to the existing grammar before ε-productions are removed.

Scanned by CamScanner

Productions compensating the effect of ε-productions :

For each production of the form $A \rightarrow \alpha_1$, create all possible productions of the form $A \rightarrow \alpha_2$, where α_2 is obtained from α_1 by removing one or more occurrences of nullable variables.

Example :

Sr. No.	Grammar with ε-productions	Nullable symbols	Grammar after addition of effect of nullable symbols
1.	$S \rightarrow aS$ $S \rightarrow \varepsilon$	{S}	$S \rightarrow aS$ $S \rightarrow a$ $S \rightarrow \varepsilon$ [$S \rightarrow aS$ is written as $S \rightarrow a$ and $S \rightarrow aS$. S is nullable and hence $S \rightarrow aS$ can also be written as $S \rightarrow a$]
2.	$S \rightarrow ABA$ $A \rightarrow \varepsilon$ $B \rightarrow \varepsilon$	 A and B are nullable as $A \rightarrow \varepsilon$ and $B \rightarrow \varepsilon$. S is null as every symbol on the right of $S \rightarrow ABA$ is nullable. ∴ Nullable symbols = {A, B, S}	$S \rightarrow ABA$ $S \rightarrow AB$ $S \rightarrow AA$ $S \rightarrow BA$ $S \rightarrow A$ $S \rightarrow B$ $S \rightarrow \varepsilon$ $A \rightarrow \varepsilon$ $B \rightarrow \varepsilon$ Productions derived from $S \rightarrow ABA$ by making one or more symbols null.
3.	$S \rightarrow aS \mid AB$ $A \rightarrow \varepsilon$ $B \rightarrow \varepsilon$	 Nullable set = {A, B, S}	$S \rightarrow aS$ $S \rightarrow a$ Derived from $S \rightarrow As$ $S \rightarrow AB$ $S \rightarrow A$ $S \rightarrow B$ Derived from $S \rightarrow AB$ $S \rightarrow \varepsilon$ $A \rightarrow \varepsilon$ $B \rightarrow \varepsilon$

Step 3 : Remove ε-productions from the grammar obtained in step 2.

1. $\{S \rightarrow aS \mid a\}$
2. $\{S \rightarrow ABA \mid AB \mid AA \mid BA \mid A \mid B\}$
3. $\{S \rightarrow aS \mid a \mid AB \mid S \rightarrow A \mid S \rightarrow B\}$

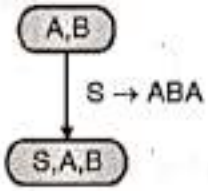
Scanned by CamScanner

Example 5.4.5 : Remove ε-productions from the given grammar.

$S \rightarrow ABA,\ A \rightarrow aA \mid \varepsilon,\ B \rightarrow bB \mid \varepsilon$

Solution :

Step 1 : Find nullable variables



A and B are nullable as $A \rightarrow \varepsilon$ and $B \rightarrow \varepsilon$.

S is nullable as A and B are nullable and there is a production $S \rightarrow ABA$, with every symbol on the right as nullable.

∴ Nullable set of symbols = {A, B, S}

Step 2 : Addition of productions with nullable variables removed.

$$S \rightarrow ABA \text{ can be written as } \left\{ \begin{array}{l} S \rightarrow ABA \\ S \rightarrow AB \\ S \rightarrow BA \\ S \rightarrow AA \\ S \rightarrow A \\ S \rightarrow B \\ S \rightarrow \varepsilon \end{array} \right\}$$

$$A \rightarrow aA \text{ can be written as } \left\{ \begin{array}{l} A \rightarrow aA \\ A \rightarrow a \end{array} \right\}$$

$$B \rightarrow bB \text{ can be written as } \left\{ \begin{array}{l} B \rightarrow bB \\ B \rightarrow b \end{array} \right\}$$

∴ Equivalent set of productions.

$$P_1 = \left\{ \begin{array}{l} S \rightarrow ABA \mid AB \mid BA \mid AA \mid A \mid B \mid \varepsilon \\ A \rightarrow aA \mid a \mid \varepsilon \\ B \rightarrow bB \mid b \mid \varepsilon \end{array} \right\}$$

Step 3 : Remove ε – productions

The final set of productions is given by

$$P_2 = \left\{ \begin{array}{l} S \rightarrow ABA \mid AB \mid BA \mid AA \mid A \mid B \\ A \rightarrow aA \mid a \\ B \rightarrow bB \mid b \end{array} \right\}$$

Example 5.4.6 : Remove ε-productions from the given grammar.

$S \rightarrow AB$

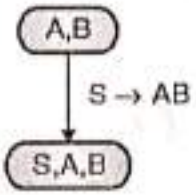
$A \rightarrow aAA \mid \varepsilon$

$B \rightarrow bBB \mid \varepsilon$

Scanned by CamScanner

Solution :

Step 1 : Find nullable variables



A and B are nullable as $A \to \varepsilon$ and $B \to \varepsilon$.

S is nullable as A and B are nullable and $S \to AB$.

$\therefore$ Nullable set of symbols = {S, A, B}

Step 2 : Addition of production with nullable variable removed.

$S \to AB$ can be written as $\left\{\begin{array}{l} S \to AB \\ S \to A \\ S \to B \\ S \to \varepsilon \end{array}\right\}$ By making one or more nullable symbols as null.

$A \to aAA$ can be written as $\left\{\begin{array}{l} A \to aAA \\ A \to aA \\ A \to a \end{array}\right\}$ By making one or more nullable symbols as null.

$B \to bBB$ can be written as $\left\{\begin{array}{l} B \to bBB \\ B \to bB \\ B \to b \end{array}\right\}$ By making one or more nullable symbols as null.

Step 3 : Remove ε-productions

$\therefore$ Equivalent set of productions.

$$P_1 = \left\{\begin{array}{l} S \to AB \mid A \mid B \\ A \to aAA \mid aA \mid a \\ B \to bBB \mid bB \mid b \end{array}\right\}$$

5.4.3 Elimination of Unit Productions

A production of the form

$$A \to B$$

It is known as a unit production. A and B are variables.

For every context free grammar G with unit productions, we can find a context free grammar G_1 having no unit productions such that

$$L(G) = L(G_1)$$

Scanned by CamScanner

The procedure for finding G_1 (an equivalent grammar without unit productions) is as follows :

- The technique is based on expansion of unit production until it disappears. This technique works in most of the cases. This technique does not work if there is a cycle of unit productions such as

  $A_1 \rightarrow A_2$, $A_2 \rightarrow A_3$, $A_3 \rightarrow A_4$ and $A_4 \rightarrow A_1$

- The steps for elimination of unit production are as follows :

Step 1 : Add all non-unit production of G to G_1.

Step 2 : Locate every pair of variables (A_i, A_j) such that $A_i \overset{*}{\underset{G}{\Rightarrow}} A_j$.

Step 3 : From pairs constructed in step 3, we can construct a chain like

$A_1 \rightarrow A_2 \ldots A_j \rightarrow \alpha$, where $A_j \rightarrow \alpha$ is a non unit production.

Each variable A_i to A_j will derive α.

Example 5.4.7 : Eliminate unit productions from

$$P = \{ \quad S \rightarrow ABA \mid BA \mid AA \mid AB \mid A \mid B$$
$$A \rightarrow aA \mid a$$
$$B \rightarrow bB \mid b$$
$$\}$$

Solution : Let G_1 with productions P_1 is an equivalent grammar without unit productions.

Step 1 : Add all non-unit productions of the given grammar to P_1.

$$P_1 = \left\{ \begin{array}{l} S \rightarrow ABA \mid BA \mid AA \mid AB \\ A \rightarrow aA \mid a \\ B \rightarrow bB \mid b \end{array} \right\}$$

Step 2 : Locate every pair of variables (A_i, A_j) such that $A_i \overset{*}{\underset{G}{\Rightarrow}} A_j$

1. (S, A) [Due to unit production $S \rightarrow A$]
2. (S, B) [Due to unit production $S \rightarrow B$]

Step 3 : Unit production $S \rightarrow A$ can be removed by expanding A.

A derives aA | a.

Unit production $S \rightarrow B$ can be removed by expanding $S \rightarrow B$. B derives bB | b.

Set of final productions for G_1 are given below :

	Productions
From step 1	$S \rightarrow ABA \mid BA \mid AA \mid AB$
	$A \rightarrow aA \mid a$
	$B \rightarrow bB \mid b$

Scanned by CamScanner

	Productions
Pair (S, A)	$S \rightarrow aA \mid a$
Pair (S, B)	$S \rightarrow bB \mid b$

Thus,

$$P_1 = \left\{ \begin{array}{l} S \rightarrow ABA \mid BA \mid AA \mid AB \mid aA \mid a \mid bB \mid b \\ A \rightarrow aA \mid a \\ B \rightarrow bB \mid b \end{array} \right\}$$

Example 5.4.8 : Eliminate unit productions from the grammar

$$P = \{ \quad E \rightarrow E+T \mid T,\ T \rightarrow T*F \mid F,$$
$$F \rightarrow (E) \mid I$$
$$I \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1$$
$$\}$$

Solution :

Let the grammar G_1, with productions P_1 is an equivalent grammar without unit productions.

Step 1 : Add all non-unit productions of the given grammar to P_1

$$P_1 = \left\{ \begin{array}{l} E \rightarrow E+T \\ T \rightarrow T*F \\ F \rightarrow (E) \\ I \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \end{array} \right\}$$

Step 2 : Locate every pair of variables (A_i, A_j) such that $A_i \overset{*}{\underset{G}{\Rightarrow}} A_j$

1. (E, T) [Due to unit production $E \rightarrow T$]
2. (T, F) [Due to unit production $T \rightarrow F$]
3. (F, I) [Due to unit production $F \rightarrow I$]
4. (E, F) [Due to (E, T) and (T, F)]
5. (E, I) [Due to (E, T), (T, F) and (F, I)]
6. (T, I) [Due to (T, F) and (F, I)]

Step 3 : Unit productions are removed as shown below :

	Productions
From step 1	$E \rightarrow E+T$
	$T \rightarrow T*F$
	$F \rightarrow (E)$
	$I \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1$

Scanned by CamScanner

	Productions
Pair (E, T)	$E \rightarrow T*F$
Pair (T, F)	$T \rightarrow (E)$
Pair (F, I)	$F \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1$
Pair (E, F)	$E \rightarrow (E)$
Pair (E, I)	$E \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1$
Pair (T, I)	$T \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1$

Thus, the final grammar is given by

$$P_1 = \left\{ \begin{array}{l} E \rightarrow E + T \mid T*F \mid (E) \mid a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \\ T \rightarrow T*F \mid (E) \mid a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \\ F \rightarrow (E) \mid a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \\ I \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \end{array} \right\}$$

Example 5.4.9 : Simplify the following grammar

$S \rightarrow ASB \mid \varepsilon$

$A \rightarrow aAS \mid a$

$B \rightarrow SbS \mid A \mid bb$

Solution : Following sequence should be followed to simplify a grammar.

1. Eliminate to ε - productions from G and obtain G_1.
2. Eliminate unit productions from G_1 and obtain G_2.
3. Eliminate useless symbols from G_2 and obtain G_3.

Step 1 : Eliminate ε - productions from G and obtain G_1.

The set of nullable symbols = {S} as $S \rightarrow \varepsilon$

Grammar with ε production (G)	Grammar after taking effect of nullable symbols (G_1)
$S \rightarrow ASB$	$S \rightarrow ASB$ $S \rightarrow AB$] By making symbol S as null.
$S \rightarrow \varepsilon$ $A \rightarrow aAS$	$S \rightarrow \varepsilon$ $A \rightarrow aAS$ $A \rightarrow aA$] By making symbol S as null
$A \rightarrow a$ $B \rightarrow SbS$	$A \rightarrow a$ $B \rightarrow SbS$ $B \rightarrow Sb$] By making one or more S null $B \rightarrow bS$] $B \rightarrow b$
$B \rightarrow A$ $B \rightarrow bb$	$B \rightarrow A$ $B \rightarrow bb$

Scanned by CamScanner

Final G_1, after deletion of ε-productions is given below :

$$G_1 = \left\{ \begin{array}{l} S \rightarrow ASB \mid AB \\ A \rightarrow aAS \mid aA \mid a \\ B \rightarrow SbS \mid Sb \mid bS \mid b \mid A \mid bb \end{array} \right\}$$

Step 2 : Eliminate unit productions from G_1 and obtain G_2.

The grammar G_1 contains a unit production $B \rightarrow A$.

A derives aAS | aA | b

$\therefore$ $\boxed{B \rightarrow A}$ $\rightarrow$ can be written as $\rightarrow$ $\boxed{B \rightarrow aAS \mid aA \mid b}$

G_2, after elimination of unit productions from G_1 is given by :

$$G_2 = \left\{ \begin{array}{l} S \rightarrow ASB \mid AB \\ A \rightarrow aAS \mid aA \mid a \\ B \rightarrow SbS \mid Sb \mid bS \mid b \mid bb \mid aAS \mid aA \mid b \mid a \end{array} \right\}$$

Step 3 : Eliminate useless symbols from G_2 and obtain G_3.

All the three variables S, A and B are both reachable and generating. Hence the grammar G_2 does not have useless symbols.

$$\therefore G_3 = G_2 = \left\{ \begin{array}{l} S \rightarrow ASB \mid AB \\ A \rightarrow aAS \mid aA \mid a \\ B \rightarrow SbS \mid Sb \mid bS \mid b \mid bb \mid aAS \mid aA \mid b \end{array} \right\}$$

Example 5.4.10 : Simplify the following grammar :

$S \rightarrow 0A0 \mid 1B1 \mid BB$

$A \rightarrow C$

$B \rightarrow S \mid A$

$C \rightarrow S \mid \varepsilon$

Solution :

Step 1 : Eliminate ε-production from G (given grammar) and obtain G_1. Set of nullable symbols are as shown in Fig. Ex. 5.4.10 :

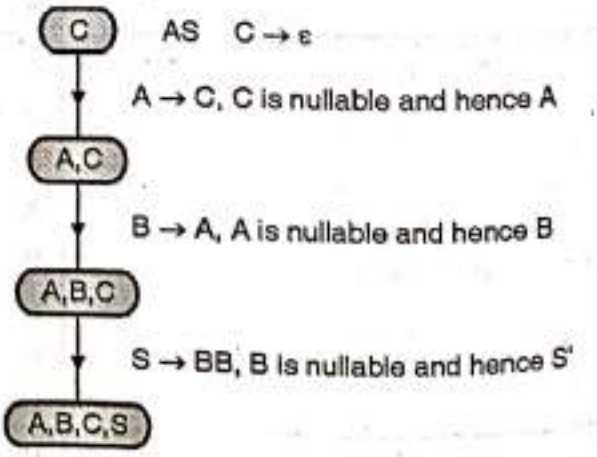


Fig. Ex. 5.4.10

Scanned by CamScanner

The grammar after removal of null production is given below :

$$\left\{\begin{array}{l} S \rightarrow 0A0 \mid 00 \mid 1B1 \mid 11 \mid B \mid BB \\ A \rightarrow C\,,\ B \rightarrow S \mid A\,,\ C \rightarrow S \end{array}\right\} \quad - \text{Grammar } G_1$$

Step 2 : Following unit productions are there in G_1.

$S \rightarrow B,\ A \rightarrow C,\ B \rightarrow S,\ B \rightarrow A,\ C \rightarrow S$

There is a chain.

$B \rightarrow A \rightarrow C \rightarrow S \rightarrow B$

and $S \rightarrow B \rightarrow S$

These chains are cyclic in nature of each is terminating in S.

The grammar without unit productions is given below :

$$\left\{\begin{array}{l} S \rightarrow 0A0 \mid 00 \mid 1B1 \mid 11 \mid BB \\ A \rightarrow 0A0 \mid 00 \mid 1B1 \mid 11 \mid BB \\ B \rightarrow 0A0 \mid 00 \mid 1B1 \mid 11 \mid BB \\ C \rightarrow 0A0 \mid 00 \mid 1B1 \mid 11 \mid BB \end{array}\right\}$$

Step 3 : The symbol C is not reachable and hence it can be deleted. The set of final productions is given below :

$S \rightarrow 0A0 \mid 00 \mid 1B1 \mid 11 \mid BB$

$A \rightarrow 0A0 \mid 00 \mid 1B1 \mid 11 \mid BB$

$B \rightarrow 0A0 \mid 00 \mid 1B1 \mid 11 \mid BB$

Example 5.4.11 : Simplify the following grammar

$S \rightarrow Ab$

$A \rightarrow a$

$B \rightarrow C \mid b$

$C \rightarrow D$

$D \rightarrow E$

$E \rightarrow a$

Solution : F

Following sequence should be followed to simplify a grammar :

1. Eliminate ε-productions from G and obtain G_1.
2. Eliminate unit productions from G_1 and obtain G_2.
3. Eliminate useless symbols from G_2 and obtain G_3.

Step 1 : Eliminate ε productions from G and obtain G_1.

Grammar does not have ε-productions.

$\therefore\ G_1 = G$

Scanned by CamScanner

Step 2 : Eliminate unit productions from G_1 and obtain G_2.

Step 2.1 : Add all non unit productions of the given grammar G_1 to productions P_2 of G_2.

$$P_2 = \begin{Bmatrix} S \to Ab, A \to a \\ E \to a \end{Bmatrix}$$

Step 2.2 : Locate every pair of variables (A_i, A_j) such that $A_i \overset{*}{\underset{G_1}{\Rightarrow}} A_j$

1.	(B, C)	[Due to unit production B → C]
2.	(C, D)	[Due to unit production C → D]
3.	(D, E)	[Due to unit production D → E]
4.	(B, D)	[Due to (B, C) and (C, D)]
5.	(B, E)	[Due to (B, C), (C, D) and (B, E)]
6.	(C, E)	[Due to (C, D) and (D, E)]

Step 2.3 : Unit productions are removed through expansion.

From Step 2.1 $S \to Ab, A \to a, E \to a$

Pair (B, C)	B → a	[there is a chain B → C → D → E → a]
Pair (C, D)	C → a	[there is a chain B → C → D → E → a]
Pair (D, E)	D → a	[there is a chain B → C → D → E → a]
Pair (B, D)	B → a	[there is a chain B → C → D → E → a]
Pair (B, E)	B → a	[there is a chain B → C → D → E → a]
Pair (C, E)	C → a	[there is a chain B → C → D → E → a]

Thus the productions after elimination of unit productions is

$$P_2 = \begin{Bmatrix} S \to Ab \\ A \to a \\ E \to a \\ B \to a \\ C \to a \\ D \to a \end{Bmatrix}$$

Step 3 : Elimination of useless symbols :

- Every symbol, in the given grammar is generating.
- Reachable symbols can be located by drawing a dependency graph.

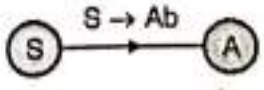


Fig. Ex. 5.4.11 : Dependency graph

Scanned by CamScanner

Two symbols {S, A} are reachable. Other symbols {B, C, D, E} are non-reachable.

Final grammar G_3 with productions P_3 is obtained by deleting useless symbols.

$$P_3 = \begin{Bmatrix} S \rightarrow Ab \\ A \rightarrow a \end{Bmatrix}$$

Example 5.4.12 : Find a reduced grammar equivalent to

$S \rightarrow aC \mid SB$

$A \rightarrow bSCa$

$B \rightarrow aSB \mid bBC$

$C \rightarrow aBC \mid ad$

Solution :

Set of generating symbols can be found as shown in Fig. Ex. 5.4.12 :

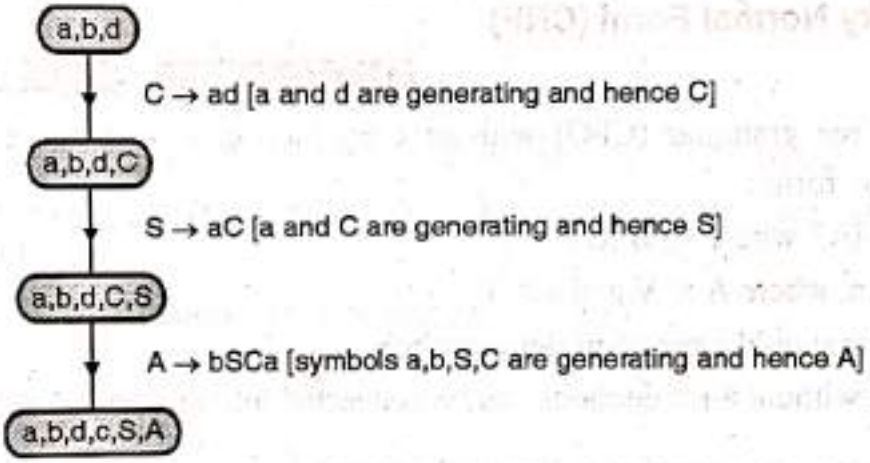


Fig. Ex. 5.4.12

$\therefore$ Non generating symbols = {B}

Thus the productions after elimination of non-generating symbols is :

$$P_1 = \begin{Bmatrix} S \rightarrow aC \\ A \rightarrow bSCa \\ C \rightarrow ad \end{Bmatrix}$$

The set of reachable symbols can be found by drawing a dependency graph.

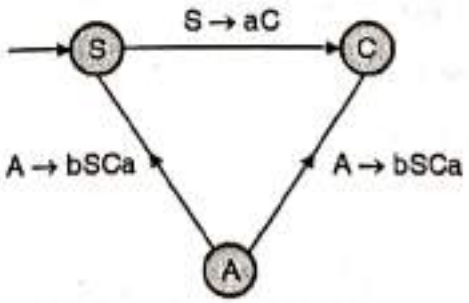


Fig. Ex. 5.4.12(a) : Dependency graph

Scanned by CamScanner

The symbol {A} is non-reachable.

Thus the final set of productions after elimination of non-reachable symbols is :

$$P_2 = \begin{Bmatrix} S \rightarrow aC \\ C \rightarrow ad \end{Bmatrix}$$

5.5 Normal Forms for CFG

Productions in G, satisfying certain restrictions are said to be in normal form.

There are two normal forms for CFG.

1. Chomsky Normal Form (CNF)
2. Greibach Normal Form (GNF)

5.5.1 Chomsky Normal Form (CNF)

MU - Dec. 2009, Dec. 2010, May 2013, May 2014

A context free grammar (CFG) without ε-production is said to be in CNF if every production is of the form :

1. $A \rightarrow BC$, where $A, B, C \in V$.
2. $A \rightarrow a$, where $A \in V$ and $a \in T$.

The grammar should have no useless symbols.

- Every CFG without ε-productions can be converted into an equivalent CNF form.

5.5.1.1 Algorithm for CFG to CNF Conversion

1. Eliminate ε-productions, unit productions and useless symbols from the grammar.
2. Every variable deriving a string of length 2 or more should consist only of variables.
   i.e. every production of the form $A \rightarrow \alpha$ with $| \alpha | \geq 2$, α should consists only of variables.

   Example : Consider a production $A \rightarrow V_1V_2aV_3bV_4$.

   Terminal symbols a and b can be removed by rewriting the production

   $A \rightarrow V_1V_2aV_3bV_4$ as

   $A \rightarrow V_1V_2C_aV_3C_bV_4$

   And adding two productions

   $C_a \rightarrow a$

   and $C_b \rightarrow b$
3. Every production deriving 3 or more variables ($A \rightarrow \alpha$ with $| \alpha | \geq 3$) can be broken down into a cascade of productions with each deriving a string of two variables.

Scanned by CamScanner

Example : Consider a production $A \rightarrow X_1X_2 \ldots X_n$ where $n \geq 3$ and as X_i's are variables.

The production $A \rightarrow X_1X_2 \ldots X_n$ should be broken down as given below :

$$A \rightarrow X_1C_1$$
$$C_1 \rightarrow X_2C_2$$
$$C_2 \rightarrow X_3C_3$$
$$\vdots$$
$$C_{n-2} \rightarrow X_{n-1}X_n$$

each with two variables on the right.

Example 5.5.1 : Find the CNF equivalent to $S \rightarrow aAbB$, $A \rightarrow aA$, $B \rightarrow bB \mid b$.

Solution :

Step 1 : Simplification of grammar.

The grammar is already in a simple form without

1. ε-productions. 2. Unit-productions. 3. Useless symbols.

Step 2 : Every symbol in α, in a production of the form $A \rightarrow \alpha$ where $|\alpha| \geq 2$ should be a variable :

This can be done by adding two productions.

$$C_a \rightarrow a$$

and $$C_b \rightarrow b$$

The set of productions after the above changes is :

$$P_2 = \left\{ \begin{array}{l} S \rightarrow C_aAC_bB \\ A \rightarrow C_aA \\ B \rightarrow C_bB \mid b \\ C_a \rightarrow a \\ C_b \rightarrow b \end{array} \right\}$$

Step 3 : Convert to CNF

$$\left. \begin{array}{l} S \rightarrow C_aC_1 \\ C_1 \rightarrow AC_2 \\ C_2 \rightarrow C_bB \end{array} \right| \quad \text{Corresponding to } S \rightarrow C_aAC_bB$$

$$A \rightarrow C_aA$$
$$B \rightarrow C_bB \mid b$$
$$C_a \rightarrow a$$
$$C_b \rightarrow b$$

$$\therefore P_3 = \left\{ \begin{array}{l} S \rightarrow C_aC_1, C_1 \rightarrow AC_2, C_2 \rightarrow C_bB \\ A \rightarrow C_aA, B \rightarrow C_bB \mid b, C_a \rightarrow a \\ C_b \rightarrow b \end{array} \right\}$$

Scanned by CamScanner

Example 5.5.2 : Convert the grammar given below to its equivalent CNF :
$S \rightarrow PQP$
$P \rightarrow 0P \mid \varepsilon$
$Q \rightarrow 1Q \mid \varepsilon$
MU - Dec. 2015

Solution :

Step 1 : Simplify the grammar

Step 1.1 : Eliminate ε productions Nullable symbols = {P, Q, S}

P and Q are nullable as $P \rightarrow \varepsilon$ and $Q \rightarrow \varepsilon$. S is also nullable as $S \rightarrow PQP$ with every symbol on the right as nullable.

Eliminating ε-productions, we get a set of productions P_1

$$P_1 = \left\{ \begin{array}{l} S \rightarrow PQP \mid PQ \mid QP \mid P \mid Q \\ P \rightarrow 0P \mid 0 \\ Q \rightarrow 1Q \mid 1 \end{array} \right\}$$

Step 1.2 : Eliminate unit productions :

There are two unit productions $S \rightarrow P$ and $S \rightarrow Q$.

Eliminating unit productions from P_1, we get a set of productions P_2

$$P_2 = \left\{ \begin{array}{l} S \rightarrow PQP \mid PQ \mid QP \mid 0P \mid 0 \mid 1Q \mid 1 \mid PP \\ P \rightarrow 0P \mid 0 \\ Q \rightarrow 1Q \mid 1 \end{array} \right\}$$

Step 2 : Every symbol in α, in a production of the form $A \rightarrow \alpha$ with $|\alpha| \geq 2$ should be a variable.

This can be done by adding two productions

$C_0 \rightarrow 0$

$C_1 \rightarrow 1$

The set of productions after the changes is given by P_3.

$$P_3 = \left\{ \begin{array}{l} S \rightarrow PQP \mid PQ \mid QP \mid PP \mid C_0P \mid 0 \mid C_1Q \mid 1 \\ P \rightarrow C_0P \mid 0 \\ Q \rightarrow C_1Q \mid 1 \\ C_0 \rightarrow 0 \\ C_1 \rightarrow 1 \end{array} \right\}$$

Step 3 : Convert to CNF :

Original production	Equivalent productions in CNF
$S \rightarrow PQP$	$S \rightarrow PC_2$ $C_2 \rightarrow QP$
$S \rightarrow PQ$	$S \rightarrow PQ$
$S \rightarrow QP$	$S \rightarrow QP$
$S \rightarrow PP$	$S \rightarrow PP$
$S \rightarrow C_0P$	$S \rightarrow C_0P$

Scanned by CamScanner

Original production	Equivalent productions in CNF
$S \rightarrow 0$	$S \rightarrow 0$
$S \rightarrow C_1Q$	$S \rightarrow C_1Q$
$S \rightarrow 1$	$S \rightarrow 1$
$P \rightarrow C_0P \mid 0$	$P \rightarrow C_0P \mid 0$
$Q \rightarrow C_1Q \mid 1$	$Q \rightarrow C_1Q \mid 1$
$C_0 \rightarrow 0$	$C_0 \rightarrow 0$
$C_1 \rightarrow 1$	$C_1 \rightarrow 1$

∴ The set of final productions in CNF is :

$$P_3 = \left\{ \begin{array}{l} S \rightarrow PC_2 \mid PQ \mid QP \mid PP \mid C_0P \mid 0 \mid C_1Q \mid 1 \\ C_2 \rightarrow QP \\ P \rightarrow C_0P \mid 0 \\ Q \rightarrow C_1Q \mid 1 \\ C_0 \rightarrow 0 \\ C_1 \rightarrow 1 \end{array} \right\}$$

Example 5.5.3 : Check whether the given grammar is in CNF
$S \rightarrow bA \mid aB$
$A \rightarrow bAA \mid aS \mid a$
$B \rightarrow aBB \mid bS \mid b$
If it is not in CNF, find the equivalent CNF. **MU - Dec. 2010**

Solution : The given grammar is not in CNF. Following productions are not allowed in CNF.

$S \rightarrow bA \mid aB$

$A \rightarrow bAA \mid aS$

$B \rightarrow aBB \mid bS$

Finding an equivalent CNF :

Step 1 : Simplification of grammar – already simplified

Step 2 : Every symbol in α, in production of the form $A \rightarrow \alpha$ where $| \alpha | \geq 2$ should be a variable.

This can be done by adding two productions.

$C_a \rightarrow a$

and $C_b \rightarrow b$

The set of productions after the above changes is

$$\left\{ \begin{array}{l} S \rightarrow C_bA \mid C_aB \\ A \rightarrow C_bAA \mid C_aS \mid a \\ B \rightarrow C_bBB \mid C_bS \mid b \\ C_a \rightarrow a\,,\ C_b \rightarrow b \end{array} \right\}$$

Scanned by CamScanner

Step 3 : Finding an equivalent CNF :

Original production	Equivalent productions in CNF
$S \to C_bA \mid C_aB$	$S \to C_bA \mid C_aB$
$A \to C_bAA$	$A \to C_bC_1$ $C_1 \to AA$
$A \to C_aS \mid a$	$A \to C_aS \mid a$
$B \to C_bBB$	$B \to C_bC_2$ $C_2 \to BB$
$B \to C_bS \mid b$	$B \to C_bS \mid b$
$C_a \to a$	$C_a \to a$
$C_b \to b$	$C_b \to b$

$\therefore$ The set of final productions in CNF is :

$$\left\{\begin{array}{l} S \to C_bA \mid C_aB \\ A \to C_bC_1 \mid C_aS \mid a \\ C_1 \to AA \\ B \to C_bC_2 \mid C_bS \mid b \\ C_2 \to BB \\ C_a \to a \\ C_b \to b \end{array}\right\}$$

Example 5.5.4 : Design a CNF grammar for the set of strings of balanced parenthesis.

Solution :

A context free grammar for balanced parentheses is given by

$$V = \{\{S\}, \{(,)\}, \{S \to (S), S \to SS, S \to \varepsilon\}, S\}$$

where the set of productions

$$P = \left\{\begin{array}{l} S \to (S) \\ S \to SS \\ S \to \varepsilon \end{array}\right\}$$

An equivalent set of productions without ε-productions can be written as

$$P_1 = \left\{\begin{array}{l} S \to (S) \mid (\ ) \\ S \to SS \end{array}\right\}$$

Conversion of P_1 in CNF can be done in two steps.

Step 1 : Addition of two productions $\boxed{C_1 \to (}$ and $\boxed{C_2 \to)}$ will change the set of productions to :

$$P_2 = \left\{\begin{array}{l} S \to C_1SC_2 \mid C_1C_2 \\ S \to SS \, , C_1 \to (\, , C_2 \to) \end{array}\right\}$$

Scanned by CamScanner

Step 2 : Finding an equivalent CNF :

Production in CFG	Equivalent productions in CNF
$S \rightarrow C_1SC_2$	$S \rightarrow C_1C_3$ $C_3 \rightarrow SC_2$
$S \rightarrow C_1C_2$	$S \rightarrow C_1C_2$
$S \rightarrow SS$	$S \rightarrow SS$
$C_1 \rightarrow ($	$C_1 \rightarrow ($
$C_2 \rightarrow)$	$C_2 \rightarrow)$

∴ The set of final productions in CNF is

$$\left\{\begin{array}{l} S \rightarrow C_1C_3 \mid C_1C_2 \mid SS \\ S \rightarrow SC_2 \\ C_1 \rightarrow (\\ C_2 \rightarrow) \end{array}\right\}$$

Example 5.5.5 : Convert the following grammar to CNF.

S→Aba, S→aab, B→Ac

Solution :

Step 1 : Symbol B is non-reachable. The symbol can be deleted. The grammar after simplification is :

S→Aba | aab

Step 2 : Every symbol is α, in a production of the from A →α with |α| ≥ 2 should be a variable :

This can be done by adding two productions

$$Ca \rightarrow a$$

and $$Cb \rightarrow b$$

The set of productions after above changes is :

$$S \rightarrow AC_bC_a \mid C_aC_aC_b$$

Step 3 : Convert to CNF

$$S \rightarrow AC_1$$
$$C_1 \rightarrow C_bC_a$$
$$S \rightarrow C_aC_2$$
$$C_2 \rightarrow C_aC_b$$

Example 5.5.6 : Convert the following grammar to CNF

S → AACD
A → aAb | ε
C → aC | a
A → aDa | bDb | ε

MU - May 2008

Scanned by CamScanner

Solution : First of all, the grammar must be simplified.

Step 1 : Removing null productions.

Nullable set = {A}

Null productions are removed with the resulting set of production as :

$S \rightarrow AACD|ACD|CD$

$A \rightarrow aAb|ab$

$C \rightarrow aCa$

$A \rightarrow aDa|bDb$

Step 2 : Removing non-generating symbol

Symbol S and D are non-generating.

Since, the starting symbol itself is non-generating, it is an **invalid grammar.**

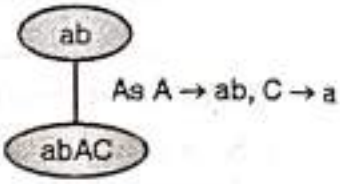


Fig. Ex. 5.5.6

Example 5.5.7 : Given a CFG G, find G' in CNF generating L (G) - ε

$S \rightarrow ASB \mid \varepsilon$

$A \rightarrow AaS \mid a$

$B \rightarrow SbS \mid A \mid bb$

MU - May 2006, May 2009, May 2010, Dec. 2011

Solution :

Step 1 : Simplification of grammar

Symbol S is nullable.

After removing ε-productions, the set of productions is given by

$S \rightarrow ASB \mid AB$

$A \rightarrow AaS \mid Aa \mid a$

$B \rightarrow SbS \mid Sb \mid bS \mid b \mid A \mid bb$

Unit production $B \rightarrow A$ is removed, the resulting set of productions is given by

$S \rightarrow ASB \mid AB$

$A \rightarrow AaS \mid Aa \mid a$

$B \rightarrow SbS \mid Sb \mid bS \mid b \mid AaS \mid Aa \mid a \mid bb$

Step 2 : Every symbol in α, in productions of the form $A \rightarrow \alpha$ where $|\alpha| \geq 2$ should be a variable.

This can be done by adding two productions :

$C_a \rightarrow a$

and $C_b \rightarrow b$

The set of productions after the above changes is

$S \rightarrow ASB \mid AB$

$A \rightarrow AC_aS \mid AC_a \mid a$

$B \rightarrow SC_bS \mid SC_b \mid C_bS \mid b \mid AC_aS \mid AC_a \mid a \mid C_bC_b$

$C_a \rightarrow a$

$C_b \rightarrow b$

Scanned by CamScanner

Step 3 : Finding an equivalent CNF

Original production	Equivalent productions in CNF
$S \rightarrow ASB$	$S \rightarrow AC_1$ $C_1 \rightarrow SB$
$S \rightarrow AB$	$S \rightarrow AB$
$A \rightarrow AC_aS$	$A \rightarrow AC_2$ $C_2 \rightarrow C_aS$
$A \rightarrow AC_a$	$A \rightarrow AC_a$
$A \rightarrow a$	$A \rightarrow a$
$B \rightarrow SC_bS$	$B \rightarrow SC_3$ $C_3 \rightarrow C_bS$
$B \rightarrow SC_b \mid C_bS \mid b$	$S \rightarrow SC_b \mid C_bS \mid b$
$B \rightarrow AC_aS$	$B \rightarrow AC_2$
$B \rightarrow AC_a \mid a \mid C_bC_b$	$B \rightarrow AC_a \mid a \mid C_bC_b$
$C_a \rightarrow a$	$C_a \rightarrow a$
$C_b \rightarrow b$	$C_b \rightarrow b$

Example 5.5.8 : Convert the given grammar to CNF

$S \rightarrow aSB|aA$

$A \rightarrow Aa|Sala$

Solution :

Step 1 : Every symbol in α, in production of the form $A \rightarrow \alpha$ where $|\alpha| \geq 2$ should be a variable.

This can be done by adding the production

$$C_a \rightarrow a$$

The set of productions after the above changes is :

$$\left\{\begin{array}{l} S \rightarrow C_aSB|C_aA \\ A \rightarrow AC_a|SC_a|a \\ C_a \rightarrow a \end{array}\right\}$$

Step 2 : Finding an equivalent CNF

Original production	Equivalent productions in CNF
$S \rightarrow C_aSB$	$S \rightarrow C_aC_1$ $C_1 \rightarrow SB$
$S \rightarrow C_aA$	$S \rightarrow C_aA$
$A \rightarrow AC_a \mid SC_a \mid a$	$A \rightarrow AC_a \mid SC_a \mid a$
$C_a \rightarrow a$	$C_a \rightarrow a$

Scanned by CamScanner

Example 5.5.9 : Convert the following grammars to CNF
$A \rightarrow aBb \mid bBa$
$B \rightarrow aB \mid bB \mid \varepsilon$
MU - May 2007, May 2011

Solution :

Step 1 : Removing ε-production.

The symbol B is nullable. The null production is removed. The resulting set of productions is,

$A \rightarrow aBb \mid bBa \mid ab \mid ba$
$B \rightarrow aB \mid bB \mid a \mid b$

Step 2 : Every symbol in α, in productions of the form $A \rightarrow \alpha$ where $| \alpha | \geq 2$ should be a variable.

This can be done by adding two productions :

$C_a \rightarrow a$
$C_b \rightarrow b$

The set of productions after above substitution is :

$A \rightarrow C_a B C_b \mid C_b B C_a \mid C_a C_b \mid C_b C_a$
$B \rightarrow C_a B \mid C_b B \mid a \mid b$
$C_a \rightarrow a$
$C_b \rightarrow b$

Step 3 : Finding an equivalent CNF

Sr. No.	Original prosecution	Equivalent productions in CNF
1.	$A \rightarrow C_a B C_b$	$A \rightarrow C_a C_1, C_1 \rightarrow B C_b$
2.	$A \rightarrow C_b B C_a$	$A \rightarrow C_b C_2, C_2 \rightarrow B C_a$
3.	$A \rightarrow C_a C_b$	$A \rightarrow C_a C_b$
4.	$A \rightarrow C_b C_a$	$A \rightarrow C_b C_a$
5.	$B \rightarrow C_a B$	$B \rightarrow C_a B$
6.	$B \rightarrow C_b B$	$B \rightarrow C_b B$
7.	$B \rightarrow a$	$B \rightarrow a$
8.	$B \rightarrow b$	$B \rightarrow b$
9.	$C_a \rightarrow a$	$C_a \rightarrow a$
10.	$C_b \rightarrow b$	$C_b \rightarrow b$

Example 5.5.10 : Begin with the grammar :
$S \rightarrow ABC \mid BaB$
$A \rightarrow aA \mid BaC \mid aaa$
$B \rightarrow bBb \mid a \mid D$
$C \rightarrow CA \mid AC$
$D \rightarrow \epsilon$
(i) Eliminate ϵ Productions.
(ii) Eliminate any unit production in the resulting grammar.

Scanned by CamScanner

(iii) Eliminate any useless symbols in the resulting grammar.
(iv) Put the resulting grammar into Chomsky Normal Form.
MU - May 2012

Solution :

Step 1 : Eliminate $\in$-production

Nullable set = {D, B}

$\therefore$ The resulting set of productions without $\in$-production is :

$S \rightarrow ABC \mid AC \mid BaB \mid aB \mid Ba \mid a$

$A \rightarrow aA \mid BaC \mid aC \mid aaa$

$B \rightarrow bBb \mid bb \mid a$

$C \rightarrow CA \mid AC$

Step 2 : The grammar does not have unit productions

Step 3 : Removing useless symbols.

{C} is a non-generating symbol. It is eliminated and the equivalent grammar is written.

$S \rightarrow BaB \mid aB \mid Ba \mid a$

$A \rightarrow aA \mid aaa$

$B \rightarrow bBb \mid bb \mid a$

The symbol {A} is non-reachable. It is eliminated and the equivalent grammar is written.

$S \rightarrow BaB \mid aB \mid Ba \mid a$

$B \rightarrow bBb \mid bb \mid a$

Step 4 : Putting grammar in CNF

Replacing 'a' by X and 'b' by Y, we get

$S \rightarrow BXB \mid XB \mid BX \mid a$

$B \rightarrow YBY \mid YY \mid a$

$X \rightarrow a, Y \rightarrow b$

Now, the above set of productions can be written in CNF

$S \rightarrow BA_1 \mid XB \mid BX \mid a$

$A_1 \rightarrow XB$

$B \rightarrow YA_2 \mid YY \mid a$

$A_2 \rightarrow BY$

$X \rightarrow a$

$Y \rightarrow b$

Example 5.5.10(A) : Convert the following Grammar to CNF form :
$S \rightarrow ABA$
$A \rightarrow aA \mid bA \mid \in$
$B \rightarrow bB \mid aA \mid \in$
PU - May 2014

Scanned by CamScanner

Solution :

1. The non-terminals {S, A, B} are nullable. Null productions are removed. The resulting grammar is :

   $S \rightarrow ABA \mid BA \mid AB \mid AA \mid A \mid B$

   $A \rightarrow aA \mid bA \mid a \mid b$

   $B \rightarrow bB \mid aA \mid b \mid a$

2. Removing unit productions, we get

   $S \rightarrow ABA \mid BA \mid AB \mid AA \mid aA \mid bA \mid a \mid b \mid bB \mid aA$

   $A \rightarrow aA \mid bA \mid a \mid b$

   $B \rightarrow bB \mid aA \mid b \mid a$

3. Every symbol in α, in production of the form $A \rightarrow \alpha$ where $| \alpha | \geq 2$ should be a variable.

   This can be done by adding two productions.

   $C_a \rightarrow a$

   $C_b \rightarrow b$

   The set of productions after the above changes is :

   $S \rightarrow ABA \mid BA \mid AB \mid AA \mid C_a A \mid C_b A \mid a \mid b \mid C_b B \mid C_a A$

   $A \rightarrow C_a A \mid C_b A \mid a \mid b$

   $B \rightarrow C_b B \mid C_a A \mid b \mid a$

   $C_a \rightarrow a, C_b \rightarrow b$

4. Finding an equivalent CNF.

Original production	Equivalent productions in CNF
$S \rightarrow ABA$	$S \rightarrow A C_1, C_1 \rightarrow BA$
$S \rightarrow BA \mid AB \mid AA \mid C_a A \mid C_b A \mid a \mid b \mid C_b B \mid C_a A$	$S \rightarrow BA \mid AB \mid AA \mid C_a A \mid C_b A \mid a \mid b \mid C_b B \mid C_a A$
$A \rightarrow C_a A \mid C_b A \mid a \mid b$	$A \rightarrow C_a A \mid C_b A \mid a \mid b$
$B \rightarrow C_b B \mid C_a A \mid b \mid a$	$B \rightarrow C_b B \mid C_a A \mid b \mid a$
$C_a \rightarrow a$	$C_a \rightarrow a$
$C_b \rightarrow b$	$C_b \rightarrow b$

5.5.2 Greibach Normal Form (GNF)

MU - Dec. 2005, May 2007, Dec. 2008, Dec. 2009, May 2013, May 2014, May 2017

A context free grammar G = (V, T, P, S) is said to be in GNF if every production is of the form :

$$A \rightarrow a\alpha,$$

Where, $a \in T$ is a terminal and α is a string of zero or more variables.

Scanned by CamScanner

The language L (G) should be without ε.

- Right hand side of each production should start with a terminal followed by a string of non-terminals of length zero or more.

5.5.2.1 Removing Left Recursion

Elimination of left recursion is an important step in algorithm used in conversion of a CFG into GNF form.

Left recursive grammar :

A production of the form $A \rightarrow A\alpha$ is called left recursive as the left hand side variable appears as the first symbol on the right hand side.

Language generated by left recursive grammar :

Let us consider a CFG containing productions of the form

$A \rightarrow A\alpha$ ... [Left recursive]

$A \rightarrow \beta$ ... [For termination of recursion]

The language generated by above production is :

$A \rightarrow A\alpha$ [From production $A \rightarrow A\alpha$]

$\rightarrow A\alpha\alpha$ [From production $A \rightarrow A\alpha$]

$\rightarrow A\alpha\alpha\alpha$ [From production $A \rightarrow A\alpha$]

: :

$\rightarrow A\alpha^n$ [From production $A \rightarrow A\alpha$]

$\rightarrow \beta\alpha^n$ [From production $A \rightarrow \beta$]

Right recursive grammar for $\beta\alpha^n$:

A right recursive grammar for $\beta\alpha^n$ can be written as :

$A \rightarrow \beta B \mid \beta$ [where B generates a string α^n, production $A \rightarrow \beta$ is for termination of recursion]

$B \rightarrow \alpha B \mid \alpha$

Thus a left recursive grammar

$$\boxed{A \rightarrow A\alpha \mid \beta}$$

can be written using a right recursive grammar as :

$$\boxed{\begin{array}{l} A \rightarrow \beta B \mid \beta \\ B \rightarrow \alpha B \mid \alpha \end{array}}$$ Right recursive grammar

Example : A number of examples are given below for removing left-recursion.

Scanned by CamScanner

Grammar with left recursion	Language generated by grammar	Grammar without left recursion
1. $A \rightarrow Aa \mid b$	$\{b, ba, baa, \ldots ba^n\}$	$A \rightarrow b \mid bB$ $B \rightarrow aB \mid a$
2. $A \rightarrow Aa \mid b \mid c$	$\{b, ba, baa, \ldots ba^n, c, ca, caa, \ldots ca^n\}$	$A \rightarrow b \mid c \mid bB \mid cB$ $B \rightarrow aB \mid a$
3. $A_1 \rightarrow A_1A_2A_3 \mid A_2A_3$	$\{A_2A_3, A_2A_3A_2A_3, \ldots, (A_2A_3)^n\}$	$A_1 \rightarrow A_2A_3 \mid A_2A_3B_1$ $B_1 \rightarrow A_2A_3B_1 \mid A_2A_3$
4. $A_1 \rightarrow A_1A_2A_3 \mid A_4A_1 \mid A_5A_3$	$\{A_4A_1, A_4A_1A_2A_3\ldots, A_4A_1(A_2A_3)^n, A_5A_3, A_5A_3A_2A_3, \ldots, A_5A_3(A_2A_3)^n\}$	$A_1 \rightarrow A_4A_1 \mid A_5A_3 \mid A_4A_1B_1 \mid A_5A_3B_1$ $B_1 \rightarrow A_2A_3B_1 \mid A_2A_3$
5. $S \rightarrow S10 \mid 0$	$\{0, 010, 01010, \ldots, 0(10)^n\}$	$S \rightarrow 0B \mid 0$ $B \rightarrow 10B \mid 10$

5.5.2.2 Algorithm for Conversion from CFG to GNF

1. Eliminate ε-productions, unit productions and useless symbols from the grammar.
2. In production of the form $A \rightarrow X_1X_2\ldots X_i\ldots X_n$, other than X_1, every other symbol should be a variable. X_1 could be a terminal.

   Example : Consider a production $A \rightarrow V_1V_2aV_3bV_4$

   Terminal symbols a and b can be removed by rewriting the production

   $A \rightarrow V_1V_2aV_3bV_4$ as

   $A \rightarrow V_1V_2C_aV_3C_bV_4$ and adding two productions.

   $C_a \rightarrow a$

   $C_b \rightarrow b$

   Thus, at the end of step 2 all productions must be of the forms.

(a)	$A \rightarrow \alpha$	(b)	$A \rightarrow a$
(c)	$A \rightarrow a\alpha$		

   Where, 'a' is a terminal and α is a string of non-terminals.
3. Rename variables as $A_1, A_2, A_3 \ldots A_n$ to create A-productions.

   Example : Consider a grammar given below

   $S \rightarrow aXSY \mid YSX \mid b$

Scanned by CamScanner

The variables S, X and Y can be renamed as A_1, A_2 and A_3 respectively. Then the productions become

$$A_1 \rightarrow aA_2A_1A_3 \mid A_3A_1A_2 \mid b \qquad \text{[A-productions]}$$

4. Modify the productions to ensure that if there is a production $A_i > A_j\ \alpha$ then i should be $\leq$ j. If there is a production $A_i \rightarrow A_j\alpha$ with i > j. then we must generate productions substituting for A_j.
5. Repeating step 4, several times will guarantee that for every production $A_i \rightarrow A_j\alpha$, $i \leq j$.
6. Remove left recursion from every production of the form $A_k \rightarrow A_k\alpha$. B-productions should be added to remove left recursion.
7. Modify A_i–productions to the form $A_i \rightarrow a\alpha$, where a is a terminal and α is a string of non-terminals.
8. Modify B_i–productions to the form $B_i \rightarrow a\alpha$, where a is a terminal and α is a string of non-terminals.

Example 5.5.11 : Construct a grammar in GNF which is equivalent to the grammar $S \rightarrow AA \mid a, A \rightarrow SS \mid b$. **MU - May 2008, Dec. 2011, Dec. 2016**

Solution :

Step 1 : Grammar is already in a simple form without :

1. ε-productions.
2. Unit productions.
3. Useless symbol.

We can proceed for renaming of variables, Variables S and A are renamed as A_1 and A_2 respectively. The set of productions after renaming becomes :

Productions after renaming
$A_1 \rightarrow A_2A_2$
$A_1 \rightarrow a$
$A_2 \rightarrow A_1A_1$
$A_2 \rightarrow b$

Step 2 : Every production of the form $A_i \rightarrow A_j\alpha$ with i > j must be modified to make $i \leq j$.

A_2 – production $A_2 \rightarrow A_1\ A_1$ should be modified.

$\Downarrow$

We must substitute $A_2A_2 \mid a$ for the first A_1. We should not touch the second A_1 of A_1A_1.

$$[A_2 \rightarrow A_1A_1] \Rightarrow \begin{bmatrix} A_2 \rightarrow A_2A_2A_1 \\ A_2 \rightarrow a\ A_1 \end{bmatrix}$$

Scanned by CamScanner

The resulting set of productions is :

$A_1 \rightarrow A_2A_2 \mid a$

$A_2 \rightarrow A_2A_2A_1 \mid aA_1 \mid b$

Step 3 : Removing left recursion :

The A_2 – productions $A_2 \rightarrow A_2A_2A_1 \mid aA_1 \mid b$ contains left recursion. Left recursion from A_2–production can be removed through introduction of B_2-production.

$A_2 \rightarrow aA_1B_2 \mid bB_2$

$B_2 \rightarrow A_2A_1B_2 \mid A_2A_1$

The resulting set of productions is :

$A_1 \rightarrow A_2A_2 \mid a$

$A_2 \rightarrow aA_1B_2 \mid aB_2 \mid aA_1 \mid b$

$B_2 \rightarrow A_2A_1B_2 \mid A_2A_1$

Step 4 : A_2 – productions are in GNF.

A_1 and B_2 productions can be converted to GNF with the help of A_2–productions.

$A_2 \rightarrow aA_1B_2 \mid bB_2 \mid aA_1 \mid b$ … in GNF

$A_1 \rightarrow A_2\ A_2$

⇓ Substitute $aA_1B_2 \mid bB_2 \mid aA_1 \mid b$ for first A_2

$\boxed{A_1 \rightarrow aA_1B_2A_2 \mid bB_2A_2 \mid aA_1A_2 \mid bA_2}$

$A_1 \rightarrow a$ … in GNF

Now, for B_2 – Production

$B_2 \rightarrow A_2A_1B_2$

⇓ Substitute $aA_1B_2 \mid bB_2 \mid aA_1 \mid b$ for the first A_2

$\boxed{B_2 \rightarrow aA_1B_2A_1B_2 \mid bB_2A_1B_2 \mid aA_1A_1B_2 \mid bA_1B_2}$

$B_2 \rightarrow A_2A_1$

⇓ Substitute $a\ A_1\ B_2 \mid b\ B_2 \mid a\ A_1 \mid b$ for the first A_2

$\boxed{B_2 \rightarrow a\ A_1B_2A_1 \mid bB_2A_1 \mid aA_1A_1 \mid bA_1}$

The final set of productions is :

$A_2 \rightarrow aA_1B_2 \mid bB_2 \mid aA_1 \mid b$

$A_1 \rightarrow aA_1B_2A_2 \mid bB_2A_2 \mid aA_1A_2 \mid bA_2 \mid a$

$B_2 \rightarrow aA_1B_2A_1B_2 \mid bB_2A_1B_2 \mid aA_1A_1B_2 \mid bA_1B_2 \mid$
$a\ A_1B_2A_1 \mid bB_2A_1 \mid aA_1A_1 \mid bA_1$

} A set of productions P

Scanned by CamScanner

where, Set of variables V = (A_1, A_2, B_2)

Set of terminals T = (a, b)

Start symbol = A_1

Set of productions P = Given above.

Example 5.5.12 : Find a grammar in GNF for the given CFG.

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid a$

Solution :

Step 1 : Grammar does not contain ε-productions and useless symbol.

Grammar has unit productions.

We first eliminate unit productions :

$E \rightarrow T$ and $T \rightarrow F$ are two unit productions with the chain $E \rightarrow T \rightarrow F$

Non unit productions are taken as it is :

$E \rightarrow E + T$

$T \rightarrow T * F$

$F \rightarrow (E) \mid a$

Productions for the following pairs are added :

$(E, T) \Rightarrow \{E \rightarrow T * F\}$

$(E, F) \Rightarrow \{E \rightarrow (E) \mid a\}$

$(T, F) \Rightarrow \{T \rightarrow (E) \mid a\}$

The resulting set of production is :

$E \rightarrow E + T \mid T * F \mid (E) \mid a$

$T \rightarrow T * F \mid (E) \mid a$

$F \rightarrow (E) \mid a$

Step 2 : Bringing every productions to the form $A \rightarrow a\alpha$, where α is a string of variables.

We can make the following substitutions : A for +, B for * and C for)

The resulting set of productions after the above substitutions is :

$E \rightarrow EAT \mid TBF \mid (EC \mid a$

$T \rightarrow TBF \mid (EC \mid a$

$F \rightarrow (EC \mid a$

$A \rightarrow +, B \rightarrow *, C \rightarrow)$

Scanned by CamScanner

Step 3 : Renaming of variables :

The variables E, A, T, B, F, C are renamed as A_1, A_2, A_3, A_4, A_5 , A_6.

The resultant set of productions after the above substitutions is :

$A_1 \rightarrow A_1A_2A_3 \mid A_3A_4A_5 \mid (A_1A_6 \mid a$

$A_3 \rightarrow A_3A_4A_5 \mid (A_1A_6 \mid a$

$A_5 \rightarrow (A_1A_6 \mid a, A_2 \rightarrow +, A_4 \rightarrow *, A_6 \rightarrow)$

Step 4 : Every productions of the form $A_i \rightarrow A_j \alpha$ with $i > j$ must be modified to make $i \leq j$.

This step is not required as the productions are in the required form.

Step 5 : Remove left recursion.

A_1 production and A_3 production have left recursion.

- Left recursion from A_1-production can be removed through introduction of B_1 – production.

$A_1 \rightarrow A_3A_4A_5B_1 \mid (A_1A_6B_1 \mid aB_1$

$B_1 \rightarrow A_2A_3B_1 \mid A_2A_3$

- Left recursion from A_3-production can be removed through introduction of B_3 – production

$A_3 \rightarrow (A_1A_6B_3 \mid aB_3$

$B_3 \rightarrow A_4A_5B_3 \mid A_4A_5$

The resulting set of productions is :

$A_1 \rightarrow A_3A_4A_5B_1 \mid (A_1A_6B_1 \mid aB_1 \mid A_3A_4A_5 \mid (A_1A_6 \mid a$

$B_1 \rightarrow A_2A_3B_1 \mid A_2A_3$

$A_3 \rightarrow (A_1A_6B_3 \mid aB_3 \mid (A_1A_6 \mid a$

$B_3 \rightarrow A_4A_5B_3 \mid A_4A_5$

$A_5 \rightarrow (A_1A_6 \mid a$

$A_2 \rightarrow +$

$A_4 \rightarrow *$

$A_6 \rightarrow)$

Scanned by CamScanner

Step 6 : Productions for A_2, A_3, A_4, A_5 and A_6 are in GNF. Productions for A_1, B_1 and B_2 can be converted to GNF.

$A_2 \rightarrow +$

$A_3 \rightarrow (A_1A_6B_3 \mid aB_3 \mid (A_1A_6 \mid a$ | Already in GNF

$A_4 \rightarrow *$

$A_5 \rightarrow (A_1A_6 \mid a$

$A_6 \rightarrow)$

$A_1 \rightarrow A_3A_4A_5B_1$

$\Downarrow$ Substituting $(A_1 A_6 B_3 \mid a B_3 \mid (A_1 A_6 \mid a$ for first A_3

$A_1 \rightarrow (A_1A_6B_3A_4A_5B_1 \mid aB_3A_4A_5B_1 \mid (A_1A_6A_4A_5B_1 \mid aA_4A_5B_1$

$A_1 \rightarrow (A_1A_6B_1 \mid aB_1$ ...in GNF

$A_1 \rightarrow A_3A_4A_5$

$\Downarrow$ Substituting $(A_1A_6B_3 \mid aB_3 \mid (A_1A_6 \mid a$ for the first A_3

$A_1 \rightarrow (A_1A_6B_3A_4A_5 \mid aB_3A_4A_5 \mid (A_1A_6A_4A_5 \mid aA_4A_5$

$A_1 \rightarrow (A_1 A_6 \mid a$ ...in GNF

$B_1 \rightarrow A_2A_3B_1 \mid A_2A_3$

Substituting + for A_2.

$B_1 \rightarrow +A_3B_1 \mid +A_3$

$B_3 \rightarrow A_4A_5B_3 \mid A_4A_5$

Substituting • for A_4

$B_3 \rightarrow \bullet A_5B_3 \mid \bullet A_5$

The final set of productions is :

$A_2 \rightarrow +$

$A_3 \rightarrow (A_1A_6B_3 \mid aB_3 \mid (A_1A_6 \mid a$

$A_4 \rightarrow *$

Scanned by CamScanner

$A_5 \rightarrow (A_1A_6 \mid a$

$A_6 \rightarrow)$

$A_1 \rightarrow (A_1A_6B_3A_4A_5B_1 \mid aB_3A_4A_5B_1 \mid (A_1A_6A_4A_5B_1 \mid aA_4A_5B_1 \mid$
$(A_1A_6B_1 \mid aB_1 \mid (A_1A_6B_3A_4A_5 \mid aB_3A_4A_5 \mid (A_1A_6A_4A_5 \mid aA_4A_5 \mid (A_1A_6 \mid a$

$B_1 \rightarrow +A_3B_1 \mid +A_3$

$B_3 \rightarrow *A_5B_3 \mid *A_5.$

Example 5.5.13 : Give the GNF for following CFG

$S \rightarrow AB$

$A \rightarrow BS \mid b$

$B \rightarrow SA \mid a$

Solution :

Step 1 : Renaming of variables :

The variables S, A and B are renamed as A_1, A_2 and A_3 respectively.

The resultant set of productions after above substitutions is :

$A_1 \rightarrow A_2A_3$

$A_2 \rightarrow A_3A_1 \mid b$

$A_3 \rightarrow A_1A_2 \mid a$

Step 2 : Every production of the form $A_i \rightarrow A_j \alpha$ with $i > j$. must be modified to make $i \leq j$.

A_3 – production $A_3 \rightarrow A_1A_2$ should be modified

$\Downarrow$ A_1 is changed as per $A_1 \rightarrow A_2A_3$

$A_2A_3A_2$

$\Downarrow$ A_2 is changed as per $A_2 \rightarrow A_3A_1 \mid b$

$A_3A_1A_3A_2 \mid bA_3A_2.$

The resulting set of productions is :

$A_1 \rightarrow A_2A_3$

$A_2 \rightarrow A_3A_1 \mid b$

$A_3 \rightarrow A_3A_1A_3A_2 \mid bA_3A_2 \mid a$

Step 3 : Remove left recursion

$A_3 \rightarrow$ Production has left recursion. Left recursion from A_3 – production can be removed through introduction of B_3 – Production

$A_3 \rightarrow bA_3A_2B_3 \mid aB_3$

$B_3 \rightarrow A_1A_3A_2B_3 \mid A_1A_3A_2$

The resulting set of production is

$A_1 \rightarrow A_2A_3$

Scanned by CamScanner

$A_2 \rightarrow A_3A_1 \mid b$

$A_3 \rightarrow bA_3A_2B_3 \mid aB_3 \mid bA_3A_2 \mid a$

$B_3 \rightarrow A_1A_3A_2B_3 \mid A_1A_3A_2$

Step 4 : A_3 – productions are already in GNF

A_2 – production can be converted to GNF with the help of A_3 – productions.

$A_2 \rightarrow A_3A_1 \mid b$

⇓ Substituting $bA_3A_2B_3 \mid aB_3 \mid bA_3A_2 \mid a$ for A_3.

$$A_2 \rightarrow bA_3A_2B_3A_1 \mid aB_3A_1 \mid bA_3A_2A_1 \mid aA_1 \mid b$$

A_1 – production can be converted to GNF with the help of A_2 – productions :

$A_1 \rightarrow A_2A_3$

⇓ Substituting $bA_3A_2B_3A_1 \mid aB_3A_1 \mid bA_3A_2A_1 \mid aA_1 \mid b$ for A_2

$$A_1 \rightarrow bA_3A_2B_3A_1A_3 \mid aB_3A_1A_3 \mid bA_3A_2A_1A_3 \mid aA_1A_3 \mid bA_3$$

B_3 – productions can be converted to GNF with the help of A_1 – productions.

$B_3 \rightarrow A_1A_3A_2B_3 \mid A_1A_3A_2$

⇓ Substituting $bA_3A_2B_3A_1A_3 \mid aB_3A_1A_3 \mid bA_3A_2A_1A_3 \mid aA_1A_3 \mid bA_3$ for A_1

$$\begin{aligned} B_3 \rightarrow\ & bA_3A_2B_3A_1A_3A_3A_2B_3 \mid aB_3A_1A_3A_3A_2\ B_3 \mid bA_3A_2A_1A_3A_3A_2\ B_3 \mid \\ & aA_1A_3A_3A_2\ B_3 \mid bA_3A_3A_2B_3 \\ B_3 \rightarrow\ & bA_3A_2B_3A_1A_3A_3A_2 \mid aB_3A_1A_3A_3A_2 \mid bA_3A_2A_1A_3A_3A_2 \mid aA_1A_3A_3A_2 \mid bA_3A_3A_2 \end{aligned}$$

The set of final productions in GNF is :

$A_1 \rightarrow bA_3A_2B_3A_1A_3 \mid aB_3A_1A_3 \mid bA_3\ A_2A_1A_3 \mid aA_1A_3 \mid bA_3$

$A_2 \rightarrow bA_3A_2B_3A_1 \mid aB_3A_1 \mid bA_3A_2A_1 \mid aA_1 \mid b$

$A_3 \rightarrow bA_3A_2B_3 \mid aB_3 \mid bA_3A_2 \mid a$

$B_3 \rightarrow bA_3A_2B_3A_1A_3A_3A_2B_3$

$\mid aB_3A_1A_3A_3A_2B_3$

$\mid bA_3A_2A_1A_3A_3A_2B_3$

$\mid aA_1A_3A_3A_2B_3$

$\mid bA_3A_3A_2B_3$

$\mid bA_3A_2B_3A_1A_3A_3A_2$

$\mid aB_3A_1A_3A_3A_2$

$\mid bA_3A_2A_1A_3A_3A_2$

$\mid aA_1A_3A_3A_2$

$\mid bA_3A_3A_2$

Scanned by CamScanner

Example 5.5.14 : Reduce the following grammar to GNF.
$S \rightarrow AB$, $A \rightarrow BSB \mid BB \mid b$
$B \rightarrow aAb \mid a$

MU - May 2011

Solution :

Step 1 : Making every symbol other than the first symbol (in derived string α in $A \rightarrow \alpha$) as a variable :

Variables C_b is substituted for b with resulting set of productions give as :

$S \rightarrow AB$

$A \rightarrow BSB \mid BB \mid b$

$B \rightarrow aAC_b \mid a$, $C_b \rightarrow b$

Step 2 : The variables S, A, B and C_b are renamed as A_1, A_2, A_3 and A_4 respectively. The resulting set of productions is given below.

$A_1 \rightarrow A_2A_3$

$A_2 \rightarrow A_3A_1A_3 \mid A_3A_3 \mid b$

$A_3 \rightarrow aA_1A_4 \mid a$

$A_4 \rightarrow b$

Step 3 : Convert to CFG

Given production		Equivalent Production in GNF
$A_4 \rightarrow b$	$\rightarrow$	$A_4 \rightarrow b$
$A_3 \rightarrow aA_1A_4 \mid a$	$\rightarrow$	$A_3 \rightarrow aA_1A_4 \mid a$
$A_2 \rightarrow A_3A_1A_3$		
$\Downarrow$ Substituting A_3		$A_2 \rightarrow aA_1A_4A_1A_3 \mid aA_1A_3$
$A_2 \rightarrow A_3A_3$		
$\Downarrow$ Substituting A_3		$A_2 \rightarrow aA_1A_4A_3 \mid aA_3$
$A_2 \rightarrow b$	$\rightarrow$	$A_2 \rightarrow b$
$A_1 \rightarrow A_2A_3$		
$\Downarrow$ Substituting A_2		$A_1 \rightarrow aA_1A_4A_1A_3A_3 \mid aA_1A_3A_3 \mid aA_1A_4A_1A_3A_3 \mid aA_3A_3 \mid bA_3$

$\therefore$ The final set of productions is :

$A_1 \rightarrow aA_1A_4A_1A_3A_3 \mid aA_1A_3A_3 \mid aA_1A_4A_3A_3 \mid aA_3A_3 \mid bA_3$

Scanned by CamScanner

$A_2 \rightarrow aA_1A_4A_1A_3 \mid aA_1A_3 \mid aA_1A_4A_3 \mid aA_3 \mid b$

$A_3 \rightarrow aA_1A_4 \mid a$

$A_4 \rightarrow b.$

Example 5.5.15 : Convert the following grammar to Greibach Normal Form (GNF),

$S \rightarrow BS,\ S \rightarrow Aa,\ A \rightarrow bc,\ B \rightarrow Ac$

Solution :

Step 1 : Modifying productions to ensure that on R.H.S, other than the first symbol every other symbol must be a variable.

$X \rightarrow a$

$Y \rightarrow c$

$S \rightarrow BS \mid AX$

$A \rightarrow bY$

$B \rightarrow AY$

Step 2 : Remaining of variables.

The variables S, A, B, X, Y are renamed as $A_1, A_2, A_3 A_4, A_5$

$A_4 \rightarrow a$

$A_5 \rightarrow c$

$A_1 \rightarrow A_3 A_1 \mid A_2 A_4$

$A_2 \rightarrow b A_5$

$A_3 \rightarrow A_2 A_5$

Step 3 : Productions for A_4, A_5, A_2 are in the required form. Productions for A_1 and A_3 can be converted to the required form.

$A_3 \rightarrow A_2 A_5$

$\Downarrow$ Substituting bA_5 for A_2

$A_3 \rightarrow bA_5 A_5$

$A_1 \rightarrow A_3A_1 \mid A_2A_4$

Substituting bA_5A_5 for A_3 and bA_5 for A_2

$A_1 \rightarrow bA_5A_5A_1 \mid bA_5A_4$

Scanned by CamScanner

∴ The final set of productions is given by

$A_1 \rightarrow bA_5A_5A_1 \mid bA_5A_4$

$A_2 \rightarrow bA_5$

$A_3 \rightarrow bA_5A_5$

$A_4 \rightarrow a$

$A_5 \rightarrow c$

Example 5.5.16 : Find GNF of the grammar given below

$S \rightarrow ABAb \mid ab$

$B \rightarrow ABA \mid a$

$A \rightarrow a \mid b$

Solution :

Step 1 : Making every symbol other than the first symbol (in derived string α in $A \rightarrow \alpha$) as a variable :

Variable C_b is substituted for b with resulting set of productions given as :

$S \rightarrow AB\,AC_b \mid aC_b$

$B \rightarrow ABA \mid a$

$A \rightarrow a \mid b$

$C_b \rightarrow b$

Step 2 : Substituting $A \rightarrow a \mid b$, in $S \rightarrow ABAC_b$ and $B \rightarrow ABA$, we can convert productions in GNF. The final set of productions is given by :

$S \rightarrow aBAC_b \mid bBAC_b \mid aC_b$

$B \rightarrow aBA \mid bBA \mid a$

$A \rightarrow a \mid b$

$C_b \rightarrow b$

Example 5.5.17 : Find the GNF equivalent to the CFG

$S \rightarrow AB$

$A \rightarrow aA \mid bB \mid b$

$B \rightarrow b$

Solution :

A-productions	$A \rightarrow aA \mid bB \mid b$ are in GNF
B-production	$B \rightarrow b$ is in GNF
S-production	$S \rightarrow AB$ is not in GNF.

Scanned by CamScanner

It can be brought into GNF through substitution.

$$S \rightarrow AB$$
$$\Downarrow$$
$$S \rightarrow aAB \mid bBB \mid bB$$

∴ The required get of productions in GNF is :

$$S \rightarrow aAB \mid bBB \mid bB$$
$$A \rightarrow aA \mid bB \mid b$$
$$B \rightarrow b$$

Example 5.5.18 : Find a GNF grammar equivalent to the following CFG.

$$S \rightarrow BA \mid ab$$
$$B \rightarrow AB \mid a$$
$$A \rightarrow Bb \mid BB$$

Solution :

Step 1 : Bring every production to the form $A \rightarrow a\alpha$ or $A \rightarrow \alpha$, where α is a string of non-terminals.

We can make the substitution D for b in the production $A \rightarrow Bb$ and $S \rightarrow ab$

$$S \rightarrow BA \mid aD$$
$$B \rightarrow AB \mid a$$
$$A \rightarrow BD \mid BB$$
$$D \rightarrow b$$

Step 2 : Remaining of variables.

The variables S,B,A and D are renamed as A_1, A_2, A_3 and A_4 respectively.

$$A_1 \rightarrow A_2A_3 \mid aA_4$$
$$A_2 \rightarrow A_3A_2 \mid a$$
$$A_3 \rightarrow A_2A_4 \mid A_2A_2$$
$$A_4 \rightarrow b$$

Step 3 : Every production of the form $A_i \rightarrow A_j\ \alpha$ with $i > j$ must be modified to make $i \leq j$.

$A_1 \rightarrow A_2A_3 \mid aA_4$
$A_2 \rightarrow A_3A_2 \mid a$
$A_3 \rightarrow A_2A_4 \mid A_2A_2$ – not in the required form

Substituting $A_3A_2 \mid a$ for A_2

$A_3 \rightarrow A_3A_2A_4 \mid aA_4 \mid A_3A_2A_2 \mid aA_2$

Scanned by CamScanner

Step 4 : Removing left recursion.

A_3 production has left recursion. Left recursion can be removed through introduction of B_3-production.

$$A_3 \rightarrow aA_4B_3|aA_2B_3|aA_4|aA_2$$

$$B_3 \rightarrow A_2A_4B_3|A_2A_2B_3|A_2A_4|A_2A_2$$

The resulting set of production is :

$$A_1 \rightarrow A_2A_3|aA_4$$

$$A_2 \rightarrow A_3A_2|a$$

$$A_3 \rightarrow aA_4B_3|aA_2B_3|aA_4|aA_2$$

$$B_3 \rightarrow A_2A_4B_3|A_2A_2B_3|A_2A_4|A_2A_2$$

Step 5 : Productions for A_3 are already in GNF. Productions for A_1,A_2 and B_3 can be converted to GNF.

$$A_3 \rightarrow aA_4B_3|aA_2B_3|aA_4|aA_2$$

$$A_2 \rightarrow A_3A_2|a$$

$\Downarrow$ Substituting a $A_4B_3|aA_2B_3|aA_4|aA_2$ for A_3

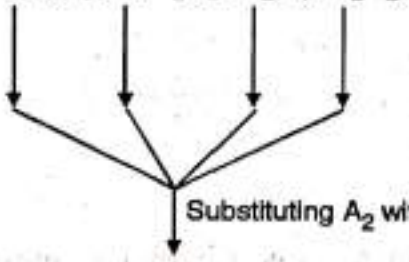
$$A_2 \rightarrow aA_4B_3A_2|\ aA_2B_3A_2|aA_4A_2|aA_2A_2|a$$

$$A_1 \rightarrow A_2A_3|aA_4$$

$\Downarrow$ Substituting A_2 with its productions

$$A_1 \rightarrow aA_4B_3A_2A_3|aA_2B_3A_2A_3|\ aA_4A_2A_3|aA_2A_2A_3|aA_3|aA_4$$

$B_3 \rightarrow A_2A_4B_3 \mid A_2A_2B_3 \mid A_2A_4 \mid A_2A_2$



Substituting A_2 with its productions

$$B_3 \rightarrow aA_4B_3A_2A_4B_3|aA_2B_2A_2A_4B_3|aA_4A_2A_4B_3|aA_2A_2A_4B_3|aA_4B_3|$$
$$aA_4B_3A_2A_2B_3|aA_2B_2A_2A_2B_3|aA_4A_2A_2B_3|aA_2A_2B_3|aA_2B_3$$
$$|aA_4B_3A_2A_4|aA_2B_2A_2A_4|aA_4A_2A_4|aA_2A_2A_4|aA_4$$
$$|aA_4B_3A_2A_2|aA_2B_2A_2A_2|aA_4A_2A_2|aA_2A_2A_2|aA_2$$

Example 5.5.19 : Convert the given grammar to GNF.
$S \rightarrow SS|aSb|ab$ **MU - Dec. 2006**

Scanned by CamScanner

Solution :

Step 1 : Other than the first symbol on the RHS of every production, every symbol must be a variable.

We can make the substitution X for b.

The resulting set of productions after the above substitution is :

$S \rightarrow SS|aSX|aX$

$X \rightarrow b$

Step 2 : Removing left recursion from s-production, we get :

$S \rightarrow aSXS_1|aXS_1|aSX|aX$

$S_1 \rightarrow SS_1|S$

$X \rightarrow b$

Step 3 : S_1-productions are not in GNF. They can be brought to GNF by substituting S.

$S \rightarrow aSXS_1|aXS_1|aSX|aX$

$S_1 \rightarrow aSXS_1S_1|aXS_1S_1|aSXS_1|aXS_1|aSXS_1|aXS_1|aSX|aX$

$X \rightarrow b$

Example 5.5.20 : Convert the following grammar into GNF

$S \rightarrow XY1|0$

$X \rightarrow 00X|Y$

$Y \rightarrow 1X1$

MU - May 2006, May 2012

Solution :

Simplification of grammar

The unit production $x \rightarrow y$ is removed, the equivalent set of productions is given by :

$S \rightarrow XY1|0$

$X \rightarrow 00X|1X1$

$Y \rightarrow 1X1$

The symbol X is non-generating.

The set of productions after elimination of X is given by :

$S \rightarrow 0$, it is in GNF

Example 5.5.21 : Convert the following CFG to GNF :

$S \rightarrow aSa \mid bSb \mid c$

MU - Dec. 2005

Solution :

The grammar can be brought to GNF through simple substitutions $C_a \rightarrow a$ and $C_b \rightarrow b$.

$S \rightarrow aSC_a \mid bSC_b \mid C$

$C_a \rightarrow a$

$C_b \rightarrow b$

Scanned by CamScanner

Example 5.5.22 : Reduce the following grammars to GNF

$B \rightarrow aAb \mid a$

$S \rightarrow AA \mid 1$

$A \rightarrow SS \mid 1$

MU - May 2011

Solution :

Step 1 : Renaming of variables by substituting A_1 for S and A_2 for A.

$A_1 \rightarrow A_2 A_2 \mid 1$

$A_2 \rightarrow A_1 A_1 \mid 1$

Step 2 : Every production of the form $A_i \rightarrow A_j \alpha$ with $i > j$ must be modified to make $i \leq j$.

A_2 – production, $A_2 \rightarrow A_1 A_1$ should be modified. We must substitute $A_2 A_2 \mid 1$ for the first A_1.

$$[A_2 \rightarrow A_1 A_1] \Rightarrow \begin{bmatrix} A_2 \rightarrow A_2 A_2 A_1 \\ A_2 \rightarrow 1A_1 \end{bmatrix}$$

The resulting set of productions is :

$A_1 \rightarrow A_2 A_2 \mid 1$

$A_2 \rightarrow A_2 A_2 A_1 \mid 1 A_1 \mid 1$

Step 3 : Removing left recursion :

The A_2 – production contains left recursion. Left recursion can be removed through introduction of B_2 – production.

$A_2 \rightarrow 1 A_1 B_2 \mid 1 B_2$

$B_2 \rightarrow A_2 A_1 B_2 \mid A_2 A_1$

The resulting set of productions is :

$A_1 \rightarrow A_2 A_2 \mid 1$

$A_2 \rightarrow 1 A_1 B_2 \mid 1 B_2 \mid 1 A_1 \mid 1$

$B_2 \rightarrow A_2 A_1 B_2 \mid A_2 A_1$

Step 4 : A_2 – productions are in GNF.

A_1 and B_2 productions can be converted to GNF with the help of A_2 – productions.

$A_2 \rightarrow 1 A_1 B_2 \mid 1 B_2 \mid 1 A_1 \mid 1$

$A_1 \rightarrow 1 A_1 B_2 A_2 \mid 1 B_2 A_2 \mid 1 A_1 A_2 \mid 1 A_2 \mid 1$

$B_2 \rightarrow 1 A_1 B_2 A_1 B_2 \mid 1 B_2 A_1 B_2 \mid 1 A_1 A_1 B_2$

$\mid 1 A_1 B_2 \mid 1 A_1 B_2 A_1 \mid 1 B_2 A_1 \mid 1 A_1 A_1 \mid 1 A_1$

Scanned by CamScanner

5.6 Pumping Lemma for CFG

MU - Dec. 2008, Dec. 2015, May 2016, May 2017

Let G be a context free grammar. Then there exists a constant n such that any string $w \in L(G)$ with $|w| \geq n$ can be rewritten as $w = uvxyz$, subject to the following conditions :

1. $|vxy| \leq n$, the middle portion is less than n.
2. $vy \neq \varepsilon$, strings v and y will be pumped.
3. For all $i \geq 0$, uv^ixy^iz is in L. The two strings v and y can be pumped zero or more times.

Proof :

Let us assume that the grammar

G is given by (V, T, P, S).

$\phi(G)$ denotes that largest number of symbols on the right-hand side of a production in P.

In pumping lemma, it is a requirement that the constant n should satisfy the following condition :

$$n \geq \phi(G)^{|V-T|}$$

Let us take a string $w \in L(G)$, such that $|w| \geq n$. Let us construct a parse tree T with root as S. The parse tree T generates w with smallest number of leaves.

The tree T will have a path length of at least $|V - T| + 1$. This path will have $|V - T| + 2$ nodes with the last node labelled as terminal and remaining non-terminals.

Fig. 5.6.1 shows paths in detail.

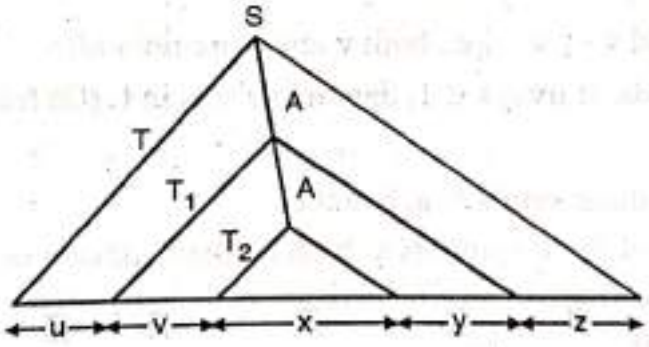


Fig. 5.6.1 : Paths in the parse tree

x is generated by T_2

v is generated by T_1

u is generated by T

T_1 excluding T_2 can be repeated any number of times.

This will yield a string of the form uv^ixy^iz where $i \geq 0$.

Scanned by CamScanner

Example 5.6.1 : Prove that the language

$L = \{a^p \mid p \text{ is a prime}\}$ is not context free language.

MU - May 2006, May 2012

Solution :

1. Let us assume that L is a CFL.
2. Let n be the natural number for L, as per the pumping lemma.
3. Let p be a prime number greater than n. Then $z = a^p \in L$. We can write $z = uvxyz$.
4. By pumping lemma $uv^0xy^0z = uxz \in L$. Therefore,

   $|uxz|$ is a prime number.

   Let us assume that $|uxz| = q$.

   Now, let us consider a string uv^qxy^qz,

   The length of uv^qxy^qz is given by :

   $|uv^q xy^q z| = q + q(|v| + |y|)$, which is not a prime with q is a factor.

   Thus, $uv^qxy^qz \notin L$. This is a contradiction.

   Therefore, L is not a context free language.

Example 5.6.2 : Prove that $L = \{a^i b^i c^i \mid i \geq 1\}$ is not a CFL.

MU - May 2008

Solution :

1. Let us assume that L is CFL.
2. Let us pick up a word $w = a^n b^n c^n$ where the constant n is given as per the pumping lemma.
3. w is rewritten as uvxyz.

   Where $|vxy| \leq n$ and $v \cdot y \neq \varepsilon$ i.e., both v and y are not null.
4. From pumping lemma, if $uvxyz \in L$ then uv^ixy^iz is in L (G) for each $i = 0, 1, 2, \ldots$

   There are two cases :

Case I : vy contains all three symbols a, b and c.

If vy contains all three symbols a, b and c then either v or y contains two symbols. The exact ordering of a, b and c will be broken in uv^2xy^2z and hence $uv^2xy^2z \notin L(G)$

Case II : If vy does not contain three symbols a, b and c then uv^2xy^2z will have unequal number of a's, b's and c's and hence $uv^2xy^2z \notin L(G)$.

Hence, it is proved by contradiction.

Example 5.6.3 : Prove that $L = \{a^i b^i c^j \mid j \geq i\}$ is not a CFL.

Solution :

1. Let us assume that L is CFL.
2. Let us pick up a word $\omega = a^n b^n c^n$, where n is a constant given as per the pumping lemma.

Scanned by CamScanner

3. ω is rewritten as uvxyz where, $|vxy| \leq n$ and $v.y \neq \varepsilon$, both v and y are not null.
4. From pumping lemma, if $uvxyz \in L$ then $uv^i xy^i z$ is in L(G) for each i = 0,1,2....

There are two cases :

Case I : vy contains all three symbols a,b and c. If vy contains all three symbols a,b, and c then either v or y contains two symbols. The exact ordering of a,b and c will be broken in uv^2xy^2z and hence $uv^2xy^2z \notin L(G)$.

Case II : If vy does not contain three symbols a,b and c then uv^2xy^2z will have either :

(a) Unequal number of a and b

(b) Count of either a or b can be increased from the count of c.

Hence, proved by contradiction.

Example 5.6.4 : $A = \{a^{n^2} \mid n \geq 1\}$ is context free. If so, enumerate some members of the equivalent CFL.

Solution :

$L = \{ a^{n^2} \mid n \geq 1 \}$ is not a context free language

Proof that L is not a CFL :

It can be proved using a contradiction. Let us assume that L is a context free language.

1. Let n be the constant as per the pumping lemma.
2. Let us choose a word $w = a^{n^2}$
3. w is rewritten as uvxyz.

Where $|vxy| \leq n$ and $v \cdot y \neq \varepsilon$, both v and y are not null.

$$|uvxyz| = |a^{n^2}| = n^2$$

$$|uv^2xy^2z| = |uvxyz| + |vy| > n^2$$

And $|uv^2xy^2z| \leq n^2 + n$ [as $|vy| \leq n$]

$$\leq n^2 + 2n + 1$$

$$\leq (n+1)^2$$

$|uv^2ny^2z|$ is a square for every i, $uv^ixy^iz \in L$. But there is no square between n^2 and $(n+1)^2$. This is a contradiction. Therefore, L is not a CFL.

Example 5.6.5 : Prove that the following language is not a context free language.

$L = \{a^n b^m c^p \mid 0 \leq n < m < p\}$

Solution :

1. Let us assume that $L = \{a^n b^m c^p \mid 0 \leq n < m < p\}$ is CFL.
2. Let us pick up a word $\omega = a^n b^{n+1} c^{n+2}$, where the constant n is given as per the pumping lemma.

Scanned by CamScanner

3. ω is rewritten as uvxyz, where $|vxy| \leq n$ and $v.y \neq \in$, both v and y are not null.
4. From pumping lemma, if $uvxyz \in L$ then $uv^i xy^i z$ is in L(G) for each i = 0,1,2...

There are two cases :

Case I : vy contains all three symbols a,b and c. If vy contains all three symbols a,b and c, then either v or y contains two symbols the exact ordering of a,b and c will be broken in uv^2xy^2z and hence $uv^2xy^2z \notin L(G)$.

Case II : If vy does not contain three symbols a,b and c then the count of the missing symbols/symbol will be less than or equal to the other symbols/symbol in uv^2xy^2z. Similarly, the count of the missing symbols/symbol will be more than or equal to the other symbol/symbols in uv^0xy^0z.

Thus the sequence $n < m < p$ in $a^n b^m c^p$ can be made to violate either for i = 0 or i = 2 in uv^ixy^iz.

Hence, it is proved by contradiction.

Example 5.6.6 : Prove that the language

$L = \{\omega\omega|\omega \text{ is in } (0 + 1)^*\}$ is not a CFL.

Solution :

1. Let us assume that $L = \{\omega\omega|\omega \text{ is in } (0 + 1)^*\}$ is a CFL.
2. Let us pick up a word $\omega = 0^n1^n0^n1^n$, where the constant n is given as per the pumping lemma.
3. ω is rewritten as uvxyz, when $|vxy| \leq n$ and $v.y \neq \in$.
4. From pumping lemma, if $uvxyz \in L$, then uv^ixy^iz is in L for each i =0,1,2...
5. vxy must be in one of the following forms :
   $0^i, 1^j, 0^i1^k, 1^j0^k$

Case I : vxy is of the form 0^j - Two sets of 0's will be unequal in uv^0xy^0z.

Case II : vxy is of the form i^j - two sets of 1's will be unequal in uv^0xy^0z.

Case III : vxy is of the form 0^j1^k or 1^j0^k-Two set of 0's or two sets of 1's will be unequal in uv^0xy^0z.

Thus the string uv^0xy^0z does not belong to L. Hence, it is proved by contradiction.

Example 5.6.7 : Prove that $L = \{0^i 1^j 2^i 3^j | i >= 1 \text{ and } j >= 1\}$ is not context free. **MU - Dec. 2007**

Solution :

1. Let us assume that L is CFL
2. Let us pick up a word $\omega = 0^n 1^n 2^n 3^n$, where the constant n is given as per the pumping lemma.

Scanned by CamScanner

3. ω is rewritten as uvxyz where $|vxy| \leq n$ and $v \cdot y \neq \varepsilon$ i.e. both v and y are not null.
4. From pumping lemma, if $uvxyz \in L$ then $uv^i xy^i z$ is in L(G) for each $i = 0,1,2,...$

There are two case :

Case I : vy contains three symbols. These three symbols could be 0,1,2 or 1,2,3. The exact ordering of 0,1,2,3 will be broken in uv^2xy^2z and hence $uv^2xy^2z \notin L(G)$

Case II : If vy does not contain three symbols then uv^2xy^2z will have either unequal number of 0's and 2's or unequal number of 1's and 3's. Hence, $uv^2xy^2z \notin L(G)$.

Thus, proved by contradiction.

5.7 Properties of Context-free Languages

In this section, we will consider some general properties of CFL. These properties can be classified in two groups :

1. Closure properties
2. Algorithmic properties (Decision properties)

5.7.1 Closure Properties

MU - Dec. 2011, Dec. 2012 , May 2014, May 2015, Dec. 2017

A context free language is closed under following operations :

1. Union
2. Concatenation
3. Kleene star

Context free language is not closed under intersection.

The intersection of a context-free language with a regular language is a context free language.

The CFL is not closed under complementation.

The CFL is closed under reversal.

5.7.1.1 CFL is Closed under Union

Theorem :

If L_1 and L_2 are context-free languages, then $L_1 \cup L_2$ is a context free language.

Proof :

Let L_1 be a CFL. It is generated by a context free grammar $G_1 = (V_1, T_1, P_1, S_1)$.

Similarly, L_2 is another CFL generated by a context-free grammar $G_2 = (V_2, T_2, P_2, S_2)$

We can combine the two grammars G_1 and G_2 into one grammar G that will generate the union of the two languages.

- A new start symbol S is added to G.

Scanned by CamScanner

- Two new productions are added to G.

  $S \rightarrow S_1$

  $S \rightarrow S_2$

The grammar G can be written as :

$$G = (V_1 \cup V_2 \cup \{ S \}, T_1 \cup T_2, P_1 \cup P_2 \cup \{ S \rightarrow S_1 \mid S_2 \}, S)$$

S can generate a string of terminals either by selecting start symbol S_1 of G_1 or start symbol S_2 of G_2. Thus, S can generate a string from L_1 or from L_2.

$$\therefore \quad L(G) = L_1 \cup L_2$$

5.7.1.2 CFL is Closed under Concatenation

Theorem :

If L1 and L2 are context-free languages, then L_1L_2 is a context-free language.

Proof :

Let L_1 be a CFL with the grammar $G_1 = (V_1, T_1, P_1, S_1)$.

Let L_2 be a CFL with the grammar $G_2 = (V_2, T_2, P_2, S_2)$

A new language L is constructed by combining the two grammars G1 and G2 into one grammar G that will generate the concatenation of the two languages.

- A new start symbol S is added to G.
- A new production is added to G.

  $S \rightarrow S_1S_2$

The start symbol S will generate a string w of the form :

$$w = w_1w_2, \text{ where } w_1 \in L_1 \text{ and } w_2 \in L_2.$$

The grammar G can be written as :

$$G = (V_1 \cup V_2 \cup \{ S \}, T_1 \cup T_2, P_1 \cup P_2 \cup \{ S \rightarrow S_1S_2 \}, S)$$

5.7.1.3 CFL is Closed under Kleene Star

Theorem :

If L is a context-free language, then L^* is a context-free language.

Proof :

Let L_1 be a CFL with the grammar $G_1 = (V_1, T_1, P_1, S_1)$

A new language L is constructed from L_1, which is L_1^*.

i.e. $L = L_1^*$

- A new start symbol S is added to the grammar G of L.
- Two new productions are added to G.

Scanned by CamScanner

$S \rightarrow SS_1$

$S \rightarrow \varepsilon$

The production $S \rightarrow SS_1 \mid \varepsilon$ will generate a string w* where $w \in L_1$.

The grammar G can be written as :

$$G = (V_1, T_1, P_1 \cup \{ S \rightarrow SS_1 \mid \varepsilon \}, S)$$

5.7.1.4 CFL is not Closed under Intersection

Theorem :

Context-free languages are not closed under intersection.

Proof :

Let us consider two context-free languages L_1 and L_2.

Where,

$$L_1 = \{ a^n b^n c^m \mid n, m \geq 0 \}$$

$$L_2 = \{ a^m b^n c^n \mid n, m \geq 0 \}$$

The language L_1 is a CFL with set of productions given below :

$S \rightarrow AB$

$A \rightarrow aAb \mid \varepsilon$

$B \rightarrow cB \mid \varepsilon$

The language L_2 is a CFL with set of productions given below :

$S \rightarrow AB$

$A \rightarrow aA \mid \varepsilon$

$B \rightarrow bBc \mid \varepsilon$

- A string $\omega_1 \in L_1$ contains equal number of a's and b's.
- A string $\omega_2 \in L_2$ contains equal number of b's and c's.

A string $\omega \in L_1 \cap L_2$ will contain equal number of a's and b's and equal number of b's and c's.

Thus, $L_1 \cap L_2 = \{ a^n b^n c^n \mid n \geq 0 \}$. From pumping lemma for CFL, a string of the form $a^n b^n c^n$ cannot be generated by a CFG.

Therefore, the class of context-free languages is not closed under intersection.

5.7.1.5 CFL is not Closed under Complementation

Theorem :

The set of context-free languages is not closed under complementation.

Scanned by CamScanner

Proof :

This theorem can be proved through contradiction.

Let us assume that CFL is closed under complementation.

If L_1 is context-free then L_1' is also context-free.

If L_2 is context-free then L_2' is also context-free.

Now, $L_1 \cap L_2$ can be written as $\left(L_1' \cup L_2' \right)'$, which should also be a context-free.

Since, $L_1 \cap L_2$ is not guaranteed to be context-free, our assumption that CFL is closed under complementation is wrong.

5.7.1.6 Intersection of CFL and RL

Theorem :

If L is a CFL and R is a regular language, then $R \cap L$ is a CFL.

Proof :

Let us assume that L is accepted by a PDA

$$M_1 = (Q_1, \Sigma_1, \Gamma_1, \delta_1, q_1, z_1, F_1)$$

and R is accepted by a FA

$$M_2 = (Q_2, \Sigma_2, \delta_2, q_2, F_2)$$

We can combine M_1 and M_2 into a single PDA $M = (Q, \Sigma, \Gamma, \delta, q, z, F)$. The PDA M will accept a string w if it accepted by the PDA M_1 and FA M_2 both executing in parallel.

The construction of M is given below :

$Q = Q_1 \times Q_2$, the Cartesian product of states of M_1 and M_2

$\Sigma = \Sigma_1 \cup \Sigma_2$

$\Gamma = \Gamma_1$

$q = (q_1, q_2)$

$z = z_1$

$F = F_1 \times F_2$

The transition function δ is defined as :

$$\delta((q_1, q_2), u, \beta) = ((p_1, p_2), y) \qquad \text{[transition for M]}$$

if and only if

$$\delta_1(q_1, u, \beta) = (p_1, y) \qquad \text{[transition for } M_1 \text{]}$$

and

$$(q_2, u) \overset{*}{\underset{M_2}{\Rightarrow}} (p_2, \varepsilon)$$

- When M passes from state (q_1, q_2) to state (p_1, p_2), M_1 passes from state q_1 to p_1.

Scanned by CamScanner

- Since, M_2 will read one symbol at a times, it requires | u | steps to reach the state p_2 from q_2.

Thus M is a PDA for intersection of $L(M_1)$ and $L(M_2)$.

$$L(M) = L(M_1) \cap L(M_2)$$

5.7.1.7 CFL is Closed under Reversal

Theorem :

If L is a context-free language, then so is L^R.

Proof :

Let us assume that L = L(G) for some context-free grammar G = (V, T, P, S)

A grammar generating reverse L is given by

$$G^R = (V, T, P^R, S)$$

P^R can be obtained from P by reversing the right hand side of the production.

If $A \rightarrow \alpha$ is a production in P then

$A \rightarrow \alpha^R$ is a production in P^R.

5.7.2 Algorithmic Properties (Decision Properties)

MU - Dec. 2012

There are algorithms for testing :

1. Whether the language generated by a CFG is empty i.e. Is $L(G) = \phi$?
2. Given a grammar G and a string w, is $w \in L(G)$?

The method of finding whether a string w belongs to L(G) is known as parsing. There are two types of parsing :

1. Top-down parsing
2. Bottom-up parsing

In top-down parsing, we try to derive w, starting from the start symbol S.

In bottom up parsing, we try to reduce the word w into the start symbol S.

5.8 University Questions and Answers

Dec. 2005

Q. 1 Use pumping lemma to show the following : **(5 Marks)**

$a^n b^m c^p$ is not context free for $n \geq 0$, $m = 2n$ and $p = m + n$.

(Similar to Example 5.6.2)

Scanned by CamScanner

Q. 2 Design a GRAMMER for accepting Even Palindrome over Σ = (a, b). **(Section 5.2.3)** **(8 Marks)**

Q. 3 Design an unambiguous grammar to accept arithmetic statements having terminals as : +, −, *, +, =, ∧ (power), ;, (,) and 'id'. **(Example 5.3.5)** **(10 Marks)**

Q. 4 Convert the following CFG to CNF :

S →aSa | bSb | c. **(Example 5.5.21)** **(10 Marks)**

Q. 5 Write short note on GNF. **(Section 5.5.2)** **(5 Marks)**

May 2006

Q. 6 Proof L = {a^p | p is prime} is not context free **(Example 5.6.1)** **(5 Marks)**

Q. 7 Consider the following grammar :

S → ASB | ε

A → AAS | a

B → SbS | A | bb

Put the above grammar in CNF. **(Example 5.5.7)** **(10 Marks)**

Q. 8 Convert the following grammar into GNF

S → XY1 | 0

X → 00X | Y

Y → 1X1 **(Example 5.5.20)** **(10 Marks)**

Dec. 2006

Q. 9 Find CFG for generating

(i) Strings containing alternate sequence of 0's and 1's, Σ = {0, 1} **(Example 5.2.33(i))**

(ii) Strings over Σ = {a, b} with atleast two a's **(Example 5.2.17, Refer 'a' as 0)**

(10 Marks)

Q. 10 Convert the given grammars to GNF

S → SS | aSb | ab **(Example 5.5.19)** **(10 Marks)**

May 2007

Q. 11 Convert the following to CNF :

S → aSb | bSa | ab | ba | ε **(Similar to Example 5.5.9)** **(10 Marks)**

Q. 12 Write short notes on :

(a) GNF **(Section 5.5.2)**

(b) Ambiguity resolution **(Section 5.3)** **(15 Marks)**

Scanned by CamScanner

Dec. 2007

Q. 13 Prove that $L = \{0^i 1^i 2^j 3^j \mid i >= 1 \text{ and } j >= 1\}$ is not context free. **(Example 5.6.7)** **(8 Marks)**

May 2008

Q. 14 Prove that $L = \{a^n b^n c^n \mid n >= 1\}$ is not context free. **(Example 5.6.2)** **(5 Marks)**

Q. 15 Convert the following grammar in CNF

$S \rightarrow AACD$

$A \rightarrow aAb \mid \varepsilon$

$C \rightarrow aC \mid a$

$D \rightarrow aDa \mid bDb \mid \varepsilon$ **(Example 5.5.6)** **(10 Marks)**

Q. 16 Convert the following grammar in GNF

$S \rightarrow AA \mid 0$

$A \rightarrow SS \mid 1$ **(Example 5.5.10) (Note : '0' as 'a' and '1' as 'b')** **(10 Marks)**

Dec. 2008

Q. 17 State and prove pumping lemma for context free languages. **(Section 5.6)** **(10 Marks)**

Q. 18 Explain GNF with suitable example. **(Section 5.5.2)** **(10 Marks)**

Q. 19 Design a grammar for accepting an even palindrome over $\Sigma = \{a,b\}$ **(Section 5.2.3)** **(10 Marks)**

Q. 20 Write short note on Ambiguity resolution. **(Section 5.3)** **(5 Marks)**

May 2009

Q. 21 Let $G = \{V,T,P,S\}$ be the CFG having following set of productions. Derive the string "aabbaa" using leftmost derivation and rightmost derivation.

$S \rightarrow aAS \mid a$, $A \rightarrow SbA \mid SS \mid ba$. **(Example 5.2.4)** **(5 Marks)**

Q. 22 Given a CFG G, find G' in CNF generating $L(G) - \varepsilon$.

$S \rightarrow ASB \mid \varepsilon$

$A \rightarrow AaS \mid a$

$B \rightarrow SbS \mid A \mid bb$ **(Example 5.5.7)** **(10 Marks)**

Q. 23 Write short note on ambiguity resolution. **(Section 5.3)** **(4 Marks)**

Scanned by CamScanner

Q. 24 Give context free grammar generating the following sets.

(i) The string containing no consecutive 'b's but 'a's can be consecutive.

(ii) The set of all string over alphabet {a,b} with exactly twice as many a's as b's.

(Example 5.2.33(ii) and (iii)) **(4 Marks)**

Dec. 2009

Q. 25 Explain CNF and GNF with example. **(Sections 5.5.1 and 5.5.2)** **(6 Marks)**

Q. 26 Let G be the grammar

$S \rightarrow aB \mid bA$

$A \rightarrow a \mid aS \mid bAA$

$B \rightarrow b \mid bS \mid aBB$

Find leftmost derivation, rightmost derivation and parse tree for the string "bbaaabbaba". **(Example 5.3.9)** **(10 Marks)**

Q. 27 Write CFG for language having number of a's greater than number of b's.

(Example 5.2.33(iv)) **(10 Marks)**

May 2010

Q. 28 Obtain a grammar to generate the language $L = \{ 0^n 1^{2n} \mid n \geq 0) \}$.

(Example 5.2.36) **(5 Marks)**

Q. 29 Convert the following grammar in Chomsky Normal form –

$S \rightarrow ASB \mid \varepsilon$

$B \rightarrow SbS \mid A \mid bb$

$A \rightarrow AAS \mid a$ **(Similar to Example 5.5.7)** **(10 Marks)**

Q. 30 Show that the following grammar is ambiguous?

$S \rightarrow aSbS$

$S \rightarrow bSaS$

$S \rightarrow \varepsilon$ **(Example 5.3.12(1), Refer '0' as 'a' and '1' as 'b')** **(10 Marks)**

Dec. 2010

Q. 31 Let G be the grammar. Find the leftmost derivation, rightmost derivation and parse tree for the string 00110101.

G : $S \rightarrow 0B \mid 1A$

$A \rightarrow 0 \mid 0S \mid 1AA$

$B \rightarrow 1 \mid 1S \mid 0BB$ **(Example 5.2.7)** **(10 Marks)**

Scanned by CamScanner

Q. 32 Convert the following CFG to CNF

$S \rightarrow bA \mid aB$

$A \rightarrow bAA \mid aS \mid a$

$B \rightarrow aBB \mid bS \mid b$ **(Example 5.5.3)** **(5 Marks)**

Q. 33 Write short note on : Chomsky Hierarchy **(Section 5.5.1)** **(6 Marks)**

May 2011

Q. 34 Reduce the following grammars to GNF

$S \rightarrow AB$

$A \rightarrow BSB \mid BB \mid b$ **(Example 5.5.14)** **(5 Marks)**

Q. 35 Reduce the following grammars to GNF

$B \rightarrow aAb \mid a$

$S \rightarrow AA \mid 1$

$A \rightarrow SS \mid 1$ **(Example 5.5.22)** **(5 Marks)**

Q. 36 Convert the following grammars to CNF

$A \rightarrow aBb \mid bBa$

$B \rightarrow aB \mid bB \mid \varepsilon$ **(Example 5.5.9)** **(10 Marks)**

Dec. 2011

Q. 37 Write a short note on Ambiguity Resolution. **(Section 5.3)** **(5 Marks)**

Q. 38 Consider the grammar :

$S \rightarrow 0B \mid 1A$

$A \rightarrow 0 \mid 0S \mid 1AA$

$B \rightarrow 1 \mid 1S \mid 0BB$

For the string 00110101 find the following :

(i) Leftmost derivation

(ii) Rightmost derivation

(iii) Parse tree. **(Example 5.2.6)** **(10 Marks)**

Q. 39 Convert the following grammar into CNF :

$S \rightarrow ASB \mid \epsilon$

$A \rightarrow AaS \mid a$

$B \rightarrow SbS \mid A \mid bb$ **(Example 5.5.7)** **(10 Marks)**

Q. 40 Convert the following grammar in GNF :

$S \rightarrow AA \mid 0$

$A \rightarrow SS \mid 1$ **(Example 5.5.11) (Note : take 0 as a and 1 as b)** **(8 Marks)**

Scanned by CamScanner

Q. 41 Write short note on : Closure properties of context free language.
(Section 5.7.1) **(6 Marks)**

May 2012

Q. 42 Prove that it is undecidable whether Context free grammar is ambiguous.
(Section 5.3) **(5 Marks)**

Q. 43 Begin with the grammar :

$S \rightarrow ABC \mid BaB$

$A \rightarrow aA \mid BaC \mid aaa$

$B \rightarrow bBb \mid a \mid D$

$C \rightarrow CA \mid AC$

$D \rightarrow \epsilon$

(i) Eliminate ϵ Productions.

(ii) Eliminate any unit production in the resulting grammar.

(iii) Eliminate any useless symbols in the resulting grammar.

(iv) Put the resulting grammar into Chomsky Normal Form.

(Example 5.5.10) **(14 Marks)**

Q. 44 Prove that $L = \{a^n \mid n \text{ is prime}\}$ is not context free. **(Example 5.6.1)** **(6 Marks)**

Q. 45 Convert the following grammar into Greibach Normal Form.

$S \rightarrow AB1 \mid 0$

$A \rightarrow 00A \mid B$

$B \rightarrow 1A1$ **(Example 5.5.20, Refer 'X' as 'A' and 'Y' as 'B')** **(10 Marks)**

Dec. 2012

Q. 46 Give and explain ambiguous context free language. **(Section 5.3)** **(5 Marks)**

Q. 47 Let G be the grammar

$S \rightarrow aB \mid bA$

$A \rightarrow a \mid aS \mid bAA$

$B \rightarrow b \mid bS \mid aBB$

Find the leftmost derivation, right most derivation and parse tree for the string "bbaaabbaba". **(Example 5.3.9)** **(10 Marks)**

Q. 48 Write short notes on : Closure property and decision algorithm for CFL.
(Sections 5.7.1 and 5.7.2) **(20 Marks)**

Scanned by CamScanner

May 2013

Q. 49 What is ambiguous CFG ? Give one example of ambiguous CFG. **(Section 5.3)** **(5 Marks)**

Q. 50 Let G be the grammar, find the leftmost derivation, rightmost derivation and parse tree for the string 00110101.

$S \rightarrow 0B \mid 1A$

$A \rightarrow 0 \mid 0S \mid 1AA$

$B \rightarrow 1 \mid 1S \mid 0BB$ **(Example 5.3.9) (Note : take '0' as 'a' and '1' as 'b')**

Q. 51 Explain CNF and GNF with example. **(Sections 5.5.1 and 5.5.2)** **(10 Marks)**

May 2014

Q. 52 Explain CNF and GNF with example. **(Sections 5.5.1 and 5.5.2)** **(5 Marks)**

Q. 53 State and prove closure properties of Context Free Languages. **(Section 5.7.1)** **(5 Marks)**

Q. 54 Consider the following grammar :

$S \rightarrow iCtS \mid iCtSeS \mid a$

$C \rightarrow b$

For the String 'ibtibtaea' find the following :

(i) Leftmost derivation

(ii) Rightmost derivation

(iii) Parse Tree

(iv) Check if the above grammar is ambiguous **(Section 5.2.7)** **(10 Marks)**

Q. 55 Convert the following Grammar to CNF form :

$S \rightarrow ABA$

$A \rightarrow aA \mid bA \mid \epsilon$

$B \rightarrow bB \mid aA \mid \epsilon$ **(Example 5.5.10(A))** **(10 Marks)**

Dec. 2014

Q. 56 Obtain leftmost derivation, rightmost derivation and derivation tree for the string "cccbaccba". The grammar is

$S \rightarrow SS\ a \mid SSb \mid c$ **(10 Marks)**

Scanned by CamScanner

Ans. : [Hint : Refer as Example 5.2.3(A)]

Derivation tree :

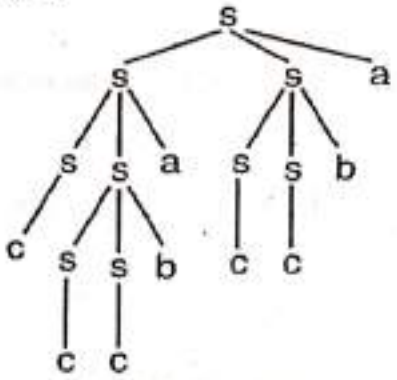


Fig. 1-Q. 4

Left most derivation

S → SSa
→ SSaSa
→ cSaSa
→ cSSbaSa
→ ccSbaSa
→ cccbaSa
→ cccbaSSba
→ cccbacSba
→ cccbaccba

Right most derivation

S → SSa
→ SSSba
→ SScba
→ Sccba
→ SSaccba
→ SSSbaccba
→ SScbaccba
→ Sccbaccba
→ cccbaccba

May 2015

Q. 57 State and Explain closure properties of Context Free Language. **(Section 5.7.1)** **(5 Marks)**

Q. 58 Convert following grammar to CNF and GNF. **(10 Marks)**

S → ASB | a | bb

A→ aSA | a

B → SbS | bb

Ans. : [Hint : Refer as Example 5.5.20(A)]

S → ASB | a | bb

A → aSA | a

B → SbS | bb

Scanned by CamScanner

Converting to CNF :

Re-writing the grammar, we get,

$S \rightarrow ASB \mid a \mid V_1V_1$

$A \rightarrow V_2SA \mid a$

$B \rightarrow SV_1S \mid V_1V_1$

$V_1 \rightarrow b$

$V_2 \rightarrow a$

Now, re-writing each production in its equivalent CNF form, we get,

Productions	**CNF forms**
$S \rightarrow ASB$	$S \rightarrow AV_3, V_3 \rightarrow SB$
$S \rightarrow a$	$S \rightarrow a$
$S \rightarrow V_1 V_1$	$S \rightarrow V_1 V_1$
$A \rightarrow V_2 SA \mid a$	$S \rightarrow V_2 V_4, V_4 \rightarrow SA$
	$A \rightarrow a$
$B \rightarrow S V_1S \mid V_1 V_1$	$B \rightarrow S V_5, V_5 \rightarrow V_1 S$
	$B \rightarrow V_1 V_1$
$V_1 \rightarrow b$	$V_1 \rightarrow b$
$V_2 \rightarrow a$	$V_2 \rightarrow a$

Converting to GNF :

Step 1 : Substituting symbols, we get,

$S \rightarrow ASB \mid a \mid b X_1$

$A \rightarrow a S A \mid a$

$B \rightarrow S X_2 S \mid b X_1$

$X_1 \rightarrow b$

$X_2 \rightarrow a$

Step 2 : Re-writing production in GNF :

	Productions	**CNF forms**
(1)	$X_1 \rightarrow b$	$X_1 \rightarrow b$
(2)	$X_2 \rightarrow a$	$X_2 \rightarrow a$
(3)	$A \rightarrow aSA \mid a$	$A \rightarrow a S A \mid a$
(4)	$S \rightarrow ASB \mid a \mid b X_1$	$S \rightarrow aSASB \mid aSB$ [substituting A]
		$S \rightarrow a \mid b X_1$
(5)	$B \rightarrow SX_2 S \mid bX$	$S \rightarrow aSASBX_2S \mid aSBX_2S \mid aX_2S \mid b X_1 X_2 S$ [substituting for S]
		$S \rightarrow bX$

Scanned by CamScanner

Dec. 2015

Q. 59 Consider the following grammar G = (V, T, P, S), V = {S, X, Y}, T = {a, b} and productions P are

$S \rightarrow XYX$

$X \rightarrow aX|\epsilon$

$Y \rightarrow bY|\epsilon$

Convert this grammar in Chomsky Normal Form (CNF).

(Similar to Example 5.5.2) **(10 Marks)**

Q. 60 State and prove pumping lemma for context free language.

(Section 5.6) **(10 Marks)**

Q. 61 Consider the following grammar G = (V, T P, S), V = (S, X), T = {0, 1} and productions P are

$S \rightarrow 0 \mid 0X1 \mid 01S1$

$X \rightarrow 0XX1 \mid 1S$

S is start symbol. Show that above grammar is ambiguous. **(5 Marks)**

Ans. : **[Hint : Refer as Example 5.3.12(A)]**

A grammar is said to be ambiguous grammar if the language generated by the grammar contains some strings that has 2 parse trees.

Ex. : Let us consider the given grammar

$S \rightarrow 0 \mid 0X1 \mid 01S1$

$X \rightarrow 0XX1 \mid 1S$

where, S is the start symbol.

A string 010011 is generated by the given grammar.

The grammar generates the string 010011 in 2 different ways. The 2 deviations are shown in Fig. 1(a)-Q. 61 and Fig. 1(b)-Q. 61. As the same string has 2 different parse trees. The given grammar is ambiguous grammar.

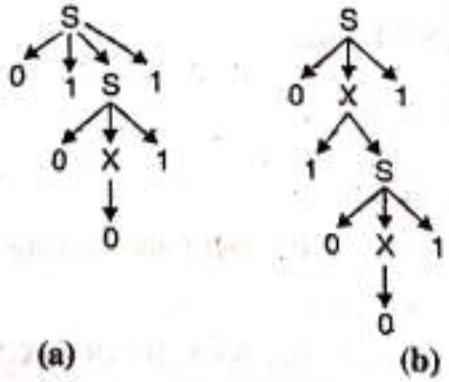


Fig. I-Q. 61

Scanned by CamScanner

May 2016

Q. 62 State and prove pumping lemma for context free languages. **(Section 5.6)** **(10 Marks)**

Q. 63 Consider the following grammar G = (V, T, P, S) , V = {S, X}, T = {a, b} and productions P are

$S \rightarrow aSb \mid aX$

$X \rightarrow Xa \mid Sa \mid a$

Convert this grammar in Greibach Normal Form (GNF). **(10 Marks)**

Ans. : [Hint : Refer Example as Example 5.5.22(A)]

Given set of productions

$S \rightarrow aSb \mid aX$

$X \rightarrow Xa \mid Sa \mid a$

Substituting C_a for a, C_b for b, A_1 for S and A_2 for X.

$A_1 \rightarrow a\,A_1\,C_b \mid a\,A_2$

$A_2 \rightarrow A_2\,C_a \mid A_1\,C_a \mid a$

$C_a \rightarrow a$

$C_b \rightarrow b$

Removing left recursion form A_2 production, we get

$C_a \rightarrow a, \quad C_b \rightarrow b$

$A_1 \rightarrow a\,A_1\,C_b \mid a\,A_2$

$A_2 \rightarrow A_1\,C_a\,A_3 \mid a\,A_3 \mid A_1\,C_a \mid a$

$A_3 \rightarrow C_a\,A_3 \mid A_2$

Re-writing productions in GNF from

$A_1 \rightarrow a\,A_1\,C_b \mid a\,A_2$

$A_2 \rightarrow a\,A_1\,C_b\,C_a\,A_3 \mid a\,A_2\,C_a\,A_3 \mid a\,A_3 \mid a\,A_1\,C_b\,C_a \mid a\,A_2\,C_a \mid a$

$A_3 \rightarrow a\,A_3 \mid a\,A_1\,C_b\,C_a\,A_3 \mid a\,A_2\,C_a\,A_3 \mid a\,A_3 \mid a\,A_1\,C_b\,C_a \mid a\,A_2\,C_a \mid a$

$C_a \rightarrow a$

$C_b \rightarrow b$

Scanned by CamScanner

Dec. 2016

Q. 65 Convert the following CFG to GNF **(10 Marks)**

$S \rightarrow AA \mid a$

$A \rightarrow SS \mid b$ **(Example 5.5.11)**

May 2017

Q. 66 Explain Pumping Lemma for CFG. **(Section 5.6)** **(5 Marks)**

Q. 67 Write short note on : Greibach Normal form. **(Section 5.5.2)** **(5 Marks)**

Dec. 2017

Q. 68 Write short note on Closure properties of Context Free Language. **(Section 5.7.1)** **(5 Marks)**

Q. 69 Consider the following grammar : **(10 Marks)**

$S \rightarrow iCtS \mid iCtSeS \mid a$

$C \rightarrow b$

For the string 'ibtibtaea' find the following :

(i) Leftmost derivation

(ii) Rightmost derivation

(iii) Parse tree

(iv) Check if above grammar is ambiguous.

Ans. : [Hint : Refer Example as Example 5.3.4(A)]

(i) Leftmost derivation

$$S \Rightarrow iCtSeS \xrightarrow{C \rightarrow b} ibtSeS \xrightarrow{S \rightarrow iCtS}$$

$$ibtiCtSeS \xrightarrow{C \rightarrow b} ibtibtSeS \xrightarrow{S \rightarrow a} ibtibtaeS$$

$$\Rightarrow ibtibtaea$$

(ii) Rightmost derivation

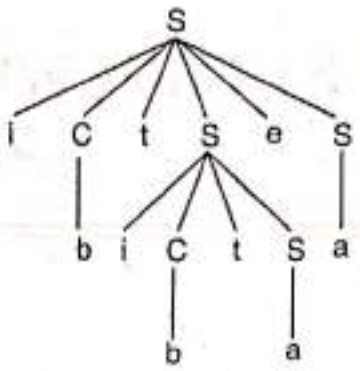
$$S \Rightarrow iCtSeS \xrightarrow{S \rightarrow a} iCtSea \xrightarrow{S \rightarrow iCtS}$$

$$iCtiCtSea \xrightarrow{S \rightarrow a} iCtiCtaea$$

$$\xrightarrow{C \rightarrow b} iCtibtaea \xrightarrow{C \rightarrow b} ibtibtaea$$

Scanned by CamScanner

(iii) Parse tree



(iv) It is an ambiguous grammar due to laughing if problem.

Q. 70 Reduce following grammar to GNF. **(10 Marks)**

$S \rightarrow AB$

$A \rightarrow BSB|BB|b$

$B \rightarrow a|aAb$

(i) $S \rightarrow 01S|01$

$S \rightarrow 10S|10$

$S \rightarrow 00|\in$

Ans. : [Hint : Refer Example as Example 5.5.22(A)]

Removing $\in$-production, we get,

$S \rightarrow 01S|01|10S|10|00$

It can be converted into GNF in an easy way by introducing two production

$X \rightarrow 1$ and $Y \rightarrow 0$

$\therefore$ Productions in GNF are

$S \rightarrow 0XS|0X|1YS|1Y|0Y$

$X \rightarrow 1$

$Y \rightarrow 0$

Chapter Ends...

❑❑❑

Scanned by CamScanner

CHAPTER 6

Module 4

Pushdown Automata (PDA)

6.1 Introduction to Pushdown Automata (PDA)

MU - Dec. 2005, Dec. 2012, May 2015

Informally, pushdown automata can be viewed as finite automata with stack. An added stack provides memory and increases the capability of the machine.

A pushdown automata can do the followings :

1. Read input symbol [as in case of FA]
2. Perform stack operations.
   - 2.1 Push operation
   - 2.2 Pop operation
   - 2.3 Check empty condition of a stack through an initial stack symbol.
   - 2.4 Read top symbol of stack without a pop.
3. Make state changes.

PDA is more powerful than FA. A context-free language (CFL) can be recognized by a PDA. Only a subset of CFL that are regular can be recognized by finite automata.

- A context free language can be recognized by PDA.
- For every context-free language, there exists a PDA.
- The language of PDA is a context-free language.

Example : A string of the form $a^n b^n$ can not be handled by a finite automata. But the same can be handled by a PDA.

- Any machine recognizing a string of the form $a^n b^n$, must keep track of a's [first half of $a^n b^n$] as number of b's must be equal to the number of a's.
- First half of the string can be remembered through a stack.

Scanned by CamScanner

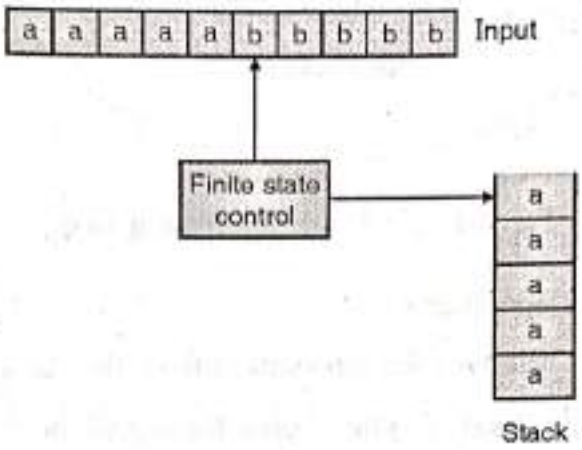


Fig. 6.1.1 : Stack after reading the first half of a^5b^5

- As the machine reads the first half of a^nb^n, it remembers it by pushing it on top of the stack. As shown in Fig. 6.1.1, after reading first 5 a's, the stack contains 5 a's.
- While reading the second half of the input string consisting of b's, the machine pops out an 'a' from the stack for every 'b' as input.
- After reading 5 b's, input will finish and the stack will become empty. This will indicate that the input string is of the form $a^n b^n$.
- The machine will have two states q_0 and q_1.

  State q_0 – while the machine is reading a's

  State q_1 – While the machine is reading b's.

  While in state q_1; an input 'a' is not allowed and hence there is a need for two states.
- A transition in PDA depends on :
  1. Current state
  2. Current input
  3. Top symbol of the stack.

A transition in PDA can be shown as a directed edge from the state q_i to q_j. While moving to state q_j, the machine can also perform stack operation. A transition edge from q_i to q_j should be marked with current input, current stack symbol (topmost symbol of the stack) and the stack operation. It is shown in Fig. 6.1.2.

Scanned by CamScanner

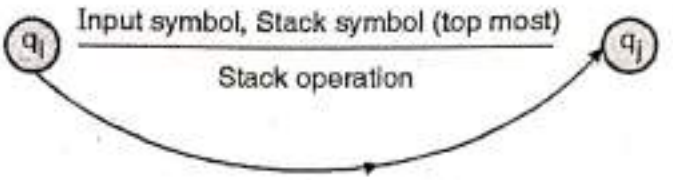


Fig. 6.1.2 : A transition from q_i to q_j

- A PDA uses three stack operations :
  1) **Pop** operation, it removes the top symbol from the stack.
  2) **Push** operation, it inserts a symbol onto the top of the stack.
  3) **Nop** operation, it does nothing to stack.
- The language $\{a^n b^n \mid n \geq 1\}$ can be accepted by the PDA of Fig. 6.1.3.

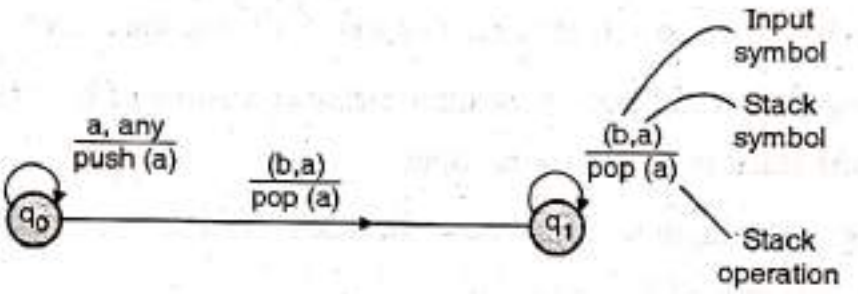


Fig. 6.1.3 : PDA for $a^n b^n$

- The state q_0 will keep track of the number of a's in an input string, by pushing symbol 'a' onto the stack for each input 'a'. A second state q_1 is used to pop an 'a' from the stack for each input symbol 'b'. Finally, after consuming the entire input the stack will become empty.

6.2 The Formal Definition of PDA

MU - Dec. 2016

A pushdown automata M is defined as 7-tuple :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$$

Where,

Q = The set of states

Σ = Input alphabet

Γ = Stack symbols

δ = The transition function is a transition form $Q \times (\Sigma \cup \varepsilon) \times \Gamma$ to $Q \times \Gamma^*$

q_0 = $q_0 \in Q$ is the initial state

F = $F \subseteq Q$ is the set of final states

Scanned by CamScanner

z_0 = An initial stack symbol

Transition function in detail :

A transition function is a mapping from

$Q \times (\Sigma \cup \varepsilon) \times \Gamma$ to $Q \times \Gamma^*$

Where

$Q \times (\Sigma \cup \varepsilon) \times \Gamma$ implies that a transition is based on :

1. Current state
2. Next input (including ε)
3. Stack symbol (topmost)

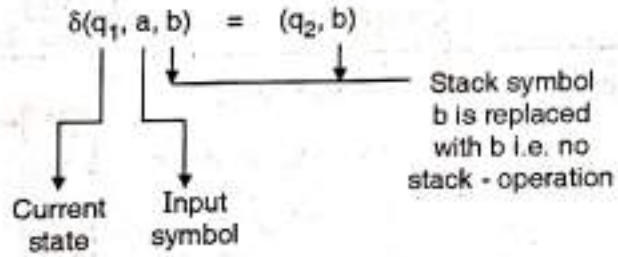
and

$Q \times \Gamma^*$ implies that :

1. The next state could be any state belonging to Q.
2. It can perform push, pop or no-operation (NOP) on stack.

In general, we can have the following types of transition behaviours.

1. Read input with no-operation on stack :

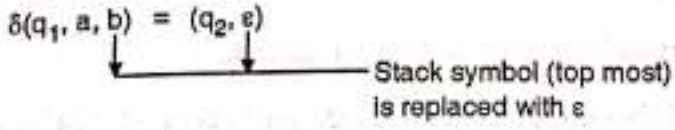


- Left hand side of the transition, $\delta (q_1, a, b)$ implies :
  - q_1 is the current state,
  - a is the current input,
  - b is the stack symbol (topmost).
- Right hand side of the transition, (q_2, b) implies that

[The machine enters a new state q_2. The top symbol of the stack , which is b is replaced with b.]

Thus, the transition $\delta(q_1, a, b) = (q_2, b)$ does not modify a stack.

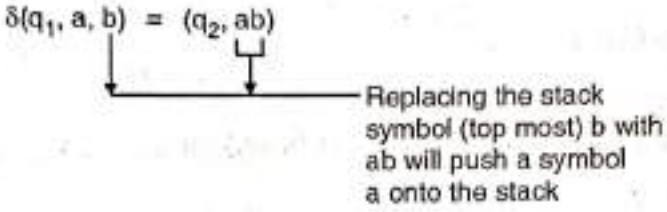
2. Pop operation :



Scanned by CamScanner

The above transition will erase the stack symbol (topmost). Replacing, b with ε amounts to erasing b from the stack top.

3. Push operation :



The above transition will perform a push operation. It will push 'a' onto the stack. Replacing 'b' with 'ab' amounts to push 'a' onto the stack.

The three operations for stack are shown in Table 6.2.1.

Table 6.2.1 : Transitions rules for stack operations

Stack before transition (assumed)	Transition rule	Stack after transition
1.	$\delta(q_1, a, b) = (q_2, b)$	No - operation
2.	$\delta(q_1, a, b) = (q_2, \varepsilon)$	Pop - operation
3.	$\delta(q_1, a, b) = (q_2, ab)$	Push - operation

Example : Transition function for a PDA accepting a language

$L = \{\omega \in \{a, b\}^* \mid \omega \text{ is of the } a^n b^n \text{ with } n \geq 1\}$, can be givgen in various forms :

1. Using a set of rules.
2. Using graphical representation (transition diagram)

Using a set of equations :

$\delta(q_0, a, z_0) = (q_0, az_0)$ [First a of $a^n b^n$ is pushed onto the stack]

$\delta(q_0, a, a) = (q_0, aa)$ [Subsequent a's of $a^n b^n$ are pushed one by one onto the stack.]

$\delta(q_0, b, a) = (q_1, \varepsilon)$ [On Seeing the first b, the machine will make a move to q_1 with a POP]

Scanned by CamScanner

$\delta(q_1, b, a) = (q_1, \varepsilon)$ [For each subsequent b, a POP operation is performed]

$\delta(q_1, \varepsilon, z_0) = (q_2, z_0)$ [After the end of input string, machine will transit to a final state]

Note : Failure transitions are not shown as it is a standard practice.

The transition rules mentioned above can be written in two different forms.

	Form 1		Form 2
$\delta(q_0, a, z_0)$	$= (q_0, a\, z_0)$	$\Rightarrow$	$((q_0, az_0), (q_0, az_0))$
$\delta(q_0, a, a)$	$= (q_0, aa)$	$\Rightarrow$	$((q_0, a, a), (q_0, aa))$
$\delta(q_0, b, a)$	$= (q_1, \varepsilon)$	$\Rightarrow$	$((q_0, b, a), (q_1, \varepsilon))$
$\delta(q_1, b, a)$	$= (q_1, \varepsilon)$	$\Rightarrow$	$((q_1, b, a), (q_1, \varepsilon))$
$\delta(q_1, \varepsilon, z_0)$	$= (q_2, z_0)$	$\Rightarrow$	$((q_1, \varepsilon, z_0), (q_2, z_0))$

Using graphical representation (transition diagram) :

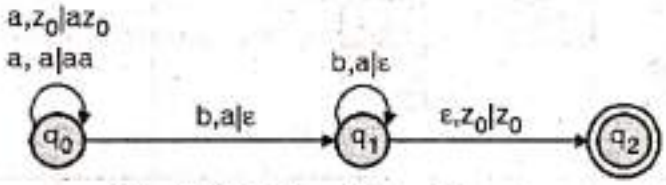


Fig. 6.2.1 : Transition diagram

6.3 Instantaneous Description of a PDA

Instantaneous behaviour is useful in showing the processing of a string by PDA. Processing of the input string a^3b^3 by the PDA of Fig. 6.2.1 is shown in Fig. 6.3.1.

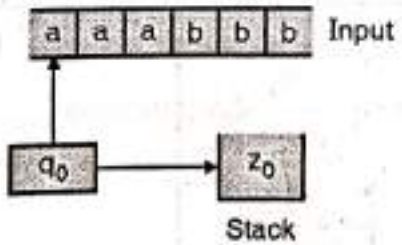
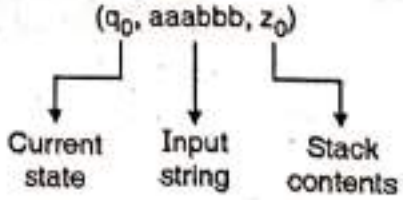
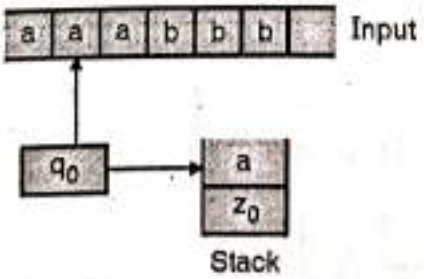
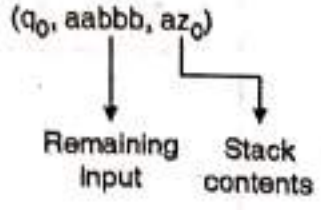
Sr. No.	Graphical representation (instantaneous behaviour)	Tuple format (instantaneous behaviour)
1.		
2.		

Fig. 6.3.1 contd....

Scanned by CamScanner

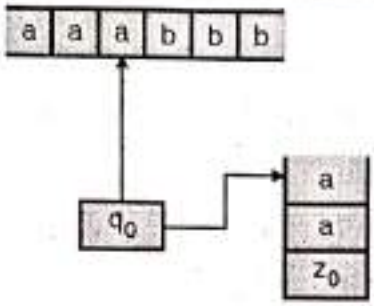
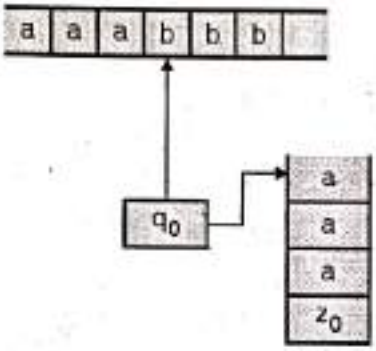
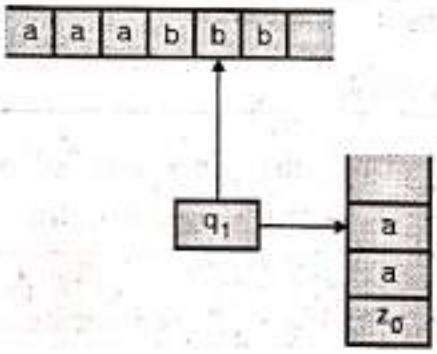
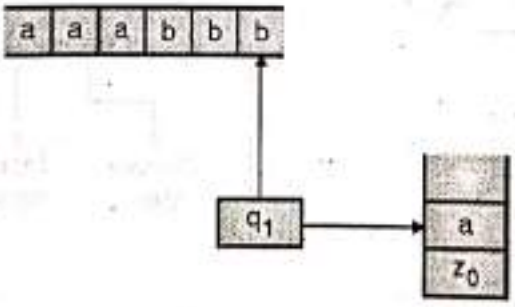
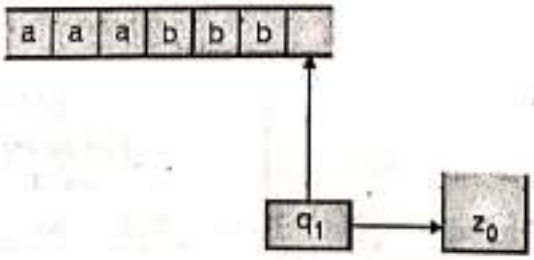
Sr. No.	Graphical representation (instantaneous behaviour)	Tuple format (instantaneous behaviour)
3.		$(q_0, abbb, aaz_0)$
4.		$(q_0, bbb, aaaz_0)$
5.		(q_1, bb, aaz_0)
6.		(q_1, b, aZ_0)
7.		(q_1, ε, z_0)

Fig. 6.3.1 contd....

Scanned by CamScanner

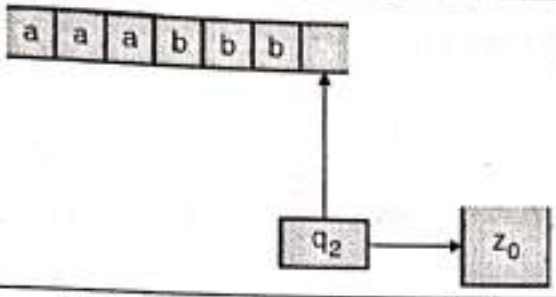
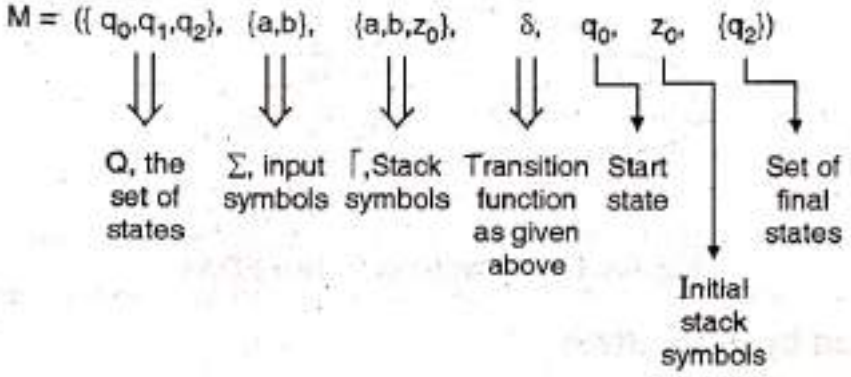
Sr. No.	Graphical representation (instantaneous behaviour)	Tuple format (instantaneous behaviour)
8.		(q_2, ε, z_0)

Fig. 6.3.1 : Processing of a^3b^3 by the PDA of Fig. 6.2.1

The PDA for recognizing a string of the form a^nb^n can be described as:



Example 6.3.1 : Suppose the PDA M = ({q_0, q_1}, {a, b, c}, {a, b, Z_0}, δ, q_0, Z_0, {q_1}) has the following transition function.

1. $\delta(q_0, a, \varepsilon) = (q_0, a)$
2. $\delta(q_0, b, \varepsilon) = (q_0, b)$
3. $\delta(q_0, c, \varepsilon) = (q_1, \varepsilon)$
4. $\delta(q_1, a, a) = (q_1, \varepsilon)$
5. $\delta(q_1, b, b) = (q_1, \varepsilon)$

Show the acceptance of abbcbba by the above PDA through an instantaneous description.

Solution :

$$(q_0, abbcbba, z_0) \xrightarrow{\text{(Rule 1)}} (q_0, bbcbba, az_0)$$
$$\xrightarrow{\text{(Rule 2)}} (q_0, bcbba, baz_0)$$
$$\xrightarrow{\text{(Rule 2)}} (q_0, cbba, bbaz_0)$$
$$\xrightarrow{\text{(Rule 3)}} (q_1, bba, bbaz_0)$$
$$\xrightarrow{\text{(Rule 5)}} (q_1, ba, baz_0)$$
$$\xrightarrow{\text{(Rule 5)}} (q_1, a, az_0)$$
$$\xrightarrow{\text{(Rule 4)}} (q_1, \varepsilon, z_0)$$

Since, the machine is in a final state q_1, the string abbcbba is accepted.

Scanned by CamScanner

6.4 The Language of a PDA

MU - Dec. 2008

A language L can be accepted by a PDA in two ways :

1. Through final state.
2. Through empty stack.

It is possible to convert between the two classes.

1. From final state to empty stack.
2. From empty stack to final state.

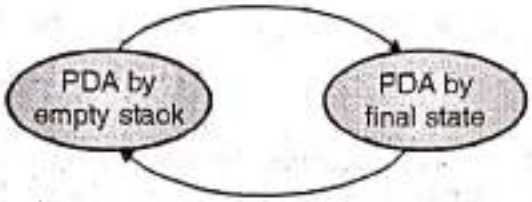


Fig. 6.4.1 : Equivalence of two PDAs

6.4.1 Acceptance by Final State

Let the PDA, $M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$ then the language accepted by M through a final state is given by :

$$L(M) = \left\{ w \mid (q_0, w, z_0) \overset{*}{\underset{M}{\Rightarrow}} (q_1, \varepsilon, \alpha) \right\}$$

Where the state $q_1 \in F$. α, the final contents of the stack are irrelevant as a string is accepted through a final state.

6.4.2 Acceptance by Empty Stack

Let the PDA, $M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, \phi)$ then the language accepted through an empty stack is given by :

$$L(M) = \left\{ w \mid (q_0, w, z_0) \overset{*}{\underset{M}{\Rightarrow}} (q_1, \varepsilon, \varepsilon) \right\}$$

Where q_1 is any state belonging to Q and the stack becomes empty on application of input string w.

Example 6.4.1 : Give a PDA to accept the language

$L = \{0^n 1^m \mid n \leq m\}$

1. Through empty stack.
2. Through final state.

Scanned by CamScanner

Solution :

Algorithm :

1. Sequence of 0's should be pushed onto the stack in state q_0.

   $\delta(q_0, 0, z_0) = (q_0, 0z_0)$ [Push the first 0]

   $\delta(q_0, 0, 0) = (q_0, 00)$ [Push subsequent 0's]

2. A '0' should be popped for every 1 as input till the stack becomes empty.

   $\delta(q_0, 1, 0) = (q_1, \varepsilon)$ [Pop on first 1 and change the state to q_1]

   $\delta(q_1, 1, 0) = (q_1, \varepsilon)$ [Pop on subsequent 1 as input till every 0 is erased from the stack]

3. Subsequent 1's (m – n) will have no effect on the stack.

   $\delta(q_1, 1, z_0) = (q_1, z_0)$

4. Finally, the symbol Z_0 should be popped out to make the stack empty.

   $\delta(q_1, \varepsilon, z_0) = (q_1, \varepsilon)$

   This step is required if the language is to be accepted through an empty stack.

Transition behaviour of the PDA is shown in Fig. Ex. 6.4.1(a to d)

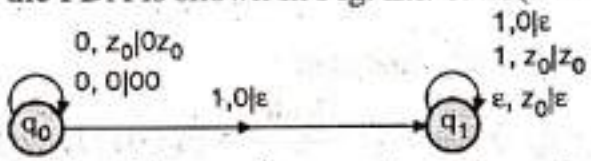


Fig. Ex. 6.4.1(a) : Transition diagram for acceptance through an empty stack

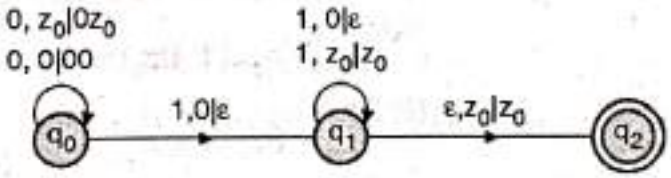


Fig. Ex. 6.4.1(b) : Transition diagram for acceptance through a final state

1.	$\delta(q_0, 0, z_0)$	$= (q_0, 0z_0)$
2.	$\delta(q_0, 0, 0)$	$= (q_0, 00)$
3.	$\delta(q_0, 1, 0)$	$= (q_1, \varepsilon)$
4.	$\delta(q_1, 1, 0)$	$= (q_1, \varepsilon)$
5.	$\delta(q_1, 1, z_0)$	$= (q_1, z_0)$
6.	$\delta(q_1, \varepsilon, z_0)$	$= (q_1, \varepsilon)$

Fig. Ex. 6.4.1(c) : State transition rules for acceptance through an empty stack

1.	$\delta(q_0, 0, z_0)$	$= (q_0, 0z_0)$
2.	$\delta(q_0, 0, 0)$	$= (q_0, 00)$
3.	$\delta(q_0, 1, 0)$	$= (q_1, \varepsilon)$
4.	$\delta(q_1, 1, 0)$	$= (q_1, \varepsilon)$
5.	$\delta(q_1, 1, z_0)$	$= (q_1, z_0)$
6.	$\delta(q_1, \varepsilon, z_0)$	$= (q_2, z_0)$

Fig. Ex. 6.4.1(d) : State transition rules for acceptance through a final state

The PDA accepting through an empty stack is given by :

$$M = (\{q_0, q_1\}, \{0, 1\}, \{0, 1, z_0\}, \delta, q_0, z_0, \phi)$$

Scanned by CamScanner

Where δ is given in Fig. Ex. 6.4.1(c).

The PDA accepting through final state is given by :

$$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \{0, 1, z_0\}, \delta, q_0, z_0, \{q_2\})$$

Where δ is given in Fig. Ex. 6.4.1(d).

Example : Processing of string 00111 by the PDA

Case I : Acceptance through empty stack.

$$(q_0, 00111, z_0) \xrightarrow{\text{(Rule 1)}} (q_0, 0111, 0z_0)$$
$$\xrightarrow{\text{(Rule 2)}} (q_0, 111, 00z_0)$$
$$\xrightarrow{\text{(Rule 3)}} (q_1, 11, 0z_0)$$
$$\xrightarrow{\text{(Rule 4)}} (q_1, 1, z_0)$$
$$\xrightarrow{\text{(Rule 5)}} (q_1, \varepsilon, z_0)$$
$$\xrightarrow{\text{(Rule 6)}} (q_1, \varepsilon, \varepsilon)$$

Case II : Acceptance through final state :

$$(q_0, 00111, z_0) \xrightarrow{\text{(Rule 1)}} (q_0, 0111, 0z_0)$$
$$\xrightarrow{\text{(Rule 2)}} (q_0, 111, 00z_0)$$
$$\xrightarrow{\text{(Rule 3)}} (q_1, 11, 0z_0)$$
$$\xrightarrow{\text{(Rule 4)}} (q_1, 1, z_0)$$
$$\xrightarrow{\text{(Rule 5)}} (q_1, \varepsilon, z_0)$$
$$\xrightarrow{\text{(Rule 6)}} (q_2, \varepsilon, z_0)$$

Example 6.4.2 : Let $L = \{a^m b^n \mid n < m\}$. Construct,

i) PDA accepting L by empty store.

ii) PDA accepting L by final state.

Solution :

i) Algorithm (Accepting L by empty store)

1. Sequence of a's should be pushed onto the stack in state q_0.

$$\delta(q_0, a, z_0) = (q_0, az_0)$$
$$\delta(q_0, a, a) = (q_0, aa)$$

Scanned by CamScanner

2. An 'a' should be popped for every b as input till the end of input.

$$\delta(q_0, b, a) = (q_1, \varepsilon)$$

$$\delta(q_1, b, a) = (q_1, \varepsilon)$$

3. Additional m – n a's should be popped from the stack.

$$\delta(q_1, \varepsilon, a) = (q_1, \varepsilon)$$

4. Finally, the symbol z_0 should be popped out to make the state empty.

$$\delta(q_1, \varepsilon, z_0) = (q_1, \varepsilon)$$

Transition diagram is given in Fig. Ex. 6.4.2 and transition table in Table Ex. 6.4.2

Table Ex. 6.4.2

$\delta(q_0,a,z_0) = (q_0,az_0)$
$\delta(q_0,a,a) = (q_0,aa)$
$\delta(q_0,b,a) = (q_1,\varepsilon)$
$\delta(q_1,b,a) = (q_1,\varepsilon)$
$\delta(q_1,\varepsilon,a) = (q_1,\varepsilon)$
$\delta(q_1,\varepsilon,z_0) = (q_1,\varepsilon)$

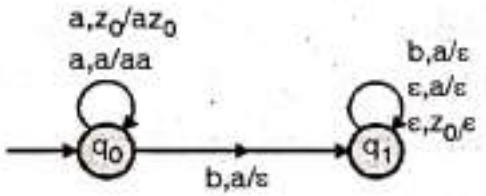


Fig. Ex. 6.4.2

$$M = (\{q_0, q_1\}, \{a, b\}, \{a, b, z_0\}, \delta, q_0, z_0, \phi)$$

ii) Accepting L through final state

Following modifications are required to accept through a final state.

In step3, when the input ends with stack containing m-n a's, the machine should enter the final state.

Table Ex. 6.4.2(a)

$\delta(q_0,a,z_0) = (q_0,az_0)$
$\delta(q_0,a,a) = (q_0,aa)$
$\delta(q_0,b,a) = (q_1,\varepsilon)$
$\delta(q_1,b,a) = (q_1, \varepsilon)$
$\delta(q_1, \varepsilon,a) = (q_2, \varepsilon)$

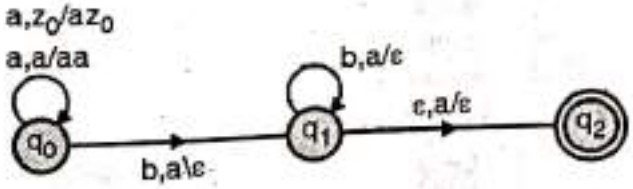


Fig. Ex. 6.4.2(a)

$$M = (\{q_0, q_1\}, \{a, b\}, \{a, b, z_0\}, \delta, q_0, z_0, \{q_2\})$$

Example 6.4.3 : Design a PDA for accepting the set of all strings over {a, b} with an equal number of a's and b's. The string should be accepted both by

(1) Final state (2) Empty stack.

Scanned by CamScanner

Solution :

- Stack will be used to store excess of a's over b's or excess of b's over a's out of input seen so far.
- The status of stack on input abaabbbbaa is shown in the Fig. Ex. 6.4.3.

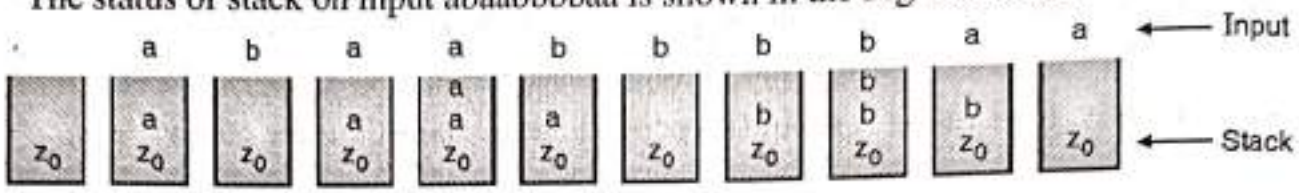


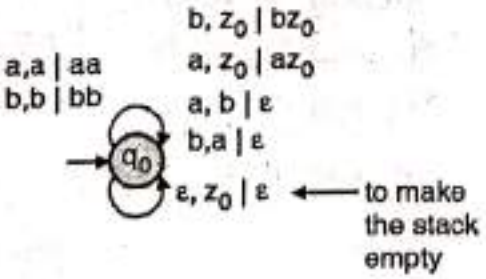
Fig. Ex. 6.4.3 : Stack preserving excess of a's over b's or b's over a's

Algorithm :

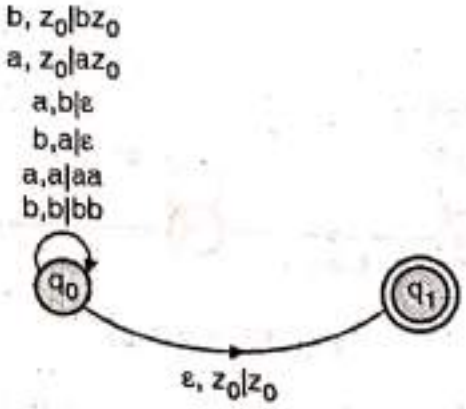
At any point of time, there could be one the six situations :

1. Stack contains z_0(topmost) and the input is a
   $\delta(q_0, a, z_0) = (q_0, az_0)$ [Extra a is pushed]
2. Stack contain z_0(topmost) and the input is b
   $\delta(q_0, b, z_0) = (q_0, bz_0)$ [Extra b is pushed]
3. Stack contains a(topmost) and the input is a.
   $\delta(q_0, a, a) = (q_0, aa)$ [Excess a's will increase by 1]
4. Stack contains a(topmost) and the input is b
   $\delta(q_0, b, a) = (q_0, \varepsilon)$ [Excess a's will decrease by 1]
5. Stack contains b(topmost) and the input is a.
   $\delta(q_0, a, b) = (q_0, \varepsilon)$ [Excess b's will decrease by 1]
6. Stack contains b(topmost) and the input is b.
   $\delta(q_0, b, b) = (q_0, bb)$ [Excess b's will increase by 1]

The PDA is shown in Fig. Ex. 6.4.3(a,b).



(a) Transition diagram for the PDA accepting through an empty stack



(b) Transition diagram for the PDA accepting through a final state

Fig. Ex. 6.4.3

Scanned by CamScanner

$\delta(q_0, a, z_0) = (q_0, az_0)$
$\delta(q_0, b, z_0) = (q_0, bz_0)$
$\delta(q_0, a, b) = (q_0, \varepsilon)$
$\delta(q_0, b, a) = (q_0, \varepsilon)$
$\delta(q_0, a, a) = (q_0, aa)$
$\delta(q_0, b, b) = (q_0, bb)$
$\delta(q_0, \varepsilon, z_0) = (q_0, \varepsilon)$

Fig. Ex. 6.4.3(c) : Transition rules for the PDA accepting through empty stack

$\delta(q_0, a, z_0) = (q_0, az_0)$
$\delta(q_0, b, z_0) = (q_0, bz_0)$
$\delta(q_0, a, b) = (q_0, \varepsilon)$
$\delta(q_0, b, a) = (q_0, \varepsilon)$
$\delta(q_0, a, a) = (q_0, aa)$
$\delta(q_0, b, b) = (q_0, bb)$
$\delta(q_0, \varepsilon, z_0) = (q_1, z_0)$

Fig. Ex. 6.4.3(d) : Transition rules for the PDA accepting through final state

Fig. Ex. 6.4.3 (a to d) shows state transition behaviour of PDA of Example 6.4.3.

The PDA accepting through empty stack is given by :

$$M = (\{q_0\}\ \{a, b\}\ \{a, b, z_0\}, \delta, q_0, z_0, \phi)$$

Where, δ is given in Fig. Ex. 6.4.3(c).

The PDA accepting through final state is given by :

$$M = (\{q_0, q_1\}, \{a, b\}, \{a, b, z_0\}, \delta, q_0, z_0, \{q_1\}),$$

Where, δ is given in Fig. Ex. 6.4.3(d).

Example 6.4.4 : Design a PDA to accept $(ab)^n(cd)^n$.

MU - May 2007

Solution : To solve this problem, we can take a stack symbol x. For every 'ab', one x will be pushed on top of the stack. After reading $(ab)^n$, the stack should contain n number of x's. These x's will be matched with $(cd)^n$. For every 'cd' one x will be popped.

The transitions for the PDA accepting through an empty stack are given in Fig. Ex.6.4.4.

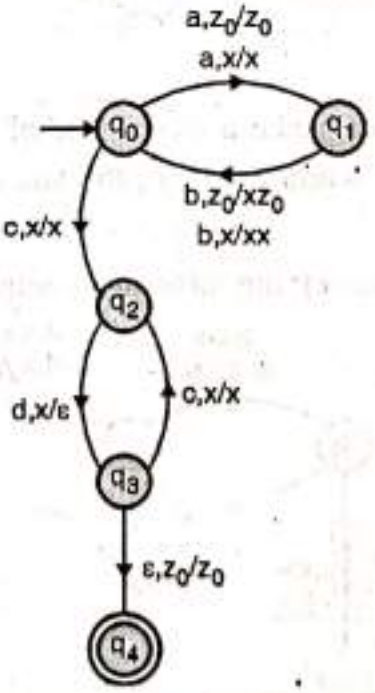


Fig. Ex. 6.4.4

Scanned by CamScanner

- PDA accepts through the final state q_4.

$$\text{The PDA } M = \{Q, \Sigma, \Gamma, \delta, q_0, z_0, F\}$$

Where,

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{a, b, c, d\}$$

$$\Gamma = \{x, z_0\}$$

The transition function δ is given by,

$$\delta(q_0, a, z_0) = (q_1, z_0)$$

$$\delta(q_0, a, x) = (q_1, x)$$

$$\delta(q_1, b, z_0) = (q_0, x\, z_0)$$

$$\delta(q_1, b, x) = (q_0, xx)$$

$$\delta(q_0, c, x) = (q_2, x)$$

$$\delta(q_2, d, x) = (q_3, \varepsilon)$$

$$\delta(q_3, c, x) = (q_2, x)$$

$$\delta(q_2, \varepsilon, z_0) = (q_4, z_0)$$

q_0 is initial state,

z_0 is initial stack symbol.

Set of final states $F = \{q_4\}$

Example 6.4.5 : Design a PDA to accept $(bdb)^n c^n$

MU - Dec. 2005

Solution :

To solve this problem, we can take a stack symbol x. For every 'bdb', one x will be pushed on top of the stack. After reading $(bdb)^n$, the stack should contain n number of x's. These x's will be matched with c's.

The transitions for the PDA accepting through an empty stack are given in Fig. Ex.6.4.5.

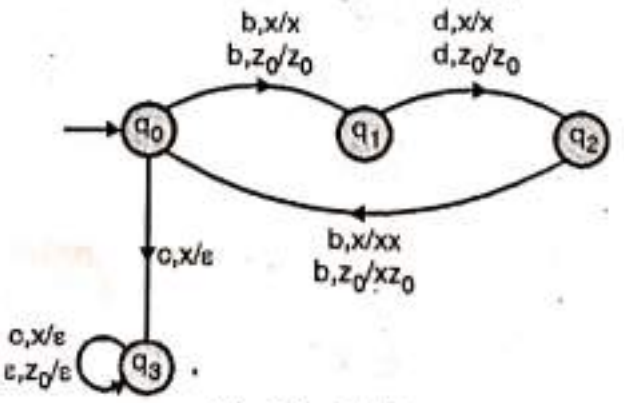


Fig. Ex. 6.4.5

Scanned by CamScanner

- A cycle through $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_0$ traces a group of bdb.

The PDA $M = \{Q, \Sigma, \Gamma, \delta, q_0, z_0, \phi\}$

Where,

$Q = \{q_0, q_1, q_2, q_3\}$, $\Sigma = \{b, d, c\}$, $\Gamma = \{x, z_0\}$

q_0 is the initial state, z_0 is initial stack symbol.

The transition function δ is given by,

$$\delta(q_0, b, z_0) = (q_1, z_0)$$
$$\delta(q_0, b, x) = (q_1, x)$$
$$\delta(q_1, d, z_0) = (q_2, z_0)$$
$$\delta(q_1, d, x) = (q_2, x)$$
$$\delta(q_2, b, z_0) = (q_0, xz_0)$$
$$\delta(q_2, b, x) = (q_0, xx)$$
$$\delta(q_0, c, x) = (q_3, \varepsilon)$$
$$\delta(q_3, c, x) = (q_3, \varepsilon)$$
$$\delta(q_3, \varepsilon, z_0) = (q_3, \varepsilon)$$ Accept through empty stack.

Example 6.4.6 : Construct pushdown automata for the following language :

L = {the set of strings over alphabet {a, b} with exactly twice as many a's as b's}

Solution :

To solve this example, we can take three stack symbols :

$\Gamma = (x, y, z_0)$

Where, x stands for 2 a's,
y stands for a 'b',
z_0 is initial stack symbol.

- Stack will be used to store excess of x's over y's or excess of y's over x's.
- Since, a 'x' will be pushed onto the stack after 2 a's, after first 'a' the machine will transit to q_1 to remember that one 'a' has already come and the second 'a' in q_1 will complete 2 a's.
- Status of the stack and the state of the machine is shown in Fig. Ex. 6.4.6. Input applied to the machine is abaabbaaaaabaab.

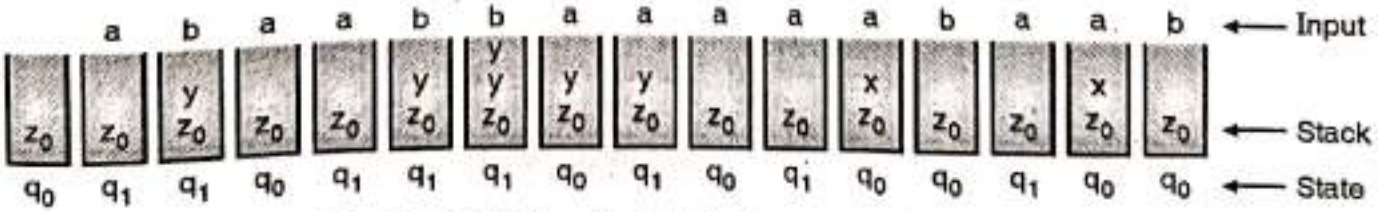


Fig. Ex. 6.4.6 : Status of stack for input abaabbaaaaabaab

Scanned by CamScanner

Implementation of PDA using final state :

Let the PDA, $M = \{Q, \Sigma, \Gamma, \delta, q_0, z_0, F\}$

Where,

$$Q = \{q_0, q_1, q_2\}, \Sigma = \{a, b\}, \Gamma = \{x, y, z_0\}$$

q_0 is initial state, z_0 is initial stack symbol.

$$F = \{q_2\}$$

Transition function δ is given by :

1.	$\delta(q_0, a, z_0) = (q_1, z_0)$	First input could be either a or b
2.	$\delta(q_0, b, z_0) = (q_0, yz_0)$	
3.	$\delta(q_0, a, x) = (q_1, x)$	[goes to q_1 to remember first a of aa]
4.	$\delta(q_0, a, y) = (q_1, y)$	[goes to q_1 to remember first a of 2 a's]
5.	$\delta(q_0, b, x) = (q_0, \varepsilon)$	[Excess x is removed]
6.	$\delta(q_0, b, y) = (q_0, yy)$	[Excess y increases by 1]
7.	$\delta(q_1, a, z_0) = (q_0, xz_0)$	[Push a x for 2 a's]
8.	$\delta(q_1, a, x) = (q_0, xx)$	[Push a x for 2 a's]
9.	$\delta(q_1, a, y) = (q_0, \varepsilon)$	[Two a's will remove a 'y']
10.	$\delta(q_1, b, z_0) = (q_1, yz_0)$	[Excess y is saved]
11.	$\delta(q_1, b, x) = (q_1, \varepsilon)$	[Excess x is removed]
12.	$\delta(q_1, b, y) = (q_1, yy)$	[Excess y increases by 1]
13.	$\delta(q_0, \varepsilon, z_0) = (q_2, z_0)$	[goes to final state to accept the string]

Example : Processing of string abaabbaaaaabaab by the PDA

$$(q_0, abaabbaaaaabaab, z_0) \xrightarrow{\text{(Rule 1)}} (q_1, baabbaaaaabaab, z_0)$$

$$\xrightarrow{\text{(Rule 10)}} (q_1, aabbaaaaabaab, yz_0)$$

$$\xrightarrow{\text{(Rule 9)}} (q_0, abbaaaaabaab, z_0)$$

$$\xrightarrow{\text{(Rule 1)}} (q_1, bbaaaaabaab, z_0)$$

$$\xrightarrow{\text{(Rule 10)}} (q_1, baaaaabaab, yz_0)$$

$$\xrightarrow{\text{(Rule 12)}} (q_1, aaaaabaab, yyz_0)$$

$$\xrightarrow{\text{(Rule 9)}} (q_0, aaaabaab, yz_0)$$

$$\xrightarrow{\text{(Rule 4)}} (q_1, aaabaab, yz_0)$$

Scanned by CamScanner

$$\xrightarrow{\text{(Rule 9)}} (q_0, aabaab, z_0)$$

$$\xrightarrow{\text{(Rule 1)}} (q_1, abaab, z_0)$$

$$\xrightarrow{\text{(Rule 7)}} (q_0, baab, xz_0)$$

$$\xrightarrow{\text{(Rule 5)}} (q_0, aab, z_0)$$

$$\xrightarrow{\text{(Rule 1)}} (q_1, ab, z_0)$$

$$\xrightarrow{\text{(Rule 7)}} (q_0, b, xz_0)$$

$$\xrightarrow{\text{(Rule 5)}} (q_0, \varepsilon, z_0)$$

$$\xrightarrow{\text{(Rule 13)}} (q_2, \varepsilon, z_0)$$

Example 6.4.7 : Give the transition table for PDA recognizing the following language

$$L = \{ a^n x \mid n \geq 0 \text{ and } x \in \{a, b\}^* \text{ and } |x| \leq n \}$$

Solution :

Algorithm :

- Sequence of initial a's should be pushed onto the stack in state q_0.
- For every symbol in x, an 'a' should be erased from the stack.

1. Push the first 'a' using the transition δ (q0, a, z0) = (q0, az0)
2. Push subsequent a's using the transition δ (q0, a, a) = (q0, aa).
3. On the first 'b' as input, the machine will transit to q_1 with a pop operation using the transition
   $\delta(q_0, b, a) = (q_1, \varepsilon)$.
4. On subsequent a's or b's in x, an 'a' should be erased from the stack.
   $\delta(q_1, a, a) = (q_1, \varepsilon)$
   $\delta(q_1, b, a) = (q_1, \varepsilon)$
5. After the end of input string, contents of stack should be erased one by one.
   $\delta(q_1, \varepsilon, a) = (q_1, \varepsilon)$
   $\delta(q_1, \varepsilon, z_0) = (q_1, \varepsilon)$

The PDA is given by :

$$M = (\{q_0, q_1\}, \{a, b\}, \{a, z_0\}, \delta, q_0, z_0, \phi)$$

Where, δ is given below :

$\delta(q_0, a, z_0) = (q_0, az_0)$

Scanned by CamScanner

$\delta(q_0, a, a) = (q_0, aa)$

$\delta(q_0, \varepsilon, z_0) = (q_0, \varepsilon)$ [To accept a null string]

$\delta(q_0, b, a) = (q_1, \varepsilon)$

$\delta(q_1, a, a) = (q_1, \varepsilon)$

$\delta(q_1, b, a) = (q_1, \varepsilon)$ erase, everything from the stack

$\delta(q_1, \varepsilon, a) = (q_1, \varepsilon)$

$\delta(q_1, \varepsilon, z_0) = (q_1, \varepsilon)$

Example 6.4.8 : Let $L = \{ a^n b^n c^m d^m \mid n, m \geq 1 \}$ find a PDA that accepts L.

Solution :

Algorithm :

1. Sequence of a's should be pushed onto the stack.
2. For every b as input, an 'a' should be erased from the stack.
3. Sequence of c's should be pushed onto the stack.
4. For every d as input, a 'c' should be erased from the stack.

The PDA is given by :

$$M = (\{q_0, q_1, q_2, q_3\}, \{a, b, c, d\}, \{a, c, z_0\}, \delta, q_0, z_0, \phi)$$

Where the transition function δ is given below :

$\delta(q_0, a, z_0) = (q_0, az_0)$ [Push the first a]

$\delta(q_0, a, a) = (q_0, aa)$ [Push remaining a's]

$\delta(q_0, b, a) = (q_1, \varepsilon)$ [Erase an 'a' on first b]

$\delta(q_1, b, a) = (q_1, \varepsilon)$ [Erase remaining a's on subsequent b's]

$\delta(q_1, c, z_0) = (q_2, cz_0)$ [First c is pushed]

$\delta(q_2, c, c) = (q_2, cc)$ [Subsequent c's are pushed]

$\delta(q_2, d, c) = (q_3, \varepsilon)$ [On first d, machine transits to q_3 with a pop]

$\delta(q_3, d, c) = (q_3, \varepsilon)$ [For every d, a 'c' is erased]

$\delta(q_3, \varepsilon, z_0) = (q_3, \varepsilon)$ [String is accepted through empty stack]

Example 6.4.9 : Let $L = \{ a^i b^j c^k \mid i, j, k \geq 0 \text{ and } i + j = k \}$

(i) Find a PDA accepting through final state.

(ii) Find a PDA accepting through empty stack.

Scanned by CamScanner

Solution :

Algorithm :

1. For every input symbol 'a', a symbol x is pushed onto the stack.
2. For every input symbol 'b', a symbol x is pushed onto the stack.
3. For every input symbol 'c', x is erased from the stack.

(i) PDA accepting through final state is given by

$$M = (\{q_0, q_1, q_2, q_3\}, \{a, b, c\}, \{x, z_0\}, q_0, z_0, \delta, \{q_3\}),$$

Where the transition function δ is :

1. $\delta(q_0, a, z_0) = (q_0, xz_0)$ [x is pushed for the first a]
2. $\delta(q_0, a, x) = (q_0, xx)$ [x is pushed for every subsequent a]
3. $\delta(q_0, b, z_0) = (q_1, xz_0)$ [First b without a's]
4. $\delta(q_0, b, x) = (q_1, xx)$ [First b after a's]
5. $\delta(q_1, b, x) = (q_1, xx)$ [Subsequent b's]
6. $\delta(q_1, c, x) = (q_2, \varepsilon)$ [x is erased for the first c]
7. $\delta(q_2, c, x) = (q_2, \varepsilon)$ [x is erased for subsequent c's]
8. $\delta(q_2, \varepsilon, z_0) = (q_3, z_0)$ [Accept through q_3]
9. $\delta(q_0, \varepsilon, z_0) = (q_3, z_0)$ [Accept a null string]

(ii) PDA accepting through empty stack is given by,

$$M = (\{q_0, q_1, q_2\}, \{a, b, c\}, \{x, z_0\}, q_0, z_0, \delta, \phi),$$

where the transition function δ is :

1. $\delta(q_0, a, z_0) = (q_0, xz_0)$
2. $\delta(q_0, a, x) = (q_0, xx)$
3. $\delta(q_0, b, z_0) = (q_1, xz_0)$
4. $\delta(q_0, b, x) = (q_1, xx)$
5. $\delta(q_1, b, x) = (q_1, xx)$
6. $\delta(q_1, c, x) = (q_2, \varepsilon)$
7. $\delta(q_2, c, x) = (q_2, \varepsilon)$
8. $\delta(q_2, \varepsilon, z_0) = (q_2, \varepsilon)$ [Stack is made empty]
9. $\delta(q_0, \varepsilon, z_0) = (q_0, \varepsilon)$ [Accept a null string]

Scanned by CamScanner

Example 6.4.10 : Construct a PDA accepting $\{a^n b^m a^n \mid m, n \geq 1\}$ by null store.

MU - Dec. 2006, Dec. 2010, May 2012, May 2013

Solution :

Algorithm :

1. The sequence of a's should be pushed onto the stack in state q_0

   $\delta(q_0, a, z_0) = (q_0, az_0)$

   $\delta(q_0, a, a) = (q_0, aa)$

2. On first b, the machine moves to q_1 and remains there for b's. b's will have no effect on the stack.

3. For every 'a', an 'a' is erased from the stack.

   The PDA accepting through empty stack is given by

   $$M = (\{q_0, q_1, q_2\}, \{a, b\}, \{a, z_0\}, \delta, q_0, z_0, \phi)$$

Where the transition function δ is :

1.	$\delta(q_0, a, z_0) = (q_0, az_0)$	[First 'a' is pushed]
2.	$\delta(q_0, a, a) = (q_0, aa)$	[Subsequent a's are pushed]
3.	$\delta(q_0, b, a) = (q_1, a)$	[Input symbols b's are skipped]
4.	$\delta(q_1, b, a) = (q_1, a)$	
5.	$\delta(q_1, a, a) = (q_2, \varepsilon)$	[An a is erased on first a of last a's]
6.	$\delta(q_2, a, a) = (q_2, \varepsilon)$	[An a is erased on subsequent a's of last a's]
7.	$\delta(q_2, \varepsilon, z_0) = (q_2, \varepsilon)$	[Accepting through empty stack]

Example 6.4.11 : Design a PDA for accepting a language

$$L = \{ WcW^T \mid W \in (a, b)^* \}$$

MU - May 2008, May 2010, May 2011

Solution :

W^T stands for reverse of W. A string of the form WcW^T is an odd length palindrome with the middle character as c.

Algorithm :

If the length of the string is 2n + 1, then the first n symbols should be matched with the last n symbols in the reverse order. A stack can be used to reverse the first n input symbols.

- Status of the stack and state of the machine is shown in Fig. Ex. 6.4.11. Input applied is abbcbba.

Scanned by CamScanner

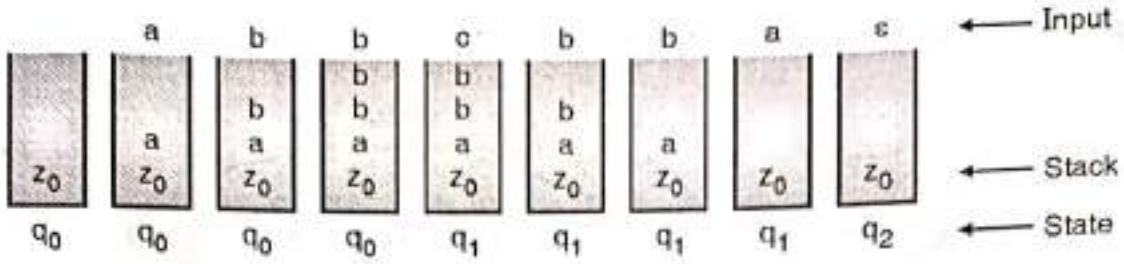


Fig. Ex. 6.4.11 : A PDA on input abbcbba

The PDA accepting through final state is given by

$$M = (\{q_0, q_1, q_2\}, \{a, b, c\}, \{a, b, z_0\}, \delta, q_0, z_0, \{q_2\})$$

Where the transition function δ is given below :

1. $\delta(q_0, a, \varepsilon) = (q_0, a)$]
2. $\delta(q_0, b, \varepsilon) = (q_0, b)$] First n symbols are pushed onto the stack
3. $\delta(q_0, c, \varepsilon) = (q_1, \varepsilon)$ [State changes on c]
4. $\delta(q_1, a, a) = (q_1, \varepsilon)$]
5. $\delta(q_1, b, b) = (q_1, \varepsilon)$] Last n symbols are matched with first n symbols in reverse order
6. $\delta(q_1, \varepsilon, z_0) = (q_2, z_0)$ [Accepted through final state]

A transition of the form $\delta(q_0, a, \varepsilon) = (q_0, a)$ implies that always push a, irrespective of stack symbol.

Example 6.4.12 : Design a PDA to check whether a given string over {a, b} ends in abb.

Solution :

Strings ending in abb form a regular language. A regular language is accepted by DFA.

A PDA for a regular language can be constructed in two steps :

1. Design a DFA.
2. Convert DFA to PDA by appending no-stack-operation to every transition in DFA.

Step 1 : DFA for the given language is :

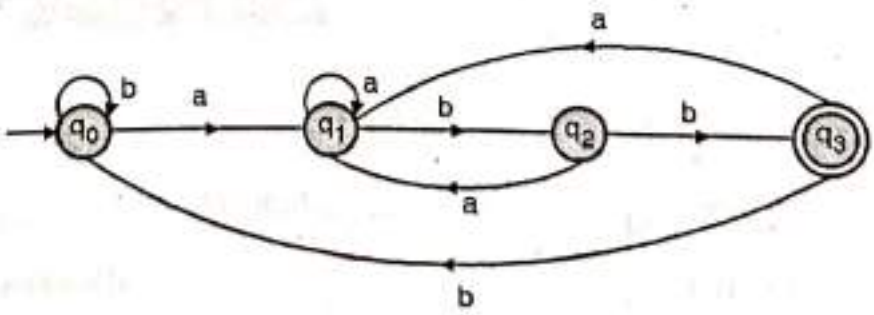


Fig. Ex. 6.4.12 : A DFA for string ending in abb

Scanned by CamScanner

Step 2 : From DFA to PDA

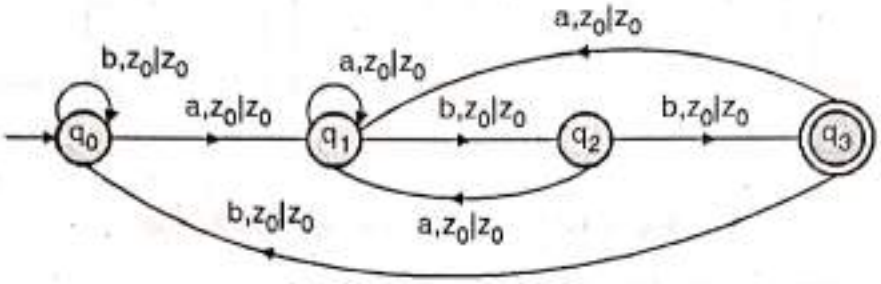


Fig. Ex. 6.4.12(a) : A PDA constructed from DFA

In every transition of DFA, a stack operation z_0/z_0 is added. z_0/z_0 implies no-stack operation.

The PDA accepting strings ending in abb through final state is given by :

$$M = (\{q_0, q_1, q_2, q_3\}, \{a, b\}, \{z_0\}, \delta, q_0, z_0, \{q_3\})$$

Where the transition function δ is given below,

$\delta(q_0, b, z_0) = (q_0, z_0)$

$\delta(q_0, a, z_0) = (q_1, z_0)$

$\delta(q_1, a, z_0) = (q_1, z_0)$

$\delta(q_1, b, z_0) = (q_2, z_0)$

$\delta(q_2, a, z_0) = (q_1, z_0)$

$\delta(q_2, b, z_0) = (q_3, z_0)$

$\delta(q_3, a, z_0) = (q_1, z_0)$

$\delta(q_3, b, z_0) = (q_0, z_0)$

Example 6.4.13 : Design a PDA for CFL that checks the well formedness of parenthesis i.e. the language L of all "balanced" string of two types of parenthesis say "( )" and "[ ]". Trace the sequence of moves made corresponding to input string (([ ])[ ]). **MU - May 2009, May 2014, Dec. 2017**

Solution :

The transition function of the PDA is given below :

1. $\delta(q_0, (, z_0) = (q_0, (z_0)$
2. $\delta(q_0, (, () = (q_0, (()$
3. $\delta(q_0, (, [) = (q_0, ([)$

Push the opening bracket '('

Scanned by CamScanner

4. $\delta(q_0, [, z_0) = (q_0, [z_0)$
5. $\delta(q_0, [, ( ) = (q_0, [( )$
6. $\delta(q_0, [, [ ) = (q_0, [[ )$

Push the opening bracket '['

7. $\delta(q_0,),)) = (q_0, \varepsilon)$
8. $\delta(q_0,],]) = (q_0, \varepsilon)$

POP an opening bracket for a closing bracket.

9. $\delta(q_0, \varepsilon, z_0) = (q_f, z_0)$

Accept through a final state.

Simulation of PDA for the input string (([])[])

$$(q_0, (([])[]), z_0) \xrightarrow{\text{Rule 1}} (q_0, ([])[]), (z_0)$$
$$\xrightarrow{\text{Rule 2}} (q_0, [])[]), ((z_0)$$
$$\xrightarrow{\text{Rule 5}} (q_0,])[]), [((z_0)$$
$$\xrightarrow{\text{Rule 8}} (q_0,)[]), ((z_0)$$
$$\xrightarrow{\text{Rule 7}} (q_0, []), (z_0)$$
$$\xrightarrow{\text{Rule 5}} (q_0,]), [(z_0)$$
$$\xrightarrow{\text{Rule 8}} (q_0,), (z_0)$$
$$\xrightarrow{\text{Rule 7}} (q_0, \varepsilon, z_0)$$
$$\xrightarrow{\text{Rule 9}} (q_f, \varepsilon, z_0)$$

Example 6.4.14 : Construct PDA for the following language : $L = \{0^m 1^n 0^{m+n} \mid m, n \geq 1\}$ **MU - Dec. 2011, May 2012**

Solution :

1. Algorithm :

(a) An x is pushed for every leading 0 of the string $0^m 1^n 0^{m+n}$.

(b) An x is pushed for every 1 in the input of the string $0^m 1^n 0^{m+n}$.

(c) x's on the stack are matched with trailing 0's in the input of the string $0^m 1^n 0^{m+n}$.

2. Transition function δ :

$\delta(q_0, 0, z_0) = (q_0, x\, z_0)$

$\delta(q_0, 0, x) = (q_0, x\, x)$

Scanned by CamScanner

$\delta(q_0, 1, x) = (q_1, x\,x)$

$\delta(q_1, 1, x) = (q_1, x\,x)$

$\delta(q_1, 0, x) = (q_2, \in)$

$\delta(q_2, 0, x) = (q_2, \in)$

$\delta(q_2, \in, z_0) = (q_2, \in)$ [Accept through an empty stack]

Example 6.4.15 : Define PDA and construct PDA for the grammar :
$E \rightarrow E + E \mid E - E \mid (E) \mid id$ **MU - Dec. 2011**

Solution :

The equivalent PDA, M is given by :

$$M = (\{q\}, \{+, -, (,), id\}, \{+, -, (,), id, E\}, \delta, q, E, \phi)$$

Where δ is given by :

$$\delta(q, \in, E) = \{(q, E + E), (q, E - E), (q, (E)), (q, id)\}$$

$$\delta(q, +, +) = \{(q, \in)\}$$

$$\delta(q, -, -) = \{(q, \in)\}$$

$$\delta(q, (, () = \{(q, \in)\}$$

$$\delta(q,),)) = \{(q, \in)\}$$

$$\delta(q, id, id) = \{(q, \in)\}$$

6.5 Non-deterministic PDA (NPDA)

MU - Dec. 2005, May 2010, Dec. 2011, May 2013, May 2015, Dec. 2017

There are two types of push down automata :

1. DPDA (Deterministic PDA)
2. NPDA (Non-deterministic PDA)

A NPDA provides non-determinism to PDA.

In a DPDA there is only one move in every situation. Where as, in case of NPDA there could be multiple moves under a situation.

DPDA is less powerful than NPDA. Every context free language can not be recognized by a DPDA but it can be recognized by NPDA. The class of language a DPDA can accept lies in between a regular language and CFL. A palindrome can be accepted by NPDA but it can not be accepted by a DPDA.

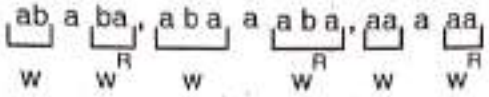
Scanned by CamScanner

Example 6.5.1 : Design a PDA for detection of odd palindrome over {a, b}. **MU - Dec. 2007**

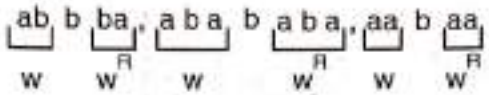
Solution :

An odd palindrome will be of the form :

1. waw^R



2. wbw^R



If the length of w is n then a palindrome of odd length is :

First n characters are equal to the last n characters in reverse order with middle character as 'a' or 'b'.

Algorithm :

There is no way of finding the middle position of a string by a PDA, therefore the middle position is fixed non-deterministically.

1. First n characters are pushed onto the stack, where n is non-deterministic.
2. The n characters on the stack are matched with the last n characters of the input string.
3. n is decided non-deterministically. Every character out of first n characters should be considered for two cases :

   (a) It is not the middle character – push the current character using the transition :

   $$\delta(q_0, a, \varepsilon) \Rightarrow (q_0, a)$$

   $$\delta(q_0, b, \varepsilon) \Rightarrow (q_0, b)$$

   (b) It is a middle character – go for matching of second half with the first half.

   $$\delta(q_0, a, \varepsilon) \Rightarrow (q_1, \varepsilon)$$

   $$\delta(q_0, b, \varepsilon) \Rightarrow (q_1, \varepsilon)$$

The status of the stack and the state of the machine is shown in the Fig. Ex. 6.5.1. Input applied is ababa.

- Left child → current input is taken as the middle character
- Right child → current input is not a middle character.

Scanned by CamScanner

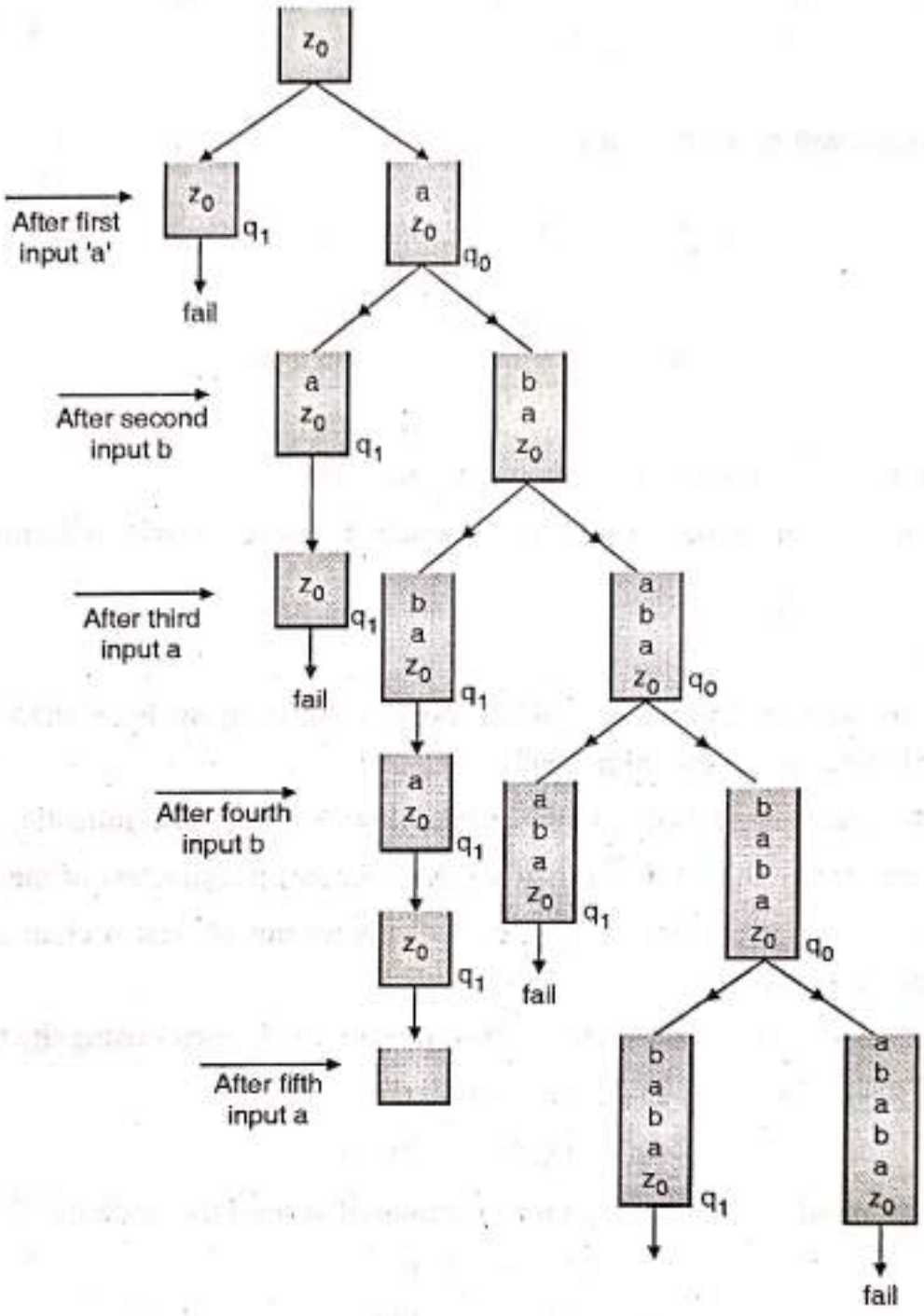


Fig. Ex. 6.5.1 : Processing of string by the PDA. String is taken as "ababa"

The transition table for the PDA is given below,

$\delta(q_0, a, \varepsilon) \Rightarrow \{(q_1, \varepsilon), (q_0, a)\}$

↳ ε - indicates that irrespective of the current stack symbol, perform the transition.

$\delta(q_0, b, \varepsilon) \Rightarrow \{(q_1, \varepsilon), (q_0, b)\}$

$\delta(q_1, a, a) \Rightarrow \{(q_1, \varepsilon)\}$

$\delta(q_1, b, b) \Rightarrow \{(q_1, \varepsilon)\}$

Scanned by CamScanner

$\delta(q_1, \varepsilon, z_0) \Rightarrow \{(q_1, \varepsilon)\}$ [Accept through an empty stack]

Where, The set of states $Q = \{q_0, q_1\}$

The set input alphabet $\Sigma = \{a, b\}$

The set of stack symbols $\Gamma = \{a, b, z_0\}$

Starting state $= q_0$

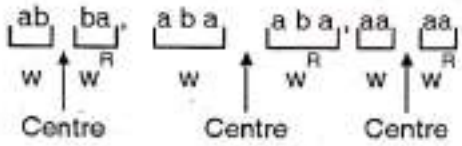
Initial stack symbol $= z_0$

Example 6.5.2 : Design a PDA for detection of even palindrome over {a, b}.

MU - Dec. 2005, May 2006, May 2007, May 2016

Solution :

An even palindrome will be of the form ww^R



If the length of w is n then a palindrome of even length is :

First n characters are equal to the last n characters in the reverse order.

The character immediately before the middle position will be identical to the character immediately after the middle position.

Algorithm :

There is no way of finding the middle position by a PDA; therefore the middle position is fixed non-deterministically.

1. First n characters are pushed onto the stack. n is non-deterministic.
2. The n characters on the stack are matched with the last n characters of the input string.
3. n is decided non-deterministically. Every character out of first n characters, whose previous character is same as itself should be considered for two cases :

   (a) It is first character of the second half.

   - Pop the current stack symbol using the transitions :

   $\delta(q_0, a, a) \Rightarrow (q_1, \varepsilon)$

   $\delta(q_0, b, b) \Rightarrow (q_1, \varepsilon)$

   Must be identical

   (b) It belongs to first half.

   - Push the current input

   $\delta(q_0, a, \varepsilon) \Rightarrow (q_0, a)$

   $\delta(q_0, b, \varepsilon) \Rightarrow (q_0, b)$

Scanned by CamScanner

4. n is decided non-deterministically. Every character out of first n characters, whose previous character is not same as itself should be pushed onto the stack.

- Push the current symbol using the transitions :

$\delta(q_0, a, b) \Rightarrow (q_0, ab)$

$\delta(q_0, b, a) \Rightarrow (q_0, ba)$

The transition table for the PDA is given below :

$\delta(q_0, a, z_0) \Rightarrow \{(q_0, az_0)\}$

$\delta(q_0, b, z_0) \Rightarrow \{(q_0, bz_0)\}$

$\delta(q_0, a, a) \Rightarrow \{(q_0, aa)\ (q_1, \varepsilon)\}$

$\delta(q_0, a, b) \Rightarrow \{(q_0, ab)\}$

$\delta(q_0, b, a) \Rightarrow \{(q_0, ba)\}$

$\delta(q_0, b, b) \Rightarrow \{(q_0, bb), (q_1, \varepsilon)\}$

$\delta(q_1, a, a) \Rightarrow \{(q_1, \varepsilon)\}$

$\delta(q_1, b, b) \Rightarrow \{(q_1, \varepsilon)\}$

$\delta(q_1, \varepsilon, z_0) \Rightarrow \{(q_1, \varepsilon)\}$ [Accept through an empty stack]

Where, the set of states $Q = \{q_0, q_1\}$

the set of input symbols $\Sigma = \{a, b\}$

the set of stack symbols $\Gamma = \{a, b, z_0\}$

Starting state $= q_0$

Initial stack symbol $= z_0$

Example 6.5.3 : Design a PDA for detection of palindromes over {a, b}. **MU - Dec. 2012**

Solution :

A palindrome will be of the form :

1. ww^R – even palindrome
2. waw^R
3. wbw^R – odd palindrome

If the length of w is n then a palindrome is :

First n characters are equal to the last n characters in the reverse order with the middle character as :

(1) a [For odd palindrome]

(2) b [For odd palindrome]

(3) ε [For even palindrome]

Scanned by CamScanner

The transition table for the PDA is given below :

$\delta(q_0, a, z_0) \Rightarrow \{(q_1, z_0), (q_0, az_0)\}$

$\delta(q_0, b, z_0) \Rightarrow \{(q_1, z_0), (q_0, bz_0)\}$

$\delta(q_0, a, a) \Rightarrow \{(q_0, aa)\ (q_1, a), (q_1, \varepsilon)\}$

$\delta(q_0, a, b) \Rightarrow \{(q_0, ab), (q_1, b)\}$

$\delta(q_0, b, a) \Rightarrow \{(q_0, ba), (q_1, a)\}$

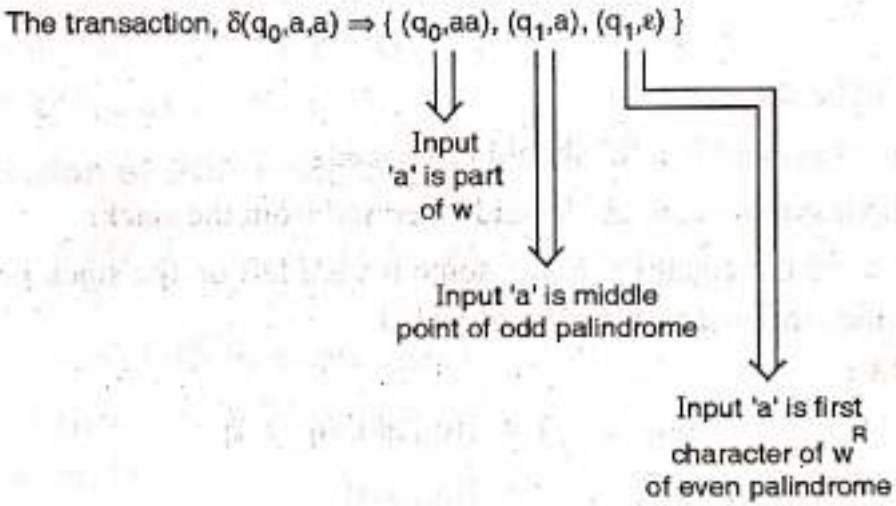
$\delta(q_0, b, b) \Rightarrow \{(q_0, bb), (q_1, b), (q_1, \varepsilon)\}$

$\delta(q_1, a, a) \Rightarrow \{(q_1, \varepsilon)\}$

$\delta(q_1, b, b) \Rightarrow \{(q_1, \varepsilon)\}$

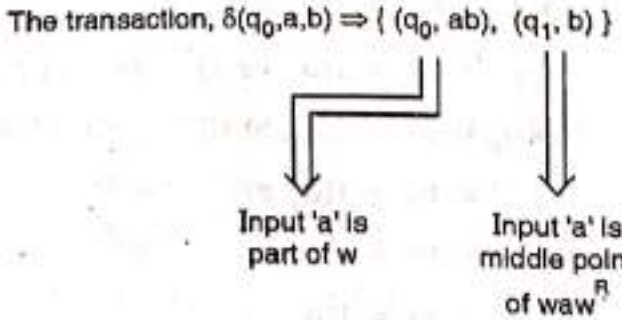
$\delta(q_1, \varepsilon, z_0) \Rightarrow \{(q_1, \varepsilon)\}$ [Accept through an empty stack].

Details of important transitions :



The transition rule for $\delta(q_0, a, a)$, must consider the three cases :

1. Input 'a' is part of w of the palindrome.
2. Input 'a' is middle character of waw^R
3. Input 'a' is the first character of w^R.



Scanned by CamScanner

Example 6.5.4 : Construct PDA for the language

$$L = \{a^i b^j c^k \mid i \neq j \text{ or } j \neq k\}$$

Solution :

Algorithm :

Machine has to non-deterministically figure out which of the two conditions will be satisfied by the string :

1. $i \neq j$, by entering the state q_1 on first input.
2. $j \neq k$, by entering the state q_2 on first input.

$$\delta(q_0, a, z_0) = \{(q_1, aa), (q_2, z_0)\}$$

In state q_1

1. Initial a's will be pushed.
2. For every input symbol b, an 'a' should be erased from the stack.
3. If a's and b's are not equal i.e. either some a's are left on the stack or some b's are still in the input then the string should be accepted.

In state q_2

1. Initial a's will be skipped.
2. For every input symbol b, a 'b' should be pushed.
3. For every input symbol c, a 'ab' should be erased from the stack.
4. If b's and c's are not equal i.e. either some b's are left on the stack or some c's are still in the input then the string should be accepted.

Transitions for PDA :

$$\delta(q_0, a, z_0) = \{(q_1, aa), (q_2, z_0)\}$$
$$\delta(q_1, a, a) = \{(q_1, aa)\}$$
$$\delta(q_1, b, a) = \{(q_3, \varepsilon)\}$$
$$\delta(q_3, b, z_0) = \{(q_f, z_0)\}$$
$$\delta(q_3, c, a) = \{(q_f, \varepsilon)\}$$
$$\delta(q_3, b, a) = \{(q_3, \varepsilon)\}$$
$$\delta(q_2, a, z_0) = \{(q_2, \varepsilon)\}$$
$$\delta(q_2, b, z_0) = \{(q_4, bz_0)\}$$
$$\delta(q_4, b, b) = \{(q_4, bb)\}$$
$$\delta(q_4, c, b) = \{(q_4, \varepsilon)\}$$
$$\delta(q_4, \varepsilon, b) = \{(q_f, \varepsilon)\}$$
$$\delta(q_f, b, \varepsilon) = \{(q_f, \varepsilon)\}$$

Scanned by CamScanner

$$\delta(q_f, c, \varepsilon) = \{(q_f, \varepsilon)\}$$

$$\delta(q_f, \varepsilon, \varepsilon) = \{(q_f, \varepsilon)\}$$

6.6 Pushdown Automata and Context Free Language

The class of languages accepted by pushdown automata is exactly the class of context-free languages. The following three classes of languages are same :

1. Context Free Language defined by CFG.
2. Languages accepted by PDA by final state.
3. Languages accepted by PDA by empty stack.

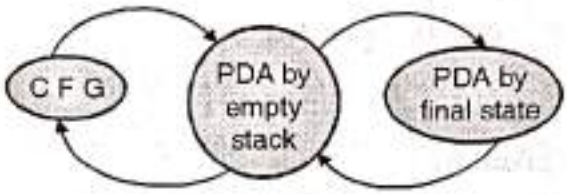


Fig. 6.6.1 : Equivalence of PDA and CFG

- It is possible to find a PDA for a CFG
- It is possible to find a CFG for a PDA.

6.6.1 Construction of PDA from CFG

From a given CFG G = (V, T, P, S), we can construct a PDA, M that simulates the leftmost derivation of G.

The PDA accepting L(G) by empty stack is given by :

$M = (\{q\}, T, V \cup T, \delta, q, S, \phi)$ [M is a PDA for L(G)]

Where δ is defined by :

1. For each variable $A \in V$, include a transition,

$$\delta(q, \varepsilon, A) \Rightarrow \{(q, \alpha) \mid A \rightarrow \alpha \text{ is a production in G}\}$$

2. For each terminal $a \in T$, include a transition

$$\delta(q, a, a) \Rightarrow \{(q, \varepsilon)\}$$

Example 6.6.1 : Find a PDA for the given grammar

$S \rightarrow 0S1 \mid 00 \mid 11$

Solution : The equivalent PDA, M is given by :

$M = (\{q\}, \{0, 1\}, \{0, 1, S\}, \delta, q, S, \phi)$, where δ is given by :

$$\delta(q, \varepsilon, S) = \{(q, 0S1), (q, 00), (q, 11)\}$$

$$\delta(q, 0, 0) = \{(q, \varepsilon)\}$$

$$\delta(q, 1, 1) = \{(q, \varepsilon)\}$$

Scanned by CamScanner

Example 6.6.2 : Convert the grammar

$S \rightarrow 0S1 \mid A$

$A \rightarrow 1A0 \mid S \mid \varepsilon$

to PDA that accepts the same language by empty stack.

Solution :

Step 1 : for each variable $A \in V$, include a transition $\delta(q, \varepsilon, A) \Rightarrow \{(q, \alpha) \mid A \rightarrow \alpha$ is a production in G $\}$

$\therefore \quad \delta(q, \varepsilon, S) \Rightarrow \{(q, 0S1), (q, A)\}$

$\delta(q, \varepsilon, A) \Rightarrow \{(q, 1A0), (q, S), (q, \varepsilon)\}$

Step 2 : For each terminal $a \in T$, include a transition $\delta(q, a, a) \Rightarrow (q, \varepsilon)$

$\therefore \quad \delta(q, 0, 0) = \{(q, \varepsilon)\}$

$\delta(q, 1, 1) = \{(q, \varepsilon)\}$

Therefore, the PDA is given by :

$M = (\{q\}, \{0, 1\}, \{S, A, 0, 1\}, \delta, q, S, \phi)$

where δ is :

$\delta(q, \varepsilon, S) = \{(q, 0S1), (q, A)\}$

$\delta(q, \varepsilon, A) = \{(q, 1A0), (q, S), (q, \varepsilon)\}$

$\delta(q, 0, 0) = \{(q, \varepsilon)\}$

$\delta(q, 1, 1) = \{(q, \varepsilon)\}$

Example 6.6.3 : Let G be the grammar given by
$S \rightarrow aABB \mid aAA$, $A \rightarrow aBB \mid a$, $B \rightarrow bBB \mid A$.
Construct NPDA that accepts the language generated by this grammar.

Solution : The equivalent PDA, M is given by :

$M = (\{q\}, \{a, b\}, \{a, b, S, A, B\}, \delta, q, S, \phi)$

Where, δ is given by :

$\delta(q, \varepsilon, S) \Rightarrow \{(q, aABB), (q, aAA)\}$

$\delta(q, \varepsilon, A) \Rightarrow \{(q, aBB), (q, a)\}$

$\delta(q, \varepsilon, B) \Rightarrow \{(q, bBB), (q, A)\}$

} For each production in given grammar

$\delta(q, a, a) \Rightarrow (q, \varepsilon)$

$\delta(q, b, b) \Rightarrow (q, \varepsilon)$

} For each terminal in T

Example 6.6.4 : Construct a PDA equivalent to the following CFG.

$S \rightarrow 0BB$

$B \rightarrow 0S \mid 1S \mid 0$

Test if 010^4 is in the language **MU - May 2006, May 2011, May 2012**

Scanned by CamScanner

Solution : The equivalent PDA, M is given by

$$M = (\{q\}, \{0, 1\}, \{0, 1, S, B\}, \delta, q, S, \phi),$$

where δ is given by

$\delta(q, \varepsilon, S) \Rightarrow \{(q, 0BB)\}$	For each production in the given grammar
$\delta(q, \varepsilon, B) \Rightarrow \{(q, 0S), (q, 1S), (q, 0)\}$	
$\delta(q, 0, 0) \Rightarrow \{(q, \varepsilon)\}$	For each terminal
$\delta(q, 1, 1) \Rightarrow \{(q, \varepsilon)\}$	

Acceptance of 010^4 by M :

$$\delta(q, 010000, S) \xrightarrow{\delta(q, \varepsilon, S) = (q, 0BB)} (q, 010000, 0BB)$$

$$\xrightarrow{\delta(q, 0, 0) = (q, \varepsilon)} (q, 10000, BB)$$

$$\xrightarrow{\delta(q, \varepsilon, B) = (q, 1S)} (q, 10000, 1SB)$$

$$\xrightarrow{\delta(q, 1, 1) = (q, \varepsilon)} (q, 0000, SB)$$

$$\xrightarrow{\delta(q, \varepsilon, S) = (q, 0BB)} (q, 0000, 0BBB)$$

$$\xrightarrow{\delta(q, 0, 0) = (q, \varepsilon)} (q, 000, BBB)$$

$$\xrightarrow{\delta(q, \varepsilon, B) = (q, 0)} (q, 000, 0BB)$$

$$\xrightarrow{\delta(q, 0, 0) = (q, \varepsilon)} (q, 00, BB)$$

$$\xrightarrow{\delta(q, \varepsilon, B) = (q, 0)} (q, 00, 0B)$$

$$\xrightarrow{\delta(q, 0, 0) = (q, \varepsilon)} (q, 0, B)$$

$$\xrightarrow{\delta(q, \varepsilon, B) = (q, 0)} (q, 0, 0)$$

$$\xrightarrow{\delta(q, 0, 0) = (q, \varepsilon)} (q, \varepsilon, \varepsilon)$$

Scanned by CamScanner

Thus the string 010^4 is accepted by M using an empty stack.

$$\therefore \quad 010^4 \in L$$

Example 6.6.5 : Design a PDA to recognize the language generated by the following grammar :

$S \rightarrow S + S \mid S * S \mid 4 \mid 2$

Show the acceptance of the input string $2 + 2 * 4$ by this PDA.

Solution : The equivalent PDA, M is given by :

$$M = (\{q\}, \{+, *, 4, 2\}, \{+, *, 4, 2, S\}, \delta, q, S, \phi)$$

Where δ is given by :

$\delta(q, \varepsilon, S) \Rightarrow \{(q, S + S), (q, S * S), (q, 4), (q, 2)\}$ | For every production in G

$\delta(q, +, +) = \{(q, \varepsilon)\}$

$\delta(q, *, *) = \{(q, \varepsilon)\}$

$\delta(q, 2, 2) = \{(q, \varepsilon)\}$

$\delta(q, 4, 4) = \{(q, \varepsilon)\}$

| for every terminal in T.

Acceptance of 2 + 2 * 4 by this PDA :

$\delta(q, 2 + 2 * 4, S)$

$\xrightarrow{\delta(q, \varepsilon\ S) \Rightarrow (q, S + S)}$ $(q, 2 + 2 * 4, S + S)$

$\xrightarrow{\delta(q, \varepsilon, S) \Rightarrow (q, 2)}$ $(q, 2 + 2 * 4, 2 + S)$

$\xrightarrow{\delta(q, 2, 2) \Rightarrow (q, \varepsilon)}$ $(q, + 2 * 4, + S)$

$\xrightarrow{\delta(q, +, +) \Rightarrow (q, \varepsilon)}$ $(q, 2 * 4, S)$

$\xrightarrow{\delta(q, \varepsilon, S) \Rightarrow (q, S * S)}$ $(q, 2 * 4, S * S)$

$\xrightarrow{\delta(q, \varepsilon, S) \Rightarrow (q, 2)}$ $(q, 2 * 4, 2 * S)$

$\xrightarrow{\delta(q, 2, 2) \Rightarrow (q, \varepsilon)}$ $(q, * 4, * S)$

$\xrightarrow{\delta(q, *, *) \Rightarrow (q, \varepsilon)}$ $(q, 4, S)$

Scanned by CamScanner

$$\delta(q, \varepsilon, S) = (q, 4)$$

$$\longrightarrow \quad (q, 4, 4)$$

$$\delta(q, 4, 4) = (q, \varepsilon)$$

$$\longrightarrow \quad (q, \varepsilon, \varepsilon)$$

Example 6.6.6 : Convert the following expression grammar to PDA

$I \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1$

$E \rightarrow I \mid E * E \mid E * E \mid (E)$

MU - Dec. 2008

Solution : The equivalent PDA, M is given by,

$$M = (\{q\}, \{0, 1, a, b, *, +, (,)\}, \{0, 1, a, b, *, +, (,), I, E\}, \delta, q, E, \phi)$$

where, δ is given by,

$\delta(q, \varepsilon, E) = \{(q, I), (q, E * E), (q, E + E), (q, (E))\}$

$\delta(q, \varepsilon, I) = \{(q, a), (q, b), (q, Ib), (q, Ia), (q, I0), (q, I1)\}$

$\delta(q, 0, 0) = \{(q, \varepsilon)\}$

$\delta(q, 1, 1) = \{(q, \varepsilon)\}$

$\delta(q, a, a) = \{(q, \varepsilon)\}$

$\delta(q, b, b) = \{(q, \varepsilon)\}$

$\delta(q, +, +) = \{(q, \varepsilon)\}$

$\delta(q, *, *) = \{(q, \varepsilon)\}$

$\delta(q, (, () = \{(q, \varepsilon)\}$

$\delta(q,),)) = \{(q, \varepsilon)\}$

Example 6.6.7 : Show that if L is generated by a CFG then there exists a PDA accepting L.

Solution :

Proof :

Let G = (V, T, P, S) be a Context Free Grammar and a PDA, M is constructed as given below :

$$M = (\{q\}, T, V \cup T, \delta, q, S, \phi)$$

where δ is defined by :

1. For each variable $A \in V$, include a transition,

$$\delta(q, \varepsilon, A) \Rightarrow \{(q, \alpha) \mid A \rightarrow \alpha \text{ is a production in G }\}$$

2. For each terminal $a \in T$, include a transition

$$\delta(q, a, a) \Rightarrow \{(q, \varepsilon)\}$$

- The transitions of M are designed to simulate a leftmost derivation of a string.
  1. The transition of the form

$$\delta(q, \varepsilon, A) \Rightarrow (q, \alpha)$$

is for expansion of the topmost variable of the stack.

Scanned by CamScanner

2. The transition of the form

$$\delta(q, a, a) \Rightarrow (q, \varepsilon)$$

is for removing terminals from the stack so that a variable is exposed for further expansion.

To prove that a PDA constructed using above rules is equivalent to G.

We can prove that a word $w \in L(M)$ if and only if $w \in L(G)$.

where, L(M) is the language of PDA

and L(G) is the language of the given CFG.

Let us take a word $w \in L(G)$. The word w can be derived using the leftmost derivation.

$$S \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \ldots \Rightarrow \alpha_n = w$$

where, $\alpha_i \rightarrow \alpha_{i+1}$

is obtained by single application of leftmost derivation, using a production in grammar G. We will show that for each α_i there is a unique configuration of PDA. α_n corresponds to the configuration of PDA accepting w.

Let $\alpha_i = x_i \beta_i$

Where, $x_i \in T^*$ and $B_i \in (V \cup T)^*$

The string α_i corresponds to a unique configuration of PDA given by a pair :

$$(y_i, \beta_i)$$

The corresponding diagram is shown in Fig. Ex. 6.6.7.

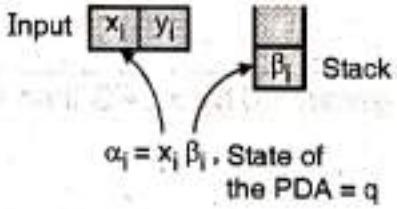


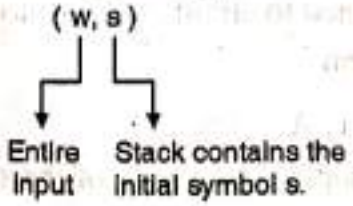
Fig. Ex. 6.6.7 : Unique configuration of PDA for α_i

- x_i portion of the input string w has already been scanned.
- The y_i is the portion of the input string, yet to be scanned.
- The stack contains β_i

The theorem can be proved by induction on i of α_i.

Base case : i = 0 :

$\alpha_0 = S$, the configuration of PDA is given by



Scanned by CamScanner

Induction step :

We have to show that the correspondence is preserved for $j = i + 1$ also, if the correspondence for α_i is assumed.

Without the loss of generality, we can assume that the given grammar is in GNF.

If $(i + 1)_{th}$ input symbol is a_{i+1} and the first variable of β_i is V_i then the top of the stack contains V_i. The variable V_i can be expanded using the production.

$$V_i \Rightarrow a_{i+1} y$$

Similarly, the top symbol of the stack can be replaced with $a_{i+1} y$ and then a_{i+1} can be popped out using the transition

$$\delta (q, a_{i+1}, a_{i+1}) \Rightarrow (q, \varepsilon)$$

Thus, there is one-to-one correspondence between the strings α_i and the configuration of PDA. Thus a string $w \in L(G)$ will be accepted by the PDA M.

6.6.2 Construction of CFG from PDA

We can find the Context Free Grammar G for any PDA, M such that

$$L(G) = L(M)$$

i.e., we can construct an equivalent CFG for a PDA.

The variables of the CFG, so constructed will be of the form :

[pXq], where $p, q \in Q$ and $X \in \Gamma$

Let the PDA is given by :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, z, \phi)$$

Where, z is the initial stack symbol.

Then an equivalent CFG is given by

$$G = (V, \Sigma, P, S) \text{ where}$$

$$V = \{S, [p^x q] \mid p, q \in Q \text{ and } X \in \Gamma\}$$

Example : If $Q = \{q_0, q_1\}$ and $\Gamma = \{a, b, z\}$ then the possible set of variables in the corresponding CFG is given by :

1. S
2. $[q_0 \, ^a \, q_0], [q_0 \, ^a \, q_1], [q_1 \, ^a \, q_0], [q_1 \, ^a \, q_1]$
3. $[q_0 \, ^b \, q_0], [q_0 \, ^b \, q_1], [q_1 \, ^b \, q_0], [q_1 \, ^b \, q_1]$
4. $[q_0 \, ^z \, q_0], [q_0 \, ^z \, q_1], [q_1 \, ^z \, q_0], [q_1 \, ^z \, q_1]$

Scanned by CamScanner

Set of productions for the equivalent CFG

1. Add the following productions for the start symbol S.

$$S \rightarrow [q_0 \overset{z}{} q_i] \text{ for each } q_i \in Q, \text{ where z is the start symbol}$$

2. For each transition of the form

$\delta(q_i, a, B) \Rightarrow (q_j, C)$

Where,

(a) $q_i, q_j \in Q$

(b) a belongs to $(\Sigma \cup \varepsilon)$

(c) B and C belong to $(\Gamma \cup \varepsilon)$

Then for each $q \in Q$, we add the production :

$$[q_i \overset{B}{} q] \rightarrow a [q_j \overset{C}{} q]$$

3. For each transition of the form

$\delta(q_i, a, B) \Rightarrow (q_j, C_1 C_2)$

Where,

(a) $q_i, q_j \in Q$

(b) a belongs to $(\Sigma \cup \varepsilon)$

(c) B, C_1 and C_2 belongs to Γ

then for each $p_1, p_2 \in Q$, we add the production

$$[q_i \overset{B}{} p_1] \rightarrow a [q_j \overset{C_1}{} p_2] [p_2 \overset{C_2}{} p_1]$$

Example 6.6.8 : Convert PDA to CFG. PDA is given by

M = ({ p, q}, {0, 1}, {x, z}, δ, q, z), transition function δ is defined by :

$\delta(q, 1, z) \Rightarrow \{(q, xz)\}$

$\delta(q, 1, x) \Rightarrow \{(q, xx)\}$

$\delta(q, \varepsilon, x) \Rightarrow \{(q, \varepsilon)\}$

$\delta(q, 0, x) \Rightarrow \{(p, x)\}$

$\delta(p, 1, x) \Rightarrow \{(p, \varepsilon)\}$

$\delta(p, 0, z) \Rightarrow \{(q, z)\}$

Solution :

Step 1 : Add productions for the start symbol.

$$S \rightarrow [q \overset{z}{} q]$$

$$S \rightarrow [q \overset{z}{} p]$$

Step 2 : Add productions for $\delta(q, 1, z) \Rightarrow \{(q, xz)\}$

$$[q \overset{z}{} q] \rightarrow 1 [q \overset{x}{} q] [q \overset{z}{} q]$$

Scanned by CamScanner

$[q\ ^{z}\ q] \rightarrow 1\ [q\ ^{x}\ p]\ [p\ ^{z}\ q]$

$[q\ ^{z}\ p] \rightarrow 1\ [q\ ^{x}\ q]\ [q\ ^{z}\ p]$

$[q\ ^{z}\ p] \rightarrow 1\ [q\ ^{x}\ p]\ [p\ ^{z}\ p]$

Step 3 : Add productions For $\delta(q, 1, x) \Rightarrow \{(q, xx)\}$

$[q\ ^{x}\ q] \rightarrow 1\ [q\ ^{x}\ q]\ [q\ ^{x}\ q]$

$[q\ ^{x}\ q] \rightarrow 1\ [q\ ^{x}\ p]\ [p\ ^{x}\ q]$

$[q\ ^{x}\ p] \rightarrow 1\ [q\ ^{x}\ q]\ [q\ ^{x}\ p]$

$[q\ ^{x}\ p] \rightarrow 1\ [q\ ^{x}\ p]\ [p\ ^{x}\ p]$

Step 4 : Add productions for $\delta(q, \varepsilon, x) \Rightarrow \{(q, \varepsilon)\}$

$[q\ ^{x}\ q] \rightarrow \varepsilon$

Step 5 : Add productions for $\delta(q, 0, x) \Rightarrow \{(p, x)\}$

$[q\ ^{x}\ q] \rightarrow 0\ [p\ ^{x}\ q]$

$[q\ ^{x}\ p] \rightarrow 0\ [p\ ^{x}\ p]$

Step 6 : Add productions for $\delta(p, 1, x) \Rightarrow \{(p, \varepsilon)\}$

$[p\ ^{x}\ p] \rightarrow 1$

Step 7 : Add productions for $\delta(p, 0, z) \Rightarrow \{(q, z)\}$

$[p\ ^{z}\ q] \rightarrow 0\ [q\ ^{z}\ q]$

$[p\ ^{z}\ p] \rightarrow 0\ [q\ ^{z}\ p]$

Step 8 : Renaming of variables :

Original name	New name
$[q\ ^{z}\ q]$	A
$[q\ ^{z}\ p]$	B
$[p\ ^{z}\ q]$	C
$[p\ ^{z}\ p]$	D
$[q\ ^{x}\ q]$	E
$[q\ ^{x}\ p]$	F

Scanned by CamScanner

Original name	New name
$[p\ ^{x}\ q]$	G
$[p\ ^{x}\ p]$	H

The set of productions can be written as :

$S \rightarrow A \mid B$

$A \rightarrow 1EA \mid 1FC$

$B \rightarrow 1EB \mid 1FD$

$E \rightarrow 1EE \mid 1FG$

$F \rightarrow 1EF \mid 1FH$

$E \rightarrow \varepsilon$

$E \rightarrow 0G$

$F \rightarrow 0H$

$H \rightarrow 1$

$C \rightarrow 0A$

$D \rightarrow 0B$

Step 9 : Simplification of grammar

Symbol G does not come on the left side of the production, hence it can be eliminated.

The equivalent set of productions is :

$S \rightarrow A \mid B$

$A \rightarrow 1EA \mid 1FC$

$B \rightarrow 1EB \mid 1FD$

$E \rightarrow 1EE \mid \varepsilon$

$F \rightarrow 1EF \mid 1FH \mid 0H$

$H \rightarrow 1$

$C \rightarrow 0A$

$D \rightarrow 0B$

Example 6.6.9 : Give the CFG generating the language accepted by the following PDA : $M = (\{q_0, q_1\}, \{0, 1\}, \{z_0, x\}, \delta, q_0, z_0, \phi)$ when δ is given below :

$\delta(q_0, 1, z_0) = \{(q_0, xz_0)\}$

$\delta(q_0, 1, x) = \{(q_0, xx)\}$

$\delta(q_0, 0, x) = \{(q_1, x)\}$

$\delta(q_0, \varepsilon, z_0) = \{(q_0, \varepsilon)\}$

$\delta(q_1, 1, x) = \{(q_1, \varepsilon)\}$

$\delta(q_1, 0, z_0) = \{(q_0, z_0)\}$

MU - Dec. 2007

Scanned by CamScanner

Solution :

Step 1 : Add productions for the start symbol

$S \rightarrow [q_0 \, {}^{z_0} \, q_0]$

$S \rightarrow [q_0 \, {}^{z_0} \, q_1]$

Step 2 : Add productions for $\delta(q_0, 1, z_0) = \{(q_0, xz_0)\}$

$[q_0 \, {}^{z_0} \, q_0] \rightarrow 1 \, [q_0 \, {}^{x} \, q_0] \, [q_0 \, {}^{z_0} \, q_0]$

$[q_0 \, {}^{z_0} \, q_0] \rightarrow 1 \, [q_0 \, {}^{x} \, q_1] \, [q_1 \, {}^{z_0} \, q_0]$

$[q_0 \, {}^{z_0} \, q_1] \rightarrow 1 \, [q_0 \, {}^{x} \, q_0] \, [q_0 \, {}^{z_0} \, q_1]$

$[q_0 \, {}^{z_0} \, q_1] \rightarrow 1 \, [q_0 \, {}^{x} \, q_1] \, [q_1 \, {}^{z_0} \, q_1]$

Step 3 : Add productions for $\delta(q_0, 1, x) \Rightarrow \{(q_0, xx)\}$

$[q_0 \, {}^{x} \, q_0] \rightarrow 1 \, [q_0 \, {}^{x} \, q_0] \, [q_0 \, {}^{x} \, q_0]$

$[q_0 \, {}^{x} \, q_0] \rightarrow 1 \, [q_0 \, {}^{x} \, q_1] \, [q_1 \, {}^{x} \, q_0]$

$[q_0 \, {}^{x} \, q_1] \rightarrow 1 \, [q_0 \, {}^{x} \, q_0] \, [q_0 \, {}^{x} \, q_1]$

$[q_0 \, {}^{x} \, q_1] \rightarrow 1 \, [q_0 \, {}^{x} \, q_1] \, [q_1 \, {}^{x} \, q_1]$

Step 4 : Add productions for $\delta(q_0, 0, x) \Rightarrow \{(q_1, x)\}$

$[q_0 \, {}^{x} \, q_0] \rightarrow 0 \, [q_1 \, {}^{x} \, q_0]$

$[q_0 \, {}^{x} \, q_1] \rightarrow 0 \, [q_1 \, {}^{x} \, q_1]$

Step 5 : Add productions for $\delta(q_0, \varepsilon, z_0) = \{(q_1, \varepsilon)\}$

$[q_0 \, {}^{z_0} \, q_1] \rightarrow \varepsilon$

Step 6 : Add production for $\delta(q_1, 1, x) \Rightarrow \{(q_1, \varepsilon)\}$

$[q_1 \, {}^{x} \, q_1] \rightarrow 1$

Step 7 : Add productions for $\delta(q_1, 0, z_0) \Rightarrow \{(q_0, z_0)\}$

$[q_1 \, {}^{z_0} \, q_0] \Rightarrow 0 \, [q_0 \, {}^{z_0} \, q_0]$

$[q_1 \, {}^{z_0} \, q_1] \Rightarrow 0 \, [q_0 \, {}^{z_0} \, q_1]$

Scanned by CamScanner

Example 6.6.10 : Consider the PDA with the following moves :

$\delta(q_0, a, z_0) = \{(q_0, az_0)\}$

$\delta(q_0, a, a) = \{(q_0, aa)\}$

$\delta(q_0, b, a) = \{(q_1, \varepsilon)\}$

$\delta(q_1, b, a) = \{(q_1, \varepsilon)\}$

$\delta(q_1, \varepsilon, z_0) = \{(q_1, \varepsilon)\}$

Obtain CFG equivalent to PDA. **MU - May 2009**

Solution :

Step 1 : Add productions for the start symbol.

$S \rightarrow [q_0 \, {}^{z_0} \, q_0]$

$S \rightarrow [q_0 \, {}^{z_0} \, q_1]$

Step 2 : Add productions for $(q_0, a, a) = \{(q_0, aa)\}$

$[q_0 \, {}^{a} \, q_0] \rightarrow a \, [q_0 \, {}^{a} \, q_0] \, [q_0 \, {}^{a} \, q_0]$

$[q_0 \, {}^{a} \, q_0] \rightarrow a \, [q_0 \, {}^{a} \, q_1] \, [q_1 \, {}^{a} \, q_0]$

$[q_0 \, {}^{a} \, q_1] \rightarrow a \, [q_0 \, {}^{a} \, q_0] \, [q_0 \, {}^{a} \, q_1]$

$[q_0 \, {}^{a} \, q_1] \rightarrow a \, [q_0 \, {}^{a} \, q_1] \, [q_1 \, {}^{a} \, q_1]$

Step 3 : Add productions for $\delta(q_0, b, a) = \{(q_1, \varepsilon)\}$

$[q_0 \, {}^{a} \, q_1] \rightarrow b$

Step 4 : Add productions for $\delta(q_1, b, a) = \{(q_1, \varepsilon)\}$

$[q_1 \, {}^{a} \, q_1] \rightarrow b$

Step 5 : Add productions for $\delta(q_1, \varepsilon, z_0) \rightarrow \{(q_1, \varepsilon)\}$

$[q_1 \, {}^{z_0} \, q_1] \rightarrow \varepsilon$

Example 6.6.11 : For the PDA

$(\{ q_0, q_1 \}, \{ 0, 1 \}, \{ 0, 1, z_0 \}, \delta, q_0, z_0, \phi)$ where δ is

$\delta(q_0, \varepsilon, z_0) = \{(q_1, \varepsilon)\}$

$\delta(q_0, 0, z_0) = \{(q_0, 0z_0)\}$

$\delta(q_0, 0, 0) = \{(q_0, 00)\}$

$\delta(q_0, 1, 0) = \{(q_0, 10)\}$

$\delta(q_0, 1, 1) = \{(q_0, 11)\}$

$\delta(q_0, 0, 1) = \{(q_1, \varepsilon)\}$

$\delta(q_1, 0, 1) = \{(q_1, \varepsilon)\}$

$\delta(q_1, 0, 0) = \{(q_1, \varepsilon)\}$

$\delta(q_1, \varepsilon, z_0) = \{(q_1, \varepsilon)\}$

Obtain CFG accepted by the above PDA and simplify the CFG and describe the language it accepts.

Scanned by CamScanner

Solution :

Sr. No.	PDA transition	Corresponding productions
1.	Productions due to start symbol S.	$S \to [q_0\ ^{z_0}\ q_0]$ $S \to [q_0\ ^{z_0}\ q_1]$
2.	$\delta(q_0, \varepsilon, z_0) = (q_1, \varepsilon)$	$[q_0\ ^{z_0}\ q_1] \to \varepsilon$
3.	$\delta(q_0, 0, z_0) = (q_0, 0z_0)$	$[q_0\ ^{z_0}\ q_0] \to 0\ [q_0\ ^{0}\ q_0]\ [q_0\ ^{z_0}\ q_0]$ $[q_0\ ^{z_0}\ q_0] \to 0\ [q_0\ ^{0}\ q_1]\ [q_1\ ^{z_0}\ q_0]$ $[q_0\ ^{z_0}\ q_1] \to 0\ [q_0\ ^{0}\ q_0]\ [q_0\ ^{z_0}\ q_1]$ $[q_0\ ^{z_0}\ q_1] \to 0\ [q_0\ ^{0}\ q_1]\ [q_1\ ^{z_0}\ q_1]$
4.	$\delta(q_0, 0, 0) = (q_0, 00)$	$[q_0\ ^{0}\ q_0] \to 0\ [q_0\ ^{0}\ q_0\ ]\ [q_0\ ^{0}\ q_0]$ $[q_0\ ^{0}\ q_0] \to 0\ [q_0\ ^{0}\ q_1]\ [q_1\ ^{0}\ q_0]$ $[q_0\ ^{0}\ q_1] \to 0\ [q_0\ ^{0}\ q_0]\ [q_0\ ^{0}\ q_1]$ $[q_0\ ^{0}\ q_1] \to 0\ [q_0\ ^{0}\ q_1]\ [q_1\ ^{0}\ q_1]$
5.	$\delta(q_0, 1, 0) = (q_0, 10)$	$[q_0\ ^{0}\ q_0] \to 1\ [q_0\ ^{1}\ q_0]\ [q_0\ ^{0}\ q_0]$ $[q_0\ ^{0}\ q_0] \to 1\ [q_0\ ^{1}\ q_1]\ [q_1\ ^{0}\ q_0]$ $[q_0\ ^{0}\ q_1] \to 1\ [q_0\ ^{1}\ q_0]\ [q_0\ ^{0}\ q_1]$ $[q_0\ ^{0}\ q_1] \to 1\ [q_0\ ^{1}\ q_1]\ [q_1\ ^{0}\ q_1]$
6.	$\delta(q_0, 1, 1) = (q_0, 11)$	$[q_0\ ^{1}\ q_0] \to 1\ [q_0\ ^{1}\ q_0]\ [q_0\ ^{1}\ q_0]$ $[q_0\ ^{1}\ q_0] \to 1\ [q_0\ ^{1}\ q_1]\ [q_1\ ^{1}\ q_0]$ $[q_0\ ^{1}\ q_1] \to 1\ [q_0\ ^{1}\ q_0]\ [q_0\ ^{1}\ q_1]$ $[q_0\ ^{1}\ q_1] \to 1\ [q_0\ ^{1}\ q_1]\ [q_1\ ^{1}\ q_1]$
7.	$\delta(q_0, 0, 1) = (q_1, \varepsilon)$	$[q_0\ ^{1}\ q_1] \to 0$
8.	$\delta(q_1, 0, 1) = (q_1, \varepsilon)$	$[q_1\ ^{1}\ q_1] \to 0$
9.	$\delta(q_1, 0, 0) = (q_1, \varepsilon)$	$[q_1\ ^{0}\ q_1] \to 0$
10.	$\delta(q_1, \varepsilon, z_0) = (q_1, \varepsilon)$	$[q_1\ ^{z_0}\ q_1] \to \varepsilon$

Scanned by CamScanner

Simplification of grammar :

We can rename the variables as given below :

$[q_0 \, {}^{Z_0} q_0] - A$, $[q_0 \, {}^{Z_0} q_1] - B$, $[q_1 \, {}^{Z_0} q_0] - C$, $[q_1 \, {}^{Z_0} q_1] - D$

$[q_0 \, {}^{Z_0} q_0] - E$, $[q_0 \, {}^{0} q_1] - F$, $[q_1 \, {}^{0} q_0] - G$, $[q_1 \, {}^{0} q_1] - H$

$[q_0 \, {}^{1} q_0] - I$, $[q_0 \, {}^{1} q_1] - J$, $[q_1 \, {}^{1} q_0] - K$, $[q_1 \, {}^{1} q_1] - L$

With the above substitutions, the resulting set of productions can be written as :

$S \rightarrow A \mid B$

$B \rightarrow \varepsilon$

$A \rightarrow 0EA \mid 0FC$

$B \rightarrow 0EB \mid 0FD$

$E \rightarrow 0EE \mid 0FG$

$F \rightarrow 0EF \mid 0FH$

$E \rightarrow 1IE \mid 1JG$

$F \rightarrow 1IF \mid 1JH$

$I \rightarrow 1II \mid 1JK$

$J \rightarrow 1IJ \mid 1JL$

$J \rightarrow 0$

$L \rightarrow 0$

$H \rightarrow 0$

$D \rightarrow \varepsilon$

1. Removing ε-productions :

Nullable set = {D, B, S}

ε-productions are removed with resulting set of productions as given below :

$S \rightarrow A \mid B$

$A \rightarrow 0EA \mid 0FC$

$B \rightarrow 0EB \mid 0FD \mid 0E \mid 0F$

$E \rightarrow 0EE \mid 0FG \mid 1IE \mid 1JG$

$F \rightarrow 0EF \mid 0FH \mid 1IF \mid 1JH$

$H \rightarrow 0$

$I \rightarrow 1II \mid 1JK$

$J \rightarrow 1IJ \mid 1JL \mid 0$

$L \rightarrow 0$

Scanned by CamScanner

2. Removing non-generating symbols

Set of productions after elimination of non-generating symbols {A, C, D, E, G, I} is given below :

$S \rightarrow B$

$B \rightarrow 0F$

$F \rightarrow 0FH \mid 1JH$

$H \rightarrow 0$

$J \rightarrow 1JL \mid 0$

$L \rightarrow 0$

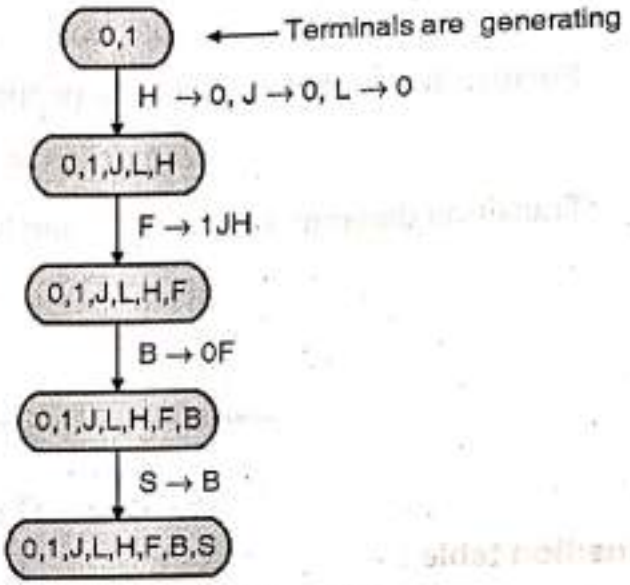


Fig. Ex. 6.6.11

3. The unit production $S \rightarrow B$ should be removed. The set of productions after elimination of the unit production $S \rightarrow B$ is given below :

$S \rightarrow 0F$

$F \rightarrow 0FH \mid 1JH$

$H \rightarrow 0$

$J \rightarrow 1JL \mid 0$

$L \rightarrow 0$

Language accepted by the PDA :

The language accepted by the PDA is given by :

$$L = \{0^n 1^m 0^{n+m} \mid n, m \geq 1\} \cup \varepsilon$$

Example 6.6.12 : Construct a PDA accepting $\{a^n b^m a^n \mid m, n > 1\}$ by null store. From the PDA construct the corresponding CFG.

Solution :

Algorithm :

1. Sequence of a's should be pushed onto the stack in state q_0.

$$\delta(q_0, a, z_0) = (q_0, az_0)$$

$$\delta(q_0, a, a) = (q_0, aa)$$

2. Sequence of input b's should be skipped. These b's will have no effect on the stack.

$$\delta(q_0, b, a) = (q_1, a)$$

$$\delta(q_1, b, a) = (q_1, a)$$

3. Initial a's which are on the stack should be matched with the trailing a's in the input. An 'a' should be popped for every 'a' as input till the end of input.

$$\delta(q_1, a, a) = (q_2, \varepsilon)$$

Scanned by CamScanner

$$\delta(q_2,a,a) = (q_2, \varepsilon)$$

4. Finally, the symbol z_0 should be popped but to make the stack empty.

$$\delta(q_2, \varepsilon,z_0) = (q_2, \varepsilon)$$

Transition diagram and the transition table are given below.

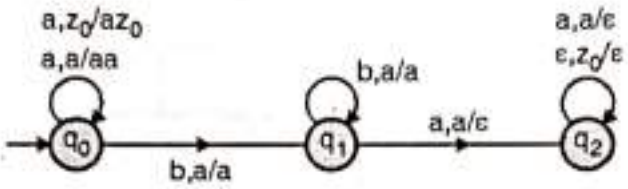


Fig. Ex. 6.6.12 : Transition diagram

Transition table :

$$\delta(q_0,a,z_0) = (q_0,a,z_0)$$
$$\delta(q_0,a,a) = (q_0,aa)$$
$$\delta(q_0,b,a) = (q_1,a)$$
$$\delta(q_1,b,a) = (q_1,a)$$
$$\delta(q_1,a,a) = (q_2, \varepsilon)$$
$$\delta(q_2,a,a) = (q_2, \varepsilon)$$
$$\delta(q_2, \varepsilon, z_0) = (q_2, \varepsilon)$$
$$\text{The PDA M} = (\{q_0,q_1,q_2\},\{a,b\},\{a,b,z_0\},\delta,q_0,z_0,\phi)$$

PDA to CFG :

Step 1 : Add productions for the start symbol

$$S \rightarrow [q_0\ ^{z_0}\ q_0]$$
$$S \rightarrow [q_0\ ^{z_0}\ q_1]$$
$$S \rightarrow [q_0\ ^{z_0}\ q_2]$$

Step 2 : Add productions for $\delta(q_0,a,z_0) = (q_0,az_0)$

$$[q_0\ ^{z_0}\ q_0] \rightarrow a\ [q_0\ ^{a}\ q_0]\ [q_0\ ^{z_0}\ q_0]$$
$$[q_0\ ^{z_0}\ q_0] \rightarrow a\ [q_0\ ^{a}\ q_1]\ [q_1\ ^{z_0}\ q_0]$$
$$[q_0\ ^{z_0}\ q_0] \rightarrow a\ [q_0\ ^{a}\ q_2]\ [q_2\ ^{z_0}\ q_0]$$
$$[q_0\ ^{z_0}\ q_1] \rightarrow a\ [q_0\ ^{a}\ q_0]\ [q_0\ ^{z_0}\ q_1]$$
$$[q_0\ ^{z_0}\ q_1] \rightarrow a\ [q_0\ ^{a}\ q_1]\ [q_1\ ^{z_0}\ q_1]$$

Scanned by CamScanner

$$[q_0\ ^{z_0}\ q_1] \rightarrow a\ [q_0\ ^{a}\ q_2]\ [q_2\ ^{z_0}\ q_1]$$

$$[q_0\ ^{z_0}\ q_2] \rightarrow a\ [q_0\ ^{a}\ q_0]\ [q_0\ ^{z_0}\ q_2]$$

$$[q_0\ ^{z_0}\ q_2] \rightarrow a\ [q_0\ ^{a}\ q_1]\ [q_1\ ^{z_0}\ q_2]$$

$$[q_0\ ^{z_0}\ q_2] \rightarrow a\ [q_0\ ^{a}\ q_2]\ [q_2\ ^{z_0}\ q_2]$$

Step 3 : Add productions for $\delta(q_0,a,a) = (q_0,aa)$

$$[q_0\ ^{a}\ q_0] \rightarrow a\ [q_0\ ^{a}\ q_0]\ [q_0\ ^{a}\ q_0]$$

$$[q_0\ ^{a}\ q_0] \rightarrow a\ [q_0\ ^{a}\ q_1]\ [q_1\ ^{a}\ q_0]$$

$$[q_0\ ^{a}\ q_0] \rightarrow a\ [q_0\ ^{a}\ q_2]\ [q_2\ ^{a}\ q_0]$$

$$[q_0\ ^{a}\ q_1] \rightarrow a\ [q_0\ ^{a}\ q_0]\ [q_0\ ^{a}\ q_1]$$

$$[q_0\ ^{a}\ q_1] \rightarrow a\ [q_0\ ^{a}\ q_1]\ [q_1\ ^{a}\ q_1]$$

$$[q_0\ ^{a}\ q_1] \rightarrow a\ [q_0\ ^{a}\ q_2]\ [q_2\ ^{a}\ q_1]$$

$$[q_0\ ^{a}\ q_2] \rightarrow a\ [q_0\ ^{a}\ q_0]\ [q_0\ ^{a}\ q_2]$$

$$[q_0\ ^{a}\ q_2] \rightarrow a\ [q_0\ ^{a}\ q_1]\ [q_1\ ^{a}\ q_2]$$

$$[q_0\ ^{a}\ q_2] \rightarrow a\ [q_0\ ^{a}\ q_2]\ [q_2\ ^{a}\ q_2]$$

Step 4 : Add productions for $\delta(q_0,b,a) = (q_1,a)$

$$[q_0\ ^{a}\ q_0] \rightarrow b\ [q_1\ ^{a}\ q_0]$$

$$[q_0\ ^{a}\ q_1] \rightarrow b\ [q_1\ ^{a}\ q_1]$$

$$[q_0\ ^{a}\ q_2] \rightarrow b\ [q_1\ ^{a}\ q_2]$$

Step 5 : Add productions for $\delta(q_1,b,a) = (q_1,a)$

$$[q_1\ ^{a}\ q_0] \rightarrow b\ [q_1\ ^{a}\ q_0]$$

$$[q_1\ ^{a}\ q_1] \rightarrow b\ [q_1\ ^{a}\ q_1]$$

$$[q_1\ ^{a}\ q_2] \rightarrow b\ [q_1\ ^{a}\ q_2]$$

Step 6 : Add productions for $\delta(q_1,a,a) = (q_2,\varepsilon)$

$$[q_1\ ^{a}\ q_2] \rightarrow a$$

Step 7 : Add productions for $\delta(q_2,a,a) = (q_2,\varepsilon)$

$$[q_2\ ^{a}\ q_2] \rightarrow a$$

Scanned by CamScanner

Step 8 : Add productions for $\delta(q_2, \varepsilon, z_0) = (q_2, \varepsilon)$

$$[q_2 \, {}^{z_0} \, q_2] \rightarrow \varepsilon$$

Example 6.6.13 : Design a PDA and then corresponding CFG for the language that accepts the simple palindrome.

$$L = \{xcx^R \mid x \in \{a,b\}^*\}$$

Solution :

Transitions for the PDA are given by :

$$\delta(q_0, a, z) = (q_0, az)$$
$$\delta(q_0, b, z) = (q_0, bz)$$
$$\delta(q_0, c, z) = (q_0, \varepsilon)$$
$$\delta(q_0, a, a) = (q_0, aa)$$
$$\delta(q_0, a, b) = (q_0, ab)$$
$$\delta(q_0, b, a) = (q_0, ba)$$
$$\delta(q_0, b, b) = (q_0, bb)$$
$$\delta(q_0, c, a) = (q_1, a)$$
$$\delta(q_0, c, b) = (q_1, b)$$
$$\delta(q_1, a, a) = (q_1, \varepsilon)$$
$$\delta(q_1, b, b) = (q_1, \varepsilon)$$
$$\delta(q_1, \varepsilon, z) = (q_1, \varepsilon)$$

- z is the initial stack symbol.
- q_0 is the initial state.
- String is accepted through an empty stack.

Productions for the corresponding CFG are given below :

Step 1 : Add productions for the start symbol.

$$S \rightarrow [q_0 \, {}^{z} \, q_0]$$
$$S \rightarrow [q_0 \, {}^{z} \, q_1]$$

Step 2 : Add productions for $\delta(q_0, a, z) = (q_0, az)$

$$[q_0 \, {}^{z} \, q_0] \rightarrow a \, [q_0 \, {}^{a} \, q_0] \; [q_0 \, {}^{z} \, q_0]$$
$$[q_0 \, {}^{z} \, q_0] \rightarrow a \, [q_0 \, {}^{a} \, q_1] \; [q_1 \, {}^{z} \, q_0]$$
$$[q_0 \, {}^{z} \, q_1] \rightarrow a \, [q_0 \, {}^{a} \, q_0] \; [q_0 \, {}^{z} \, q_1]$$
$$[q_0 \, {}^{z} \, q_1] \rightarrow a \, [q_0 \, {}^{a} \, q_1] \; [q_1 \, {}^{z} \, q_1]$$

Scanned by CamScanner

Step 3 : Add productions for $\delta(q_0, c, z) = (q_0, \varepsilon)$

$$[q_0 \, ^{z} \, q_0] \rightarrow c$$

Step 4 : Add productions for $\delta(q_0, a, a) = (q_0, aa)$

$$[q_0 \, ^{a} \, q_0] \rightarrow a \, [q_0 \, ^{a} \, q_0] \; [q_0 \, ^{a} \, q_0]$$

$$[q_0 \, ^{a} \, q_0] \rightarrow a \, [q_0 \, ^{a} \, q_1] \; [q_1 \, ^{a} \, q_0]$$

$$[q_0 \, ^{a} \, q_1] \rightarrow a \, [q_0 \, ^{a} q_0] \; [q_0 \, ^{a} \, q_1]$$

$$[q_0 \, ^{a} \, q_1] \rightarrow a \, [q_0 \, ^{a} \, q_1] \; [q_1 \, ^{a} \, q_1]$$

Step 5 : Add productions for $\delta(q_0, a, b) = (q_0, ab)$

$$[q_0 \, ^{b} \, q_0] \rightarrow a \, [q_0 \, ^{a} \, q_0] \; [q_0 \, ^{b} \, q_0]$$

$$[q_0 \, ^{b} \, q_0] \rightarrow a \, [q_0 \, ^{a} q_1] \; [q_1 \, ^{b} \, q_0]$$

$$[q_0 \, ^{b} \, q_1] \rightarrow a \, [q_0 \, ^{a} \, q_0] \; [q_0 \, ^{b} \, q_1]$$

$$[q_0 \, ^{b} \, q_1] \rightarrow a \, [q_0 \, ^{a} q_1] \; [q_1 \, ^{b} \, q_1]$$

Step 6 : Add productions for $\delta(q_0, b, a) = (q_0, ba)$

$$[q_0 \, ^{a} \, q_0] \rightarrow b \, [q_0 \, ^{b} \, q_0] \; [q_0 \, ^{a} \, q_0]$$

$$[q_0 \, ^{a} \, q_0] \rightarrow b \, [q_0 \, ^{b} \, q_1] \; [q_1 \, ^{a} \, q_0]$$

$$[q_0 \, ^{a} \, q_1] \rightarrow b \, [q_0 \, ^{b} q_0] \; [q_0 \, ^{a} \, q_1]$$

$$[q_0 \, ^{a} \, q_1] \rightarrow b \, [q_0 \, ^{b} \, q_1] \; [q_1 \, ^{a} \, q_1]$$

Step 7 : Add productions for $\delta(q_0, b, b) = (q_0, bb)$

$$[q_0 \, ^{b} \, q_0] \rightarrow b \, [q_0 \, ^{b} \, q_0] \; [q_0 \, ^{b} \, q_0]$$

$$[q_0 \, ^{b} \, q_0] \rightarrow b \, [q_0 \, ^{b} \, q_1] \; [q_1 \, ^{b} \, q_0]$$

$$[q_0 \, ^{b} \, q_1] \rightarrow b \, [q_0 \, ^{b} \, q_0] \; [q_0 \, ^{b} \, q_1]$$

$$[q_0 \, ^{b} \, q_1] \rightarrow b \, [q_0 \, ^{b} \, q_1] \; [q_1 \, ^{b} \, q_1]$$

Step 8 : Add productions for $\delta(q_0, c, a) = (q_1, a)$

$$[q_0 \, ^{a} \, q_0] \rightarrow c [q_1 \, ^{a} \, q_0]$$

$$[q_0 \, ^{a} \, q_1] \rightarrow c [q_1 \, ^{a} \, q_1]$$

Scanned by CamScanner

Step 9 : Add productions for $\delta(q_0, c, b) = (q_1, b)$

$$[q_0\ ^{b}\ q_0] \rightarrow c[q_1\ ^{b}\ q_0]$$

$$[q_0\ ^{b}\ q_1] \rightarrow c[q_1\ ^{b}\ q_1]$$

Step 10 : Add productions for $\delta(q_1, a, a) = (q_1, \varepsilon)$

$$[q_1\ ^{a}\ q_1] \rightarrow a$$

Step 11 : Add productions for $\delta(q_1, b, b) = (q_1, \varepsilon)$

$$[q_1\ ^{b}\ q_1] \rightarrow b$$

Step 12 : Add production for $\delta(q_1,\varepsilon,z) = (q_1,\varepsilon)$

$$[q_1\ ^{z}\ q_1] \rightarrow \varepsilon$$

Step 13 : Add productions for $\delta(q_0,b,z) = (q_0,bz)$

$$[q_0\ ^{z}\ q_0] \rightarrow b[q_0\ ^{b}\ q_0]\ [q_0\ ^{z}\ q_0]$$

$$[q_0\ ^{z}\ q_0] \rightarrow b[q_0\ ^{b}\ q_1]\ [q_1\ ^{z}\ q_0]$$

$$[q_0\ ^{z}\ q_1] \rightarrow b[q_0\ ^{b}\ q_0]\ [q_0\ ^{z}\ q_1]$$

$$[q_0\ ^{z}\ q_1] \rightarrow b[q_0\ ^{b}\ q_1]\ [q_1\ ^{z}\ q_1]$$

Example 6.6.14 : For the PDA

$(\{q_0, q_1\}, (0, 1), \{ z_0, x\}, \delta, q_0, z_0, \phi)$ where δ is

$\delta(q_0, 1, z_0) = \{(q_0, xz_0)\}$

$\delta(q_0, 1, x) = \{(q_0, xx)\}$

$\delta(q_0, 0, x) = \{(q_1, x)\}$

$\delta(q_0, \varepsilon, z_0) = \{(q_0, \varepsilon)\}$

$\delta(q_1, 1, x) = \{(q_1, \varepsilon)\}$

$\delta(q_1, 0, z_0) = \{(q_0, z_0)\}$

Obtain CFG accepted by the above PDA and simplify the CFG and describe the language it accepts.

Solution :

Sr. No.	PDA transition	Corresponding productions
1.	Productions due to start symbol S.	$S \rightarrow [q_0\ ^{z_0}\ q_0]$ $S \rightarrow [q_0\ ^{z_0}\ q_1]$

Scanned by CamScanner

Sr. No.	PDA transition	Corresponding productions
2.	$\delta(q_0, 1, z_0) = (q_0, xz_0)$	$[q_0\ ^{z_0}\ q_0] \rightarrow 1\ [q_0\ ^{x}\ q_0]\ [q_0\ ^{z_0}\ q_0]$ $[q_0\ ^{z_0}\ q_0] \rightarrow 1\ [q_0\ ^{x}\ q_1]\ [q_1\ ^{z_0}\ q_0]$ $[q_0\ ^{z_0}\ q_1] \rightarrow 1\ [q_0\ ^{x}\ q_0]\ [q_0\ ^{z_0}\ q_1]$ $[q_0\ ^{z_0}\ q_1] \rightarrow 1\ [q_0\ ^{x}\ q_1]\ [q_1\ ^{z_0}\ q_1]$
3.	$\delta(q_0, 1, x) = (q_0, xx)$	$[q_0\ ^{x}\ q_0] \rightarrow 1\ [q_0\ ^{x}\ q_0]\ [q_0\ ^{x}\ q_0]$ $[q_0\ ^{x}\ q_0] \rightarrow 1\ [q_0\ ^{x}\ q_1]\ [q_1\ ^{x}\ q_0]$ $[q_0\ ^{x}\ q_1] \rightarrow 1\ [q_0\ ^{x}\ q_0]\ [q_0\ ^{x}\ q_1]$ $[q_0\ ^{x}\ q_1] \rightarrow 1\ [q_0\ ^{x}\ q_1]\ [q_1\ ^{x}\ q_1]$
4.	$\delta(q_0, 0, x) = (q_1, x)$	$[q_0\ ^{x}\ q_0] \rightarrow 0[q_1\ ^{x}\ q_0]$
		$[q_0\ ^{x}\ q_1] \rightarrow 0[q_1\ ^{x}\ q_1]$
5.	$\delta(q_0, \varepsilon, z_0) = (q_0, \varepsilon)$	$[q_0\ ^{z_0}\ q_0] \rightarrow \varepsilon$
6.	$\delta(q_1, 1, x) = (q_1, \varepsilon)$	$[q_1\ ^{x}\ q_1] \rightarrow 1$
7.	$\delta(q_1, 0, z_0) = (q_1, z_0)$	$[q_1\ ^{z_0}\ q_0] \rightarrow 0\ [q_0\ ^{z_0}\ q_0]$
		$[q_1\ ^{z_0}\ q_1] \rightarrow 0\ [q_0\ ^{z_0}\ q_1]$

Simplification of grammar :

We can rename the variables as given below :

$[q_0\ ^{z_0}\ q_0]$ – A, $[q_0\ ^{z_0}\ q_1]$ – B, $[q_1\ ^{z_0}\ q_0]$ – C, $[q_1\ ^{z_0}\ q_1]$ – D

$[q_0\ ^{x}\ q_0]$ – E, $[q_0\ ^{x}\ q_1]$ – F, $[q_1\ ^{x}\ q_0]$ – G, $[q_1\ ^{x}\ q_1]$ – H

With the above substitutions, the resulting set of productions can be written as :

$S \rightarrow A \mid B$

$A \rightarrow 1EA \mid 1FC$

$B \rightarrow 1EB \mid 1FD$

$E \rightarrow 1EE \mid 1FG$

$F \rightarrow 1EF \mid 1FH$

Scanned by CamScanner

$E \rightarrow 0G$
$F \rightarrow 0H$
$A \rightarrow \varepsilon$
$H \rightarrow 1$
$C \rightarrow 0A$
$D \rightarrow 0B$

1. Removing ε-production.

Nullable set = {S, A}

ε-productions are removed with resulting set of productions as given below :

$S \rightarrow A \mid B$
$A \rightarrow 1EA \mid 1FC \mid 1E$
$B \rightarrow 1EB \mid 1\ FD$
$C \rightarrow 0A \mid 0$
$D \rightarrow 0B$
$E \rightarrow 1EE \mid 1\ FG \mid 0G$
$F \rightarrow 1EH \mid 1FH \mid 0H$
$H \rightarrow 0$

2. Removing non-generating symbols

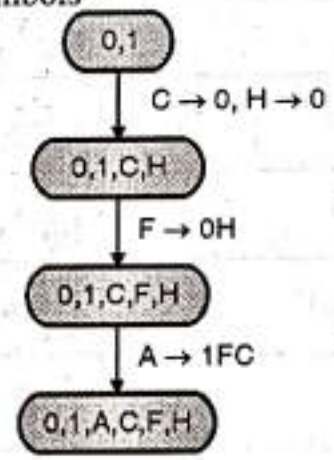


Fig. Ex. 6.6.14

Following symbols are non-generating

= (B, D, E, G)

Set of productions after elimination of non-generating symbols :

$S \rightarrow A$
$A \rightarrow 1FC$
$C \rightarrow 0A \mid 0$
$F \rightarrow 1FH \mid 0H$
$H \rightarrow 0$

3. Unit production $S \rightarrow A$ is removed, the resulting set of productions :

$S \rightarrow 1FC$

Scanned by CamScanner

$A \rightarrow 1FC$

$C \rightarrow 0A \mid 0$

$F \rightarrow 1FH \mid 0H$

$H \rightarrow 0$

The language accepted by the PDA is given by :

$$L = \{1^n 0 1^n 0 \ldots\ldots 1^n 0 \mid n \geq 1\} \cup \varepsilon$$

Example 6.6.15 : Show that if a language L is accepted by a PDA then there exists a CFG generating L.

Solution :

Proof :

For simplicity, we will consider a normalized PDA with following properties:

1. There is a single final state q_f
2. It empties the stack before accepting
3. Each transition either pushes a symbol onto the stack or performs a **pop** operation, but not both.

A transition,

$$\delta(q_i, x, b) \Rightarrow (q_j, c) \text{ [pop b and push c]}$$

can be replaced by couple of transitions.

1. $\delta(q_i, x, b) \Rightarrow \delta(q_{temp}, \varepsilon)$
2. $\delta(q_{temp}, \varepsilon, \varepsilon) \Rightarrow \delta(q_j, c)$

A new state q_{temp} has been introduced.

Similarly, a transition that neither pushes nor pops anything can be replaced with two transitions :

1. Transition pushing a dummy stack symbol
2. Transition popping the dummy stack symbol.

Let us consider a normalized PDA,

$$N = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F), \text{ where } F = \{ q_f \}$$

- A word w will be accepted by N if it starts in q_0 and ends up in q_f with an empty stack.
- The language $L_{p, q}$, where $p, q \in Q$, is defined as consisting of those strings that start at state p with an empty stack and ends up in q with an empty stack.

Example 6.6.16 : Describe the language L(M) in English for push down automation.

$$M = (K, \Sigma, \Gamma, \Delta, S, F)$$

where, $k = \{S, F\}$

$F = \{F\}$

$\Sigma = \{a, b\}$

Scanned by CamScanner

$T = \{a\}$

$\Delta = \{((S, a, \varepsilon), (S, a)), ((S, b, \varepsilon), (S, a)), ((S, a, \varepsilon), (F, \varepsilon)), ((F, a, a), (F, \varepsilon)), ((F, b, a), (F, \varepsilon))\}$

Solution :

It recognizes a string of the form $(a + b)^m a(a + b)^n \mid n \leq m$ and $n, m \geq 0$

- The middle character 'a' is determined non-deterministically.
- For every symbol from the first $(a + b)^m$ of $(a + b)^m$ a $(a + b)^n$ an 'a' is pushed onto the stack using the given two moves.

  $((S, a, \varepsilon),(S,a)),((S,b,\varepsilon),(S,a))$
- The middle a of the input string $(a + b)^m a (a + b)^n$ takes the PDA to state F. 'a' is fixed non-deterministically.
- For every symbol from $(a + b)^n$ of $(a + b)^m$ a $(a + b)^n$ an a is erased from the stack in state F.

6.7 Deterministic Pushdown Automata (DPDA)

MU - Dec. 2009, May 2013, May 2017

In a DPDA there is only one move in every situation. A DPDA is less powerful than NPDA. Every context free language cannot be accepted by a DPDA. For example, a string of the form ww^R can not be processed by a DPDA. The class of a language a DPDA can accept lies in between a regular language and CFL.

A DPDA is defined as :

$M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$, where

$\delta(q, a, x)$ has one move for any $q \in Q$, $X \in \Gamma$ and $a \in \Sigma$.

6.7.1 Regular Language and DPDA

We can always design a DPDA for a regular language. A DPDA can be designed for regular language in two steps :

Step 1 : Construct an equivalent DFA for the given regular language.

Step 2 : For every transition $\delta(q_i, a) = q_j$ in FA (where $q_i, q_j \in Q$ and $a \in \Sigma$), we can write an equivalent transition for DPDA.

$$\delta(q_i, a, z_0) = \{(q_j, z_0)\}.$$

The move for PDA involves neither a **Push** nor a **Pop** operation.

Scanned by CamScanner

Example 6.7.1 : Design a DPDA for a binary number divisible by 3.

Solution :

Step 1 : Construction of DFA.

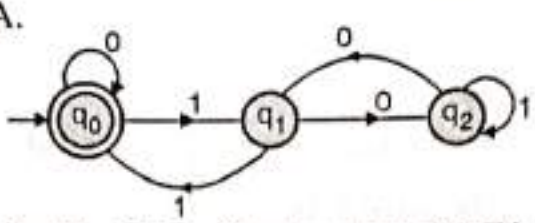


Fig. Ex. 6.7.1 : Construction of DFA

Step 2 : The DPDA is given by :

$$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \{0, 1, 1, z_0\}, \delta, q_0, z_0, \{q_0\})$$

where the transition function δ is :

$$\delta(q_0, 0, z_0) = (q_0, z_0)$$
$$\delta(q_0, 1, z_0) = (q_1, z_0)$$
$$\delta(q_1, 0, z_0) = (q_2, z_0)$$
$$\delta(q_1, 1, z_0) = (q_0, z_0)$$
$$\delta(q_2, 0, z_0) = (q_1, z_0)$$
$$\delta(q_2, 1, z_0) = (q_2, z_0)$$

Example 6.7.2 : Show that if L is accepted by a PDA in which no symbols are removed from the stack, then L is regular.

Solution : Every regular language is accepted by some FA. Every transition of an FA can be converted into a PDA move without any push or pop operation.

Let there be a transition $\delta(q_i, a) = q_j$ in the FA. Where,

$$q_i, q_j \in Q \quad \text{and} \quad a \in \Sigma.$$

The FA move $\delta(q_i, a) = q_j$ can be converted into an equivalent PDA move as given below :

$$\delta(q_i, a, z_0) = \{(q_j, z_0)\}$$

This move for the PDA involves neither a **Push** nor a **Pop** operation.

Example 6.7.3 : Prove "Let L be a language accepted by deterministic PDA, then the complement of L, can also be accepted by a DPDA".

Solution :

Let the DPDA for the given language L is

$$M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$$

We can construct M′ from M, such that M′ accepts L′.

$$M' = (Q, \Sigma, \Gamma, \delta, q_0, z_0, Q\text{-}F)$$

Scanned by CamScanner

i.e. an accepting state in M becomes a non-accepting state in M′ and a non-accepting state in M becomes an accepting state in M′.

Proof that M′ accepts L′ :

Case I : Let us take a string $\omega \in L$.

On application of ω, the machine M will reach a final state.

$$(q_0, \omega, z_0) \overset{*}{\underset{M}{\vdash}} (p, -), \text{ where } p \in F.$$

Since the state of $p \notin Q\text{-}F$, it will not be accepted by M′.

Case II : Let us take a string $\omega \in \Sigma^*$ but $\omega \notin L$. i.e. $\omega \in L'$.

On application of ω, the machine M will reach a non-final state.

$$(q_0, \omega, z_0) \overset{*}{\underset{M}{\vdash}} (r, -), \text{ where } r \notin F.$$

Since, the state $r \in Q\text{-}F$, it will be accepted by M′.

Example 6.7.4 : Enlist the difference between PDM and FSM.

Solution :

Difference :

1. PDM is more powerful than FSM.
2. PDM has additional memory in the form of a stack.
3. PDM can handle CFL but FSM can not handle CFL.

6.8 Application of PDA

MU - Dec. 2012

PDA is a machine for CFL. A string belonging to a CFL can be recognized by a PDA. PDA is extensively used for parsing. PDA is an abstract machine; it can also used for giving proofs of lemma on CFL.

6.9 Parsing

We can always find a method for determining whether a particular string is generated by a CFG. Parsing a string is nothing but finding a derivation of the string in the given grammar G. A great deal of work has been done in finding an efficient algorithm for parsing. These algorithms depend on specific properties of the grammar.

PDA is a machine for CFG. A PDA can be used for parsing. A PDA can be enhanced to record its moves, so that the sequence of moves leading to an acceptance of the string can be remembered.

Scanned by CamScanner

6.9.1 Top-Down Parsing

A top down parser for a given grammar G tries to derive a string through a sequence of derivations starting with the start symbol.

A top down parser, normally uses leftmost derivation to derive an input string.

- **Recursive-descent parser** is a general top down parser.
- A recursive descent parser may require backtracking and repeated scan of input string. working of a recursive descent parser is being explained with the help of an example.

Example : Consider the grammar

$S \rightarrow aXb$

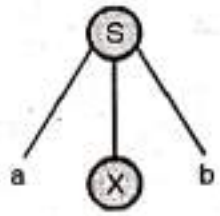
$X \rightarrow ab \mid b$

and the input string abb.

The parse tree of abb can be constructed as given below :

Step 1 : We create a parse tree with single node S. S is the start symbol.

Step 2 : We use the first production $S \rightarrow aXb$ to expand the node



The leftmost leaf, labelled a, matches the first symbol of input string abb. Input pointer is advanced to the next symbol b of abb.

Step 3 : Now, X is expanded using $X \rightarrow ab$ and if it fails to generate remaining input symbols, we backtrack and try the next production $X \rightarrow b$.

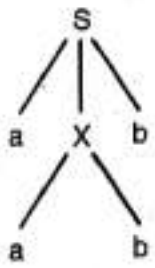


Fig. 6.9.1

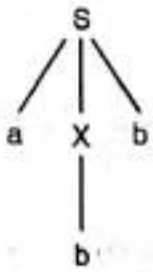


Fig. 6.9.2

The first tree will not generate the input string abb. The leftmost symbol 'a' of subtree, rooted at X does not match the next input symbol 'b'. We must backtrack and try the next production $X \rightarrow b$, as shown in Fig. 6.9.2.

- A recursive-descent parser may enter an infinite loop.

Scanned by CamScanner

- We can write a predictive parser (without backtracking) by modifying the grammar :
  1. Left recursion should be eliminated.
  2. Apply left factoring to the grammar.

  We can easily write a program for a recursive descent parser.
- There must be a function corresponding to each variable.
- A global variable 'position' points to the current input character.
- If the current input character matches the terminal in the production then the next input character is read by advancing the variable 'position' to the next location.

Example 6.9.1 : Write a program for recursive descent parser for the following grammar.

$E = E + T \mid T$

$T = T * F \mid F$

$F = (E) \mid a \mid b$

Solution :

Step 1 : Removing left recursion from the grammar we get :

$$E = TE_1$$
$$E_1 = +TE_1 \mid \varepsilon$$
$$T = FT_1$$
$$T_1 = *FT_1 \mid \varepsilon$$
$$F = (E) \mid a \mid b$$

Step 2 : The program will have a function for each variable.

Function corresponding to $E = TE_1$

```
E( )
  { T( );
    E1( );
  }
```

Function corresponding to $E_1 = +TE_1 \mid \varepsilon$

```
E1( )
    {    if (input [position] == '+')
         {    match( );
              T( );
              E1( );
         }
    }
```

Scanned by CamScanner

```
//The function match ( ) will increment the current position pointer for input string by 1.
```

Function corresponding to $T = F\ T_1$

```
T( )
{  F( );
   T1( );
}
```

Function corresponding to $T_1 = *F\ T_1 \mid \varepsilon$

```
T1( )
  {  if(input [position] == '*')
           {  match( );
              F( );
              T1( );
           }
  }
```

Function corresponding to F = (E) | a | b

```
F( )
  {
          if(input[position] == 'C')
          {       match( );
                  E ( );
                  if (input[position] == ')')
                      match( );
                  else
                      error =1;
          }
          else
            if(input [position] == 'a'  || input [position] == 'b')
                        match( );
  }
// main program
char input [30];
int error = 0, position = 0;
void main( )
  {
```

Scanned by CamScanner

```
    printf("\n enter a string :");
    gets(input);
    E( );
    if(error == 1 || input [position]! = '\0')
        printf("\n invalid string");
    else
        printf("\n valid string");
}
```

6.9.2 Bottom-Up Parsing

In bottom up parsing, the source string is reduced to the start symbol of the grammar. The bottom up parsing is also known as shift-reduce parsing.

- A shift reduce parser constructs a parse tree by beginning at the leaves and then working up towards the root.
- A shift reduce parser can be constructed using a stack.

Implementation of shift reduce parser

- A stack is taken to hold grammar symbols.
- An array is taken to store input string to be parsed.
- Initially, the stack is empty.

At each step the parser can take one of the following two steps :

1. Reduce : The process of reduction is shown in the Fig. 6.9.3.
   If there is a production $X \rightarrow \alpha$ and the string α is found at the top end of the stack then α should be replaced by X.

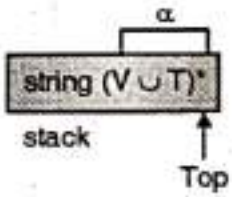


Fig. 6.9.3

2. If the reduction step cannot be carried out then the next input symbol is shifted on the stack.

The parser repeats this cycle of shift-reduce until it has detected an error or until the stack contains the start symbol and the input is empty.

Example 6.9.2 : Consider the grammar

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

and the input string id + id * id. Show the working of the shift reduce parser.

Scanned by CamScanner

Solution :

Sr. No.	Stack	Input	Action
1.	empty	id + id *id	Shift
2.	id	+ id * id	Reduce by F → id
3.	F	+ id * id	Reduce by T → F
4.	T	+ id * id	Reduce by E → T
5.	E	+ id * id	Shift
6.	E +	id * id	Shift
7.	E + id	* id	Reduce by F → id
8.	E + F	* id	Reduce by T → F
9.	E + T	* id	Shift
10.	E + T*	id	Shift
11.	E + T * id	–	Reduce by F → id
12.	E + T * F	–	Reduce by T → T * F
13.	E + T	–	Reduce by
14.	E	–	Accept

6.10 Multistack DPDA

MU - May 2017

Adding multiple stacks to a PDA enhances its power. A PDA with two or more stacks can accept any language that a Turing machine can accept. A PDA with two stacks is shown in Fig. 6.10.1.

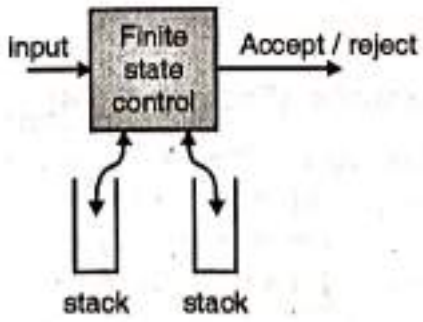


Fig. 6.10.1 : A PDA with 2 stacks

- A K-stack PDA is a deterministic PDA with K stacks.
- It accepts an input like a normal PDA.

Scanned by CamScanner

- In each move, the multistack machine can :
  1. Change to a new stack.
  2. Replace the top symbol of each stack with a string of zero or more stack symbols.

6.11 Power of Various Machines

MU - Dec. 2012

- A post machine can simulate functioning of a turing machine. Hence a turing machine and a post machine have equal power.
- A post machine is more powerful than a PDA. A string of the $a^n b^n c^n$ can be handled by a post machine but it can not be handled by a PDA.
- An NPDA is more powerful than a DPDA. A string of the form $\omega\omega^R$ can be handled by an NPDA but it cannot be handled by a DPDA.
- DPDA is more powerful than FA. A string of the form $a^n b^n$ can be handled by a DPDA but it cannot be handled by FA.

6.12 University Questions and Answers

Dec. 2005

Q. 1 Distinguish between NPDA and DPDA. **(Section 6.5)** **(3 Marks)**

Q. 2 Design a PDA to accept $(bdb)^n C^n$. **(Example 6.4.5)** **(7 Marks)**

Q. 3 Design a suitable PDA for even palindrome. **(Example 6.5.2)** **(10 Marks)**

Q. 4 Write short note on power of PDA. **(Section 6.1)** **(5 Marks)**

May 2006

Q. 5 Construct the PDA equivalent to the following context free grammar :

$S \rightarrow 0BB$

$B \rightarrow 0S \mid 1S \mid 0$

Test whether 010^4 is in language. **(Example 6.6.4)** **(10 Marks)**

Q. 6 Design the PDA that will recognize the language $L = WW^R$: W is in $\{a, b\}^*$ i.e. even length palindrome over $\Sigma = \{a, b\}$ **(Example 6.5.2)** **(10 Marks)**

Dec. 2006

Q. 7 Construct PDA accepting

$L = \{ a^n b^m a^n \mid m, n \geq 1 \}$ **(Example 6.4.10)** **(5 Marks)**

Scanned by CamScanner

May 2007

Q. 8 Design a PDA to accept $(ab)^n (cd)^n$ **(Example 6.4.4)** **(5 Marks)**

Q. 9 Design a suitable PDA to accept an even palindrome over {a, b} **(Example 6.5.2)** **(10 Marks)**

Dec. 2007

Q. 10 Design the PDA to accept language containing all odd length palindromes over $\Sigma = \{0, 1\}$. **(Example 6.5.1)** **(10 Marks)**

Q. 11 Give the CFG generating the language accepted by the following PDA :

$M = (\{ q_0, q_1\}, \{0, 1\}, \{z_0, x\}, \delta, q_0, z_0, \phi)$ when δ is given below :

$\delta(q_0, 1, z_0) = \{(q_0, xz_0)\}$

$\delta(q_0, 1, x) = \{(q_0, xx)\}$

$\delta(q_0, 0, x) = \{(q_1, x)\}$

$\delta(q_0, \varepsilon, z_0) = \{(q_0, \varepsilon)\}$

$\delta(q_1, 1, x) = \{(q_1, \varepsilon)\}$

$\delta(q_1, 0, z_0) = \{(q_0, z_0)\}$ **(Example 6.6.9)** **(5 Marks)**

May 2008

Q. 12 Design a PDA for accepting a language

$L = \{ WcW^R \mid W \in \{a, b\}^* \}$ **(Example 6.4.11)** **(10 Marks)**

Dec. 2008

Q. 13 Prove the variations and equivalence of the push down automata.

(Section 6.4) **(10 Marks)**

Q. 14 Convert the following expression grammar to PDA

$I \rightarrow a \mid b \mid Ia \mid Ib \mid ID \mid I1$

$E \rightarrow I \mid E * E \mid E + E \mid (E)$ **(Example 6.6.6)** **(10 Marks)**

May 2009

Q. 15 Design a PDA for CFL that checks the well formedness of parenthesis i.e. the language L all "balanced" string of two types of parenthesis say "( )" and "[ ]". Trace the sequence of moves made corresponding to input string (([ ])[ ]).

(Example 6.4.13) **(8 Marks)**

Scanned by CamScanner

Q. 16 Consider the PDA with the following moves :

$\delta(q_0, a, z_0) = \{(q_0, az_0)\}$

$\delta(q_0, a, a) = \{(q_0, aa)\}$

$\delta(q_0, b, a) = \{(q_1, \varepsilon)\}$

$\delta(q_1, b, a) = \{(q_1, \varepsilon)\}$

$\delta(q_1, \varepsilon, z_0) = \{(q_1, \varepsilon)\}$

Obtain CFG equivalent to PDA. **(Example 6.6.10)** **(10 Marks)**

Dec. 2009

Q. 17 Write short note on DPDA. **(Section 6.7)** **(5 Marks)**

May 2010

Q. 18 Differentiate Deterministic Push Down Automata and Non-deterministic Push Down Automata. **(Section 6.5)** **(5 Marks)**

Q. 19 Design PDA for the following language

L (M) = { WcW^R } W ε { a, b }* where W^R is reverse of W and C is constant.

(Example 6.4.11) **(10 Marks)**

Dec. 2010

Q. 20 Construct a PDA accepting the following language $L = \{a^n b^m a^n \mid m,n >= 1\}$

(Example 6.4.10) **(5 Marks)**

May 2011

Q. 21 Design a PDA for the language L = { WcW^R | W ∈ {a, b}* }

(Example 6.4.11) **(10 Marks)**

Q. 22 Design a PDA for the following grammar and test whether 010^4 is in the language defined by that PDA.

$S \rightarrow 0BB$

$B \rightarrow 0S \mid 1S \mid 0$ **(Example 6.6.4)** **(10 Marks)**

Dec. 2011

Q. 23 Construct PDA for the following language :

$L = \{0^m 1^n 0^{m+n} \mid m, n \geq 1\}$ **(Example 6.4.14)** **(8 Marks)**

Q. 24 Differentiate between DPDA and NPDA. **(Section 6.5)** **(4 Marks)**

Q. 25 Define PDA and construct PDA for the grammar :

$E \rightarrow E + E \mid E - E \mid (E) \mid id$ **(Section 6.7 and Example 6.4.15)** **(10 Marks)**

Scanned by CamScanner

May 2012

Q. 26 Design a PDA to accept each of the following languages
(i) $\{0^n 1^m 0^n \mid m, n \geq 1\}$ **(Example 6.4.10, Refer 0 as a and 1 as b)**
(ii) $(0^n 1^m 0^m 0^n \mid m, n \geq 1)$ **(Example 6.4.14)** **(10 Marks)**

Q. 27 Convert the grammar
$S \rightarrow 0AA$
$A \rightarrow 0S \mid 1S \mid 0$
to a PDA that accepts the same language by empty stack.**(Example 6.6.4)** **(10 Marks)**

Dec. 2012

Q. 28 Explain PDA and power of PDM. **(Sections 6.1 and 6.12)** **(10 Marks)**

Q. 29 Construct a PDA accepting the language of Palindromes.
(Example 6.5.3) **(10 Marks)**

Q. 30 Write application of PDA. **(Section 6.8)** **(5 Marks)**

May 2013

Q. 31 Explain DPDA with languages of them.
(Sections 6.5 and 6.7) **(10 Marks)**

Q. 32 Construct the PDA accepting following language
$L = \{a^n b^m a^n \mid m, n >= 1\}$ **(Example 6.4.10)** **(10 Marks)**

May 2014

Q. 33 Design PDA that checks for well-formed parentheses. **(Example 6.4.13)** **(10 Marks)**

Dec. 2014

Q. 34 Design a PDA to accept language $\{a^{n-1} b^{2n+1} \mid n \geq 1\}$ **(10 Marks)**

Ans. : [Hint : Refer as Example 6.4.8(A)]

- For every 'a' in the input, 2 b's are pushed onto the stack.
- For the first 'b' in the input, 2 b's are pushed onto the stack.
- For every 'b' in the input, 1 'b' is popped out from the stack.
- Finally the stack should become empty.

Transitions :

$$\delta(q_0, a, z_0) = (q_0, bb\, z_0)$$

Scanned by CamScanner

$\delta(q_0, a, b) = (q_0, bbb)$

$\delta(q_0, b, z_0) = (q_1, bb\, z_0)$

$\delta(q_0, b, b) = (q_1, bbb)$

$\delta(q_1, b, b) = (q_1, \in)$

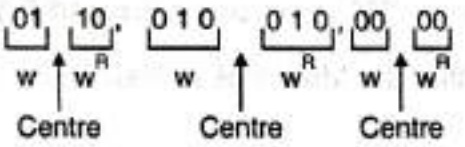
$\delta(q_1, \in, z_0) = (q_1, \in)$

[Accept using empty stack]

Q. 35 Design PDA to check even palindrome over $\Sigma = \{0, 1\}$ **(10 Marks)**

Ans. : [Hint : Refer as Example 6.5.2(A)]

An even palindrome will be of the form ww^R



If the length of w is n then a palindrome of even length is :

First n characters are equal to the last n characters in the reverse order.

The character immediately before the middle position will be identical to the character immediately after the middle position.

Algorithm :

There is no way of finding the middle position by a PDA; therefore the middle position is fixed non-deterministically.

1. First n characters are pushed onto the stack. n is non-deterministic.
2. The n characters on the stack are matched with the last n characters of the input string.
3. n is decided non-deterministically. Every character out of first n characters, whose previous character is same as itself should be considered for two cases :

   (a) It is first character of the second half.

   - Pop the current stack symbol using the transitions :

$\delta(q_0, 0, 0) \Rightarrow (q_1, \varepsilon)$

$\delta(q_0, 1, 1) \Rightarrow (q_1, \varepsilon)$

Must be identical

   (b) It belongs to first half.

   - Push the current input

$\delta(q_0, 0, \varepsilon) \Rightarrow (q_0, 0)$

$\delta(q_0, 1, \varepsilon) \Rightarrow (q_0, 1)$

Scanned by CamScanner

4. n is decided non-deterministically. Every character out of first n characters, whose previous character is not same as itself should be pushed onto the stack.

- Push the current symbol using the transitions :

$\delta(q_0, 0, 1) \Rightarrow (q_0, 01)$

$\delta(q_0, 1, 0) \Rightarrow (q_0, 10)$

The transition table for the PDA is given below :

$\delta(q_0, 0, z_0) \Rightarrow \{(q_0, 0z_0)\}$

$\delta(q_0, 1, z_0) \Rightarrow \{(q_0, 1z_0)\}$

$\delta(q_0, 0, 0) \Rightarrow \{(q_0, 00)\ (q_1, \varepsilon)\}$

$\delta(q_0, 0, 1) \Rightarrow \{(q_0, 01)\}$

$\delta(q_0, 1, 0) \Rightarrow \{(q_0, 10)\}$

$\delta(q_0, 1, 1) \Rightarrow \{(q_0, 11), (q_1, \varepsilon)\}$

$\delta(q_1, 0, 0) \Rightarrow \{(q_1, \varepsilon)\}$

$\delta(q_1, 1, 1) \Rightarrow \{(q_1, \varepsilon)\}$

$\delta(q_1, \varepsilon, z_0) \Rightarrow \{(q_1, \varepsilon)\}$ [Accept through an empty stack]

Where, the set of states $Q = \{q_0, q_1\}$

the set of input symbols $\Sigma = \{0, 1\}$

the set of stack symbols $\Gamma = \{0, 1, z_0\}$

Starting state $= q_0$

Initial stack symbol $= z_0$

May 2015

Q. 36 Construct PDA accepting the language $L = \{a^n, b^n | n > 0\}$ **(Section 6.1)** **(10 Marks)**

Q. 37 Write short note on : Difference between PDA and NPDA. **(Section 6.5)** **(5 Marks)**

Dec. 2015

Q. 38 Design DPDA to accept language $L = \{x \in \{a, b\}^* \ N_a(x) > N_b(x)\}$,
$N_a(x) > N_b(x)$ means number of a's are greater than number of b's in string x.
(10 Marks)

Ans. : [Hint : Refer Example as Example 6.6.6(A)]

The PDA is being designed to accept the string using final state. The stack is being used to store excess of a's over b's or excess of b's over a's out of input seen so far.

Scanned by CamScanner

Transitions

1. $\delta(q_0, a, z_0) = (q_0, a\, z_0)$ [Extra 'a' is pushed]
2. $\delta(q_0, b, z_0) = (q_0, b\, z_0)$ [Extra 'b' is pushed]
3. $\delta(q_0, a, a) = (q_0, aa)$ [Excess a's are pushed]
4. $\delta(q_0, a, b) = (q_0, \in)$ [Excess b's decreased by 1]
5. $\delta(q_0, b, b) = (q_0, bb)$ [Excess b's are pushed]
6. $\delta(q_0, b, a) = (q_0, \in)$ [Excess a's decreased by 1]
7. $\delta(q_0, \in, a) = (q_1, \in)$ [Input ends with excess a's on the stack]

The PDA is given by :

$$M = (\{q_0, q_1\}\ \{a, b\}, \{a, b, z_0\}, \delta, q_0, z_0, \{q_1\})$$

May 2016

Q. 4(b) Construct PDA accepting the language $L = \{ a^{2n} b^n \mid n > 0 \}$. **(10 Marks)**

Ans. : [Hint : Refer Example as Example 6.4.16]

Algorithm :

1. For every pair of leading a's, one X is inserted in the stack.
2. X's on the stack are matched with trailing b's.

The PDA is given by

$M = (\{q_0, q_1, q_2, q_3\}, \{a, b\}\ \{X, Z_0\}, \delta, q_0, Z_0, \phi)$

where the transition function δ is

1. $\delta(q_0, a, Z_0) = (q_1, Z_0)$
2. $\delta(q_1, a, Z_0) = (q_2, X\, Z_0)$
3. $\delta(q_2, a, X) = (q_1, X)$
4. $\delta(q_1, a, X) = (q_2, XX)$
5. $\delta(q_2, b, X) = (q_3, \in)$
6. $\delta(q_3, b, X) = (q_3, \in)$
7. $\delta(q_3, \in, Z_0) = (q_3, \in)$

Accept through empty stack.

Q. 39 Design PDA to check even palindrome over $\sum = \{0, 1\}$ **(Example 6.5.2)** **(10 Marks)**

Dec. 2016

Q. 40 Give formal definition of a Push Down Automata. **(Section 6.2)** **(5 Marks)**

Scanned by CamScanner

Q. 41 Design a PDA corresponding to the grammar : **(10 Marks)**

$S \rightarrow aSA \mid \in$

$A \rightarrow bB$

$B \rightarrow b$

Ans. : [Hint : Please refer Example as Example 6.6.4(A)]

The equivalent PDA, M is given by :

$M = (\{q\}, \{a, b\}, \{a, b, S, A, B\}, \delta, q, S, \phi)$

where δ is given by :

$\delta(q, \in, S) \Rightarrow \{(q, aSA), (q, 6)\}$

$\delta(q, \in, A) \Rightarrow \{(q, bB)\}$

$\delta(q, \in, B) = \{(q, b)\}$

$\delta(q, a, a) = \{(q, \in)\}$

$\delta(q, b, b) = \{(q, \in)\}$

May 2017

Q. 42 Write short note on : Deterministic PDA and multistack PDA. **(5 Marks)**
(Sections 6.7 and 6.10)

Dec. 2017

Q. 43 Design a PDA to accept language $\{ a^{n-1} b^{2n+1} \mid n >= 1 \}$ **(10 Marks)**

Ans. : [Hint : Refer Example as Example 6.4.17]

1. $\delta(q_0, a, Z_0) \Rightarrow (q_1, aaZ_0)$
2. $\delta(q_1, a, a) \Rightarrow (q_1, aa)$
3. $\delta(q_1, b, a) \Rightarrow (q_2, a)$
4. $\delta(q_2, b, a) \Rightarrow (q_1, \in)$
5. $\delta(q_2, \in, Z_0) \Rightarrow (q_2, \in)$

Accept through empty stack.

Q. 44 Design a PDA that checks for well-formed parenthesis. **(10 Marks)**
(Example 6.4.13)

Q. 45 Write short note on Differentiation between DPDA and NPDA. **(5 Marks)**
(Section 6.5)

Chapter Ends...

❑❑❑

Scanned by CamScanner

CHAPTER 7

Module 5

Turing Machine (TM)

7.1 Introduction to Turing Machine

MU - Dec. 2010, Dec. 2012, May 2013

Turing machine is an example of computing machine. So far we have discussed three types of machines :

1. Finite state machine
2. Pushdown machine
3. Post machine

These machines have no control over the input and they cannot modify their own inputs. Turing machine is a writing machine; it can modify its own input symbols. Turing machine is more powerful than a pushdown machine. Power of various machines is shown below :

$$FA \leq DPDA \leq NPDA \leq \text{Post machine} = \text{Turing machine}$$

Turing machine is capable of performing computations on inputs and producing a new result.

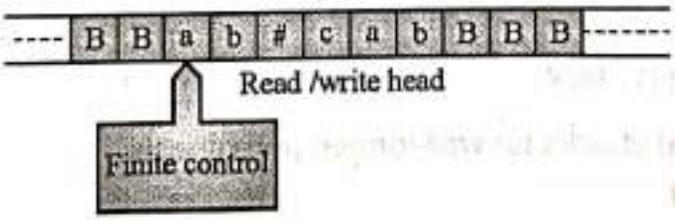


Fig. 7.1.1 : An example of a Turing Machine

An abstract model of a turing machine is shown in Fig. 7.1.1.

- Input to a turing machine is provided through a long tape.
- Turing machine is provided with a read / write head.

Scanned by CamScanner

- The tape is divided into squares. Each square holds a single symbol.
- Blank squares hold a special character 'B'.
- The head is capable of performing three operations :
  1. Reading a symbol being scanned.
  2. Modifying a symbol being scanned.
  3. Shifting either to previous square (L) or next square (R).

Example 7.1.1 : Design a turing machine to perform the following computation :

Initially the tape contains two finite blocks of 1's separated by finite block of blanks. The machine should delete the block of blanks between two blocks of 1's.

(a) Input tape

(b) Output tape

Fig. Ex. 7.1.1 : Input, Output tapes

Solution :

Let us call the first block of 1's as ω_1 and the second block of 1's as ω_2.

There are three blanks between ω_1 and ω_2. These three blanks will be deleted in three cycle.

Each cycle will consist of following steps :

Step 1 : Leftmost 1 of ω_1 is erased.

Step 2 : Head moves to first 'B' between ω_1 and ω_2 and replaces it by 1.

Step 3 : If some B's are yet to be deleted then head comes back to first 1 of ω_1.

These cycles of computation are shown in Fig. Ex. 7.1.1(c).

Scanned by CamScanner

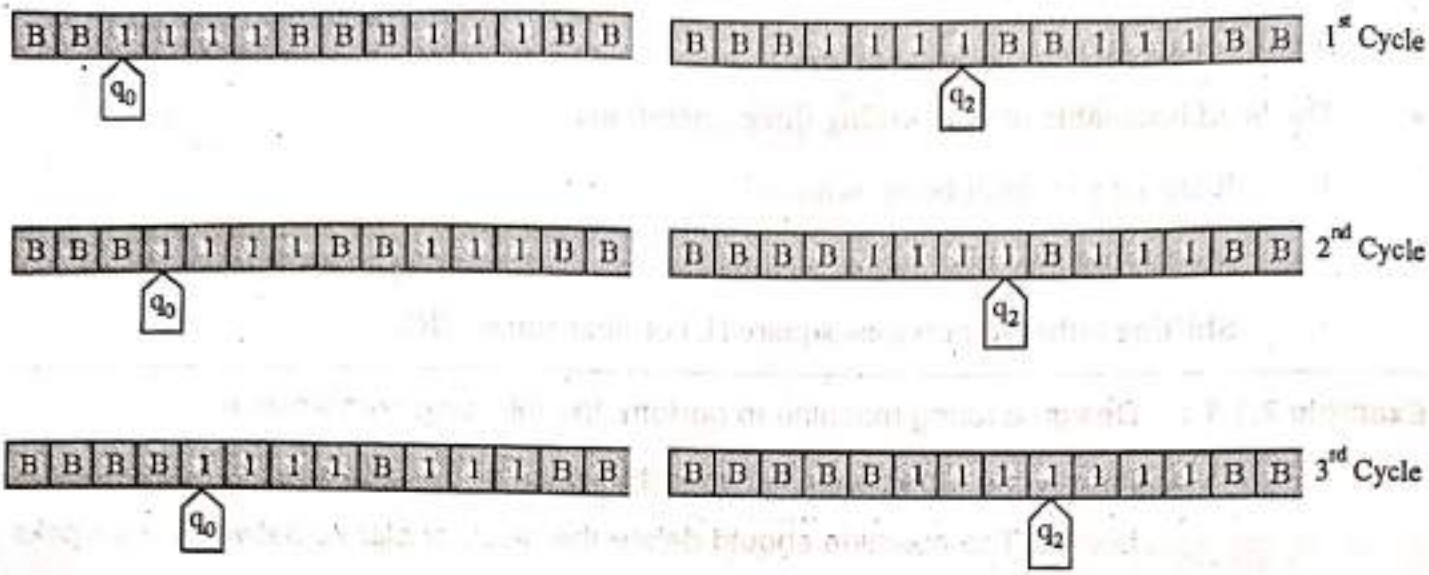


Fig. Ex. 7.1.1(c) : Cycles of computation

The transition behaviour of the turing machine is shown in Fig. Ex. 7.1.1(d).

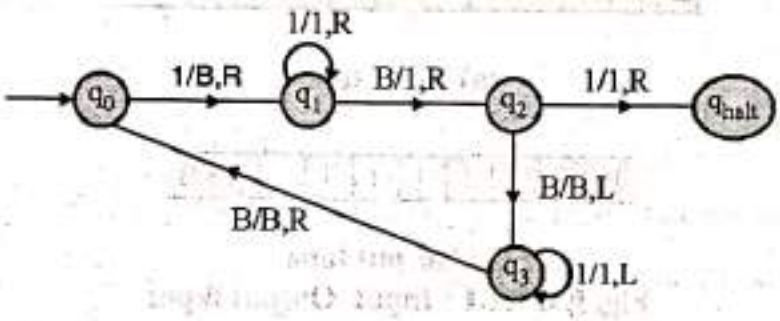
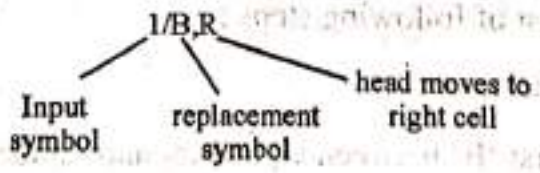


Fig. Ex. 7.1.1(d) : Transition diagram

Arc between q_0 and q_1 is marked as (1/B, R). It implies that the square being scanned contains 1 and it will be replaced by B, then the head will move to right cell.



Scanned by CamScanner

Meaning of various transitions and their effect on input tape :

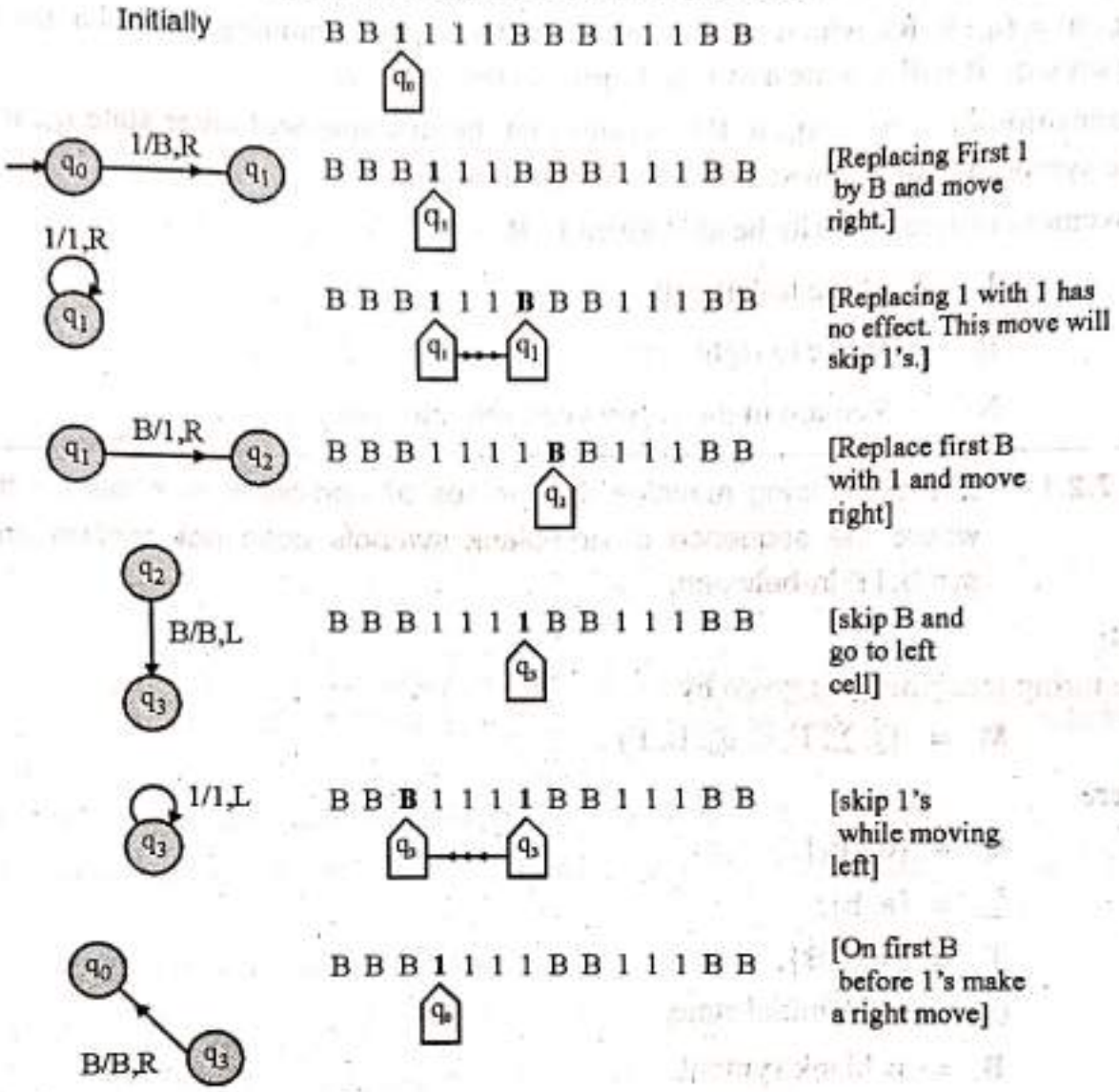


Fig. Ex. 7.1.1(e) : Transitions in detail for first cycle

7.2 The Formal Definition of Turing Machine

MU - Dec. 2008, May 2013

A Turing machine M is a 7-tuple given by :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

where

1. Q is finite set of states
2. Σ is finite set of input alphabet not containing B.
3. Γ is a finite set of tape symbols. Tape symbols include B.
4. $q_0 \in Q$ is the initial symbol.
5. $B \in \Gamma$ is a special symbol representing an empty cell.
6. $F \subseteq Q$ is the set of final states, final states are also known as halting states.
7. The transition function δ is a function from
   $Q \times \Gamma$ to $Q \times \Gamma \times (L,R,N)$

Scanned by CamScanner

A transition in turing machine is written as,

$\delta(q_0, a) = (q_1, b, R)$, which implies, when in state q_0 and scanning symbol a, the machine will enter state q_1, it will rewrite a as b and move to the right cell.

A transition $\delta(q_0, a) = (q_1, a, R)$, implies that the machine will enter state q_1, it will not change the symbol being scanned and move to the right cell.

Movement of Read / Write head is given L, R or N

L → Move to left cell

R → Move to right cell

N → Remain in the current cell (No movement)

Example 7.2.1 : Design a turing machine that erases all non-blank symbols on the tape, where the sequence of non-blank symbols does not contain any blank symbol B in-between.

Solution :

The turing machine M is given by :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

where

$Q = \{q_0, q_1\}$,

$\Sigma = \{a, b\}$,

$\Gamma = \{a, b, B\}$,

q_0 = is the initial state,

B = is blank symbol.

$F = \{q_1\}$ is a final state or 'Halt state'.

The transition function δ is defined below :

$\delta(q_0, a) = (q_0, B, R)$ [erase 'a' and move right]

$\delta(q_0, b) = (q_0, B, R)$ [erase 'b' and move right]

$\delta(q_0, B) = (q_1, B, N)$ [stop in state q_1]

The transition function can also be given in a tabular form as shown in Fig. Ex. 7.2.1(a).

	a	b	B	
$\rightarrow q_0$	(q_0,B,R)	(q_0,B,R)	(q_1,B,N)	
q_1^*	q_1	q_1	q_1	← Halting state

Fig. Ex. 7.2.1(a) : Transition table

The transition function can also be given as transition diagram as shown in Fig. Ex. 7.2.1(b).

Scanned by CamScanner

Fig. Ex. 7.2.1(b) : Transition diagram

7.2.1 A String Accepted by TM

A string α over Σ is said to be accepted by a turing machine $M = (Q,\Sigma,\Gamma,\delta,q_0,B,F)$ if when the string α is placed on the tape and head is positioned on leftmost cell containing a non-blank cell, machine is started in state q_0, then after a finite number of moves the machine is in a "Halt-state" $\in$ F.

$$(q_0,\omega) \overset{*}{\underset{M}{\vdash}} \text{Halt-state}$$

A string is rejected by a TM if the machine enters a state $q \notin F$ and scans a symbol x for which $\delta(q,x)$ is not defined.

Example 7.2.2 : Design a TM which accepts all strings of the form $a^n b^n$ for $n \geq 1$ and rejects all other strings. **MU - Dec. 2009, May 2011, May 2013**

Solution :

Let us try to understand the design process with the help of the input string a^3b^3.

There are three a's and three b's. Three a's will be matched with three b's in three cycles.

Each cycle will consist of following steps :

Step 1 : Leftmost a is changed to x.

Step 2 : Rightmost b is changed to y.

Step 3 : Head comes back to first a.

These cycles of computations are shown in Fig. Ex. 7.2.2(a).

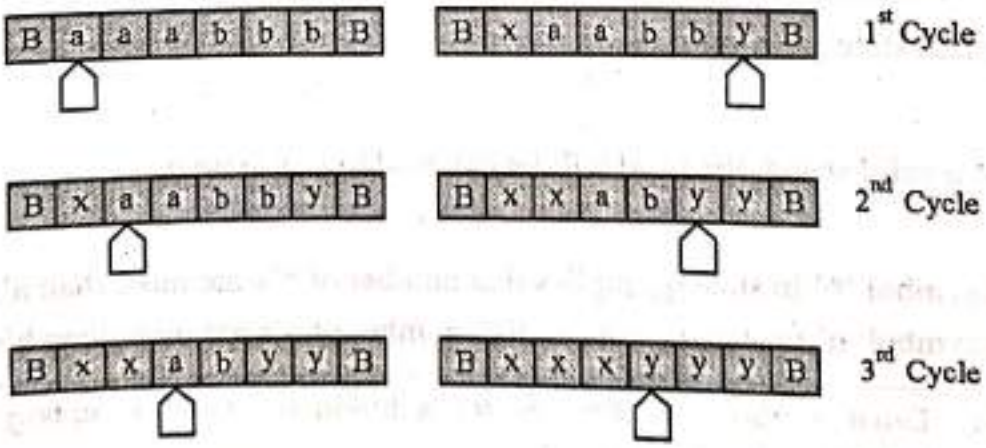


Fig. Ex. 7.2.2(a) : Cycles of computation for a^3b^3

Scanned by CamScanner

The turing machine M is given by :

$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$

Where,

$Q = \{q_0, q_1, q_2, q_3, q_4\}$

$\Sigma = \{a, b\}$

$\Gamma = \{a, b, B\}$

q_0 = Initial state

B = Blank symbol.

$F = \{q_4\}$ is a final state or 'Halt-state'

The transition function δ is given in Fig. Ex. 7.2.2(b).

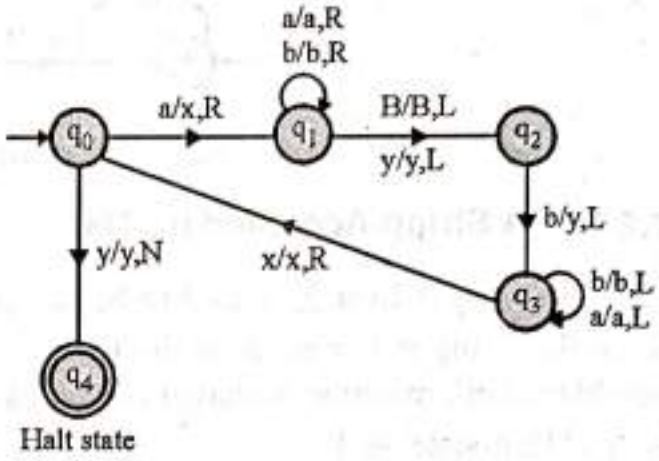


Fig. Ex. 7.2.2(b) : Transition diagram

	a	b	x	y	B	
$\rightarrow q_0$	(q_1, x, R)	–	–	(q_4, y, N)	–	
q_1	(q_1, a, R)	(q_1, b, R)	–	(q_2, y, L)	(q_2, B, L)	
q_2	–	(q_3, y, L)	–	–	–	
q_3	(q_3, a, L)	(q_3, b, L)	(q_0, x, R)	–	–	
q_4^*	q_4	q_4	q_4	q_4	q_4	← Halting state

Fig. Ex. 7.2.2(c) : Transition table

Meaning of Various States :

q_0 – Leftmost a is replaced by x.

q_1 – State q_1 locates the last b by skipping a's and b's and taking a left turn on first B or y.

q_2 – Rightmost b is replaced by y.

q_3 – State q_3 locates the first a by skipping a's and b's while moving left and taking a right turn on first x from right side and entering state q_0.

q_4 – It is halt state for a valid string.

Acceptance :

In case of a valid string, the head will be exposed to y in state q_0.

Rejections :

1. An input symbol 'b' in state q_0 implies that number of b's are more than a's.
2. An input symbol 'a' in state q_2 implies that number of a's are more than b's.

Example 7.2.3 : Draw a transition diagram for a turing machine accepting the following language. $L = \{ a^i b^j \mid i < j \}$

Solution : Solution to this example follows from Ex. 7.2.2. In Fig. 7.2.2(b), if the machine finds a string of only b's in state q_0 then number of a's is less than number of b's in $a^i b^j$.

Scanned by CamScanner

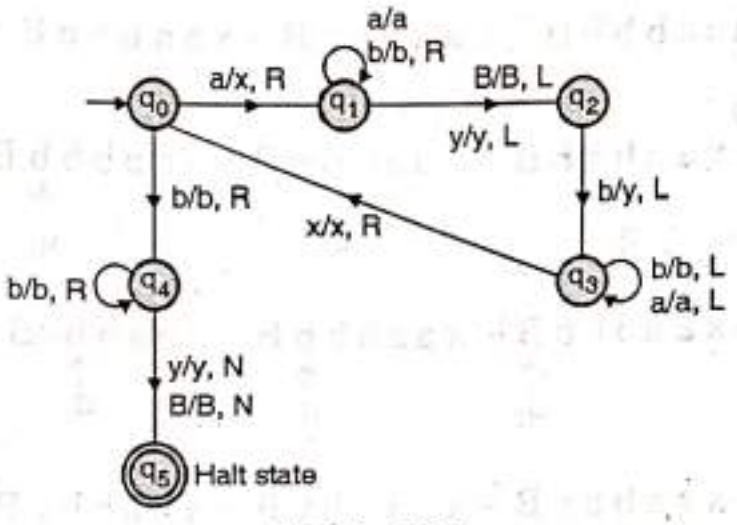


Fig. Ex. 7.2.3

7.2.2 Instantaneous Descriptions for Turing Machines

Processing of a string by a turing machine can be shown using the instantaneous description - An instantaneous description of a turing machine include :

1. The input string at any point of time
2. Position of head
3. State of the machine

The string $a_1\ a_2\ \ldots\ a_{i-1}\ \boxed{q\ a_i}\ a_{i+1}\ \ldots\ a_n$ gives the snapshot of the machine in which :

1. q is the state of the turing machine.
2. The head is scanning the symbol a_i.

The representation of machine with head scanning the symbol a_i and the machine in state q is shown in Fig. 7.2.1.

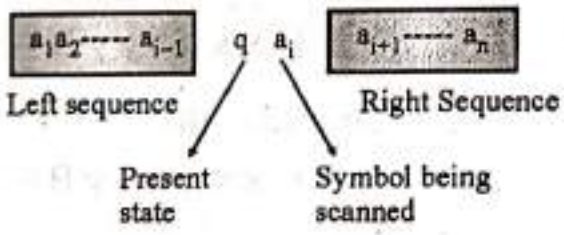


Fig. 7.2.1 : Instantaneous description of TM

Example 7.2.4 : A turing machine is represented in Fig. Ex. 7.2.4 show the processing sequence for the input string aaabbb.

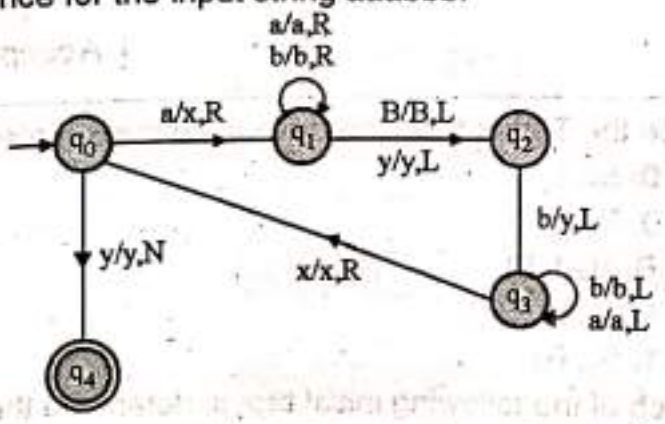


Fig. Ex. 7.2.4 : TM under consideration

Scanned by CamScanner

Solution :

$\mathbf{a}aabbbB \vdash x\mathbf{a}abbbB \vdash xa\mathbf{a}bbbB$
$q_0 \qquad q_1 \qquad q_1$

$\vdash xaa\mathbf{b}bbB \vdash xaab\mathbf{b}bB \vdash xaabb\mathbf{b}B$
$q_1 \qquad q_1 \qquad q_1$

$\vdash xaabbb\mathbf{B} \vdash xaabb\mathbf{b}B \vdash xaab\mathbf{b}yB$
$q_1 \qquad q_2 \qquad q_3$

$\vdash xaa\mathbf{b}byB \vdash xa\mathbf{a}byB \vdash x\mathbf{a}abyB$
$q_3 \qquad q_3 \qquad q_3$

$\vdash \mathbf{x}aabbyB \vdash x\mathbf{a}abyB$
$q_3 \qquad q_0$

$\vdash xx\mathbf{a}bbyB \vdash xx\mathbf{a}byB \vdash xxab\mathbf{y}B$
$q_1 \qquad q_1 \qquad q_1$

$\vdash xxabby\mathbf{B} \vdash x\,xab\mathbf{b}yB \vdash xxab\mathbf{y}yB$
$q_1 \qquad q_2 \qquad q_3$

$\vdash xx\mathbf{a}byyB \vdash x\mathbf{x}abyyB \vdash xx\mathbf{a}byyB$
$q_3 \qquad q_3 \qquad q_0$

$\vdash xxx\mathbf{b}yyB \vdash xxxb\mathbf{y}yB \vdash xxx\mathbf{b}yyB$
$q_1 \qquad q_1 \qquad q_2$

$\vdash xx\mathbf{x}yyyB \vdash xxx\mathbf{y}yyB \vdash xxx\mathbf{y}yyB$
$q_3 \qquad q_0 \qquad q_4$

[Accept]

Example 7.2.5 : Let T be the Turing Machine defined by the 5-tuples,

$(S_0, 0, 0, S_1, L)$
$(S_0, 1, 0, S_0, L)$
$(S_0, B, B, Halt, L)$
$(S_1, 0, 1, S_0, L)$
$(S_1, 1, 1, S_0, R)$

For each of the following initial tapes, determine the final tape when T halts, assuming that T begins in initial position.

1) 110B 2) 0011B 3) 0101B

Scanned by CamScanner

Solution :

1) 110B :

$$B\,1\,1\,0\,B \vdash B\,0\,1\,0\,B \quad [\text{As per } (S_0, 1, 0, S_0, L)]$$

(S_0 under the 1; S_0 under the second B)

$$\vdash B\,B\,0\,1\,0\,B$$

(Halt under the second B)

[If there is less then 2 blanks on the left then the tape head halt with a crash]

2) 0011B :

$$B\,0\,0\,1\,1\,B \vdash B\,0\,0\,1\,1\,B \quad [\text{As per } (S_0, 0, 0, S_1, L)]$$

(S_0 under the first 0; S_1 under the second 0)

The machine will fail at this point. There is no transition for input B in state S_1

3) 0101B :

$$B\,0\,1\,0\,1\,B \vdash B\,0\,0\,1\,1\,B \quad [\text{As per } (S_0, 0, 0, S_1, L)]$$

(S_0 under the first 0; S_1 under the second 0)

This machine will fail at this point. There is no transition for input B in state S_1.

Example 7.2.6 : Design a right shifting turing machine over alphabet {0,1}

Solution :

A right shifting turing machine will shift the input string, right by 1 place.

Algorithm :

Right shifting should start from the rightmost character. Each character is shifted right starting from right end and working towards left end.

Cycles of computation for an input string 1010 are shown in Fig. Ex. 7.2.6 :

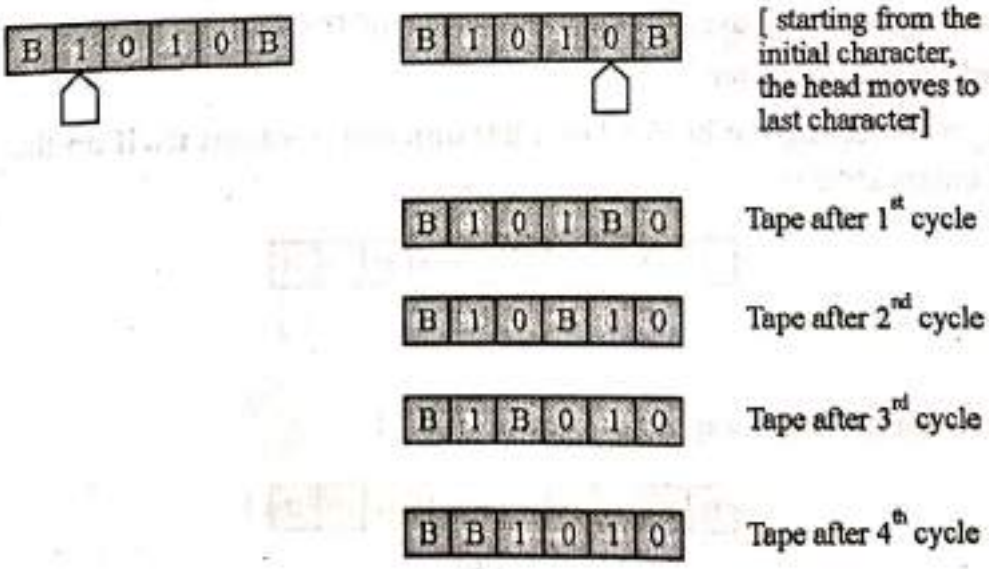


Fig. Ex. 7.2.6 : Cycles of computation

Scanned by CamScanner

Turing machine is shown in Fig. Ex. 7.2.6(a).

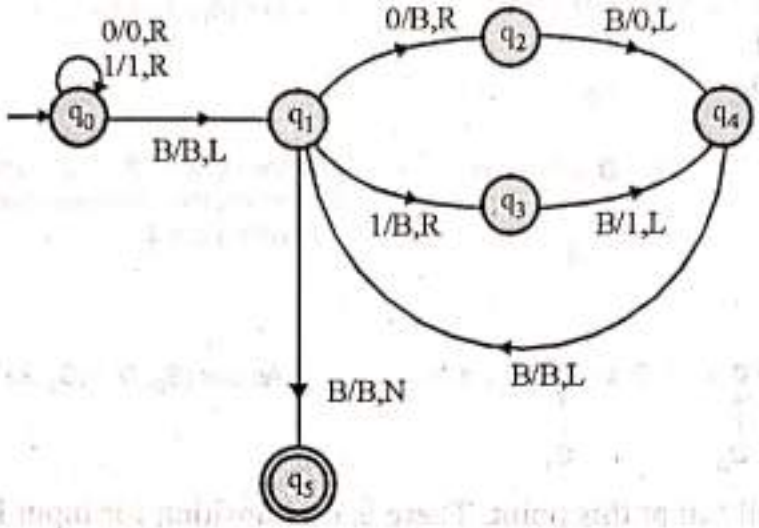


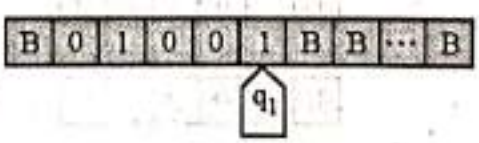
Fig. Ex. 7.2.6(a) : Transition diagram

	0	1	B
$\rightarrow q_0$	$(q_0,0,R)$	$(q_0,1,R)$	(q_1,B,L)
q_1	(q_2,B,R)	(q_3,B,R)	(q_5,B,N)
q_2	–	–	$(q_4,0,L)$
q_3	–	–	$(q_4,1,L)$
q_4	–	–	(q_1,B,L)
q_5^*	q_5	q_5	q_5

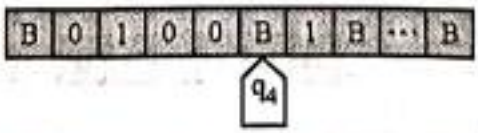
Fig. Ex. 7.2.6(b) : Transition table

Meaning of various states :

- Initial state q_0 is being used to skip the string of 0's and 1's so that the head can be positioned on last character.
- On seeing a B (blank), the head takes a left turn and positions itself on the last character. Machine enters state q_1.

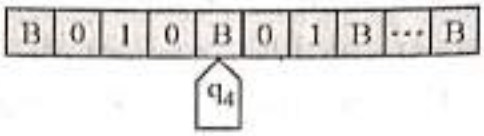


- A path through $q_1 \rightarrow q_3 \rightarrow q_4$ is for right shifting 1.

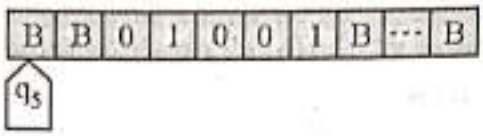


Scanned by CamScanner

- A path through $q_1 \rightarrow q_2 \rightarrow q_4$ is for right shifting 0.



- Machine halts in state q_5.



Example 7.2.7 : Design a TM to make a copy of a string over {0,1}

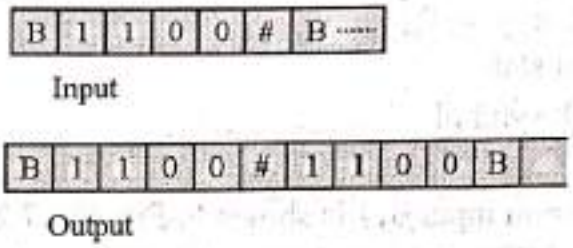


Fig. Ex. 7.2.7

Solution :

- Two copies of the strings are separated by #.
- Machine is described in Fig. Ex. 7.2.7(a).

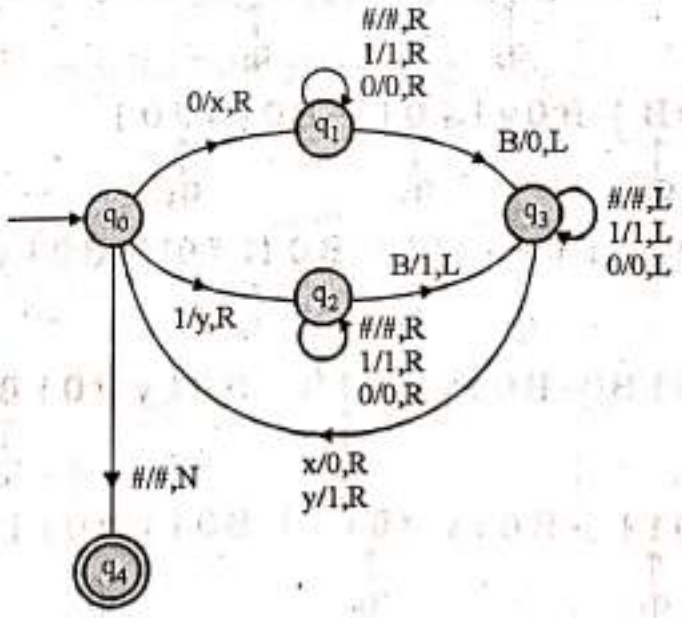


Fig. Ex. 7.2.7(a) : Transition diagram

Scanned by CamScanner

		0	1	#	B	x	y
	$\rightarrow q_0$	(q_1,x,R)	(q_2,y,R)	$(q_4,\#,N)$	–	–	–
	q_1	$(q_1,0,R)$	$(q_1,1,R)$	$(q_1,\#,R)$	$(q_3,0,L)$	–	–
	q_2	$(q_2,0,R)$	$(q_2,1,R)$	$(q_2,\# R)$	$(q_3,1,L)$	–	–
	q_3	$(q_3,0,L)$	$(q_3,1,L)$	$(q_3\ \#,L)$	–	$(q_0,0,R)$	$(q_0,1,R)$
Halting state	$\rightarrow q_4^*$	q_4	q_4	q_4	q_4	q_4	q_4

Fig. Ex. 7.2.7(b) : Transition table

- The turing machine M is given by :

$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$

Where, $Q = \{q_0, q_1, q_2, q_3, q_4\}$

$\Sigma = \{0, 1\}$

$\Gamma = \{0, 1, x, y, \#, B\}$

q_0 = Initial state

B = Blank symbol

$F = \{q_4\}$

- Working of machine for an input 011 is shown in Fig. Ex. 7.2.7(c) :

$\mathbf{B}\,0\,1\,1\,\#\,B \vdash B\,x\,\mathbf{1}\,1\,\#\,B \vdash B\,x\,1\,\mathbf{1}\,\#\,B \vdash B\,x\,1\,1\,\#\,B \vdash B\,x\,1\,1\,\#\,\mathbf{B}$

$q_0 \quad q_1 \quad q_1 \quad q_1 \quad q_1$

$\vdash B\,x\,1\,1\,\#\,0 \vdash B\,x\,1\,\mathbf{1}\,\#\,0 \vdash B\,x\,\mathbf{1}\,1\,\#\,0 \vdash B\,\mathbf{x}\,1\,1\,\#\,0$

$q_3 \quad q_3 \quad q_3 \quad q_3$

$\vdash B\,0\,\mathbf{1}\,1\,\#\,0 \vdash B\,0\,y\,\mathbf{1}\,\#\,0 \vdash B\,0\,y\,1\,\#\,0 \vdash B\,0\,y\,1\,\#\,\mathbf{0}\,B$

$q_0 \quad q_2 \quad q_2 \quad q_2$

$\vdash B\,0\,y\,1\,\#\,0\,\mathbf{B} \vdash B\,0\,y\,1\,\#\,\mathbf{0}\,1 \vdash B\,0\,y\,1\,\#\,0\,1$

$q_2 \quad q_3 \quad q_3$

$\vdash B\,0\,y\,\mathbf{1}\,\#\,0\,1 \vdash B\,0\,\mathbf{y}\,1\,\#\,0\,1 \vdash B\,0\,\mathbf{1}\,1\,\#\,0\,1 \vdash B\,0\,1\,y\,\#\,0\,1\,B$

$q_3 \quad q_3 \quad q_0 \quad q_2$

$\vdash B\,0\,1\,y\,\#\,\mathbf{0}\,1\,B \vdash B\,0\,1\,y\,\#\,0\,1\,B \vdash B\,0\,1\,y\,\#\,0\,1\,\mathbf{B}$

$q_2 \quad q_2 \quad q_2$

$\vdash B\,0\,1\,y\,\#\,0\,\mathbf{1}\,1 \vdash B\,0\,1\,y\,\#\,\mathbf{0}\,1\,1 \vdash B\,0\,1\,y\,\#\,0\,1\,1$

$q_3 \quad q_3 \quad q_3$

$\vdash B\,0\,1\,\mathbf{y}\,\#\,0\,1\,1 \vdash B\,0\,1\,1\,\#\,0\,1\,1 \vdash B\,0\,1\,1\,\#\,0\,1\,1$

$q_3 \quad q_0 \quad q_4$

Fig. Ex. 7.2.7(c)

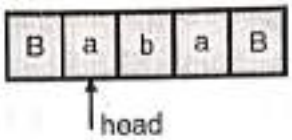
Scanned by CamScanner

Example 7.2.8 : Design TM to reverse a string $\Sigma = \{a, b\}$

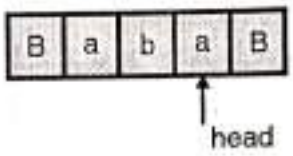
Solution :

The working of the TM is being explained with the help of the string aba.

Initial configuration of the tape

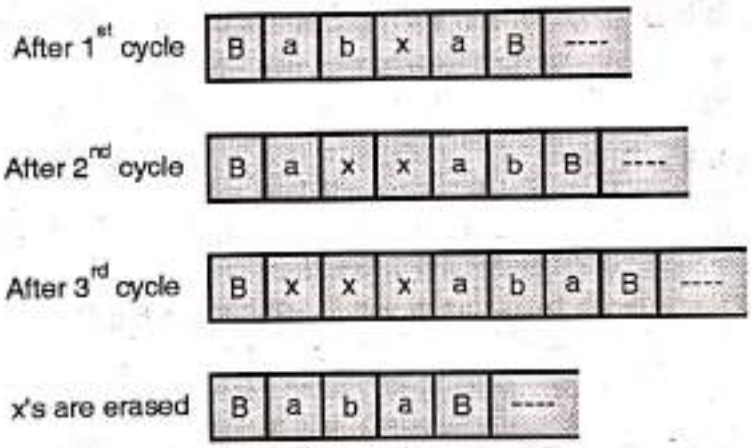


Step 1 : Head is moved to the rightmost symbol



Step 2 : We create another copy of aba in the reverse order. It will require 3 cycles as the length of aba is 3. In each cycle, the input symbol 'a' and 'b' are renamed as x.

Step 3 :



The transition diagram of the TM is given in Fig. Ex. 7.2.8.

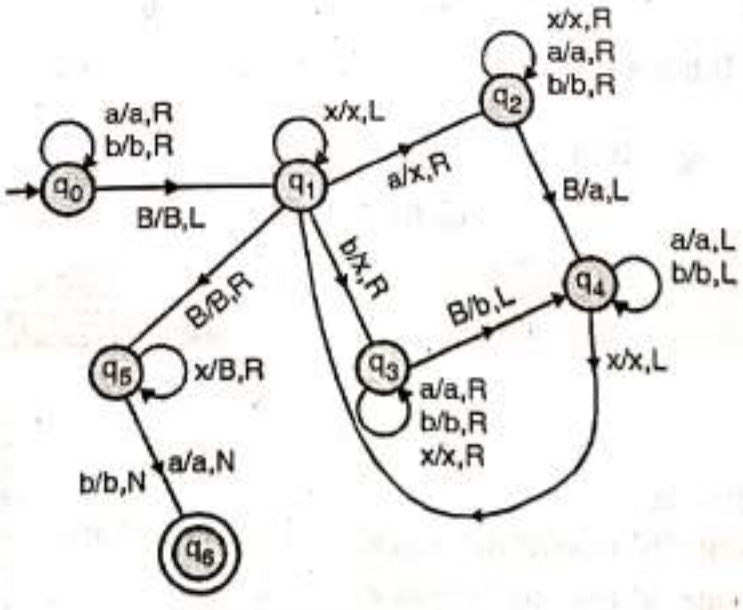


Fig. Ex. 7.2.8

Scanned by CamScanner

Simulation of machine for input abb is shown in Fig. Ex. 7.2.8(a).

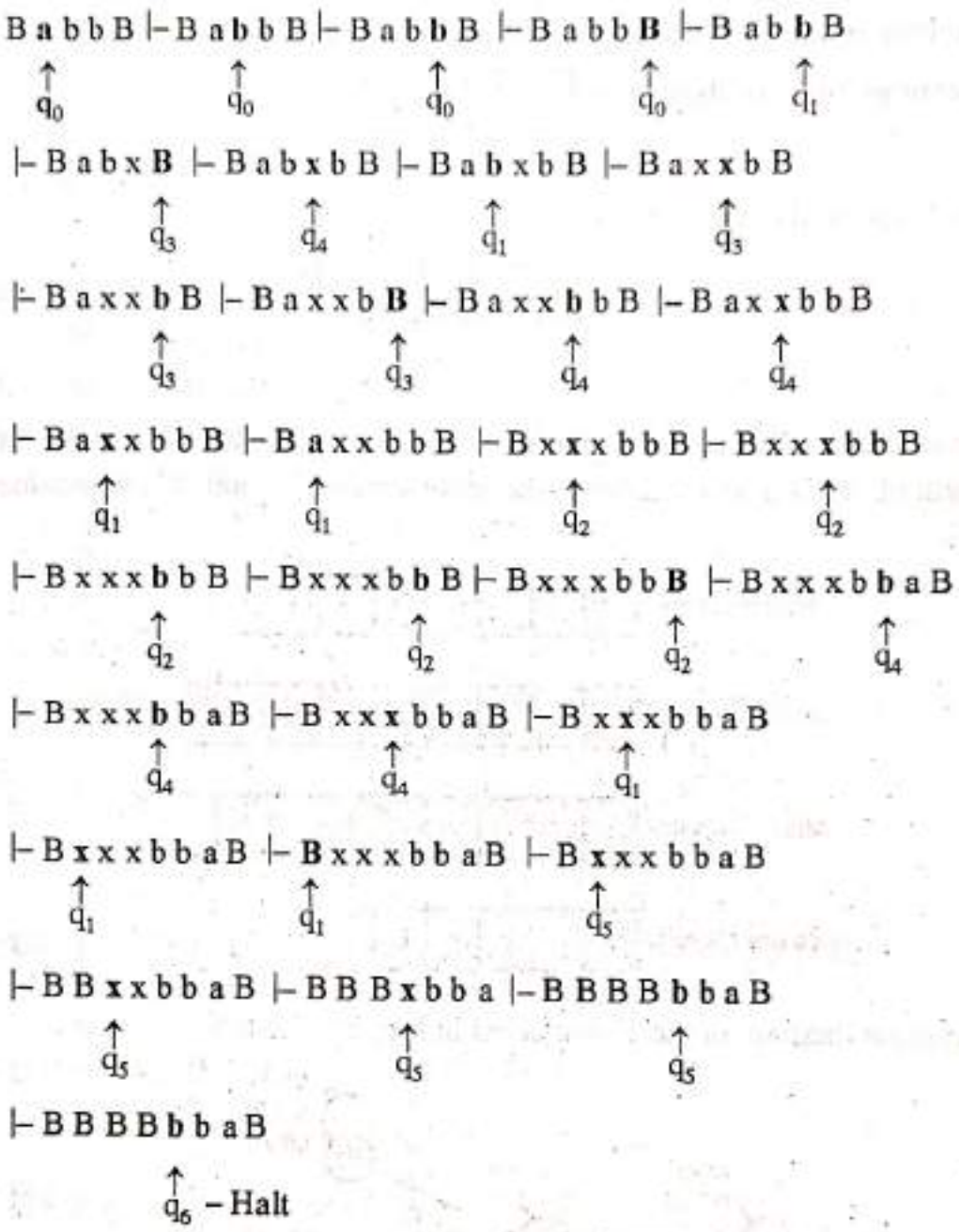


Fig. Ex. 7.2.8(a)

Example 7.2.9 : Design a turing machine to check whether a string over {a,b} contains equal number of a's and b's. **MU – Dec. 2009, May 2008, Dec. 2015**

Solution :

Algorithm :

1. Locate first a or first b.
2. If it is 'a' then locate 'b' rewrite them as x.
3. If it is 'b' then locate 'a' rewrite them as x.
4. Repeat steps from 1 to 3 till every a or b is re-written as x.

Scanned by CamScanner

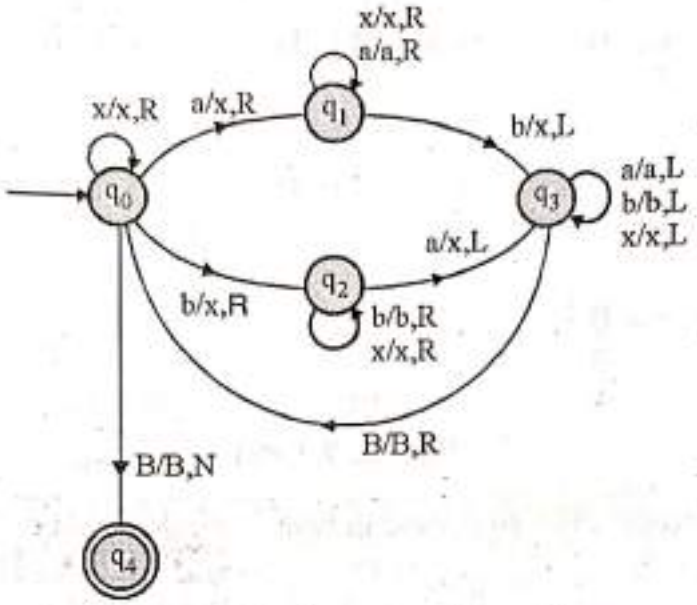


Fig. Ex. 7.2.9(a) : State transition diagram

	a	b	X	B	
$\rightarrow q_0$	(q_1,X,R)	(q_2,X,R)	(q_0,X,R)	(q_4,B,N)	
q_1	(q_1,a,R)	(q_3,X,L)	(q_1,X,R)	–	
q_2	(q_3,X,L)	(q_2,b,R)	(q_2,X,R)	–	
q_3	(q_3,a,L)	(q_3,b,L)	(q_3,X,L)	(q_0,B,R)	
q_4^*	q_4	q_4	q_4	q_4	← Halting state

Fig. Ex. 7.2.9(b) : Transition table

The turing machine M is given by :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Where,
$Q = \{q_0, q_1, q_2, q_3, q_4\}$
$\Sigma = \{a, b\}$
$\Gamma = \{a, b, X, B\}$
q_0 = Initial state
B = Blank symbol
$F = \{q_4\}$

Working of machine for an input abba is shown in Fig. Ex. 7.2.9(c) :

$$\vdash B\,\mathbf{a}\,b\,b\,a\,B \vdash B\,x\,\mathbf{b}\,b\,a\,B \vdash B\,\mathbf{x}\,x\,b\,a\,B \vdash \mathbf{B}\,x\,x\,b\,a\,B$$
(q_0, q_1, q_3, q_3)

$$\vdash B\,\mathbf{x}\,x\,b\,a\,B \vdash B\,x\,\mathbf{x}\,b\,a\,B \vdash B\,x\,x\,\mathbf{b}\,a\,B \vdash B\,x\,x\,x\,\mathbf{a}\,B$$
(q_0, q_0, q_0, q_2)

Fig. Ex. 7.2.9(c) Contd,...

Scanned by CamScanner

$$\vdash B\,x\,x\,\mathbf{x}\,x\,B \vdash B\,x\,\mathbf{x}\,x\,x\,B \vdash B\,\mathbf{x}\,x\,x\,x\,B \vdash \mathbf{B}\,x\,x\,x\,x\,B$$
$$\uparrow q_3 \qquad \uparrow q_3 \qquad \uparrow q_3 \qquad \uparrow q_3$$

$$\vdash B\,\mathbf{x}\,x\,x\,x\,B \vdash B\,x\,\mathbf{x}\,x\,x\,B \vdash B\,x\,x\,\mathbf{x}\,x\,B \vdash B\,x\,x\,x\,\mathbf{x}\,B$$
$$\uparrow q_0 \qquad \uparrow q_0 \qquad \uparrow q_0 \qquad \uparrow q_0$$

$$\vdash B\,x\,x\,x\,x\,\mathbf{B} \vdash B\,x\,x\,x\,x\,\mathbf{B}$$
$$\uparrow q_0 \qquad \uparrow q_4$$

Fig. Ex. 7.2.9(c)

Example 7.2.10 : Construct a TM for checking well formedness of parentheses.

MU - May 2012, May 2015, May 2017

Solution :

In each cycle, the left-most ')' is written as X, then the head moves left to locate the nearer '(' and it is changed to X.

The cycles of computation are shown below.

Input string is assumed to be (()())().

Cycle No.	Tape
Initial	B (()()() B
1.	B (XX()()() B
2.	B (XXXX)()() B
3.	B XXXXXX()() B
4.	B XXXXXXXX B

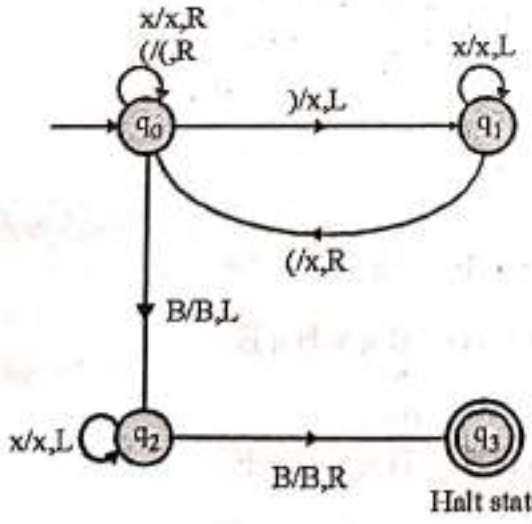


Fig. Ex. 7.2.10(a) : State transition diagram

	(	)	x	B
q_0	$(q_0,(,R)$	(q_1,x,L)	(q_0,x,R)	(q_2,B,L)
q_1	(q_0,x,R)	–	(q_1,x,L)	–
q_2	–	–	(q_2,x,L)	(q_3,B,R)
q_3^*	q_3	q_3	q_3	q_3

⇓ Halting state

Fig. Ex. 7.2.10(b) : State transition table

Scanned by CamScanner

The Turing machine M is given by :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

where, $Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{(,)\}$

$\Gamma = \{(,), x, B\}$

δ is given in Fig. Ex. 7.2.10(a) or Ex. 7.2.10(b)

q_0 = Initial state

B = Blank symbol

F = $\{q_3\}$, halting state

Making of the machine for input (()())() is given in Fig. Ex. 7.2.10(c) :

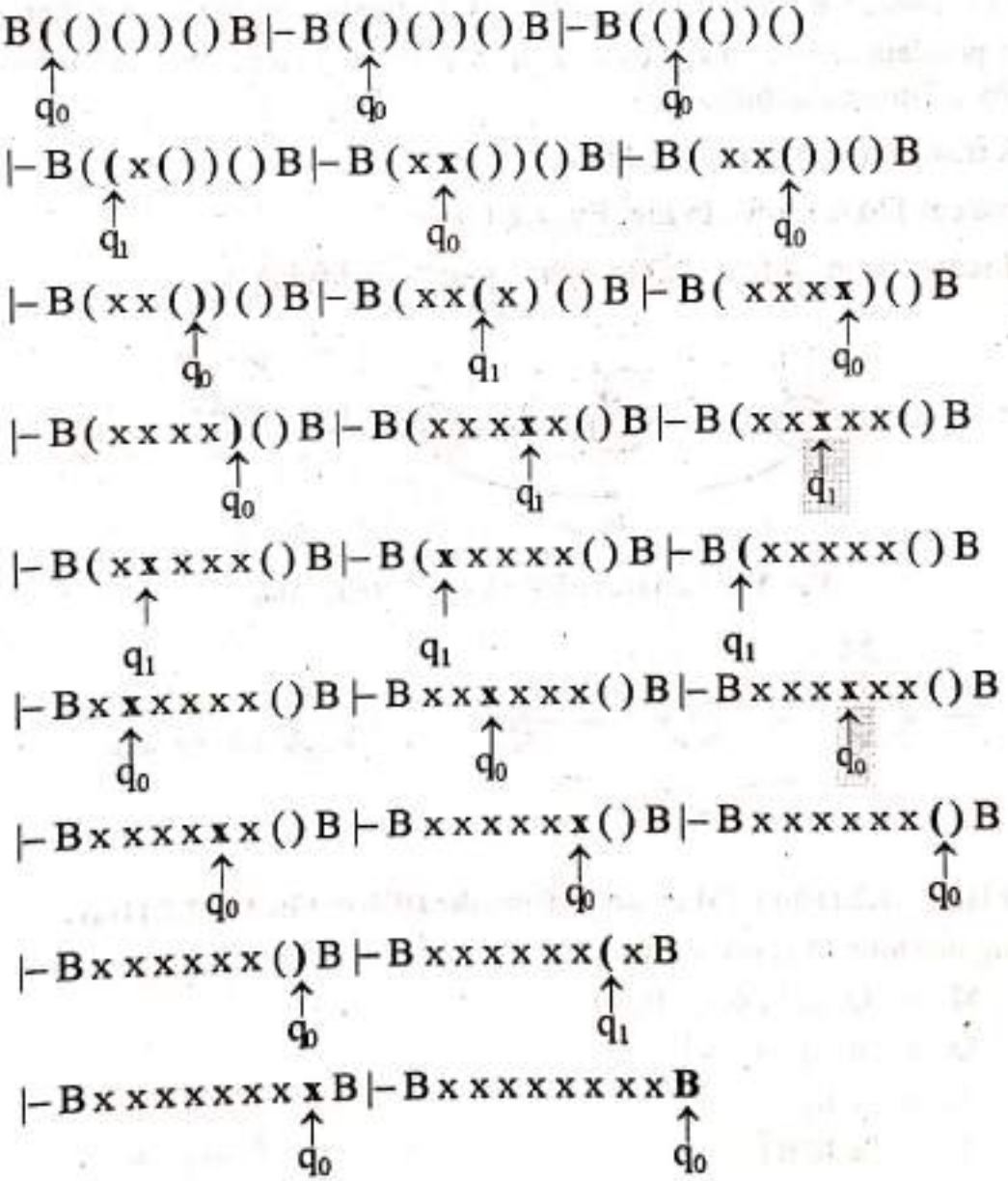


Fig. Ex. 7.2.10(c) Contd...

Scanned by CamScanner

$$\vdash B\,x\,x\,x\,x\,x\,x\,x\,\mathbf{x}\,B \vdash B\,x\,x\,x\,x\,x\,x\,\mathbf{x}\,x\,B \vdash B\,x\,x\,x\,x\,x\,\mathbf{x}\,x\,x\,B$$
$$\uparrow q_2 \qquad \uparrow q_2 \qquad \uparrow q_2$$

$$\vdash B\,x\,x\,x\,x\,\mathbf{x}\,x\,x\,x\,B \vdash B\,x\,x\,x\,\mathbf{x}\,x\,x\,x\,x\,B \vdash B\,x\,x\,\mathbf{x}\,x\,x\,x\,x\,x\,B$$
$$\uparrow q_2 \qquad \uparrow q_2 \qquad \uparrow q_2$$

$$\vdash B\,x\,\mathbf{x}\,x\,x\,x\,x\,x\,x\,B \vdash B\,\mathbf{x}\,x\,x\,x\,x\,x\,x\,x\,B \vdash B\,\mathbf{B}\,x\,x\,x\,x\,x\,x\,x\,B$$
$$\uparrow q_2 \qquad \uparrow q_2 \qquad \uparrow q_2$$

$$\vdash B\,\mathbf{x}\,x\,x\,x\,x\,x\,x\,x\,B$$
$$\uparrow q_3$$

Fig. Ex. 7.2.10(c)

Example 7.2.11 : Design a TM that recognizes a string containing aba as a substring.

Solution : This problem can be solved by a DFA. A problem that can be solved by a DFA can also be solved by a Turing machine.

- The DFA is shown in Fig. Ex. 7.2.11(a).
- The equivalent TM is shown in Fig. Ex. 7.2.11(b).
- The head scans the input from left to right without modifying it.

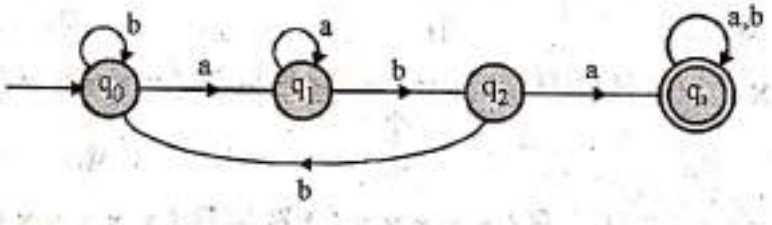


Fig. Ex. 7.2.11(a) : DFA for substring aba

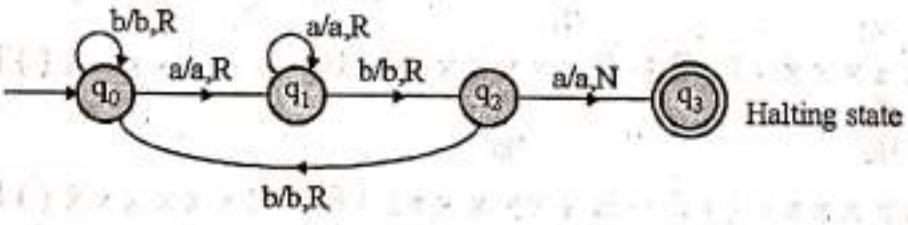


Fig. Ex. 7.2.11(b) : TM obtained from the DFA of Fig. Ex. 7.2.11(a)

The Turing machine M is given by :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Where, $Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{a, b\}$

$\Gamma = \{a, b, B\}$

Scanned by CamScanner

δ is given in Fig. Ex. 7.2.11(b)

q_0 = Initial state

B = Blank symbol,

F = $\{q_0\}$, Halting state

Example 7.2.12 : Design a TM that replaces every occurrence of abb by baa.

Solution :

- The state q_2 implies that the preceding two characters are ab.
- In input b in state q_2, completes the sequence abb.
- abb is changed to baa by a transition through

$q_2 \rightarrow q_3 \rightarrow q_4 \rightarrow q_5$.

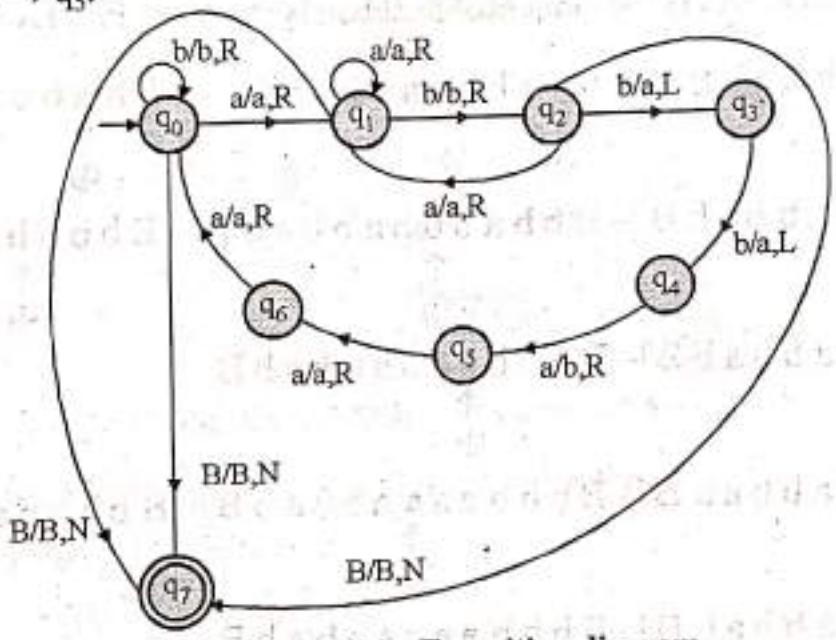


Fig. Ex. 7.2.12(a) : Transition diagram

- The head is positioned after the converted baa, by a transition through $q_5 \rightarrow q_6 \rightarrow q_0$.
- Above cycle is repeated to replace every occurrence of abb by baa.

	a	b	B	
$\rightarrow q_0$	(q_1,a,R)	(q_0,b,R)	(q_7,B,N)	
q_1	(q_1,a,R)	(q_2,b,R)	(q_7,B,N)	
q_2	(q_1,a,R)	(q_3,a,L)	(q_7,B,N)	
q_3	–	(q_4,a,L)	–	
q_4	(q_5,b,R)	–	–	
q_5	(q_6,a,R)	–	–	
q_6	(q_6,a,R)	–	–	
q_7^*	q_7	q_7	q_7	$\leftarrow$ Halting state

Fig. Ex. 7.2.12(b) : State transition table

Scanned by CamScanner

The Turing machine M is given by

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

where, $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7\}$

$\Sigma = \{a, b\}$

$\Gamma = \{a, b, B\}$

δ is given is Fig. Ex. 7.2.12(a) or (b)

q_0 = initial state

B = blank symbol

$F = \{q_7\}$, Halting state

Working of the machine for input bbabbaabbab is shown in Fig. Ex. 7.2.12(c) :

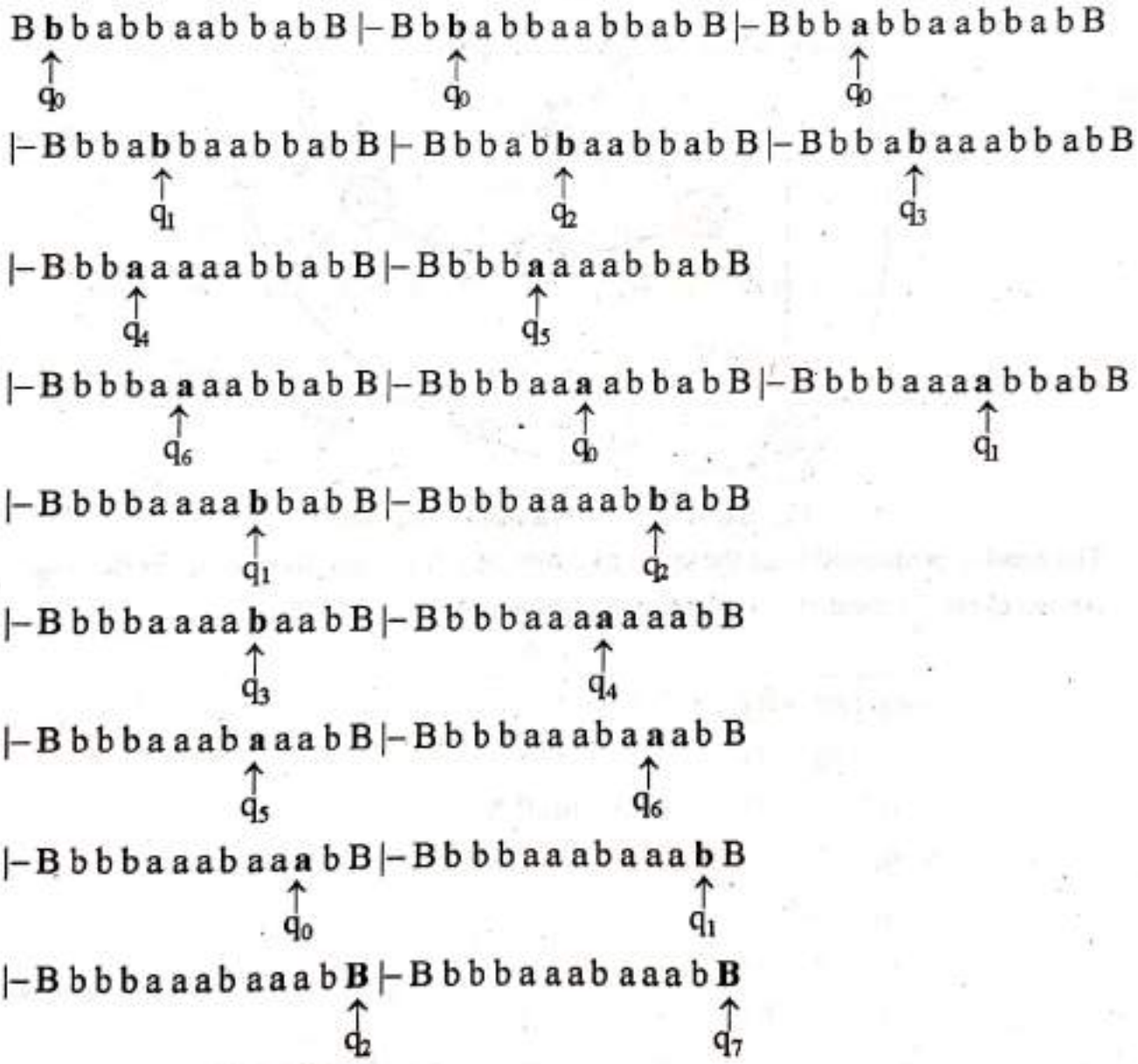


Fig. Ex. 7.2.12(c)

Scanned by CamScanner

Example 7.2.13 : Construct Turing Machine that recognizes the language :

$$L = \{x \in \{0,1\}^* \mid x \text{ ends in } 00\}$$

Solution : The given language L can have an equivalent DFA. Therefore, we will first design a DFA and then convert this DFA into a TM.

Step 1 : DFA for the given language L.

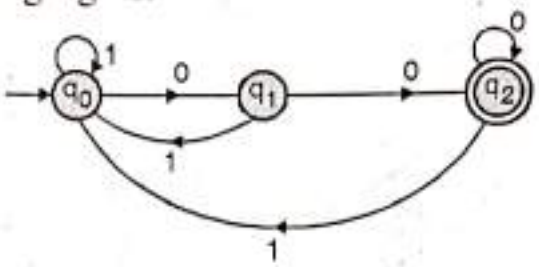


Fig. Ex. 7.2.13(a) : DFA for L

Step 2 : TM from the DFA.

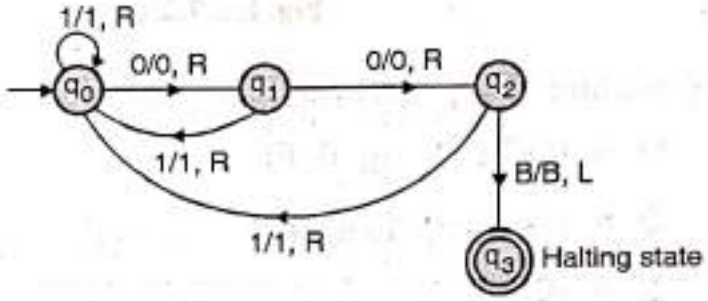


Fig. Ex. 7.2.13(b) : TM from DFA

- An input symbol B in either q_0 or q_1 will imply rejection.

The turing machine M is given by :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Where, $Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{0, 1\}$

$\Gamma = \{0, 1, B\}$

δ = Transition function is given in step 2

q_0 = initial state

B = Blank symbol for the tape

$F = \{q_3\}$, halting state

Example 7.2.14 : Design a turing machine to replace string 110 by 101 in binary input string. **MU - May 2010**

Solution :

The turing machine will look for every occurrence of the string 110.

- State q_2 is for previous two symbols as 11.
- Next symbol as 0 in state q_2, will initiate the replacement process to replace 110 by 101.

Scanned by CamScanner

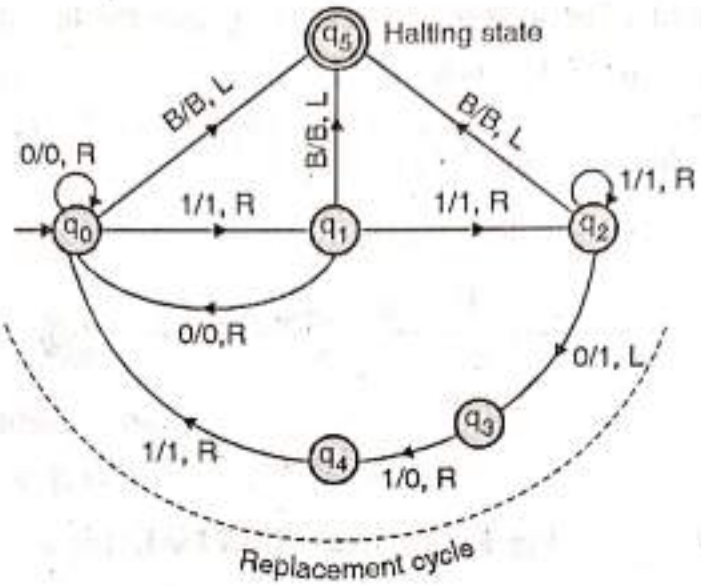


Fig. Ex. 7.2.14

The turing machine M is given by :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Where, $Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$

$\Sigma = \{0, 1\}$

$\Gamma = \{0, 1, B\}$

δ = Transition function is shown using the transition diagram

B = Blank symbol for the tape

$F = \{q_5\}$, halting state

Working of the machine for input 0101101 is shown in Fig. Ex. 7.2.14(a) :

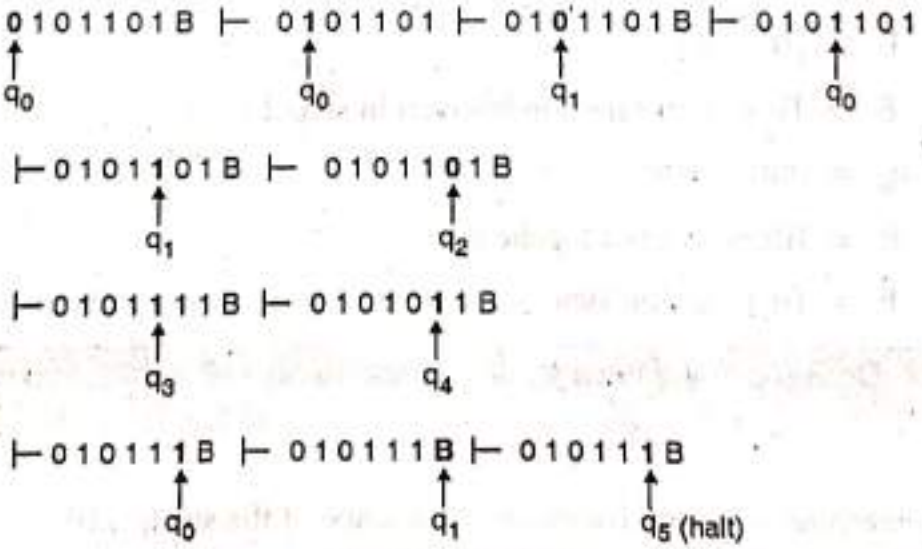


Fig. Ex. 7.2.14(a)

Scanned by CamScanner

Example 7.2.15 : Design and write out in full a turing machine that scans to the right until it finds two consecutive a's and then halts over the language of {a,b}.

Solution :

The solution can be given in two steps :

1. Design a DFA to recognize strings with substring aa.
2. Designing a TM from the DFA.

Step 1 : DFA for strings having aa in it.

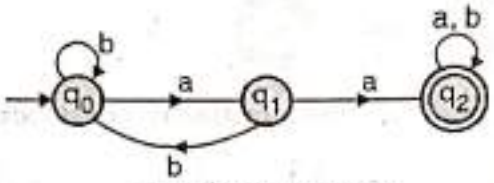


Fig. Ex. 7.2.15(a)

Step 2 : TM from the DFA

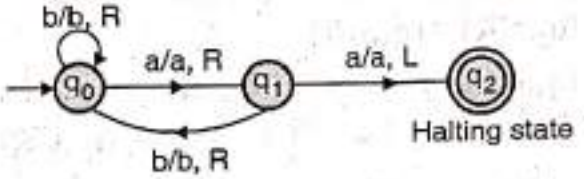


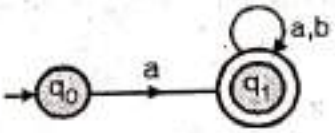
Fig. Ex. 7.2.15(b)

Example 7.2.16 : Design a Turing machine with no more than three states that accepts the

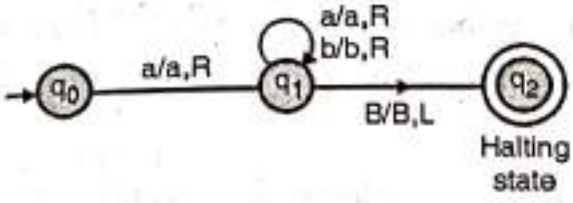
Solution :

This problem can be solved by a DFA. A problem that can be solved by a DFA can also be solved by a TM.

Step 1 : DFA for $(a(a + b)^*)$



Step 2 : Equivalent TM from the DFA.



The Turing machine $M = (\{q_0, q_1, q_2\}, \{a, b\}, \{a, b, B\}, \delta, q_0, B, \{q_2\})$

Example 7.2.17 : Design a TM which recognizes words of the form $a^n b^n c^n \mid n \geq 1$.

MU - May 2006, May 2008, Dec. 2011, Dec. 2016

Scanned by CamScanner

Solution :

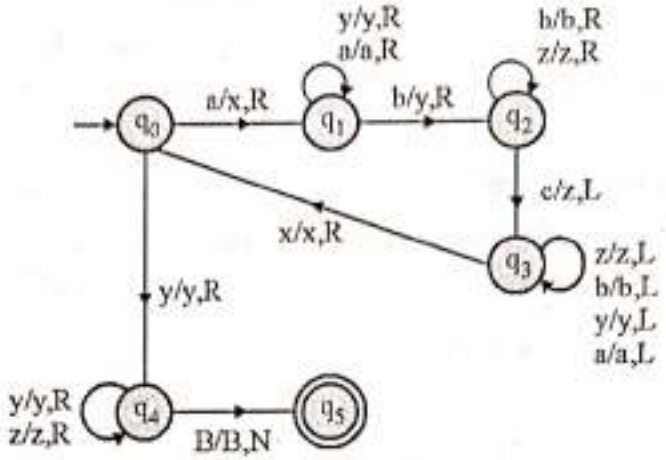


Fig. Ex. 7.2.17(a) : Transition diagram

	a	b	c	x	y	z	B
$\rightarrow q_0$	(q_1,x,R)	–	–	–	(q_4,y,R)	–	–
q_1	(q_1,a,R)	(q_2,y,R)	–	–	(q_1,y,R)	–	–
q_2	–	(q_2,b,R)	(q_3,z,R)	–	–	$(q_2,z\,R)$	–
q_3	(q_3,a,L)	(q_3,b,L)	–	(q_0,x,R)	(q_3,y,L)	(q_3,z,L)	–
q_4	–	–	–	–	(q_4,y,R)	(q_4,z,R)	(q_5,B,N)
q_5^*	q_5	q_5	q_5	q_5	q_5	q_5	q_5

Hating state

Fig. Ex. 7.2.17(b) : Transition table

The Turing machine M is given by :

$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$

Where, $Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$

$\Sigma = \{a, b, c\}$

$\Gamma = \{a, b, c, x, y, z, B\}$

δ = The transition is given Fig. Ex. 7.2.17(a, b)

q_0 = Initial state

B = Blank symbol

F = $\{q_5\}$, Halting state

Algorithm :

For a string $a^n b^n c^n$, the TM will need n cycles. In each cycle :

1. Leftmost a is written as x
2. Leftmost b is written as y
3. Leftmost c is written as z

At the end of n cycles, the tape should contain only x's, y's and z's .

Scanned by CamScanner

Working of the TM for input $a^3b^3c^3$ is shown in Fig. Ex. 7.2.17(c) :

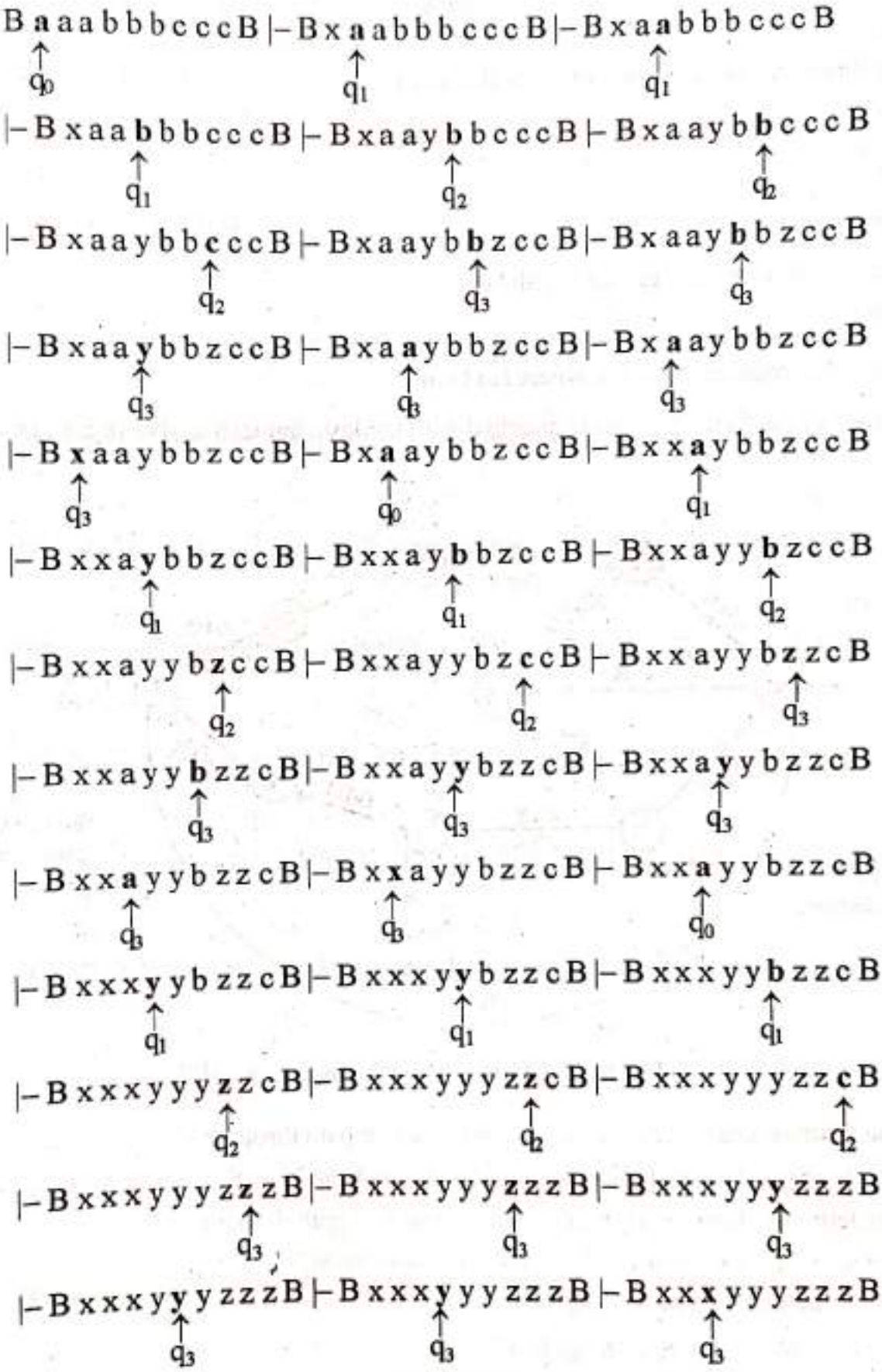


Fig. Ex. 7.2.17(c)

Scanned by CamScanner

Example 7.2.18 : Design a TM which recognizes palindromes over alphabet {a,b}

MU - May 2006, May 2009, May 2014, Dec. 2017

Solution :

A palindrome can have one of the following forms :

1. $\omega\omega^R$
2. $\omega a\omega^R$
3. $\omega b\omega^R$

Where ω is a string over {a,b} with $|\omega| \geq 0$

Algorithm :

1. Algorithm requires n cycles, where $|\omega| = n$.
2. In each cycle, first character is matched with the last character and both are erased.

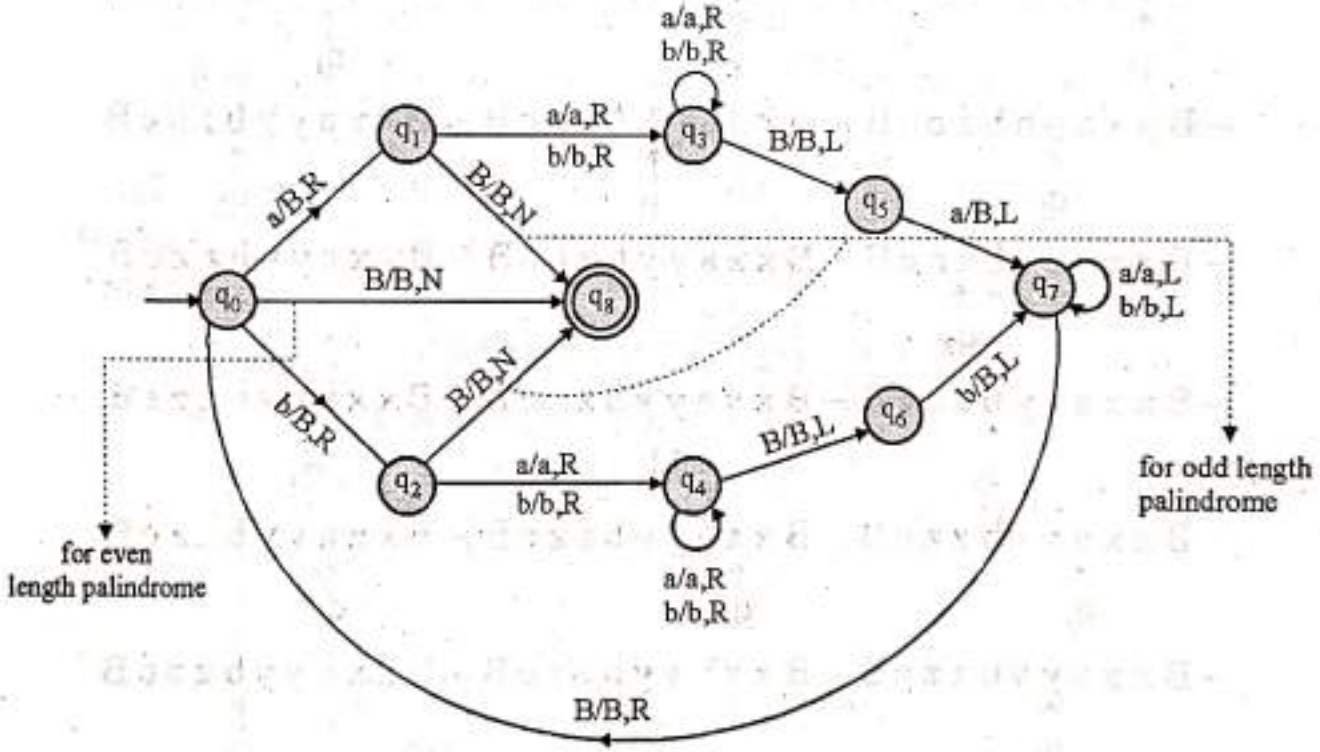


Fig. Ex. 7.2.18(a) : Transition diagram

- If the leftmost character is 'a' the machine takes a path through $q_0 \rightarrow q_1 \rightarrow q_3 \rightarrow q_5 \rightarrow q_7$, looking for last character as 'a'.
- If the leftmost character is 'b', the machine takes a path through $q_0 \rightarrow q_2 \rightarrow q_4 \rightarrow q_6 \rightarrow q_7$, looking for last character as 'b'.

The Turing machine M is given by :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Scanned by CamScanner

where, $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8\}$

$\Sigma = \{a, b\}$

$\Gamma = \{a, b, B\}$

The transition function δ is given in Fig. Ex. 7.2.18(a)

q_0 = initial state

B = blank symbol

$F = \{q_8\}$, halting state

Working of TM for input abbabba is shown in Fig. Ex. 7.2.18(a) :

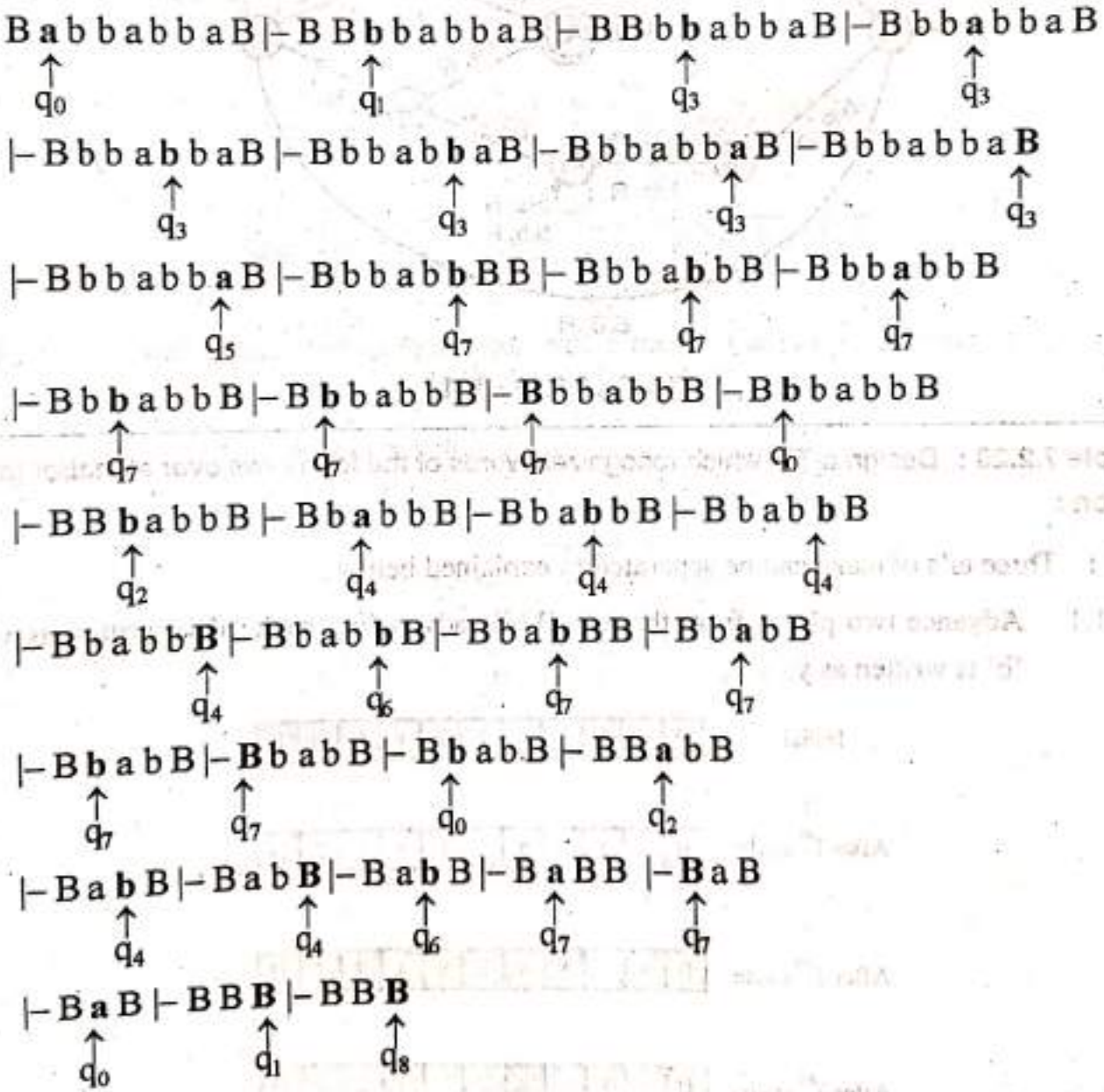


Fig. Ex. 7.2.18(a)

Scanned by CamScanner

Example 7.2.19 : Draw a transition diagram for a Turning machine accepting the following language.

L = The language of all non-palindromes over {a,b}

Solution : The solution follows from Example 7.2.18.

In Fig. Ex. 7.2.18, if the tape does not contain 'a' in the state q_5 or the tape does not contain 'b' in state q_6 then it is a non-palindrome.

The required TM is given below :

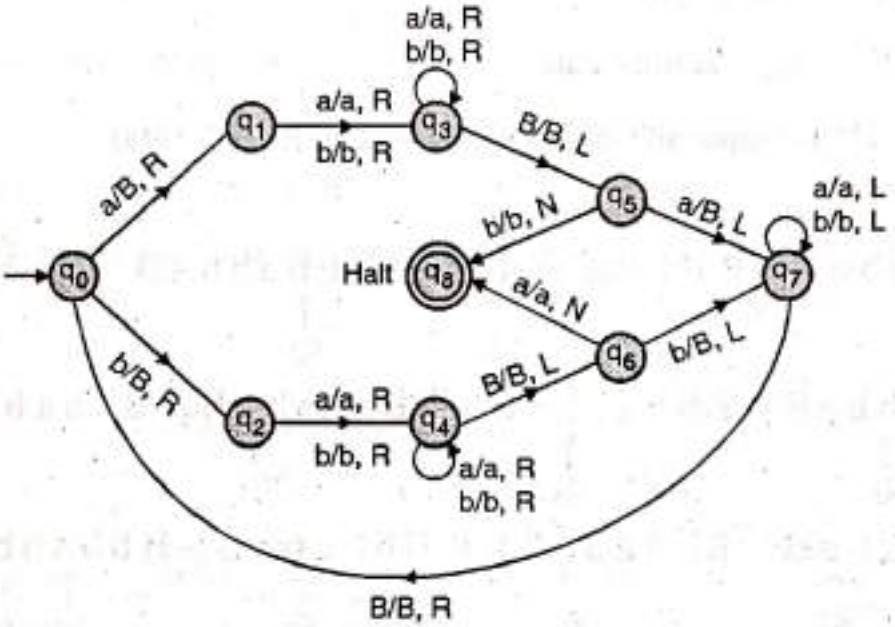


Fig. Ex. 7.2.19 : TM

Example 7.2.20 : Design a TM which recognizes words of the form ωωω over alphabet {a,b}.

Solution :

Step 1 : Three ω's of ωωω can be separated as explained below :

1.1 Advance two places from the left. While advancing, each 'a' is written as x and 'b' is written as y.

Initial	B	a	b	b	a	b	b	a	b	b	B
After 1st cycle	B	x	y	b	a	b	b	a	b	2	B
After 2nd cycle	B	x	y	y	x	b	b	a	2	2	B
After 3rd cycle	B	x	y	y	x	y	y	1	2	2	B

Scanned by CamScanner

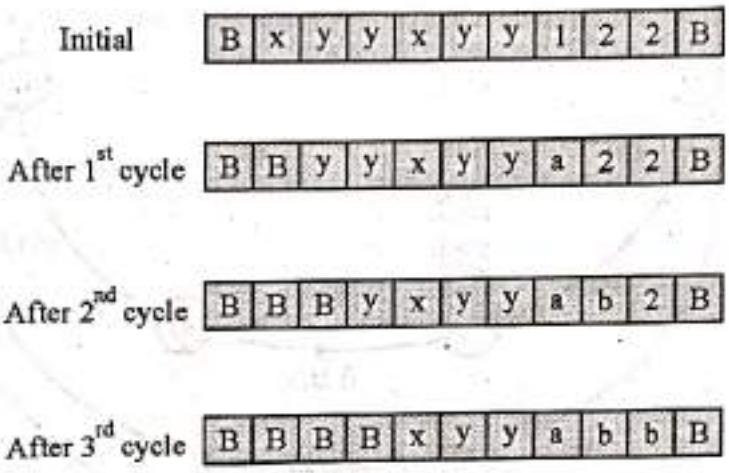
1.2 Locate the last alphabet of ωωω and change it to 1 if it is a, otherwise change it to 2.

1.3 Step number 1.1 over 1.2 is performed several times.

Situation of tape for ω = abb is shown below.

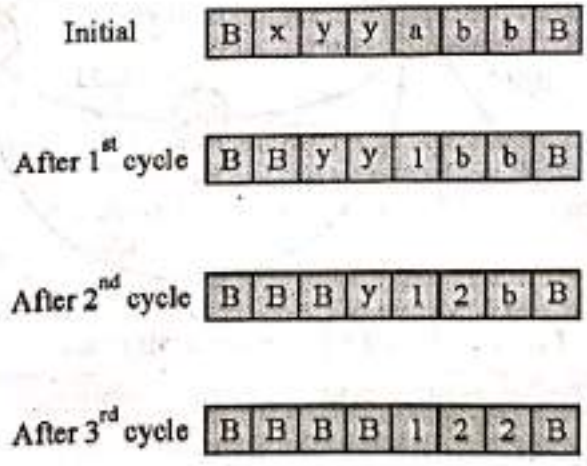
Step 2 : Fist ω of ωωω is matched with last ω. While matching, the first ω is erased and the last ω is converted to a string of {a,b}.

Situation of the tape is shown below :



Step 3 : After step 2, the tape contains ωω, where the first ω is a string over {x,y} . In this step first ω is matched with 2nd ω.

Situation of the tape is shown below :



Scanned by CamScanner

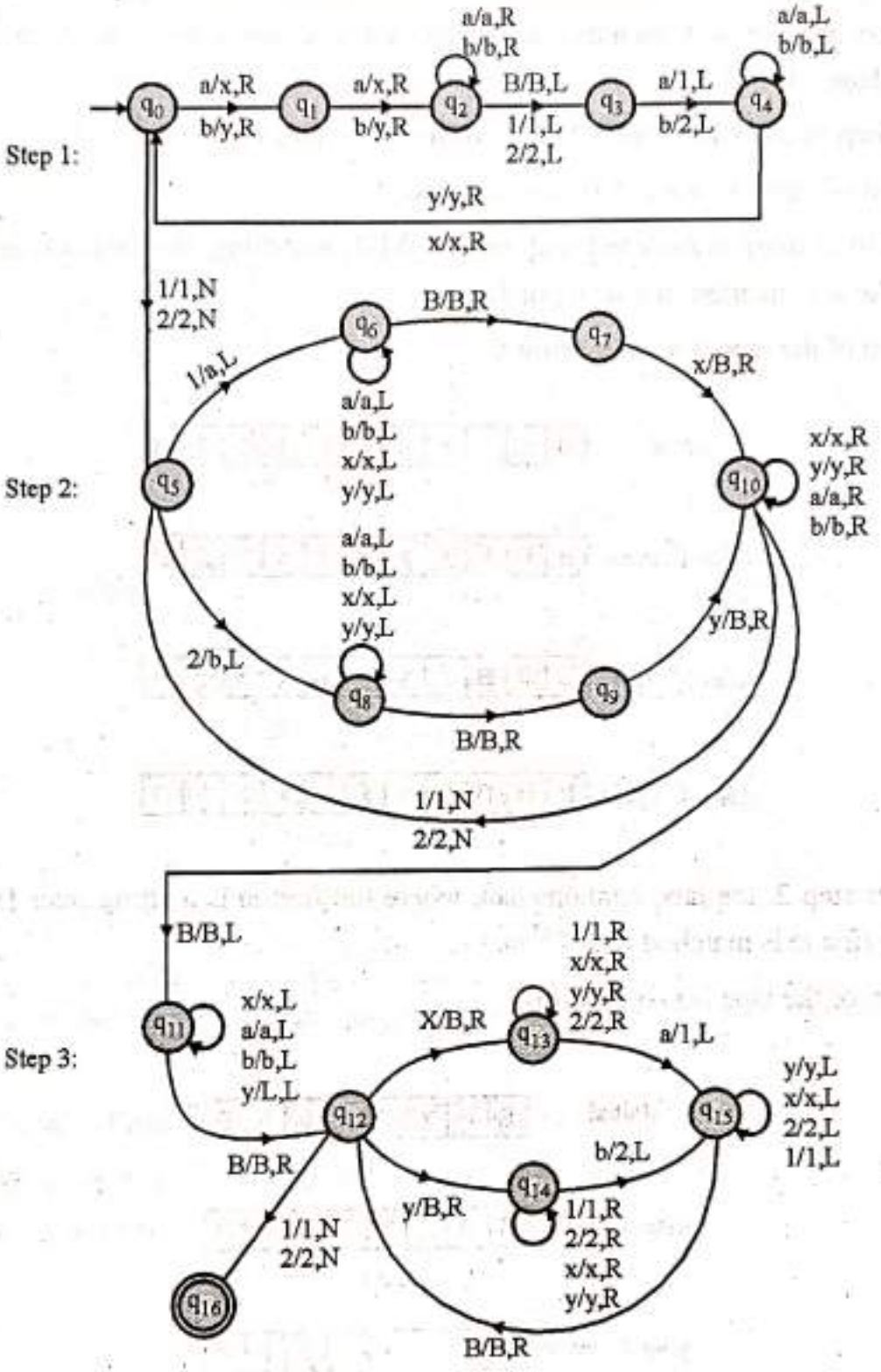
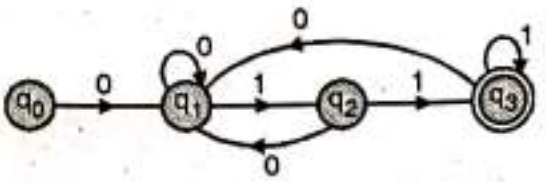


Fig. Ex. 7.2.20 : Transition diagram

Example 7.2.21 : Design a Turing machine to accept the language given by a regular expression 0(0+1)*11. **MU - Dec. 2010**

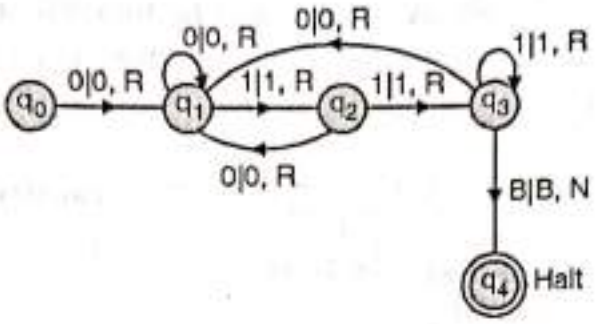
Solution :

Step 1 : First, we can design an FA to recognize a string staring with 0 and ending in 11.



Scanned by CamScanner

Step 2 : From FA to TM



7.3 Turing Machines as Computer of Functions

MU - May 2012

A turing machine can be used for computation. It can perform several operations including :

1. Addition
2. Subtraction
3. Multiplication
4. Division

Computation of a function f can be represented as

$$f : \Sigma_1^* \rightarrow \Sigma_2^*$$

A function f, $f : \Sigma_1^* \rightarrow \Sigma_2^*$ is said to be turing computable if there is a turing machine $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ such that :

$$(q_0 B\omega B) \overset{*}{\underset{M}{\vdash}} (q_f B\mu B).$$

Where $\omega \in \Sigma_1^*$ and $\mu \in \Sigma_2^*$ satisfying

$$f(\omega) = \mu \quad \text{and} \quad q_f \in F$$

Representation of a number :

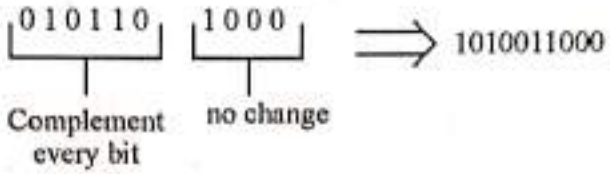
The unary number system is often used while computing a function using a Turing machine. The unary system uses only one symbol. This symbol is represented as 0. Representation of some decimal numbers in unary system is given below :

Decimal Number	Unary Number
0	B [Represented by blank string]
1	0
2	00
5	00000
n	0 .... n times .... 0, or 0^n

Scanned by CamScanner

Example 7.3.1 : Design a TM to find 2's complement of a binary machine.

Solution : 2's complement of a binary number can be found by not changing bits from right end till the first '1' and then complementing remaining bits. For example, the 2's complement of a binary number 0101101000 is calculated as given below :



Algorithm :

1. Locate the last bit (right most)
2. Move towards left till the first 1.
3. Complement remaining bits, while moving towards left.

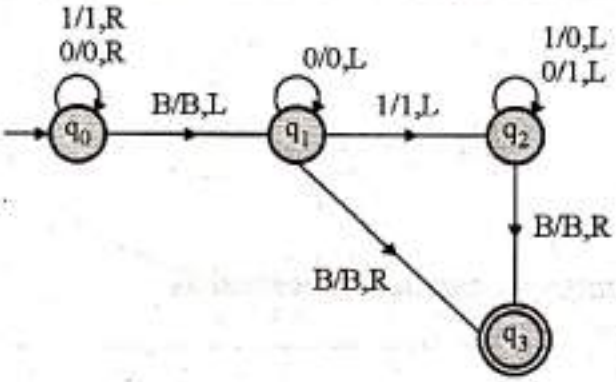


Fig. Ex. 7.3.1(a) : Transition diagram

	0	1	B	
$\rightarrow q_0$	$(q_0,0,R)$	$(q_0,1,R)$	(q_1,B,L)	
q_1	$(q_1,0,L)$	$(q_2,1,L)$	(q_3,B,R)	
q_2	$(q_2,1,L)$	$(q_2\ 0,L)$	(q_3,B,R)	
q_3^*	q_3	q_3	q_3	$\leftarrow$ Halting state

Fig. Ex. 7.3.1(b) : Transition table

The Turing machine M is given by :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Where,

$Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{0, 1\}$

$\Gamma = \{0, 1, B\}$

The transition function δ is given in Fig. Ex. 7.3.1(a) or (b),

q_0 = Initial state

B = Blank symbol

Scanned by CamScanner

$F = \{q_3\}$, Halting state

Working of TM for input 0101101000 is shown in Fig. Ex. 7.3.1(c) :

$$\mathrm{B\,\mathbf{0}\,1\,0\,1\,1\,0\,1\,0\,0\,0\,B} \vdash \mathrm{B\,0\,\mathbf{1}\,0\,1\,1\,0\,1\,0\,0\,0\,B} \vdash \mathrm{B\,0\,1\,\mathbf{0}\,1\,1\,0\,1\,0\,0\,0\,B}$$
(q_0 under the first 0, under the 1, under the 0)

$$\vdash \mathrm{B\,0\,1\,0\,\mathbf{1}\,1\,0\,1\,0\,0\,0\,B} \vdash \mathrm{B\,0\,1\,0\,1\,\mathbf{1}\,0\,1\,0\,0\,0\,B} \vdash \mathrm{B\,0\,1\,0\,1\,1\,\mathbf{0}\,1\,0\,0\,0\,B}$$
(q_0 under the 1, under the 1, under the 0)

$$\vdash \mathrm{B\,0\,1\,0\,1\,1\,0\,\mathbf{1}\,0\,0\,0\,B} \vdash \mathrm{B\,0\,1\,0\,1\,1\,0\,1\,\mathbf{0}\,0\,0\,B} \vdash \mathrm{B\,0\,1\,0\,1\,1\,0\,1\,0\,\mathbf{0}\,0\,B}$$
(q_0 under the 1, under the 0, under the 0)

$$\vdash \mathrm{B\,0\,1\,0\,1\,1\,0\,1\,0\,\mathbf{0}\,B} \vdash \mathrm{B\,0\,1\,0\,1\,1\,0\,1\,0\,0\,\mathbf{B}} \vdash \mathrm{B\,0\,1\,0\,1\,1\,0\,1\,0\,\mathbf{0}\,B}$$
(q_0 under the 0, under B, q_1 under the 0)

$$\vdash \mathrm{B\,0\,1\,0\,1\,1\,0\,1\,\mathbf{0}\,0\,0\,B} \vdash \mathrm{B\,0\,1\,0\,1\,1\,0\,\mathbf{1}\,0\,0\,0\,B}$$
(q_1 under the 0, q_1 under the 1)

$$\vdash \mathrm{B\,0\,1\,0\,1\,1\,\mathbf{0}\,1\,0\,0\,0\,B} \vdash \mathrm{B\,0\,1\,0\,1\,\mathbf{1}\,1\,1\,0\,0\,0\,B}$$
(q_2 under the 0, q_2 under the 1)

$$\vdash \mathrm{B\,0\,1\,0\,\mathbf{1}\,0\,1\,1\,0\,0\,0\,B} \vdash \mathrm{B\,0\,1\,0\,0\,0\,1\,1\,0\,0\,0\,B} \vdash \mathrm{B\,0\,\mathbf{1}\,1\,0\,0\,1\,1\,0\,0\,0\,B}$$
(q_2 under the 1, q_2 under the 0, q_2 under the 1)

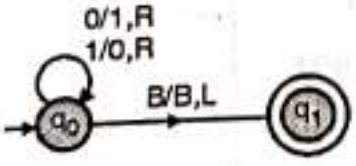
$$\vdash \mathrm{B\,\mathbf{0}\,0\,1\,0\,0\,1\,1\,0\,0\,0\,B} \vdash \mathrm{\mathbf{B}\,1\,0\,1\,0\,0\,1\,1\,0\,0\,0\,B} \vdash \mathrm{B\,\mathbf{1}\,0\,1\,0\,0\,1\,1\,0\,0\,0\,B}$$
(q_2 under the 0, q_2 under B, q_3 under the 1)

Fig. Ex. 7.3.1(c)

Example 7.3.2 : Design a TM to find the 1's complement of a given binary input.

Solution :

1's complement of a binary number can be computed by complementing every bit of the given binary number.



(a) Transition diagram

	0	1	B
q_0	$(q_0, 1, R)$	$(q_0, 0, R)$	(q_1, B, L)
q_1^*	q_1	q_1	q_1 ←Halting state

(b) Transition table

Fig. Ex. 7.3.2

Scanned by CamScanner

The Turing machine $M = (\{q_0, q_1\}, \{0, 1\}, \{0, 1, B\}, \delta, q_0, B, \{q_1\})$

The transition function δ is given through transition diagram/table.

Example 7.3.3 : Design a TM to compute proper subtraction of two unary numbers. The proper subtraction function f is defined as follows :

$$f(m, n) = \begin{cases} m - n & \text{if } m > n \\ 0 & \text{otherwise} \end{cases}$$

MU - May 2009, Dec. 2009

Solution :

The working of the TM is being explained with subtraction of 3 from 5.

In unary system, 5 is represented as 00000.

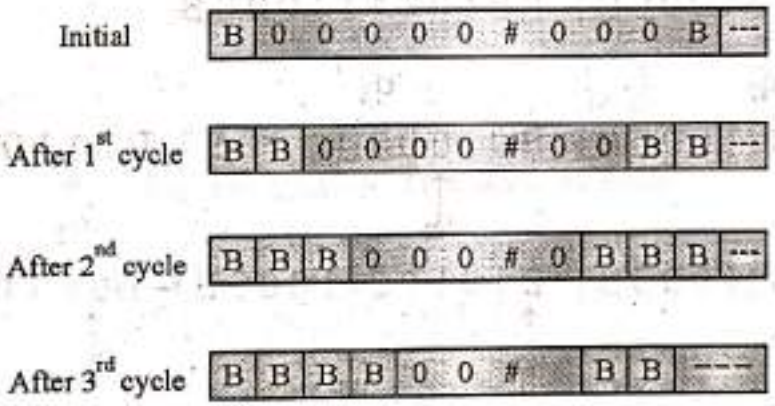
In unary system, 3 is represented as 000.

In unary system, 0 is represented by a blank tape.

Subtraction will require several cycle. In each cycle :

1. Leftmost 0 is erased
2. Rightmost 0 is erased.

Situation of tape after each cycle is shown below :



Transition diagram and transition table are given in Fig. Ex. 7.3.3(a) and (b).

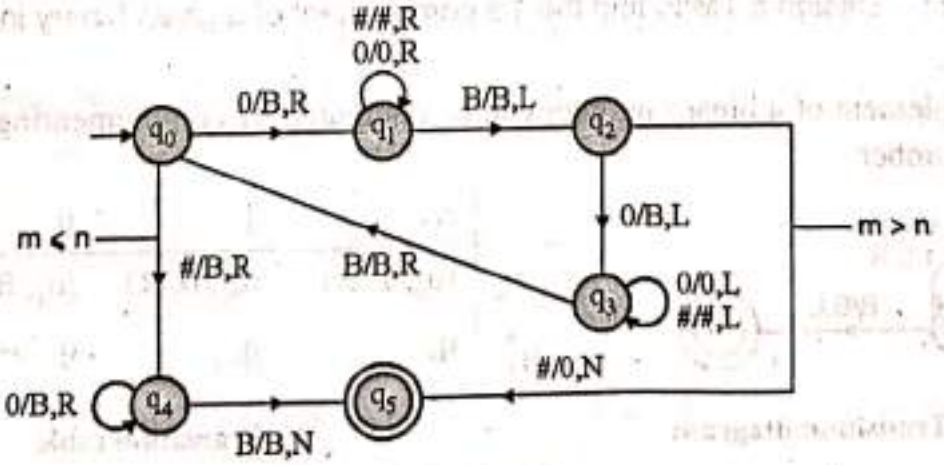


Fig. Ex. 7.3.3(a) : Transition diagram

Scanned by CamScanner

	0	#	B	
$\rightarrow q_0$	(q_1,B,R)	(q_4,B,R)	–	
q_1	$(q_1,0,R)$	$(q_1,\#,R)$	(q_2,B,L)	
q_2	(q_3,B,L)	$(q_5\ 0,N)$	–	
q_3	$(q_3,0,L)$	$(q_3,\#,L)$	(q_0,B,R)	
q_4	(q_4,B,R)	–	(q_5,B,N)	
q_5^*	q_5	q_5	q_5	← Halting state

Fig. Ex. 7.3.3(b) : Transition table

The Turing machine M is given by :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

where,

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$

$\Sigma = \{0, 1, \#\}$

$\Gamma = (0, 1, \#, B)$

The transition function δ is given in Fig. Ex. 7.3.3(a) and (b)

q_0 = initial state,

B = blank symbol

F = $\{q_5\}$, Halting state

The working of TM is being simulated for 5-3 is shown in Fig. Ex. 7.3.3(c) :

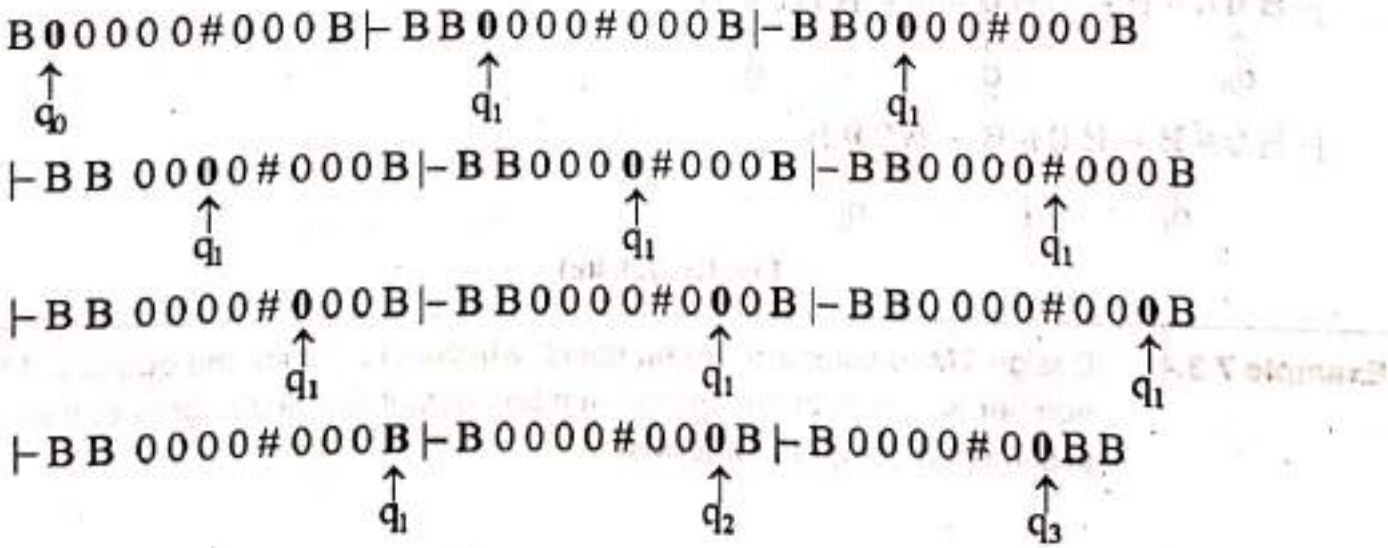


Fig. Ex. 7.3.3(c) Contd...

Scanned by CamScanner

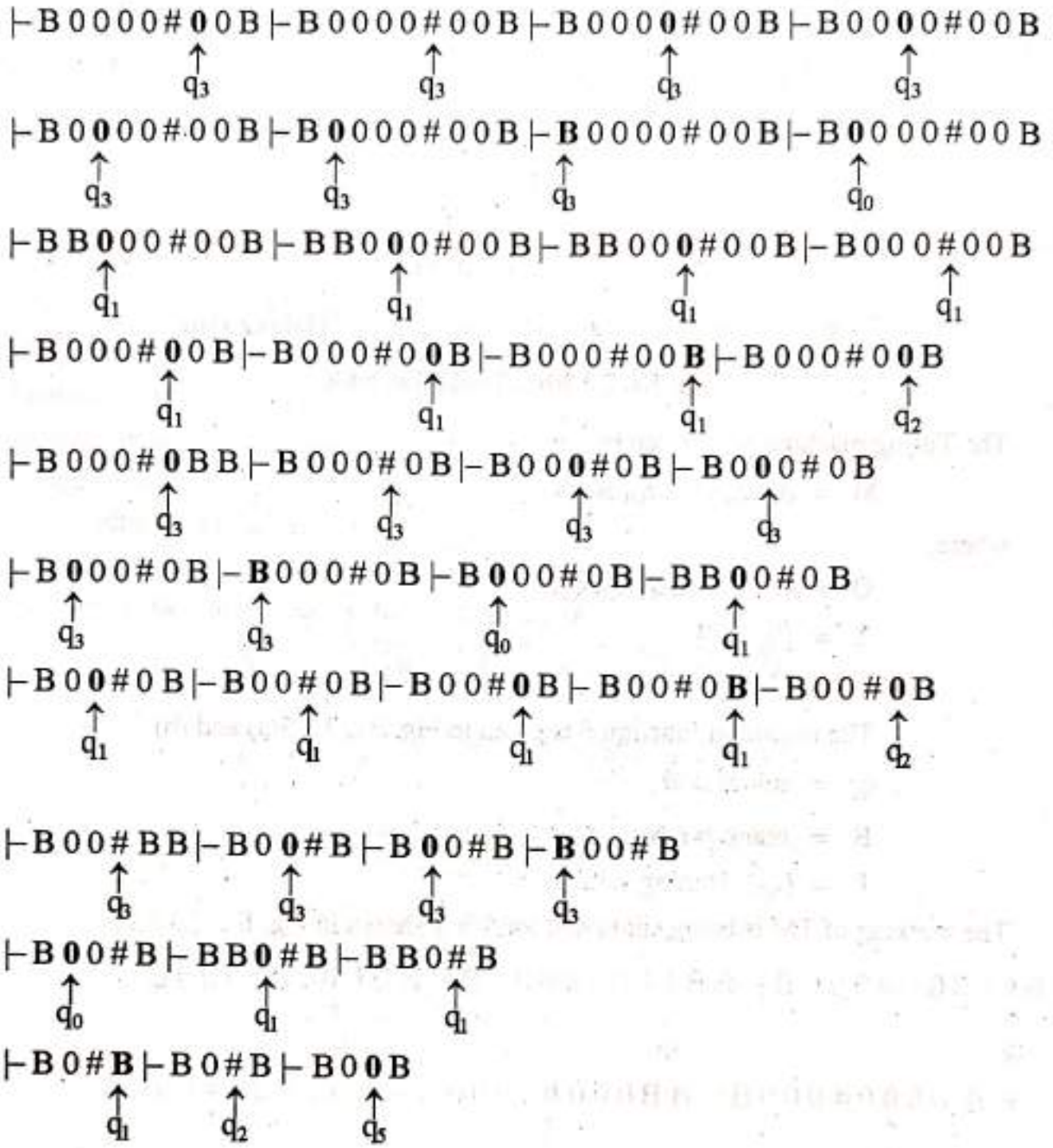


Fig. Ex. 7.3.3(c)

Example 7.3.4 : Design TM to compare two numbers, which will produce the output L if first number is less than the second number, output G if first is greater than the second number and E otherwise.

Scanned by CamScanner

Solution :

This machine is an extension of machine for proper subtraction given in Example 7.3.3.

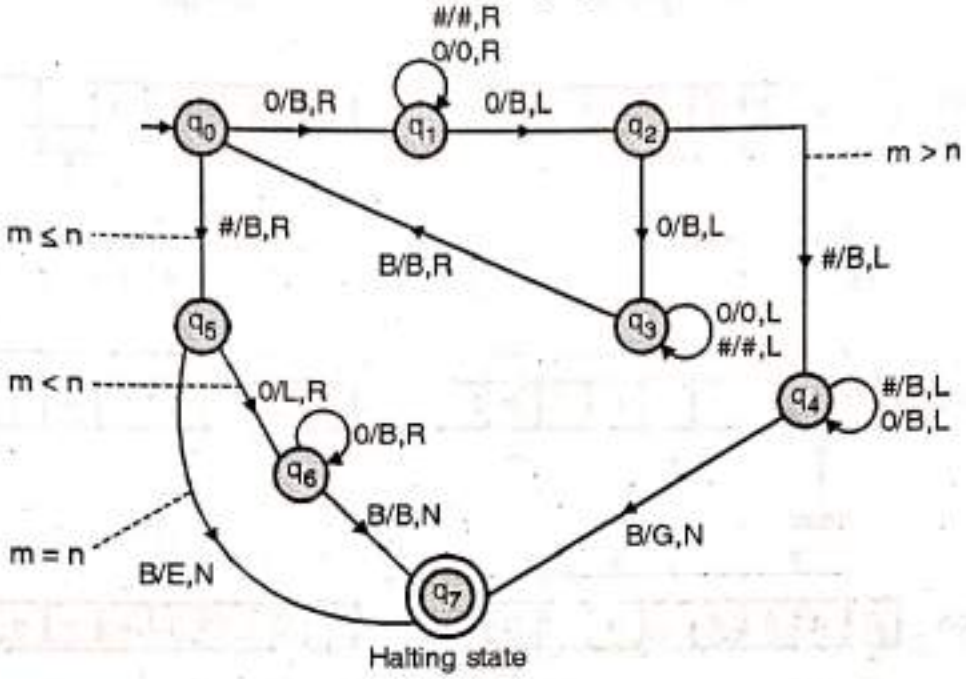


Fig. Ex. 7.3.4

- An input # in state q_2 means m > n. Everything is erased from the tape and an alphabet G is written on the tape.
- A # followed by 0 in state q_0 means m < n. L is written on the tape.
- A # followed by B(blank) in state q_0 means m = n. E is written on the tape.

The Turing machine

$$M = (\{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7\}, \{0\}, \{0, \#, B, G, L, E\}, \delta, q_0, B, \{q_7\})$$

The transition function δ is specified using the transition diagram.

Example 7.3.5 : Design a TM that computes the function

$g(x, y) = x + y$, if $x \geq y$

$= y - x$, if $y > x$

Simulate working for

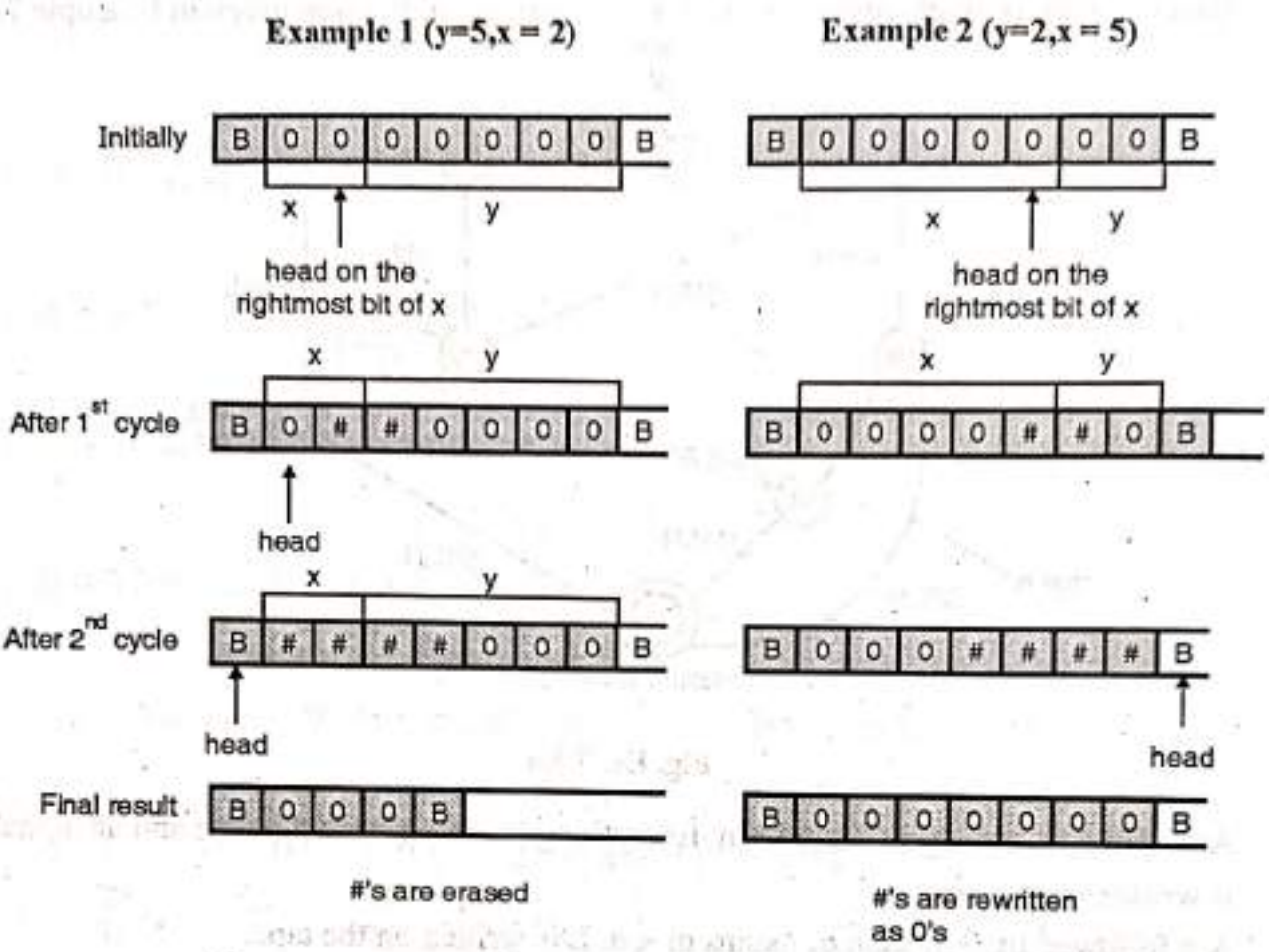
(i) $x = 2, y = 2$ (ii) $x = 0, y = 2$.

Solution :

Working of the machine is being explained with the help of two examples :

1. $y > x$ [y = 5, x = 2]
2. $y \leq x$ [y = 2, x = 5]

Scanned by CamScanner



- Initially, head is positioned on the rightmost bit of x.
- In each cycle :
  1. Nearest (from head) zero of x is written as #
  2. Nearest zero of y is written as #
- If the head hits the B(blank) on the left side before B(blank) on the right side then #'s are erased to perform y – x.
- If the head hits the B(blank) on the right side before B(blank) on the left side then #'s are re-written as 0's to perform x + y.

Scanned by CamScanner

The transition diagram of the TM is shown in Fig. Ex. 7.3.5.

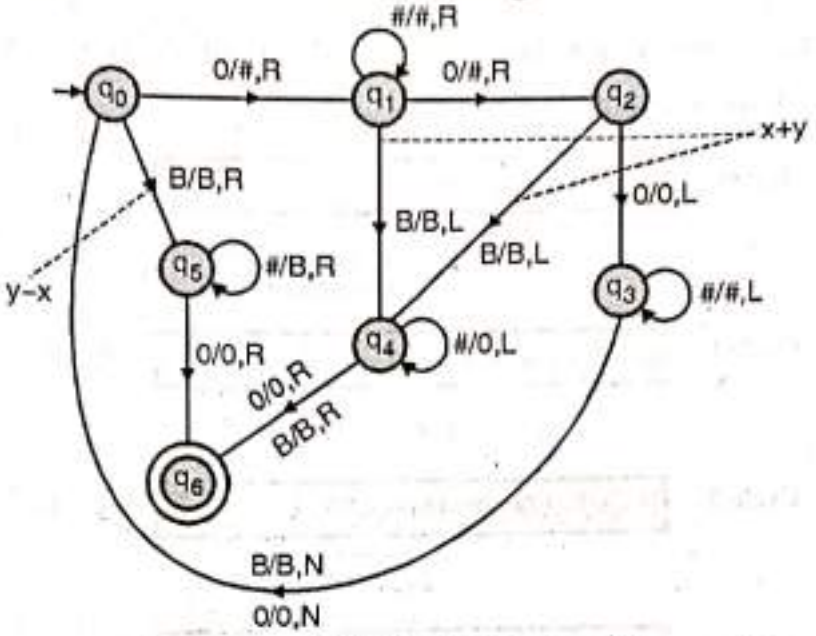


Fig. Ex. 7.3.5

The Turing machine $M = (\{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}, \{0\}, \{0, \#, B\}, \delta, q_0, B, \{q_6\})$

Where δ is specified through the transition diagram.

Simulation of machine :

(i) $x = 2, y = 2$

$\vdash$ B 0 **0** 0 0 B $\vdash$ B 0 # **0** 0 B $\vdash$ B 0 # # **0** B $\vdash$ B 0 # # 0 B
(q_0 q_1 q_2 q_3)

$\vdash$ B 0 # # 0 B $\vdash$ B **0** # # 0 B $\vdash$ B **0** # # 0 B
(q_3 q_3 q_0)

$\vdash$ B # # # 0 B $\vdash$ B # # # 0 B $\vdash$ B # # # **0** B $\vdash$ B # # # # **B**
(q_1 q_1 q_1 q_2)

$\vdash$ B # # # # B $\vdash$ B # # # 0 B $\vdash$ B # # 0 0 B $\vdash$ B # 0 0 0 B
(q_4 q_4 q_4 q_4)

$\vdash$ **B** 0 0 0 0 B $\vdash$ B **0** 0 0 0 B
(q_4 q_6 – halt)

(ii) $x = 0, y = 2$

x = 0, will be represented by blanks and y = 2 will be represented by two 0's.

B 0 0 B $\vdash$ B **0** 0 B $\vdash$ B 0 **0** B
(q_0 q_5 q_6 – halt)

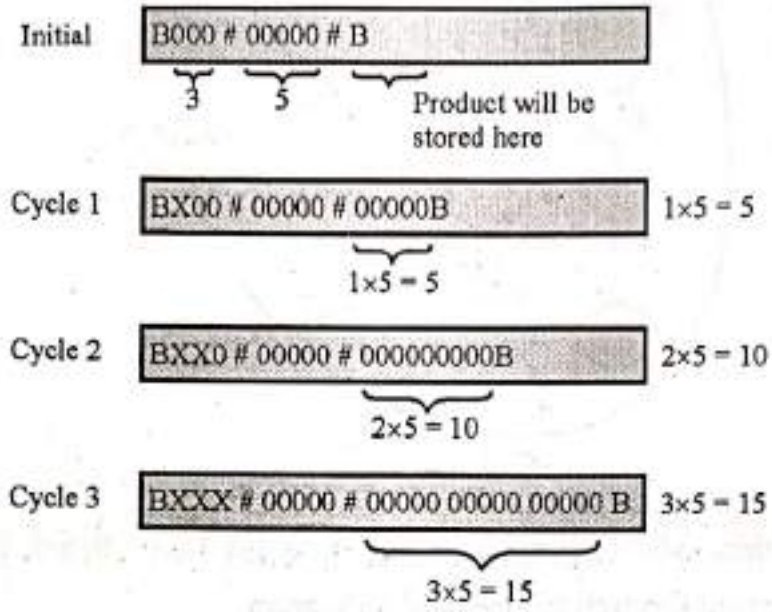
Scanned by CamScanner

Example 7.3.6 : Design a TM to compute multiplication of two unary numbers.

MU - May 2007

Solution : Multiplication algorithm is being explained with the help of an example.

3×5 will require three cycles.



- To calculate 3×5, three times, 5 zero's are appended.
  Unary representation of 3 is 000.
  Unary representation of 5 is 00000.
- 3, 5 and the result, are separated by #.
- Inside each major cycles (three cycles for 3), there will be a number of minor cycles (5 minor cycles for 5) to append 0's one at a time.

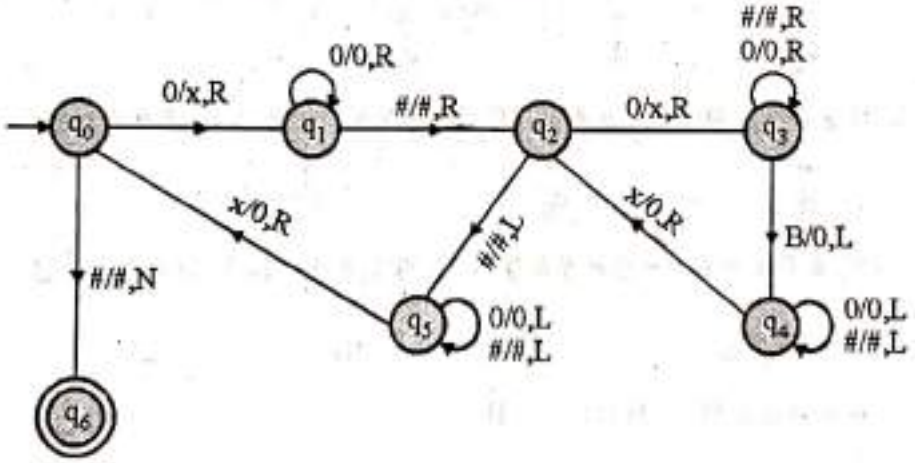


Fig. Ex. 7.3.6 : Transition diagram for TM of example 7.3.6

- Let us assume that the two numbers to be multiplied are x_1 and x_2.
  x_1 is represented by ω_1, where ω_1 is a string of 0's.
  x_2 is represented by ω_2, where ω_2 is a string of 0's.
  $x_1 * x_2$ is represented by ω_3, where ω_3 is a string 0's.
  # separates ω_1 and ω_2, ω_2 and ω_3.

Scanned by CamScanner

- In the TM shown in Fig. Ex. 7.3.6, there are two cycles.
- The cycle $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_5 \rightarrow q_0$ appends ω_2 to ω_3 for every zero in ω_1, with the help of cycle $q_2 \rightarrow q_3 \rightarrow q_4 \rightarrow q_2$

Working of TM for 2×2 is shown in Fig. Ex. 7.3.6(a) :

$\vdash$ B **0** 0 # 0 0 # B $\vdash$ B x **0** # 0 0 # B $\vdash$ B x 0 # 0 0 # B $\vdash$ B x 0 # **0** 0 # B
(q_0 q_1 q_1 q_2)

$\vdash$ B x 0 # x **0** # B $\vdash$ B x 0 # x 0 # B
(q_3 q_3)

$\vdash$ B x 0 # x 0 # **B** $\vdash$ B x 0 # x 0 # 0 B
(q_3 q_4)

$\vdash$ B x 0 # x **0** # 0 B $\vdash$ B x 0 # **x** 0 # 0 B
(q_4 q_4)

$\vdash$ B x 0 # 0 **0** # 0 B $\vdash$ B x 0 # 0 x # 0 B
(q_2 q_3)

$\vdash$ B x 0 # 0 x # **0** B $\vdash$ B x 0 # 0 x # 0 **B**
(q_3 q_3)

$\vdash$ B x 0 # 0 x # **0** 0 B $\vdash$ B x 0 # 0 x # 0 0 B
(q_4 q_4)

$\vdash$ B x 0 # 0 **x** # 0 0 B $\vdash$ B x 0 # 0 0 # 0 0 B
(q_4 q_2)

$\vdash$ B x 0 # 0 **0** # 0 0 B $\vdash$ B x 0 # **0** 0 # 0 0 B $\vdash$ B x 0 # 0 0 # 0 0 B
(q_5 q_5 q_5)

$\vdash$ B x **0** # 0 0 # 0 0 B $\vdash$ B **x** 0 # 0 0 # 0 0 B $\vdash$ B 0 **0** # 0 0 # 0 0 B
(q_5 q_5 q_0)

$\vdash$ B 0 x # 0 0 # 0 0 B $\vdash$ B 0 x # **0** 0 # 0 0 B
(q_1 q_2)

Fig. Ex. 7.3.6(a) Contd...

Scanned by CamScanner

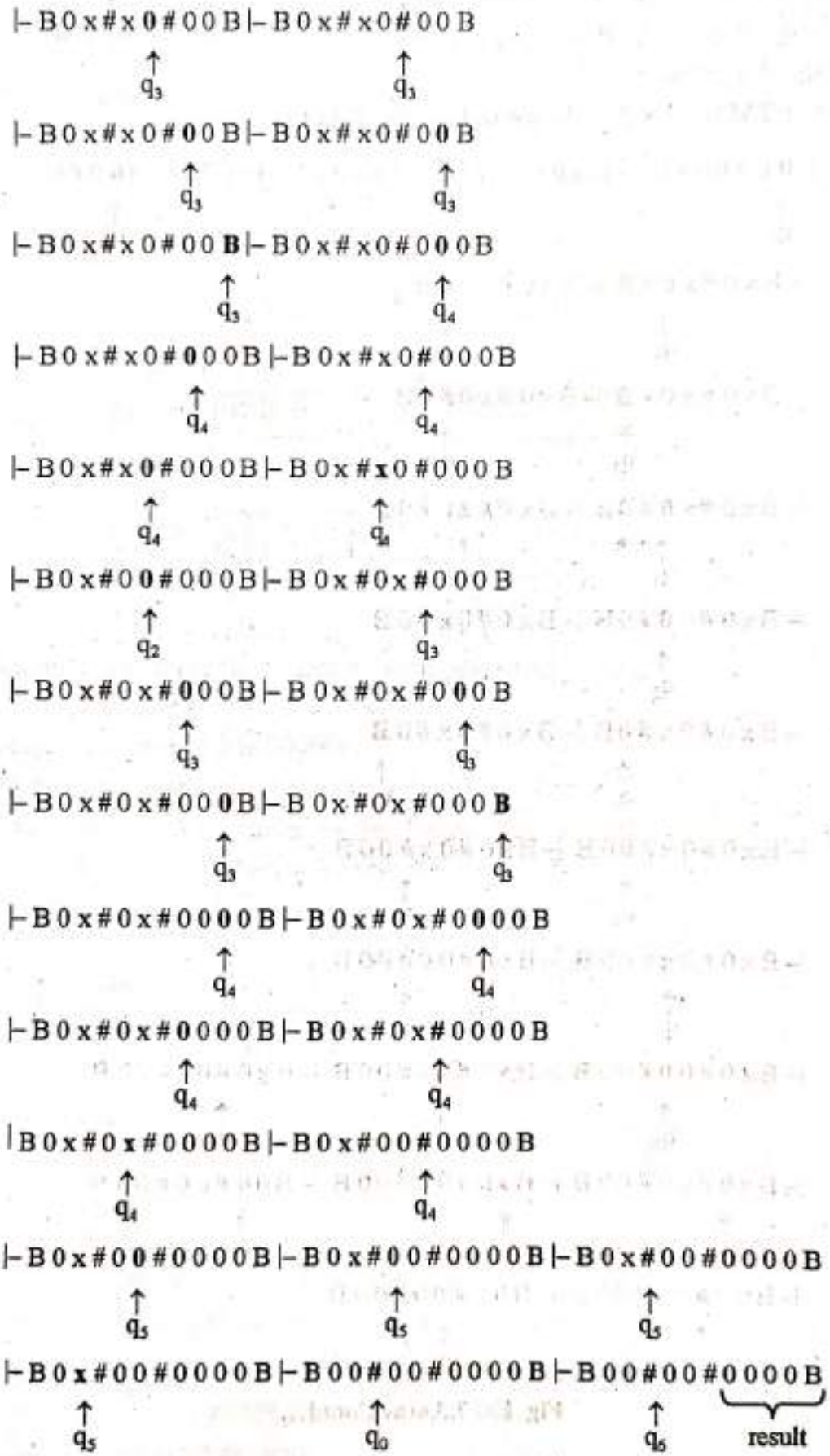


Fig. Ex. 7.3.6(a)

Scanned by CamScanner

Example 7.3.7 : Design TM for n^2 where 'n' is integer and $n \geq 0$.

Solution : Computation of n^2 is same as multiplication of n with n. Computation of n^2 is being explained with the help of an example.

Computation of 3^2 :

3 is represented as 000.

Initially the tape will contain B 0 0 0 # B ,

we make a copy of the string B 000 # 000 B .

Now, the multiplication of 3 with 3 will require three cycles as shown in Fig. Ex. 7.3.7(a).

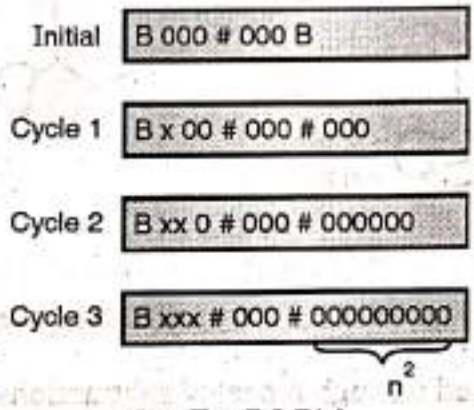


Fig. Ex. 7.3.7(a)

The transition diagram of the required TM is shown in Fig. Ex. 7.3.7(b).

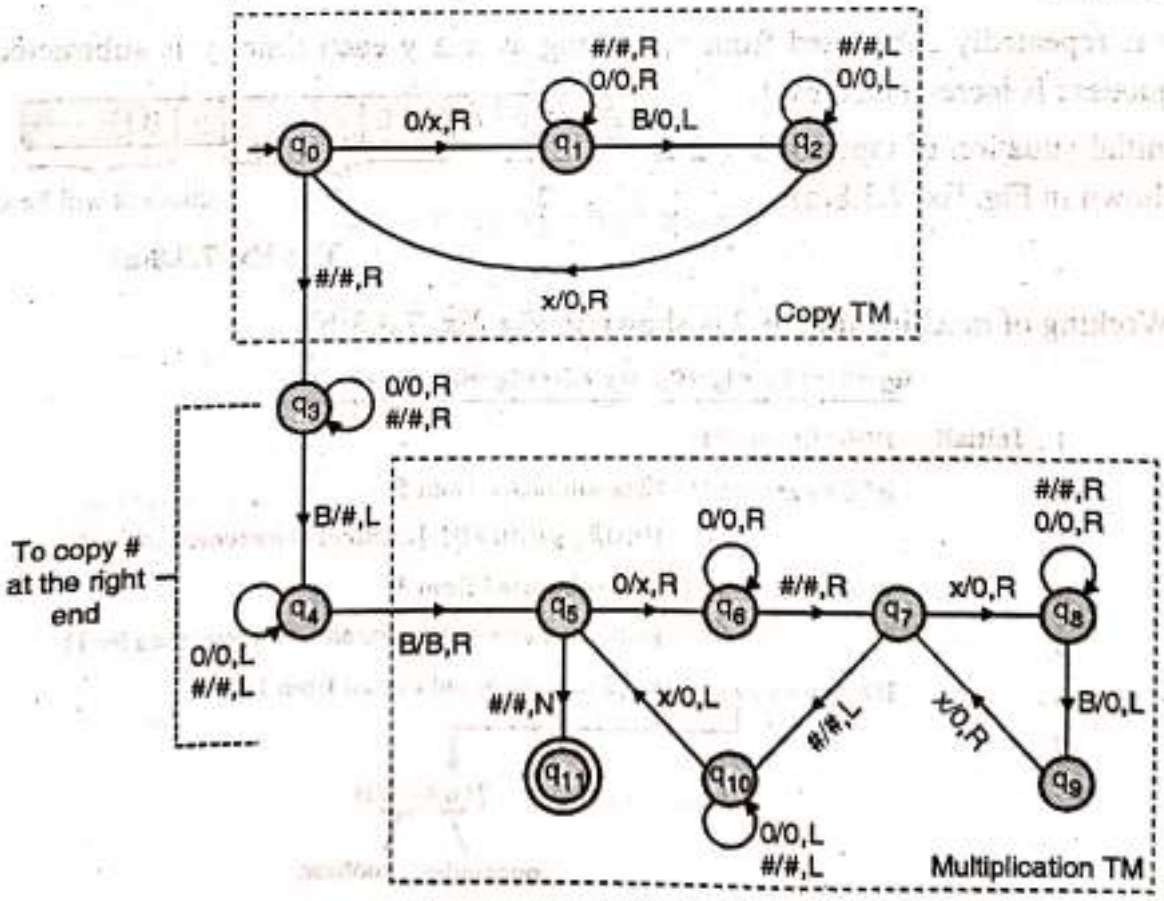


Fig. Ex. 7.3.7(b)

Scanned by CamScanner

- TM for multiplication is given in Example 7.3.6.
- TM for copy operation is given in Example 7.2.7.

Example 7.3.8 : Design a TM to compute remainder and quotient when a unary number is divided by another unary number.

Solution :

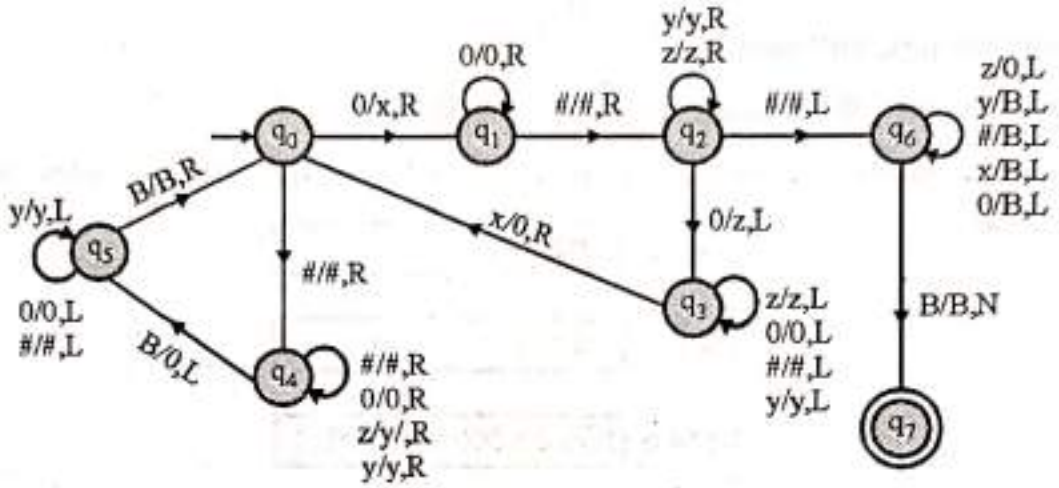


Fig. Ex. 7.3.8 : Transition diagrams

- Division can be performed through repeated subtractions.
- To perform x ÷ y, both x and y are represented in unary system. y is divisor and x is dividend.
- y is repeatedly subtracted from x, as long as x ≥ y each time, y is subtracted from x, quotient is incremented by 1.
- Initial situation of tape for 5 ÷ 2 is shown in Fig. Ex. 7.3.8(a).

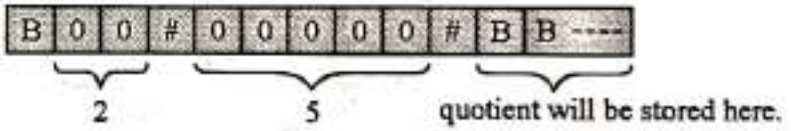


Fig. Ex. 7.3.8(a)

Working of machine for 5 ÷ 2 is shown in Fig. Ex. 7.3.8(b).

$q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_0$ $q_0 \rightarrow q_4 \rightarrow q_5 \rightarrow q_0$ $q_2 \rightarrow q_6 \rightarrow q_7$

1. **Initially** B00#00000#B
2. B00#zz000#B [2 is subtracted from 5]
3. B00#yy000#0B [quotient is incremented by 1]
4. B00#yyzz0#0B [2 is subtracted from 3]
5. B00#yyyy0#00B [quotient is incremented by 1]
6. Bx0#yyyyz#00B [2 can not be subtracted from 1]
7. B0#00B

remainder quotient

Fig. Ex. 7.3.8(b)

Scanned by CamScanner

- If y cannot be subtracted from x then z's become remainder.

Working of TM for 5 + 2 is shown in Fig. Ex. 7.3.8(c)

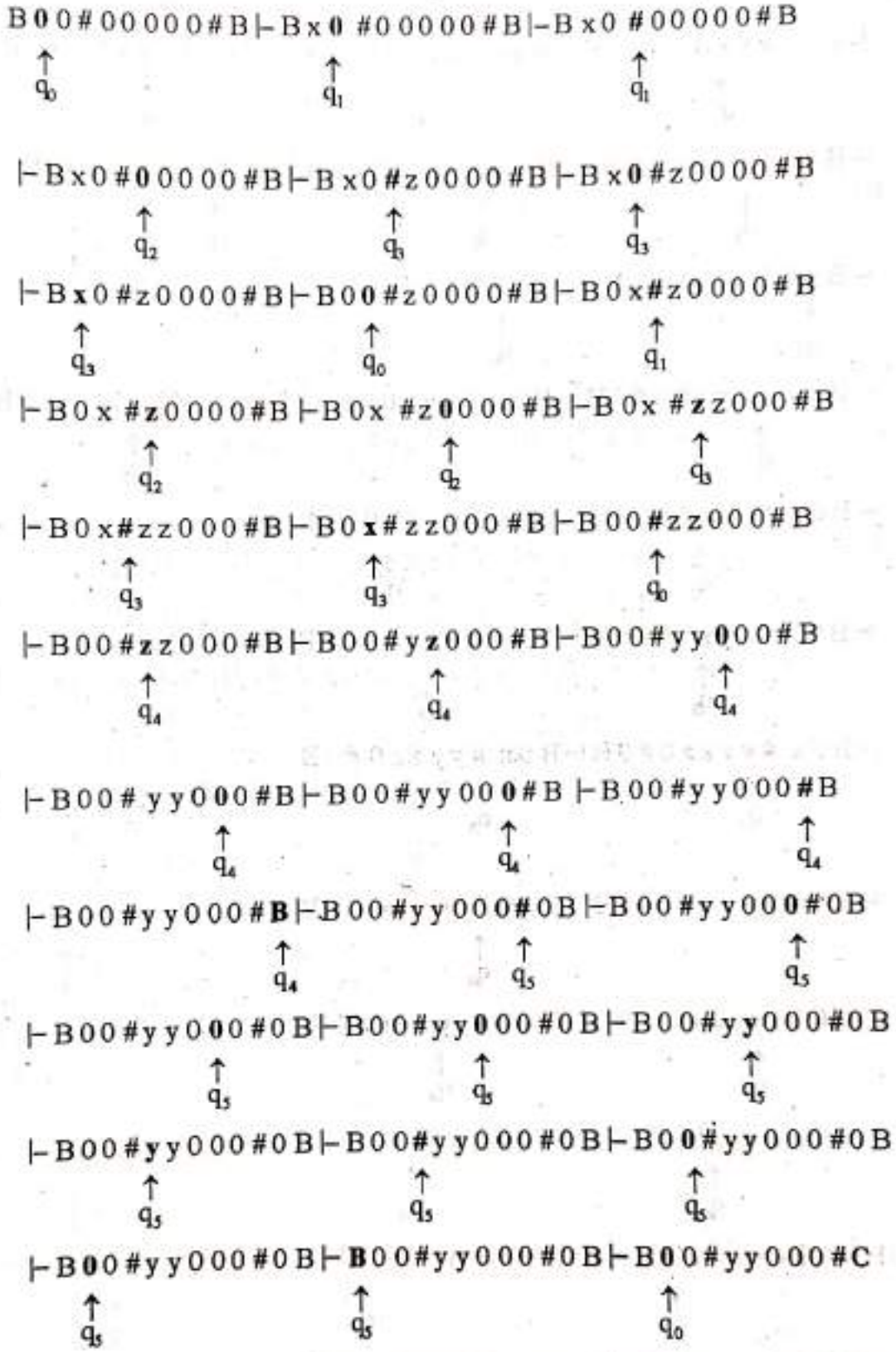


Fig. Ex. 7.3.8(c) Contd...

Scanned by CamScanner

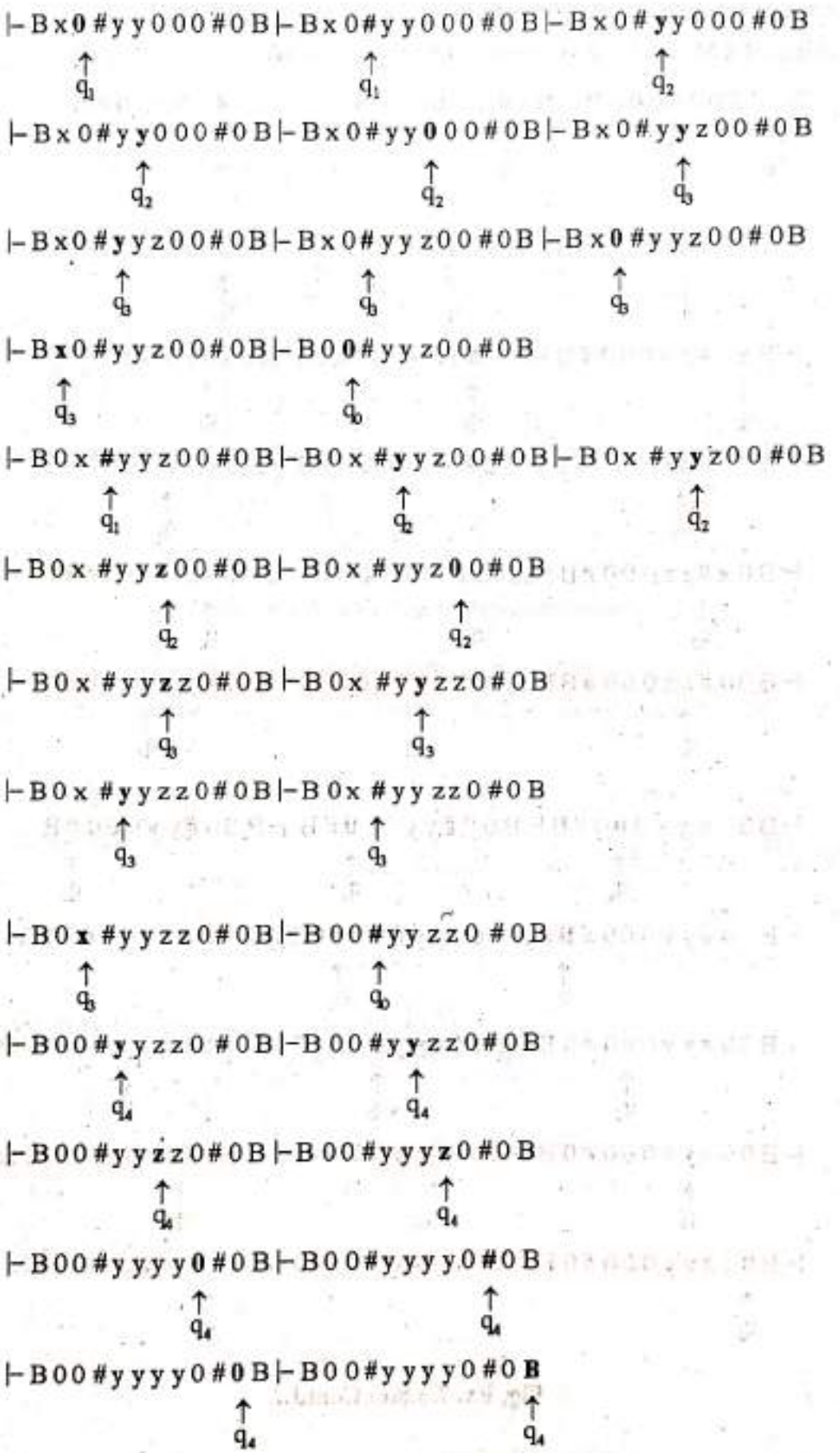


Fig. Ex. 7.3.8(c) Contd...

Scanned by CamScanner

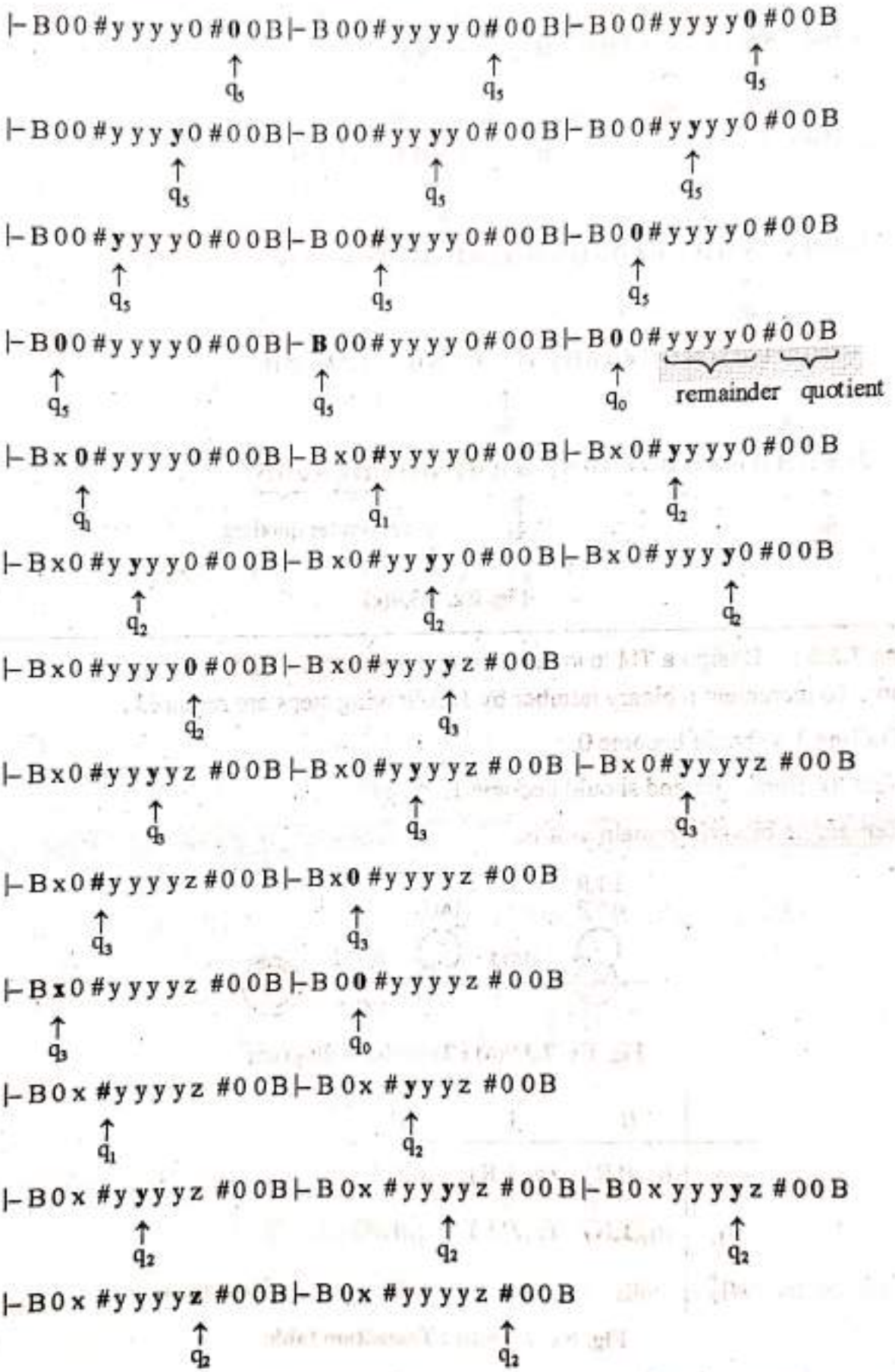


Fig. Ex. 7.3.8(c) Contd...

Scanned by CamScanner

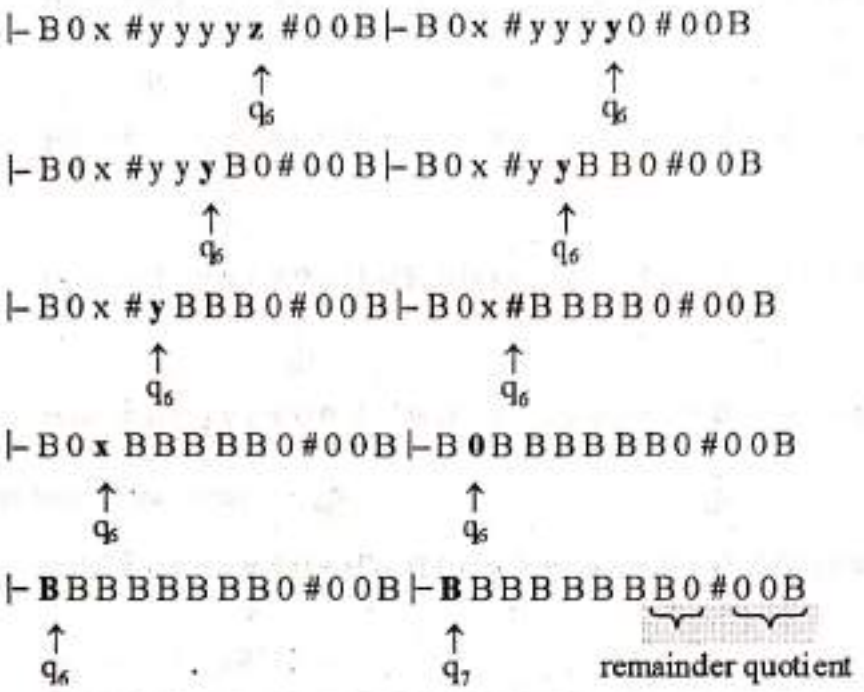


Fig. Ex. 7.3.8(c)

Example 7.3.9 : Design a TM to increment a binary number by 1.

Solution : To increment a binary number by 1, following steps are required :

1. Trailing 1's should become 0.
2. First '0' from right end should become 1.
3. Remaining bits will remain as it is.

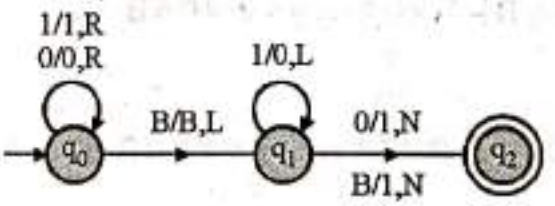


Fig. Ex. 7.3.9(a) : Transition diagram

	0	1	B	
$\rightarrow q_0$	$(q_0,0,R)$	$(q_0,1,R)$	(q_1,B,L)	
q_1	$(q_2,1,N)$	$(q_1,0,L)$	$(q_2,1,N)$	
eq_2^*	q_2	q_2	q_2	← Halting state

Fig. Ex. 7.3.9(b) : Transition table

- State q_0 moves the head to the first blank on the right side.
- A transition from q_0 to q_1, positions the head on the rightmost 0 or 1.

Scanned by CamScanner

- Trailing 1's are written as 0's in state q_1.
- A transition from q_1 to q_2 changes the first '0' or a blank from the right end towards left to 1.
- q_2 is halting state.

The working of TM being simulated for 01010111, is shown in Fig. Ex. 7.3.9(c)

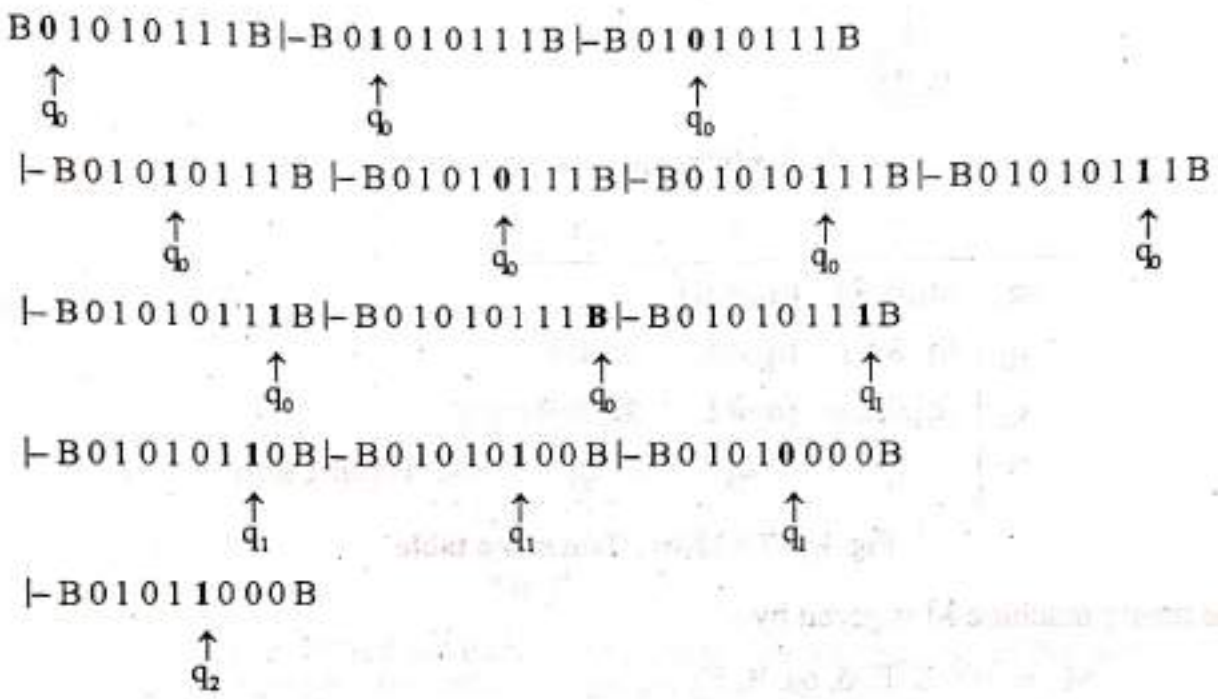


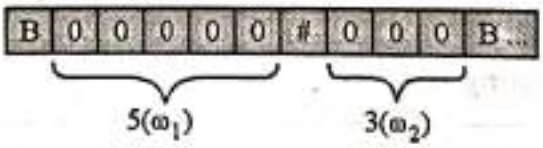
Fig. Ex. 7.3.9(c)

Example 7.3.10 : Design a TM to add two unary numbers. **MU - May 2011, Dec. 2016**

Solution :

Addition of two unary numbers can be performed through append operation. To add two numbers 5 (say ω_1) and 3 (say ω_2) will require following steps :

1. Initial configuration of tape :



2. ω_1 is appended to ω_2.

While every '0' from ω_1 is getting appended to ω_2, '0' from ω_1 is erased. ω_2 contains 8 0's, which is sum of 5 and 3.

Scanned by CamScanner

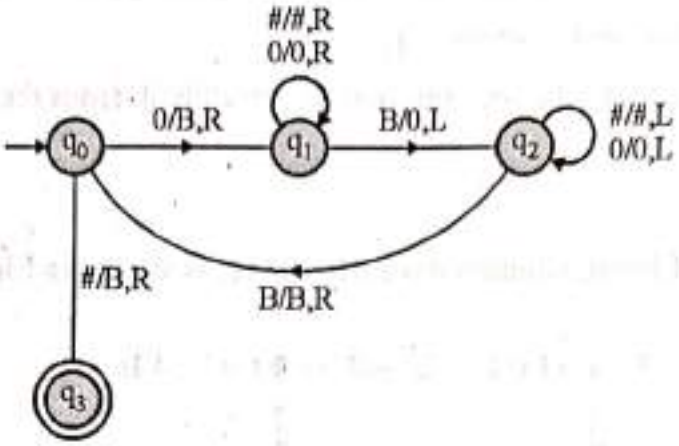


Fig. Ex. 7.3.10(a) : Transition diagram

	0	#	B	
$\rightarrow q_0$	(q_1,B,R)	(q_3,B,R)	–	
q_1	$(q_1,0,R)$	$(q_1,\#,R)$	$(q_2,0,L)$	
q_2	$(q_2,0,L)$	$(q_2,\#,L)$	(q_0,B,R)	
q_3^*	q_3	q_3	q_3	← Halting state

Fig. Ex. 7.3.10(b) : Transition table

The turing machine M is given by :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Where $Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{0, \#\}$

$\Gamma = \{0, \#, B\}$

δ = Transition function is given in Fig. Ex. 7.3.10 (a), (b)

q_0 = initial state

B = blank symbol

$F = \{q_3\}$, halting state.

Example 7.3.11 : Design a TM to find the GCD (greatest common divisor) of two unary numbers.

Solution :

Two numbers x and y are stored on the tape without any separator. Head is positioned on the last digit of x.

- Algorithm for finding GCD is based on repeated subtraction of x from y if $y \geq x$ or y from x if $y < x$. ultimately either x or y will become 0.
  - If x becomes 0 then y is GCD

Scanned by CamScanner

- If y becomes 0 then x is GCD

- Algorithm for finding GCD is given below in C-language :

```
while(x != 0 && y != 0)
{       if(x >= y)
            x = x – y;
        else
            y = y – x;
}
if(x > 0)
   printf (" % d",x);
else
   printf (" % d",y) ;
```

Situation of tape for x = 6 and y = 4 is shown in Fig. Ex. 7.3.11 :

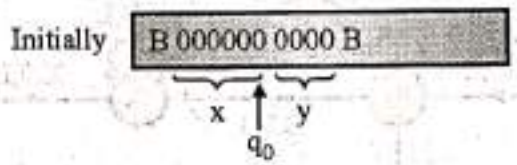


Fig. Ex. 7.3.11

1st cycle : Subtraction is carried out through several cycles. In each cycle, the last '0' of 'x' is written as x and the first '0' of y is written as Y.

1. B00000 X Y 000 B
2. B0000 XX YY 00B
3. B000 XXX YYY 0B
4. B00 XXXX YYYY B

Since, x is greater than y, y will end first.

At this point Y's are erased and X's are written as 0's

B 00 0000 B
x y
q_0

2nd cycle :

1. B0XY000B
2. BXXYY00B

B 00 00 B
x y
q_0

Since, y is greater than x, x will end first.

Scanned by CamScanner

At this point, X's are erased and Y's are written as 0's.

$$\underset{q_0}{B\,\overset{x}{0}\,0\,\overset{y}{0}\,B}$$

3rd cycle :

1. B0XY0B
2. BXXYYB

Since, y is equal to x, both x and y, have ended at the same time. At this point, x's are erased and Y's are written as 0's. The tape contains the final result.

$$\underbrace{B00B}_{\text{result}}$$

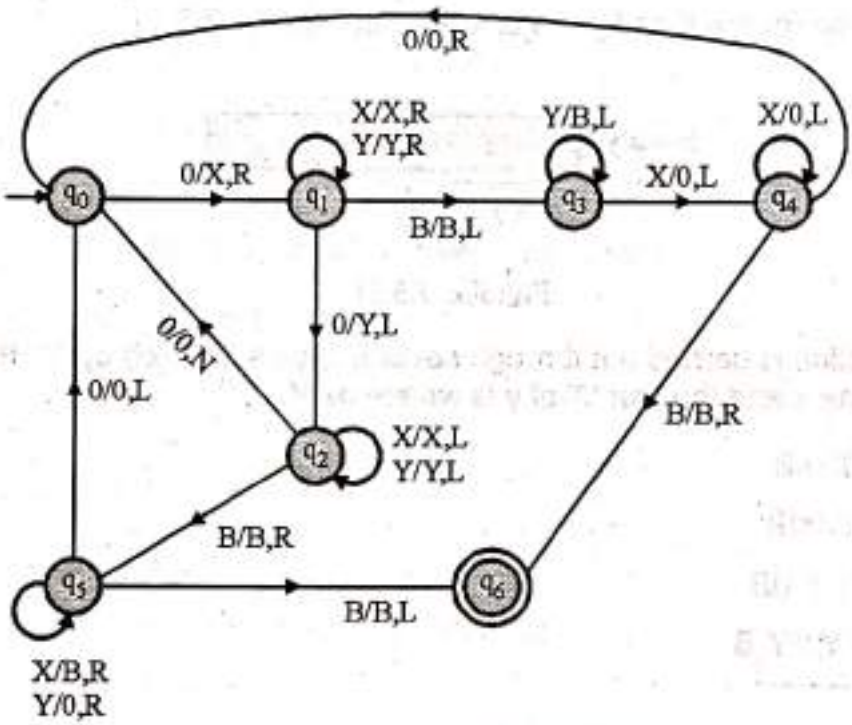


Fig. Ex. 7.3.11(a) : Transition diagram

The transition diagram of the TM is given in Fig. Ex. 7.3.11(a).

The TM can be defined as :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Where

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$

$\Sigma = \{0\}$

$\Gamma = \{0, X, Y, B\}$

δ = Transition function is given in Fig. Ex. 7.3.11(a)

Scanned by CamScanner

q_0 = Initial state

B = Blank symbol

F = $\{q_6\}$, halting state

Working of TM for GCD (6, 4) is shown in Fig. Ex. 7.3.11(b) :

6 is represented as 000000

4 is represented as 0000

Initial configuration $\overbrace{\text{B000000}}^{6}\overbrace{\text{0000B}}^{4}$ head is positioned on the last digit of 6.

$$\underset{q_0}{\text{B}\,0\,0\,0\,0\,0\,\mathbf{0}\,0\,0\,0\,0\,\text{B}} \vdash \text{B}\,0\,0\,0\,0\,\text{x}\,\underset{q_1}{\mathbf{0}}\,0\,0\,0\,\text{B} \vdash \text{B}\,0\,0\,0\,0\,\underset{q_2}{\mathbf{x}}\,\text{y}\,0\,0\,0\,\text{B}$$

$$\vdash \text{B}\,0\,0\,0\,0\,\underset{q_2/q_0}{\mathbf{0}}\,\text{x}\,\text{y}\,0\,0\,0\,\text{B} \vdash \text{B}\,0\,0\,0\,0\,\text{x}\,\underset{q_1}{\mathbf{x}}\,\text{y}\,0\,0\,0\,\text{B} \vdash \text{B}\,0\,0\,0\,0\,\text{x}\,\text{x}\,\underset{q_1}{\mathbf{y}}\,0\,0\,0\,\text{B}$$

$$\vdash \text{B}\,0\,0\,0\,0\,\text{x}\,\text{x}\,\text{y}\,\underset{q_1}{\mathbf{0}}\,0\,0\,\text{B} \vdash \text{B}\,0\,0\,0\,0\,\text{x}\,\text{x}\,\underset{q_2}{\mathbf{y}}\,\text{y}\,0\,0\,\text{B} \vdash \text{B}\,0\,0\,0\,0\,\text{x}\,\underset{q_2}{\mathbf{x}}\,\text{y}\,\text{y}\,0\,0\,\text{B}$$

$$\vdash \text{B}\,0\,0\,0\,0\,\underset{q_2}{\mathbf{x}}\,\text{x}\,\text{y}\,\text{y}\,0\,0\,\text{B} \vdash \text{B}\,0\,0\,0\,\underset{q_2}{\mathbf{0}}\,\text{x}\,\text{x}\,\text{y}\,\text{y}\,0\,0\,\text{B} \vdash \text{B}\,0\,0\,0\,\underset{q_0}{\mathbf{0}}\,\text{x}\,\text{x}\,\text{y}\,\text{y}\,0\,0\,\text{B}$$

$$\vdash \text{B}\,0\,0\,0\,\text{x}\,\underset{q_1}{\mathbf{x}}\,\text{x}\,\text{y}\,\text{y}\,0\,0\,\text{B} \vdash \text{B}\,0\,0\,0\,\text{x}\,\text{x}\,\underset{q_1}{\mathbf{x}}\,\text{y}\,\text{y}\,0\,0\,\text{B} \vdash \text{B}\,0\,0\,0\,\text{x}\,\text{x}\,\text{x}\,\underset{q_1}{\mathbf{y}}\,\text{y}\,0\,0\,\text{B}$$

$$\vdash \text{B}\,0\,0\,0\,\text{x}\,\text{x}\,\text{x}\,\underset{q_1}{\mathbf{y}}\,0\,0\,\text{B} \vdash \text{B}\,0\,0\,0\,\text{x}\,\text{x}\,\text{x}\,\text{y}\,\underset{q_1}{\mathbf{y}}\,0\,\text{B} \vdash \text{B}\,0\,0\,0\,\text{x}\,\text{x}\,\text{x}\,\text{y}\,\underset{q_2}{\mathbf{y}}\,\text{y}\,0\,\text{B}$$

$$\vdash \text{B}\,0\,0\,0\,\text{x}\,\text{x}\,\underset{q_2}{\mathbf{x}}\,\text{y}\,\text{y}\,\text{y}\,0\,\text{B} \vdash \text{B}\,0\,0\,0\,\text{x}\,\text{x}\,\underset{q_2}{\mathbf{x}}\,\text{y}\,\text{y}\,\text{y}\,0\,\text{B} \vdash \text{B}\,0\,0\,0\,\text{x}\,\underset{q_2}{\mathbf{x}}\,\text{x}\,\text{y}\,\text{y}\,\text{y}\,0\,\text{B}$$

$$\vdash \text{B}\,0\,0\,0\,\underset{q_2}{\mathbf{x}}\,\text{x}\,\text{y}\,\text{y}\,\text{y}\,0\,\text{B} \vdash \text{B}\,0\,0\,\underset{q_2}{\mathbf{0}}\,\text{x}\,\text{x}\,\text{y}\,\text{y}\,\text{y}\,0\,\text{B} \vdash \text{B}\,0\,0\,\underset{q_0}{\mathbf{0}}\,\text{x}\,\text{x}\,\text{y}\,\text{y}\,\text{y}\,0\,\text{B}$$

Fig. Ex. 7.3.11(b) contd....

Scanned by CamScanner

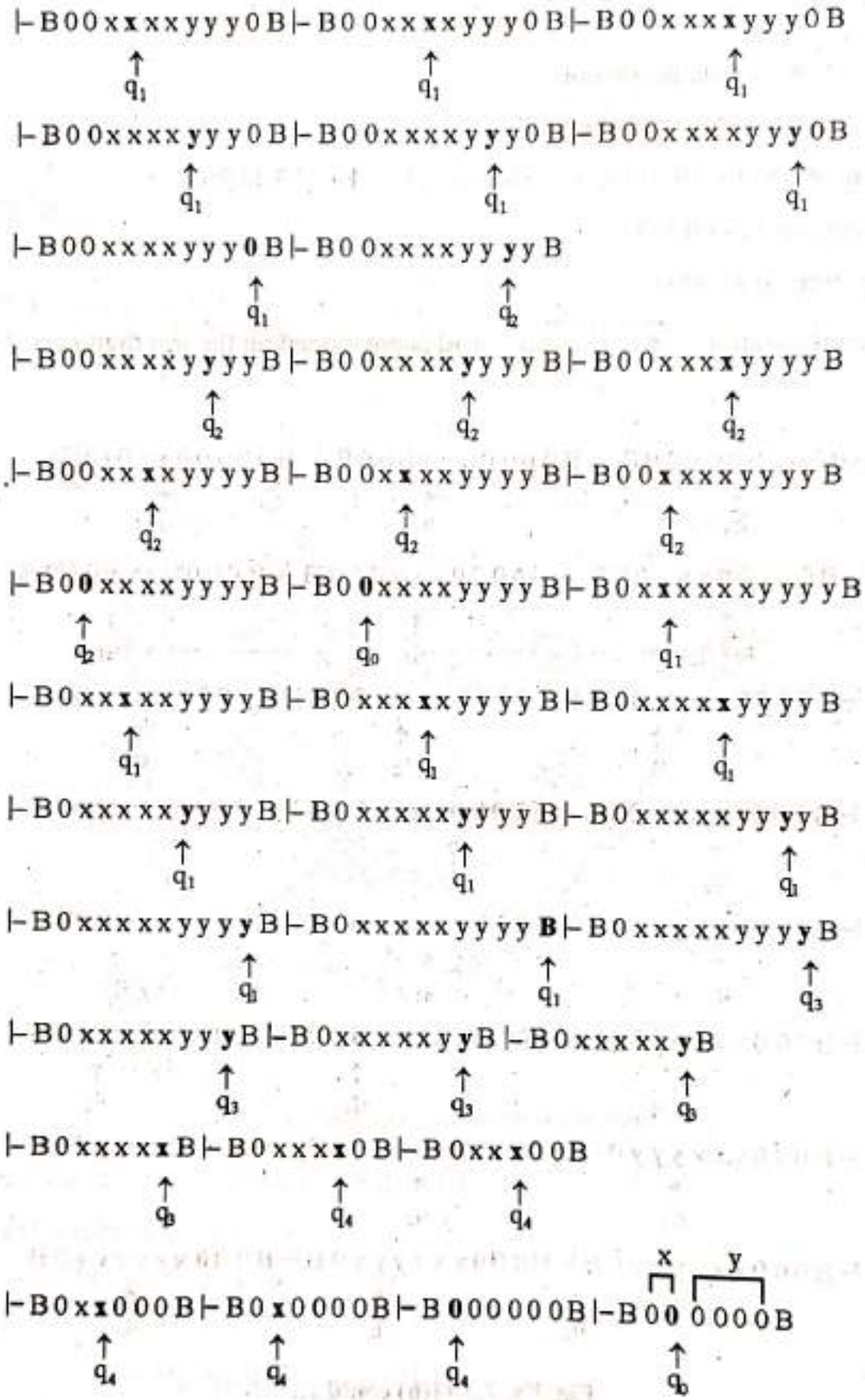


Fig. Ex. 7.3.11(b) contd....

Scanned by CamScanner

$\vdash$ B 0 x **0** 0 0 B $\vdash$ B 0 **x** y 0 0 B $\vdash$ B **0** x y 0 0 B
(q_1) (q_2) (q_2)

$\vdash$ B **0** x y 0 0 B $\vdash$ B x **x** y 0 0 B $\vdash$ B x x **y** 0 0 B $\vdash$ B x x y **0** 0 B
(q_0) (q_1) (q_1) (q_1)

$\vdash$ B x x **y** y 0 0 B $\vdash$ B x **x** y y 0 0 B $\vdash$ B **x** x y y 0 0 B
(q_2) (q_2) (q_2)

$\vdash$ **B** x x y y 0 0 B $\vdash$ B **x** x y y 0 0 B $\vdash$ B **x** y y 0 0 B $\vdash$ B **y** y 0 0 B
(q_2) (q_3) (q_5) (q_5)

x y

$\vdash$ B 0 **y** 0 0 B $\vdash$ B 0 0 **0** 0 B $\vdash$ B 0 **0** 0 0 B
(q_5) (q_5) (q_0)

$\vdash$ B 0 x **0** 0 B $\vdash$ B 0 **x** y 0 B $\vdash$ B **0** x y 0 B $\vdash$ B **0** x y 0 B
(q_1) (q_2) (q_2) (q_0)

$\vdash$ B x **x** y 0 B $\vdash$ B x x **y** 0 B $\vdash$ B x x y **0** B $\vdash$ B x x **y** y B
(q_1) (q_1) (q_1) (q_2)

$\vdash$ B x **x** y y B $\vdash$ B **x** x y y B $\vdash$ **B** x x y y B $\vdash$ B **x** x y y B
(q_2) (q_2) (q_2) (q_5)

$\vdash$ B **x** y y B $\vdash$ B **y** y B $\vdash$ B 0 **y** B $\vdash$ B 0 0 **B** $\vdash$ B 0 **0** B
(q_5) (q_5) (q_5) (q_5) (q_6)

Result = 00, which is 2.

Fig. Ex. 7.3.11(b)

Example 7.3.12 : Design a TM to find the value of $\log_2(n)$, where n is any binary number.

MU - Dec. 2007

Solution : $\log_2(n)$ of any number n lying between 2^n and 2^{n+1} is given by n.

i.e. if $2^n \leq n < 2^{n+1}$, then $\log_2(n) = n$

Let us consider the case of a number

$n = 36$

$2^5 \leq 36 < 2^6$

Therefore, $\log_2(36) = 5$

36 can be written as 100100.

Scanned by CamScanner

- Any number n satisfying the condition $2^5 \leq n < 2^6$ can be written as 1XXXXX (where X stands for either 1 or 0).
- $\log_2$ (1XXXXX) can be calculated by erasing the most significant bit 1 and renaming other bits as '0'. Unary representation of 5 is 00000.

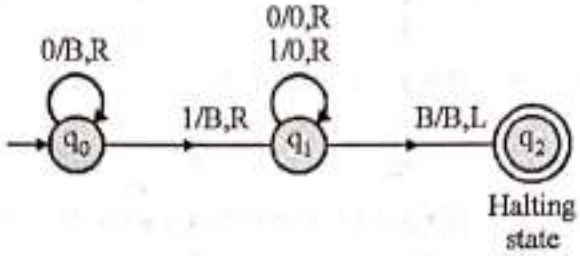


Fig. Ex. 7.3.12(a) : Transition diagram

	0	1	B	
$\rightarrow q_0$	(q_0,B,R)	(q_1,B,R)	–	
q_1	$(q_1,0,R)$	$(q_1,0,R)$	(q_2,B,L)	
q_2^*	q_2	q_2	q_2	← Halting state

Fig. Ex. 7.3.12(b) : Transition table

Working of TM for $(36)_{10}$ is shown in Fig. Ex. 7.3.12(c) :

$(36)_{10} = (0100100)_2$

$$\underset{\uparrow q_0}{B\,0}\,1\,0\,0\,1\,0\,0\,B \vdash B\,\underset{\uparrow q_0}{1}\,0\,0\,1\,0\,0\,B \vdash B\,0\,0\,\underset{\uparrow q_1}{1}\,0\,0\,B$$

$$\vdash B\,0\,0\,\underset{\uparrow q_1}{1}\,0\,0\,B \vdash B\,0\,0\,\underset{\uparrow q_1}{1}\,0\,0\,B \vdash B\,0\,0\,0\,0\,\underset{\uparrow q_1}{0}\,B$$

$$\vdash B\,0\,0\,0\,0\,\underset{\uparrow q_1}{0}\,B \vdash B\,0\,0\,0\,0\,\underset{\uparrow q_1}{0}\,B \vdash B\,0\,0\,0\,0\,\underset{\uparrow q_2}{0}\,B$$

Fig. Ex. 7.3.12(c)

Example 7.3.13 : Design a Turing machine to compute n!.

MU - Dec. 2008

Solution : It is assumed that n is represented in unary system.

Factorial of n can be calculated through repeated application of :

1. Multiplication
2. Copy

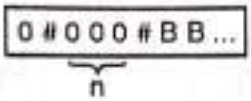
Operations.

Algorithm is being explained with the help of example.

Scanned by CamScanner

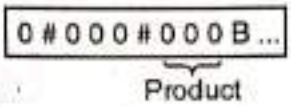
Algorithm for $\lfloor 3 \rfloor$.

Initial configuration

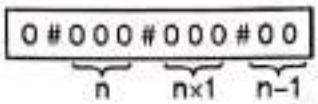


Cycle 1 :

1. Multiplication

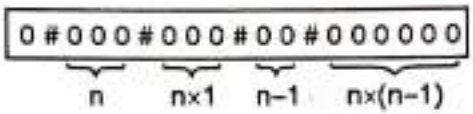


2. Copy n – 1, i.e. 2

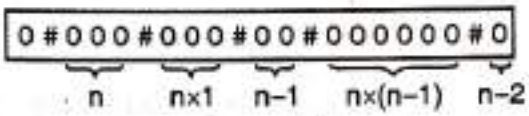


Cycle 2 :

1. Multiplication

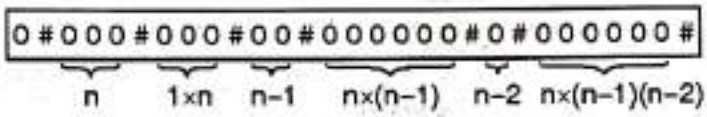


2. Copy n – 2, i.e. 1



Cycle 3 :

1. Multiplication



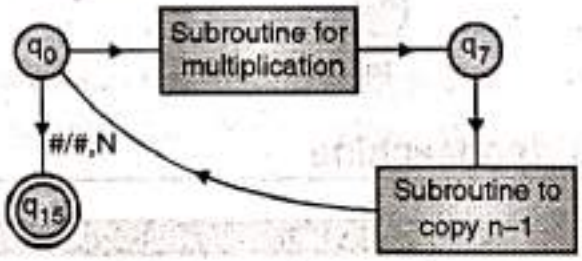


Fig. Ex. 7.3.13(a)

Subroutine for multiplication :

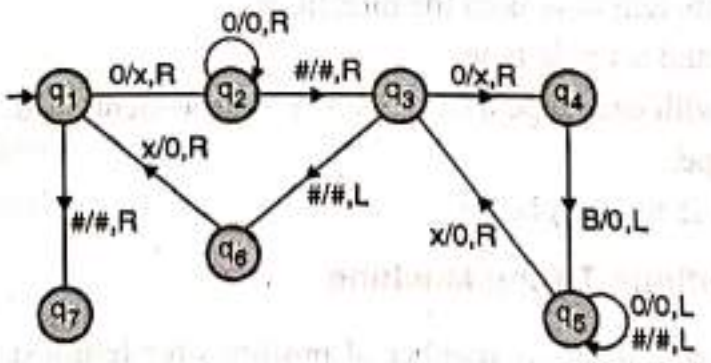


Fig. Ex. 7.3.13(b)

Scanned by CamScanner

Subroutine to copy n – 1 :

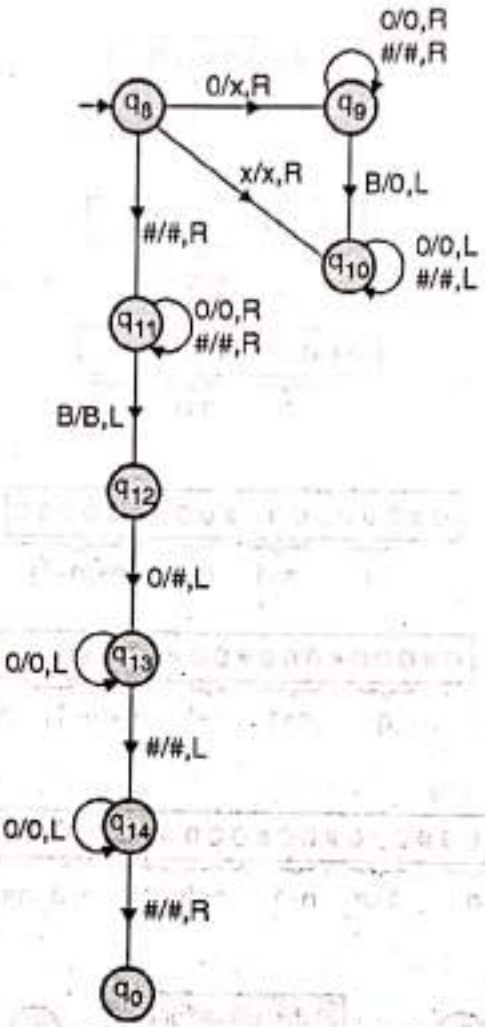


Fig. Ex. 7.3.13(c)

7.4 Extension of Turing Machine

MU - Dec. 2006, Dec. 2008, Dec. 2009, Dec. 2010, May 2014, May 2015, May 2017

In a standard Turing machine, the tape is semi-infinite. It is bounded on the left and unbounded on the right side. Some of the extensions of turing machine are given below :

1. Tape is of infinite length in both the directions.
2. Multiple heads and a single tape.
3. Multiple tapes with each tape having its own independent head.
4. K-dimension tape.
5. Non-deterministic turing machine.

7.4.1 Two-way Infinite Turing Machine

In a standard turing machine number of positions for leftmost blanks is fixed and they are included in instantaneous description, where the right-hand blanks are not included.

In the two way infinite Turing machine, there is an infinite sequence of blanks on each

Scanned by CamScanner

side of the input string. In an instantaneous description, these blanks are never shown.

7.4.2 A Turing Machine with Multiple Heads

MU - Dec. 2007

A turing machine with single tape can have multiple heads. Let us consider a turing machine with two heads H_1 and H_2. Each head is capable of performing read/write /move operation independently.

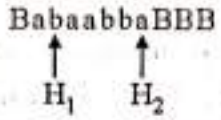


Fig. 7.4.1 : A Turing machine with two heads

The transition behavior of 2-head one tape Turing machine can be defined as given below :

δ (State, Symbol under H_1, Symbol under H_2) = (New state, (S_1, M_1), (S_2, M_2))

Where,

S_1 is the symbol to be written in the cell under H_1.

M_1 is the movement (L, R, N) of H_1.

S_2 is the symbol to be written in the cell under H_2.

M_2 is the movement (L, R, N) of H_2.

7.4.3 Multi-Tape Turing Machine

MU - Dec. 2007

Multi-Tape turing machine has multiple tuples with each tape having its own independent head. Let us consider the case of a two tape turing machine. It is shown in Fig. 7.4.2.

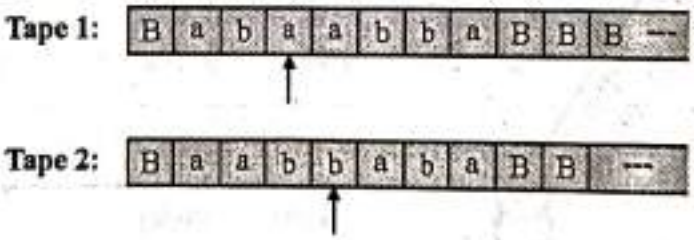


Fig. 7.4.2 : A two-tape turing machine

The transition behavior of a two-tape Turing machine can be defined as :

$\delta(q_1,a_1,a_2) = (q_2,(S_1,M_1),(S_2,M_2))$

Where,

q_1 is the current state,

q_2 is the next state,

a_1 is the symbol under the head on tape 1,

a_2 is the symbol under the head on tape 2,

Scanned by CamScanner

S_1 is the symbol written in the current cell on tape 1,

S_2 is the symbol written in the current cell on tape 2,

M_1 is the movement (L, R, N) of head on tape 1,

M_2 is the movement (L, R, N) of head on tape 2.

Example 7.4.1 : Construct a two-tape turing machine to recognize words of the form ωωω over alphabet {a,b}.

Solution : Let the initial string be placed on tape 1 and tape 2 may contain all blanks. Initial configuration for ω = abb is shown in Fig. Ex. 7.4.1(a).

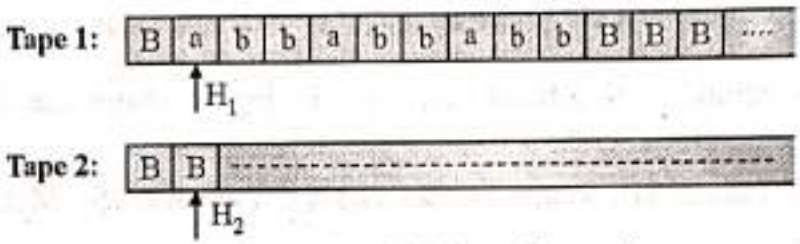


Fig. Ex. 7.4.1(a) : Initial configuration

Step 1: This step will copy the last ω from ωωω on tape 1 to tape 2 in reverse order.

- H_1 is advanced by two places while re-writing a as x and b as y.
- H_1 is moved to the last character and the last character from tape 1 is copied to tape 2, last character of type 1 is erased.

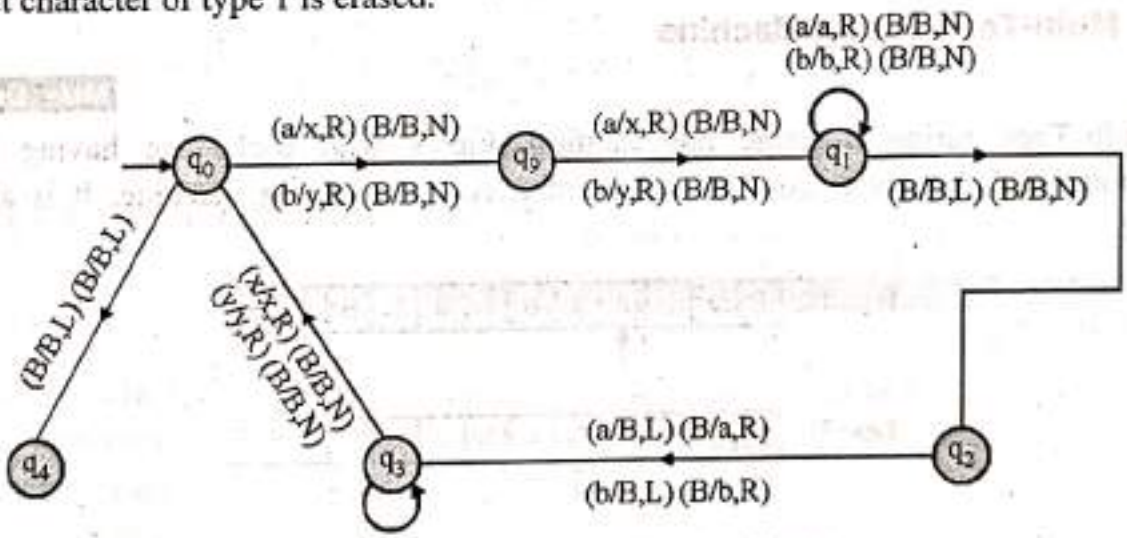


Fig. Ex. 7.4.1(b)

Contents of tapes after step 1 are shown in Fig. Ex. 7.4.1(c)

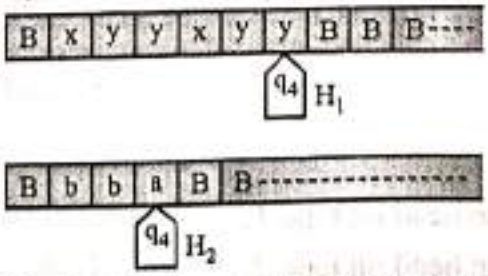


Fig. Ex. 7.4.1(c) : Last ω from tape 1 is copied as ω^R to tape 2

Scanned by CamScanner

Step 2 : H_2 is moved to leftmost character on tape 2.

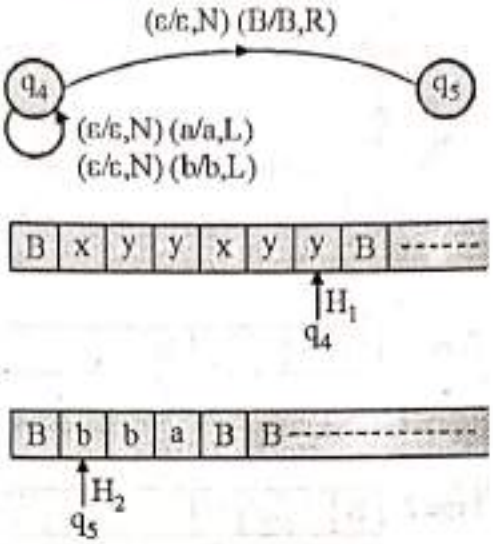


Fig. Ex. 7.4.1(d) : Situation after step 2

Step 3 : Two sets of xyy are matched with bba in the reverse order.

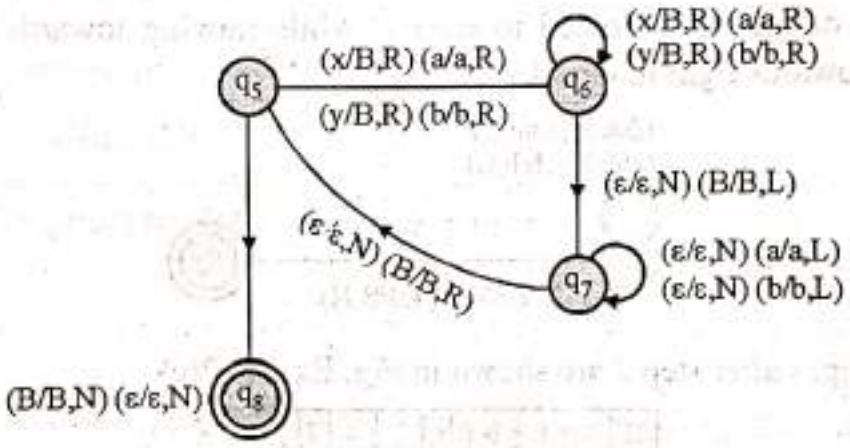


Fig. Ex. 7.4.1(e) : Two sets of xyy are matched with bba in the reverse order

---

Example 7.4.2 : Construct a two-tape turing machine to convert an input ω into $\omega\omega^R$.

Solution :

Initially, tape 1 contains ω and the tape 2 is blank String ω is assumed to be abb.

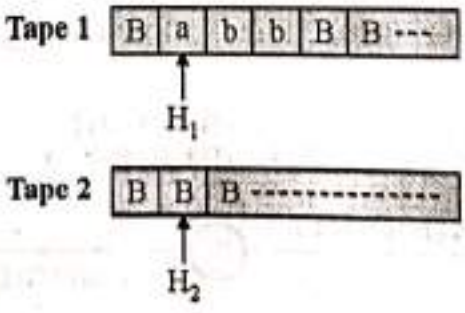
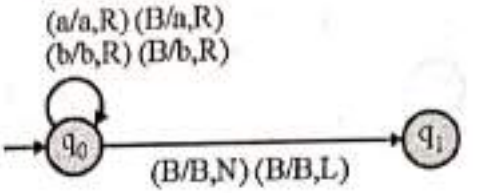


Fig. Ex. 7.4.2(a) : Initial configuration

Scanned by CamScanner

Step 1 : String abb is copied to tape 2.



Contents of tapes after step1 are shown in Fig. Ex. 7.4.2(b).

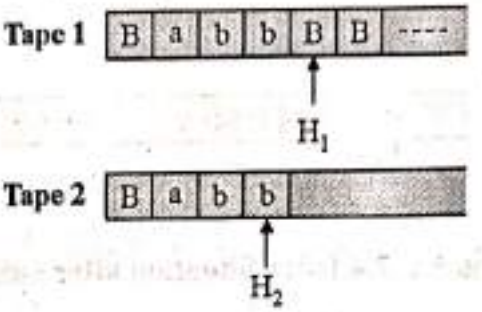
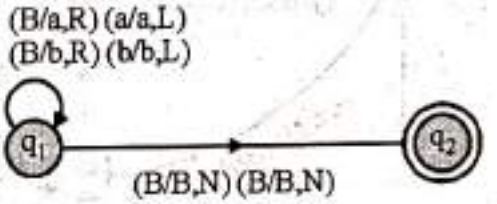


Fig. Ex. 7.4.2(b) : Contents of tapes after step 1

Step 2 : Contents of tape 2 is copied to tape 1, while moving towards left in tape 2 and moving towards right in tape 1.



Contents of tapes after step 2 are shown in Fig. Ex. 7.4.2(c)

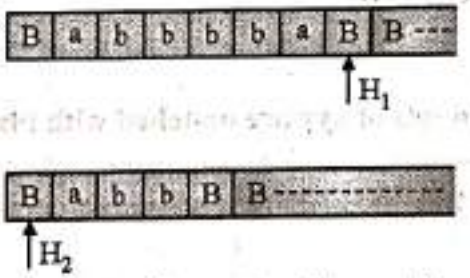


Fig. Ex. 7.4.2(c) : Contents of tapes after step 2

Transition diagram of the turing machine is shown in Fig. Ex. 7.4.2(d)

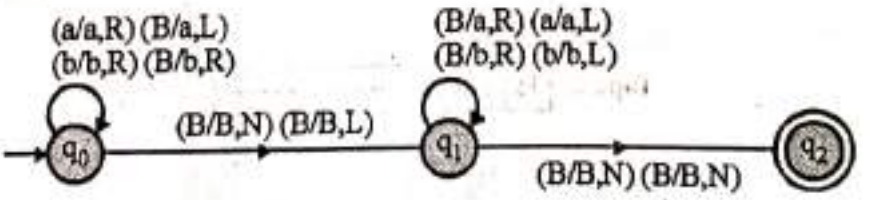


Fig. Ex. 7.4.2(d) : Transition diagram for Example 7.4.2

Scanned by CamScanner

7.4.4 Non-deterministic Turing Machine

Non-deterministic is a powerful feature. A non-deterministic TM machine might have, on certain combinations of state and symbol under the head, more than one possible choice of behaviour.

- Non-deterministic does not make a TM more powerful.
- For every non-deterministic TM, there is an equivalent deterministic TM.
- It is easy to design a non-deterministic TM for certain class of problems.
- A string is said to be accepted by a NDTM, if there is at least one sequence of moves that takes the machine to final state.
- An example of non-deterministic move for a TM is shown in Fig. 7.4.3.

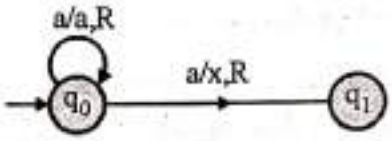


Fig. 7.4.3 : A sample move for NDTM

The transition behaviour for state q_0 for TM of Fig. 7.4.3 can be written as

$$\delta(q_0, a) = \{(q_0, a, R)(q_1, x, R)\}$$

Example 7.4.3 : Construct NDTM to recognize words of the form ωω over alphabet {a,b}

Solution :

First character of the second ω in ωω can be located non-deterministically.

1. TM skips the first ω and re-writes every a as x and every b as y for every a, b belonging to first ω of ωω. Contents of tape after step 1 are shown in Fig. Ex. 7.4.3(b).

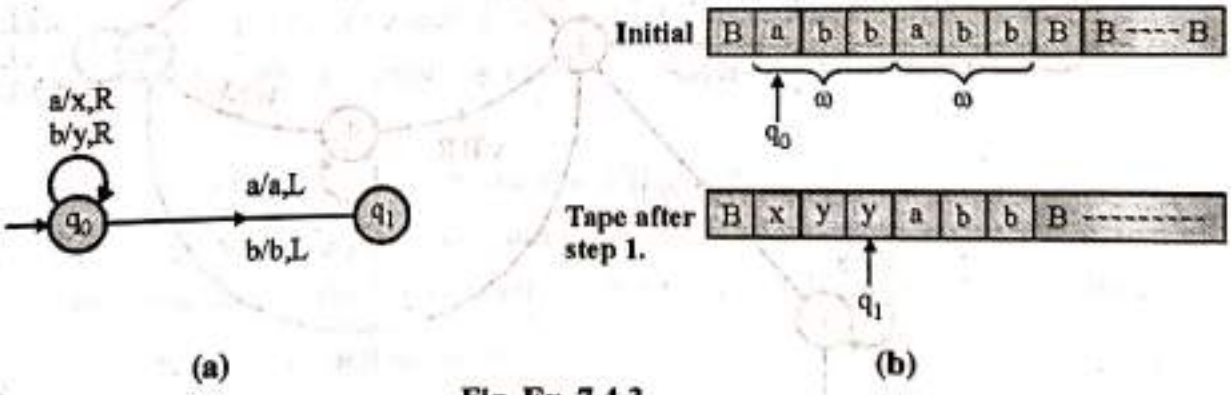


Fig. Ex. 7.4.3

2. Head is sent back to the first character and then first half is matched with the second half while matching the two ω's every character of second ω is re-written as z.

- A cycle through $q_2 \rightarrow q_3 \rightarrow q_4 \rightarrow q_2$ matched x with a.
- A cycle through $q_2 \rightarrow q_5 \rightarrow q_4 \rightarrow q_2$ matched y with b.

Scanned by CamScanner

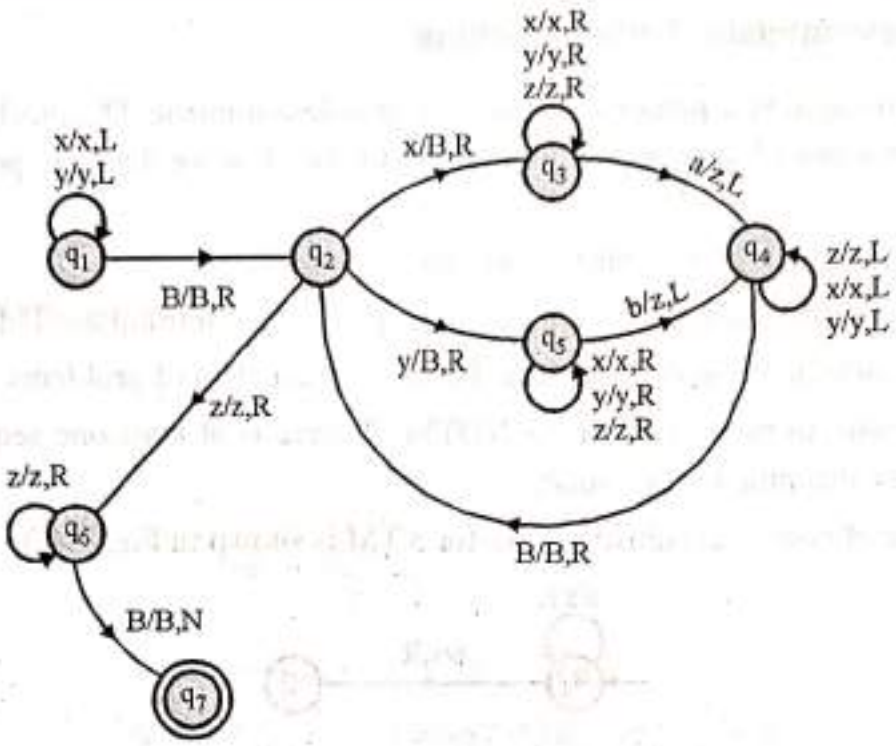


Fig. Ex. 7.4.3(c)

- After every x, y ∈ first ω is matched, machine moves to state q_6.
- The state q_6 is to establish that every character from the second ω of ωω has been matched.
- q_7 is the final state.
- The final TM is shown in Fig. Ex. 7.4.3(d).

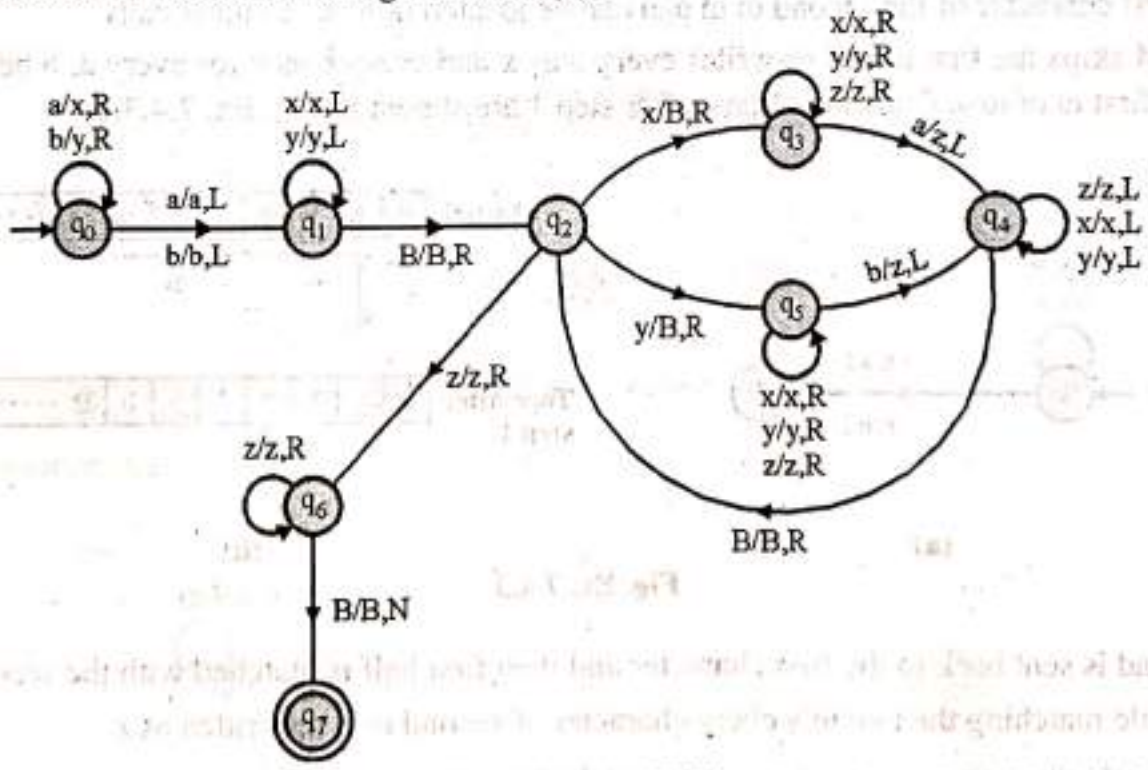


Fig. Ex. 7.4.3(d) : NDTM for ωω

Scanned by CamScanner

7.5 Universal Turing Machine

MU - Dec. 2005, May 2007, Dec. 2007, May 2008, Dec. 2008, May 2009, May 2010, Dec. 2011, May 2012, Dec. 2012, Dec. 2015

A general-purpose computer can be programmed to solve different types of problems. A TM can also behave like a general-purpose computer. A general purpose computer solves a problem as given below :

1. A program is written in a high level language and its machine-code is obtained with the help of a complier.
2. Machine code is loaded in main memory.
3. Input to the program can also be loaded in memory.
4. Program stored in memory is executed line by line. Execution involves reading a line of code pointed by IP (instruction pointer), decoding the code and executing it.

We can follow a similar approach for a TM. Such, a TM is known as **Universal Turing Machine.** Universal Turing Machine (UTM) can solve all sorts of solvable problems.

- A Turing machine M is designed to solve a particular problem p, can be specified as :
  1. The initial state q_0 of the TM M.
  2. The transition function δ of M can be specified as given :

If the current state of M is q_i and the symbol under the head is a_i then the machine moves to state q_j while changing a_i to a_j. The move of tape head may be :

1. To-left,
2. To-Right or
3. Neutral

Such a move of TM can be represented by tuple

$\{(q_i,a_i,q_j,a_j,m_f) : q_i,q_j,\in Q ; a_i,a_j,\in \Gamma ; m_f \in \{\text{To- left, To-Right, Neutral}\}\}$

- UTM should be able to simulate every turing machine. Simulation of a Turing will involve :
  1. Encoding behaviour of a particular TM as a program.
  2. Execution of the above program by UTM.
- A move of the form (q_i,a_i,q_j,a_j,m_f) can be represented as $10^{i+1}\ 10^{j}\ 10^{j+1}\ 10^{j}\ 10^{K}$,

  Where $K = 1$, if move is to the left

  $K = 2$, if move is to the right

  $K = 3$, if move is 'no-move'

  State q_0 is represented by 0,

  State q_1 is represented by 00,

Scanned by CamScanner

State q_n is represented by 0^{n+1}.

First symbol can be represented by 0,

Second symbol can be represented by 00 and so on.

Two elements of a tuple representing a move are separated by 1.

- Two moves are separated by 11.

Execution by UTM : We can assume the UTM as a 3-tape turing machine.

1. Input is written on the first tape.
2. Moves of the TM in encoded form is written on the second tape.
3. The current state of TM is written on the third tape.

The control unit of UTM by counting number of 0's between 1's can find out the current symbol under the head. It can find the current state from the tape 3. Now, it can locate the appropriate move based on current input and the current state from the tape 2. Now, the control unit can extract the following information from the tape 2 :

1. Next state 2. Next symbol to be written
3. Move of the head.

Based on this information, the control unit can take the appropriate action.

7.5.1 Church-Turing Hypothesis

MU - May 2017

The Turing machine is a general model of computation. Any algorithmic procedure can be solved by G computer can also be solved by a TM. Problems computed by a computer or a TM are also known as partial recursive functions. Some enhancements to TM made the Church-Turing thesis acceptable. These enhancements are :

1. Multi-tape 2. Multi-head
3. Infinite tapes 4. Non-determinism.

Since the introduction of TM, no one has suggested an algorithm than can be solved by a computer but cannot be solved by a TM.

7.5.2 Power of Various Machines

MU - Dec. 2015, May 2016, Dec. 2016

- A post machine can simulate functioning of a turing machine. Hence a turing machine and a post machine have equal power.
- A post machine is more powerful than a PDA. A string of the $a^nb^nc^n$ can be handled by a post machine but it cannot be handled by a PDA.
- An NPDA is more powerful than a DPDA. A string of the form $\omega\omega^R$ can be handled by

Scanned by CamScanner

an NPDA but it cannot be handled by a DPDA.

- DPDA is more powerful than FA. A string of the form $a^n b^n$ can be handled by a DPDA but it cannot be handled by FA.

Example 7.5.1 : Explain the power of turing machine over finite state machine.

MU - Dec. 2012, Dec. 2016

Solution :

1. Turing machine is more powerful than FA.
2. FA has no memory; Turing machine has additional memory in the form of a tape.
3. FA cannot modify its input. A TM can modify its own input.
4. FA cannot be used for arithmetic operations. A TM can perform arithmetic operations.
5. A language accepted by finite automata is regular language. It cannot handle context free language, context sensitive or recursive languages. These languages can be handled by a TM.

7.6 Linear Bounded Automata

A linear bounded automata(LBA) restricts the length of the tape. The LBA is important as the set of context-sensitive languages is accepted by it.

A linear bounded automata is same as a non-deterministic Turing machine except in the followings :

1. There is a special symbol < (say), marking the left end of the tape.
2. There is a special symbol > (say), marking the right end of the tape.
3. The head is not allowed to move beyond these end-markers.
4. The head is not allowed to change these end-markers.

An LBA can be denoted by :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F, <, >)$$

Where < and > are symbols in Σ, the left and right end-markers. Q, Σ, Γ, δ, q_0 and F are same as for a non-deterministic TM.

7.7 University Questions and Answers

Dec. 2005

Q. 1 Write short note on : Universal TM **(Section 7.5)** **(5 Marks)**

Scanned by CamScanner

May 2006

Q. 2 Design a Turing machine that can accept the set of all even palindromes over alphabet {0, 1}. **(Example 7.2.18) (Note : '0' as 'a' and '1' as 'b')** **(8 Marks)**

Q. 3 Design turing machine to recognize the language $L = \{1^n2^n3^n \mid n \geq 1\}$ **(Example 7.2.17) (Note : '0' as 'a' , '1' as 'b' and '3' as 'c')** **(10 Marks)**

Dec. 2006

Q. 4 Prove the variations and equivalence of Turing machine. **(Section 7.4)** **(10 Marks)**

Q. 5 Design TM to accept the language.

$L = \{x, \varepsilon, \{0, 1\}^* \mid x$ contains equal number of 0's and 1's$\}$. Simulate the operation for the string 110100. **(Similar to Example 7.2.9) (Note : 0 as 'a' and 1 as 'b')(10 Marks)**

May 2007

Q. 6 Design a TM to multiply 2 unary numbers. **(Example 7.3.6)** **(12 Marks)**

Q. 7 Write note on :Universal TM **(Section 7.5)** **(5 Marks)**

Dec. 2007

Q. 8 Design the turing machine to find the value of $\log_2 n$ where n is any binary number and a prefect square. **(Example 7.3.12)** **(10 Marks)**

Q. 9 Write note on 'Multiple Turing machine'. **(Sections 7.4.2 and 7.4.3)** **(5 Marks)**

Q. 10 Write note on 'Universal Turing Machine **(Section 7.5)** **(5 Marks)**

May 2008

Q. 11 Design a Turing Machine which recognizes the language $L = \{a^nb^nc^n \mid n \geq 1\}$. **(Example 7.2.17)** **(10 Marks)**

Q. 12 Write note on 'Universal turing machine'. **(Section 7.5)** **(5 Marks)**

Q. 13 Design TM which will recognize strings containing equal number of 0's and 1's. **(Example 7.2.9) (Note : '0' as 'a' and '1' as 'b')** **(10 Marks)**

Scanned by CamScanner

Dec. 2008

Q. 14 What is Turing machine ? Explain different techniques for Turing machine construction. **(Sections 7.2 and 7.4)** **(10 Marks)**

Q. 15 Design a Turing machine to compute n!. **(Example 7.3.13)** **(10 Marks)**

Q. 16 Write detail note on : Universal TM **(Section 7.5)** **(3 Marks)**

May 2009

Q. 17 Write a short note on Universal Turing Machine. **(Section 7.5)** **(5 Marks)**

Q. 18 Construct a Turing machines M to accept the language $L = \{\omega c \omega^R \mid \omega \in \{0, 1\}^*$ and ω^R means reverse of w}. Trace the sequence of moves made corresponding to input string 01c10. **(Similar to Example 7.2.18)** **(10 Marks)**

Q. 19 Construct a turing machine M for performing proper subtraction m-n is defined to be m-n for $m \geq n$ and zero for $m < n$. The TM started with $0^m\ 10^n$ its tape, halts with 0^{m-n} on its tape. Trace the sequence to moves for the input string 0000100.

(Similar to Example 7.3.3) **(10 Marks)**

Dec. 2009

Q. 20 Design a TM to subtract two numbers (e.g. m and n are two integers and m – n is to be evaluated) assume m > n. **(Example 7.3.3)** **(10 Marks)**

Q. 21 Design a TM to accept language $\{0^n1^n \mid n > 0\}$

(Example 7.2.2) (Note : '0' as 'a' and '1' as 'b') **(10 Marks)**

Q. 22 Write short note on Variants of TM **(Section 7.4)** **(5 Marks)**

May 2010

Q. 23 Design TM that replaces all occurrences of "111" by "101" from sequence of 0's and 1's. **(Example 7.2.14)** **(10 Marks)**

Q. 24 Write short note on : Universal Turing Machine **(Section 7.5)** **(5 Marks)**

Dec. 2010

Q. 25 Explain different types of machines and state at least one application of each

(Section 7.1) **(10 Marks)**

Q. 26 Design a Turing machine to accept the language given by a regular expression 0(0+1)*11. **(Example 7.2.21)** **(10 Marks)**

Q. 27 Write short note on : Variants of a Turing Machine **(Section 7.4)** **(6 Marks)**

Scanned by CamScanner

May 2011

Q. 28 Design a Turing machine to accept the language $L = \{ a^n b^n \mid n >= 1 \}$. **(Example 7.2.2)** **(10 Marks)**

Q. 29 Design a Turing machine that computes a function $f(m, n) = m + n$ for the addition of 2 integers. **(Example 7.3.10)** **(10 Marks)**

Dec. 2011

Q. 30 Design Turing machine for recognizing the following Language : $L = \{a^n b^n c^n \mid n \geq 1\}$ **(Example 7.2.17)** **(5 Marks)**

Q. 31 Write short note on : Universal turning machine **(Section 7.5)** **(6 Marks)**

May 2012

Q. 32 Design a Turing Machine for the following language. "set of all the strings of balanced parentheses" **(Example 7.2.10)** **(10 Marks)**

Q. 33 Short note on :Universal Turing Machine. **(Section 7.5)** **(5 Marks)**

Q. 34 Write short on application areas of Turing machine. **(Section 7.3)** **(5 Marks)**

Dec. 2012

Q. 35 What is TM ? Give the power of TM over FSM. **(Section 7.1, Example 7.5.1)** **(10 Marks)**

Q. 36 Write short note on :Universal TM **(Section 7.5)** **(6 Marks)**

May 2013

Q. 37 Define TM and give its variants. **(Sections 7.1 and 7.2)** **(5 Marks)**

Q. 38 Design a turing machine to accept the language $L = \{a^n b^n \mid n >= 1\}$ **(Example 7.2.2)** **(10 Marks)**

May 2014

Q. 39 Design a TM that recognizes palindrome strings where $\Sigma = \{0, 1\}$ **(Similar to Example 7.2.18)** **(10 Marks)**

Q. 40 Write short note on : Variants of Turing Machine **(Section 7.4)** **(5 Marks)**

Scanned by CamScanner

Dec. 2014

Q. 41 Design Turing machine as generator to add two binary numbers and hence simulate for "110 + 10". **(Hint :** Assume two way infinite tape.) **(10 Marks)**

Ans. : [Hint : Refer Example as Example 7.2.14(A)]

This problem can be solved using a 3-tape Turing machine.

First machine T1 stores the first binary number. Second machine T2 stores the second binary number. Third machine T3 stores the result.

The Turing machine will have 3 states :

q_0 – previous carry as 0

q_1 – previous carry as 1

q_2 – Halting state

(0, 0, L) (0, 0, L) (B, 0, L)
(1, 1, L) (0, 0, L) (B, 0, L)
(1, 1, L) (0, 0, L) (B, 1, L)
(1, 1, L) (B, B, L) (B, 0, L)
(0, 0, L) (1, 1, L) (B, 1, L)
(0, 0, L) (1, 1, L) (B, 0, L)
(B, B, L) (0, 0, L) (B, 0, L)
(B, B, L) (1, 1, L) (B, 0, L)
(0, 0, L) (B, B, L) (B, 0, L)
(1, 1, L) (1, 1, L) (B, 1, L)
(B, B, L) (1, 1, L) (B, 1, L)
(1, 1, L) (B, B, L) (B, 1, L)

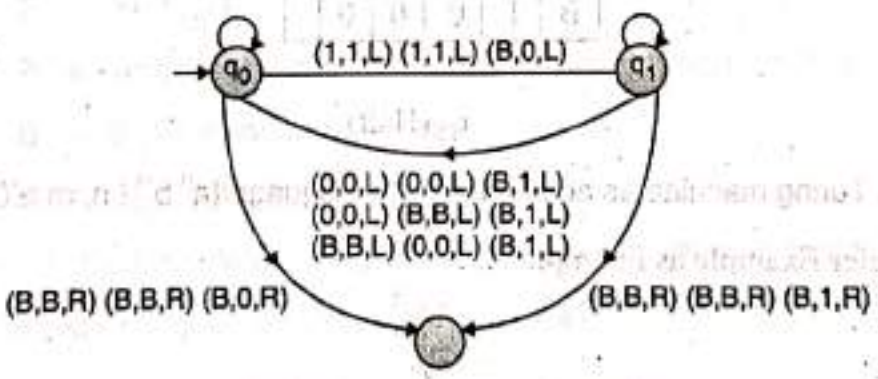


Fig. 1-Q. 41

Simulation for 110 + 10

	B	B	1	0	
	B	1	1	0	
	B	B	B	B	

↑

State q_0

Scanned by CamScanner

	B	B	1	0	
	B	1	1	0	
	B	B	B	0	

↑

State q_0

	B	B	1	0	
	B	1	1	0	
	B	B	0	0	

↑

State q_1

	B	B	1	0	
	B	1	1	0	
	B	0	0	0	

↑

State q_1

B	B	B	1	0	
B	B	1	1	0	
B	1	0	0	0	

↑

q_2 (Halt)

Q. 42 Design a Turing machine as acceptor for the language $\{a^n b^m \mid n, m \geq 0 \text{ and } m \geq n\}$.

Ans. : [Hint : Refer Example as Example 7.2.17(A)]

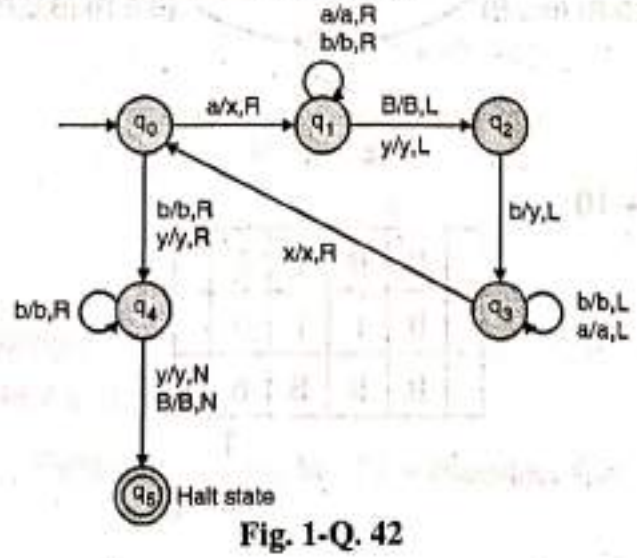


Fig. 1-Q. 42

Scanned by CamScanner

May 2015

Q. 43 Construct turning machine that accepts the string over $\Sigma = \{0, 1\}$ and converts every occurrence of 111 to 101. **(10 Marks)**

Ans. : [Hint : Refer Example as Example 7.2.13(A)]

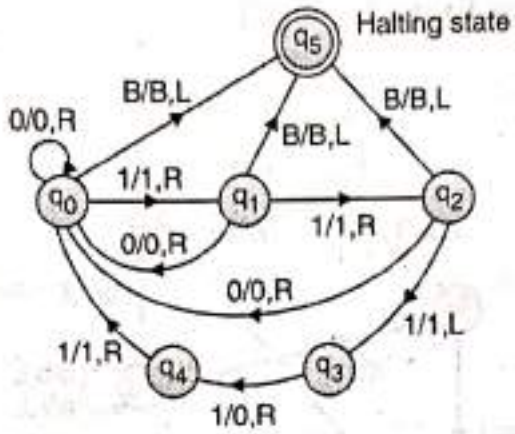


Fig. 1-Q. 43

The turing machine M is given by :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Where,

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$

$\Sigma = \{0, 1\}$

$\Gamma = \{0, 1, B\}$

δ = Transition function is shown using the transition diagram

B = Blank symbol for the tape

$F = \{q_5\}$, halting state

Q. 44 Construct TM to check wellformed ness of parenthesis. **(Example 7.2.10)** **(10 Marks)**

Q. 45 Write short note on : Variants of TM **(Section 7.4)** **(5 Marks)**

Dec. 2015

Q. 46 Design Turing machine to accept the language L = set of string with equal number of a's and b's. **(Example 7.2.9)** **(10 Marks)**

Q. 47 Write short note on "Universal Turing machine" **(Section 7.5)** **(5 Marks)**

Q. 48 Explain variations and equivalences of Turing machine. **(Section 7.5.2)** **(10 Marks)**

Scanned by CamScanner

May 2016

Q. 49 Explain variations and equivalence of Turing machine. **(Section 7.5.2)** **(10 Marks)**

Q. 50 Design a Turing machine as an acceptor for the language $\{a^n b^m \mid n, m \geq 0 \text{ and } m \geq n\}$ **(10 Marks)**

Ans. : **[Hint :** Refer Example as Example 7.2.22]

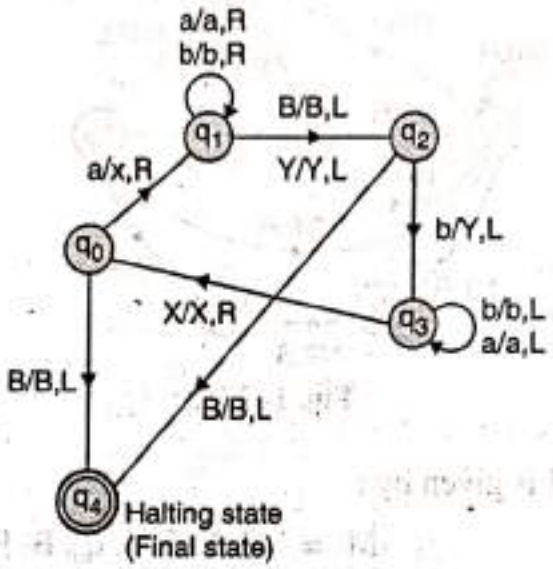


Fig. 1-Q. 50

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Where,

$Q = \{q_0, q_1, q_2, q_3, q_4\}$

$\Sigma = \{a, b\}$

$\Gamma = \{a, b, X, Y, B\}$

q_0 = initial state

B = Blank symbol

$F = \{q_4\}$

Dec. 2016

Q. 51 State and explain the power and limitations of a Turning machine. **(Section 7.5.2 and Example 7.5.1)** **(5 Marks)**

Q. 52 Design a Turing machine that computes a function f(m, n) = m + n i.e. addition of two integers.**(Example 7.3.10)** **(10 Marks)**

Scanned by CamScanner

Q. 53 Design a Turning machine to accept the language $0^n 1^n 2^n$. **(Similar to Example 7.2.17)** **(10 Marks)**

May 2017

Q. 54 Design Turning machine to recognize wellformedness of parenthesis. **(Example 7.2.10)** **(10 Marks)**

Q. 55 Write short note on : Variants of turning Machine. **(Section 7.4)** **(5 Marks)**

Q. 56 Write short note on : Church-Turing Thesis. **(Section 7.5.1)** **(5 Marks)**

Dec. 2017

Q. 57 Design a Turing Machine that recognizes palindrome string where $\Sigma = (a, b)$. **(Example 7.2.18)** **(10 Marks)**

Chapter Ends...

❑❑❑

Scanned by CamScanner

Undecidability

8.1 Recursive and Recursively Enumerable Language

MU - Dec. 2005, Dec. 2009, Dec. 2010, May 2014, Dec. 2014, May 2015, Dec. 2015, May 2016, Dec. 2016, Dec. 2017

There is a difference between recursively enumerable (Turing Acceptable) and recursive (Turing Decidable) language.

- Following statements are equivalent :
  1. The language L is **Turing acceptable.**
  2. The language L is **recursively enumerable.**
- Following statements are equivalent
  1. The language L is **Turing decidable.**
  2. The language L is **recursive.**
  3. There is an algorithm for recognizing L.
- Every Turing decidable language is Turing acceptable.
- Every Turing acceptable language need not be Turing decidable.

8.1.1 Turing Acceptable Language

A language $L \subseteq \Sigma^*$ is said to be a Turing Acceptable language if there is a Turing machine M which halts on every $\omega \in L$ with an answer **'YES'**. However, if $\omega \notin L$, then M may not halt.

Turing Decidable Language

A language $L \subseteq \Sigma^*$ is said to be turing being decidable if there is a turing machine M which always halts on every $\omega \in \Sigma^*$. If $\omega \in L$ then M halts, with answer 'YES', and if $\omega \notin L$ then M halts, with answer 'NO'.

Scanned by CamScanner

- A set of solutions for any problem defines a language.
- A problem P is said to be decidable /solvable if the language $L \subseteq \Sigma^*$ representing the problem (set of solutions) is turing decidable.
- If P is solvable / decidable then there is an algorithm for recognizing L, representing the problem. It may be noted that an algorithm terminates on all inputs.
- Following statements are equivalent :
  1. The language L is Turing decidable.
  2. The language L is recursive.
  3. There is an algorithm for recognizing L.

Remarks

1. Every turing decidable language is turing acceptable.
2. Every turing acceptable language need not be turing decidable.
3. A language $L \subseteq \Sigma^*$ many not be turing acceptable and hence not turing decidable. Thus we cannot design a turing machine / algorithm which halts for every $\omega \in L$.

Example 8.1.1 : Show that for two recursive languages L_1 and L_2, each of the following is recursive.

(i) $L_1 \cup L_2$ (ii) $L_1 \cap L_2$ (iii) L_1'

Solution :

(i) $L_1 \cup L_2$ is recursive

Let the turing machine M_1 decides L_1 and M_2 decides L_2.

If a word $\omega \in L_1$ then M_1 returns "Y" else it returns "N". Similarly, if a word $\omega \in L_2$ then M_2 returns "Y" else it returns "N".

Let us construct a turing machine M_3 as shown in Fig. Ex. 8.1.1(a).

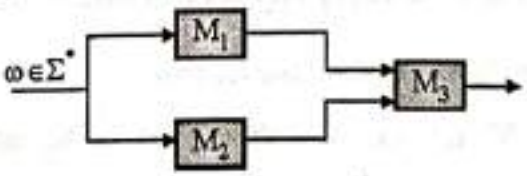


Fig. Ex. 8.1.1(a) : A turing machine for $L_1 \cup L_2$

- Output of machine M_1 is written on the tape of M_3.
- Output of machine M_2 is written on the tape of M_3.
- The machine M_3 returns "Y" as output, if at least one of the outputs of M_1, or of M_2 is "Y".

Scanned by CamScanner

It should be clear that M_3 decides $L_1 \cup L_2$. As both L_1 and L_2 are turing decidable, after a finite time both M_1 and M_2 will halt with answer "Y" or "N". The machine M_3 is activated after M_1 and M_2 are halted. The machine M_3 halts with answer "Y" if $\omega \in L_1$ or $\omega \in L_2$, else M_3 halts with output "N".

Thus $L_1 \cup L_2$ is turing decidable or $L_1 \cup L_2$ is recursive.

(ii) $L_1 \cap L_2$ is recursive

Let the turing machine M_1 decides L_1 and M_2 decides L_2. If a word $\omega \in L$, then M_1 returns "Y" else it returns "N". Similarly, if a word $\omega \in L_2$ then M_2 returns "Y" else it returns "N".

Let us construct a turing machine M_4 as shown in Fig. Ex. 8.1.1(b).

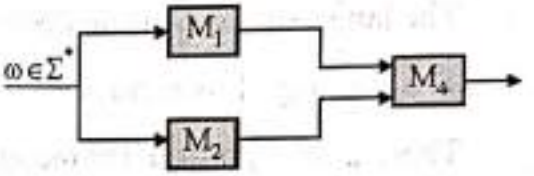


Fig. Ex. 8.1.1(b) : A turing machine for $L_1 \cap L_2$

- Output of machine M_1 is written on the tape of M_4.
- Output of machine M_2 is written on the tape of M_4.
- The machine M_4 returns "Y" as output, if both outputs of M_1 and M_2 are "Y"; otherwise, M_4 returns "N".

It should be clear that M_4 decides $L_1 \cap L_2$. As both L_1 and L_2 are turing decidable, after a finite time both M_1 and M_2 will halt with answer "Y" or 'N". The machine M_4 is activated after M_1 and M_2 are halted. The machine M_4 halts with answer "Y" if $\omega \in L_1$ and $\omega \in L_2$, else M_4 halts with answer "N".

(iii) L_1' is Recursive

Let the turing machine M_1 decide L_1.

Let us construct a turing machine M_5 as shown in Fig. Ex. 8.1.1(c).

Fig. Ex. 8.1.1(c) : A turing machine for L_1'

- Output of machine M_1 is written on the tape of M_5.
- The machine M_5 returns "Y" as output, if the output of M_1 is "N"; otherwise, M_5 returns "N".

It should be clear that M_5 decides L_1'. As L is turing decidable, after a finite time M_1 will halt with answer "Y" or "N". The machine M_5 is activated after M_1 halts. The machine M_5 halts with answer "Y" if $\omega \notin L_1$, else M_5 halts with answer "N".

Scanned by CamScanner

Example 8.1.2 : Show that the language $L = \{a^n b^n c^n \mid n \geq 0\}$ is turing decidable.

Solution :

In order to prove that L is turing decidable, we must construct TM accepting both L and L′. We can construct a 3-tape TM. The construction is given below.

Step 1 : b^n is copied to tape 2 and c^n is copied to tape 3 using the following moves. b's and c's are erased from the first tape. Initially the three heads H_1, H_2 and H_3 are positioned on the leftmost symbol.

$\delta(q_0(B,B,B)) = (q_1,(B,B,B),(R,R,R))$ [Skip the first blank]

$\delta(q_1,(a,B,B)) = (q_1,(a,B,B),(R,N,N))$ [skip a's on tape 1]

$\delta(q_1,(b,B,B)) = (q_2,(B,b,B),(R,R,N))$ [copy first b to tape 2]

$\delta(q_2,(b,B,B)) = (q_2,(B,b,B),(R,R,N))$ [copy remaining b's to tape 2]

$\delta(q_2,(c,B,B)) = (q_3,(B,B,c),(R,N,R))$ [copy first c to tape 3]

$\delta(q_3,(c,B,B)) = (q_3,(B,B,c),(R,N,R))$ [copy remaining c's to tape 3]

$\delta(q_3,(B,B,B)) = (q_4,(B,B,B),(L,N,N))$

Step 2 : H_1 is positioned on the rightmost a

$\delta(q_4(B,B,B)) = (q_4,(B,B,B),(L,N,N))$

$\delta(q_4,(a,B,B)) = (q_5,(a,B,B),(N,L,L))$

H_2 is positioned on the rightmost b and H_3 is positioned on the rightmost c.

Step 3 : a's on tape 1, b's on tape 2 and c's on tape 3 are matched.

$\delta(q_5,(a,b,c)) = (q_5,(a,b,c),(L,L,L))$

$\delta(q_5,(B,B,B)) = (q_6,(B,B,B),(N,N,N))$

q_6 is a halting state.

If the state q_6 is reached then the string is in the language, $\{a^n b^n c^n\}$.

If at any stage, the TM does not have any move then the string $\omega \in L'$

Thus a string of the form $a^n b^n c^n$ is turing decidable.

Example 8.1.3 : If L_1 and L_2 are two recursive languages and if L is defined as : $L = \{\omega \mid \omega$ is in L_1 and not in L_2, or ω is in L_2 and not in $L_1\}$. Prove or disprove that L is recursive.

Scanned by CamScanner

Solution :

Let the turing machine M_1 decides L_1 and M_2 decides L_2. If a word $\omega \in L_1$, then M_1 returns "Y" else it returns "N". Similarly, if a word $\omega \in L_2$ then M_2 return "Y" else it returns "N".

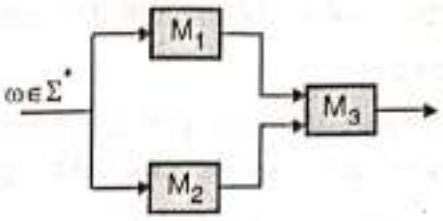


Fig. Ex. 8.1.3

Let us construct a Turing machine M_3 as shown in Fig. Ex. 8.1.3.

- Output of machine M_1 is written on the tape of M_3.
- Output of machine M_2 is written on the tape of M_3.
- The machine M_3 returns "Y" as output, if one of the outputs of M_1, or M_2 is "Y" and other is "N".
- The machine M_3 returns "N" as output, if the outputs of both M_1 and M_2 are either "Y" or "N".

It should be clear that M_3 decides L. As both L_1 and L_2 are recursive, after a finite time both M_1 and M_2 will halt with answer "Y" or "N". The machine M_3 is activated after M_1 and M_2 are halted. Finally, the machine M_3 will halt with answer "Y" or "N".

Example 8.1.4 : Prove the theorem.

"A language is recursive if and only if both it and its complement are recursively enumerable.

Solution : Let us consider a recursive language L. Let the turing machine M_1 accepts L and M_2 accepts L'.

If a word $\omega \in L$ then M_1 returns "Y" and the machine M_2 returns either "Y" or it loops forever.

Similarly, if a word $\omega \notin L$ then M_2 returns "N" and the machine M_1 returns either "N" or it loops forever.

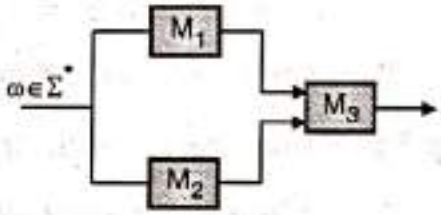


Fig. Ex. 8.1.4

Let us construct a Turing machine M_3 as shown in Fig. Ex. 8.1.4.

- Output of machine M_1 is written on the tape of M_3.
- Output of machine M_2 is written on the tape of M_3.
- The machine M_3 returns "Y" if a value "Y" is found on its tape.
- The machine M_3 returns "N" if a value "N" is found on its tape.

It should be clear that if $\omega \in L$ then at least M_1 will halt with answer "Y". Similarly, if $\omega \in L$ then, at least M_2 will halt with answer "N".

Thus, if a language and its complement are recursively enumerable then the language is recursive.

If a language is recursive then its complement is also recursive. A recursive language is subset of a recursively enumerable language. Every recursive language is recursively enumerable.

Hence, proved.

Scanned by CamScanner

8.2 Enumerating a Language

A Turing enumerable language can be enumerated by some Turing machine. To enumerate a language means to list the elements one at a time.

A Turing machine enumerating a language

- A Turing machine M can be made to list elements of a language L.
- A multi-tape Turing machine, with K-tapes can be considered for this purpose. Tape 1 can be reserved exclusively for output.
- Let M be a K-tape Turing machine with $K \geq 1$ and $L \subseteq \Sigma^*$. M is made to operate in such a fashion so that the following conditions are satisfied :
  1. The tape head on first tape moves only in the forward direction replacing blank symbols with valid strings $\omega_i \in L$.
  2. For every $\omega_i \in L$, there is some point in the operation of M when tape 1 has following contents :

     $$\omega_1 \# \omega_2 \# .... \# \omega_n \# \omega \#$$

     Where, the strings $\omega_1, \omega_2 \ldots \omega_n$ are distinct strings in L.
     (a) If L is finite than nothing is printed after the # following the last word.
     (b) If L is finite than machine can either halt or continue to loop forever after the last word has appeared on the tape.
     (c) If L is infinite, M continues to move forever.

The machine M cannot hope to finish its computation on each string ω_i before beginning to work on the next. If $\omega_i \notin L$ then M may loop forever. The solution for this is given below :

Instead of completing the computation on each string as it is generated, M carries out the following sequence of operations :

1. ω_{i+1} is computed from ω_i where $\omega_{i+1} = \omega_i \cdot \Sigma$ for each alphabet in Σ.
2. If ω_{i+1} takes the machine M to an accepting state than ω_{i+1} is written to tape 1.
3. Step 1 and step 2 are carried out infinitely if L is infinite.

8.2.1 Finite and Infinite Sets

Number of elements in a set is also known as its cardinality.

- If $\mathbf{A \subseteq B}$, then the size of **A** is less than or equal to that of **B**.
- Two sets A and B are said to be equinumerous (having same number of elements) if there is a **bijection.**

  $\mathbf{f : A \rightarrow B}$ [bijection means one-to-one and onto]
- A finite set has finite number of elements.

Scanned by CamScanner

- A set is **infinite** if it is not **finite**.
- The set **N** of **natural numbers** is **infinite**.
- A set **S** is said to be countably infinite if there is a bijection from **N** to **S**.

  $\mathbf{f : N \rightarrow S}$ is a bijection.
- To prove that a set **S** is countably infinite, we must show a bijection between some countably infinite set and the set **S**.

Example 8.2.1 : Show that any subset of a countable set is countable.

Solution :

Let A be a given countably infinite set.

Let **S** be an infinite subset of the set A.

Obviously, there exists a bijection between the set **N** of natural numbers and the set **A**.

$\mathbf{f : N \rightarrow A}$, $f(n) = x$ for $x \in \mathbf{A}$ is a bijective relation.

Elements of the set **A** can be arranged as :

$f(1), f(2), f(3)\ldots.$

Now, we can delete from the set **A**, those elements which are not present in **S**. The remaining elements in **A** must be infinite. Let us denote these elements by :

$f(i_1), f(i_2)\ldots..$

Now, we can define a function

$\mathbf{g : N \rightarrow S}$ such that $g(n) = f(i_n)$

Thus, **g** is one-to-one and onto.

Hence, **S** is countable.

Example 8.2.2 : Show that an infinite recursively enumerable set has an infinite recursive sub set.

Solution :

Let $n_1, n_2, n_3\ldots\ldots$be a recursive enumeration of an infinite set S. For each n_i there must be, in this sequence, a later $n_j > n_i$.

Let $m_1 = n_1$,

$m_2 = n_{i_2}$, the first n_i greater than n_1

$m_3 = n_{i_3}$, the first n_i beyond n_{i_2} greater than n_{i_2}.

⋮

The sequence $m_1, m_2, m_3\ldots.$ is a recursive enumeration of a subset of S without repetition in order of magnitude. This subset is infinite and recursive.

Scanned by CamScanner

Example 8.2.3 : Show that if S is uncountable and T is countable than S-T is uncountable.

Solution :

We can prove this by contradiction.

Suppose **S-T** is countable set

$$S\text{-}T = \{\omega_1, \omega_2 \ldots \omega_n\}$$

where, $\omega_1 = a_{11}, a_{12} \ldots\ldots$

$\omega_2 = a_{21}, a_{22} \ldots\ldots$

$\omega_n = a_{n1}, a_{n2} \ldots\ldots$

It is given that the set T is countable.

$$T = \{x_1, x_2 \ldots x_m\}$$

Since, the set **S** is uncountable, we can always find a word y such that

$y \in S$, $y \notin$ S-T and $y \notin T$

Hence, a contradiction.

Therefore, S-T is uncountable.

Example 8.2.4 : Show that the set of languages L over {0,1} so that neither L nor L′ is recursively enumerable is uncountable.

Solution :

To prove that the set of languages over {0,1} that are not recursively enumerable is uncountable, we will first prove that the set of recursively enumerable languages is countable.

Proof that set of recursively enumerable language is countable

- Transitions of a Turing machine can be represented by a binary number. If ω_i represent a Turing machine then the set of all Turing machines.

$$\mathbf{T} = \{\omega_1, \omega_2, \ldots..\}$$

Obviously, there exists a bijection between the set **N** of natural numbers and the set **T**.

$$\mathbf{f : N \rightarrow T},\ f(n) = \omega \text{ for } \omega \in \mathbf{T} \text{ is a bijective relation}$$

A recursively enumerable language L can be accepted by some Turing machine. For each L, let g(L) be such a Turing machine. Since every TM accepts only one language, the relation **g** is one-to-one.

RE is subset of **T**.

Since, **T** is countable, RE is also countable.

Now, that we have shown the set of recursively enumerable language to be countable; proving that there are uncountable many languages which are not recursively enumerable.

Scanned by CamScanner

Example 8.2.5 : Prove that "The set of real numbers, R, is not countable.

Solution :

Let us define a set S.

$$S = \{x \mid (x \in R) \text{ and } (0 < x < 1)\}$$

Now, the proof can be given in two steps :

1. By proving that S and R have the same cardinality.
2. The set S is not countable.

These two steps will imply that R is not countable.

Step 1 : Proof that S and R have the same cardinality

Let R be the set of all positive real numbers. Let $f : R \rightarrow S$ be defined by $f(x) = \frac{x}{1+x}$ for $x \in R$. Obviously the range of f is in S.

Further, for any $y \in S$, we have $x = \frac{y}{1-y}$. Thus f is one-to-one and onto. Therefore, the set S and the set R have the same cardinality.

Step 2 : The proof that the set S given by $S = \{x \mid x \in R \text{ and } 0 < x < 1\}$ is not countable.

We can prove this by contradiction.

Suppose S is a countable set.

$$S = \{\omega_1, \omega_2 \ldots . \omega_n\}$$

where, $\omega_1 = a_{11}\, a_{12}\, a_{13} \ldots \ldots$

$\omega_2 = a_{21}\, a_{22}\, a_{23} \ldots \ldots$

$\omega_n = a_{n1}\, a_{n2}\, a_{n3} \ldots \ldots$

Assumptions

1. Each ω_i is distinct
2. $a_{ij} \in [0,1,2,3,4,5,6,7,8,9]$

Using the principle of diagonalization, we can find a string $\omega_k = x_{11}\, x_{22}\, x_{33} \ldots$, such that $\omega_k \notin S$.

$$x_{ii} = a_{ii} + 1, \quad \text{if} \quad a_{ii} < 5$$
$$= 4, \quad \text{if} \quad a_{ii} \geq 5$$

Now, it should be clear that :

ω_k will differ from ω_1 at the first place

ω_k will differ from ω_2 at the second place

⋮

ω_k will differ from ω_n at the n^{th} place.

Therefore, the set S is uncountable.

Hence, proved by contradiction.

Scanned by CamScanner

8.3 Compare Type 0, Type 1, Type 2 and Type 3 Grammars

Type	Name of Languages Generated	Restriction on Productions	Accepting Machine
0	Recursively enumerable	$\alpha_1 \rightarrow \alpha_2$ where $\alpha_1, \alpha_2 \in (V \cup T)^*$	Turing machine
1	Context sensitive	$\alpha_1 \rightarrow \alpha_2$ where $\alpha_1, \alpha_2 \in (V \cup T)^*$ and $\lvert\alpha_1\rvert \leq \lvert\alpha_2\rvert$	Turing machine with bounded tape. Length of the tape in finite
2	Context free	$X \rightarrow \alpha_1$ where $X \in V$ and $\alpha_1 \in (V \cup T)^*$	PDA
3	Regular	$X \rightarrow aY \mid a \mid \varepsilon \mid Ya$ where, $X, Y \in V$ and $a \in T$	FA

A general hierarchy of various languages is given below :

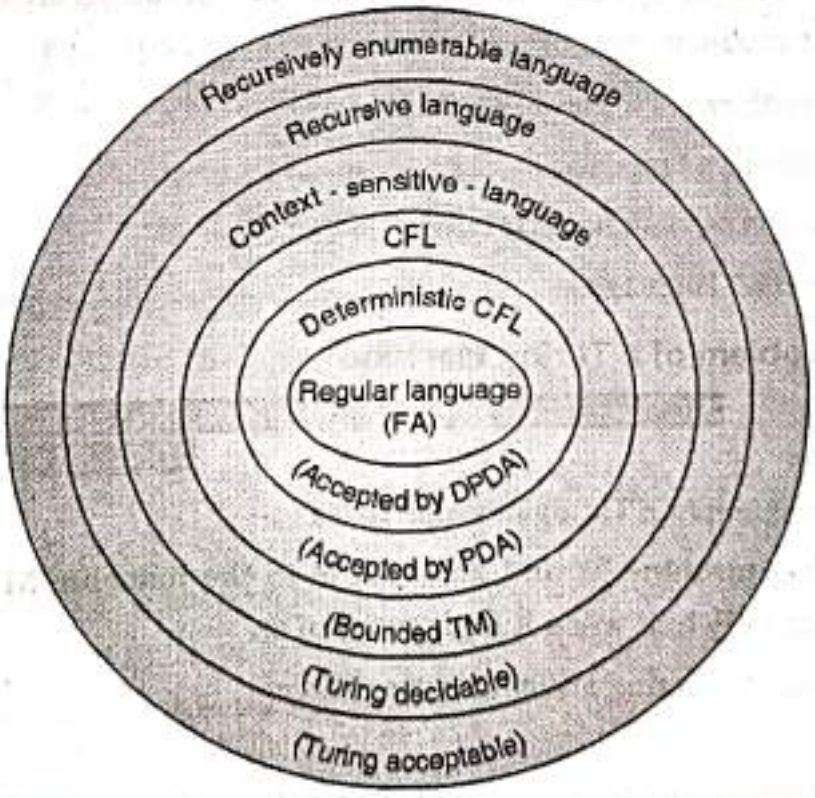


Fig. 8.3.1 : Hierarchy of languages

Scanned by CamScanner

8.4 Un-decidability

MU - May 2013

A problem is said to be decidable if there exists a Turing machine that gives the correct answer for every statement in the domain of the problem. Otherwise, the class of problems is said to be un-decidable. These two statements are equivalent :

1. A class of problems is un-decidable.
2. A class of problems is un-solvable.

How to prove that a given language is un-decidable ?

A language can be proved to be un-decidable through a method of reduction. We have already seen that the halting problem is un-decidable.

- To show that a problem A is un-decidable, we must reduce another problem that is known to be un-decidable to A.
- Having proved that the halting problem is un-decidable, we can use problem reduction to show that other problems are un-decidable.

First of all, we will provide proof for the un-decidability of some standard problems. Subsequently, these problems will be used to show that other problems are un-decidable. Some standard un-decidable problems are :

1. Halting problem of a Turing machine.
2. Diagonalization language.
3. The post correspondence problem.
4. The universal language.

8.4.1 Halting Problem of a Turing Machine

MU - Dec. 2006, Dec. 2007, May 2008, Dec. 2008, May 2011, Dec. 2011, Dec. 2015, Dec. 2016, May 2017

The halting problem of a Turing machine states :

Given a Turing machine M and an input ω to the machine M, determine if the machine M will eventually halt when it is given input ω.

Halting problem of a Turing machine is unsolvable.

Proof :

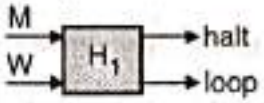
- Moves of a turing machine can be represented using a binary number. Thus, a Turing machine can be represented using a string over $\Sigma^*(0,1)$. This concept has already been explained in the chapter.
- Insolvability of halting problem of a Turing machine can be proved through the method of contradiction.

Scanned by CamScanner

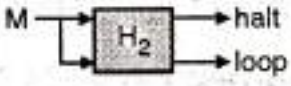
Step 1 : Let us assume that the halting problem of a Turing machine is solvable. There exists a machine H_1(say). H_1 takes two inputs :

1. A string describing M.
2. An input ω for machine M.

H_1 generates an output **"halt"** if H_1 determines that M stops on input ω; otherwise H outputs "loop". Working of the machine H_1 is shown below.

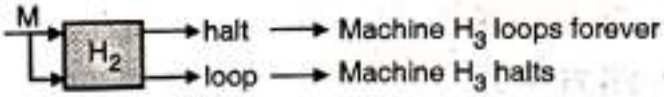


Step 2 : Let us revise the machine H_1 as H_2 to take M as both inputs and H_2 should be able to determine if M will halt on M as its input. Please note that a machine can be described as a string over 0 and 1.



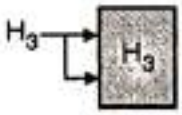
Step 3 : Let us construct a new Turing machine H_3 that takes output of H_2 as input and does the following :

1. If the output of H2 is "loop" than H3 halts.
2. If the output of H_2 is "halt" than H_3 will loop forever.



H_3 will do the opposite of the output of H_2.

Step 4 : Let us give H_3 itself as inputs to H_3.



If H_3 halts on H_3 as input then H_3 would loop (that is how we constructed it).
If H_3 loops forever on H_3 as input H_3 halts (that is how we constructed it).
In either case, the result is wrong.
Hence,
H_3 does not exist.
If H_3 does not exist than H_2 does not exist.
If H_2 does not exist than H_1 does not exist.

Scanned by CamScanner

Example 8.4.1 : Show that the following problem is un-decidable. "Given a Turing machine T,T halts on every input string".

Solution : This is also known as **totality problem**

This can be proved by showing that the halting problem is reducible to the totality problem.

- That is, if an algorithm can solve the totality problem, it can also solve the halting problem. Since, no algorithm can solve the halting problem; the totality problem must also be unsolvable.

Reduction step (from halting problem to totality problem) :

For any Turing machine M and input ω, we construct M_1 :

1. M_1 takes an arbitrary input, ignores it, and runs M on ω.
2. M_1 halts on all inputs if and only if M halts on input ω.

Thus, M_1 halts on all inputs also tells that M halts on input ω, which is solution to the halting problem. Hence, the totality problem is un-decidable.

8.4.2 Un-decidability of Post Correspondence Problem :

MU - May 2006, Dec. 2006, May 2007, May 2008, Dec. 2008, May 2009, May 2010, Dec. 2010, May 2011, Dec. 2011, May 2012, May 2016

Definition of post correspondence problem (PCP) : Let A and B be two non-empty lists of strings over Σ. A and B are given as below :

$$A = \{x_1, x_2, x_3 \ldots x_k\}$$

$$B = \{y_1, y_2, y_3 \ldots y_k\}$$

We say, there is a post correspondence between A and B if there is a sequence of one or more integers i, j, k ...m such that :

The string $x_i\, x_j \ldots x_m$ is equal to $y_i\, y_j \ldots y_m$.

Example : Does the PCP with two lists :

$$A = \{a, aba^3, ab\} \text{ and}$$

$$B = \{a^3, ab, b\}$$

have a solution ?

We will have to find a sequence using which when the elements of A and B are listed, will produce identical strings.

The required sequence is (2, 1, 1 , 3)

$$A_2\, A_1\, A_1\, A_3 = aba^3\, a\, aab = ab\, a^6\, b$$

$$B_2\, B_1\, B_1\, B_3 = aba^3\, a^3\, b = aba^6\, b$$

Thus, the PCP has solution.

We are accepting the un-decidability of post correspondence problem without proof.

Scanned by CamScanner

Example 8.4.2 : Determine the solution for following instance of PCP. **MU - Dec. 2007**

	List A	List B
I	ωI	xI
1	01	0
2	110010	0
3	1	1111
4	11	01

Solution :

The PCP has a solution. The required sequence is (1, 3, 2, 4, 4, 3)

$$\omega_1\omega_3\omega_2\omega_4\omega_4\omega_3 = 01111001011111$$

$$x_1\, x_3\, x_2\, x_4\, x_4 x_3 = 01111001011111$$

Example 8.4.3 : Does PCP with following two list : A = (10, 011, 101) and B = (101, 11, 011) have a solution ? Justify your answer. **MU - May 2009**

Solution :

A_2 and A_3 differ from B_2 and B_3 at the first of place. Therefore, we must pick A_1 and B_1

Sequence	String
(1)	$(A_1 = 10)\ (B_1 = 101)$

The next string to be picked up must be A_3 and B_3. Any other sequence will not lead to a solution.

Sequence	String
(1, 3)	$(A_1A_3 = 10101)\ (B_1B_3 = 101011)$

The next string to be picked up must be A_3 and B_3. Any other sequence will not lead to a solution.

Sequence	String
(1, 3, 3)	$(A_1A_3A_3 = 10101101)\ (B_1B_3B_3 = 101011011)$

There is only choice of next string. This choice is A_3 and B_3. This does not lead to a solution. The PCP has no solution.

Example 8.4.4 : Prove that there exists no algorithm for deciding whether a given CFG is ambiguous. **MU - May 2006, Dec. 2007, Dec. 2008**

Solution :

The post correspondence problem can be used to prove the un-decidability of whether a given CFG is ambiguous.

Scanned by CamScanner

Let us consider two sequences of strings over Σ.

$$A = \{u_1, u_2, u_3 \dots u_m\}$$

$$B = \{v_1, v_2, v_3 \dots v_m\}$$

Let us take a new set of symbols $a_1, a_2 \dots a_m$ such that

$$\{a_1, a_2 \dots a_m\} \cap \Sigma = \phi.$$

Symbols $a_1, a_2 \dots a_m$ are being taken as index symbols. The index symbol a_i represents a choice of u_i from A and v_i from the list B.

A string of the form $u_i\, u_j\, u_k \dots a_k\, a_j\, a_i$. Over alphabet $\Sigma \cup \{a_1, a_2, \dots a_m\}$ can be defined using the set of productions :

$$G_A = \begin{Bmatrix} A \rightarrow u_1\, A\, a_i \mid u_2\, A\, a_2 \mid \dots \mid u_m\, Aa_m \\ u_1\, a_1 \mid u_2\, a_2 \mid \dots \mid u_m\, a_m \end{Bmatrix}$$

Similarly a string of the form $v_i\, v_j\, v_k \dots a_k\, a_j\, a_i$ over alphabet $\Sigma \cup \{a_1, a_2 \dots a_m\}$ can be defined using the set of productions :

$$G_B = \begin{Bmatrix} B \rightarrow v_1\, A\, a_1 \mid v_2\, A\, a_2 \mid \dots \mid v_m\, Aa_m \\ v_1\, a_1 \mid v_2\, a_2 \mid \dots \mid v_m\, a_m \end{Bmatrix}$$

Finally, we can combine the languages and grammars of two lists to form a grammar G_{AB} :

- A new start symbol S is added to G_{AB}
- Two new productions are added to G_{AB}

$$S \rightarrow A$$

$$S \rightarrow B$$

- All productions of G_A and G_B are taken.

Now, we will show that G_{AB} is ambiguous if and only if an instance (A, B) of PCP has a solution.

Assumption :

Suppose the sequence $i_1, i_2, \dots, i_m$ is a solution to this instance of PCP. Two derivations for the above string in G_{AB} is :

$$S \Rightarrow A \Rightarrow u_{i_1} A\, a_{i_1} \Rightarrow u_{i_1} u_{i_2} A\, a_{i_1} a_{i_2} \Rightarrow \dots \Rightarrow$$

$$u_{i_1} u_{i_2} \dots u_{i_m} a_{i_1} a_{i_2} \dots a_{i_m}$$

$$S \Rightarrow B \Rightarrow v_{i_1} B\, a_{i_1} \Rightarrow v_{i_1} v_{i_2} B\, a_{i_1} a_{i_2} \Rightarrow \dots \Rightarrow$$

$$v_{i_1} v_{i_2} \dots v_{i_m} a_{i_1} a_{i_2} \dots a_{i_m}$$

Consequently, if G_{AB} is ambiguous, then the post correspondence problem with the pair (A, B) has a solution. Conversely, if G_{AB} is unambiguous, then the post correspondence cannot have a solution.

Scanned by CamScanner

If there exists an algorithm for solving the ambiguous problem, then there exists an algorithm for solving the post correspondence problem. But, since there is no algorithm for the post correspondence problem, the ambiguity of CFG problem is unsolvable.

Example 8.4.5 : Prove that the blank tape halting problem is un-decidable.

Solution :

The halting problem of Turing machine can be reduced to blank-tape halting problem. Given a Turing machine M, determine whether or not M halts if started with a blank tape. This is un-decidable.

Reduction :

Suppose, we are given some Turing machine M and some string ω. We first construct from M a new machine M_1 that starts with a blank tape, writes ω on it, then positions itself on the first symbol of ω in starting state q_0.

After this, M_1 will halt on a blank tape if and only if M halts on ω.

Conclusion :

Since, the halting problem of the Turing machine is un-decidable, the same must be true for the blank tape halting problem.

Example 8.4.6 : Let G_1 and G_2 be context free grammars, and r be a regular expression. Then the following are un-decidable :

(a) $L(G_1) \cap L(G_2) = \phi$ (b) $L(G_1) = L(G_2)$

(c) $L(G_1) = L(r)$ (d) $L(G_1) \subseteq L(G_2)$

(e) $L(R) \subseteq L(G_1)$

Solution :

(a) $L(G_1) \cap L(G_2) = \phi$ is un-decidable :

Proof :

Let G_1 be a grammar that generates

$$L(G_1) = \{\omega C \omega^R \text{ such that } \omega \in \Sigma^* \text{ and } C \notin \Sigma\}$$

Let $P \subseteq \Sigma^+ \times \Sigma^+$, a member of P will be of the form $(u_i\ v_i)$

Let G_2 be a grammar that generates

$$L(G_2) = \{u_1 u_2 \ldots u_n C v_n^R v_{n-1}^R \ldots v_1^R . \mid n \geq 1 \text{ and } (u_i, v_i) \in P\}$$

- G_1 and G_2 can be constructed easily.
- $L(G_1) \cap L(G_2) = \{\omega C \omega^R$ such that ω is match of P}

$L(G_1) \cap L(G_2)$ is the set of solutions to this instance of PCP. The intersection is empty if and only if there is no solution.

Scanned by CamScanner

If there is an algorithm for finding whether $L(G_1) \cap L(G_2)$ is ϕ then there exists an algorithm for solving the post correspondence problem. But, since there is no algorithm for the post correspondence problem there is no solution for

$$L(G_1) \cap L(G_2) = \phi ?$$

(b) $L(G_1) = L(G_2)$ is un-decidable :

Proof :

Un-decidability of $L(G_1) = L(G_2)$ will imply un-decidability of similar unsolvable problem about Turing machine.

Assumptions :

$L(G_1) = L(G_2)$ is decidable

Construction of TM for G_1 and G_2.

We can design a TM for $L(G_1)$ and similarly for $L(G_2)$. We can design a Turing machine M_1 for $L(G_1)$ as follows :

1. Given an input string ω, machine M_1 saves the string ω on its tape.
2. Machine M_1 can produce all strings over the alphabet of G_1 in a lexicographic order.
3. Whenever M_1 produces a string derivable by $L(G_1)$, it compares the last string in derivation with the saved string ω. M_1 halts if they are same otherwise M_1 continues to generate later strings.

Obviously, $L(G_1)$ is Turing acceptable. If a language is Turing acceptable then it must be the output language of some Turing machine.

Let us assume that $L(G_1)$ is acceptable by M_1 and $L(G_2)$ is acceptable by M_2.

From the assumption, $L(G_1) = L(G_2)$

For every string $\omega_i \in L(G_1)$, ω_i should be in $L(G_2)$ and for every string $\omega_i \in L(G_2)$, ω_i should be in $L(G_1)$.

$\omega_i \in L(G_1)$ is similar to whether M_1 accepts ω_i and

$\omega_i \in L(G_2)$ is similar to whether M_2 accepts ω_i.

Solvability of above problem contradicts the halting problem of Turing machine.

(c) $L(G_1) = L(r)$ is un-decidable :

Proof :

Let the grammar G_1 and the regular expressions r is defined over the alphabet Σ.

Let the language $L(G_1)$ is the output language of the Turing machine M_1.

Scanned by CamScanner

The decidability of $L(G_1) = L(r)$ will imply :

1. For every ω_i in $L(r)$, ω_i is in $L(M_1)$ is Turing decidable.
2. For every ω_i in $\overline{L(r)}$, ω_i is in $\overline{L(M_1)}$ is Turing decidable.

8.4.3 Rice'sTheorem

MU - Dec. 2012, May 2013, May 2014, May 2015, Dec. 2015, May 2016, Dec. 2016, May 2017, Dec. 2017

"Every property that is satisfied by some but not all recursively enumerable language is un-decidable". Any property that is satisfied by some recursively enumerable language but not all is known as nontrivial property. We have seen many properties of R.E. languages that are un-decidable. These properties include :

1. Given a TM M, is L(M) nonempty ?
2. Given a TM M, is L(M) finite ?
3. Given a TM M, is L(M) regular ?
4. Given a TM M, is L(M) recursive ?

The Rice's theorem can be proved by reducing some other unsolvable problem to nontrivial property of recursively enumerable language.

8.4.4 Greibach Theorem

MU - May 2006, Dec. 2007, May 2011

The Theorem states that :

"Let σ be a class of languages that is effectively closed under concatenation with regular sets and union, and for which $L = \Sigma^*$ is un-decidable for any sufficiently large fixed Σ. Let P be any non-trivial property that is true for all regular sets and that is preserved under a, where a is single symbol in Σ. Then P is un-decidable for σ".

Greibach theorem can be used to prove that many problems related to CFG are un-decidable.

8.5 Partial Recursive Function

Turing machine is viewed as a mathematical model of partial recursive function.

Some basic concepts :

1. Total function :

A total function from x to y assigns a unique element of y to every element of x.

$f_1(x) = 2x$ is a **total function**

Scanned by CamScanner

2. Partial function :

A partial function from x to y assigns at-most one element of y to every element of x.

$f_2(x) = +\sqrt{x}$ is a partial function.

$f_2(x)$ is not defined if x is a negative real number.

3. Function of K-variables :

A function of K-variables is represented as $f(x_1, x_2 \ldots x_k)$.

$f(x_1, x_2) = x_1 + 2x_2$ is a function of two variables.

4. Primitive recursive function :

- Some initial functions are taken as primitive recursive functions. These initial functions are
  1. Zero function
  2. Successor function
  3. Projection function
- A function derived from combination/composition of primitive recursive functions is primitive recursive.

The **zero function** Z is defined by $Z(x) = 0$

The **successor function** S is defined by $S(x) = x + 1$

The **projection function** U_i^n is defined by

$$U_i^n (x_1, x_2, \ldots x_n) = x_i$$

The projection functions for selecting i^{th} element from a tape containing n elements.

Example 8.5.1 : Show that $f(x, y) = x + y$ is primitive recursive.

Solution :

$$f(x, 0) = x + 0 = x = U_1^1 (x)$$

Only element x is selected from the tape.

$$f(x, y + 1) = f(x,y) + 1 = S(f(x,y))$$

$$= S\left[U_3^3 (x, y, f(x, y))\right]$$

Select the third element

Increment it by 1

Scanned by CamScanner

Thus the function f is obtained by applying composition and recursion finite number of times to initial functions. Hence, f is primitive recursive.

8.6 University Questions and Answers

Dec. 2005

Q. 1 Write short note on : Recursively Enumerable Languages. **(Section 8.1)** **(5 Marks)**

May 2006

Q. 2 Prove that it is un-decidable whether a context free grammar is ambiguous. **(Example 8.4.4)** **(10 Marks)**

Q. 3 Write short notes on post correspondence problem and Greibach Theorem. **(Sections 8.4.2 and 8.4.4)** **(8 Marks)**

Dec. 2006

Q. 4 Write short notes on :

(a) Halting problem **(Section 8.4.1)**

(b) Post's correspondence problem. **(Section 8.4.2)** **(10 Marks)**

May 2007

Q. 5 Write note on : Post correspondence problem. **(Section 8.4.2)** **(5 Marks)**

Dec. 2007

Q. 6 Prove that is un-decidable whether Context Free Grammar is ambiguous. **(Example 8.4.4)** **(10 Marks)**

Q. 7 Determine the solution for following instance of post correspondence problem.

	List A	List B
I	ωI	xI
1	01	0
2	110010	0
3	1	1111
4	11	01

(Example 8.4.2) **(5 Marks)**

Scanned by CamScanner

Q. 8 Write notes on :

(i) Halting problem **(Section 8.4.1)**

(ii) Greibach Theorem **(Section 8.4.4)** **(10 Marks)**

May 2008

Q. 9 Write notes on :

(i) Post correspondence problem **(Section 8.4.2)**

(ii) Halting problem **(Section 8.4.1)** **(10 Marks)**

Dec. 2008

Q. 10 Prove that it is un-decidable whether a Context Free Grammer is ambiguous. **(Example 8.4.4)** **(10 Marks)**

Q. 11 Write detail notes on :

(a) Post correspondence problem **(Section 8.4.2)**

(b) Halting problem **(Section 8.4.1)** **(7 Marks)**

May 2009

Q. 12 Explain the un-decidability of PCP ? Does PCP with following two list : A = (10, 011, 101) and B = (101, 11, 011) have a solution ? Justify your answer. **(Section 8.4.2, Example 8.4.3)** **(6 Marks)**

Dec. 2009

Q. 13 Write short note on :Recursive and recursively enumerated languages. **(Section 8.1)** **(5 Marks)**

May 2010

Q. 14 Define post correspondence problem and prove that PCP with two lists $x = \{ b, bab^3, ba \}$ and $y = \{ b^3, ba, a \}$ have a solution. **(Sections 8.4.2) (Note : a and b are interchanged)** **(10 Marks)**

Scanned by CamScanner

Dec. 2010

Q. 15 Write short notes on :

(a) Post Correspondence Problem **(Section 8.4.2)**

(b) Recursive and recursively enumerable languages **(Section 8.1)** **(10 Marks)**

May 2011

Q. 16 Write short notes on :

(a) Halting problem **(Section 8.4.1)**

(b) Post Correspondence Problem **(Section 8.4.2)**

(c) Greibach Theorem. **(Section 8.4.4)** **(15 Marks)**

Dec. 2011

Q. 17 Write short notes on :

(a) Post correspondence problem **(Section 8.4.2)**

(b) Halting problem **(Section 8.4.1)** **(10 Marks)**

May 2012

Q. 18 Short note on : Post Correspondence Problem. **(Section 8.4.2)** **(5 Marks)**

Dec. 2012

Q. 19 Write short note on : Rice Theorem **(Section 8.4.3)** **(5 Marks)**

May 2013

Q. 20 Define the following terms : Undecidability **(Section 8.4)** **(2 Marks)**

Q. 21 Write short notes on : Rice's theorem. **(Section 8.4.3)** **(5 Marks)**

May 2014

Q. 22 Write short notes on :

(c) Rice's Theorem **(Section 8.4.3)**

(d) Recursive and Recursively enumerable languages. **(Section 8.1)** **(10 Marks)**

Dec. 2014

Q. 23 Compare recursive and recursively enumerable languages. **(Section 8.1)** **(5 Marks)**

Scanned by CamScanner

May 2015

Q. 24 Compare recursive and recursively enumerable languages. **(Section 8.1)** **(5 Marks)**

Q. 25 Write short note on : Rice theorem. **(Section 8.4.3)** **(5 Marks)**

Dec. 2015

Q. 26 State and prove the halting problem. **(Section 8.4.1)** **(5 Marks)**

Q. 27 Compare recursive and recursively enumerable language. **(Section 8.1)** **(5 Marks)**

Q. 28 State and prove Rice's theorem. **(Section 8.4.3)** **(10 Marks)**

May 2016

Q. 29 Explain post correspondence problem. **(Section 8.4.2)** **(5 Marks)**

Q. 30 Compare recursive and recursively enumerable languages. **(Section 8.1)** **(5 Marks)**

Q. 31 State and prove Rice's theorem. **(Section 8.4.3)** **(10 Marks)**

Dec. 2016

Q. 32 Write detailed note on : Recursive and Recursively Enumerable Languages. **(Section 8.1)** **(10 Marks)**

Q. 33 Write detailed note on : Rice's Theorem. **(Section 8.4.3)** **(10 Marks)**

Q. 34 Write detailed note on : Halting problem. **(Section 8.4.1)** **(10 Marks)**

May 2017

Q. 35 Explain Rice's Theorem. **(Section 8.4.3)** **(5 Marks)**

Q. 36 Write short note on : Halting Problem. **(Section 8.4.1)** **(5 Marks)**

Dec. 2017

Q. 37 Explain Recursive and Recursively enumerable languages. **(Section 8.1)** **(5 Marks)**

Q. 38 Write short note on Rices' Theorem. **(Section 8.4.3)** **(5 Marks)**

Chapter Ends...

❑❑❑

Scanned by CamScanner

Appendix A

Solved University Question Paper of Dec. 2018

Theory of Computer Science

Dec. 2018

Q. 1(a) Explain Chomsky Hierarchy

(Section 1.3) **(5 Marks)**

Q. 1(b) Differentiate between PDA and NPDA. **(5 Marks)**

Ans. :

Sr. No.	PDA	NPDA
1.	Always a single move on a new input.	Multiple moves are possible on a new input.
2.	Less powerful than NPDA.	More powerful than a PDA.
3.	Algorithms related to PDA are simple.	Algorithms related to NPDA are complex.
4.	Algorithms related to PDA do not require backtracking.	Algorithms related to NPDA require backtracking.

Q. 1(c) Define Regular Expression and give regular expression for :

(i) Set of all strings over {0, 1} that end with 1 has no substring 00 **(Section 3.1)** **(5 Marks)**

Ans. :

The transition graph is shown in Fig. 1-Q. 1(c) :

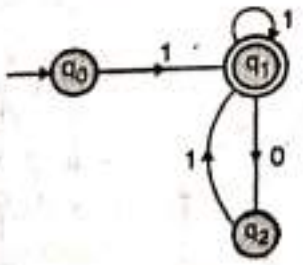


Fig. 1-Q.1(c)

∴ R.E. can be written from the transition graph. The required R. E. = $1(1+01)^*$

Q. 1(d) Explain Halting Problem.

(Section 8.4.1) **(5 Marks)**

Q. 2(a) Design a Finite State machine to determine whether ternary number (base 3) is divisible 5.

(Example 2.2.25) **(10 Marks)**

Q. 2(b) Give and explain formal definition of Pumping Lemma for Regular Language and prove that following language is not regular.

$L = \{a^m b^{m-1} \mid m > 0\}$ **(10 Marks)**

Ans. :

Step 1 : Let us assume that the given language $L(a^n b^{n-1} \mid n > 0)$ is regular and L is accepted by a FA with n states.

Step 2 : Let us choose a string

$$\omega = a^n b^{n-1}$$

$$|\omega| = 2^{n-1} \geq n \text{ for } n > 0$$

Let us write ω as xyz, with

$|y| > 0$

and $|xy| \leq n$

since, $|xy| \leq n$, y must be of the form $a^r \mid r > 0$.

since $|xy| \leq n$, x must be of the form a^s.

Now, $a^n b^{n-1}$ can be written as

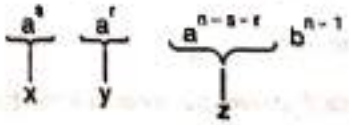


Fig. 1-Q. 2(b)

Scanned by CamScanner

Step 3 : Let us check whether $xy^{\overline{L}}z$ for $\overline{L} = 2$ belongs to L.

$$xy^2z = a^s (a^r)^2 a^{n-s-r} b^{n-1}$$
$$= a^3 a^{2r} a^{n-s-r} b^{n-1}$$
$$= a^{n+r} b^{n-1}$$

Since $r > 0$, $a^{n+r} b^{n-1} \notin L$.

Hence, by contradiction, we can say that the given language is not regular.

Q. 3(a) Construct PDA accepting the language $L = \{a^{2n} b^n \mid n \geq 0\}$. **(10 Marks)**

Ans. :

1. For every pair of a's one x is pushed on to the stack
2. For every b, one x is popped out from the stack.
3. Finally the stack should contain the initial stack symbol Z_0.

Transition table (δ)

1. $\delta(q_0, a, Z_0) = (q_1, Z_0)$
2. $\delta(q_1, a, Z_0) = (q_0, x Z_0)$
3. $\delta(q_0, a, x) = (q_1, x)$
4. $\delta(q_1, a, x) = (q_0, xx)$
5. $\delta(q_0, b, x) = (q_2, \epsilon)$
6. $\delta(q_2, b, x) = (q_2, \epsilon)$
7. $\delta(q_2, \epsilon, Z_0) = (q_2, \epsilon)$

Accepting through empty stack

Thus, the PDA M = $(\{q_0, q_1, q_2\}, \{a, b\}, \{x, Z_0\}, \delta, q_0, Z_0, \{\phi\})$

Q. 3(b) Consider the following grammar **(10 Marks)**

S→ iCtS | iCtSeS | a

C→b

For the string 'ibtaeibta' find the following :

(i) Leftmost derivation

(ii) Rightmost derivation

(iii) Parse tree

(iv) Check if above grammar is ambiguous.

Ans. :

(i) Left most derivation :

S → iCtSeS [using S → iCtSeS]
→ ibtSeS [using C → b]
→ ibtaeS [using S → a]
→ ibtaeiCtS [using S→ iCtS]
→ ibtaeibts [using C → b]
→ ibtaeibta

(ii) Rightmost derivation :

S → iCtSeS [using S → iCtSeS]
→ iCtSeiCtS [using S → iCtS]
→ iCtSeiCta [using S → a]
→ iCtSeibta [using C → b]
→ iCtaeibta [using S → a]
→ ibtaeibta [using C → b]

(iii) Parse tree as shown in Fig. 1-Q. 3(b)

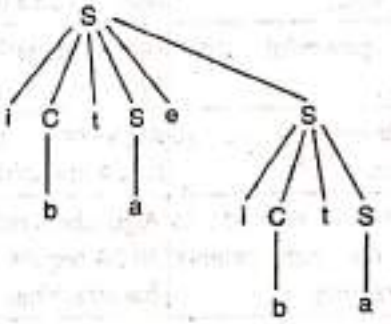


Fig. 1-Q. 3(b)

(iv) The grammar can be shown to be ambiguous by drawing two different derivation trees for the string ibtibtaea as shown in Fig. 2-Q. 3(b)

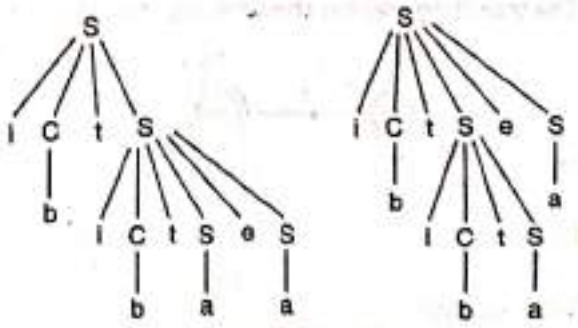


Fig. 2-Q. 3(b)

Scanned by CamScanner

Q. 4(a) Construct TM to check wellformedness of parenthesis. **(Example 7.2.10)** **(10 Marks)**

Q. 4(b) Convert following CFG to CNF **(10 Marks)**

$S \rightarrow ASA \mid Ab$

$A \rightarrow B \mid S$

$B \rightarrow b \mid \in$

Ans. :

1. Nullable set of symbols = (B, A)

   Re-writing grammar after removing $\in$-production, we get,

   $S \rightarrow AS \mid SA \mid ASA \mid aB \mid a$

   $A \rightarrow B \mid S$

   $B \rightarrow b$

2. Re-writing grammar after removing unit productions ($A \rightarrow B$, $A \rightarrow S$), we get

   $S \rightarrow AS \mid SA \mid ASA \mid aB \mid a$

   $A \rightarrow b \mid AS \mid SA \mid ASA \mid aB \mid a$

   $B \rightarrow b$

3. Every symbol in α, in production of the form $A \rightarrow \alpha$ where $|\alpha| \geq 2$ should be a variable. This can be done by adding the production

   $C_1 \rightarrow a$

   The set of productions become,

   $S \rightarrow AS \mid SA \mid ASA \mid C_1B \mid a$

   $A \rightarrow b \mid AS \mid SA \mid ASA \mid C_1B \mid a$

   $B \rightarrow b$

   $C_1 \rightarrow a$

4. Finding an equivalent grammar in CNF.

   $S \rightarrow AS \mid SA \mid AC_2 \mid C_1B \mid a$ [Replacing SA by C_2]

   $C_2 \rightarrow SA$

   $A \rightarrow b \mid AS \mid SA \mid AC_2 \mid C_1B \mid a$

   $B \rightarrow b$

   $C_1 \rightarrow a$

Q. 5(a) Convert (0 + 1) (10)*(0 + 1) into NFA with $\in$-moves and obtain DFA. **(10 Marks)**

Ans. :

R. E. to NFA

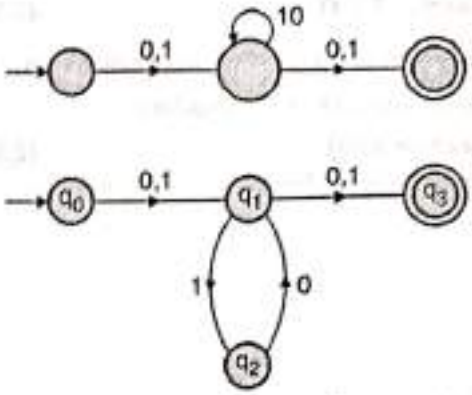


Fig. 1-Q. 5(a)

NFA to DFA using direct method

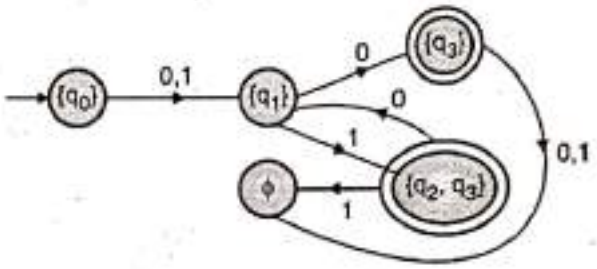


Fig. 2-Q. 5(a)

Q. 5(b) Construct Moore and Mealy Machine to convert each occurrence of 100 by 101. **(10 Marks)**

Ans. :

1. Mealy Machine

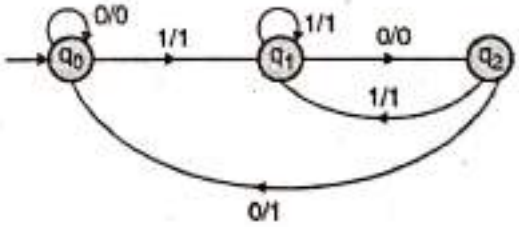


Fig.1-Q. 5(b)

2. Moore Machine

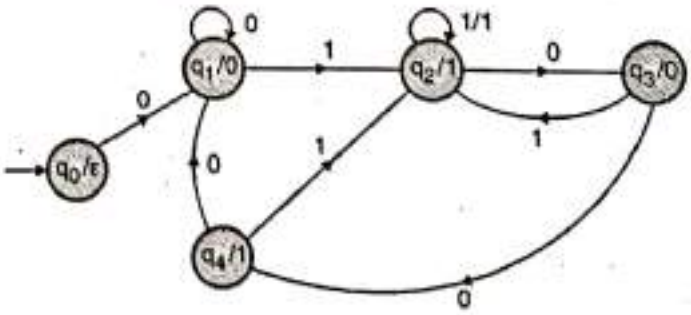


Fig. 2-Q. 5(b)

Scanned by CamScanner

Q. 6(a) Write short note on Closure properties of Context Free Language.
(Section 5.7.1) **(2.5 Marks)**

Q. 6(b) Write short note on Applications of Regular expression and Finite automata.
(Section 3.10) **(2.5 Marks)**

Q. 6(c) Write short note on Rice's Theorem.
(Section 8.4.3) **(2.5 Marks)**

Q. 6(d) Write short note on Moore and Mealy Machine
(Section 2.4) **(2.5 Marks)**

Q. 6(e) Write short note on Universal Turing Machine
(Section 7.5) **(2.5 Marks)**

❑❑❑

Scanned by CamScanner

Your Success is Our Goal

Strictly as per the New Revised Syllabus (Rev - 2016) of Mumbai University w.e.f. academic year 2018-2019 (As per Choice Based Credit and Grading System)

MU

Semester V - Computer Engineering

Computer Networks
J. S. Katre

Database Management System
Mahesh Mali

Microprocessor
Harish G. Narula

Theory of Computer Science
Dilip Kumar Sultania

Multimedia System (Dept. Elective I)
Tulsiram Sule, Poonam Kadam

Advance Operating System (Dept. Elective I)
Rajesh D. Kadu

coming soon.....
es easy-solutions
now with
TechKnowledge Publications

Head Office :
B/5, First Floor, Maniratna Complex, Taware Colony, Aranyeshwar Corner,
Pune - 411009. Maharashtra State, India. Tel. : 91-20-24221234, 91-20-24225678

ISBN : 978-93-89299-95-3

Price ₹ 455/-

MO46A

Distributors

Student's Agencies (I) Pvt. Ltd.
T. : (022) 40496161, 91672 90777

Vidyarthi Sales Agencies
T. : (022) 23867279, 98197 76110

Bharat Sales Agency
T. : (022) 23819350, 86572 92797

Our Branches : Pune | Mumbai | Kolhapur | Nagpur | Solapur | Nashik

For Library Orders Contact - Ved Book Distributors M : 80975 71421 / 92208 77214

Email : info@techknowledgebooks.com | Website : www.techknowledgebooks.com

Like us at: TechknowledgePublications

Scanned by CamScanner